Operating Systems

For Java Developers

How the operating system runs a Java service: processes, threads, scheduling, memory, files and I/O, containers and concurrency, with diagrams, real commands, and production incidents shown from symptom to diagnosis, fix and prevention.

Processes · Threads · Scheduling · Virtual Memory · Page Cache · OOM Killer
Deadlocks · File Systems · I/O · Containers · cgroups · Incident Playbooks

```
$ ps -o pid,stat,rss,cmd -p $(pgrep -f app.jar)
  PID STAT    RSS CMD
 4127 Sl   812344 java -jar app.jar
$ cat /proc/4127/status | grep -E 'VmRSS|Threads'
VmRSS:    812344 kB
Threads:       61
```


Maruthi Sreeram

Prepared for Java full stack interview preparation

Maruthi Sreeram

Version basis

Experiments were run on a Linux 6.x virtual machine (2 CPUs, 8 GB) with JDK 21, gcc, strace and procfs
Output from other kernels, distributions or production systems is labelled typical
Prepared for Maruthi Sreeram

How to use this guide

- **Who this is for.** A Java full stack developer who must explain, and diagnose, what the operating system does beneath a running service: process and thread management, memory, files, I/O, containers and synchronisation.
- **Every chapter follows the same steps:** (1) the concept and the problem it solves, (2) the mechanism, shown with diagrams, (3) a worked trace or calculation, (4) a hands-on experiment with real commands, C or Java code, (5) **real-world incidents**, each presented as symptoms, diagnosis, root cause, fix and prevention, and (6) the questions interviewers ask, with answers.
- **How to study.** Read each diagram before the prose and narrate it aloud. Reproduce every experiment on a Linux machine or virtual machine (never on a production host). Treat each incident as an exercise: cover the diagnosis, decide which command to run first, then compare with the text.
- **Running the code.** Save a Java listing as `Main.java` and run `java Main.java`; compile C with `gcc -O0 -o demo demo.c`. Terminal sessions begin with `$`. Output shown was produced on the build machine unless labelled typical.
- **Scope.** The focus is Linux, the platform on which most Java services run, together with the JVM and container behaviour that interacts with it. Windows and macOS are mentioned only where the difference matters.

Contents

Part 1: Operating System Foundations 4

1.1 What an Operating System Does 4
1.2 Hardware Essentials for Operating Systems 12
1.3 System Calls 22
1.4 Boot and Init 32
1.5 The Linux File-Based View 40
1.6 The Diagnostic Toolbox 48

Part 2: Processes, Threads and Scheduling 59

2.1 Processes 59
2.2 Signals, Process Groups and Daemons 71
2.3 Inter-Process Communication 78
2.4 Threads 86
2.5 CPU Scheduling 94
2.6 Scheduling in Linux 97
2.7 CPU Limits in Containers 106

Part 3: Memory Management 112

3.1 Virtual Memory and Address Spaces 112
3.2 Paging and Address Translation 120
3.3 Page Replacement, Working Sets and Swapping 131
3.4 The Page Cache and Memory Used for File I/O 139
3.5 Memory Limits, Overcommit and the OOM Killer 146
3.6 The Memory of a JVM Process 156

Part 4: Concurrency and Synchronisation 166

4.1 Race Conditions and Critical Sections 166

sreeram

4.2 Mutexes, Semaphores, Condition Variables and Spinlocks . . . 175
4.3 Deadlock, Livelock, Starvation and Priority Inversion . . . 188
4.4 Locks in Java and Their Mapping to the Operating System . . . 200
4.5 Diagnosing Concurrency Problems in Production . . . 208

Part 5: Storage, File Systems and I/O . . . 216

5.1 Files and Directories . . . 216
5.2 File System Internals . . . 226
5.3 Storage Devices and the Block Layer . . . 237
5.4 Buffered I/O, the Page Cache and Durability . . . 244
5.5 I/O Models . . . 253
5.6 Logging and Log Rotation in Production . . . 262

Part 6: Containers, Security and Production Playbook . . . 269

6.1 Containers from First Principles . . . 269
6.2 Linux Security, Services and Graceful Shutdown . . . 281
6.3 The Production Playbook . . . 291
6.4 Interview Question Bank . . . 306
6.5 Cheat Sheets and Study Plan . . . 310
6.6 Putting It Together: One Request Through the Whole Stack . . . 314

sreeram

PART 1

Operating System Foundations

The kernel boundary, the hardware beneath it, and the tools used to observe both on a running Linux system.

In this part

1.1 What an Operating System Does
1.2 Hardware Essentials for Operating Systems
1.3 System Calls
1.4 Boot and Init
1.5 The Linux File-Based View
1.6 The Diagnostic Toolbox

1.1 What an Operating System Does

A computer consists of processors, memory, storage devices and network adapters, each with its own programming interface and its own failure modes. A program that talked to this hardware directly would need to contain a disk driver, a memory allocator, a network stack and a scheduler, and two such programs running at the same time would overwrite each other's data. The operating system exists to remove both problems. It presents every program with a uniform, simplified machine, and it arbitrates between programs that compete for the real one.

For a Java developer the operating system is rarely visible in source code, yet it determines the behaviour of every deployed service. The size of the heap that the JVM selects, the number of threads that can be created, the point at which a request becomes slow, and the exit status that a container orchestrator reports are all decisions or side effects of the kernel. This chapter defines the operating system precisely, distinguishes the kernel from the rest of the system, and traces a single line of Java output through every layer between the source code and the hardware.

1.1.1 Concept and mechanism

The three functions of an operating system

An operating system performs three functions that are easier to understand separately than as a single definition.

- **Resource management.** The processor, memory, storage and network bandwidth are finite. The kernel decides which thread runs on which core, which pages stay in memory, and which request is served next by a disk.
- **Abstraction.** Hardware details are hidden behind stable interfaces. A program opens a file, not a set of disk sectors; it uses a socket, not a network card queue; it addresses virtual memory, not physical frames.
- **Protection and isolation.** One program must not read or corrupt the memory of another, and an unprivileged program must not issue instructions that control the hardware. The processor enforces the rule, and the kernel configures the processor to do so.

Preview

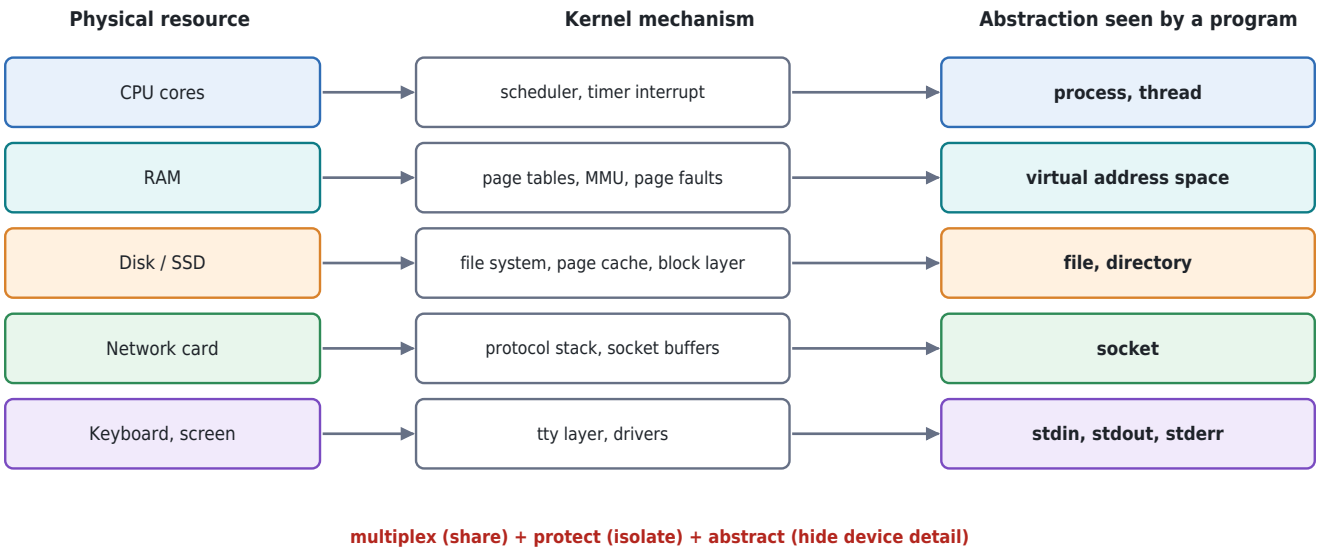


Figure 1.1. Each physical resource is multiplexed by a kernel mechanism and offered to programs as an abstraction; the Linux terms in the right column are the ones that appear in diagnostic output throughout this book.

Kernel and user space

The **kernel** is the part of the operating system that runs with full hardware privileges and is always resident in memory. Everything else runs in **user space**: shells, compilers, the JVM, web servers, and also the system utilities that are conventionally considered part of the operating system, such as `ps`, `systemd` and the C library. A *distribution* such as Ubuntu is the Linux kernel plus a large collection of user-space programs. Linux itself names only the kernel.

> **DEFINITION**
> Kernel: the privileged component of an operating system that manages the processor, memory and devices, and that user programs enter only through a controlled interface, the system call.

The separation is enforced by the processor. x86-64 processors define four privilege levels, called rings. Linux and Windows use two of them: ring 0 for the kernel and ring 3 for applications. The current ring is recorded in the low two bits of the code segment register. Privileged instructions, such as those that load page tables, disable interrupts or halt the processor, are legal only in ring 0. A program in ring 3 can leave the ring only through designated entry points, which are the system call instruction, interrupts and processor faults.

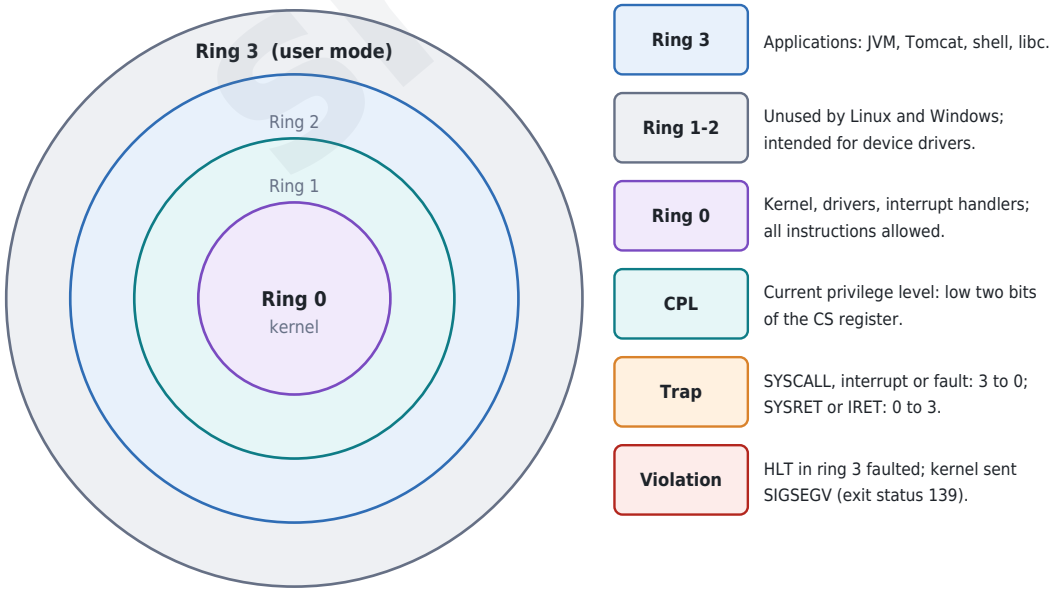


Figure 1.2. Rings 1 and 2 are unused by mainstream systems; the only transitions that matter are entry to ring 0 through a trap and the return to ring 3.

The separation can be demonstrated directly. The program in the following listing executes `HLT`, an instruction that stops the processor until the next interrupt, from user mode.

priv.c C

```c
#include <stdio.h>

int main(void) {
    printf("about to execute HLT in user mode\n");
    fflush(stdout);
    __asm__ volatile("hlt");
    printf("not reached\n");
    return 0;
}
```

Privileged instruction in user mode (Linux 6.18.44, x86-64) TERMINAL

```
$ gcc -o priv priv.c && ./priv; echo "exit status $?"
about to execute HLT in user mode
/bin/bash: line 23:  2561 Segmentation fault      ./priv
exit status 139
$ strace -e trace=none ./priv 2>&1 | tail -2
--- SIGSEGV {si_signo=SIGSEGV, si_code=SI_KERNEL, si_addr=NULL} ---
+++ killed by SIGSEGV +++
```

The processor raised a general protection fault. The kernel's fault handler found that the faulting code belonged to a user process and converted the fault into the signal `SIGSEGV`. The marker `SI_KERNEL` in the signal information states that the kernel itself generated the signal, not a bad memory reference by the program. The shell reports the exit status 139, which is 128 plus the signal number 11. The same arithmetic recurs for container exit codes later in this chapter.

Kernel architectures

Operating-system designs differ in how much functionality runs in ring 0. The choice is a trade between speed, robustness and the amount of privileged code that must be trusted.

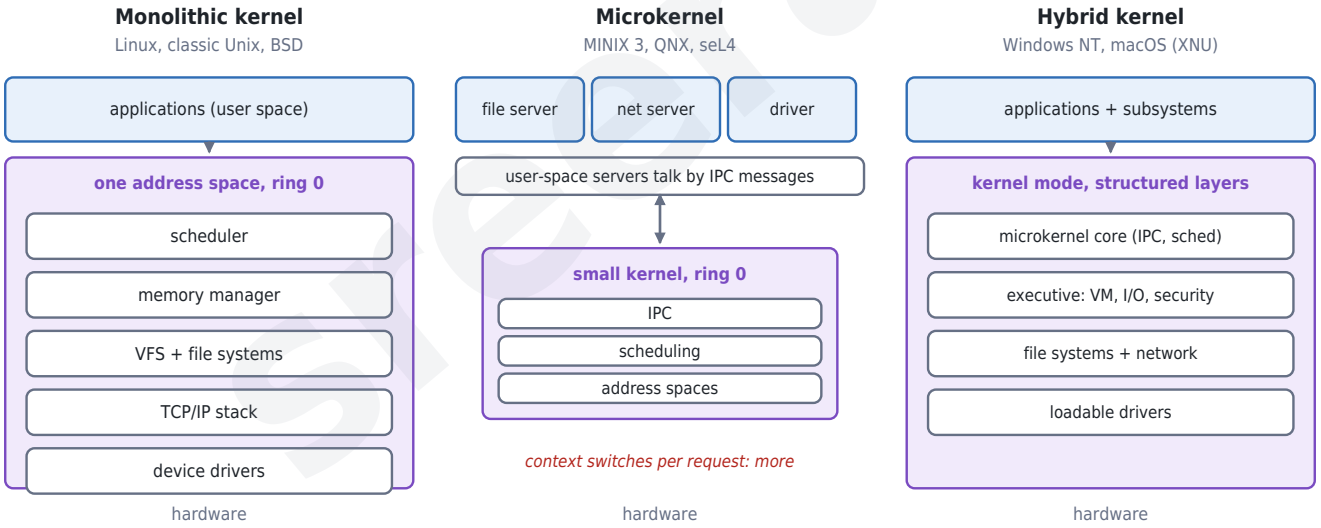


Figure 1.3. In a monolithic kernel all subsystems share one privileged address space; a microkernel moves file systems and drivers into user-space servers that communicate by messages; a hybrid design keeps microkernel structure but runs most services in kernel mode.

Property	Monolithic (Linux)	Microkernel (MINIX 3, QNX, seL4)	Hybrid (Windows NT, macOS)
Code in ring 0	Very large (tens of millions of lines)	Very small	Large
Driver crash	Can crash the whole system	Server restarts, kernel survives	Usually crashes the system
Call between subsystems	Function call	Message plus context switch	Function call or message
Typical use	Servers, phones, containers	Embedded, safety-critical	Desktop operating systems

Comparison of kernel architectures.

Linux is monolithic but not static. Drivers and file systems can be built as **loadable kernel modules**, which are linked into the running kernel on demand; the modularity is organisational, because a loaded module runs in ring 0 with full privilege.

Where a Java program sits

A Java program never invokes the kernel directly. Every request passes through the layers shown below. The JVM translates bytecode into native code and implements Java library methods with native methods. Those native methods call functions in the C library, and the C library contains the thin wrappers that execute the system call instruction.

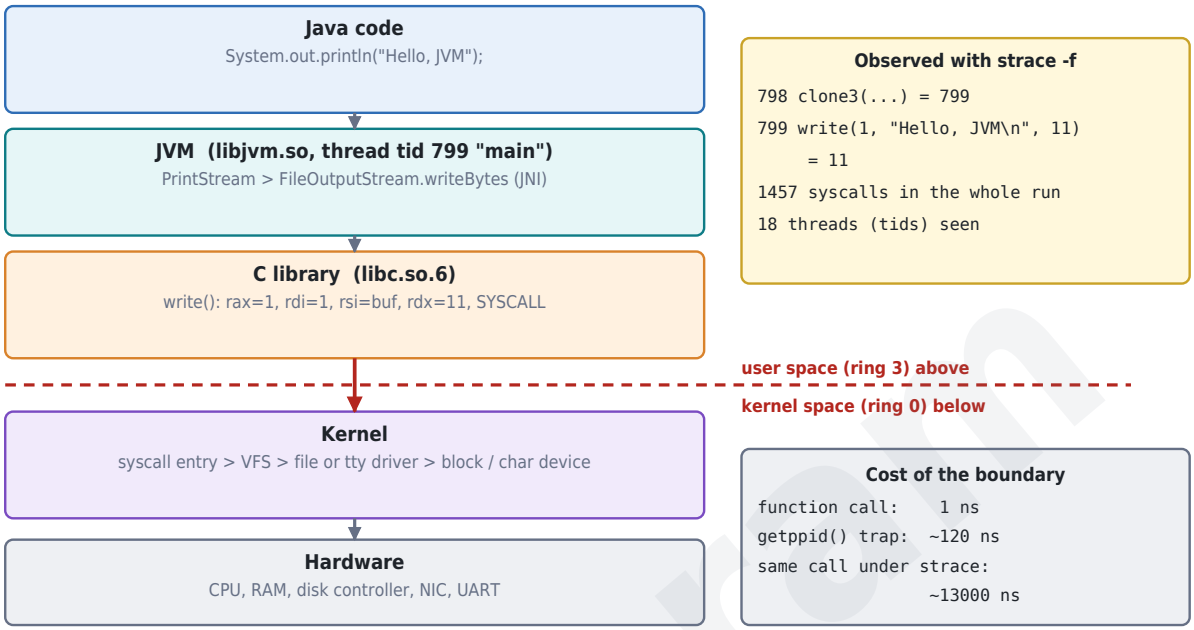


Figure 1.4. The call `System.out.println` reaches the kernel as one `write` system call; the numbers in the side panels were measured on the build machine and appear again in the worked trace and in the system call chapter.

Two consequences follow. First, any operation that touches a device, the clock, the network or the memory map has a cost that includes a ring transition. Second, every observable behaviour of a JVM is ultimately a sequence of system calls, which makes tools that observe system calls, principally `strace`, applicable to Java programs without changes to the code.

1.1.2 Worked trace

The following steps trace `System.out.println("Hello, JVM")` from a terminal. The values come from `strace -f` runs of the program `Hello` on the build machine (Linux 6.18.44, OpenJDK 21.0.12); thread identifiers vary between runs.

1. The shell calls `fork` and then `execve("/usr/bin/java", ...)`. The kernel replaces the address space of the child with that of the `java` launcher. The launcher has thread identifier 798 in this run.
2. The launcher loads `libjvm.so` with `openat` and `mmap`, then calls `clone3` with the flags `CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD`. The new native thread, identifier 799, becomes the Java `main` thread. Linux treats it as a task that shares the address space of its creator.
3. The JVM maps `lib/modules` (the class library image, about 140 MB) and the class data sharing archive `classes.jsa`, then starts the garbage collector, compiler and housekeeping threads. Eighteen thread identifiers appear in the full trace.
4. The bytecode of `main` calls `PrintStream.println`. The text is encoded to bytes and handed to `FileOutputStream.writeBytes`, a native method.
5. The native method calls the C library function `write(1, buf, 11)`. The wrapper places 1 in `rax` (the number of `write` in the x86-64 system call table), the descriptor 1 in `rdi`, the buffer address in `rsi` and the length 11 in `rdx`, and executes `SYSCALL`.
6. The processor switches to ring 0, loads the kernel stack and jumps to the system call entry code. The kernel indexes its system call table with `rax`, runs `ksys_write`, which resolves descriptor 1 to the terminal or pipe through the virtual file system, and copies 11 bytes into it.

7. The kernel places the return value 11 in `rax` and executes `SYSRET`. The processor returns to ring 3. The wrapper returns 11 to the JVM, which returns normally to the Java code.

The trace line for step 5 is:

The single system call behind println TERMINAL

```
$ strace -f -o java.full java Hello > /dev/null
$ grep 'write(1' java.full
799   write(1, "Hello, JVM\n", 11 <unfinished ...>
$ wc -l java.full
1821 java.full
```

The `<unfinished ...>` marker is an artefact of `strace -f`: another thread produced output between the entry and the exit of the call, so `strace` prints the entry and the result on separate lines.

1.1.3 Hands-on

The programs of this chapter are short. `Hello` prints one line and `Sleeper` prints its process identifier and then sleeps for the number of milliseconds given as its argument.

Hello.java JAVA

```java
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, JVM");
    }
}
```

Sleeper.java JAVA

```java
public class Sleeper {
    public static void main(String[] args) throws Exception {
        System.out.println("pid=" + ProcessHandle.current().pid());
        Thread.sleep(Long.parseLong(args[0]));
    }
}
```

Identify the kernel, the distribution and the first process

Kernel, distribution and process 1 on the build machine TERMINAL

```
$ uname -srm
Linux 6.18.44-fc-v139 x86_64
$ head -2 /etc/os-release
PRETTY_NAME="Ubuntu 24.04.5 LTS"
NAME="Ubuntu"
$ ps -p 1 -o pid,comm,args | cut -c1-70
  PID COMMAND         COMMAND
    1 process_api     /process_api --firecracker-init --addr 0.0.0.0:2
$ java -version 2>&1 | head -1
openjdk version "21.0.12.1" 2026-08-18
```

The kernel version is independent of the distribution release. The first process on this machine is not `systemd`; the machine is a micro virtual machine whose kernel starts a purpose-built init program directly. The boot chapter explains the difference. On a typical server or in a full virtual machine, `ps -p 1` reports `systemd`, and inside a container it reports the container's entry point.

Observe the native threads of a JVM

The JVM maps each Java platform thread to one kernel thread, and the kernel exposes every thread as an entry in `/proc/PID/task`. The program `Sleeper` prints its process identifier and sleeps; it was started with `-Xmx64m -Xss512k`.

Threads of an idle JVM (typical output; identifiers vary) TERMINAL

```
$ java -Xmx64m -Xss512k Sleeper 60000 &
pid=8323
$ ls /proc/8323/task | wc -l
18
$ for t in $(ls /proc/8323/task); do echo "$t $(cat /proc/8323/task/$t/comm)"; done | head -8
8323 java
8325 java
8326 GC Thread#0
8327 G1 Main Marker
```

Threads of an idle JVM (typical output; identifiers vary) (continued) — TERMINAL

```
8328 G1 Conc#0
8329 G1 Refine#0
8330 G1 Service
8331 VM Periodic Tas
$ grep -E 'State|Threads|VmSize|VmRSS' /proc/8323/status
State:  S (sleeping)
VmSize:  2646632 kB
VmRSS:     40740 kB
Threads:        18
```

An idle JVM that executes `Thread.sleep` owns eighteen kernel threads, 2.6 GB of virtual address space and 40 MB of resident memory. The thread named `java` with the lowest identifier is the launcher thread; the second is the Java `main` thread. The comm field is limited to 15 characters, which explains the truncated names. The large gap between virtual size and resident size is a first example of the difference between address space reserved by the JVM and memory that the kernel has actually backed with physical pages. The memory part of this book treats it fully.

Compare the cost of starting a native and a managed program

Elapsed time of a C hello and a Java hello (measured, typical) — TERMINAL

```
$ time ./hello > /dev/null
real    0m0.002s
user    0m0.001s
sys     0m0.000s
$ time java Hello > /dev/null
real    0m0.046s
user    0m0.039s
sys     0m0.013s
```

The native program issues 34 system calls; the JVM issues about 1450, creates 17 additional threads and maps hundreds of megabytes of address space. Most of the 46 ms is the work of preparing the runtime, which is the reason that short-lived command-line tools are rarely written in Java and that serverless platforms care about JVM start-up.

1.1.4 Real-world incidents

Incident: Spring Boot service cannot bind port 80 after migration to a non-root user

Scenario. A team hardens a Spring Boot service by running it under an unprivileged account instead of root. The service previously listened on port 80 behind a load balancer health check. The new release starts on the staging host, which runs as root during tests, and fails on the production host.

Symptoms. The unit enters a restart loop. The application log contains `java.net.BindException: Permission denied` shortly after the banner, and the load balancer marks every instance unhealthy.

Diagnosis. The exception text is the message of the system call error `EACCES`. The first step is to confirm which call failed and what the kernel policy is. The reproduction uses a minimal server socket program.

Bind.java — JAVA

```java
import java.net.ServerSocket;

public class Bind {
    public static void main(String[] args) throws Exception {
        int port = Integer.parseInt(args[0]);
        try (ServerSocket s = new ServerSocket(port)) {
            System.out.println("listening on " + s.getLocalPort());
        }
    }
}
```

Reproduction as uid 65534 (the nobody account) — TERMINAL

```
$ id -u
0
$ java Bind 80
listening on 80
$ setpriv --reuid=65534 --regid=65534 --clear-groups java Bind 80
Exception in thread "main" java.net.BindException: Permission denied
        at java.base/sun.nio.ch.Net.bind0(Native Method)
        at java.base/sun.nio.ch.Net.bind(Net.java:577)
        ...
```

Reproduction as uid 65534 (the nobody account) (continued) TERMINAL

```
$ setpriv --reuid=65534 --regid=65534 --clear-groups \
    strace -f -e trace=bind java Bind 80 2>&1 | grep 'bind('
5260  bind(4, {sa_family=AF_INET, sin_port=htons(80), ...}, 16) = -1 EACCES (Permission denied)
$ cat /proc/sys/net/ipv4/ip_unprivileged_port_start
1024
```

The `strace` line shows the failing system call, `bind`, with the result `EACCES`. The sysctl `net.ipv4.ip_unprivileged_port_start` is the policy: ports below 1024 may be bound only by a process that holds the capability `CAP_NET_BIND_SERVICE` in its effective set. The root user passes the check because root holds all capabilities, which is also why the staging tests succeeded.

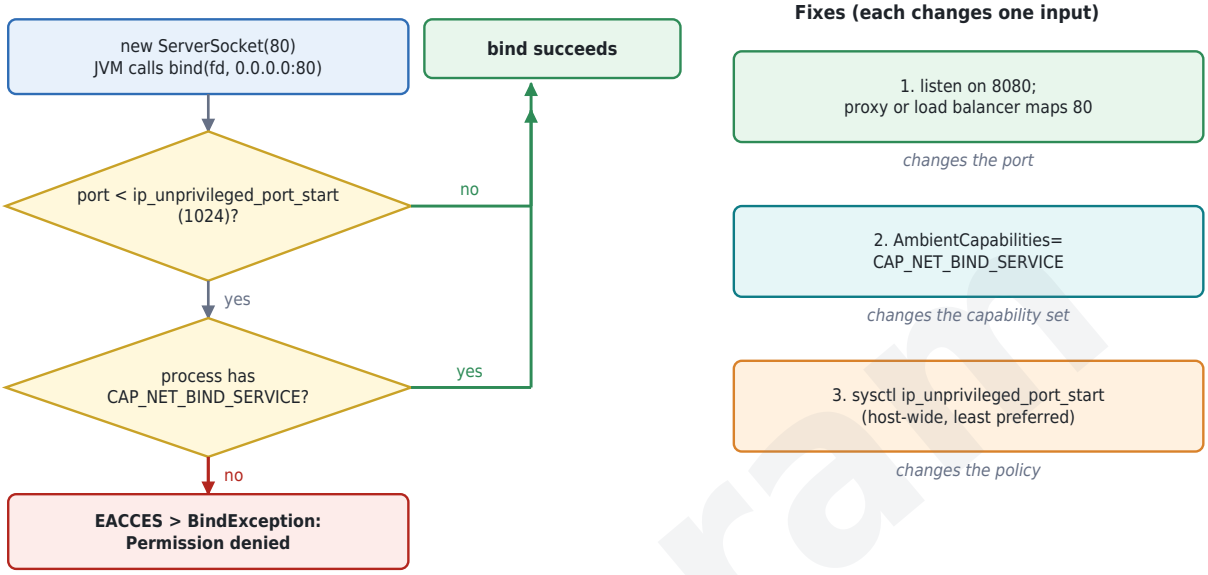


Figure 1.5. The kernel decides the outcome of `bind` from two facts, the port number and the capability set of the caller; each fix changes one of those facts.

Root cause. The service was moved to an unprivileged account without granting it the right to use a privileged port. The failure is deterministic and independent of the Java code.

Fix. The immediate mitigation is to move the application to a port above 1023, for example 8080, and to let the load balancer or a reverse proxy listen on port 80. The permanent options, in order of preference, are:

1. Bind to port 8080 and map the port externally (container port mapping, load balancer listener, or an `iptables` redirect). No privilege is involved.
2. Grant only the needed capability to the process. With systemd this is `AmbientCapabilities=CAP_NET_BIND_SERVICE`. The following run shows the effect with `setpriv`:

The same unprivileged user with one capability TERMINAL

```
$ setpriv --reuid=65534 --regid=65534 --clear-groups \
    --inh-caps +net_bind_service --ambient-caps +net_bind_service java Bind 80
listening on 80
```

3. Lower the sysctl `net.ipv4.ip_unprivileged_port_start` for the whole host. This weakens a host-wide boundary for one service and is the least preferred option.

Prevention. Run the production start-up path under the production account in a pre-production environment. A start-up test that executes the unit as the service user would have caught the failure before release. Document the listening port in the deployment descriptor, and prefer ports above 1023 for application processes.

Incident: the orchestrator reports exit code 143 for a service that stopped cleanly

Scenario. A deployment pipeline stops a Java service by sending `SIGTERM` and treats any non-zero exit status as a failed stop. After a harmless redeployment, the pipeline marks every stop as failed and rolls the release back.

Symptoms. The service log ends with the normal shutdown messages, but the supervisor records `exited, code=143` or `Terminated`. Container dashboards show exit code 143 with no error in the log.

Diagnosis. A shell and most supervisors encode death by signal as 128 plus the signal number. The statuses can be reproduced without any application bug. The program `Hook` registers a shutdown hook and sleeps.

Hook.java JAVA

```java
public class Hook {
    public static void main(String[] args) throws Exception {
        Runtime.getRuntime().addShutdownHook(
                new Thread(() -> System.out.println("shutdown hook ran")));
        System.out.println("started");
        Thread.sleep(60000);
    }
}
```

Exit status after SIGTERM and SIGKILL TERMINAL

```
$ java Sleeper 60000 > s2.out & P=$!; sleep 2; kill -TERM $P; wait $P; echo "TERM: $?"
TERM: 143
$ java Sleeper 60000 > s3.out & P=$!; sleep 2; kill -KILL $P; wait $P; echo "KILL: $?"
KILL: 137
$ java Hook > h.out & P=$!; sleep 2; kill -TERM $P; wait $P; echo "exit=$?"; cat h.out
exit=143
started
shutdown hook ran
```

The last run uses a program that registers a shutdown hook. The hook ran, so the stop was orderly, and the status was still 143. The JVM handles `SIGTERM` by running the shutdown hooks and then terminating, and the termination is reported as death by signal 15.

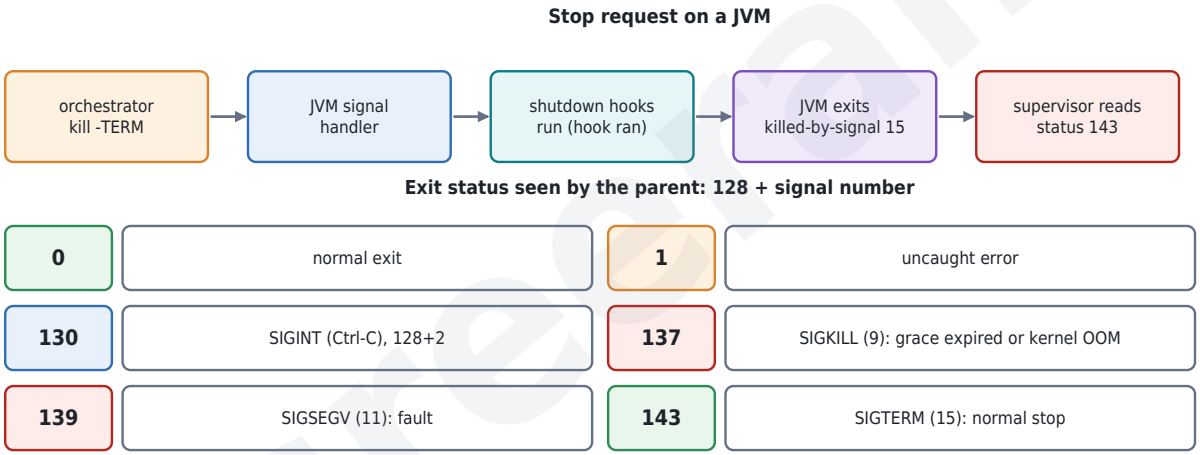


Figure 1.6. A status above 128 means that the process was ended by signal (status minus 128); 143 is the normal result of a polite stop, 137 is the result of a forced kill and usually indicates that the grace period expired or that the kernel killed the process for memory.

Root cause. The pipeline interpreted any non-zero status as failure. The status 143 is the expected status of a JVM that received `SIGTERM`, whether or not the shutdown was clean.

Fix. Configure the supervisor to accept the status: `SuccessExitStatus=143` in a systemd unit, or compare the exit code with 143 in the pipeline. The actual health of the shutdown is better judged from the log (the shutdown hooks completed) and from the absence of status 137.

Prevention. Treat 143 as success and 137 as an alert condition. A 137 after a stop request means that the process ignored or outlived `SIGTERM` and was killed by `SIGKILL` after the grace period (`TimeoutStopSec` in systemd, `terminationGracePeriodSeconds` in Kubernetes); in a container without a memory limit increase it can also be the kernel out-of-memory killer, which the memory part distinguishes by reading the kernel log.

1.1.5 Questions to expect

Questions and model answers

1. **What is the difference between the kernel and the operating system?** The kernel is the privileged core that manages the processor, memory and devices. The operating system, in the common sense, also includes the system libraries, the init system and the utilities that make the kernel usable. Linux is a kernel; Ubuntu is an operating system built around it.
2. **Why do user programs call the kernel through system calls instead of jumping into it?** The processor allows ring 3 code to enter ring 0 only at addresses that the kernel registered. This prevents arbitrary code from running with kernel privilege and lets the kernel validate every argument.
3. **Name the privilege levels used by Linux on x86-64.** Ring 0 for the kernel and ring 3 for user programs. Rings 1 and 2 exist in the architecture but are not used.
4. **Compare monolithic and microkernel designs.** A monolithic kernel runs file systems, drivers and the network stack in one privileged address space: calls are fast and a driver fault can crash the system. A microkernel keeps only scheduling, IPC and address-space management in ring 0 and runs the rest as servers: faults are contained and each service call costs messages and context switches.
5. **Is Linux a microkernel because it has loadable modules?** No. Modules are loaded into ring 0 and share the kernel address space; they provide modularity of packaging, not isolation.
6. **How does a Java `println` reach the screen?** Bytecode calls `PrintStream`, which calls a native method; the native method calls libc `write`, which executes `SYSCALL`; the kernel resolves the file descriptor through the virtual file system to a terminal or pipe and copies the bytes.
7. **A Java thread corresponds to what in Linux?** One kernel task created by `clone`, listed under `/proc/PID/task`, scheduled independently. Virtual threads in Java 21 are different: they are scheduled by the JVM onto a small number of platform threads.
8. **What does exit status 139 mean?** The process was killed by signal 11 (`SIGSEGV`), because 139 is 128 plus 11. Status 143 is `SIGTERM` and status 137 is `SIGKILL`.
9. **Why can a non-root service not listen on port 80, and how is it permitted safely?** Binding below 1024 requires `CAP_NET_BIND_SERVICE`. The service is granted that capability only, or it listens on a high port behind a proxy or load balancer.

KEY POINTS

The operating system multiplexes, protects and abstracts the hardware; the kernel is its privileged part and runs in ring 0, while every other program, including the JVM, runs in ring 3.

- Linux is a monolithic kernel with loadable modules; a distribution adds the user-space system.
- A Java program reaches the kernel through native methods, libc wrappers and the `SYSCALL` instruction; each Java platform thread is a kernel task visible in `/proc/PID/task`.
- Privileged instructions in user mode cause a fault that the kernel turns into `SIGSEGV`.
- Exit statuses above 128 encode death by signal: 137 is `SIGKILL`, 139 is `SIGSEGV`, 143 is `SIGTERM`.
- Policy failures such as `EACCES` on `bind` come from kernel checks on capabilities, not from the Java code.

1.2 Hardware Essentials for Operating Systems

An operating system is a program written for a particular machine, and several of its central mechanisms exist only because of properties of that machine: processors are far faster than memory, devices are far slower than processors, and several processors share one memory. This chapter reviews the hardware concepts needed to understand scheduling, virtual memory, interrupts and synchronisation, and it measures the memory hierarchy on the build machine, because the numbers explain behaviour that otherwise appears arbitrary.

1.2.1 Concept and mechanism

Processor state and modes

The state of a running thread is small: the general-purpose registers, the instruction pointer (`RIP`), the stack pointer (`RSP`), the flags register and the segment and control registers. Switching from one thread to another means saving this state of the old thread in kernel memory and loading the state of the new one. That operation is a **context switch**, and its cost, together with the cost of the cache and TLB contents that become useless, is why thread counts matter.

Besides the privilege level, the processor has a mode that selects how addresses are interpreted. In 64-bit mode on the build machine (an Intel Xeon reporting 52 physical and 57 virtual address bits) the processor translates every address used by a program through page tables that the kernel maintains. The translation hardware is the memory management unit (MMU). It is what makes the isolation promised in the previous chapter enforceable, and the memory part of this book covers it in detail.

The memory hierarchy

A processor executes several instructions per nanosecond but main memory needs about a hundred nanoseconds to answer a request. Caches bridge the gap. Each level is smaller and faster than the one below it, and data moves between levels in fixed-size blocks called **cache lines**, 64 bytes on this machine.

Level	Size	Measured latency	Notes
Registers	about 16 general	below 1 ns	Named in the instruction
L1 data cache	48 KiB per core	1.7 to 2.0 ns	Private to a core
L2 cache	2 MiB per core	5.0 to 6.2 ns	Private to a core
L3 cache	reported 260 MiB	41 ns at 4 MiB	Shared; the host shares it with other tenants
Main memory (DRAM)	8 GiB	80 to 176 ns	Reached through the memory controller
SSD or virtual disk	hundreds of GiB	tens of microseconds and above	Block device; reached through the kernel

Memory hierarchy of the build machine (sizes from lscpu and sysfs; latencies measured).

The latency column came from a program that follows a chain of pointers through a randomly permuted array, so that every load depends on the previous one and the hardware prefetcher cannot hide the delay. The source is included because the technique is reusable.

lat.c C

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

static double now_ns(void) {
    struct timespec t;
    clock_gettime(CLOCK_MONOTONIC, &t);
    return t.tv_sec * 1e9 + t.tv_nsec;
}

static double chase(size_t bytes) {
    size_t n = bytes / sizeof(void *);
    void **a = malloc(n * sizeof(void *));
    size_t *idx = malloc(n * sizeof(size_t));
    for (size_t i = 0; i < n; i++) idx[i] = i;
    srand(42);
    for (size_t i = n - 1; i > 0; i--) {
        size_t j = (size_t)rand() % (i + 1);
        size_t t = idx[i]; idx[i] = idx[j]; idx[j] = t;
    }
    for (size_t i = 0; i < n; i++) a[idx[i]] = &a[idx[(i + 1) % n]];
    void **p = &a[idx[0]];
    long steps = 20000000;
    for (long i = 0; i < 1000000; i++) p = (void **)*p;
    double t0 = now_ns();
    for (long i = 0; i < steps; i++) p = (void **)*p;
    double t1 = now_ns();
    if (p == NULL) puts("x");
    free(a); free(idx);
    return (t1 - t0) / steps;
```

lat.c (continued) C

```c
}

int main(void) {
    size_t kb[] = {16, 32, 256, 1024, 4096, 16384, 65536, 131072};
    for (int i = 0; i < 8; i++)
        printf("%7zu KiB : %6.1f ns per dependent load\n", kb[i], chase(kb[i] * 1024));
    return 0;
}
```

Measured on the build machine (typical output) TERMINAL

```
$ gcc -O2 -o lat lat.c && (ulimit -v 600000; timeout 120 ./lat)
     16 KiB :    1.7 ns per dependent load
     32 KiB :    2.0 ns per dependent load
    256 KiB :    5.0 ns per dependent load
   1024 KiB :    6.2 ns per dependent load
   4096 KiB :   41.1 ns per dependent load
  16384 KiB :   80.6 ns per dependent load
  65536 KiB :  148.8 ns per dependent load
 131072 KiB :  175.9 ns per dependent load
```

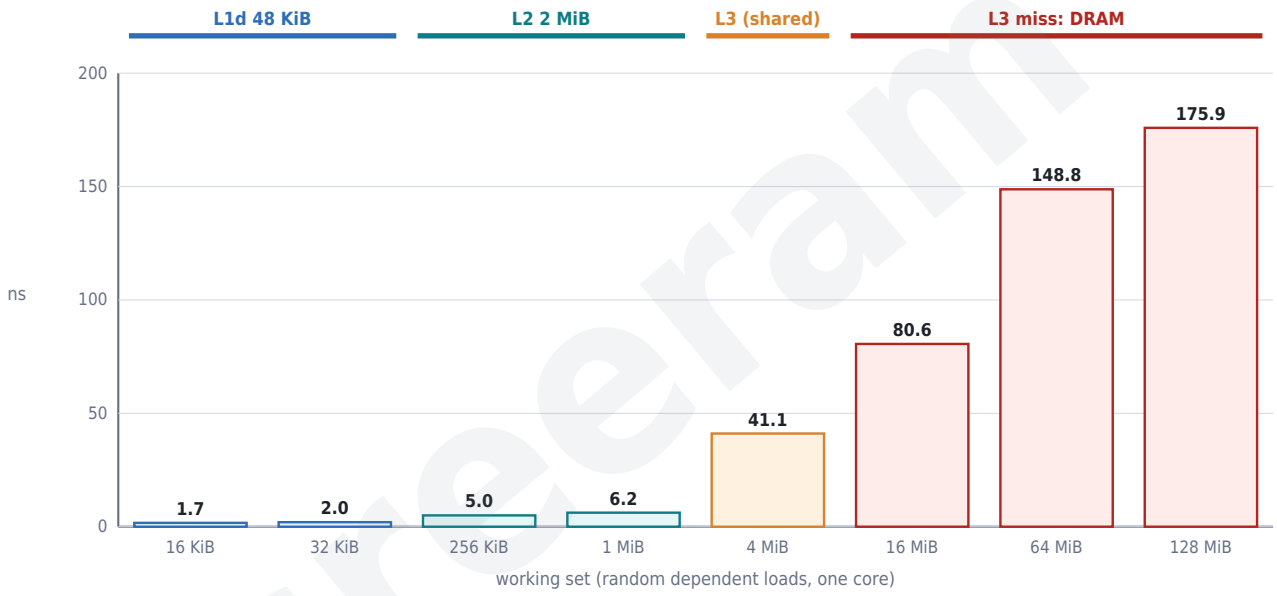


Figure 1.7. Latency rises in steps at the boundaries of the cache levels: the jump between 1 MiB and 4 MiB marks the end of the 2 MiB L2 cache, and the final plateau is main memory. The 260 MiB L3 reported by `lscpu` is not usable by one tenant, because the hardware is shared.

The practical law that follows is **locality**. Temporal locality means that recently used data is likely to be used again; spatial locality means that data near recently used data is likely to be used next. Programs and data structures that respect locality run several times faster than equivalent ones that do not, with identical instruction counts.

Interrupts, exceptions and the timer

Devices complete work at times that the processor cannot predict. Instead of polling, the processor lets a device raise an **interrupt**: an electrical or message signal that makes the processor stop the current instruction stream after the current instruction, save a small amount of state, switch to ring 0 and run a handler found through the interrupt descriptor table. Three kinds of event use this mechanism.

- **Hardware interrupts** come from devices (a network card with a received packet, a disk with a completed request) and from the timer.
- **Exceptions** are raised by the executing instruction itself: a page fault, a division by zero, an illegal instruction. The `HLT` experiment of the previous chapter was an exception.
- **Software interrupts and system calls** are deliberate entries into the kernel by the program.

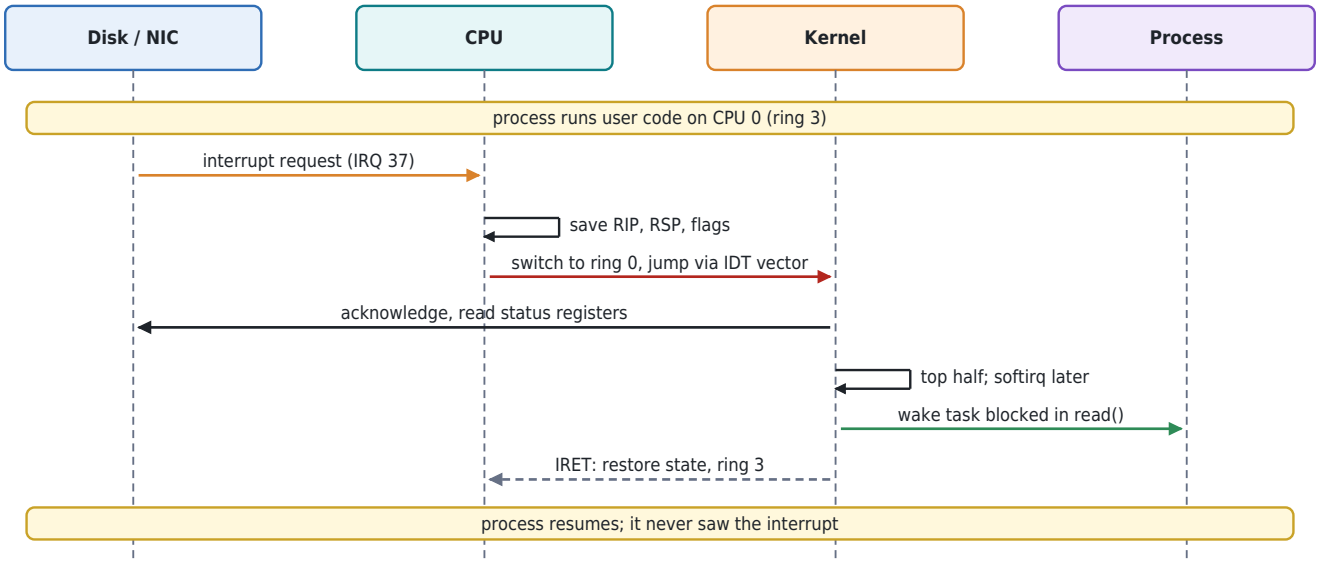


Figure 1.8. An interrupt suspends a process without its knowledge: the kernel handles the device in ring 0, may wake a blocked task, and returns to the interrupted code with its registers intact.

The kernel splits interrupt work in two. The **top half** runs with interrupts masked and does the minimum, for example acknowledging the device and queueing the data. The **bottom half**, called a softirq or a tasklet, runs shortly afterwards with interrupts enabled and does the bulk of the work, for example protocol processing. Both are visible in `/proc`.

Interrupt and softirq counters (typical output on the 2-CPU build machine) TERMINAL

```
$ head -3 /proc/interrupts
           CPU0       CPU1
 24:          1          0  IO-APIC   5-edge      ACPI:Ged
 25:          0          1  IO-APIC   6-edge      ACPI:Ged
$ grep -E 'virtio7|virtio1-req' /proc/interrupts
 37:          0       9194 PCI-MSIX-0000:00:02.0   1-edge      virtio1-req.0
 49:      10720          0 PCI-MSIX-0000:00:08.0   1-edge      virtio7-input.0
 50:          0      10608 PCI-MSIX-0000:00:08.0   2-edge      virtio7-output.0
$ head -6 /proc/softirqs
                    CPU0       CPU1
          HI:          0          0
       TIMER:       8695       5317
      NET_TX:          3          6
      NET_RX:      11279      11541
       BLOCK:       1608      11987
$ grep -E '^ctxt|^intr' /proc/stat | cut -c1-40
intr 2095530 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
ctxt 18931868
```

The listing shows virtual devices (`virtio`) of a micro virtual machine; on bare metal the right-hand column names real network cards and disk controllers (`nvme0q1`, `eth0-TxRx-0`). The counters per CPU reveal whether interrupt load is spread: one CPU with a counter far above the others is a common cause of an uneven load. The `ctxt` line of `/proc/stat` counts all context switches since boot, and `intr` counts all interrupts.

The **timer interrupt** gives the kernel the periodic control that user programs cannot take away. On the build machine the kernel is configured with `CONFIG_HZ=250` (a periodic tick every 4 ms), and modern kernels additionally use high-resolution timers programmed for exact deadlines (`HRTIMER` row). At each tick the scheduler can decide that the running thread has used its time and must yield. Without a timer interrupt, a thread that never calls the kernel would hold the processor indefinitely.

Direct memory access

Copying a megabyte from a disk controller into memory one word at a time would occupy the processor for the entire duration of the transfer. **Direct memory access** (DMA) lets the controller write to memory by itself. The kernel programs the transfer, the processor runs other threads, and the device raises an interrupt when the transfer is complete.

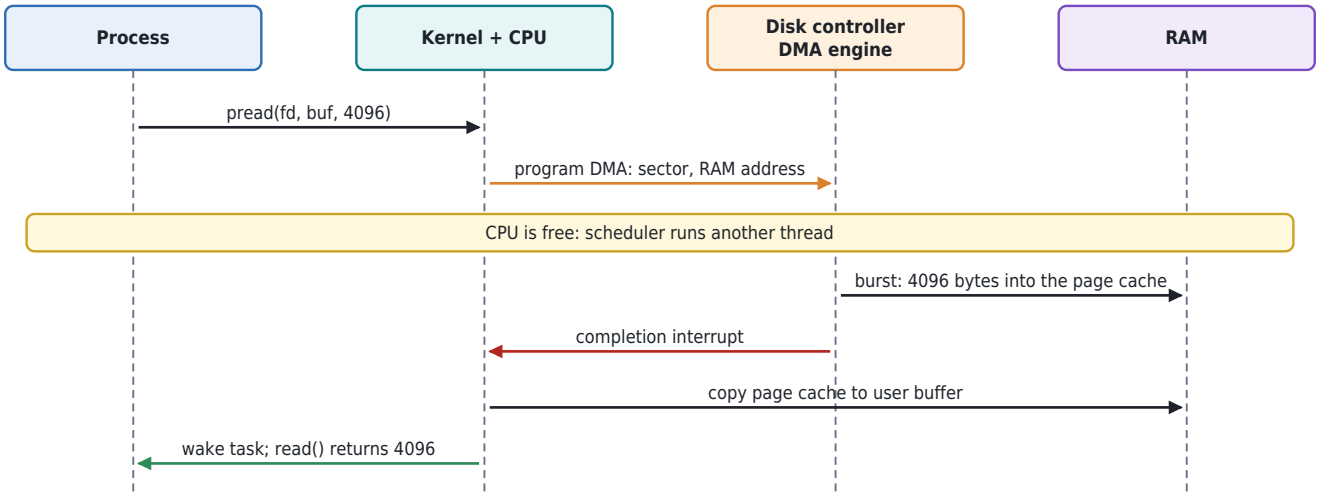


Figure 1.9. During a DMA read the processor is not involved in moving data; the kernel blocks the requesting thread, schedules another and is woken by the completion interrupt. This is the reason that blocking I/O does not waste a core.

The consequence for application design is that an I/O wait is cheap for the processor, expensive for latency, and visible as the state of the waiting thread (state `S` or `D` in `ps`) and as the `wa` figure in `top` and `vmstat`.

Multicore processors and cache coherence

Each core has private L1 and L2 caches and shares an L3 cache and main memory with the other cores. When two cores cache the same line and one of them writes, the other copy becomes stale. Hardware **cache coherence protocols**, of which MESI is the textbook form, prevent this by tracking the state of every line and invalidating other copies before a write completes.

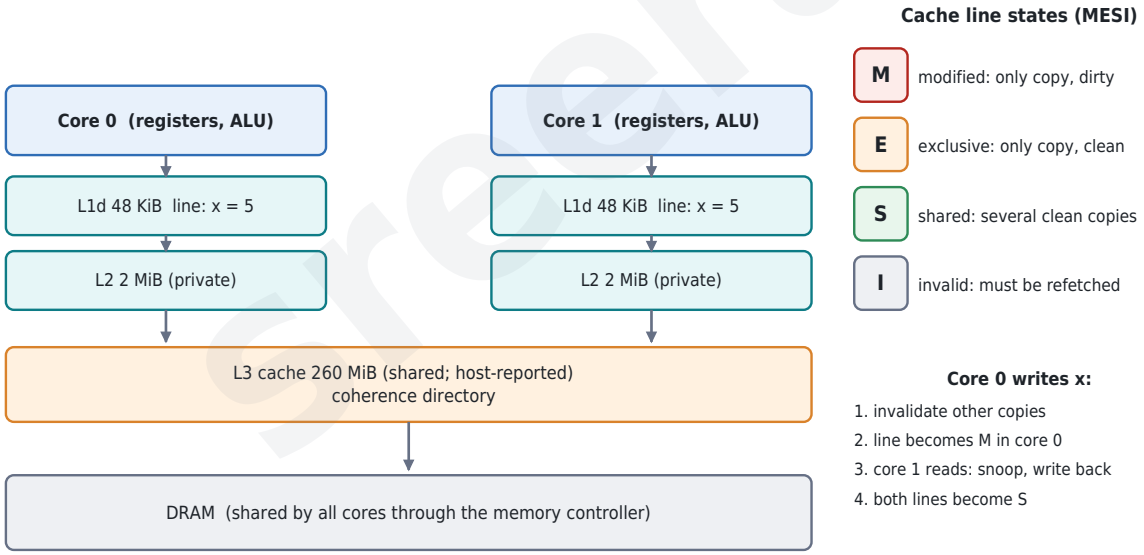


Figure 1.10. A write by one core forces other cores to drop their copy of the line, so repeated writes to one line from several cores make the line bounce between caches; the cost is paid in nanoseconds per operation, not in correctness.

Coherence makes a shared-memory program correct at the hardware level, but not necessarily at the language level. Compilers and processors reorder independent operations, and a store may sit in a core's store buffer before other cores see it. The Java Memory Model and the `volatile` and `synchronized` constructs exist to restore ordering where it matters; the concurrency part of this book explains how they map to memory barriers. A related performance hazard is **false sharing**: two threads that write different variables placed on the same 64-byte line cause the line to bounce between their cores even though no data is shared logically. The standard remedy is padding or, in the JDK, the internal `@Contended` annotation.

NOTE
False sharing is real but difficult to measure on a shared virtual machine. Repeated trials of a two-thread counter on the build machine, with the counters adjacent or 128 bytes apart, gave overlapping timings (between 630 and 1210 ms per run), so the effect is not claimed here as a measurement. On dedicated hardware with pinned cores the difference is typically clear.

1.2.2 Worked trace

A load that misses every cache level

The following sequence traces a read of a variable whose address is not in any cache, on the machine described above. The timings are the measured latencies of the table.

1. The thread executes `mov rax, [rbx]`. The MMU translates the virtual address in `rbx` to a physical address (the translation itself may need page-table reads, which are cached in the TLB; the memory part of this book follows it).
2. The core looks up the physical address in L1d, a 48 KiB cache of 64-byte lines. The lookup misses (about 2 ns have passed since the previous level began).
3. The L2 lookup misses, which adds about 5 ns of latency.
4. The L3 lookup misses. The request leaves the core through the shared interconnect.
5. The memory controller reads the 64-byte line from DRAM. Total latency is between 80 ns (16 MiB working set) and 176 ns (128 MiB working set) in the measurements.
6. The line is installed in L3, L2 and L1d, and `rax` is loaded. The following 7 loads from the same line (8 bytes each) hit in L1d and cost about 2 ns in total.

At a clock of 2.1 GHz, 100 ns correspond to 210 clock cycles, in which the core could have executed several hundred instructions. The prefetcher hides most of that for sequential access and none of it for random access, which is why the random-pointer experiment is slow and a sequential scan of the same memory is fast.

1.2.3 Hands-on

Inspect the machine

Processor and cache facts (excerpt of lscpu and sysfs) TERMINAL

```
$ lscpu | grep -E 'Model name|^CPU\(s\)|Thread|Core|L1d|L2|L3|Hypervisor'
CPU(s):                               2
Model name:                           Intel(R) Xeon(R) Processor @ 2.10GHz
Thread(s) per core:                   1
Core(s) per socket:                   2
Hypervisor vendor:                    KVM
L1d cache:                            96 KiB (2 instances)
L2 cache:                             4 MiB (2 instances)
L3 cache:                             260 MiB (1 instance)
$ for i in 0 1 2 3; do d=/sys/devices/system/cpu/cpu0/cache/index$i; \
    echo $(cat $d/level) $(cat $d/type) $(cat $d/size) line=$(cat $d/coherency_line_size); done
1 Data 48K line=64
1 Instruction 32K line=64
2 Unified 2048K line=64
3 Unified 266240K line=64
```

The `Hypervisor vendor` line confirms that this machine is itself a guest, and that the figures for caches describe the host's hardware. On a cloud instance the same applies, which is one reason why measurements on shared virtual machines vary from run to run.

Measure the effect of access order

The next program sums a 4096 by 4096 matrix of `int` twice, once row by row and once column by column. Both versions perform 16,777,216 additions.

CacheDemo.java JAVA

```
public class CacheDemo {
    static final int N = 4096;

    static long sumRowMajor(int[][] m) {
        long s = 0;
        for (int i = 0; i < N; i++)
```

CacheDemo.java (continued) JAVA

```
            for (int j = 0; j < N; j++) s += m[i][j];
        return s;
    }

    static long sumColMajor(int[][] m) {
        long s = 0;
        for (int j = 0; j < N; j++)
            for (int i = 0; i < N; i++) s += m[i][j];
        return s;
    }

    public static void main(String[] args) {
        int[][] m = new int[N][N];
        for (int i = 0; i < N; i++) java.util.Arrays.fill(m[i], 1);
        for (int r = 0; r < 3; r++) {
            long t0 = System.nanoTime();
            long a = sumRowMajor(m);
            long t1 = System.nanoTime();
            long b = sumColMajor(m);
            long t2 = System.nanoTime();
            System.out.printf("row-major %4d ms, column-major %4d ms (sum %d, %d)%n",
                    (t1 - t0) / 1_000_000, (t2 - t1) / 1_000_000, a, b);
        }
    }
}
```

Result (typical output; Java 21, 2 vCPUs, -Xmx300m) TERMINAL

```
$ java -Xmx300m CacheDemo.java
row-major   17 ms, column-major  211 ms (sum 16777216, 16777216)
row-major   12 ms, column-major  192 ms (sum 16777216, 16777216)
row-major    5 ms, column-major  180 ms (sum 16777216, 16777216)
```

Row-major order reads consecutive addresses, so each 64-byte cache line serves 16 consecutive `int` values and the prefetcher keeps ahead of the loop. Column-major order touches a different row, and therefore a different cache line, on every access, and the 64 MiB matrix is far larger than the caches; each access pays a miss. The result is identical, the instruction count is nearly identical, and the second version is between 10 and 36 times slower.

Observe interrupts and context switches while a workload runs

Context switches in one second (typical; the machine is shared with other workloads) TERMINAL

```
$ grep -E '^ctxt' /proc/stat; sleep 1; grep -E '^ctxt' /proc/stat
ctxt 30766906
ctxt 30769624
```

The difference, 2,718 switches in one second on a machine with two cores, is the background activity of the system: every wake-up of a thread, every blocked call and every preemption is one switch. The figure becomes meaningful in comparison with a baseline, which the diagnostic chapters build with `vmstat` (column `cs`).

1.2.4 Real-world incidents

Incident: a nightly batch job becomes ten times slower after a refactoring that changed only the loop order

Scenario. A reporting job aggregates a large two-dimensional array of measurements held as `double[sensors][timestamps]`. A refactoring renames the loop variables and, in the process, swaps the nesting so that the inner loop iterates over sensors. The unit tests, which use ten by ten arrays, pass. The job runs on a Kubernetes node with a four-core limit.

Symptoms. The batch window grows from 25 minutes to more than three hours. CPU utilisation stays at 100 percent of one core, the heap is unchanged, garbage collection is quiet, and no error appears in any log. Profilers attribute the time to the innermost loop.

Diagnosis. CPU time without waiting, no GC, and a hot loop point to the memory system. The commands, in order, were:

1. `top -H -p PID` shows one thread at 100 percent in state `R` and `us` (user time) near 100 percent, so the process is computing, not waiting for I/O or the kernel.

2. `jcmd PID Thread.print` and a few repeated samples show the thread inside the aggregation method every time.
3. A micro-benchmark of the loop in isolation reproduces the slowdown with the sizes of production data (see the listing earlier in this chapter: 5 ms against 180 ms for the same additions).
4. `git diff` of the method shows the swapped loop order; no algorithmic change exists.

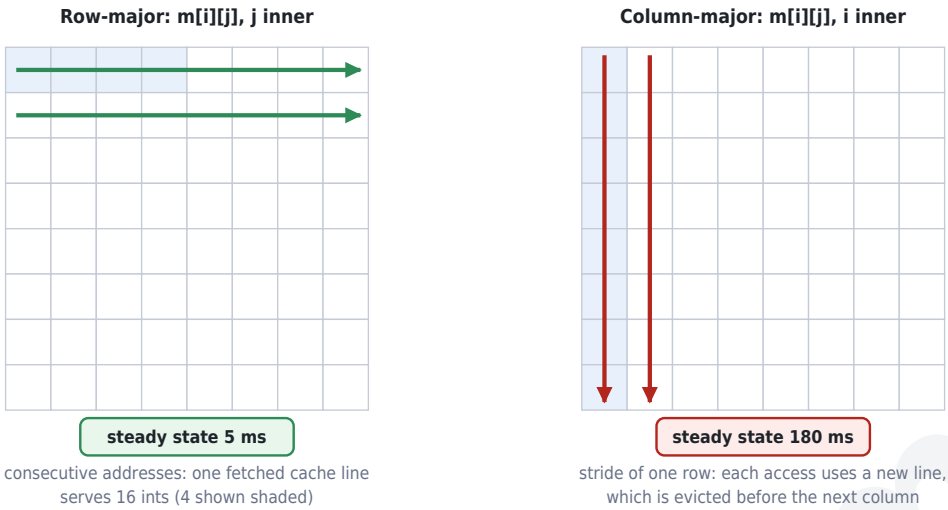


Figure 1.11. Java stores each row of a two-dimensional array contiguously; the loop that walks along a row uses every byte of each fetched cache line, the loop that walks down a column uses 4 bytes of each and evicts the line before it is needed again.

Root cause. The new loop order accessed memory with a stride of one row (32 KiB for 4096 doubles) so that every access missed the L1 and L2 caches and, for a working set above the cache sizes, paid the latency of main memory. The instruction count did not change; the number of cache misses grew by roughly the ratio of the cache line size to the element size: 8 for doubles and 16 for the 4-byte integers of the experiment.

Fix. Restore the row-major order, or iterate in blocks (tiling) when both orders are needed. Add a benchmark with realistic dimensions to the build.

Prevention. Small unit-test inputs cannot detect locality regressions, because they fit in the cache. A performance regression test with production-sized data, executed in continuous integration with a time budget, would have failed. Hardware counters (`perf stat` on a host where it is available, reporting cache misses and instructions per cycle) identify the cause directly; they are not available in the build environment of this book, so the evidence above is the measured timing.

Incident: a latency-sensitive service slows down after switching from an array of primitives to a list of boxed values

Scenario. A price-aggregation service holds four million integer prices. A developer replaces an `int[]` with an `ArrayList<Integer>` to use the stream API conveniently. Entries arrive in random order and are inserted at random positions.

Symptoms. Throughput of the aggregation falls, the young-generation garbage collections become more frequent, and the 99th-percentile latency rises, although the total heap use grows by only a modest amount.

Diagnosis. The traversal of the collection is measured in isolation on the same hardware.

Layout.java — JAVA

```java
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Random;

public class Layout {
    public static void main(String[] args) {
        int n = 4_000_000;
        int[] prim = new int[n];
        List<Integer> boxed = new ArrayList<>(n);
        for (int i = 0; i < n; i++) { prim[i] = i % 1000 + 1000; boxed.add(prim[i]); }
        List<Integer> shuffled = new ArrayList<>(boxed);
        Collections.shuffle(shuffled, new Random(1));
        for (int r = 0; r < 4; r++) {
            long t0 = System.nanoTime();
```

Layout.java (continued) JAVA

```
            long a = 0;
            for (int v : prim) a += v;
            long t1 = System.nanoTime();
            long b = 0;
            for (int v : boxed) b += v;
            long t2 = System.nanoTime();
            long c = 0;
            for (int v : shuffled) c += v;
            long t3 = System.nanoTime();
            System.out.printf("int[] %3d ms | boxed in order %3d ms | "
                    + "boxed shuffled %3d ms (%d %d %d)%n",
                    (t1 - t0) / 1_000_000, (t2 - t1) / 1_000_000,
                    (t3 - t2) / 1_000_000, a, b, c);
        }
    }
}
```

Typical output (Java 21, -Xmx512m, 2 vCPUs; the first rounds include JIT warm-up) TERMINAL

```
$ java -Xmx512m Layout.java
int[]  14 ms | boxed in order  17 ms | boxed shuffled  63 ms (5998000000 5998000000 5998000000)
int[]  23 ms | boxed in order  68 ms | boxed shuffled 119 ms (5998000000 5998000000 5998000000)
int[]   1 ms | boxed in order   5 ms | boxed shuffled  39 ms (5998000000 5998000000 5998000000)
int[]   1 ms | boxed in order   5 ms | boxed shuffled  64 ms (5998000000 5998000000 5998000000)
```

In the steady state, the same sum takes about 1 ms over the primitive array, 5 ms over boxed objects that were allocated in order, and between 39 and 64 ms over the same objects after the list order was shuffled, which separates object order in memory from list order.

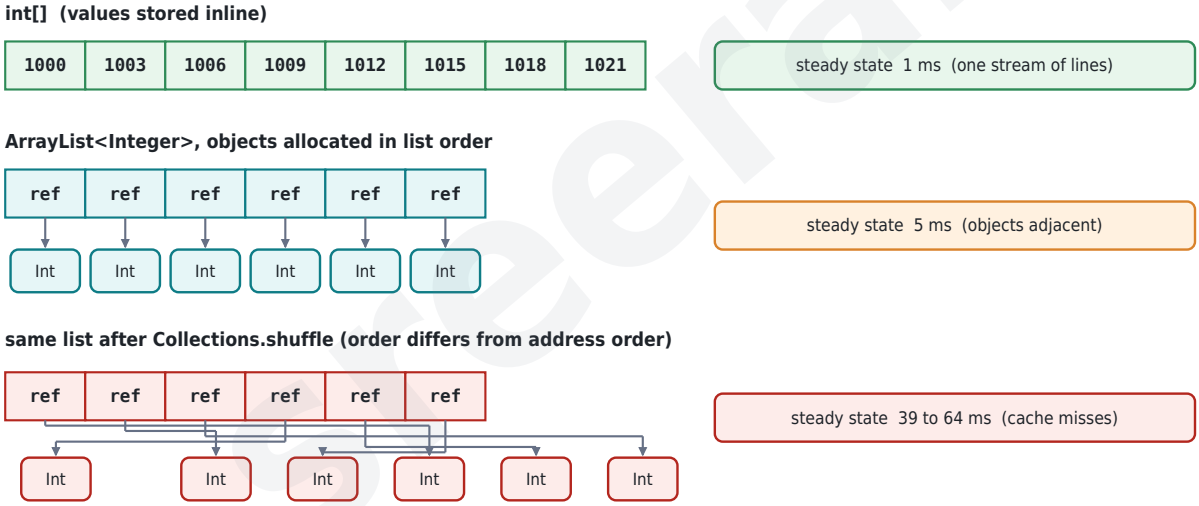


Figure 1.12. An `int[]` stores values side by side; an `ArrayList<Integer>` stores references to separate objects, and when list order differs from allocation order each step of the loop is a dependent load that may miss every cache level.

Root cause. Each element became a 4-byte reference (with compressed object pointers) to a separate 16-byte `Integer` object. Sequential allocation kept the objects near each other at first, so the in-order list stayed reasonably fast. After random insertion, consecutive list positions referred to objects scattered across the heap, so every element cost a dependent cache miss, the same pattern as the pointer-chasing experiment. The additional objects also increased allocation and young-collection work.

Fix. Use a primitive array or a primitive-specialised structure on hot paths, and keep boxed collections for the cold ones. Where the stream API is required, `IntStream` over an `int[]` retains the layout.

Prevention. Review the memory layout of data structures that are traversed in tight loops, and measure with representative sizes and order. The mean latency hides the problem; the percentile does not.

1.2.5 Questions to expect

Questions and model answers

1. **What is a context switch and why is it costly?** Saving the registers of one thread and loading those of another, usually with a change of address space. The direct cost is small, a few microseconds at most; the larger cost is the loss of warm caches and TLB entries for the thread that was switched out.
2. **What is the difference between an interrupt and an exception?** An interrupt is asynchronous and comes from a device or timer; an exception is synchronous and caused by the executing instruction, such as a page fault or a division by zero. Both enter the kernel through the interrupt descriptor table.
3. **Why does the timer interrupt matter to the operating system?** It guarantees that the kernel regains control at regular intervals, which makes preemptive scheduling and timeouts possible even when a program never calls the kernel.
4. **What does DMA do and why does it improve utilisation?** DMA lets a device transfer data to or from memory without processor involvement. The processor programs the transfer, runs other work and receives an interrupt when the transfer is complete.
5. **Explain spatial and temporal locality with an example.** Spatial: reading `a[i+1]` right after `a[i]` uses data in the same cache line. Temporal: a loop counter reused in each iteration stays in a register or in L1. Row-major traversal of a Java matrix has both; column-major traversal loses the spatial part.
6. **What are the approximate access times of L1, L2, L3 and main memory?** On the measured machine, 2 ns, 5 to 6 ns, about 40 ns and 80 to 176 ns, which means a factor of 50 to 100 between L1 and main memory.
7. **What is cache coherence?** The guarantee that all cores observe a consistent value for a memory location in spite of private caches, implemented by protocols such as MESI that invalidate or update other copies when a core writes.
8. **What is false sharing?** Two threads writing different variables that lie on the same cache line cause the line to bounce between cores, which reduces throughput without any logical sharing. Padding or separate allocation avoids it.
9. **Why does the same loop run at different speeds in different orders although it executes the same instructions?** The speed is determined by cache misses, not by instruction count. A stride-one access pattern uses whole cache lines and is prefetched; a large stride does not.
10. **Why can a virtual machine report a 260 MiB L3 cache?** The guest reports the cache of the host processor, which is shared by many guests. The share available to one guest is smaller and varies with the load of the others.

KEY POINTS

Memory is two orders of magnitude slower than the processor cache, devices are slower still, and the operating system is built around hiding both gaps.

- The cache hierarchy on the build machine: about 2 ns (L1), 5 ns (L2), 40 ns (L3) and 80 to 176 ns (DRAM); locality decides which of these a program pays.
- Interrupts, exceptions and the timer return control to the kernel; DMA moves data without the processor, so blocking I/O does not waste a core.
- A context switch saves and restores register state; its hidden cost is the loss of warm caches.
- Cache coherence keeps cores consistent; false sharing and contended writes show up as lost throughput, not as wrong results.
- Data layout (primitive arrays against boxed lists, row against column order) can change run time by an order of magnitude with identical instruction counts.

1.3 System Calls

A program cannot read a file, allocate memory from the kernel, create a thread or send a network packet by itself, because every one of those actions touches state that only ring 0 may modify. The **system call** is the interface through which user programs request such actions. It is the narrowest and most observable boundary in a running system: everything that a process does to the outside world, including everything that a JVM does, passes through it. A developer who can read a system call trace can answer questions that logs and profilers cannot, for example which file a service tried to open, which call it is blocked in, and how many times per second it enters the kernel.

1.3.1 Concept and mechanism

The x86-64 Linux calling convention

A system call is a function call with a different calling convention and a privilege transition. On x86-64 Linux the convention is fixed by the kernel's application binary interface.

- The system call number is placed in `rax`.
- Up to six arguments are placed in `rdi`, `rsi`, `rdx`, `r10`, `r8` and `r9`, in that order. (The ordinary C calling convention uses `rcx` instead of `r10`; the `SYSCALL` instruction overwrites `rcx` with the return address.)
- The `SYSCALL` instruction switches the processor to ring 0, saves the return address in `rcx` and the flags in `r11`, and jumps to the address stored in a model-specific register, `LSTAR`, that the kernel set at boot.
- The kernel stores the result in `rax`. A value between -4095 and -1 is the negated error number (`errno`); any other value is a successful result.
- The `SYSRET` instruction returns to ring 3.

The kernel keeps a **system call table**, an array of function pointers indexed by the call number. On this build machine the header `asm/unistd_64.h` defines 373 numbers.

Name	Number	Purpose
`read`	`0`	Read bytes from a file descriptor
`write`	`1`	Write bytes to a file descriptor
`mmap`	`9`	Map memory or a file into the address space
`getpid`	`39`	Return the process identifier
`clone`	`56`	Create a process or a thread
`execve`	`59`	Replace the program of the calling process
`futex`	`202`	Block or wake on a user-space lock word
`exit_group`	`231`	Terminate all threads of the process
`openat`	`257`	Open a file relative to a directory descriptor
`clone3`	`435`	Extended form of clone

A few x86-64 Linux system call numbers (from asm/unistd_64.h).

The numbers are specific to the processor architecture. Other architectures, such as 64-bit ARM, assign different numbers and use a different instruction (`SVC`), so a system call trace is portable in meaning but not in numbering.

sreram

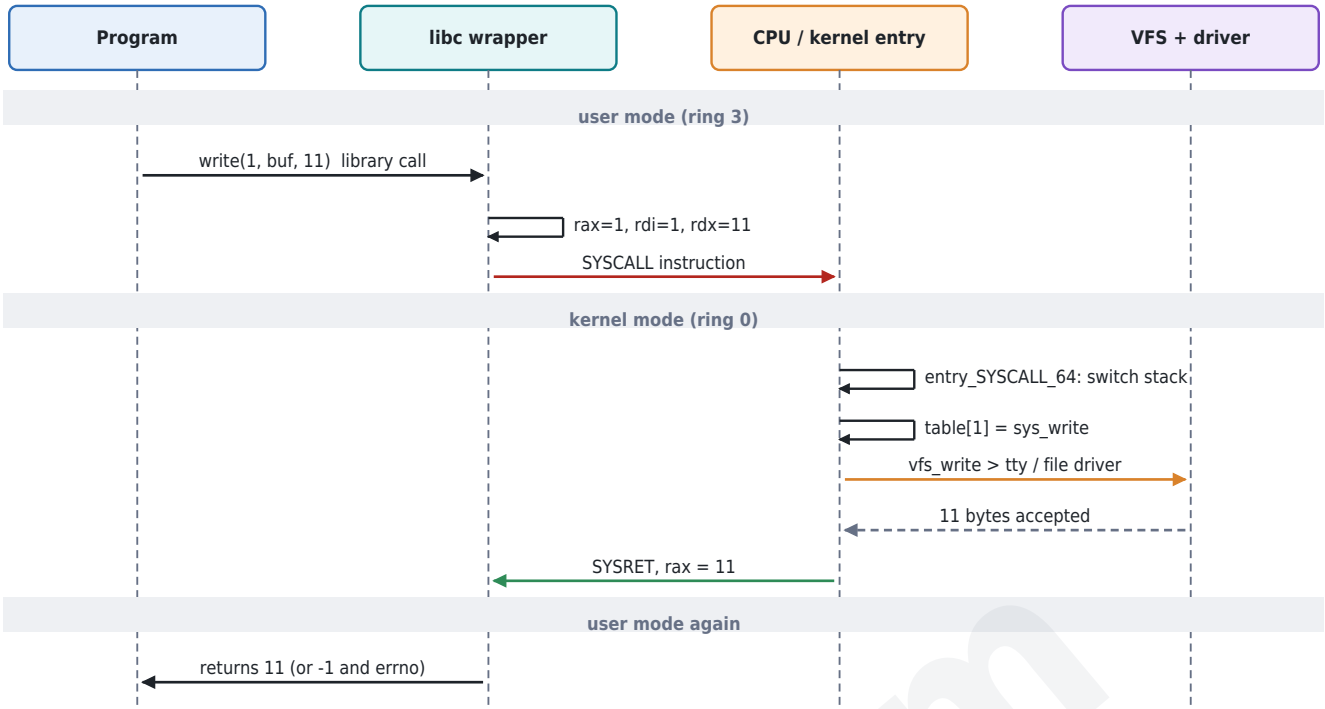


Figure 1.13. The transition into the kernel is one instruction in each direction; the work between them is the kernel's handler, selected by the number in `rax`.

Categories of system calls

Group	Purpose	Examples
Process control	Create, replace, wait for and end processes and threads	`clone`, `execve`, `wait4`, `exit_group`
File and device I/O	Open, read, write, position and close	`openat`, `read`, `write`, `lseek`, `close`
Memory	Change the address space	`brk`, `mmap`, `munmap`, `mprotect`
Information	Query the system and the process	`getpid`, `uname`, `clock_gettime`, `fstat`
Communication	Sockets, pipes, shared memory, signals	`socket`, `connect`, `epoll_wait`, `pipe2`, `kill`
Synchronisation	Block and wake threads	`futex`, `sched_yield`, `nanosleep`

Major system call groups and examples.

The C library wrapper and errno

Programs rarely execute `SYSCALL` themselves. The C library provides one wrapper per call that loads the registers, executes the instruction and converts a negative result into `errno` and a return value of -1. The disassembly of `write` in the C library of the build machine shows this directly.

The write wrapper in libc.so.6 (Ubuntu glibc, trimmed) TERMINAL

```
$ gdb -batch -ex 'disassemble write' /lib/x86_64-linux-gnu/libc.so.6 | head -12
Dump of assembler code for function __GI___libc_write:
   0x000000000011c830 <+0>:     endbr64
   0x000000000011c834 <+4>:     cmpb   $0x0,0xef805(%rip)        # 0x20c040 <__libc_single_threaded>
   0x000000000011c83b <+11>:    je     0x11c850 <__GI___libc_write+32>
   0x000000000011c83d <+13>:    mov    $0x1,%eax
   0x000000000011c842 <+18>:    syscall
   0x000000000011c844 <+20>:    cmp    $0xfffffffffffff000,%rax
   0x000000000011c84a <+26>:    ja     0x11c8a0 <__GI___libc_write+112>
   0x000000000011c84c <+28>:    ret
   0x000000000011c8a0 <+112>:   mov    0xe7551(%rip),%rdx        # 0x203df8
   0x000000000011c8a7 <+119>:   neg    %eax
   0x000000000011c8a9 <+121>:   mov    %eax,%fs:(%rdx)
```

The instruction at offset +13 loads 1 (the number of `write`) into `eax`, and the instruction at +18 is `SYSCALL`. The comparison with `0xfffffffffffff000` tests whether the result lies in the error range; the code at +112 negates it and stores it in `errno`, which lives in thread-local storage addressed through the `fs` register. The first branch handles the common single-threaded case without the cancellation bookkeeping that the multi-threaded path performs.

The same sequence can be written by hand with inline assembly. The program in the following listing performs `write` twice, once with a valid descriptor and once with an invalid one, without calling the C library wrapper.

rawsys.c C

```c
#include <stdio.h>
#include <unistd.h>

static long raw_write(int fd, const char *buf, unsigned long len) {
    long ret;
    __asm__ volatile("syscall"
                     : "=a"(ret)
                     : "a"(1L), "D"((long)fd), "S"(buf), "d"(len)
                     : "rcx", "r11", "memory");
    return ret;
}

int main(void) {
    const char msg[] = "written with a raw SYSCALL instruction\n";
    long n = raw_write(1, msg, sizeof msg - 1);
    printf("returned %ld\n", n);
    long bad = raw_write(99, msg, 3);
    printf("write to fd 99 returned %ld (negative errno, -EBADF = -9)\n", bad);
    return 0;
}
```

Raw system calls and the negative-errno convention TERMINAL

```
$ gcc -o rawsys rawsys.c && ./rawsys
written with a raw SYSCALL instruction
returned 39
write to fd 99 returned -9 (negative errno, -EBADF = -9)
$ strace -e trace=write ./rawsys 2>&1 | grep 'write(99'
write(99, "wri", 3)                     = -1 EBADF (Bad file descriptor)
```

The kernel returned -9 in `rax`. `strace` decodes that value as `-1 EBADF` because it knows the convention, while the raw wrapper exposes the unprocessed value.

The cost of the transition and the vDSO

Entering and leaving the kernel is more expensive than an ordinary call because the processor changes privilege, the kernel switches stacks, and on many processors additional mitigations for speculative-execution vulnerabilities add work. The following program measures the cost of four operations: a normal function call, the cheapest system call (`getppid`), the clock read through the C library, and the same clock read forced through the `SYSCALL` instruction.

sysc.c C

```c
#define _GNU_SOURCE
#include <stdio.h>
#include <time.h>
#include <unistd.h>
#include <sys/syscall.h>

static double now_ns(void) {
    struct timespec t;
    clock_gettime(CLOCK_MONOTONIC, &t);
    return t.tv_sec * 1e9 + t.tv_nsec;
}

static volatile long sink;

__attribute__((noinline)) long plain_call(long x) { return x + 1; }

int main(void) {
    const long N = 2000000;
    double t0, t1;
    struct timespec ts;

    t0 = now_ns();
    for (long i = 0; i < N; i++) sink = plain_call(i);
```

sysc.c (continued) C

```c
    t1 = now_ns();
    printf("function call        : %7.1f ns/call\n", (t1 - t0) / N);

    t0 = now_ns();
    for (long i = 0; i < N; i++) sink = syscall(SYS_getppid);
    t1 = now_ns();
    printf("getppid() syscall    : %7.1f ns/call\n", (t1 - t0) / N);

    t0 = now_ns();
    for (long i = 0; i < N; i++) clock_gettime(CLOCK_MONOTONIC, &ts);
    t1 = now_ns();
    printf("clock_gettime (vDSO) : %7.1f ns/call\n", (t1 - t0) / N);

    t0 = now_ns();
    for (long i = 0; i < N; i++)
        sink = syscall(SYS_clock_gettime, CLOCK_MONOTONIC, &ts);
    t1 = now_ns();
    printf("clock_gettime (trap) : %7.1f ns/call\n", (t1 - t0) / N);
    return 0;
}
```

Measured on the build machine (typical output of three runs, first run shown) TERMINAL

```
$ gcc -O1 -o sysc sysc.c && ./sysc
function call        :     1.0 ns/call
getppid() syscall    :   125.2 ns/call
clock_gettime (vDSO) :    26.3 ns/call
clock_gettime (trap) :   154.7 ns/call
$ strace -c -e trace=clock_gettime ./sysc 2>&1 | tail -6 | head -5
clock_gettime (trap) : 13214.6 ns/call
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
100.00    3.924013           1   2000000           clock_gettime
------ ----------- ----------- --------- --------- ----------------
```

A real transition costs roughly 100 to 150 ns, about a hundred times a function call. The line labelled `(vDSO)` is faster because the kernel maps a small code page, the **virtual dynamic shared object** (vDSO), into every process. The C library calls the clock function in that page, which reads a time value that the kernel keeps updated in a shared data page; no ring change occurs. The mapping is visible in `/proc/PID/maps` as `[vdso]` and `[vvar]`. Only a few calls are served this way: `clock_gettime`, `gettimeofday`, `time` and `getcpu`. The last block of the experiment shows the consequence for observation: under `strace`, only the forced-trap calls appear (2,000,000 of them), because the vDSO calls never enter the kernel and therefore cannot be traced. It also shows the cost of tracing, which is the subject of one of the incidents of this chapter.

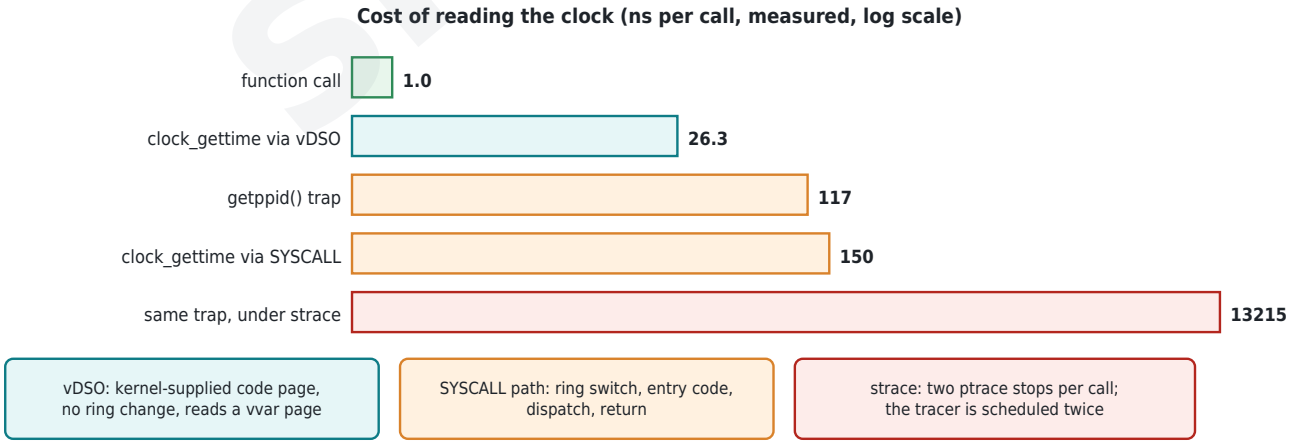


Figure 1.14. A time read through the vDSO costs about 26 ns, a real system call about 150 ns, and the same call under `strace` about 13,000 ns.

The clock source matters for this mechanism: on this machine `/sys/devices/system/clocksource/clocksource0/current_clocksource` reports `tsc`, which the vDSO can read without entering the kernel. Some virtualised environments use a clock source that the vDSO cannot read, in which case every call to `System.nanoTime` becomes a real system call.

How a JVM uses system calls

The JVM itself is a heavy user of three groups. Memory management uses `mmap`, `munmap` and `mprotect` to reserve the heap, to commit pages and to implement safepoint polling and code-cache protection. Thread synchronisation uses `futex` for monitors, `LockSupport.park`, `Thread.sleep` and condition waits. Input and output use `read`, `write`, `openat`, `epoll_wait` and the socket calls. `System.nanoTime` uses `clock_gettime` through the vDSO. The blocked state of a thread can be read without any tracer: the file `/proc/PID/task/TID/syscall` shows the call that a sleeping thread is in.

The system call in which two JVM threads are blocked (typical output) TERMINAL

```
$ cat /proc/9254/task/9254/syscall /proc/9254/task/9254/wchan; echo
202 0x7f4c9001b378 0x89 0x0 0x7f4c951fe740 0x0 0xffffffff 0x7f4c951fe610 0x7f4c96898e51
futex_do_wait
```

The first number, 202, is `futex`; the second column is the first argument (the address of the lock word) and the third is the operation (`0x89` is `FUTEX_WAIT_BITSET` with the private flag). The thread is the Java `main` thread of `Sleeper`, which is inside `Thread.sleep`, implemented in the JVM as a timed park on a futex.

1.3.2 Worked trace

A C program from `execve` to `exit_group`

The following trace is the output of `strace` for the program `hello` (a single `printf`), reduced to the lines that matter and grouped by phase. The complete trace has 34 calls.

hello.c C

```c
#include <stdio.h>

int main(void) {
    printf("Hello, kernel\n");
    return 0;
}
```

strace of the dynamically linked hello (typical addresses; trimmed) TERMINAL

```
$ gcc -O0 -o hello hello.c && strace -o hello.trace ./hello && cat hello.trace
execve("./hello", ["./hello"], 0x7ffe1a132b20 /* 212 vars */) = 0
brk(NULL)                               = 0x5570aacc7000
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f8673358000
access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
mmap(NULL, 54375, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7f867334a000
close(3)                                = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0\0"..., 832) = 832
mmap(0x7f8673028000, 1609728, PROT_READ|PROT_EXEC, ...) = 0x7f8673028000
close(3)                                = 0
arch_prctl(ARCH_SET_FS, 0x7f8673347740) = 0
set_tid_address(0x7f8673347a10)         = 640
mprotect(0x7f8673200000, 16384, PROT_READ) = 0
munmap(0x7f867334a000, 54375)           = 0
fstat(1, {st_mode=S_IFREG|0644, st_size=19, ...}) = 0
getrandom("\xdc\xd8\x7b\x9a\x4b\x2a\x58\x75", 8, GRND_NONBLOCK) = 8
brk(NULL)                               = 0x5570aacc7000
brk(0x5570aace8000)                     = 0x5570aace8000
write(1, "Hello, kernel\n", 14)         = 14
exit_group(0)                           = ?
+++ exited with 0 +++
```

1. `execve` replaces the shell's child with the new program. Everything before the first line of `main` is the work of the dynamic loader and the C library.
2. `brk` **and** `mmap` set up the loader's own memory. The `access("/etc/ld.so.preload")` call fails with `ENOENT`, which is normal: it checks for an optional preload list.
3. `openat("/etc/ld.so.cache")`, `mmap`, `close` read the cache of library locations, then `openat("libc.so.6")`, `read` of the ELF header and several `mmap` calls map the C library into the address space with the permissions of each segment (read-execute for code, read-only for constants, read-write for data).
4. `arch_prctl(ARCH_SET_FS)` points the `fs` register at the thread control block, which is where `errno` lives. `set_tid_address` registers the address that the kernel clears when the thread exits; the returned value, 640, is the thread identifier.

5. `mprotect` makes the relocated data read-only. `munmap` discards the library cache mapping. `getrandom` seeds the allocator.
6. `fstat(1)` examines standard output. In this run the output was captured to a file (`S_IFREG`), so the C library selects full buffering; on a terminal it selects line buffering and the write would follow the newline immediately. The two `brk` calls grow the heap for the buffer.
7. `write(1, "Hello, kernel\n", 14)` is the single call that the program asked for. It happens at exit, when the buffer is flushed.
8. `exit_group(0)` terminates every thread of the process and returns the status to the parent.

Thirty-three of the 34 calls are overhead of the runtime and one is the work. Linking statically removes the loader's part: the static build of the same program issues 16 calls. The JVM, with a far larger runtime, issues 1,457 for the same single line of output.

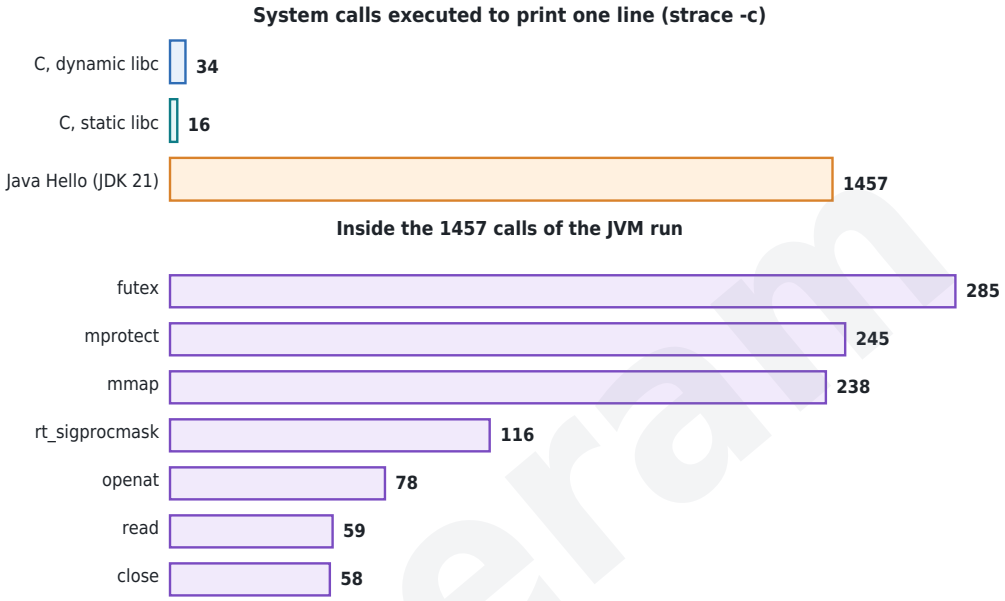


Figure 1.15. The fixed cost of starting a runtime is measured in system calls; the JVM's total is dominated by `futex` (thread coordination) and by memory-mapping calls, not by file I/O.

1.3.3 Hands-on

The `strace` toolbox

`strace` runs a program under the kernel's `ptrace` facility and prints every system call with its arguments and its result. The options below cover almost all diagnostic use.

Option	Effect	Typical use
`-f`	Follow child processes and threads	Always for a JVM, which is multi-threaded
`-o FILE`	Write the trace to a file	Keep the trace apart from the program's output
`-p PID`	Attach to a running process (add `-f` for all threads)	Diagnose a stuck service without restarting it
`-c`	Print a table of counts and times instead of the trace	Find which calls dominate
`-S calls`	Sort the `-c` table by call count	Rank by frequency
`-e trace=SET`	Trace only the named calls or a group such as `%file`, `%network`, `%memory`	Reduce volume
`-T`	Show the time spent inside each call	Find slow calls
`-tt`	Print a wall-clock time stamp with microseconds	Correlate with logs
`-y`	Print the file path next to a file descriptor	Understand which descriptor is which
`-s N`	Print up to N bytes of string arguments	See more of a read or write
`--seccomp-bpf`	Stop only on the traced calls	Lower overhead, used with `-e trace=`

The strace options used in this book.

Follow errors and timing: the open-and-read program

The next program tries to open a file that does not exist, then reads a file that does.

open_fail.c — C

```c
#include <fcntl.h>
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <unistd.h>

int main(void) {
    int fd = open("/etc/does-not-exist.conf", O_RDONLY);
    if (fd < 0) printf("open failed: errno=%d (%s)\n", errno, strerror(errno));
    fd = open("/etc/hostname", O_RDONLY);
    char buf[64];
    ssize_t n = read(fd, buf, sizeof buf);
    printf("read %zd bytes\n", n);
    close(fd);
    return 0;
}
```

Timing (-T), time stamps (-tt) and the file group (typical output) — TERMINAL

```
$ strace -tt -T -e trace=openat,read,close ./open_fail 2>&1 | grep -v -e ld.so -e libc | tail -7
02:17:05.178913 openat(AT_FDCWD, "/etc/does-not-exist.conf", O_RDONLY) = -1 ENOENT (...) <0.000011>
02:17:05.180434 openat(AT_FDCWD, "/etc/hostname", O_RDONLY) = 3 <0.000011>
02:17:05.180518 read(3, "vm\n", 64)     = 3 <0.000012>
02:17:05.180551 close(3)                = 0 <0.000004>
open failed: errno=2 (No such file or directory)
read 3 bytes
02:17:05.180657 +++ exited with 0 +++
$ strace -e trace=%file ./open_fail 2>&1 | tail -5
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
openat(AT_FDCWD, "/etc/does-not-exist.conf", O_RDONLY) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/hostname", O_RDONLY) = 3
open failed: errno=2 (No such file or directory)
read 3 bytes
```

The pattern that this shows is the most common use of `strace` in operations: the failing call (`ENOENT`) names the path that the program expected, which turns "the application cannot find its configuration" into a precise file name. The `-T` value is the duration of the call; a `read` or `connect` with a large value identifies the slow dependency.

Trace and count a JVM

Counting the system calls of a Java Hello World (typical output; counts vary by about 5 percent) — TERMINAL

```
$ strace -f -c -o java.sc java Hello > /dev/null
$ head -10 java.sc
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ------------------
 94.71    0.120780         423       285        43 futex
  0.73    0.000934           3       245           mprotect
  0.94    0.001193           5       238           mmap
  0.35    0.000440           3       116           rt_sigprocmask
  0.42    0.000538           6        78        21 openat
  0.24    0.000308           5        59           read
  0.31    0.000397           6        58           close
  0.19    0.000239           4        58           fstat
$ tail -1 java.sc
100.00    0.127520          87      1457        96 total
$ strace -f -o java.full java Hello > /dev/null
$ grep -c clone3 java.full; awk '{print $1}' java.full | sort -u | wc -l
29
18
```

The `% time` column is misleading here: 95 percent of the time attributed to `futex` is threads waiting for each other, which is idle time, not work. The reliable columns are `calls` and `errors`. Of the 96 failed calls, most are probes for optional files (`ENOENT`), for example the JVM searching for architecture-specific library variants under `glibc-hwcaps` directories, which is harmless. The trace of 1,821 lines contains 18 distinct thread identifiers and 17 `clone3` calls (the count of 29 above includes lines that `strace` split with `<unfinished ...>` and `resumed`).

The files that the JVM opens are informative too. It reads the module image `lib/modules` (140 MB) and the class data sharing archive `lib/server/classes.jsa`, probes `/proc/self/coredump_filter` and `/etc/passwd`, and reads the

control-group files `/sys/fs/cgroup/cpu/cpu.cfs_quota_us` and `/sys/fs/cgroup/memory/.../memory.limit_in_bytes`. The last group is how the JVM discovers container limits; the chapter on the Linux file-based view explains the consequences.

1.3.4 Real-world incidents

Incident: a Java service burns a core on system time because it writes byte by byte

Scenario. An export service writes a CSV file to local disk through a `FileOutputStream` obtained from a legacy helper. The helper writes each character with `write(int)`. The service handles small exports in production without issue, and a new customer requests exports of several million rows.

Symptoms. Export requests take minutes. `top` shows the Java process at 100 percent of one core, and the split of CPU time shows a large share of system time (`sy`) alongside user time. Garbage collection logs are quiet and the disk shows little throughput.

Diagnosis. The combination of high system time and low disk throughput suggests many small calls into the kernel. `strace -c` counts them.

ByteWriter.java — JAVA

```java
import java.io.BufferedOutputStream;
import java.io.FileOutputStream;
import java.io.OutputStream;

public class ByteWriter {
    public static void main(String[] args) throws Exception {
        boolean buffered = args[0].equals("buffered");
        int n = 200_000;
        long t0 = System.nanoTime();
        try (OutputStream raw = new FileOutputStream("/tmp/osw_o1/bytes.dat");
             OutputStream out = buffered ? new BufferedOutputStream(raw, 8192) : raw) {
            for (int i = 0; i < n; i++) {
                out.write('x');
            }
        }
        System.out.printf("%s: %d bytes in %d ms%n", args[0], n,
                (System.nanoTime() - t0) / 1_000_000);
    }
}
```

Reproduction with 200,000 single-byte writes (typical output) — TERMINAL

```
$ java ByteWriter unbuffered
unbuffered: 200000 bytes in 165 ms
$ java ByteWriter buffered
buffered: 200000 bytes in 19 ms
$ strace -f -c -o sc_unbuffered.txt java ByteWriter unbuffered > /dev/null
$ head -6 sc_unbuffered.txt | cut -c1-70
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ------------------
 95.95    4.493343        5673       792        87 futex
  3.92    0.183625           0    200017           write
  0.06    0.002591        1295         2           sched_yield
  0.01    0.000701           7        89        24 openat
$ strace -f -c -o sc_buffered.txt java ByteWriter buffered > /dev/null
$ grep -w write sc_buffered.txt
  0.13    0.000239           5        42           write
```

The unbuffered variant issued 200,017 `write` calls, one per byte, while the buffered variant issued 42 in total. The 200,017 count equals the number of bytes plus 17 calls from the runtime itself, which is a precise signature of a one-call-per-byte loop. The run times (165 ms against 19 ms here) grow linearly with the data, and on a loaded server, where each call also competes with other threads, the system time dominates.

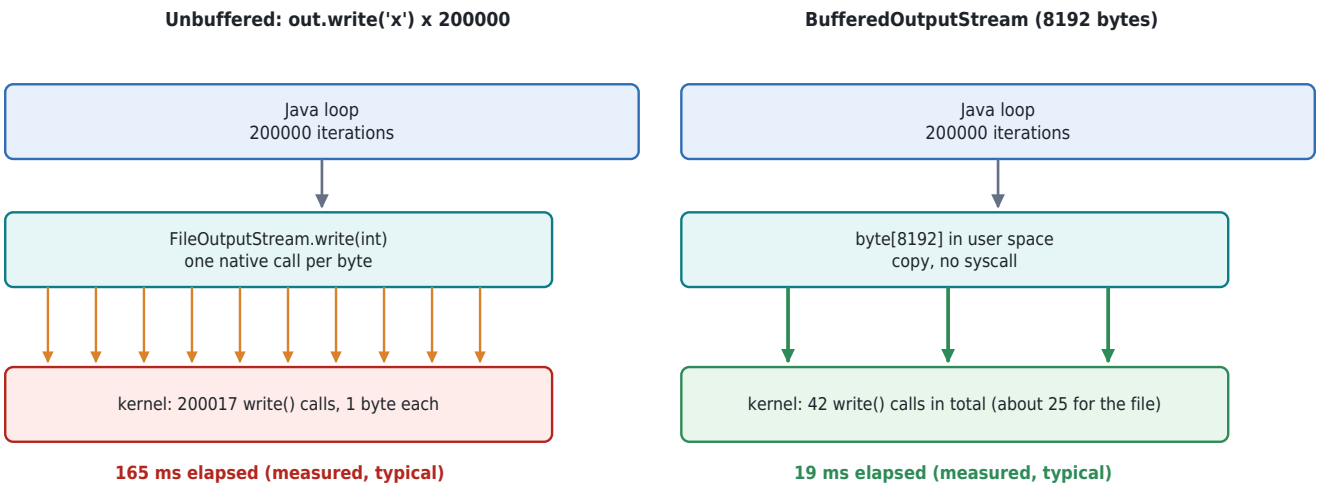


Figure 1.16. Buffering turns 200,000 kernel entries into a few dozen by collecting bytes in user space; the kernel work per byte falls to nearly zero.

Root cause. The helper used an unbuffered stream. Each `write(int)` is a native method that performs one `write` system call, so the cost of the ring transition (about 100 ns plus the file-system work of every call) was paid per byte.

Fix. Wrap the stream in `BufferedOutputStream` (8 KiB is the default) or use `Files.newBufferedWriter`; write arrays or strings, not single bytes. For large sequential output, a buffer of 64 KiB or more reduces calls further. The immediate mitigation, before a code change, is to limit export size or to move exports to a queue.

Prevention. Search for raw `FileOutputStream` and `FileWriter` use in loops during review. In load tests, record the system call rate: `strace -c -f -p PID` for ten seconds on a staging copy, or `vmstat 1` (column `sy` and `cs`) in production. A rate in the hundreds of thousands per second for one process is a defect, whatever the throughput.

Incident: attaching strace to a production JVM makes the outage worse

Scenario. A payment service shows intermittent timeouts. An engineer attaches `strace -f -p PID` to the production JVM to find the slow call, with no filter, and sends the output to the terminal.

Symptoms. Within seconds the service throughput collapses to a fraction of normal, health checks fail and the orchestrator restarts the instance, which removes the evidence.

Diagnosis. The slowdown is caused by the tool. `strace` uses `ptrace` to stop the traced process at the entry and the exit of every system call, wakes the tracer, which decodes and prints the call, and then resumes the process. Every call therefore costs two extra context switches and the work of the tracer. The overhead was measured on the build machine using the byte-writing program above, which performs 200,000 calls.

Overhead of different strace modes on the same program (typical output) TERMINAL

```
$ java ByteWriter unbuffered
unbuffered: 200000 bytes in 73 ms
$ strace -f -e trace=none -o /dev/null java ByteWriter unbuffered
unbuffered: 200000 bytes in 6553 ms
$ strace -f -c -o /dev/null java ByteWriter unbuffered
unbuffered: 200000 bytes in 4770 ms
$ strace -f --seccomp-bpf -e trace=openat -o /dev/null java ByteWriter unbuffered
unbuffered: 200000 bytes in 80 ms
```

Tracing nothing at all (`-e trace=none`) still slowed the program by a factor of about 90, because the ptrace stops occur for every call regardless of the filter. Counting (`-c`) is cheaper but remains in the same order. The `--seccomp-bpf` option installs a filter in the kernel so that only the selected calls stop the process; with a filter on a rarely used call, the run time was almost unchanged (80 ms against 73 ms).

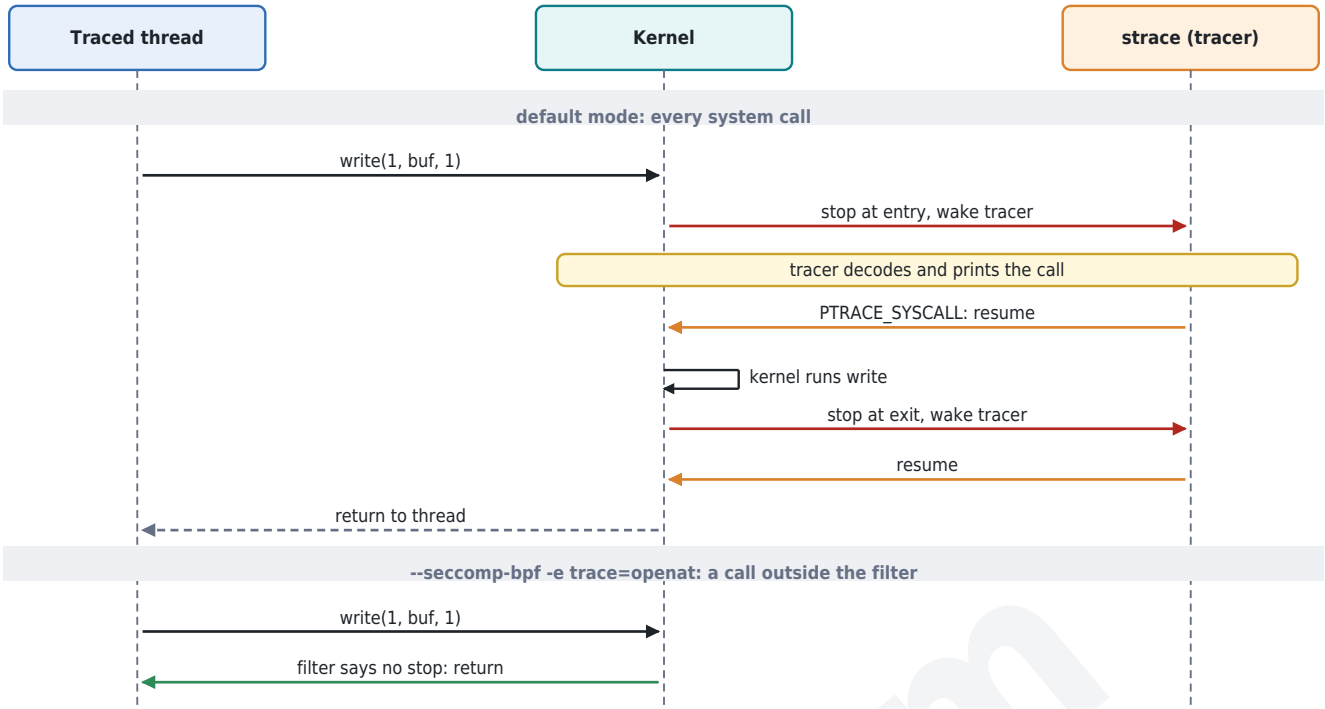


Figure 1.17. Without the filter option, `strace` stops the traced thread twice per system call and the tracer is scheduled twice; with `--seccomp-bpf` the kernel stops the thread only for the selected calls.

Root cause. An unfiltered ptrace attach on a process that executes tens of thousands of system calls per second. The JVM of a busy service performs most of them in `futex`, `read`, `write` and `epoll_wait`.

Fix. Mitigation: detach (`Ctrl-C` ends `strace` and the process continues), and re-run on one instance removed from the load balancer. Correct procedure: use `strace -c -p PID` for a few seconds to see which calls dominate; then use a filter with `--seccomp-bpf`, for example `-f --seccomp-bpf -e trace=connect,sendto,recvfrom -T -tt -o /tmp/t.txt`; or use the JVM's own tools (`jcmd PID Thread.print`, Java Flight Recorder), which do not stop every call.

Prevention. Document the safe procedure in the runbook. Prefer sampling tools in production. Perf-based tracing (`perf trace`) and eBPF tools have lower overhead; they are not available on the build machine of this book and were not run. Never write a trace to the terminal of a busy process, and write the file to a local disk with sufficient free space.

1.3.5 Questions to expect

Questions and model answers

1. **What is a system call and why is it needed?** The interface by which a user program requests a privileged operation from the kernel. It is needed because user code runs in ring 3 and cannot touch devices, page tables or other processes' memory.
2. **Describe what happens between `write()` in C and the data reaching the device.** The wrapper loads `rax` with the call number and the argument registers, executes `SYSCALL`; the processor enters ring 0 at the kernel entry point; the kernel dispatches through the table to `write`, validates the descriptor, copies data to the file or device layer; the result goes into `rax` and `SYSRET` returns.
3. **How is an error reported by a system call?** The kernel returns a negative error number in `rax`; the C wrapper stores the positive value in `errno` and returns -1.
4. **Why is a system call more expensive than a function call?** It changes privilege level, switches stacks, runs the entry and exit code, and on some processors includes mitigations for speculative-execution attacks. The measured cost here is 100 to 150 ns against 1 ns.
5. **What is the vDSO and which calls use it?** A kernel-provided page mapped into every process that implements some calls, such as `clock_gettime`, entirely in user space by reading kernel-updated data. It avoids the ring transition.
6. **What does `strace -c` show and what is the caveat in the `% time` column?** The count, error count and time of each call. The time includes waiting, so blocking calls such as `futex` and `epoll_wait` dominate without indicating work.

7. **Why does `strace` slow the traced process so much?** It stops the process at each call's entry and exit through `ptrace`, and the tracer must run, decode and resume it. `--seccomp-bpf` limits the stops to selected calls.
8. **How would you find which file an application fails to open?** `strace -f -e trace=%file -o out.txt command`, then search for `ENOENT` or `EACCES` in the trace; the argument of the failing `openat` is the path.
9. **A Java program calls `System.out.print` for each of a million characters and is slow. Why?** `System.out` is a `PrintStream`, which flushes to a `FileOutputStream`; with autoflush or without buffering, each call can become a system call. Building one string or using a buffered writer reduces the number of calls.
10. **How can a blocked thread's system call be found without tracing?** `cat /proc/PID/task/TID/syscall` shows the call number and arguments; `/proc/PID/task/TID/wchan` shows the kernel function where the thread sleeps.

KEY POINTS

The system call is the single, narrow gate between a process and the kernel, and observing it explains most process behaviour.

- Convention: number in `rax`, arguments in `rdi, rsi, rdx, r10, r8, r9`, `SYSCALL` to enter, result in `rax` with negative values meaning `-errno`.
- A transition costs roughly 100 to 150 ns on the build machine; the vDSO serves clock reads without a transition (about 26 ns).
- A Java Hello World performs about 1,450 system calls (futex, mprotect, mmap dominate) against 34 for the equivalent C program.
- `strace -f -c` counts calls, `-e trace=%file` finds missing files, `-T` finds slow calls, `/proc/PID/task/TID/syscall` shows a blocked call without tracing.
- `strace` slows a process by about two orders of magnitude; use `--seccomp-bpf` with a filter, or sampling tools, on production systems.

1.4 Boot and Init

When a machine is powered on, nothing is running: memory is uninitialised, the kernel is a file on a disk, and no process exists. The boot sequence is the chain of small programs that each prepare the machine for the next one, ending in the first user-space process, which starts everything else. Understanding the chain turns vague symptoms (the server does not come back after a reboot, the service starts by hand but not at boot, the container stops after one signal) into questions with specific places to look. This chapter follows the sequence from firmware to the service manager and then concentrates on `systemd`, which starts almost every service on a modern Linux server.

1.4.1 Concept and mechanism

The boot sequence

1. **Firmware.** The processor starts at a fixed address in firmware, either the legacy BIOS or UEFI. The firmware tests and initialises the hardware, then looks for a bootable device. UEFI reads files from a FAT-formatted EFI System Partition and can verify signatures (Secure Boot); legacy BIOS reads the first 512-byte sector of the disk.
2. **Bootloader.** A program such as GRUB loads the kernel image and an initial RAM file system (initramfs) into memory, passes the **kernel command line**, and jumps to the kernel. On virtual machines that use a direct-boot mechanism, the hypervisor performs this step and no bootloader exists.
3. **Kernel start-up.** The kernel decompresses itself, sets up memory management and the interrupt tables, starts the scheduler and initialises the built-in drivers. It then mounts the initial RAM file system as the temporary root.
4. **Initramfs.** A small archive that contains the drivers and tools needed to reach the real root file system: disk controller and file-system modules, RAID or LVM assembly, decryption of an encrypted volume. It finds and mounts the real root and switches to it.
5. **PID 1.** The kernel executes the first user-space program, `/sbin/init`, which is `systemd` on current mainstream distributions. It receives process identifier 1 and has no parent to report to; it is the ancestor of every other process.
6. **User space start-up.** The init system mounts the remaining file systems, starts services in dependency order, and reaches the default target (for a server, `multi-user.target`).

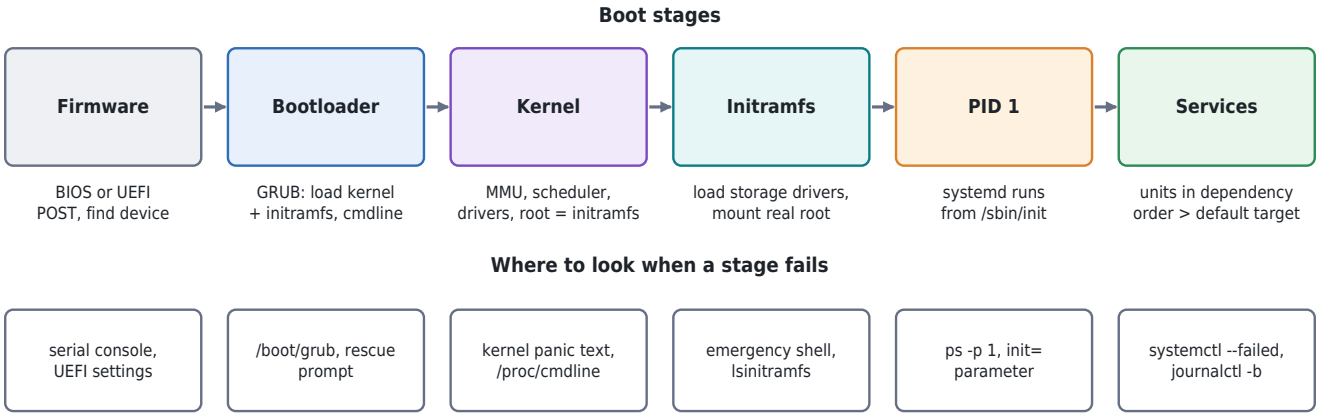


Figure 1.18. Each boot stage loads the next and hands over a smaller set of unknowns; the lower row names the place to look when a stage fails.

The kernel command line

The bootloader passes text parameters that configure the kernel and the init process. They are visible in `/proc/cmdline`. The build machine of this book is a micro virtual machine that starts the kernel directly, and its command line shows the typical elements:

The kernel command line of the build machine TERMINAL

```
$ cat /proc/cmdline | fold -w 90
mitigations=auto,no_guest_host,no_guest_guest deferred_init=lazy dhash_entries=131072 ihash_entr
ies=65536 console=ttyS0 reboot=k panic=1 nomodule random.trust_cpu=1 ipv6.disable=1 swiotlb=nof
orce psi=1 rdinit=/process_api -- --firecracker-init --addr 0.0.0.0:2024 --max-ws-buffer-size 32
768 --block-local-connections --listen-vsock-port 2024 --log-vsock-port 5002
$ ls /sys/firmware; ls /sys/firmware/efi
acpi  memmap
ls: cannot access '/sys/firmware/efi': No such file or directory
```

The parameters that matter to an engineer:

- `console=ttyS0` sends kernel messages to a serial port, the usual console of a virtual machine.
- `panic=1` reboots one second after a kernel panic; `reboot=k` selects the reboot method.
- `nomodule` forbids loading kernel modules; that is why `/proc/modules` does not exist on this machine.
- `ipv6.disable=1` and `psi=1` toggle the IPv6 stack and enable pressure stall information, which provides `/proc/pressure/*` used in the diagnostic chapters.
- `mitigations=auto` selects the default speculative-execution mitigations, which affect the cost of system calls.
- `rdinit=/process_api` names the program to run as PID 1 from the initial RAM file system, instead of `/sbin/init`; everything after the `--` is passed to that program as arguments. On the build machine the first process is therefore the control agent of the sandbox, which agrees with the `ps -p 1` result of the first chapter. The absence of `/sys/firmware/efi` confirms that no UEFI firmware was involved.

On a server the same file contains `root=UUID=...`, `ro` and often `quiet` and cgroup options such as `systemd.unified_cgroup_hierarchy=1`. A parameter that is wrong after a change to the bootloader configuration is a common cause of a machine that does not return after a reboot.

PID 1 and the init system

PID 1 has two roles that no other process has. It **starts** the system, and it **adopts orphans**: when a process exits while its children are alive, the kernel re-parents the children to PID 1, and PID 1 must call `wait` to collect their exit status or they become zombies. In addition, the kernel treats PID 1 specially for signals: a signal for which PID 1 has installed no handler is ignored, not applied with the default action, so that a mistaken `kill` cannot take the system down. The same rules apply to the first process of a PID namespace, which is what the entry point of a container is. The experiment below creates such a namespace with `unshare` and demonstrates the signal rule.

The first process of a new PID namespace ignores an unhandled SIGTERM TERMINAL

```
$ unshare --pid --fork --mount-proc --kill-child bash -c \
    'echo "inside: pid=$$"; ps -o pid,ppid,comm; kill -TERM $$; echo "survived SIGTERM"'
inside: pid=1
    PID    PPID COMMAND
      1       0 bash
      2       1 ps
survived SIGTERM
```

Inside the namespace the shell has PID 1 and its parent is 0 (outside the namespace). The `kill -TERM $$` is sent to PID 1 itself and has no effect because bash has no handler for it in this context. The consequence for containers is direct: a Java application started as `CMD java -jar app.jar` in a container becomes PID 1; the JVM installs a handler for `SIGTERM`, so it does shut down, but a shell script wrapper that starts the JVM without `exec` does not forward the signal, and an orphaned grandchild is never reaped. The container chapter returns to this.

systemd units, targets and dependencies

`systemd` models the system as a graph of **units**. A unit is described by a text file in `/lib/systemd/system` (packaged units), `/etc/systemd/system` (administrator units and overrides) or `/run/systemd/system`. The common unit types are in the following table.

Type	Meaning	Example
`.service`	A process or group of processes that systemd starts and supervises	`ssh.service`, `orders.service`
`.target`	A named synchronisation point that groups units	`multi-user.target`
`.socket`	A listening socket that starts a service on the first connection	`docker.socket`
`.timer`	A schedule that activates another unit	`logrotate.timer`
`.mount`	A mount point, usually generated from `/etc/fstab`	`data.mount`
`.path`	Activates a unit when a path changes	`spool.path`

Common systemd unit types.

Ordering and requirement are separate concepts, and confusing them is the most frequent cause of start-up problems.

- **Requirement**: `Wants=` (soft: start the other unit, tolerate its failure) and `Requires=` (hard: if the other unit fails to start or stops, this one stops).
- **Ordering**: `After=` and `Before=` say in which order two units start. They say nothing about whether the other unit is started at all.

A unit that declares `After=postgresql.service` without `Wants=` or `Requires=` is started after PostgreSQL only if something else started PostgreSQL. Conversely, `Requires=postgresql.service` without `After=` starts both at the same time.

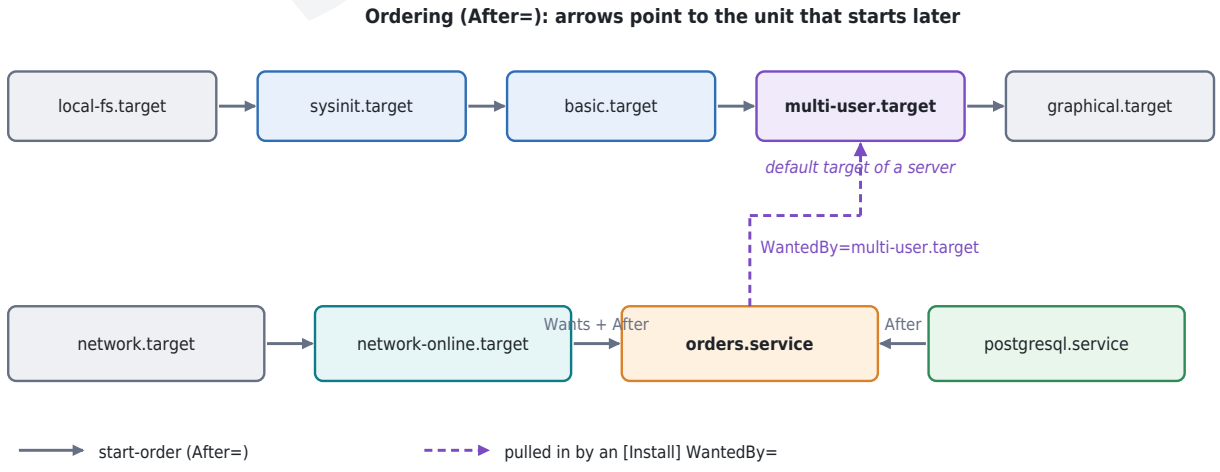


Figure 1.19. Targets form the skeleton of the boot: a service attaches itself to a target through `WantedBy=` in its `[Install]` section, and the default target pulls in everything below it.

The **start of a service** proceeds as follows. The manager resolves the dependencies of the unit and queues start jobs for them. For a unit of `Type=simple`, which is the default and the usual choice for a Spring Boot executable jar, `systemd` creates a control group for the service, applies the settings of the unit (user, working directory, resource limits such as `LimitNOFILE`), forks, executes the command, and considers the unit started as soon as the process has been forked. For `Type=notify` the unit is started only when the program reports readiness through the notification socket; `Type=forking` is for traditional daemons that fork into the background and exit. Standard output and error of the service are connected to the journal.

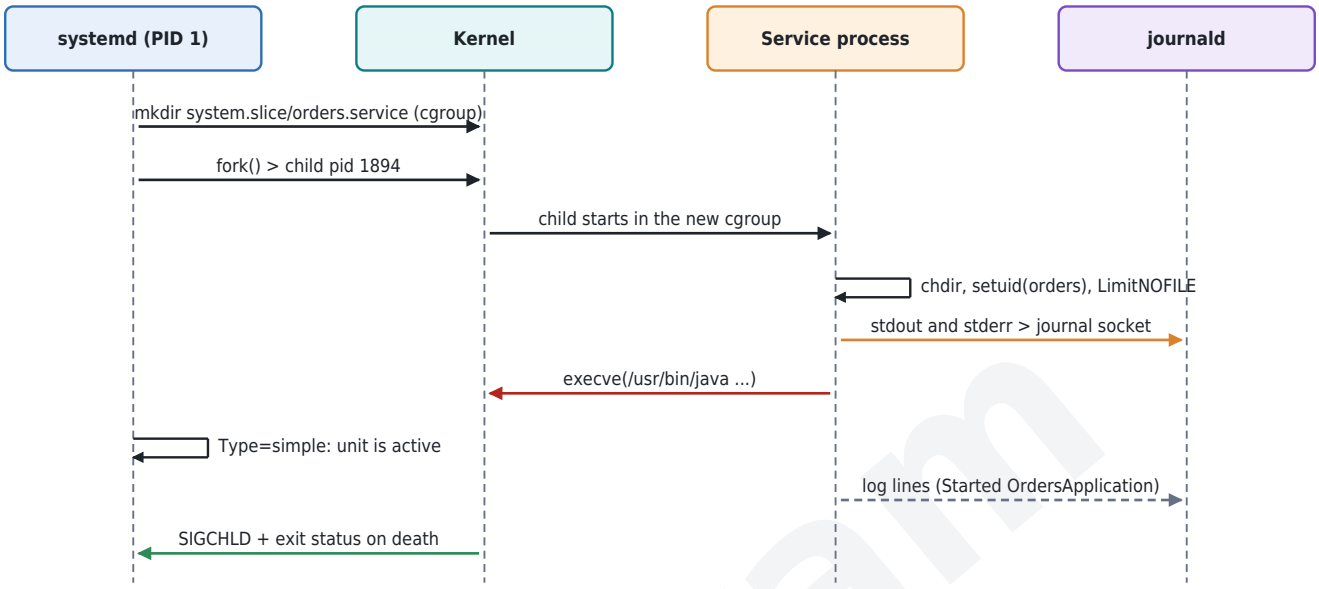
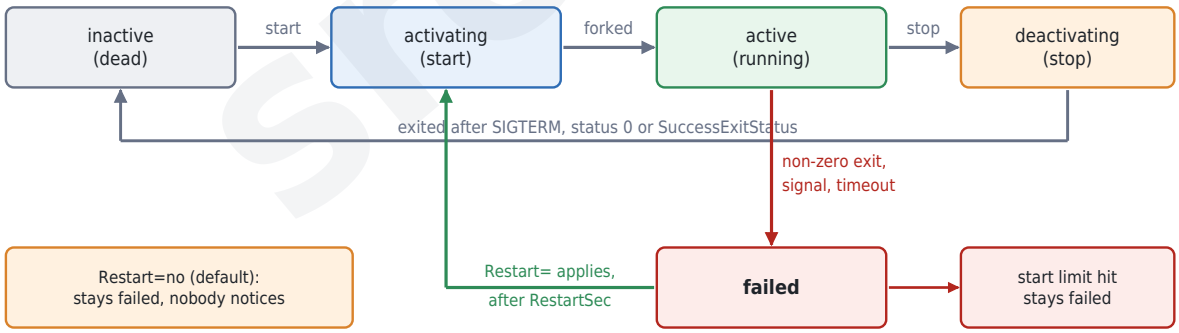


Figure 1.20. A service start is a sequence of kernel operations performed by PID 1: create a cgroup, fork, drop privileges and limits, `execve`, and log the output; the journal receives everything the process writes to its standard streams.

Three commands give the state of the system: `systemctl status UNIT` shows the state, the main process and the most recent log lines; `journalctl -u UNIT -b` shows the logs of one unit since boot; `systemctl list-dependencies UNIT` shows the dependency tree. The state of a service follows the finite state machine shown below.



Restart policy decides between the green path and a permanent failed state

Figure 1.21. A service that exits with a failure moves to `failed`, and the `Restart=` setting determines whether it is started again; after too many restarts within `StartLimitIntervalSec`, systemd gives up and stays in `failed`.

1.4.2 Worked trace

Typical timeline of a server boot

The numbers below are the typical output of `systemd-analyze` on a cloud virtual machine of the kind that runs Java services; they illustrate the proportions and were not measured on the build machine, which does not run systemd. The same command on a real server prints the stage times in its first line.

Typical systemd-analyze output on a cloud server (illustrative; not run on the build machine) TERMINAL

```
$ systemd-analyze
Startup finished in 3.201s (firmware) + 2.105s (loader) + 2.874s (kernel) + 6.412s (initrd)
 + 19.337s (userspace) = 33.929s
multi-user.target reached after 19.231s in userspace.
$ systemd-analyze blame | head -5
 9.804s cloud-init-local.service
 7.332s apt-daily.service
 5.118s orders.service
 3.907s postgresql@16-main.service
 2.443s snapd.service
$ systemd-analyze critical-chain orders.service
orders.service @14.201s +5.118s
└─network-online.target @14.198s
  └─systemd-networkd-wait-online.service @9.402s +4.793s
    └─systemd-networkd.service @9.111s +278ms
      └─systemd-udevd.service @8.201s +901ms
```

The critical chain is the sequence of units that determined when `orders.service` could start. Here the service waited for `network-online.target`, which waited 4.8 seconds for a network address. The time of `orders.service` itself (`+5.118s`) is the time until systemd regarded it as started.

Start of a Spring Boot service, step by step

The unit file that follows is a realistic definition of a Spring Boot service. It was checked with `systemd-analyze verify`, which parses the file and validates it without starting anything.

orders.service INI

```
[Unit]
Description=Orders API (Spring Boot)
After=network-online.target postgresql.service
Wants=network-online.target

[Service]
Type=simple
User=orders
WorkingDirectory=/opt/orders
ExecStart=/usr/bin/java -Xmx512m -jar /opt/orders/orders.jar
Restart=on-failure
RestartSec=5
LimitNOFILE=65536
SuccessExitStatus=143
TimeoutStopSec=30

[Install]
WantedBy=multi-user.target
```

On `systemctl start orders.service` systemd performs these steps, which can be matched to the lines above:

1. It computes the start order from `After=` and `Wants=`: `network-online.target` is pulled in by `Wants=`, and PostgreSQL is ordered before but not requested.
2. When the ordering allows, it creates the cgroup `system.slice/orders.service` and sets `LimitNOFILE=65536` for the process through `setrlimit`.
3. It forks. The child changes to `/opt/orders`, switches to the user `orders`, connects standard output and error to the journal, and calls `execve("/usr/bin/java", ...)`.
4. For `Type=simple` the unit is now `active (running)`. The JVM may still be starting; readiness of the application is not known to systemd.
5. On a stop request, systemd sends `SIGTERM` to the main process, waits up to `TimeoutStopSec=30` seconds, then sends `SIGKILL` to every process of the cgroup. The exit status 143 is accepted as success by `SuccessExitStatus=143`.
6. If the process exits with a failure, `Restart=on-failure` starts it again after `RestartSec=5` seconds.

sreeram

1.4.3 Hands-on

Validate units and time expressions without a running systemd

The tools of systemd that parse files work on any machine with the package installed. The build machine has systemd 255 installed but runs a different PID 1, so commands that talk to the manager, such as `systemctl status` and `journalctl`, are shown in this chapter as typical output only, and the following commands were run for real.

Real output of the offline systemd tools (systemd 255) TERMINAL

```
$ systemctl --version | head -1
systemd 255 (255.4-1ubuntu8.17)
$ systemd-analyze verify units/orders.service; echo "rc=$?"
rc=0
$ systemd-analyze calendar 'Mon..Fri 02:30'
  Original form: Mon..Fri 02:30
Normalized form: Mon..Fri *-*-* 02:30:00
    Next elapse: Mon 2026-10-19 02:30:00 Asia
       From now: 23h left
$ systemd-analyze timespan 1min30s
Original: 1min30s
      µs: 90000000
   Human: 1min 30s
$ systemd-escape 'my app/data'
my\x20app-data
```

An empty output with status 0 from `verify` means that the unit parsed cleanly. The `calendar` command is useful for validating `OnCalendar=` expressions of timers before deployment.

The daily commands (typical output on a systemd host)

systemctl status of the service above (typical output; not run on the build machine) TERMINAL

```
$ systemctl status orders.service
● orders.service - Orders API (Spring Boot)
     Loaded: loaded (/etc/systemd/system/orders.service; enabled; preset: enabled)
     Active: active (running) since Sun 2026-10-18 02:14:07 UTC; 3h 12min ago
   Main PID: 1894 (java)
      Tasks: 41 (limit: 18921)
     Memory: 412.3M
        CPU: 3min 42.118s
     CGroup: /system.slice/orders.service
             └─1894 /usr/bin/java -Xmx512m -jar /opt/orders/orders.jar

Oct 18 02:14:07 app01 systemd[1]: Started orders.service - Orders API (Spring Boot).
Oct 18 02:14:11 app01 java[1894]: Started OrdersApplication in 3.9 seconds
$ journalctl -u orders.service -b --no-pager -n 3
$ journalctl -p err -b
```

The `Tasks: 41` line is the number of kernel tasks in the cgroup (the JVM's 41 native threads), and the limit that follows is the `TasksMax` setting; both connect directly to the thread chapters in the next part. `Memory:` is the accounted memory of the cgroup, which is larger than the Java heap. `CGroup:` shows that the service lives in its own control group, which is how systemd finds and stops every process of a service.

1.4.4 Real-world incidents

Incident: the service starts by hand but not at boot, because the database is not ready yet

Scenario. A Spring Boot service connects to PostgreSQL on the same host at start-up. After a maintenance reboot, the service is `active (running)` but never becomes healthy, while a manual restart ten minutes later works.

Symptoms. The health endpoint does not respond for minutes after the boot. The log repeats a connection error. In the process list the JVM exists and uses almost no CPU.

Diagnosis. The questions are whether the process is alive, what it is waiting for, and how the ordering was declared. The reproduction uses a client that retries a connection to a port on which nothing listens yet.

DbWait.java JAVA

```java
import java.net.InetSocketAddress;
import java.net.Socket;

public class DbWait {
    public static void main(String[] args) throws Exception {
        int port = Integer.parseInt(args[0]);
        for (int attempt = 1; ; attempt++) {
            try (Socket s = new Socket()) {
                s.connect(new InetSocketAddress("127.0.0.1", port), 2000);
                System.out.println("connected on attempt " + attempt);
                return;
            } catch (java.io.IOException e) {
                System.out.println("attempt " + attempt + " failed: " + e.getMessage());
                Thread.sleep(1000);
            }
        }
    }
}
```

Attach to the waiting process and see the retry loop (real output; port 20101 had no listener) TERMINAL

```
$ java DbWait 20101 &
attempt 1 failed: Connection refused
attempt 2 failed: Connection refused
attempt 3 failed: Connection refused
$ strace -f -tt -p 4111 -e trace=connect,poll -o dbw.strace
$ grep -E 'connect|poll' dbw.strace | head -4 | cut -c7-
02:19:42.053080 connect(5, {sa_family=AF_INET, sin_port=htons(20101), ...}, 16) = -1 EINPROGRESS
02:19:42.053184 poll([{fd=5, events=POLLOUT}], 1, 0) = 1 ([{fd=5, revents=POLLOUT|POLLERR|POLLHUP}])
02:19:43.057646 connect(5, {sa_family=AF_INET, sin_port=htons(20101), ...}, 16) = -1 EINPROGRESS
02:19:43.057747 poll([{fd=5, events=POLLOUT}], 1, 0) = 1 ([{fd=5, revents=POLLOUT|POLLERR|POLLHUP}])
$ ls /proc/4111/task | wc -l
18
```

The trace shows one `connect` per second on port 20101, each ending in `POLLERR|POLLHUP`, which is how the kernel reports a refused connection: the SYN was answered with a reset because nothing was listening. The process is alive (18 threads) and idle. After a listener was started on the port, the loop ended on its eighth attempt with `connected on attempt 8`.

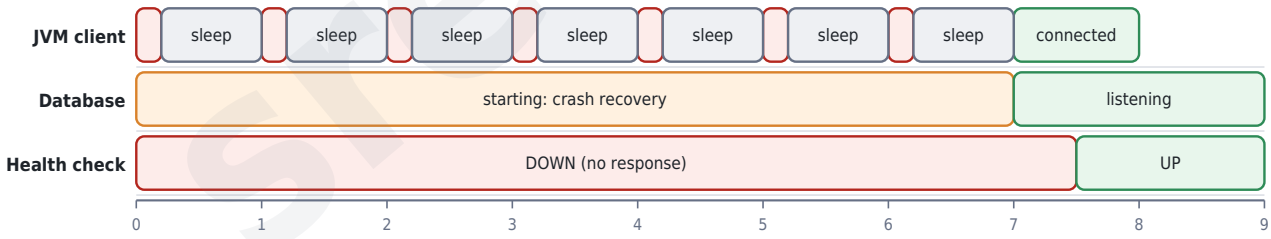


Figure 1.22. Each short red mark is a refused connect and the grey blocks are the one-second sleeps; the client alternates between a failed `connect` and a one-second sleep until the listener appears; the delay between boot and health is the time until the dependency starts, plus up to one retry interval.

On the production host the next step is to read the unit: `systemctl cat orders.service` shows whether `After=postgresql.service` is present, and `systemctl list-dependencies orders.service` shows whether PostgreSQL is in the dependency tree. The typical finding is that the service listed `After=network.target` only. `network.target` means only that the network stack is configured, not that any address is available; and ordering after a database requires the database unit to be named.

Root cause. The unit declared no ordering after the database. At boot both services started in parallel, the database needed several seconds for crash recovery, and the application, which had no retry policy beyond a long back-off, started its own connection attempts too early.

Fix. Immediate mitigation: restart the service once the database accepts connections. Permanent fix: add `After=postgresql.service` and `Wants=postgresql.service` (or `Requires=` for a hard dependency) and use `After=network-online.target` with `Wants=network-online.target`. Ordering after a unit only waits until that unit is *started*; for a database that accepts connections later, make the application resilient: a connection pool with initialisation retries (for HikariCP, `spring.datasource.hikari.initialization-fail-timeout` controls whether start-up fails or continues), and Spring Boot's readiness probe so that traffic is not routed before the pool is usable.

Prevention. Test the cold-boot path, not only a manual restart. Alert on a service that is `active` but not healthy for more than a few minutes. Keep application retry intervals short and bounded, and log the target and the reason on each failure.

Incident: a mistyped unit directive silently disables the restart policy

Scenario. A unit file is edited to enable automatic restart. A typo in the value is not noticed during review, and `systemctl daemon-reload` reports nothing.

Symptoms. Weeks later the JVM is killed by an out-of-memory condition. The service stays down until a person notices, although the file seems to request a restart.

Diagnosis. `systemctl show orders.service -p Restart` would print the effective value; the unit file is the starting point, and the offline verification of the file shows the problem.

A unit with a misspelled value (real output) — TERMINAL

```
$ sed -e 's/on-failure/on-faliure/' units/orders.service > units/bad.service
$ grep Restart= units/bad.service
Restart=on-faliure
$ systemd-analyze verify units/bad.service; echo "rc=$?"
/tmp/osw_o1/units/bad.service:11: Failed to parse service restart specifier, ignoring: on-faliure
rc=0
```

The parser reports the problem as a warning, ignores the directive and uses the default, `Restart=no`. The tool exits with status 0, so a pipeline that checks only the exit code does not detect it. The warning is written to the journal at daemon-reload time, where nobody reads it.

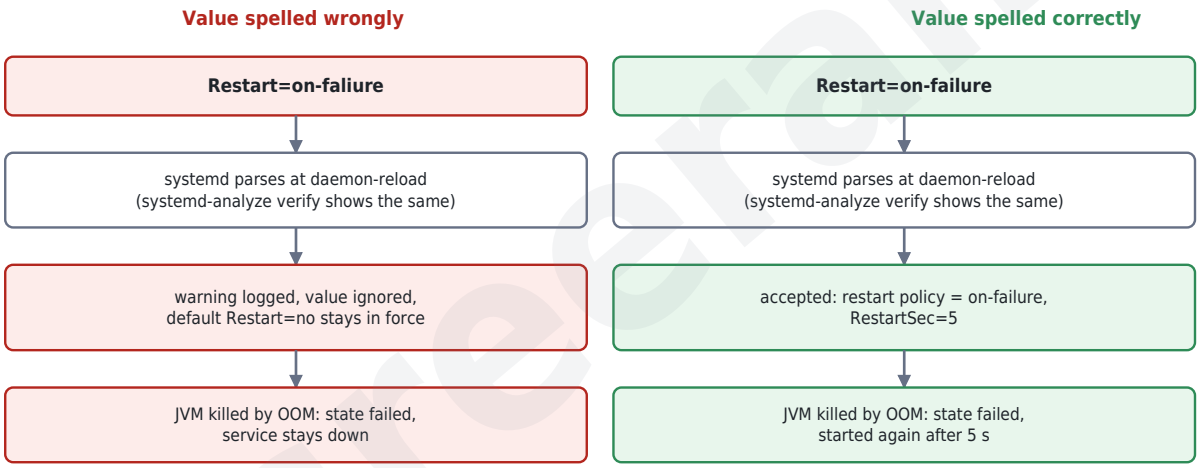


Figure 1.23. A value that systemd cannot parse is dropped, not rejected, and the unit silently runs with the default policy; the failure surfaces only when the process dies.

Root cause. systemd ignores unknown values for most directives to remain compatible with newer unit files, so an invalid `Restart=` value leaves the default `no` in force.

Fix. Correct the value, run `systemctl daemon-reload`, and confirm the effective setting with `systemctl show orders.service -p Restart -p RestartUSec`. For the immediate outage, `systemctl start orders.service`.

Prevention. Run `systemd-analyze verify` in the deployment pipeline and fail on any output, not on the exit code: `test -z "$(systemd-analyze verify unit 2>&1)"`. Confirm the effective configuration after deployment, and alert on units in the `failed` state. For crash-looping protection add `StartLimitIntervalSec` and `StartLimitBurst`.

1.4.5 Questions to expect

Questions and model answers

1. **Describe the Linux boot sequence.** Firmware initialises the hardware and starts a bootloader; the bootloader loads the kernel and the initramfs and passes the command line; the kernel initialises memory, scheduling and drivers and mounts the initramfs; the initramfs finds and mounts the real root file system; the kernel runs `/sbin/init` as PID 1, which starts services until the default target is reached.

2. **What is the initramfs for?** It holds the drivers and tools needed to reach the root file system (storage controllers, RAID, LVM, encryption), which cannot be built into the kernel in every configuration.
3. **What is special about PID 1?** It is the ancestor of all processes, it adopts and reaps orphaned processes, and the kernel ignores signals for which it has no handler.
4. **What is a systemd target?** A named grouping of units that acts as a synchronisation point, for example `multi-user.target`; it replaces the run levels of older init systems.
5. **What is the difference between `After=` and `Requires=`?** `After=` controls only the order of start; `Requires=` makes the other unit a hard dependency that is started and whose failure stops this unit. A unit usually needs both, or `Wants=` with `After=`.
6. **What does `Type=simple` mean for a Spring Boot service?** systemd regards the service as started when the process is forked, not when the application is ready; readiness must be handled by health checks or by `Type=notify` with a notification-aware launcher.
7. **How does systemd stop a service?** It sends `SIGTERM` to the main process, waits for `TimeoutStopSec`, then sends `SIGKILL` to all remaining processes of the cgroup.
8. **A service starts manually but not at boot. What is checked first?** Whether it is enabled (`systemctl is-enabled`), its ordering and dependencies (`systemctl cat`, `list-dependencies`), its logs of the boot (`journalctl -u UNIT -b`), and whether something it needs (network address, mount, database) was not yet available.
9. **Which command shows what delayed the boot?** `systemd-analyze blame` for per-unit times and `systemd-analyze critical-chain` for the sequence that determined the total.
10. **Why can a container entry point not simply be a shell script that runs `java -jar app.jar`?** The shell is PID 1 and does not forward `SIGTERM` to the child, so the JVM never runs its shutdown hooks and is killed after the grace period. The script should use `exec java ...` so that the JVM itself becomes PID 1, or the container should use a minimal init program.

KEY POINTS

The boot chain is firmware, bootloader, kernel, initramfs and PID 1; after that, systemd starts units in an order determined by explicit dependencies.

- `/proc/cmdline` shows how the kernel was started; the build machine starts its own PID 1 with `rdinit`, a real example of direct boot.
- PID 1 adopts and reaps orphans and ignores signals that it does not handle; the first process of a container has the same duties.
- `After=` orders, `Wants=` and `Requires=` request; most start-up races come from giving only one of the two.
- `Type=simple` marks a service as started when the process is forked; the application may not be ready, so health and retry logic belong in the application.
- Validate units with `systemd-analyze verify`, check the output (not only the exit code), and confirm effective values with `systemctl show`.

1.5 The Linux File-Based View

Unix introduced a design decision that Linux inherits and extends: a very large part of the system is presented as files. Regular files, directories, devices, pipes, sockets, and even the internal state of the kernel are opened, read and written with the same small set of system calls. The decision has a practical consequence for operations work. A running system can be inspected with `cat` and `ls`, tuned with `echo`, and monitored by programs that need no special interface, because the kernel publishes its state under `/proc` and `/sys` as ordinary-looking files. This chapter explains the mechanism that makes the uniform view possible, tours the directories that matter, and shows how a JVM's environment is discovered through them.

1.5.1 Concept and mechanism

The file abstraction and the virtual file system

A program that wants to use any of these objects calls `open`, receives a small integer, the **file descriptor**, and passes it to `read`, `write`, `close` and a few others. The kernel layer that gives these calls their meaning for every kind of object is the **virtual file system** (VFS). The VFS defines a common set of operations (open, read, write, seek, ...) and a common set of objects (inode, directory entry, open file). Each concrete file system or driver supplies its own implementation of the operations. When a program calls `read` on a descriptor, the VFS follows the open file to its inode and calls the implementation registered there: the block-based `ext4` code for a file on the root disk, the `procfs` code that formats text for a file under `/proc`, or the driver of a character device for a file under `/dev`.

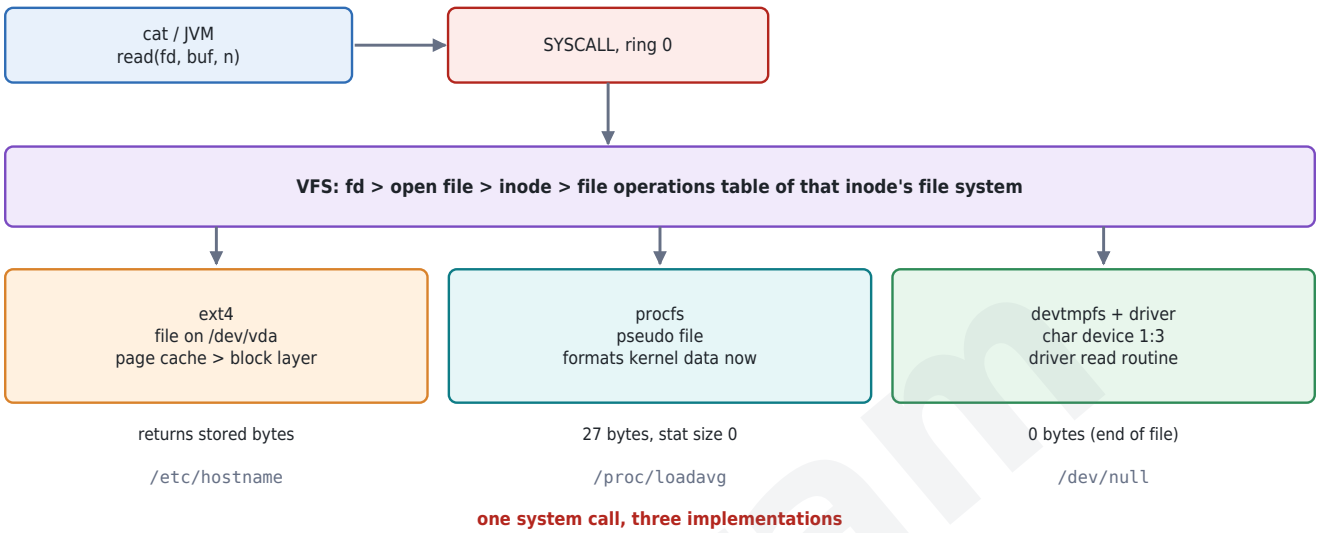


Figure 1.24. The same `read` system call reaches three different implementations; the program cannot tell them apart except by the content it receives.

Because the implementation is chosen per inode, the file type is part of the metadata. `stat` reports it, and `ls -l` shows it in the first character of the mode string.

First character	Type	Example
`-`	Regular file	`/etc/hostname`, a JAR
`d`	Directory	`/proc`, `/tmp`
`l`	Symbolic link	`/bin` (to `usr/bin`), `/dev/stdout`
`c`	Character device	`/dev/null`, `/dev/tty`
`b`	Block device	`/dev/vda`
`p`	Named pipe (FIFO)	created with `mkfifo`
`s`	Socket	Unix domain socket, such as the Docker socket

File types in Linux.

The directory hierarchy

The layout of the root directory follows the Filesystem Hierarchy Standard. Distributions have converged on a "merged `/usr`" layout in which the traditional `/bin`, `/sbin` and `/lib` are symbolic links into `/usr`.

The root directory of the build machine (Ubuntu 24.04) TERMINAL

```
$ ls -l / | grep -E ' (bin|sbin|lib|lib64) ->' | awk '{print $9,$10,$11}'
bin -> usr/bin
lib -> usr/lib
lib64 -> usr/lib64
sbin -> usr/sbin
$ ls /usr
bin  games  include  lib  lib64  libexec  local  sbin  share  src
$ stat -c '%n %a %U' /tmp /root /etc/shadow /etc/passwd /home
/tmp 1777 root
/root 700 root
/etc/shadow 640 root
/etc/passwd 644 root
/home 755 root
```

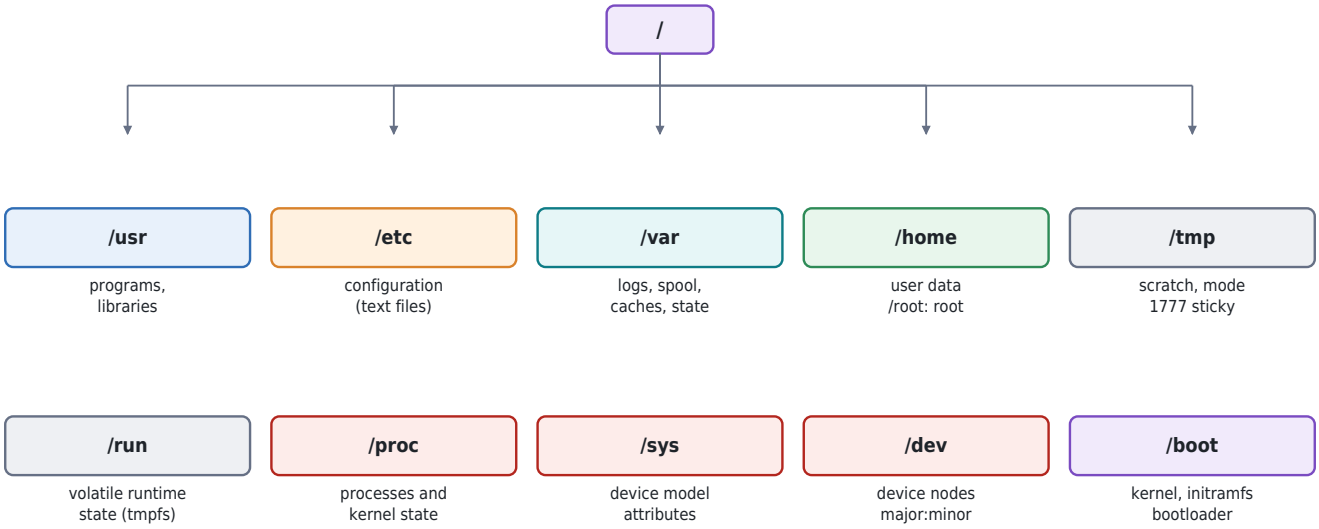


Figure 1.25. The hierarchy separates programs (`/usr`), configuration (`/etc`), variable data (`/var`), kernel views (`/proc`, `/sys`, `/dev`) and volatile runtime state (`/run`, `/tmp`).

The permission bits shown above follow a pattern worth noting: `/etc/shadow`, which holds password hashes, is mode 640 and unreadable by ordinary users; `/tmp` carries mode 1777, where the leading `1` is the *sticky bit* that lets every user create files but delete only their own.

The kernel's views: /proc, /sys and /dev

Three pseudo file systems expose the kernel. None of them is stored on disk.

- `/proc` **(procfs)** presents processes and system-wide state. Each process has a directory `/proc/PID`, and each thread has `/proc/PID/task/TID`. System-wide files include `meminfo`, `loadavg`, `cpuinfo`, `interrupts`, `stat`, `diskstats` and the tunable parameters under `/proc/sys`.
- `/sys` **(sysfs)** presents the device and driver model as a tree: one directory per bus, class and device, with one small file per attribute. A file holds one value.
- `/dev` **(devtmpfs)** holds the device nodes through which programs reach drivers. A device file has no content; it holds a pair of numbers, the *major* number (which driver) and the *minor* number (which device of that driver).

Sizes and types are not what they seem (typical output) TERMINAL

```
$ stat -c '%n: %F, size=%s' /proc/loadavg /proc/self/status /dev/null /etc/hostname
/proc/loadavg: regular empty file, size=0
/proc/self/status: regular empty file, size=0
/dev/null: character special file, size=0
/etc/hostname: regular file, size=3
$ strace -o cat.tr cat /proc/loadavg > /dev/null; grep -E 'loadavg|read\(3, "[0-9]' cat.tr
openat(AT_FDCWD, "/proc/loadavg", O_RDONLY) = 3
read(3, "1.44 1.98 1.85 2/119 15462\n", 131072) = 27
```

The file `/proc/loadavg` reports a size of zero, yet a `read` returns 27 bytes. The kernel generates the text at the moment of the read from current data structures; no stored bytes exist. The consequence is that tools should not use the size of a `/proc` file to allocate a buffer, and that every read returns a fresh snapshot.

A tour of /proc, /sys and /dev (real output) TERMINAL

```
$ ls /proc | grep -v '^[0-9]' | tr '\n' ' ' | cut -c1-150
acpi buddyinfo bus cgroups cmdline config.gz consoles cpuinfo crypto devices diskstats dma
$ cat /proc/loadavg /proc/uptime
2.46 1.05 0.40 1/125 4077
472.90 741.81
$ head -5 /proc/meminfo
MemTotal:        8222320 kB
MemFree:         7461868 kB
MemAvailable:    7595916 kB
Buffers:            6616 kB
Cached:           346444 kB
$ ls /proc/sys; cat /proc/sys/kernel/pid_max /proc/sys/fs/file-max /proc/sys/vm/swappiness
```

A tour of /proc, /sys and /dev (real output) (continued) TERMINAL

```
abi  debug  dev  fs  kernel  net  user  vm
32768
819675
60
$ ls -d /sys/*/ | tr '\n' ' ' | cut -c1-90
/sys/block/ /sys/bus/ /sys/class/ /sys/dev/ /sys/devices/ /sys/firmware/ /sys/fs/ ...
$ cat /sys/devices/system/cpu/online /sys/block/vda/queue/scheduler /sys/block/vda/queue/rotational
0-1
none [mq-deadline] kyber bfq
1
$ cat /sys/kernel/mm/transparent_hugepage/enabled
always [madvise] never
```

The fields of interest: the load average and the "running/total" count `1/125` in `/proc/loadavg`; the four main values of `meminfo`, which the memory part of this book explains; `pid_max` (32768 on this machine, the upper bound of process and thread identifiers, which limits the total number of threads on the host); `file-max` (the system-wide limit on open file descriptors); and `vm.swappiness`. In `/sys`, the brackets mark the active choice: the block device `vda` uses the `mq-deadline` I/O scheduler and reports `rotational` as 1 (this virtual disk advertises itself as rotating, which is common in virtualised environments and does not reflect the physical medium); transparent huge pages are enabled only for memory regions that request them (`madvise`).

The parameters under `/proc/sys` are writable by root, which is what the `sysctl` command does. The command and a direct read are equivalent:

sysctl is a thin wrapper over /proc/sys TERMINAL

```
$ sysctl vm.swappiness
vm.swappiness = 60
$ strace -e trace=openat sysctl vm.swappiness 2>&1 | grep proc/sys
openat(AT_FDCWD, "/proc/sys/vm/swappiness", O_RDONLY) = 3
```

Settings written to `/proc/sys` take effect immediately and vanish at reboot; persistence requires a file in `/etc/sysctl.d/`.

Device files

A device node is a name for a driver. The node's identity is its type and number pair, not its name, which the following experiment demonstrates by creating a second name for the null device.

Two names, one driver (run as root) TERMINAL

```
$ stat -c '%F %t:%T %n' /dev/null /dev/zero /dev/vda
character special file 1:3 /dev/null
character special file 1:5 /dev/zero
block special file fe:0 /dev/vda
$ mknod /tmp/osw_o1/mynull c 1 3 && ls -l /tmp/osw_o1/mynull
crw-r--r-- 1 root root 1, 3 Oct 18 02:35 /tmp/osw_o1/mynull
$ echo "test data" > /tmp/osw_o1/mynull; echo "rc=$?"; head -c 4 /tmp/osw_o1/mynull | wc -c
rc=0
0
$ dd if=/dev/zero bs=1M count=2 2>&1 > /dev/null | tail -1
2097152 bytes (2.1 MB, 2.0 MiB) copied, 0.000491884 s, 4.3 GB/s
$ echo ab > /dev/full; echo "rc=$?"
/bin/bash: line 1: echo: write error: No space left on device
rc=1
```

The device `1:3` discards writes and returns end-of-file on reads, whatever its name; `/dev/zero` (`1:5`) yields zero bytes without end, and `/dev/full` (`1:7`) fails every write with `ENOSPC`, which is a convenient way to test an application's behaviour when a disk is full. The mode and owner of a device node, not the driver, decide who may use it, which is why access to a device such as `/dev/kvm` is controlled with groups.

Kernel modules

Drivers and file systems that are not needed at every boot are built as **loadable kernel modules**: object files with extension `.ko` stored under `/lib/modules/KERNEL_VERSION/`. The commands are `lsmod` (list loaded modules, which reads `/proc/modules`), `modinfo NAME` (show the description, parameters and dependencies), `modprobe NAME` (load a module and its dependencies) and `modprobe -r NAME` (unload). The build machine boots with the parameter `nomodule` and has no `/lib/modules`, so a module listing cannot be shown from it. On a typical server the listing resembles the following.

lsmod on a typical server (typical output; not run on the build machine) TERMINAL

```
$ lsmod | head -5
Module                  Size  Used by
overlay               212992  8
nf_conntrack          196608  3 nft_ct,nf_nat,xt_conntrack
virtio_net             69632  0
ext4                 1081344  1
```

The `Used by` column is a reference count and names the dependants; a module with a non-zero count cannot be unloaded. Container hosts rely on modules such as `overlay` (the union file system used for image layers) and `nf_conntrack` (connection tracking), which is why a missing module on a minimal host image causes container start-up errors.

1.5.2 Worked trace

Path resolution of a read of /proc/loadavg

The steps below follow what happens inside `cat /proc/loadavg`, using the values of the trace above.

1. `cat` calls `openat(AT_FDCWD, "/proc/loadavg", O_RDONLY)`. The kernel resolves the path one component at a time, starting from the root directory of the process: the entry `proc` in `/` leads to the mount of the `proc` file system, and the entry `loadavg` in that file system leads to an inode that procfs created.
2. The kernel allocates an *open file* object that records the position and the flags, links it to the inode and the file operations of procfs, and returns the lowest unused descriptor, 3.
3. `cat` calls `read(3, buf, 131072)`. The VFS calls the `read` operation registered by procfs. That operation formats the current load averages and the count of runnable tasks into the buffer: `"1.44 1.98 1.85 2/119 15462\n"`. It returns 27, the number of bytes produced.
4. `cat` writes the 27 bytes to descriptor 1 and calls `read` again. The operation is positioned at the end and returns 0, which signals end of file.
5. `cat` calls `close(3)`, which releases the open file object. The program exits.

The fields of the line are the load averages over 1, 5 and 15 minutes (1.44, 1.98, 1.85), the number of currently runnable scheduling entities over the total (2/119) and the identifier of the most recently created process (15462).

1.5.3 Hands-on

How the JVM reads its environment from the file system

A JVM adapts to its surroundings by reading kernel files. The earlier `strace` of `java Hello` listed the control-group files that it opens. The program `Env` shows the result of that adaptation.

Env.java JAVA

```java
import java.lang.management.ManagementFactory;

public class Env {
    public static void main(String[] args) {
        Runtime rt = Runtime.getRuntime();
        System.out.println("availableProcessors = " + rt.availableProcessors());
        System.out.println("maxMemory (MiB)     = " + rt.maxMemory() / (1024 * 1024));
        System.out.println("GC                  = " + ManagementFactory.getGarbageCollectorMXBeans()
                .stream().map(b -> b.getName()).toList());
    }
}
```

What the JVM decides from the files it reads (Java 21; the shell runs in a 6 GB cgroup) TERMINAL

```
$ java Env
availableProcessors = 2
maxMemory (MiB)     = 1496
GC                  = [G1 Young Generation, G1 Concurrent GC, G1 Old Generation]
$ java -Xlog:os+container=trace -version 2>&1 | grep -m2 -E 'Memory Limit|hybrid'
[0.000s][debug][os,container] Detected cgroups hybrid or legacy hierarchy, using cgroups v1 ...
[0.000s][trace][os,container] Memory Limit is: 6272172032
$ java -XX:+PrintFlagsFinal -version | grep -E ' MaxHeapSize | MaxRAMPercentage ' | tr -s ' '
 size_t MaxHeapSize = 1568669696 {product} {ergonomic}
 double MaxRAMPercentage = 25.000000 {product} {default}
```

The maximum heap of 1,568,669,696 bytes is exactly 25 percent of the 6,272,172,032 byte limit of the cgroup in which the shell runs (the machine has 8 GB of RAM, but the sandbox limits the group). The JVM read `memory.limit_in_bytes` and applied `MaxRAMPercentage`. This is the mechanism that the incident later in this chapter relies on.

Read the open files and limits of a process

The limits and the descriptors of the current shell (real output) TERMINAL

```
$ cat /proc/self/limits | grep -E 'Max (processes|open files|stack size|address space)'
Max stack size            8388608              unlimited            bytes
Max processes             32044                32044                processes
Max open files            20000                20000                files
Max address space         unlimited            unlimited            bytes
$ ls -l /proc/self/fd | cut -c1-70
total 0
lr-x------ 1 root root 64 Oct 18 02:17 0 -> /dev/null
l-wx------ 1 root root 64 Oct 18 02:17 1 -> /tmp/claude-0/-home-claude/df14931c-0a1a-560d
l-wx------ 1 root root 64 Oct 18 02:17 2 -> /tmp/claude-0/-home-claude/df14931c-0a1a-560d
lr-x------ 1 root root 64 Oct 18 02:17 3 -> /proc/4249/fd
$ readlink /proc/self/ns/pid /proc/self/ns/mnt
pid:[4026531836]
mnt:[4026531832]
```

The three descriptors 0, 1 and 2 are standard input, output and error; descriptor 3 is the directory handle that `ls` itself opened to read the listing, which demonstrates that `/proc/PID/fd` shows the live descriptor table. The `limits` file is the authoritative source of the limits that apply to a running process, and it is the first thing to read when an application reports `Too many open files`. The namespace links at the end identify the PID and mount namespaces; two processes with identical values share them, a fact that the container chapter uses.

1.5.4 Real-world incidents

Incident: the service works in the shell and fails under the service manager with too many open files

Scenario. A Java service that accepts many client connections is tested on a developer laptop and in an interactive shell on a staging server, where it handles several thousand connections. In production, started by the service manager, it fails under much lower load.

Symptoms. The log fills with `java.io.IOException: Too many open files`, accept calls fail, and health checks that need a new connection fail while established connections keep working.

Diagnosis. The limit of the running process, not the limit of the shell, is what matters.

1. `cat /proc/PID/limits | grep 'open files'` reports the limits of the process that is failing.
2. `ls /proc/PID/fd | wc -l` reports how many descriptors are in use.
3. Comparing the two shows whether the process has reached its limit. The command that follows reproduces the situation with a program that opens one file repeatedly until the limit stops it.

OpenMany.java JAVA

```
import java.io.FileInputStream;
import java.util.ArrayList;
import java.util.List;

public class OpenMany {
    public static void main(String[] args) throws Exception {
        int want = Integer.parseInt(args[0]);
        List<FileInputStream> open = new ArrayList<>();
        try {
            for (int i = 0; i < want; i++) open.add(new FileInputStream("/etc/hostname"));
            System.out.println("opened " + open.size() + " files");
        } catch (java.io.IOException e) {
            System.out.println("failed after " + open.size() + " files: " + e.getMessage());
        }
        for (FileInputStream f : open) f.close();
    }
}
```

The same program under three limits (real output) TERMINAL

```
$ ulimit -n
20000
$ java OpenMany 3000
opened 3000 files
$ (ulimit -n 1024; java OpenMany 3000)
failed after 1020 files: /etc/hostname (Too many open files)
$ (ulimit -n 1024; strace -f -e trace=openat -o fd.tr java OpenMany 3000 > /dev/null; \
    grep EMFILE fd.tr | head -1 | cut -c1-80)
5691  openat(AT_FDCWD, "/etc/hostname", O_RDONLY) = -1 EMFILE (Too many open files)
$ (ulimit -Sn 1024; java OpenMany 3000)
opened 3000 files
```

Three observations follow. With a limit of 1024 the program failed after 1,020 files; the other four descriptors were already taken by standard streams and the runtime. The error number in the trace is `EMFILE`, whose message is "Too many open files". The last command is the surprise: lowering only the *soft* limit to 1024 did not prevent the program from opening 3,000 files, because the JVM, with the default `-XX:+MaxFDLimit`, raises its soft limit to the hard limit at start-up (20,000 here). Whether a limit is effective therefore depends on the hard limit and on how the process was started.

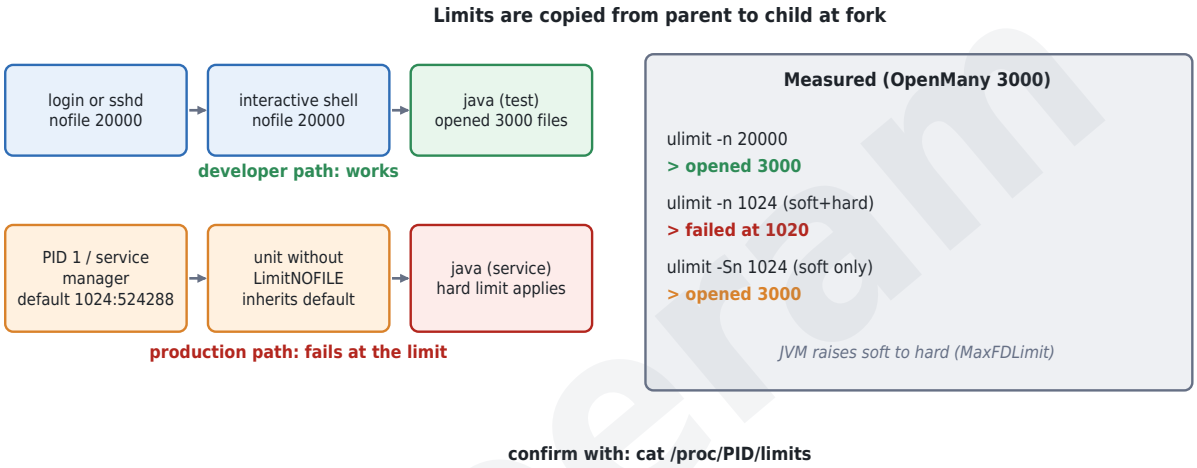


Figure 1.26. Limits are inherited from the parent at `fork`: a service manager, a container runtime or a login shell each start their children with their own values, so the effective limit of a process is a property of its ancestry.

Root cause. The interactive shell had a high limit, either inherited from a login configuration or set by the administrator, and the service manager started the process with its own default, which on many distributions has been 1,024 for the soft limit and 524,288 or a lower value for the hard limit, depending on the version of systemd and the unit settings. Each connection consumes a descriptor, so a low limit allows only a few hundred clients once files, log streams and database connections are counted.

Fix. The immediate mitigation is to restart the service with a higher limit. The permanent fix sets the limit where the process is started: `LimitNOFILE=65536` in the systemd unit, `--ulimit nofile=65536:65536` in a container run command, or the `ulimits` of the orchestrator. The limit that was applied is confirmed with `cat /proc/PID/limits`, not by reading the unit file.

Prevention. Monitor descriptor use as a ratio of the limit of each process, `ls /proc/PID/fd | wc -l` against `Max open files`. Alert at 80 percent. Include a load test in which the number of connections exceeds the expected peak, run under the same service manager as production.

Incident: the same JVM selects a different collector and heap in the container than on the laptop

Scenario. A service runs well on developer machines and in an integration environment, but after deployment to a container with a 512 MB memory limit and a CPU quota of half a core it shows long pauses and frequent garbage collection, although the same JAR and the same command line were used.

Symptoms. Response times degrade under moderate load; the garbage collector log shows a different collector name than in the other environments; and the heap, which was never configured, is far smaller than expected.

Diagnosis. The JVM chooses its collector, heap size and thread counts from the processor count and memory it observes, and in a container it observes the control-group limits. The program `Env` was run under four conditions on the build machine. The cgroup limits were created for the experiment under `/sys/fs/cgroup` and removed afterwards.

JVM ergonomics under different limits (real output; Java 21, cgroup v1) TERMINAL

```
$ java Env
availableProcessors = 2
maxMemory (MiB)     = 1496
GC                  = [G1 Young Generation, G1 Concurrent GC, G1 Old Generation]
$ taskset -c 0 java Env
availableProcessors = 1
maxMemory (MiB)     = 1446
GC                  = [Copy, MarkSweepCompact]
$ java -Xmx256m Env
availableProcessors = 2
maxMemory (MiB)     = 256
GC                  = [G1 Young Generation, G1 Concurrent GC, G1 Old Generation]
# memory cgroup limit 512m (memory.limit_in_bytes = 536870912), same machine
$ bash -c "echo \$\$ > /sys/fs/cgroup/memory/osw_o1/cgroup.procs; exec java Env"
availableProcessors = 2
maxMemory (MiB)     = 123
GC                  = [Copy, MarkSweepCompact]
# cpu cgroup: cfs_quota_us = 50000 with cfs_period_us = 100000 (half a core)
$ bash -c "echo \$\$ > /sys/fs/cgroup/cpu/osw_o1/cgroup.procs; exec java Env"
availableProcessors = 1
maxMemory (MiB)     = 1446
GC                  = [Copy, MarkSweepCompact]
```

The results explain the symptoms. The JVM selects G1 only on a "server-class" machine: at least two available processors and at least about 1.8 GB of memory. With one processor (an affinity mask, or a CPU quota below one core, which the JVM rounds up to one processor) or with a 512 MB limit it falls back to the serial collector, whose collections stop all application threads. The memory limit of 512 MiB produced a default maximum heap of 123 MiB, about a quarter of the limit, because `MaxRAMPercentage` defaults to 25. Setting `-Xmx` explicitly decouples the heap from the limit (the third run), and the processor count can be forced with `-XX:ActiveProcessorCount=N`.

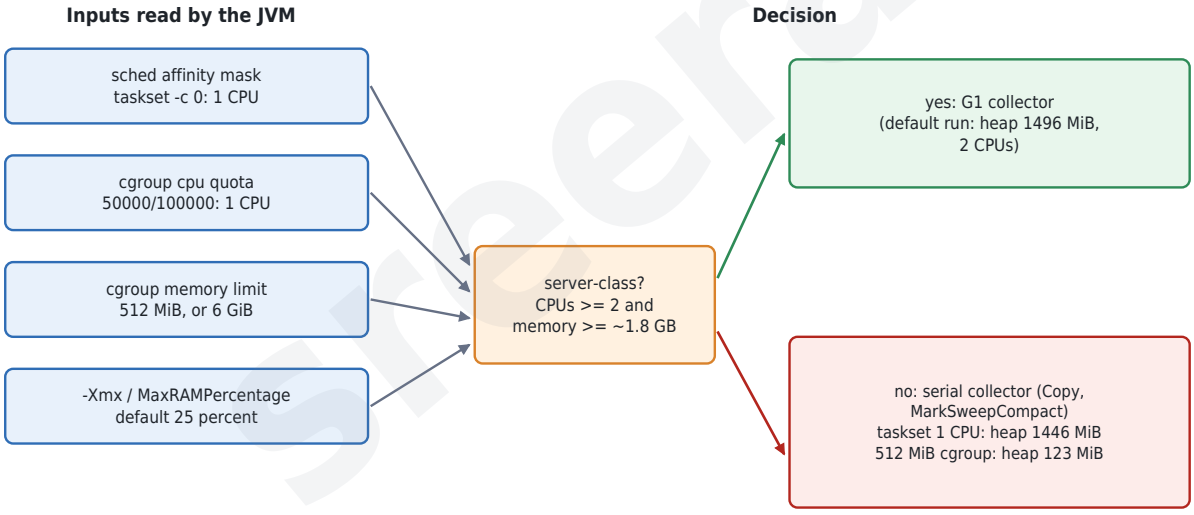


Figure 1.27. The inputs the JVM reads from the kernel (processor mask, cgroup CPU quota, cgroup memory limit) determine the collector and the heap; two environments with the same JAR can therefore behave as different runtimes.

Root cause. The deployment limits were different from the environments in which the application was tuned, and the application relied on ergonomic defaults. The 512 MB limit and fractional CPU selected the serial collector and a heap of about 123 MiB; the pauses of a serial collection on a request path are long compared with G1.

Fix. Set the collector and heap explicitly for the production resource envelope, for example `-XX:+UseG1GC -XX:MaxRAMPercentage=70` with at least 1 GB of memory, or choose `-XX:+UseSerialGC` deliberately for a very small container. Raise the memory limit so that the heap plus non-heap memory fits: a heap of 70 percent of the limit leaves 30 percent for metaspace, thread stacks, the code cache and native buffers, which the memory part of this book quantifies.

Prevention. Print the effective configuration at start-up (`-XX:+PrintFlagsFinal` or `-Xlog:gc*`, `Runtime.availableProcessors()`) and compare it between environments in the deployment pipeline. Test performance in an environment with the production limits. Document the container resource envelope next to the JVM flags.

1.5.5 Questions to expect

Questions and model answers

1. **What does "everything is a file" mean in Linux?** Devices, pipes, sockets and kernel information are reached through the same descriptor-based calls as regular files; the VFS dispatches each call to the right implementation, so tools such as `cat` and `echo` work on all of them.
2. **What is the VFS?** The kernel layer that defines common file objects and operations, and calls the implementation of the particular file system or driver for each inode.
3. **What is `/proc` and why do its files have size 0?** A pseudo file system that exposes kernel and process state. Content is generated at read time, so no stored size exists.
4. **How does a JVM find out how much memory it may use inside a container?** It reads the cgroup files (`memory.limit_in_bytes` for cgroup v1, `memory.max` for v2) and, unless `-Xmx` is given, sets the maximum heap to `MaxRAMPercentage` (25 by default) of that value.
5. **What are major and minor numbers?** The identity of a device node: the major number selects the driver, the minor number a device handled by it. `1:3` is `/dev/null`.
6. **Where are kernel parameters changed and how are they made permanent?** Under `/proc/sys`, directly or with `sysctl -w`; the change is lost at reboot unless written to a file in `/etc/sysctl.d/`.
7. **What is the difference between `/proc/sys` and `/sys`?** `/proc/sys` holds tunable kernel parameters; `/sys` exposes the device model, one attribute per file.
8. **A service reports "Too many open files" although `ulimit -n` shows 65536 in the shell. Why?** The shell's limit does not apply to a process started by a service manager or container runtime. The process limit is in `/proc/PID/limits` and is set by the unit file, container option or orchestrator.
9. **Why might the same JAR use the serial garbage collector in one environment and G1 in another?** The JVM selects the collector from the processor count and memory it sees; fewer than two processors or less than about 1.8 GB selects the serial collector.
10. **What is a kernel module?** Code loaded into the running kernel on demand, usually a driver or file system; it runs in ring 0 with full privilege, and `lsmod`, `modinfo` and `modprobe` manage it.

KEY POINTS

The Linux interface to its own state is the file system, which makes diagnosis possible with general-purpose tools.

- The VFS dispatches `open`, `read` and `write` to the implementation of each inode: disk file systems, procfs, sysfs, device drivers.
- `/proc` shows processes and system state (generated on read), `/sys` shows the device model, `/dev` holds device nodes identified by major and minor numbers.
- Kernel tunables live under `/proc/sys`; `sysctl` reads and writes them, and `/etc/sysctl.d/` makes them persistent.
- The effective limits of a process are in `/proc/PID/limits` and depend on the process's ancestry, not on the shell used for testing.
- A JVM derives its collector, heap and thread counts from the processor mask and cgroup limits it reads; set the important ones explicitly.

1.6 The Diagnostic Toolbox

Production problems are diagnosed with a small set of tools applied in a fixed order. Each tool reads kernel data that the previous chapters introduced (`/proc`, `/sys`, system calls) and presents it as one screen of numbers. The difficulty is not running the tools but reading them: a single misread field, such as treating the `free` column of `free` as the memory available or the `% time` of `strace -c` as work, sends the investigation in the wrong direction. This chapter explains the output of `ps`, `top`, `free`, `vmstat`, `lsof` and `jcmd` field by field, presents a structured method for deciding which resource to examine, and applies it to two incidents reproduced on the build machine.

1.6.1 Concept and mechanism

The USE method and the 60-second checklist

The **USE method**, proposed by Brendan Gregg, states a rule for a first pass over a system: for every resource, check its **utilisation** (the fraction of time or capacity in use), its **saturation** (the amount of work that is waiting because the resource is busy) and its **errors** (counts of failed operations). Resources are the processors, memory, storage devices, network interfaces and also software resources such as thread pools and file descriptor tables. The method terminates after a finite number of checks, and it prevents the common failure of concentrating on the first anomaly that is found.

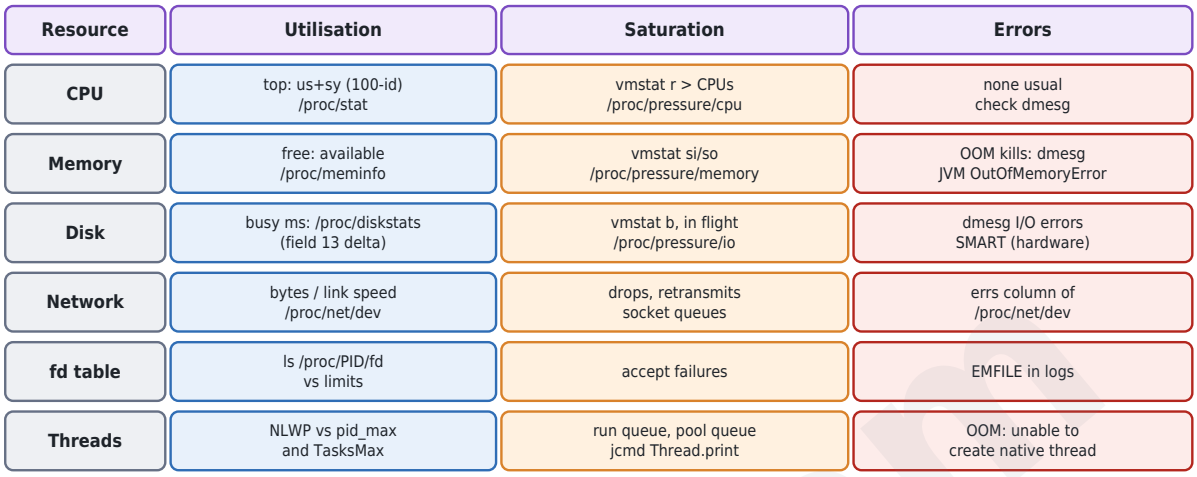


Figure 1.28. Every cell of the matrix names a concrete command or file, so that a first pass over a host is a lookup, not an exercise in recall.

A condensed procedure for a Linux host, often called the 60-second checklist, runs about ten commands. The classic list uses tools from the `sysstat` package (`mpstat`, `pidstat`, `iostat`, `sar`), which are not installed on the build machine; the equivalent data are in `/proc` and the following table gives the substitutes that were used here.

Step	Classic command	Used here
Load averages and uptime	`uptime`	`uptime`, `cat /proc/loadavg`
Kernel errors	`dmesg` (last lines)	`dmesg --level=err,warn` (last lines)
Run queue, memory, swap, CPU split	`vmstat 1`	`vmstat 1 5`
Per-CPU balance	`mpstat -P ALL 1`	`top -b -n1 -1`
Per-process CPU	`pidstat 1`	`ps -eo pid,pcpu,nlwp,comm --sort=-pcpu`
Disk utilisation and latency	`iostat -xz 1`	`/proc/diskstats`, `/proc/pressure/io`
Memory	`free -m`	`free -m`, `/proc/meminfo`
Network	`sar -n DEV 1`	`/proc/net/dev`
Overall	`top`	`top -b -n1`, `top -H`

The 60-second checklist and the commands used in this book.

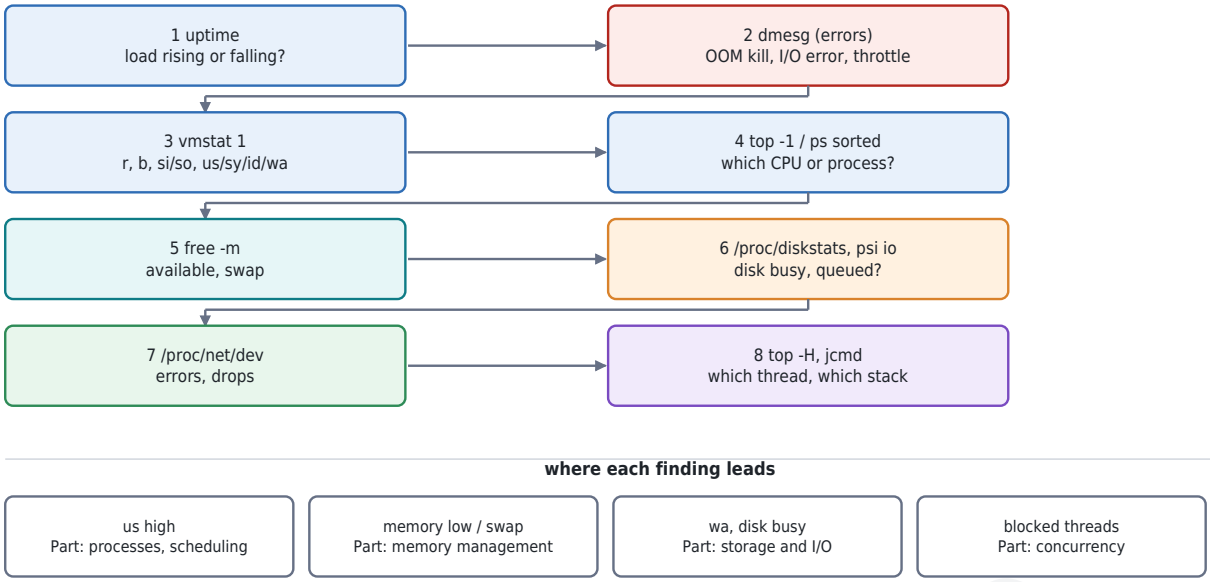


Figure 1.29. The checklist is ordered so that the cheapest and most general checks come first, and each branch points to the part of this book that treats the resource in depth.

1.6.2 Worked trace

The following listings come from one session on the build machine in which the Java program `Work` runs a busy thread named `spinner` (an endless arithmetic loop), a sleeping thread named `waiter` and 20 MiB of retained arrays, with `-Xmx128m -Xms64m`. The machine has two virtual processors and is shared with other work, so system-wide figures include other activity; per-process figures are exact.

Work.java JAVA

```java
import java.util.ArrayList;
import java.util.List;

public class Work {
    static volatile long sink;

    public static void main(String[] args) throws Exception {
        System.out.println("pid=" + ProcessHandle.current().pid());
        Thread spinner = new Thread(() -> {
            long x = 0;
            while (true) { x += x * 31 + 7; sink = x; }
        }, "spinner");
        spinner.setDaemon(true);
        spinner.start();
        Thread waiter = new Thread(() -> {
            try { Thread.sleep(600_000); } catch (InterruptedException e) { }
        }, "waiter");
        waiter.setDaemon(true);
        waiter.start();
        List<byte[]> keep = new ArrayList<>();
        for (int i = 0; i < 20; i++) keep.add(new byte[1024 * 1024]);
        Thread.sleep(600_000);
    }
}
```

ps: the process table

ps in two formats (the process has been running for 4 seconds) TERMINAL

```
$ ps aux | awk 'NR==1 || $2==15141' | cut -c1-90
USER       PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root     15141 81.0  0.7 2720628 63608 ?       Sl   02:35   0:03 java -Xmx128m -Xms64m Work
$ ps -o pid,ppid,stat,ni,pri,pcpu,pmem,vsz,rss,nlwp,etime,time,comm -p 15141
  PID  PPID STAT  NI PRI %CPU %MEM    VSZ   RSS NLWP     ELAPSED     TIME COMMAND
15141 15139 Sl     0  19 81.0  0.7 2720628 63608  21       00:04 00:00:03 java
```

Field	Meaning
`PID`, `PPID`	Process identifier and the parent's identifier. The parent 15139 is the `timeout` wrapper that started the JVM.
`%CPU`	CPU time divided by elapsed time since the start of the process: a lifetime average, not the current rate. 81 percent here because the JVM spent its first second starting.
`%MEM`	Resident memory as a share of physical memory.
`VSZ`	Virtual size in KiB: all reserved address space, 2.7 GB for a JVM with `-Xmx128m`. Not a measure of consumption.
`RSS`	Resident set size in KiB: pages currently in physical memory, 63,608 KiB (62 MiB).
`STAT`	State letter, then modifiers. `S` interruptible sleep, `R` running or runnable, `D` uninterruptible sleep (usually waiting for I/O), `Z` zombie, `T` stopped; `l` multi-threaded, `s` session leader, `<` high priority, `N` low priority.
`NI`, `PRI`	Nice value and kernel priority.
`NLWP`	Number of threads (light-weight processes): 21 here. `ps -L` lists them.
`TIME`	Cumulative CPU time consumed by all threads.

Fields of the ps output.

top: the live view

top in batch mode, process view and thread view TERMINAL

```
$ top -b -n1 -p 15141 | head -8
top - 02:35:09 up 25 min,  0 user,  load average: 1.30, 2.00, 1.85
Tasks:   1 total,   0 running,   1 sleeping,   0 stopped,   0 zombie
%Cpu(s): 52.6 us,  0.0 sy,  0.0 ni, 47.4 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   8029.6 total,   6748.1 free,    800.6 used,    705.9 buff/cache
MiB Swap:      0.0 total,      0.0 free,      0.0 used.   7229.0 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
15141 root      20   0 2720628  63608  23528 S 100.0   0.8   0:03.46 java
$ top -H -b -n1 -p 15141 | sed -n '2p;7,9p'
Threads:  22 total,   1 running,  21 sleeping,   0 stopped,   0 zombie
  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
15159 root      20   0 2721652  63824  23532 R  99.9   0.8   0:07.69 spinner
15141 root      20   0 2721652  63824  23532 S   0.0   0.8   0:00.00 java
```

The header has five parts. The first line holds the time, the uptime and the three load averages. The `Tasks` line counts processes by state (with `-H`, it counts threads, 22 here). The `%Cpu(s)` line divides CPU time among eight categories: `us` user code, `sy` kernel code on behalf of processes, `ni` user code of niced processes, `id` idle, `wa` idle while I/O was outstanding, `hi` hardware interrupts, `si` software interrupts, `st` time stolen by the hypervisor. The sum is 100 percent of the sampling interval across all CPUs. The memory lines are explained next. In the process rows, `%CPU` is the current rate over the last interval and can exceed 100 for a multi-threaded process; `RES` and `SHR` are resident and shared memory in KiB; `S` is the state letter. The thread view shows what the process view hides: one thread, `spinner` (tid 15159), uses 99.9 percent of a core, while the other 21 are idle.

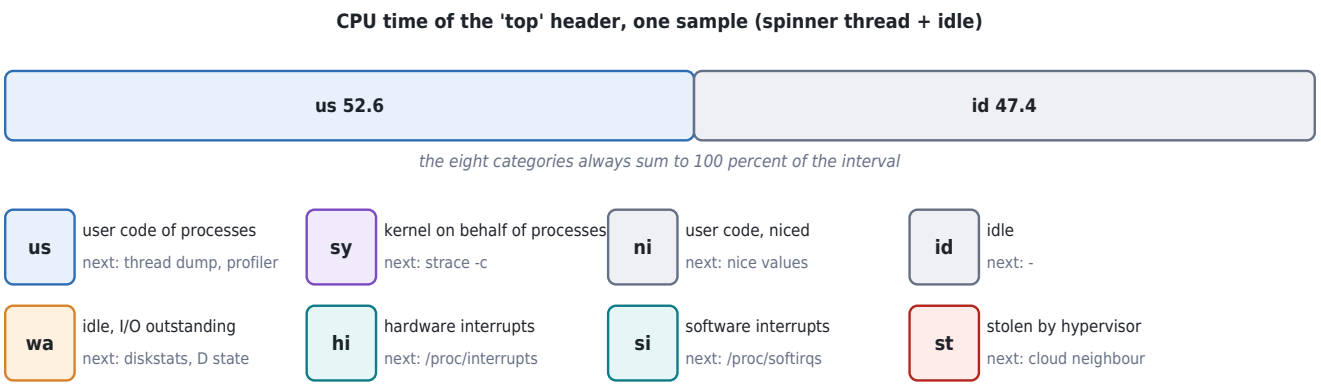


Figure 1.30. The eight CPU categories of the `top` header partition the time of every processor; reading which one grows tells which layer to examine next.

free and vmstat: memory and system activity

free and vmstat (vmstat sampled once per second) TERMINAL

```
$ free -m
               total        used        free      shared  buff/cache   available
Mem:            8029         799        6749          14         705        7230
Swap:              0           0           0
$ vmstat 1 4
procs -----------memory---------- ---swap-- -----io---- -system-- -------cpu-------
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st gu
 3  0      0 6911244  14188 708644    0    0  1383  8583 4376  118 32  7 59  2  1  0
 1  0      0 6922676  14188 708652    0    0     0     0  801  790 52  2 47  0  0  0
 4  0      0 6930360  14188 708664    0    0     0    24  929 1137 59  4 36  0  0  0
 4  0      0 6934636  14188 708684    0    0     0    12  910 1786 60  3 37  0  1  0
```

In `free`, `total` is the physical memory (8,029 MiB); `used` is `total` minus `available` (8,029 - 7,230 = 799); `free` is memory that holds nothing at all; `buff/cache` is memory holding file data and kernel buffers, which the kernel reclaims when applications need it; and `available` is the estimate of memory that can be given to new work without swapping. `available` **is the column to use**: the small `free` figure on a long-running server is normal and is not a sign of a shortage. The swap row is zero because this machine has no swap.

The first line of `vmstat` is an average since boot and is discarded. Subsequent lines show the following columns.

Column	Meaning
`r`	Runnable threads (running or waiting for a processor). A value persistently above the processor count means that the CPU is saturated; here 4 runnable tasks on 2 CPUs.
`b`	Threads in uninterruptible sleep, usually blocked on I/O.
`swpd`, `free`, `buff`, `cache`	Memory figures in KiB.
`si`, `so`	KiB per second swapped in from and out to disk. Sustained non-zero values mean memory pressure.
`bi`, `bo`	Blocks per second read from and written to block devices.
`in`, `cs`	Interrupts and context switches per second.
`us`, `sy`, `id`, `wa`, `st`, `gu`	CPU percentages: user, system, idle, I/O wait, steal, guest.

Columns of vmstat.

lsof and the process's file descriptors

lsof for the process, by descriptor type TERMINAL

```
$ lsof -p 15141 | awk 'NR==1 || $4 ~ /^(cwd|rtd|txt|0|1|2)/' | cut -c1-100
COMMAND   PID USER   FD   TYPE             DEVICE  SIZE/OFF     NODE NAME
java    15141 root  cwd    DIR              254,0      4096   966698 /tmp/osw_o1
java    15141 root  rtd    DIR              254,0      4096        2 /
java    15141 root  txt    REG              254,0     14456   241261 /usr/lib/jvm/java-21-...
java    15141 root    0r   CHR                1,3       0t0        3 /dev/null
java    15141 root    1w   REG              254,0        10   967682 /tmp/osw_o1/work2.out
$ lsof -p 15141 | awk 'NR>1{print $4}' | sed 's/[0-9]*//' | sort | uniq -c | sort -rn | head -4
     13 mem
      2 w
      2 r
      1 u
```

The `FD` column holds a number plus the access mode (`r`, `w`, `u` for both) or a special name: `cwd` the working directory, `rtd` the root, `txt` the program image, `mem` a memory-mapped file (13 libraries and archives here). The device `254,0` is the root disk; `1,3` is `/dev/null`, the same pair that appeared in the device chapter. `lsof -p PID` is the quick answer to what a process has open; `lsof -i :PORT` finds the process that owns a port; and `lsof +L1` finds files that are deleted but still open, which is a frequent cause of disk space that does not return.

jcmd: the view from inside the JVM

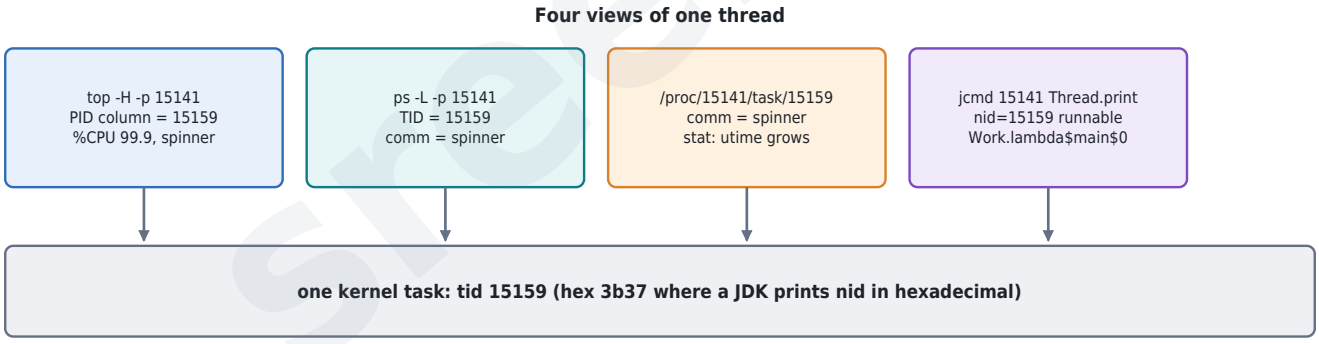
jcmd and jstat for the same process — TERMINAL

```
$ jcmd 15141 VM.uptime
15141:
7.732 s
$ jcmd 15141 GC.heap_info
15141:
 garbage-first heap   total 73728K, used 42350K [0x00000000f8000000, 0x0000000100000000)
  region size 1024K, 2 young (2048K), 1 survivors (1024K)
 Metaspace       used 355K, committed 512K, reserved 1114112K
  class space    used 26K, committed 128K, reserved 1048576K
$ jstat -gcutil 15141
  S0     S1     E      O      M     CCS    YGC     YGCT     FGC    FGCT     CGC    CGCT       GCT
     -  18.94   0.00  72.02  69.37  21.06      2     0.002     0     0.000     2     0.000     0.002
$ jcmd 15141 Thread.print | grep -A5 '"spinner"'
"spinner" #18 [15159] daemon prio=5 os_prio=0 cpu=7413.10ms elapsed=9.19s tid=0x00007f0c9c0de310
 nid=15159 runnable  [0x00007f0ca0d48000]
   java.lang.Thread.State: RUNNABLE
        at Work.lambda$main$0(Work.java:11)
        at Work$$Lambda/0x00007f0c28000a08.run(Unknown Source)
```

`jcmd` talks to the JVM through a local attach mechanism and works with the process identifier. `GC.heap_info` reports the committed and used heap (42 MiB used, 72 MiB committed, in G1 regions of 1 MiB) and the metaspace, which lies outside the heap. `jstat -gcutil` shows occupancy percentages of the heap spaces (`O` old generation 72 percent) and the counts and times of young and full collections. The thread dump joins the operating system and the JVM: `nid=15159` is the **native thread identifier**, the number that `top -H` printed as `PID` for the `spinner` row. The numbers match because a JVM thread is a kernel thread. The `cpu=7413.10ms` field gives the thread's total CPU time.

GOTCHA

Many JDK versions print `nid` in hexadecimal (`nid=0x3b37`), and the identifier from `top -H` must be converted with `printf '%x\n' 15159` before searching the dump. The JDK 21 build used here prints the identifier in decimal, so the decimal value from `top -H` is matched directly. Check the format of the dump before converting. The hexadecimal form of this thread is `3b37`.



a JVM thread is a kernel thread, so every tool shows the same number

Figure 1.31. One identifier connects four views of the same thread: `top -H`, `ps -L`, the `/proc/PID/task` directory and the `nid` field of the Java thread dump.

1.6.3 Hands-on

A USE-method snapshot from /proc

The script in this listing reads only `/proc` and prints one line per resource. It uses no external tools, which is the reason it works in a minimal container that has no `sysstat`.

use_check.py — PYTHON

```
#!/usr/bin/env python3
"""Minimal USE-method snapshot from /proc (utilisation, saturation, errors)."""
import time


def cpu():
    f = open("/proc/stat").readline().split()[1:]
    v = list(map(int, f))
```

use_check.py (continued) PYTHON

```
    return sum(v[:8]), v[3] + v[4]          # total, idle + iowait


def disk(dev):
    for line in open("/proc/diskstats"):
        p = line.split()
        if p[2] == dev:
            return int(p[12]), int(p[11])   # ms doing I/O, I/Os in flight


def net():
    rx = tx = 0
    for line in list(open("/proc/net/dev"))[2:]:
        name, data = line.split(":")
        if name.strip() == "lo":
            continue
        c = data.split()
        rx += int(c[2]) + int(c[3])
        tx += int(c[10]) + int(c[11])
    return rx + tx


def mem():
    m = {l.split(":")[0]: int(l.split()[1]) for l in open("/proc/meminfo")}
    return m["MemAvailable"] / m["MemTotal"] * 100, m["SwapTotal"] - m["SwapFree"]


def psi(name):
    return open("/proc/pressure/" + name).readline().split()[1].split("=")[1]


t0, i0 = cpu()
d0, _ = disk("vda")
n0 = net()
time.sleep(5)
t1, i1 = cpu()
d1, infl = disk("vda")
n1 = net()
avail, swap = mem()
runq = open("/proc/loadavg").read().split()[3].split("/")[0]
rows = [
    ("CPU", "%.1f%% busy" % (100.0 * (1 - (i1 - i0) / (t1 - t0))),
     "runq=%s, psi some=%s" % (runq, psi("cpu")), "none exported"),
    ("memory", "%.1f%% available" % avail, "swap used=%d kB, psi=%s" % (swap, psi("memory")),
     "OOM: dmesg"),
    ("disk vda", "%.1f%% busy" % ((d1 - d0) / 50.0), "in flight=%d, psi=%s" % (infl, psi("io")),
     "none exported"),
    ("network", "n/a (needs link speed)", "n/a", "drops+errors +%d" % (n1 - n0)),
]
print("%-9s %-24s %-30s %s" % ("resource", "utilisation", "saturation", "errors"))
for r in rows:
    print("%-9s %-24s %-30s %s" % r)
```

The snapshot while the busy Java thread runs (real output) TERMINAL

```
$ python3 use_check.py
resource  utilisation              saturation                    errors
CPU       99.9% busy               runq=3, psi some=29.58        none exported
memory    90.6% available          swap used=0 kB, psi=0.00      OOM: dmesg
disk vda  0.0% busy                in flight=0, psi=0.04         none exported
network   n/a (needs link speed)   n/a                           drops+errors +0
```

The row for the CPU shows both utilisation (99.9 percent of two processors busy, which includes other work on the shared machine) and saturation (three runnable tasks on two CPUs, and a pressure-stall figure of 29.58, the percentage of the last ten seconds in which at least one task was waiting for a CPU). The pressure stall information files `/proc/pressure/cpu`, `memory` and `io` are the most direct saturation measures in the kernel; they were enabled by the `psi=1` kernel parameter seen in the boot chapter. Memory and disk show no problem: 90.6 percent available, no swap, no disk activity.

sreram

Applying the checklist

The first three checks of the 60-second list on the same host — TERMINAL

```
$ uptime
 02:36:26 up 26 min,  0 user,  load average: 2.92, 2.30, 1.96
$ ps -eo pid,pcpu,pmem,nlwp,comm --sort=-pcpu | head -3
  PID %CPU %MEM NLWP COMMAND
15141 84.6  0.7   22 java
   95  8.5  5.0    9 claude
$ ps -eo stat= | cut -c1 | sort | uniq -c | sort -rn
     34 S
     34 I
      1 Z
      1 R
```

Reading the output: a load average of 2.92 on two processors, rising from 1.96 over fifteen minutes, indicates a recent increase of runnable work (the load average counts runnable and uninterruptible tasks; the scheduling part of this book treats it). The process list identifies one candidate, the JVM with PID 15141. The distribution of states (34 sleeping, 34 idle kernel threads `I`, one running `R`, one zombie `Z`) shows no accumulation of `D` tasks, so the machine is not blocked on I/O. The decision for the next step comes from the CPU split: all of the time is in `us` (user code), so the next tool is a thread-level view, `top -H`, followed by the thread dump.

1.6.4 Real-world incidents

Incident: an alert reports 100 percent CPU, but the CPU time is spent in the kernel

Scenario. A monitoring alert reports that a service instance has used a full core for ten minutes. A developer takes a thread dump and finds nothing unusual: the busy thread is in a harmless-looking method that polls a status value, and sampling the stack with a profiler shows only that method.

Symptoms. CPU utilisation of one core, no growth in heap, normal response times initially, and no throughput increase to explain the load.

Diagnosis. The first question is whether the time is spent in user code or in the kernel. The per-process counters `utime` and `stime` in `/proc/PID/stat` give the answer without any tracing. The reproduction is a program that calls a trivial system call in an endless loop, which models a polling loop that asks the kernel for something on every iteration.

sysburn.c — C

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/syscall.h>

int main(void) {
    printf("pid=%d\n", getpid());
    fflush(stdout);
    for (;;) syscall(SYS_getppid);
}
```

User against system time of one process, then which call it makes (real output) — TERMINAL

```
$ timeout 12 ./sysburn > sb.out & sleep 2; SB=$(grep -o '[0-9]*' sb.out | head -1)
$ awk '{printf "utime=%d stime=%d ticks (100 per second)\n", $14, $15}' /proc/$SB/stat
utime=93 stime=103 ticks (100 per second)
$ sleep 3; awk '{printf "utime=%d stime=%d ticks\n", $14, $15}' /proc/$SB/stat
utime=236 stime=256 ticks
$ strace -c -o sbc.txt -p $SB & ST=$!; sleep 3; kill -INT $ST; sleep 1; cat sbc.txt
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ------------------
100.00    0.414513           1    280404           getppid
------ ----------- ----------- --------- --------- ----------------
100.00    0.414513           1    280404           total
```

During the three seconds between the two readings, `utime` grew by 143 ticks and `stime` by 153, which is 296 ticks of 300 possible: the process used a full processor and about half of its time was accounted to the kernel. In a program that does real computation, `stime` is small. Under `strace`, the process made 280,404 `getppid` calls in three seconds. (The tracing slows the process by a large factor, as the system call chapter showed; the count demonstrates the identity of the call, not the production rate.) In the thread dump the thread would appear `RUNNABLE`, because the JVM cannot distinguish time inside a native call from time in compiled code.

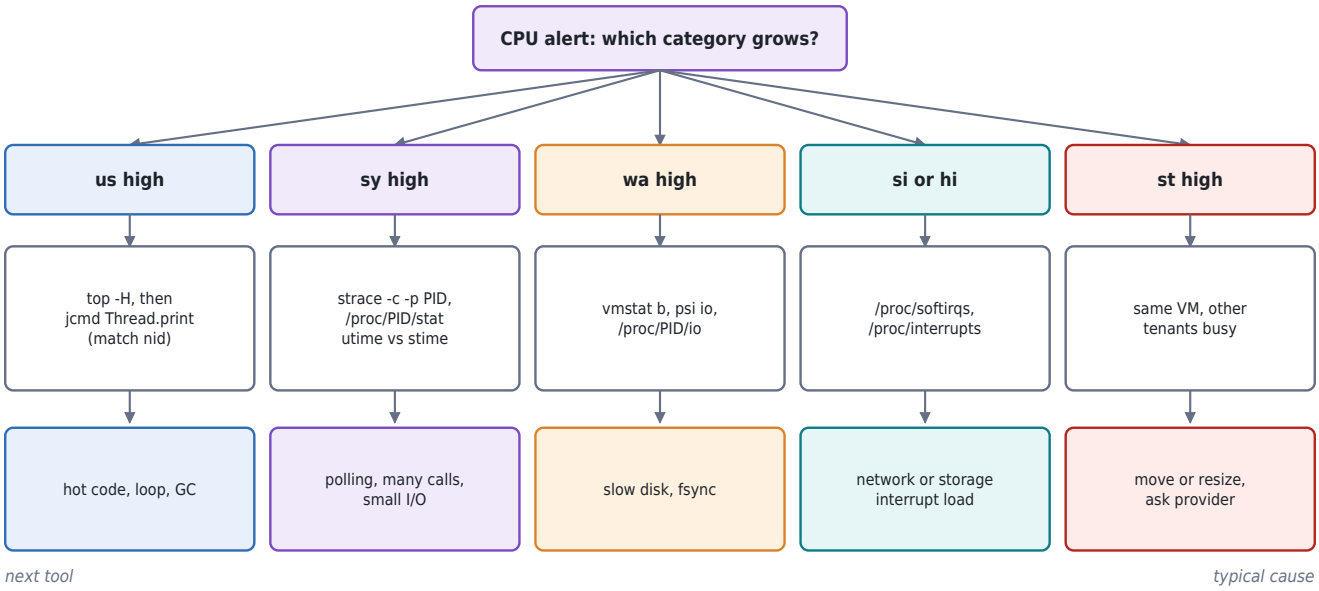


Figure 1.32. The split of CPU time selects the next tool: high `us` leads to thread dumps, high `sy` to `strace -c`, high `wa` to storage statistics, high `si` to network interrupts and high `st` to the hypervisor.

Root cause. A loop that issued a kernel call on every iteration, so that half of the time was spent entering and leaving ring 0. In real services the typical sources are polling with `System.currentTimeMillis` where the clock is not served by the vDSO, status checks with `File.exists` or `File.lastModified` in a tight loop, busy-wait loops with `Thread.yield` (a `sched_yield` call), and logging that flushes on every line.

Fix. Identify the call with `strace -c -f -p PID` for a few seconds (acceptable for a short, supervised interval) and the call site from the thread dump, then remove the polling: replace it with a blocking wait, a callback, a `WatchService` for file changes, or a cached value with a time-to-live. Mitigation until then is to limit the rate of the loop.

Prevention. Monitor the `us` and `sy` split of the process as separate series (`utime`, `stime` in `/proc/PID/stat`, or the equivalent of the container runtime). A rise of `sy` without a rise of throughput is a defect signal. Record the system call rate of a representative workload in performance tests.

Incident: the disk is busy and nobody knows which process is writing

Scenario. An alert reports that the write latency of a data disk has doubled. Several services share the host, and the disk graph shows a steady stream of small writes.

Symptoms. Higher latency for every service that writes to the disk; no obvious spike in application traffic.

Diagnosis. Device-level counters identify the disk but not the writer. The per-process counters in `/proc/PID/io` attribute bytes and calls to a process. The reproduction is a program that appends 64 KiB blocks to a file and forces each block to disk with `sync()` (an `fsync`), 400 times with a 20 ms pause between blocks.

Flusher.java JAVA

```java
import java.io.FileOutputStream;

public class Flusher {
    public static void main(String[] args) throws Exception {
        System.out.println("pid=" + ProcessHandle.current().pid());
        System.out.flush();
        byte[] block = new byte[64 * 1024];
        try (FileOutputStream out = new FileOutputStream("/tmp/osw_o1/flush.dat")) {
            for (int i = 0; i < 400; i++) {
                out.write(block);
                out.getFD().sync();
                Thread.sleep(20);
            }
        }
    }
}
```

The write counters of one process, two seconds apart, against the device counters (real output) TERMINAL

```
$ cat /proc/6090/io
rchar: 157490
wchar: 6160409
syscr: 90
syscw: 105
read_bytes: 0
write_bytes: 6201344
cancelled_write_bytes: 0
$ sleep 2; grep -E 'wchar|syscw|write_bytes' /proc/6090/io
wchar: 12451865
syscw: 201
write_bytes: 12492800
# /proc/diskstats for vda over the same two seconds, fields 8, 10, 13 (delta)
writes completed 288, sectors written 13056, ms doing I/O 48
```

In two seconds the process wrote 6,291,456 bytes (`wchar` and `write_bytes` both grew by exactly 6 MiB, which is 96 blocks of 64 KiB) and made 96 `write` calls. The device saw 288 write requests and 13,056 sectors (6.4 MiB), a few more than the application wrote because the file system adds journal and metadata writes, and because other tenants of the machine also write. The `rchar` and `wchar` counters count bytes passed through `read` and `write` calls, including those served by the page cache; `read_bytes` and `write_bytes` count bytes that were sent to or fetched from the block layer. A process with a large `wchar` and small `write_bytes` writes mostly to the cache or to files such as pipes, a process with large `write_bytes` is the disk consumer.

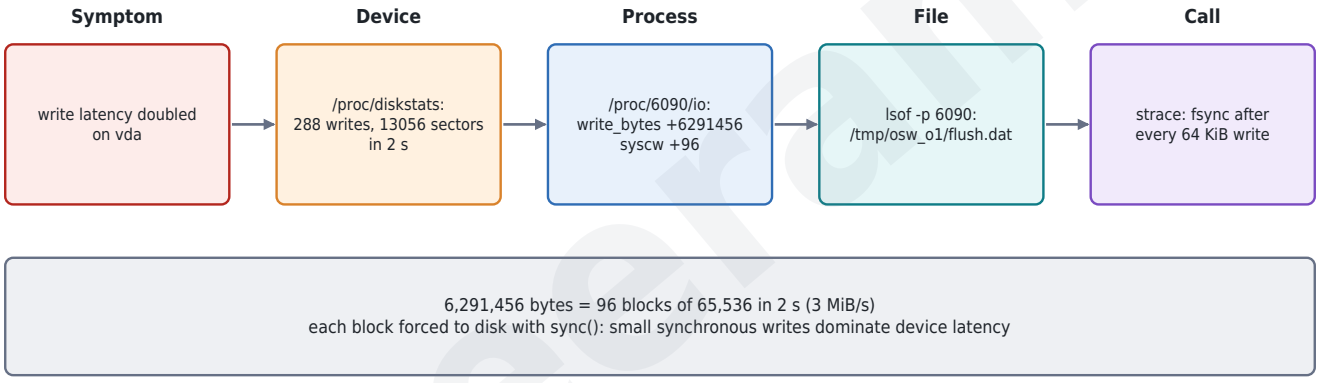


Figure 1.33. Attribution runs from the symptom at the device down to the process: device counters identify the disk, per-process counters identify the writer, and `strace` or `lsof` identify the file.

The loop to find the writer on a real host is to read `/proc/PID/io` for all processes, sort by `write_bytes` difference over an interval, and then use `lsof -p PID` to find the files. A short script that samples every process twice and prints the largest differences is sufficient. `iotop` presents the same data interactively but is not installed here.

Root cause. A process that forces small synchronous writes at a high rate. Each `fsync` makes the device flush its write cache and the file system commit its journal, which dominates the latency of every other writer on the same disk.

Fix. Mitigation: reduce the rate or the synchronisation of the offending writer (batch writes, `fdatasync` once per batch, group commit), or move it to a separate disk. Permanent: assign each write-heavy service its own volume, set I/O limits where the platform supports them (`io.max` in cgroup v2, `blkio` throttling in v1, volume IOPS in the orchestrator), and review whether each `sync` is needed for durability.

Prevention. Export per-process or per-container write bytes and `fsync` rates; alert on a change of the share of device writes per service. The storage part of this book measures the cost of `fsync` in detail.

sreram

1.6.5 Questions to expect

Questions and model answers

1. **State the USE method.** For every resource, check utilisation, saturation and errors. It gives a bounded checklist and prevents fixation on the first anomaly.
2. **Which `free` column shows how much memory is available to applications?** `available`. `free` omits reclaimable cache and is normally small on a healthy long-running host.
3. **What does the `r` column of `vmstat` mean?** The number of runnable threads, including those waiting for a CPU. Persistently more than the number of processors indicates CPU saturation.
4. **What do `us`, `sy`, `wa` and `st` mean in `top`?** Percentages of CPU time in user code, in the kernel, idle with I/O outstanding, and stolen by the hypervisor.
5. **How is a hot Java thread found from the operating system?** `top -H -p PID` identifies the thread identifier with the highest CPU; the same number is the `nid` in `jcmd PID Thread.print` (hexadecimal on older JDKs, decimal on the JDK 21 used here), which names the Java thread and its stack.
6. **Why is `VSZ` of a JVM much larger than its heap?** `VSZ` counts reserved address space (heap reservation, thread stacks, mapped libraries and archives, metaspace reservation). Only `RSS` counts pages in physical memory.
7. **What does `D` in the `STAT` column mean and why is it important?** Uninterruptible sleep, normally a task waiting for I/O. Such tasks cannot be killed until the I/O completes and they count towards the load average.
8. **How is the process responsible for disk writes identified?** `/proc/PID/io` (`write_bytes`) sampled twice for all processes, or `iotop` where installed; then `lsof -p PID` to find the files.
9. **A process shows 100 percent CPU. What is the first distinction to make?** Whether the time is user or system time (`utime` and `stime` in `/proc/PID/stat`, `us` and `sy` in `top`); the answer selects between thread dumps and `strace`.
10. **Why is `strace -c`'s `% time` column unreliable for diagnosing slowness?** It includes time blocked inside calls such as `futex` or `epoll_wait`, which is idle waiting. The call counts and error counts are the reliable columns.

KEY POINTS

A diagnosis starts by deciding which resource is limiting, using utilisation, saturation and errors, and ends with the process, thread and call responsible.

- `ps` shows state and lifetime averages, `top -H` shows current per-thread CPU, `vmstat 1` shows run queue, swapping and the CPU split, `free` shows `available` memory.
- The CPU split (`us`, `sy`, `wa`, `si`, `st`) selects the next tool: thread dump, `strace -c`, storage statistics, network interrupts or the hypervisor.
- The native thread identifier is the common key: `top -H`, `ps -L`, `/proc/PID/task` and the `nid` of the Java thread dump refer to the same number.
- `/proc/PID/io`, `/proc/PID/limits`, `/proc/PID/fd` and `/proc/pressure/*` answer who, what and how much without any tracing overhead.
- Treat figures with care: lifetime averages in `ps`, since-boot averages in the first `vmstat` line, and time-blocked counts in `strace -c` are not current work.

PART 2

Processes, Threads and Scheduling

How the kernel represents running programs, lets them communicate, divides the processors among them, and how each mechanism fails in production.

In this part

2.1 Processes
2.2 Signals, Process Groups and Daemons
2.3 Inter-Process Communication
2.4 Threads
2.5 CPU Scheduling
2.6 Scheduling in Linux
2.7 CPU Limits in Containers

2.1 Processes

A program stored on disk is inert: it is a file that contains machine code, constants and metadata. Running it requires memory, a processor, open files and an identity that the kernel can track, schedule and, if necessary, terminate. The operating system packages these resources into an abstraction called the process. Every Java service, every shell command and every container entrypoint is a process, and most operational failures of a deployed service (a service that does not stop, a defunct entry in the process table, a failed `fork`, an exit status of 137) are consequences of the rules governing process creation and termination. This chapter defines the process, shows how Linux represents and creates one, and reproduces the failure modes that appear in production.

2.1.1 Concept and mechanism

Program and process

A **program** is a passive description of a computation: an executable file in ELF format. A **process** is an executing instance of a program together with the resources that the kernel has allocated to it. One program can have many simultaneous processes; ten shells opened from `/bin/bash` are ten processes that share one program file.

> **DEFINITION**
> Process: an instance of a program in execution, consisting of a private virtual address space, one or more threads of execution, a table of open files, signal dispositions, credentials and accounting state, all identified by a process identifier (PID).

The kernel needs a data structure that holds everything it knows about a running program. In Linux this structure is `task_struct`, one per schedulable entity. A process with a single thread has one `task_struct`; a multithreaded process has one per thread, and the threads share most of the referenced objects (see the chapter on threads). The fields that matter for diagnosis are shown in the figure. Each pointer in `task_struct` leads to a separate object that can be shared: `fork` gives the child copies, whereas `clone` with sharing flags gives a thread the same objects.

sreeram

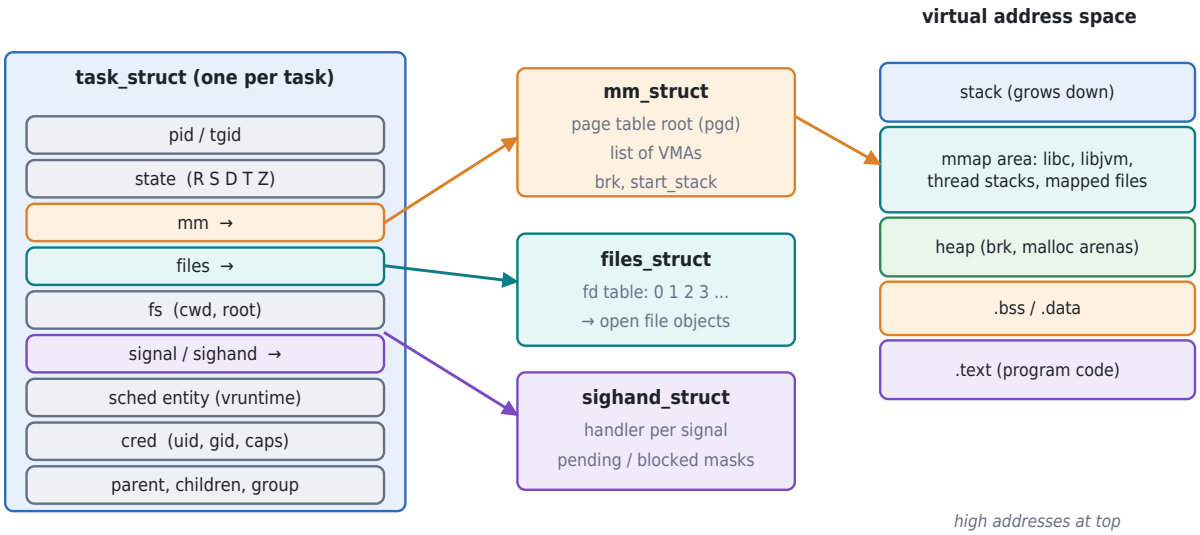


Figure 2.1. The task_struct is a hub of pointers: the address space (mm_struct), the open files (files_struct) and the signal handlers (sighand_struct) are separate objects, which is what allows threads to share them and fork to copy them.

The user-visible face of these structures is the `/proc` file system. The following listing shows the fields of the status file of an interactive shell, taken from the build machine.

Identity and accounting of one process (excerpt) TERMINAL

```
$ grep -E '^(Name|State|Tgid|Pid|PPid|Uid|Threads|VmRSS|voluntary|nonvoluntary)' /proc/$$/status
Name:   bash
State:  S (sleeping)
Tgid:   2905
Pid:    2905
PPid:   95
Uid:    0       0       0       0
VmRSS:      4080 kB
Threads:        1
voluntary_ctxt_switches:        8
nonvoluntary_ctxt_switches:     0
```

`Pid` is the identifier of the task, `Tgid` the identifier of the thread group (the number that user tools call the PID), `PPid` the parent. The four `Uid` values are the real, effective, saved and file-system user identifiers. The two context-switch counters are used in the chapter on scheduling.

Process states

A process is always in exactly one state, and the state letter printed by `ps`, `top` and `/proc/PID/status` is the first piece of evidence in most diagnoses.

Letter	Name	Meaning and typical cause
R	running or runnable	On a CPU, or on a run queue waiting for one. Many R processes with few CPUs means CPU saturation.
S	interruptible sleep	Waiting for an event (timer, socket data, lock); can be woken by a signal. The normal state of an idle service.
D	uninterruptible sleep	Waiting inside the kernel, usually for disk or network file system I/O; signals, including `SIGKILL`, are not acted on until the wait ends. Counts toward load average.
T	stopped	Stopped by `SIGSTOP`, `SIGTSTP` (Ctrl-Z) or a debugger. Resumed by `SIGCONT`.
Z	zombie	Terminated; only the process-table entry and exit status remain until the parent calls `wait`.
I	idle kernel thread	A kernel worker with nothing to do; not counted in load.

Linux process states as displayed by ps and top.

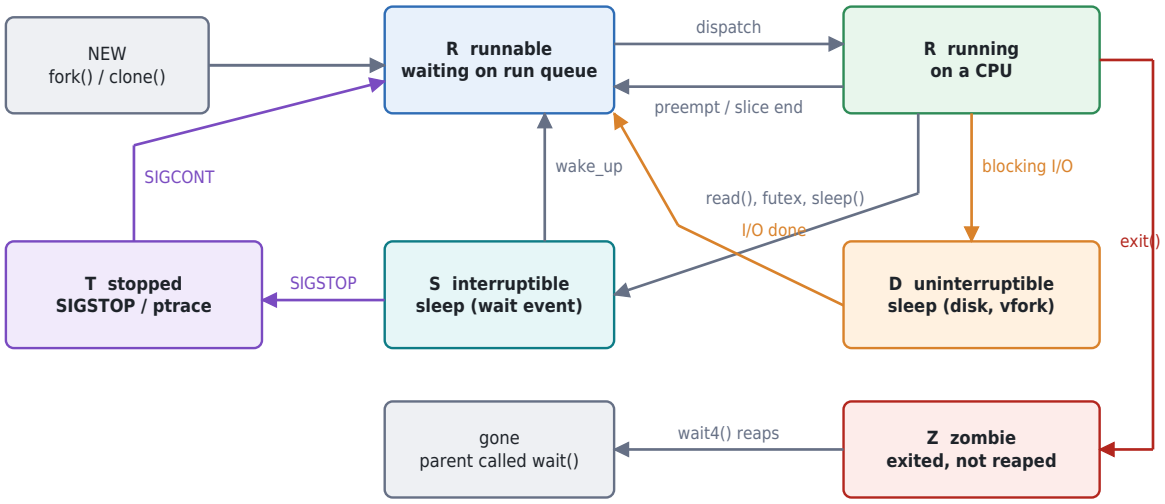


Figure 2.2. State transitions of a Linux task: only the scheduler moves a task between runnable and running, a wake-up moves S or D back to runnable, and a task that has exited stays in state Z until its parent reaps it.

The distinction between S and D is the one that matters operationally. A task in S can be interrupted: `kill` makes it runnable so that it can handle the signal or die. A task in D must complete a kernel operation that cannot be abandoned safely, such as waiting for a disk block, so no signal takes effect until the operation returns. A process stuck in D on an unresponsive network file system cannot be killed by any signal.

Creating processes: fork, exec and wait

Unix separates process creation into two operations, which is unusual compared with systems that offer a single "spawn a program" call.

- `fork()` (implemented by Linux as `clone` with the `SIGCHLD` flag) creates a child that is a copy of the caller: same code, same data, same open files, a new PID. It returns twice, with the child's PID in the parent and 0 in the child.
- `execve(path, argv, envp)` replaces the program image of the calling process with a new program. The PID, the open files (unless marked close-on-exec) and the parent are retained; the address space, the stack and the signal handlers are replaced.
- `wait4()` / `waitpid()` blocks the parent until a child terminates, then returns the child's exit status and releases its process-table entry.

The separation allows the child to configure itself between `fork` and `exec`: redirect file descriptors, change directory, drop privileges, set limits. A shell uses exactly this window to implement redirection and pipelines (see the chapter on inter-process communication).

sreeram

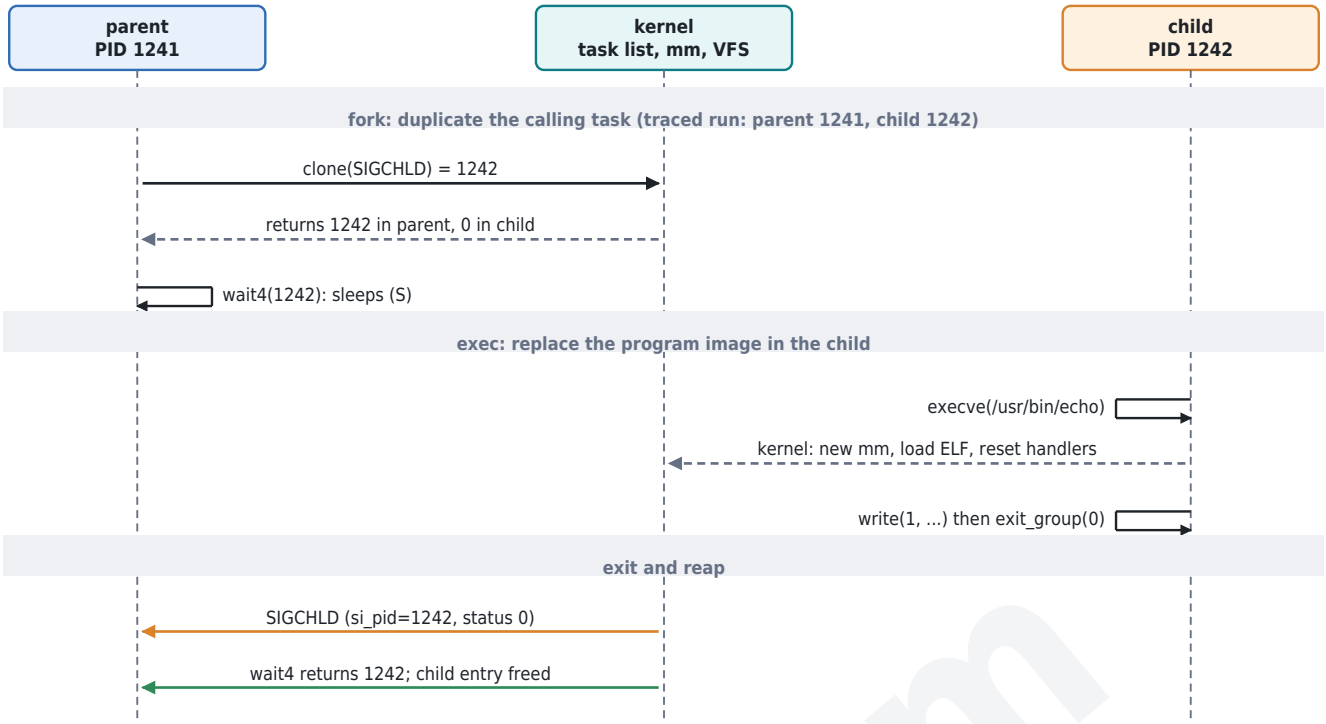


Figure 2.3. The sequence of one traced run: the parent blocks in wait4, the child replaces itself with /usr/bin/echo, and the kernel signals the parent with SIGCHLD when the child exits.

Copy-on-write

Copying the entire address space on every `fork` would be prohibitively expensive, particularly because the most frequent action after `fork` is `exec`, which discards the copy. Linux therefore applies **copy-on-write** (COW). At `fork` the kernel copies only the page tables, not the data pages, and marks every writable page read-only in both parent and child. Parent and child then reference the same physical frames. When either process writes to a shared page, the processor raises a page fault; the kernel allocates a new frame, copies the page, maps the copy writable into the writing process and resumes it. Pages that are never written stay shared for the life of both processes.

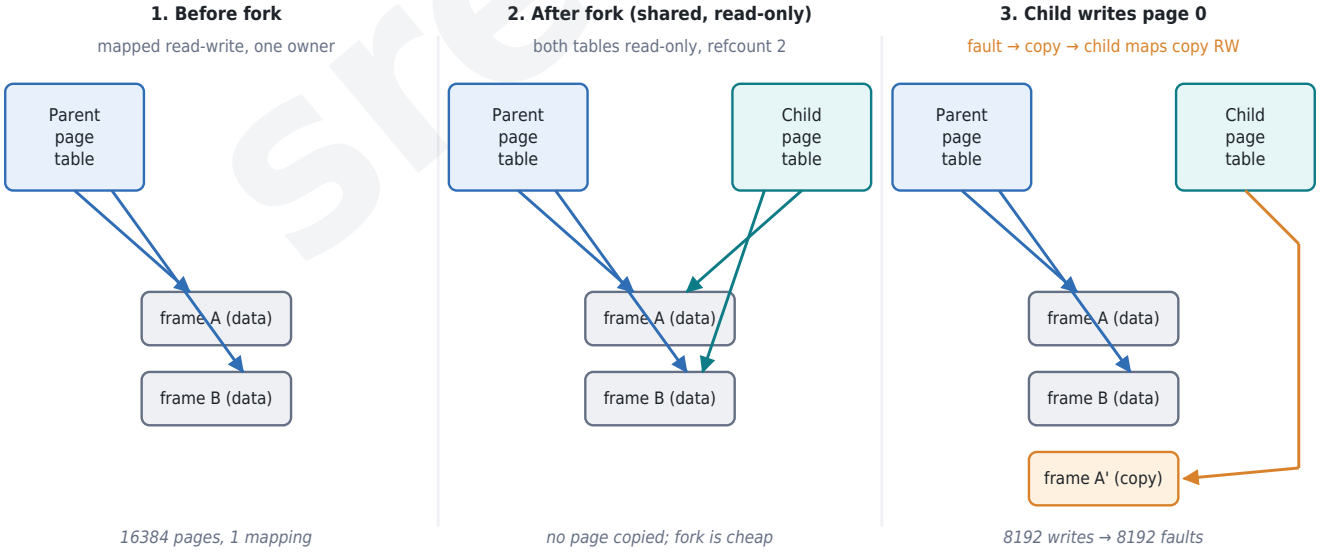


Figure 2.4. Fork shares frames read-only; a write by the child triggers a page fault whose handler allocates a private copy, so only written pages cost memory and time.

The cost of `fork` therefore depends on the number of page-table entries to copy, and the cost of a process that continues to run depends on how many pages it writes afterwards. A JVM that forks a large child to run `ls` would copy a large set of page tables; for that reason the JDK starts external commands through a small helper program, `jspawnhelper`, using `posix_spawn`, which avoids duplicating the large address space of the JVM.

Termination, zombies and orphans

When a process calls `exit`, or is killed, the kernel releases its memory, closes its files and delivers `SIGCHLD` to the parent. It cannot release everything, because the parent may later ask how the child ended. The kernel therefore keeps a minimal entry holding the PID and the exit status. A process in this condition is a **zombie** (state `Z`, shown by `ps` as `<defunct>`). The entry disappears when the parent calls `wait`; the parent is said to **reap** the child.

A zombie consumes no memory, CPU or open files, but it holds a PID, and PIDs are finite (`/proc/sys/kernel/pid_max` is 32768 on the build machine, and a container may also have a `pids.max` limit). A zombie cannot be killed, because it is already dead; the only remedies are to make the parent call `wait`, or to terminate the parent so that the zombies are adopted by another process that does.

An **orphan** is a running process whose parent has terminated. The kernel re-parents it to a **subreaper**: the nearest ancestor that has declared itself one with `prctl(PR_SET_CHILD_SUBREAPER)`, or otherwise PID 1 of the PID namespace. PID 1 must reap all adopted children that exit, and this is the reason why PID 1 inside a container needs to behave as an init process.

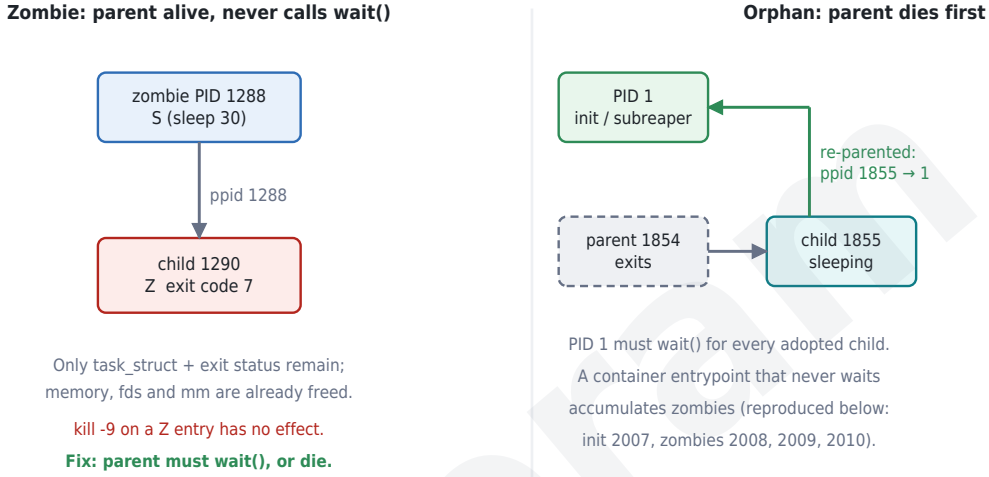


Figure 2.5. A zombie is a terminated child whose parent has not called wait; an orphan is a live child whose parent is gone and is adopted by PID 1. Both experiments are reproduced in the hands-on section.

2.1.2 Worked trace

The program below creates a child, lets the child modify a variable and run `/usr/bin/echo`, and waits for it. The trace that follows records the system calls of the same run.

forkexec.c C

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    int counter = 100;
    printf("parent: pid=%d ppid=%d\n", getpid(), getppid());
    fflush(stdout);
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); return 1; }
    if (pid == 0) {
        counter += 1;
        printf("child:  pid=%d ppid=%d counter=%d\n", getpid(), getppid(), counter);
        fflush(stdout);
        execl("/usr/bin/echo", "echo", "child now runs /usr/bin/echo", (char *)NULL);
        perror("execl");
        _exit(127);
    }
    int status;
    waitpid(pid, &status, 0);
    printf("parent: child %d exited, WIFEXITED=%d status=%d, counter=%d\n",
           pid, WIFEXITED(status), WEXITSTATUS(status), counter);
    return 0;
}
```

Run and trace forkexec.c (environment pointers shortened) TERMINAL

```
$ gcc -o forkexec forkexec.c
$ ./forkexec
parent: pid=1230 ppid=1213
child:  pid=1232 ppid=1230 counter=101
child now runs /usr/bin/echo
parent: child 1232 exited, WIFEXITED=1 status=0, counter=100
$ strace -f -e trace=clone,execve,wait4,exit_group ./forkexec > /dev/null
execve("./forkexec", ["./forkexec"], ...) = 0
clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD, ...) = 1242
strace: Process 1242 attached
[pid  1241] wait4(1242,  <unfinished ...>
[pid  1242] execve("/usr/bin/echo", ["echo", "child now runs /usr/bin/echo"], ...) = 0
[pid  1242] exit_group(0)               = ?
[pid  1242] +++ exited with 0 +++
<... wait4 resumed>[{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 1242
--- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=1242, si_uid=0, si_status=0, ...} ---
exit_group(0)                           = ?
+++ exited with 0 +++
```

The two runs have different PIDs because they are separate executions; within each run the numbers are consistent. The steps, using the PIDs of the traced run, are:

1. The shell executes `./forkexec` as PID 1241. `execve` loaded the program; no child exists yet.
2. PID 1241 calls `clone` with flags that mean "copy" (`CLONE_VM` is absent, therefore the address space is duplicated copy-on-write) and `SIGCHLD` as the termination signal. The kernel creates `task_struct` 1242 and returns 1242 to the parent and 0 to the child.
3. The parent calls `wait4(1242)` and sleeps (state S) because the child has not exited.
4. The child increments its own copy of `counter` to 101. The parent's copy remains 100, which the final line confirms: after `fork` the two address spaces are independent.
5. The child calls `execve`. The kernel discards the child's address space, maps `/usr/bin/echo`, and resets handlers. The PID remains 1242.
6. `echo` writes one line and calls `exit_group(0)`. The kernel frees the child's memory, keeps its exit status, sends `SIGCHLD` to 1241 and wakes the parent.
7. `wait4` returns 1242 with a status that `WIFEXITED` decodes as a normal exit with code 0. The kernel releases the child's process-table entry. The parent exits.

2.1.3 Hands-on

Observe every process state

The following session creates one process in each of the states R, S, T, D and Z and prints them in one `ps` call. The state D is obtained safely with `vfork`: the parent of a `vfork` is suspended in uninterruptible sleep until the child exits.

vf.c C

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/* Demonstration only: vfork() suspends the parent in state D until the child exits.
   usage: vf SECONDS */
int main(int argc, char **argv) {
    int secs = argc > 1 ? atoi(argv[1]) : 6;
    pid_t pid = vfork();
    if (pid == 0) { sleep(secs); _exit(0); }
    waitpid(pid, NULL, 0);
    return 0;
}
```

One process per state TERMINAL

```
$ (ulimit -t 8; exec ./spin) &                  # busy loop, bounded to 8 s of CPU
$ sleep 30 &                                    # S
$ sleep 30 & kill -STOP $!                      # T
$ ./vf &                                        # D (parent of vfork)
$ ./zombie > /dev/null &                        # Z child, S parent
$ ps -o pid,ppid,stat,wchan:22,comm -p 2688,2689,2690,2700,2701 --ppid 2701
```

One process per state (continued) — TERMINAL

```
  PID  PPID STAT WCHAN                COMMAND
 2688  2687 R    -                    spin
 2689  2687 S    hrtimer_nanosleep    sleep
 2690  2687 T    do_signal_stop       sleep
 2700  2687 D    kernel_clone         vf
 2701  2687 S    hrtimer_nanosleep    zombie
 2704  2701 Z    -                    zombie
```

The `WCHAN` column names the kernel function in which a sleeping task waits and is the quickest way to learn why a task is in S or D. Here it shows a timer wait for the sleepers, a signal stop for the stopped task, and `kernel_clone` for the vfork parent. (The PIDs in the command line are those of the recorded run; substitute the PIDs of the current session.)

Reproduce a zombie

zombie.c — C

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    pid_t pid = fork();
    if (pid == 0) {
        _exit(7);                /* child exits immediately */
    }
    printf("parent pid=%d child pid=%d (child exits, parent does not call wait)\n",
           getpid(), pid);
    fflush(stdout);
    sleep(30);                   /* parent idles without reaping */
    int status;
    waitpid(pid, &status, 0);    /* reap: the zombie disappears */
    printf("reaped: exit status %d\n", WEXITSTATUS(status));
    return 0;
}
```

The zombie exists for the 30 seconds before the parent calls waitpid — TERMINAL

```
$ ./zombie &
parent pid=1288 child pid=1290 (child exits, parent does not call wait)
$ ps -o pid,ppid,stat,comm --ppid 1288
  PID  PPID STAT COMMAND
 1290  1288 Z    zombie
$ grep -E 'State|PPid' /proc/1290/status
State:  Z (zombie)
PPid:   1288
reaped: exit status 7
```

The last line is printed 30 seconds later, when the parent finally calls `waitpid` and the entry disappears. Sending `SIGKILL` to PID 1290 during the wait has no effect because the target is already dead.

Reproduce an orphan and a subreaper

orphan.c — C

```c
#include <stdio.h>
#include <unistd.h>

int main(void) {
    pid_t pid = fork();
    if (pid == 0) {
        printf("child  pid=%d ppid(before)=%d\n", getpid(), getppid());
        fflush(stdout);
        sleep(2);                        /* parent exits meanwhile */
        printf("child  pid=%d ppid(after) =%d\n", getpid(), getppid());
        return 0;
    }
    sleep(1);
    printf("parent pid=%d exits now\n", getpid());
    return 0;
}
```

The orphan is adopted by PID 1 — TERMINAL

```
$ ./orphan
child  pid=1855 ppid(before)=1854
parent pid=1854 exits now
child  pid=1855 ppid(after) =1
```

On a desktop Linux system the adopting process is often a per-user `systemd` instance rather than PID 1, because that instance is registered as a subreaper; on the build machine PID 1 is the sandbox agent and adopts directly. A process can claim the subreaper role itself:

A subreaper adopts a grandchild whose parent has exited (subreaper.c uses prctl) — TERMINAL

```
$ ./subreaper
subreaper pid=1873
grandchild ppid=1873 (subreaper pid=?)
subreaper reaped adopted orphan pid=1875
```

The grandchild reports `ppid=1873`, the subreaper, instead of 1; the subreaper's own `wait` returned the grandchild's PID 1875. Container runtimes and process supervisors such as `tini` and `systemd` use this mechanism.

Measure copy-on-write

The program touches 64 MB, forks, and lets the child first read and then write half of the pages. The kernel's own accounting (`smaps_rollup`) and the minor-fault counter from `getrusage` show the copy-on-write effect.

cow.c — C

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/resource.h>
#include <sys/wait.h>

#define MB (1024 * 1024)
#define SIZE (64 * MB)

static long minflt(void) { struct rusage r; getrusage(RUSAGE_SELF, &r); return r.ru_minflt; }

static void show(const char *who) {
    char cmd[128];
    snprintf(cmd, sizeof cmd,
             "grep -E '^(Rss|Pss|Private_Dirty|Shared_Dirty):' /proc/%d/smaps_rollup"
             " | tr -s ' ' | tr '\\n' ' '", getpid());
    printf("%-22s", who); fflush(stdout);
    if (system(cmd) != 0) return;
    printf("\n");
}

int main(void) {
    char *buf = malloc(SIZE);
    memset(buf, 1, SIZE);                       /* parent touches all 16384 pages */
    show("parent before fork:");
    pid_t pid = fork();
    if (pid == 0) {
        long f0 = minflt();
        long sum = 0;
        for (long i = 0; i < SIZE; i += 4096) sum += buf[i];   /* read only */
        printf("child read  : %ld minor faults\n", minflt() - f0);
        f0 = minflt();
        for (long i = 0; i < SIZE / 2; i += 4096) buf[i] = 2;  /* write half the pages */
        printf("child write : %ld minor faults (8192 pages written)\n", minflt() - f0);
        show("child after writes:");
        printf("child sees buf[0]=%d, sum=%ld\n", buf[0], sum);
        _exit(0);
    }
    waitpid(pid, NULL, 0);
    printf("parent sees buf[0]=%d (unchanged)\n", buf[0]);
    return 0;
}
```

Output of cow.c TERMINAL

```
$ gcc -O1 -o cow cow.c && ./cow
parent before fork:   Rss: 67428 kB Pss: 65743 kB Shared_Dirty: 0 kB Private_Dirty: 65672 kB
child read  : 0 minor faults
child write : 8192 minor faults (8192 pages written)
child after writes:   Rss: 66756 kB Pss: 49283 kB Shared_Dirty: 32860 kB Private_Dirty: 32808 kB
parent sees buf[0]=1 (unchanged)
```

What to notice. Reading the shared pages caused no faults, because the page-table entries already existed and reads are allowed. Writing 8192 pages caused exactly 8192 minor faults, one per page, each followed by a 4 KiB copy. After the writes, the child's proportional set size (`Pss`) fell to 49283 kB: half of its pages (32808 kB, `Private_Dirty`) are now private copies, and the other half (32860 kB, `Shared_Dirty`) are still shared with the parent, so each shared page is charged half to each process. The parent is unaffected, as the last line shows.

2.1.4 Real-world incidents

Incident: zombie processes accumulate in a container until the node refuses new processes

Scenario. A containerised batch service starts as `CMD ["/bin/sh", "-c", "./run-job.sh"]`, and `run-job.sh` launches helper programs in the background and finishes by `exec`-ing a long-running monitor program. The monitor is written in a language whose runtime does not call `wait` for children it did not create. The pod has run for three weeks.

Symptoms. A node-level alert reports that the number of processes is approaching the limit. Inside the container `ps` lists hundreds of entries with state `Z` and the label `<defunct>`. New helper programs fail intermittently with `Resource temporarily unavailable`. CPU and memory use of the pod are normal.

Diagnosis. The first command counts the states in the container. The second shows who the parent of the zombies is; the third establishes that the parent is PID 1, which is the process that must reap adopted children. The reproduction below uses a PID namespace and a program that never calls `wait`.

noinit.c C

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

/* Acts as PID 1 in a new PID namespace and never calls wait(): children become zombies. */
int main(void) {
    printf("noinit: my pid is %d\n", getpid());
    for (int i = 0; i < 3; i++) {
        if (fork() == 0) _exit(0);
    }
    sleep(3);
    return 0;
}
```

Reproduction: the program is PID 1 of a new PID namespace and leaves its children unreaped TERMINAL

```
$ timeout 20 unshare --pid --fork --mount-proc ./noinit &
$ ps -eo pid,ppid,stat,comm | awk '$3 ~ /Z/ || $4=="noinit" || $4=="unshare"'
 2006  2004 S    unshare
 2007  2006 S    noinit
 2008  2007 Z    noinit
 2009  2007 Z    noinit
 2010  2007 Z    noinit
noinit: my pid is 1
```

The listing shows the host view of the namespace: PID 2007 is `noinit`, which prints "my pid is 1" because it is PID 1 inside the namespace. Its three exited children 2008 to 2010 are zombies whose `PPID` is 2007. In the incident the same pattern appears with the `PPID` column equal to 1 inside the container. In a real container the commands are `ps -eo stat= | sort | uniq -c` to count states and `ps -o pid,ppid,stat,comm` on the zombies to read the parent. The diagnosis is complete when the parent of every zombie is the container's PID 1 and PID 1 does not reap.

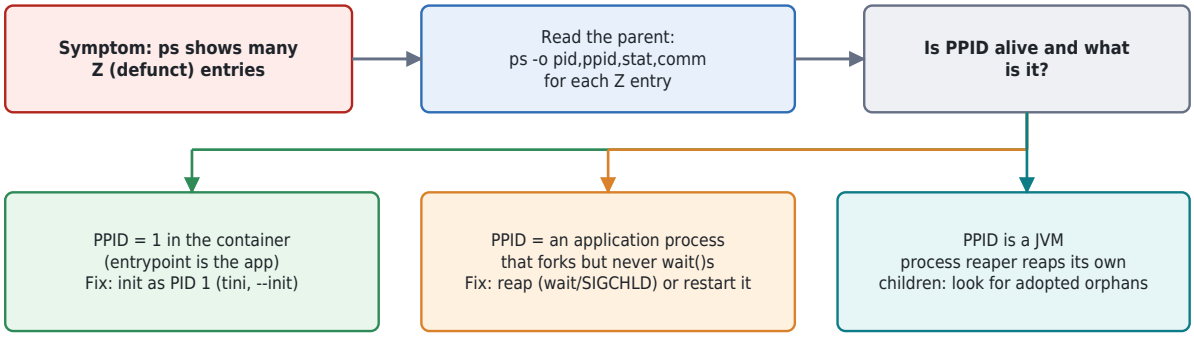


Figure 2.6. Diagnosis path for a growing number of Z entries: find the parent of the zombies; a live parent that does not wait is a defect in that program, and PID 1 that does not wait is a missing init.

Root cause. The container's PID 1 is not an init process. Orphaned descendants are re-parented to PID 1 (the PID namespace has no other subreaper), and PID 1 never calls `wait`, so every helper that ends leaves a zombie. Each zombie holds a PID and counts against the PID limit of the cgroup (`pids.max`) and of the host (`pid_max`).

Fix. Immediate: restarting the container destroys the PID namespace and with it all zombies, because the kernel reaps everything when PID 1 of a namespace exits. Permanent: run a minimal init as PID 1 that reaps adopted children and forwards signals. Docker provides `docker run --init` (which uses `tini`), Kubernetes pods may share a process namespace with a pause container that reaps (`shareProcessNamespace: true`), and an image can include `tini` explicitly:

Dockerfile fragment

```
ENTRYPOINT ["/usr/bin/tini", "--"]
CMD ["java", "-jar", "/app/app.jar"]
```

Where the program is a JVM, zombies of children started with `ProcessBuilder` do not occur, because the JVM runs a "process reaper" thread that calls `wait4` for every child it started; the problem arises for orphans adopted from other programs, and for non-Java entrypoints.

Prevention. Alert on the count of zombies (`node_processes_state{state="Z"}` in node exporters, or `ps -eo stat | grep -c Z`) and on the ratio of `pids.current` to `pids.max`. Require an init in the base image policy. Review any shell entrypoint that starts background jobs.

Incident: "Resource temporarily unavailable" when a service forks or creates threads

Scenario. A Spring Boot service runs a report generator that starts one operating-system process per request. After a traffic increase the service logs `java.io.IOException: error=11, Resource temporarily unavailable` and the container's health check, which also spawns a `curl`, begins to fail.

Symptoms. Intermittent failures of any operation that creates a process or thread. Memory and CPU are normal. The error number 11 is `EAGAIN`.

Diagnosis. `EAGAIN` from `fork` or `clone` means that a process-count limit was hit. Three limits exist: the per-user limit `RLIMIT_NPROC` (`ulimit -u`), the system limit (`kernel.pid_max`, `kernel.threads-max`), and the `pids` cgroup limit applied by container runtimes and Kubernetes (`podPidsLimit`). The commands below read the cgroup limit and counter, which is the most frequent source in containers.

forkloop.c C

```c
#include <errno.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

/* Bounded fork loop: tries to create up to 100 children that sleep for 3 seconds. */
int main(void) {
    int ok = 0;
    for (int i = 0; i < 100; i++) {
```

forkloop.c (continued) C

```c
        pid_t pid = fork();
        if (pid == 0) { sleep(3); _exit(0); }
        if (pid < 0) {
            printf("fork #%d failed: errno=%d (%s)\n", i + 1, errno, strerror(errno));
            break;
        }
        ok++;
    }
    printf("children created: %d\n", ok);
    while (wait(NULL) > 0) { }
    return 0;
}
```

Reproduction with a pids cgroup limit of 20 (cgroup v1 layout of the build machine) TERMINAL

```
$ mkdir /sys/fs/cgroup/pids/osw_o2 && echo 20 > /sys/fs/cgroup/pids/osw_o2/pids.max
$ ( echo $BASHPID > /sys/fs/cgroup/pids/osw_o2/cgroup.procs; exec ./forkloop )
fork #20 failed: errno=11 (Resource temporarily unavailable)
children created: 19
$ cat /sys/fs/cgroup/pids/osw_o2/pids.events
max 1
$ setpriv --reuid=65534 --regid=65534 --clear-groups bash -c 'ulimit -u 15; ./forkloop'
fork #15 failed: errno=11 (Resource temporarily unavailable)
children created: 14
```

The limit of 20 counts the forking process itself, so 19 children fit and the twentieth `fork` fails. The counter `max 1` in `pids.events` records the number of times the limit rejected a creation; on cgroup v2 the file is `pids.events` as well and `pids.max` and `pids.current` have the same meaning. The second experiment shows the per-user limit with a non-root identity: `RLIMIT_NPROC` is not enforced for root, so the experiment must switch user. In a pod the diagnosis is `cat /sys/fs/cgroup/pids.current /sys/fs/cgroup/pids.max` (v2) or the `pids` controller path (v1), followed by `ps -eLf | wc -l` to count tasks, because threads count as well as processes.

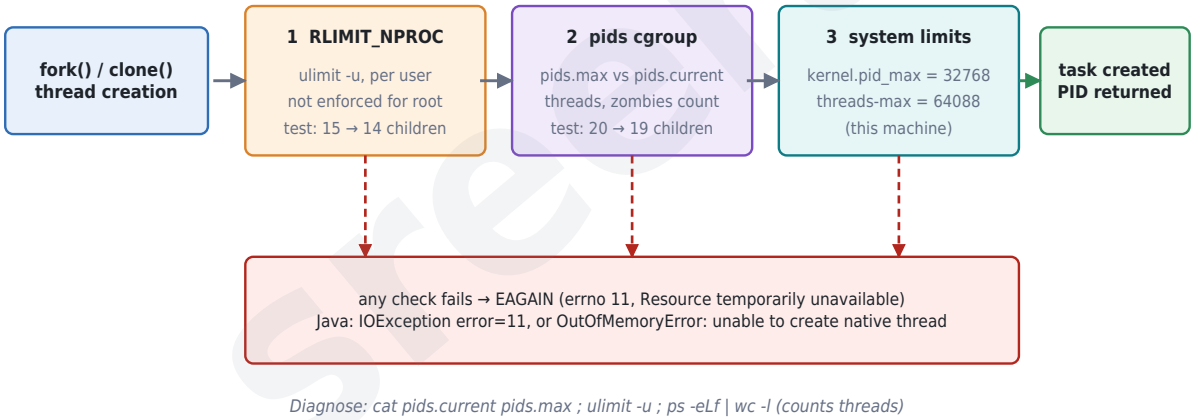


Figure 2.7. Every fork or clone is checked against three limits in turn; the pids controller counts threads and zombies, and the failing call returns EAGAIN, which Java reports as error=11 or as OutOfMemoryError "unable to create native thread".

Root cause. The pod's `pids.max` (applied by the platform, for example through the kubelet setting `podPidsLimit`) was exhausted by one process per request combined with long-running requests and un-reaped children.

Fix. Immediate: restart the pod to clear the tasks, and reduce concurrency at the load balancer. Permanent: do not start one process per request; bound concurrency with a fixed-size executor, reuse a long-lived helper process, or call a library instead of a child program. If more tasks are legitimately required, raise the platform limit deliberately (`podPidsLimit` in the kubelet configuration, `--pids-limit` in Docker) and size it from measured `pids.current`.

Prevention. Export `pids.current` against `pids.max` as a saturation metric (the USE method of the earlier part) and alert at 80 percent. Add a test that runs the service at twice the expected concurrency under the production limit.

2.1.5 Questions to expect

Questions and model answers

1. **What is the difference between a program and a process?** A program is a file with code and data; a process is a running instance with its own address space, file descriptors, credentials and at least one thread. One program can have many processes.
2. **What happens in the kernel during `fork`?** The kernel allocates a new `task_struct` and PID, copies the parent's page tables with all writable pages marked read-only (copy-on-write), duplicates the file-descriptor table and signal handlers, and returns 0 in the child and the child's PID in the parent.
3. **Why are `fork` and `exec` separate calls?** The child can change its environment (descriptors, directory, limits, credentials) between the two before the new program starts. Shells implement redirection and pipes with this window.
4. **What is copy-on-write and what does it save?** Pages are shared read-only after `fork`; a write faults, and the kernel copies only that page. It makes `fork` cheap and avoids copying pages that `exec` would discard.
5. **What is a zombie process, can it be killed, and how is it removed?** A terminated process whose exit status has not been collected. It uses no memory but a PID. It cannot be killed because it is dead; the parent must call `wait`, or the parent must exit so that init adopts and reaps it.
6. **What is an orphan process?** A live process whose parent has exited. The kernel re-parents it to the nearest subreaper or to PID 1 of its PID namespace.
7. **What is the difference between process states S and D?** S is an interruptible sleep that signals can end; D is an uninterruptible kernel wait, usually for I/O, that ignores signals until it completes and that counts toward the load average.
8. **Why does PID 1 in a container need special care?** It adopts all orphans and must reap them, and the kernel does not apply default signal actions to it. An application that is PID 1 and does neither accumulates zombies and may ignore `SIGTERM`. An init such as `tini` solves both.
9. **A call fails with `EAGAIN` on `fork`. What are the possible causes?** The per-user `RLIMIT_NPROC`, the system limit `pid_max` or `threads-max`, or a `pids` cgroup limit. In containers the cgroup limit is the most likely, and threads and zombies count toward it.
10. **How does Java start an external command without copying the JVM's address space?** On Linux, `ProcessBuilder` launches the helper `jspawnhelper` through `posix_spawn`, and a "process reaper" thread waits for the child, so the JVM does not leave zombies for children it started.

KEY POINTS

A process is an address space plus kernel-tracked resources, represented by a `task_struct` and observed through `/proc/PID`.

- `fork` copies page tables and marks pages copy-on-write; `exec` replaces the program image; `wait` collects the exit status and frees the entry.
- The state letters R, S, D, T and Z identify what a task is waiting for; `WCHAN` names the kernel function.
- A zombie is an exited child that has not been reaped; it holds a PID and is removed only by its parent's `wait` or by the parent's death.
- PID 1 of a PID namespace adopts orphans and must reap them; containers that run an application as PID 1 need an init such as `tini`.
- `EAGAIN` from `fork` or thread creation points to `RLIMIT_NPROC`, `pid_max` or the `pids` cgroup; threads and zombies count.

Preview

2.2 Signals, Process Groups and Daemons

A process cannot poll the kernel continuously to learn that the user pressed Ctrl-C, that a child has exited, or that an operator wants a service to stop. Unix solves this with the **signal**: a small, numbered, asynchronous notification that the kernel delivers to a process. Signals are the mechanism behind `kill`, behind the exit status 143 reported for a service that was stopped by a deployment tool, behind `SIGSEGV` crashes and behind the graceful shutdown of a Spring Boot application. This chapter defines the delivery model, the signals that matter for operations, the grouping of processes into process groups and sessions, and the way a daemon detaches itself from a terminal.

2.2.1 Concept and mechanism

Signal sources and dispositions

A signal originates from one of four sources: another process (`kill(pid, sig)`), the kernel in response to a hardware fault (`SIGSEGV` for an invalid memory access, `SIGFPE` for division by zero), the terminal driver in response to a key (`SIGINT` for Ctrl-C, `SIGTSTP` for Ctrl-Z), or the kernel in response to an event (`SIGCHLD` when a child changes state, `SIGPIPE` when a process writes to a pipe with no reader, `SIGALRM` when a timer expires).

For each signal, a process has a **disposition**, chosen from three possibilities.

- **Default action.** Terminate the process, terminate and write a core dump, ignore, stop, or continue, depending on the signal.
- **Ignore.** The kernel discards the signal.
- **Handler.** A function that the process registered runs when the signal is delivered.

Two signals cannot be caught, blocked or ignored: `SIGKILL`, which terminates the process, and `SIGSTOP`, which stops it. This guarantees that an administrator always retains a way to end a process. The call `sigaction(SIGKILL, ...)` fails with `EINVAL`, as the hands-on section shows.

Signal	No.	Default action	Typical use
SIGHUP	1	terminate	Terminal closed; many daemons re-read configuration on it.
SIGINT	2	terminate	Ctrl-C.
SIGQUIT	3	core dump	Ctrl-backslash; the JVM prints a thread dump and continues.
SIGABRT	6	core dump	`abort()`, failed assertion, `free()` corruption detected.
SIGKILL	9	terminate	Cannot be caught. Sent by `kill -9`, by the OOM killer and by the container runtime after the grace period.
SIGSEGV	11	core dump	Invalid memory access. The JVM uses it internally for null checks and safepoints.
SIGPIPE	13	terminate	Write to a closed pipe or socket.
SIGTERM	15	terminate	Polite request to stop; the default of `kill`; sent by `systemctl stop` and by Kubernetes.
SIGCHLD	17	ignore	A child exited or stopped; parents use it to call `wait`.
SIGSTOP	19	stop	Cannot be caught; used by debuggers and `kill -STOP`.
SIGCONT	18	continue	Resume a stopped process.

Signals that matter in operations (x86-64 Linux numbering).

Delivery and the pending mask

Signals are delivered by the kernel at a well-defined moment: when the target task is about to return from kernel mode to user mode (after a system call, an interrupt or a page fault). Until then the signal is **pending**, recorded as a bit in the task's pending set. A process can **block** a signal with a mask; blocked signals stay pending until they are unblocked. Standard signals (numbers 1 to 31) do not queue: if the same signal arrives three times while pending, the process sees it once. Real-time signals (34 to 64) queue and carry a value. The consequence for programs is that a handler for `SIGCHLD` must loop on `waitpid(..., WNOHANG)` until it returns 0, because two children that exit together can produce one signal.

sreeram

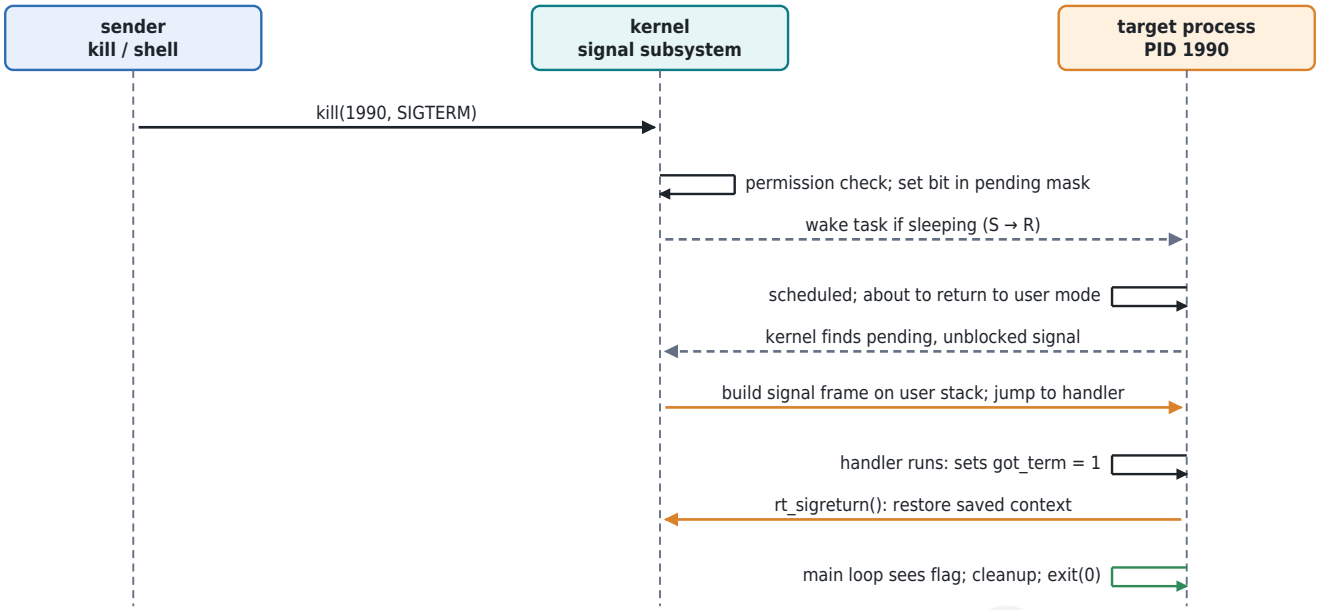


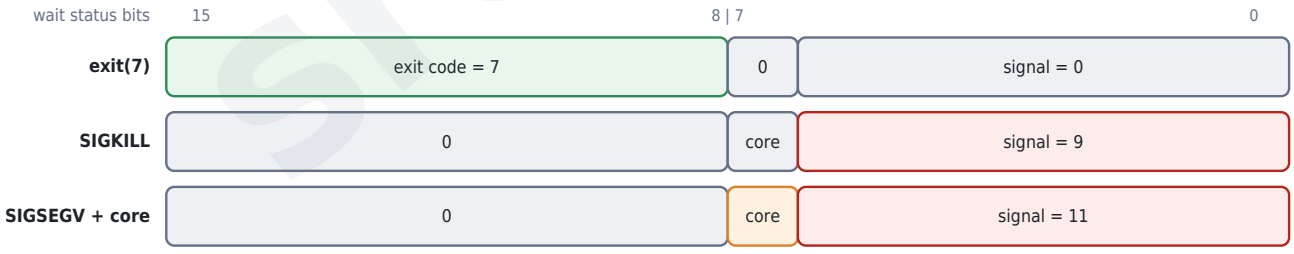
Figure 2.8. Delivery of SIGTERM: the signal is recorded as pending, the target is woken, and the handler runs on the target's own stack when it next returns to user mode; rt_sigreturn then restores the interrupted context.

A handler interrupts the program at an arbitrary instruction, possibly inside `malloc` or `printf`. For that reason a handler may call only **async-signal-safe** functions (`write`, `_exit`, `kill`, `waitpid` and a short list of others) and should do the least possible: set a flag of type `volatile sig_atomic_t` and return, leaving the work to the main loop. The same rule applies to the JVM: the JVM registers its own native handlers and turns a `SIGTERM` into a call to the registered shutdown hooks on a separate Java thread, so that Java code never runs inside a signal handler.

A blocking system call that is interrupted by a handled signal returns `EINTR` unless the handler was installed with `SA_RESTART`; portable code retries on `EINTR`.

Termination status

When a process ends, the kernel stores a 16-bit status that the parent obtains from `wait`. A normal exit places the exit code in the upper byte. A death by signal places the signal number in the lower seven bits and sets bit 7 if a core dump was written. Shells and container runtimes combine this into one number: exit codes 0 to 255 stand for themselves, and a death by signal `n` is reported as 128 plus `n`.



Shell and container runtimes report a signal death as 128 + signal number:

SIGKILL → 137 (OOM kill, grace period expired) **SIGSEGV → 139** **SIGTERM → 143** **SIGHUP → 129**

WIFEXITED / WEXITSTATUS and WIFSIGNALED / WTERMSIG in C decode these bit fields.

Figure 2.9. The 16-bit wait status distinguishes a normal exit from a death by signal; the number 137 seen in container logs is 128 plus 9, so it means the process was killed with SIGKILL and gives no information about who sent it.

The value 137 has several possible senders: the kernel OOM killer, the container runtime after a grace period, or an operator. The chapter on memory limits shows how to tell them apart; the incident in this chapter covers the grace-period case.

Process groups, sessions and daemons

Interactive use requires that a pipeline such as `a | b | c` be controlled as one unit. Linux organises processes in two levels above the process. A **process group** is a set of processes that can be signalled together; it is identified by the PID of its leader. A **session** is a set of process groups, normally created by a login or terminal and identified by the PID of the session leader. A session may have one **controlling terminal**, and exactly one process group of the session is the foreground group of that terminal. The terminal driver sends `SIGINT` (Ctrl-C) and `SIGTSTP` (Ctrl-Z) to the foreground group, and sends `SIGHUP` to the session leader when the terminal closes. `kill(-PGID, sig)` signals every member of a group.

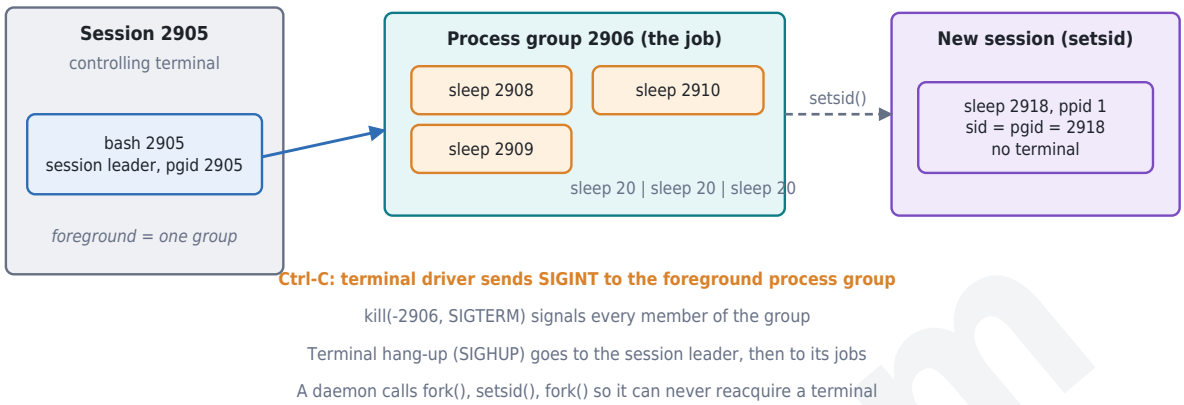


Figure 2.10. A shell is the leader of session 2905; each pipeline it starts forms a process group (here 2906); setsid creates a new session without a terminal, which is the first step in turning a program into a daemon.

A **daemon** is a long-running background process with no controlling terminal. The classic procedure has five steps: `fork` and let the parent exit (so the shell regains control and the child is not a group leader), `setsid` to leave the session and the terminal, `fork` again so that the daemon is not a session leader and can never acquire a terminal, set the working directory to `/` and the `umask`, and redirect descriptors 0, 1 and 2 to `/dev/null`. Under `systemd` none of this is needed: the service manager provides a clean environment, so a modern service runs in the foreground and logs to standard output, and the Java application is started as `ExecStart=java -jar app.jar` (see the chapter on services in the last part).

2.2.2 Worked trace

The program `sigdemo.c` installs handlers for `SIGTERM`, `SIGINT` and `SIGCHLD`, tries to catch `SIGKILL`, creates a child that exits after 300 ms, and waits until it has seen `SIGTERM`.

sigdemo.c C

```c
#define _GNU_SOURCE
#include <signal.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

static volatile sig_atomic_t got_term = 0, got_int = 0, got_chld = 0;

static void on_signal(int sig) {
    if (sig == SIGTERM) got_term = 1;
    else if (sig == SIGINT) got_int = 1;
    else if (sig == SIGCHLD) got_chld = 1;
}

int main(void) {
    struct sigaction sa;
    memset(&sa, 0, sizeof sa);
    sa.sa_handler = on_signal;
    sigemptyset(&sa.sa_mask);
    sigaction(SIGTERM, &sa, NULL);
    sigaction(SIGINT, &sa, NULL);
    sigaction(SIGCHLD, &sa, NULL);
    /* SIGKILL and SIGSTOP cannot be caught: sigaction fails with EINVAL */
```

sigdemo.c (continued) C

```c
    if (sigaction(SIGKILL, &sa, NULL) == -1) perror("sigaction(SIGKILL)");

    printf("pid=%d waiting for signals\n", getpid());
    fflush(stdout);
    if (fork() == 0) { usleep(300000); _exit(0); }   /* produces SIGCHLD */
    while (!got_term) pause();
    if (got_chld) {
        int st; pid_t p = waitpid(-1, &st, WNOHANG);
        printf("SIGCHLD seen, reaped %d\n", p);
    }
    if (got_int) printf("SIGINT seen\n");
    printf("SIGTERM seen: cleaning up and exiting with status 0\n");
    return 0;
}
```

Send SIGINT and then SIGTERM from outside TERMINAL

```
$ gcc -Wall -o sigdemo sigdemo.c
$ ./sigdemo & P=$!; sleep 1; kill -INT $P; sleep 0.2; kill -TERM $P; wait $P; echo "exit=$?"
sigaction(SIGKILL): Invalid argument
pid=2118 waiting for signals
SIGCHLD seen, reaped 2120
SIGINT seen
SIGTERM seen: cleaning up and exiting with status 0
exit=0
$ grep SigCgt /proc/2118/status          # while the program was running
SigCgt: 0000000000014002
```

What to notice. The first line is the failed attempt to catch `SIGKILL`. The `SIGCHLD` message appears first because the child exited 300 ms after start, before the two signals were sent. The mask `SigCgt` in `/proc/PID/status` lists the caught signals as a bit set in which bit `n-1` stands for signal `n`: `0x14002` has bits 1, 14 and 16 set, which are signals 2 (`SIGINT`), 15 (`SIGTERM`) and 17 (`SIGCHLD`). `SigIgn`, `SigBlk` and `SigPnd` show the ignored, blocked and pending sets in the same format, which makes the file a quick way to learn whether a stuck process has a handler for `SIGTERM`.

The same mask answers a frequent question about containers. The steps of a delivery are:

1. `kill -TERM 2118` enters the kernel; the kernel checks that the sender may signal the target (same user, or root).
2. The kernel sets bit 14 in the pending set of the target task and, since the target sleeps in `pause`, makes it runnable.
3. The scheduler runs the target. On the return to user mode the kernel finds a pending, unblocked signal with a handler.
4. The kernel builds a signal frame on the user stack, which saves the registers, and jumps to `on_signal`.
5. The handler sets `got_term` and returns into a small trampoline that calls `rt_sigreturn`; the kernel restores the saved registers.
6. `pause` returns with `EINTR`; the loop sees the flag and the program exits with status 0.

2.2.3 Hands-on

Exit statuses of signalled processes

A shell reports 128 plus the signal number TERMINAL

```
$ sleep 100 & P=$!; sleep 0.2; kill -TERM $P; wait $P; echo "TERM exit=$?"
TERM exit=143
$ sleep 100 & P=$!; sleep 0.2; kill -KILL $P; wait $P; echo "KILL exit=$?"
KILL exit=137
$ gcc -o segv segv.c && ./segv; echo "segv exit=$?"       # segv.c dereferences a null pointer
Segmentation fault
segv exit=139
$ sleep 30 & P=$!; kill -HUP $P; wait $P; echo "HUP exit=$?"
HUP exit=129
```

The JVM follows the same rule: a Spring Boot application stopped with `SIGTERM` runs its shutdown hooks and exits with status 143, which is the success status of a graceful stop under the shell convention; the build machine confirmed it with a small Java program with a shutdown hook (`java exit=143`).

Process groups and sessions

Pipeline members share a process group; setsid starts a new session — TERMINAL

```
$ (sleep 20 | sleep 20 | sleep 20) &
$ setsid sleep 20 &
$ ps -eo pid,ppid,pgid,sid,comm | grep sleep
 2908  2906  2906  2905 sleep
 2909  2906  2906  2905 sleep
 2910  2906  2906  2905 sleep
 2918     1  2918  2918 sleep
```

The three pipeline members have group 2906 and share session 2905 with their shell. The process started by `setsid` is its own group leader and session leader (`pgid = sid = 2918`), and since the intermediate process exited immediately its parent is now PID 1. `kill -TERM -2906` (note the minus sign) would signal all members of the first group.

A daemon, step by step

daemonize.c — C

```c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>

/* Classic double-fork daemonization. Sleeps 20 s so that ps can inspect it. */
int main(void) {
    if (fork() > 0) exit(0);          /* 1st fork: parent returns to the shell */
    setsid();                         /* new session, no controlling terminal */
    if (fork() > 0) exit(0);          /* 2nd fork: the daemon is not a session leader */
    chdir("/");
    umask(022);
    int fd = open("/dev/null", O_RDWR);
    dup2(fd, 0); dup2(fd, 1); dup2(fd, 2);
    sleep(20);
    return 0;
}
```

The result: parent PID 1, no terminal, own session, descriptors on /dev/null — TERMINAL

```
$ ./daemonize; sleep 2
$ ps -o pid,ppid,pgid,sid,tty,stat,comm -C daemonize
  PID  PPID  PGID   SID TT       STAT COMMAND
14442     1 14440 14440 ?        S    daemonize
$ ls -l /proc/14442/cwd /proc/14442/fd
lrwxrwxrwx 1 root root 0 Oct 18 02:33 /proc/14442/cwd → /
lrwx------ 1 root root 64 Oct 18 02:33 0 → /dev/null
lrwx------ 1 root root 64 Oct 18 02:33 1 → /dev/null
```

The `?` in the `TT` column means that no controlling terminal exists. The daemon's group and session (14440) belong to the intermediate process that has already exited, which proves that the second `fork` left the daemon as an ordinary member of a session without a leader.

2.2.4 Real-world incidents

Incident: a Spring Boot pod needs the full grace period to stop and is killed with status 137

Scenario. A Spring Boot service is deployed in Kubernetes with an image whose entrypoint is a wrapper script `start.sh` that exports environment variables and then runs `java -jar app.jar` as an ordinary command. The application registers shutdown hooks that finish in-flight requests and close a connection pool. After a change of the readiness behaviour the team observes that rolling updates take 30 seconds per pod, and the connection pool of the database reports abandoned connections after each deployment.

Symptoms. `kubectl describe pod` shows the last state as `Terminated`, reason `Error`, exit code 137. The application log lacks the line that the shutdown hook normally prints. Each pod takes exactly `terminationGracePeriodSeconds` (30 s by default) to disappear. The database logs show client connections that ended without a protocol close.

Diagnosis. Exit code 137 is 128 plus 9: the process ended by `SIGKILL`. A pod that was killed by the OOM killer also shows 137, so the first check is whether the termination coincided with the pod deletion and whether memory use was low (see the chapter on the OOM killer). When it did, the question becomes why `SIGTERM` did not stop the service. In a container the main process is PID 1 of its PID namespace, and the kernel applies special rules to that process: a signal

for which PID 1 has no handler is discarded, so that PID 1 cannot be killed accidentally by a signal from within the namespace. Signals sent from the parent namespace, the way a container runtime sends them, are also ignored when the target PID 1 has no handler, except for `SIGKILL` and `SIGSTOP`. The reproduction uses a PID namespace and a JVM with a shutdown hook.

Svc.java JAVA

```java
public class Svc {
    public static void main(String[] args) throws Exception {
        Runtime.getRuntime().addShutdownHook(new Thread(() ->
            System.out.println("shutdown hook: draining requests, closing pool")));
        System.out.println("service started, pid " + ProcessHandle.current().pid());
        Thread.sleep(60_000);
    }
}
```

Two wrapper scripts: start_noexec.sh runs java as a child, start_exec.sh uses exec TERMINAL

```
$ cat start_noexec.sh
#!/bin/sh
echo "wrapper: starting service"
java -cp /tmp/osw_o2/p/hot Svc
echo "wrapper: service ended"
$ cat start_exec.sh
#!/bin/sh
echo "wrapper: starting service"
exec java -cp /tmp/osw_o2/p/hot Svc
$ unshare --pid --fork --mount-proc ./start_noexec.sh &        # shell is PID 1; JVM is PID 2
$ kill -TERM 14276                                            # host PID of the namespace's PID 1
$ sleep 2; kill -0 14276 && echo "after SIGTERM and 2 s: PID 1 still alive"
after SIGTERM and 2 s: PID 1 still alive
wrapper: starting service
service started, pid 2
$ unshare --pid --fork --mount-proc ./start_exec.sh &          # JVM replaces the shell: PID 1
$ kill -TERM 14309
wrapper: starting service
service started, pid 1
shutdown hook: draining requests, closing pool
```

In the first run the JVM reports `pid 2`: the shell is PID 1, has installed no handler, and the kernel discards `SIGTERM`. The JVM, which would have handled the signal, never receives it, and the namespace survives until `SIGKILL`. In the second run `exec` replaced the shell by the JVM, which is now PID 1; the JVM installs a `SIGTERM` handler at start, the signal is delivered and the hook runs. The `SigCgt` mask makes the difference observable without sending anything: for a plain `sleep` as PID 1 it is `0000000000000000`, while a running JVM shows `0000000101005ccf`, whose bits include signals 1, 2, 3 and 15 (`SIGHUP`, `SIGINT`, `SIGQUIT`, `SIGTERM`).

Siderain

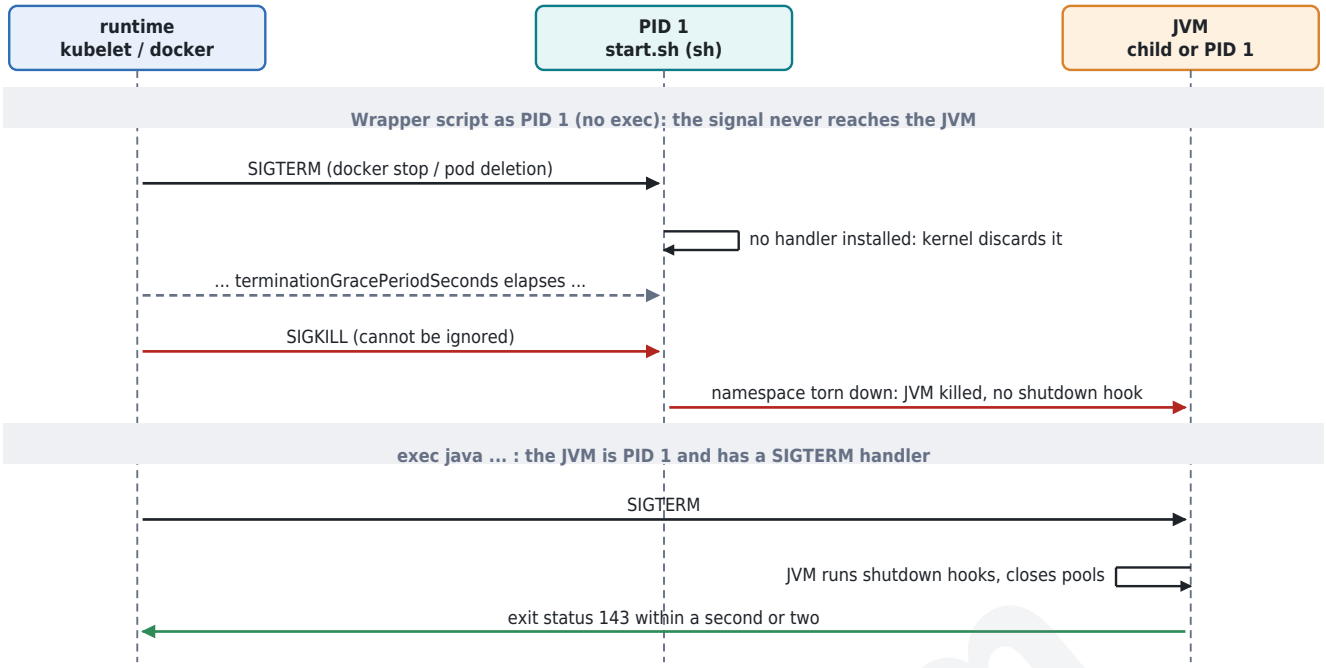


Figure 2.11. Two start-up scripts, two outcomes: without exec the signal stops at a PID 1 that has no handler and the runtime must escalate to SIGKILL; with exec the JVM receives SIGTERM directly and exits with 143.

Root cause. The wrapper script was PID 1 and did not forward signals. The shutdown hook never ran, so in-flight requests and pooled connections were cut off by `SIGKILL` after the grace period.

Fix. Immediate: none is needed beyond accepting the 30-second stop; shortening `terminationGracePeriodSeconds` would only make the abrupt termination more frequent. Permanent: end the wrapper with `exec java ...` so that the JVM takes over PID 1, or remove the wrapper and set the variables in the pod specification, or start an init (`tini`) that forwards signals to its child. Spring Boot additionally offers `server.shutdown=graceful` and `spring.lifecycle.timeout-per-shutdown-phase`; the grace period of the pod must be longer than that timeout.

Prevention. Test the stop path in the pipeline: start the container, send `SIGTERM`, and require exit status 143 within a few seconds. Alert on pods whose last exit code is 137 and which were not memory-limited.

Incident: a service started from an SSH session stops when the session ends

Scenario. An operator starts a Java batch process with `java -jar batch.jar &` from an SSH session during an urgent data fix and closes the laptop. The process was expected to run for six hours.

Symptoms. The job disappears a few minutes after the operator disconnects. No error appears in the application log. The exit status cannot be inspected because the parent shell is gone; the shell history shows the command.

Diagnosis. When the connection closes, the terminal driver sends `SIGHUP` to the session leader (the login shell), and the shell forwards `SIGHUP` to its jobs. A process without a handler for `SIGHUP` terminates by default, and the shell would have reported status 129 (128 plus 1). The reproduction compares a plain process and one started with `nohup`, which sets the disposition of `SIGHUP` to ignore:

SIGHUP kills a plain process and is ignored by nohup — TERMINAL

```
$ sleep 30 & P=$!; kill -HUP $P; wait $P; echo "plain sleep after SIGHUP: exit=$?"
plain sleep after SIGHUP: exit=129
$ nohup sleep 30 > /dev/null 2>&1 & P=$!; sleep 0.3; kill -HUP $P; sleep 0.3
$ kill -0 $P && echo "nohup sleep survived SIGHUP"
nohup sleep survived SIGHUP
```

Root cause. The job belonged to the operator's session and process group, so the hang-up of the terminal reached it, and its default disposition for `SIGHUP` is to terminate.

Fix. Immediate: restart the job with a method that detaches from the terminal: `nohup java -jar batch.jar > batch.log 2>&1 &`, or `setsid`, or a terminal multiplexer. Permanent: run production jobs as supervised services (`systemd-run`, a systemd unit, a Kubernetes `Job`) so that no terminal is involved and the exit status is recorded.

Prevention. Forbid long-running work in interactive sessions by policy, make the standard job launcher write a PID file and a log, and monitor for jobs whose parent PID is a login shell.

2.2.5 Questions to expect

Questions and model answers

1. **What is a signal and how is it delivered?** An asynchronous notification identified by a number. The kernel records it as pending for the target task and delivers it when the task next returns to user mode, running the handler on the task's stack or applying the default action.
2. **Which signals cannot be caught or ignored, and why?** `SIGKILL` and `SIGSTOP`. The kernel must always be able to terminate or stop a process regardless of its code.
3. **What is the difference between `SIGTERM` and `SIGKILL`?** `SIGTERM` can be handled, so the process can clean up and exit (shutdown hooks in Java). `SIGKILL` cannot be handled and ends the process immediately, without cleanup.
4. **What do exit codes 130, 137, 139 and 143 mean?** 128 plus the signal number: 130 is `SIGINT`, 137 is `SIGKILL`, 139 is `SIGSEGV`, 143 is `SIGTERM`.
5. **Why can a handler call only a few functions?** A handler interrupts the program at any point, possibly inside a non-reentrant function such as `malloc`. Only async-signal-safe functions may be called; the usual pattern is to set a flag and return.
6. **If the same standard signal is sent three times before the process runs, how many are delivered?** Normally one: standard signals are not queued. Handlers for `SIGCHLD` loop on `waitpid` with `WNOHANG` for this reason.
7. **What are a process group and a session?** A group is a set of processes signalled together (a pipeline or a job); a session is a set of groups associated with one controlling terminal. Ctrl-C goes to the foreground group; terminal closure sends `SIGHUP` to the session leader.
8. **Why does `docker stop` take ten seconds for some containers?** The main process is PID 1 and has no `SIGTERM` handler, or a wrapper shell does not forward the signal; the runtime waits for the grace period and then sends `SIGKILL`.
9. **How is a daemon created?** `fork`, `setsid`, `fork` again, change directory to `/`, set `umask`, redirect the standard descriptors to `/dev/null`. Under systemd the service simply stays in the foreground.
10. **What does `kill -3` do to a JVM?** `SIGQUIT` makes the JVM print a thread dump to its standard output and continue running. It is equivalent to `jstack` without attaching.

KEY POINTS

Signals are asynchronous notifications recorded as pending bits and delivered on return to user mode; they carry no data except the number.

- `SIGKILL` and `SIGSTOP` are uncatchable; `SIGTERM` is the polite stop that Spring Boot and systemd use, and the JVM turns it into shutdown hooks.
- A status above 128 means death by signal (137 `SIGKILL`, 139 `SIGSEGV`, 143 `SIGTERM`); the number does not identify the sender.
- PID 1 of a PID namespace ignores signals for which it has no handler; use `exec` or an init so that the JVM or an init forwards `SIGTERM`.
- Process groups and sessions bound the reach of Ctrl-C and `SIGHUP`; `nohup`, `setsid` and supervised services detach long jobs.

2.3 Inter-Process Communication

Processes have private address spaces, which is the basis of isolation: a fault in one cannot corrupt another. Cooperating programs nevertheless need to exchange data and coordinate, for example a web server and a log shipper, a JVM and a native helper, or the stages of a shell pipeline. The kernel offers several **inter-process communication** (IPC) mechanisms that differ in what they transfer, how much copying occurs, and who orders and blocks the participants. Choosing the correct one affects throughput and failure behaviour; misusing one is the cause of the commonly seen hanging child process in Java.

2.3.1 Concept and mechanism

The mechanisms

Two families exist. In the **message-passing** family the kernel holds a buffer between the processes: the sender copies data into it with a system call and the receiver copies it out. In the **shared-memory** family the kernel maps the same physical frames into the address spaces of both processes; after the set-up no system call is needed per byte, but the processes must synchronise themselves (see the part on concurrency).

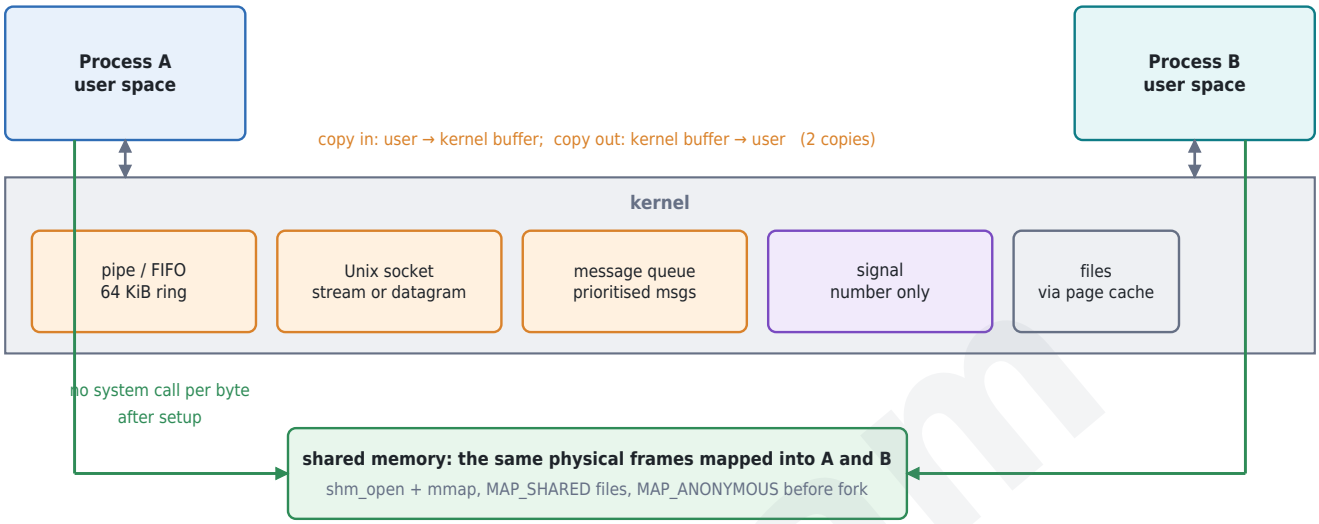


Figure 2.12. Message-passing mechanisms place a kernel object between two processes and cost two copies per transfer; shared memory maps the same frames into both and leaves ordering to the programs.

Mechanism	Properties	Typical use	Java access
`pipe (`pipe`)`	Unidirectional byte stream, 64 KiB buffer, related processes only	Shell pipelines, parent and child	`Process.getInputStream()`
`FIFO (`mkfifo`)`	A pipe with a name in the file system	Unrelated processes on one host	`FileInputStream` on the path
`Unix domain socket`	Bidirectional stream or datagram, credentials passing, file-system address	Local services (database, Docker daemon, agents)	`UnixDomainSocketAddress` (Java 16+)
`POSIX message queue`	Discrete, prioritised messages, kernel limits on size and count	Rare; real-time systems	None in the JDK
`Shared memory`	Same frames mapped in both processes; fastest; needs own locking	Large data, caches, databases	`MappedByteBuffer` on a file in `/dev/shm`
`Signal`	A number, no payload, coalesced	Stop, reload, notify	`kill`, `ProcessHandle`
`Regular file`	Persistent, any process, ordered by the page cache	Spool files, hand-over	`Files`
`TCP socket`	Works across hosts, higher overhead	Any network service	`Socket`

IPC mechanisms on Linux.

Pipes in detail

A pipe is a ring buffer inside the kernel made of 16 page-sized slots. `write` appends to it and `read` removes from it. The rules that decide the behaviour of every pipeline follow from three conditions.

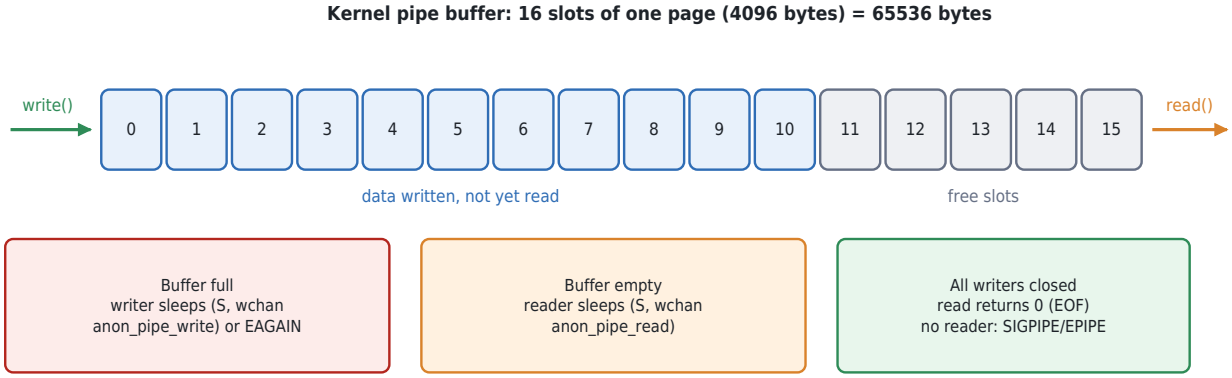


Figure 2.13. A full pipe blocks its writer, an empty pipe blocks its reader, and the closing of all write ends turns a read into end-of-file; the capacity is 65536 bytes by default.

- If the buffer is full, `write` blocks until a reader makes room (or fails with `EAGAIN` for a non-blocking descriptor).
- If the buffer is empty, `read` blocks until data arrives.
- If every write end has been closed, `read` returns 0 (end-of-file); if every read end has been closed, `write` raises `SIGPIPE` (or returns `EPIPE`).

The third rule explains why programs must close the descriptors they do not use: a reader never sees end-of-file as long as any process, including itself, still holds an open write end.

How a shell runs a pipeline

For `seq 1 5 | grep -v 3 | wc -l` the shell creates all pipes first, then one child per command. Each child connects the pipe ends to descriptors 0 and 1 with `dup2` and then calls `exec`. The shell itself waits for all of them. The trace in the worked example shows the real calls.

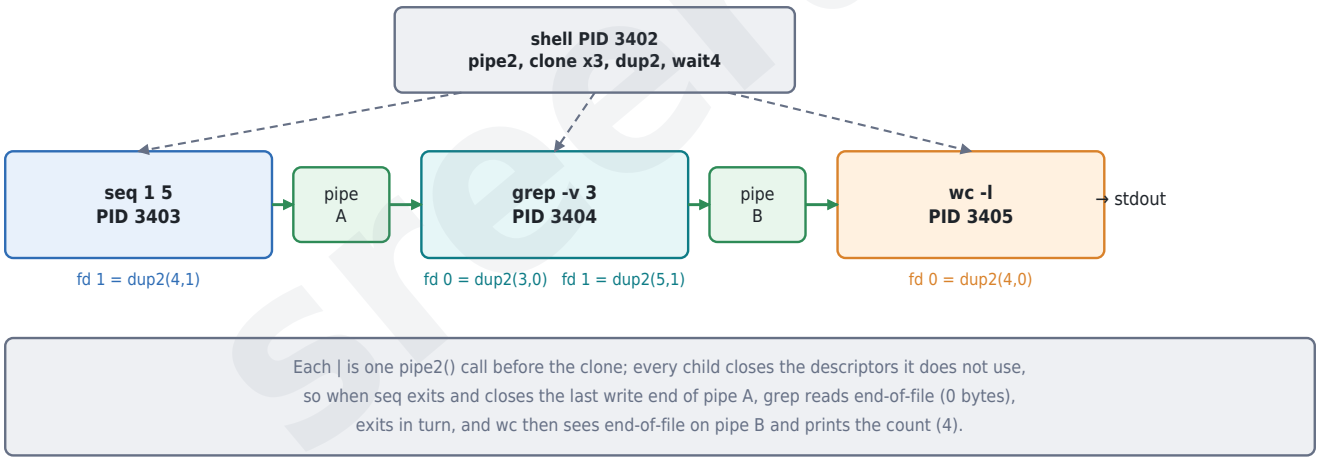


Figure 2.14. The shell creates two pipes before forking three children; each child duplicates the proper pipe ends onto descriptors 0 and 1, so the programs themselves are unaware that they are connected by pipes.

2.3.2 Worked trace

The script `pl.sh` contains only the pipeline `seq 1 5 | grep -v 3 | wc -l`. The following trace lists the selected system calls of the shell (3402) and its three children; the lines are copied from the real output and reduced to the calls that connect the pipes (`...` marks omitted lines).

strace -f of the pipeline (selected lines) TERMINAL

```
$ strace -f -e trace=pipe2,clone,dup2,execve,wait4,close -o pl.trace ./pl.sh
$ grep -E 'pipe2|clone|dup2\(|execve\("/usr|wait4' pl.trace
3402  pipe2([3, 4], 0)                  = 0
3402  clone(flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD) = 3403
3402  pipe2([4, 5], 0)                  = 0
3402  clone(flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD) = 3404
3402  clone(flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD) = 3405
3404  dup2(3, 0)                        = 0
```

strace -f of the pipeline (selected lines) (continued) — TERMINAL

```
3403  dup2(4, 1)                        = 1
3404  dup2(5, 1)                        = 1
3404  execve("/usr/bin/grep", ["grep", "-v", "3"], ...) = 0
3403  execve("/usr/bin/seq", ["seq", "1", "5"], ...) = 0
3405  dup2(4, 0)                        = 0
3405  execve("/usr/bin/wc", ["wc", "-l"], ...) = 0
3402  wait4(-1, [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 3403
3402  wait4(-1, [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 3404
```

Reading the trace in order gives the algorithm.

1. `pipe2([3, 4])` creates pipe A: read end 3, write end 4.
2. The first `clone` creates `seq` (3403). The shell closes its copy of descriptor 4, and the child does `dup2(4, 1)`, so that its standard output is the write end of pipe A.
3. `pipe2([4, 5])` creates pipe B (the numbers 4 and 5 are free again because the shell closed 4).
4. The second `clone` creates `grep` (3404): `dup2(3, 0)` makes pipe A's read end its standard input, and `dup2(5, 1)` makes pipe B's write end its standard output.
5. The third `clone` creates `wc` (3405); `dup2(4, 0)` attaches the read end of pipe B.
6. Each child calls `execve`. The descriptors survive the exec, which is why the programs need no knowledge of pipes.
7. `seq` exits and closes the last write end of pipe A; `grep` reads end-of-file, exits, and `wc` finally reads end-of-file on pipe B and prints 4. The shell's `wait4` calls return 3403 and 3404 in turn, and then 3405.

The processes run concurrently, not one after the other: `grep` starts reading while `seq` still writes, and the pipe buffer is the only coupling. This is why a pipeline processes data of unlimited size with 64 KiB of buffer.

2.3.3 Hands-on

Pipe between parent and child, and its capacity

pipedemo.c — C

```c
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    int fd[2];
    if (pipe(fd) == -1) { perror("pipe"); return 1; }
    printf("pipe: read end fd=%d, write end fd=%d\n", fd[0], fd[1]);
    pid_t pid = fork();
    if (pid == 0) {
        close(fd[0]);                                   /* child writes */
        const char *msg = "hello through the kernel pipe buffer";
        write(fd[1], msg, strlen(msg));
        close(fd[1]);
        _exit(0);
    }
    close(fd[1]);                                       /* parent reads */
    char buf[64];
    ssize_t n = read(fd[0], buf, sizeof buf - 1);
    buf[n] = '\0';
    printf("parent read %zd bytes: %s\n", n, buf);
    n = read(fd[0], buf, sizeof buf);                   /* all writers closed */
    printf("second read returns %zd (end of file)\n", n);
    waitpid(pid, NULL, 0);
    return 0;
}
```

Output, followed by the capacity measurement and a FIFO — TERMINAL

```
$ ./pipedemo
pipe: read end fd=3, write end fd=4
parent read 36 bytes: hello through the kernel pipe buffer
second read returns 0 (end of file)
$ ./pipecap                       # writes 1 KiB blocks to a non-blocking pipe until it fails
write failed after 65536 bytes: errno=11 (EAGAIN, pipe full)
F_GETPIPE_SZ = 65536
$ cat /proc/sys/fs/pipe-max-size
```

Output, followed by the capacity measurement and a FIFO (continued) — TERMINAL

```
1048576
$ mkfifo ff && ls -l ff
prw-r--r-- 1 root root 0 Oct 18 02:16 ff
$ (cat ff > ff.out &); echo "via named pipe" > ff; sleep 0.3; cat ff.out
via named pipe
```

The second `read` returning 0 is the end-of-file rule: the child has exited and the parent closed its own write end before reading. The type letter `p` in `ls -l` marks a FIFO; its data never touches the disk. A privileged process can enlarge a pipe up to `/proc/sys/fs/pipe-max-size` with `fcntl(F_SETPIPE_SZ)`.

Unix domain sockets in C and Java

A socket pair is a bidirectional channel between related processes; a bound Unix socket with a path allows unrelated processes to connect. The Java version uses the standard library since Java 16.

A connected pair created with socketpair (uds.c) — TERMINAL

```
$ ./uds
child  got request : ping
parent got response: pong
```

UdsDemo.java — JAVA

```java
import java.net.StandardProtocolFamily;
import java.net.UnixDomainSocketAddress;
import java.nio.ByteBuffer;
import java.nio.channels.ServerSocketChannel;
import java.nio.channels.SocketChannel;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;

public class UdsDemo {
    public static void main(String[] args) throws Exception {
        Path path = Path.of("/tmp/osw_o2/demo.sock");
        Files.deleteIfExists(path);
        var address = UnixDomainSocketAddress.of(path);
        try (ServerSocketChannel server = ServerSocketChannel.open(StandardProtocolFamily.UNIX)) {
            server.bind(address);
            Thread client = new Thread(() -> {
                try (SocketChannel ch = SocketChannel.open(address)) {
                    ch.write(ByteBuffer.wrap("GET /status".getBytes(StandardCharsets.UTF_8)));
                    ByteBuffer reply = ByteBuffer.allocate(64);
                    ch.read(reply);
                    System.out.println("client received: "
                            + new String(reply.array(), 0, reply.position(), StandardCharsets.UTF_8));
                } catch (Exception e) { e.printStackTrace(); }
            });
            client.start();
            try (SocketChannel conn = server.accept()) {
                ByteBuffer req = ByteBuffer.allocate(64);
                conn.read(req);
                System.out.println("server received: "
                    + new String(req.array(), 0, req.position(), StandardCharsets.UTF_8));
                conn.write(ByteBuffer.wrap("200 OK".getBytes(StandardCharsets.UTF_8)));
            }
            client.join();
        }
        Files.deleteIfExists(path);
    }
}
```

Run of UdsDemo.java — TERMINAL

```
$ java UdsDemo.java
server received: GET /status
client received: 200 OK
```

Local services such as the Docker daemon (`/var/run/docker.sock`) and many database clients use Unix sockets because they avoid the TCP/IP stack and allow the server to read the peer's user identity.

Shared memory and message queues

POSIX shared memory between two processes (shm_writer.c and shm_reader.c, linked with -lrt) — TERMINAL

```
$ ./shm_writer
writer pid=3550 wrote into the shared page and exits
$ ls -l /dev/shm/osw_o2_demo
-rw------- 1 root root 4096 Oct 18 02:17 /dev/shm/osw_o2_demo
$ ./shm_reader
reader pid=3552 sees: written by process A, no read/write syscall needed
```

The segment is a file on the `tmpfs` mounted at `/dev/shm`; `shm_open` is `open` on that directory and `mmap` with `MAP_SHARED` maps it. The writer had already exited when the reader mapped the object, so the data persisted in the kernel until `shm_unlink`. The same mechanism is available from Java with a mapped buffer:

A MappedByteBuffer on /dev/shm shared by two JVMs (SharedMap.java write, then read) — TERMINAL

```
$ java SharedMap.java write
JVM 3925 wrote to the mapping
$ java SharedMap.java read
JVM 3944 read: counter=42
```

POSIX message queues deliver whole messages in priority order, as the following run of `mq.c` shows: three messages sent with priorities 1, 9 and 5 arrive in the order 9, 5, 1.

mq.c, linked with -lrt — TERMINAL

```
$ ./mq
received (priority 9): HIGH priority
received (priority 5): medium priority
received (priority 1): low priority
```

2.3.4 Real-world incidents

Incident: a Java service hangs when it runs an external command with large output

Scenario. A Spring Boot service generates PDF thumbnails by running an external converter through `ProcessBuilder`. The code starts the process, calls `waitFor()` and then reads the output. It has worked for months with small documents. After a customer uploads a large document the request never returns and the thread pool fills with waiting request threads.

Symptoms. Requests for large inputs time out. CPU use is near zero. The thread dump shows request threads in `Process.waitFor`. The converter processes are alive and not consuming CPU. Small inputs still work.

Diagnosis. Process state and kernel wait channel come first, because a sleeping child with no CPU use means that it waits for something. The reproduction starts `seq 1 100000` (about 589 KB of output) from Java without reading its output, and inspects the child after three seconds.

PipeHang.java — JAVA

```java
import java.io.InputStream;
import java.util.concurrent.TimeUnit;

public class PipeHang {
    public static void main(String[] args) throws Exception {
        boolean drain = args.length > 0 && args[0].equals("drain");
        // The child prints about 600 KB, far more than the 64 KiB pipe buffer.
        Process p = new ProcessBuilder("/usr/bin/seq", "1", "100000").start();
        System.out.println("child pid " + p.pid() + ", drain=" + drain);
        long bytes = 0;
        if (drain) {
            try (InputStream in = p.getInputStream()) {
                bytes = in.transferTo(java.io.OutputStream.nullOutputStream());
            }
        }
        boolean done = p.waitFor(5, TimeUnit.SECONDS);
        System.out.println("finished=" + done + ", bytes read=" + bytes);
        if (!done) p.destroyForcibly();
    }
}
```

Without and with draining the output TERMINAL

```
$ java PipeHang.java &
$ ps -o pid,stat,wchan:20,comm -p $(pgrep -x seq)
  PID STAT WCHAN                COMMAND
15665 S    anon_pipe_write      seq
$ ls -l /proc/15665/fd/1
l-wx------ 1 root root 64 Oct 18 02:35 /proc/15665/fd/1 → pipe:[32364]
child pid 15665, drain=false
finished=false, bytes read=0
$ java PipeHang.java drain
child pid 15768, drain=true
finished=true, bytes read=588895
```

The child is in state S with wait channel `anon_pipe_write`: it is blocked writing to a full pipe, and its standard output (descriptor 1) is the pipe. The parent never reads the other end, because it waits for the child to exit before reading. Both wait for each other. The output size, 588895 bytes, exceeds the 65536-byte buffer, which explains why only large inputs fail.

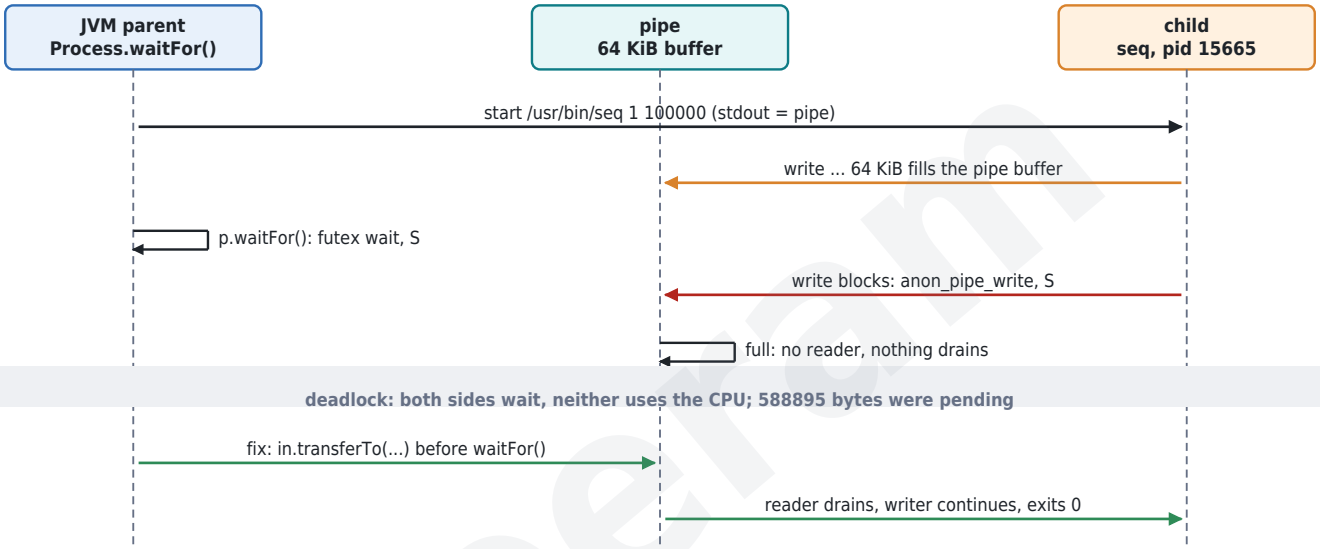


Figure 2.15. The parent waits for the exit of a child that cannot exit until the parent reads: a circular wait through a 64 KiB buffer, visible as a child in state S at anon_pipe_write.

Root cause. The program waits for process termination before consuming the child's output, so any output beyond the pipe capacity blocks the child permanently. The same applies to the standard error stream, which is a second pipe unless merged.

Fix. Immediate: kill the stuck converter processes (their PIDs are listed by `pgrep`) so that the request threads are released, and restrict input size. Permanent: consume the streams while the child runs. Use `ProcessBuilder.redirectErrorStream(true)` to merge standard error into standard output, read the stream to the end (or use `redirectOutput(File)` to send the output to a file without any pipe), and use `waitFor` with a timeout followed by `destroyForcibly`.

Prevention. Wrap external commands in one helper that applies the three rules and a timeout, ban direct `Runtime.exec` in code review, and monitor the number of child processes of the JVM.

Incident: a process dies with SIGBUS (exit 135) while writing to shared memory

Scenario. A containerised service shares a 200 MB cache between worker processes through a file mapped from a memory-backed directory. The deployment moves to a node profile in which that directory is a `tmpfs` with a smaller size limit. The mapping is created by `ftruncate` to the full size.

Symptoms. Workers terminate unexpectedly with status 135 or with the log line `Bus error`, after running correctly for some time. The failures occur when the cache is filled, not at start-up. No out-of-memory event is recorded for the cgroup.

Diagnosis. Status 135 is 128 plus 7, the signal `SIGBUS`. `SIGSEGV` signals an access to an address that is not mapped; `SIGBUS` signals an access to a mapped address that the kernel cannot back with storage. The reproduction maps 4 MiB of a sparse file in a `tmpfs` of 1 MiB:

A 1 MiB tmpfs and a 4 MiB mapping (shmfull.c touches one byte per page) TERMINAL

```
$ mount -t tmpfs -o size=1m tmpfs /tmp/osw_o2/shm
$ ./shmfull /tmp/osw_o2/shm/seg
touching offset 0 KiB
touching offset 256 KiB
touching offset 512 KiB
touching offset 768 KiB
touching offset 1024 KiB
Bus error
$ echo "exit=135"; df -h /tmp/osw_o2/shm | tail -1
exit=135
tmpfs           1.0M  1.0M     0 100% /tmp/osw_o2/shm
```

`ftruncate` succeeded for 4 MiB because tmpfs files are sparse: space is charged when a page is first written. The first write beyond 1 MiB asked for a page that did not exist, and the kernel answered with `SIGBUS` instead of an error return, because a store instruction has no way to report failure.

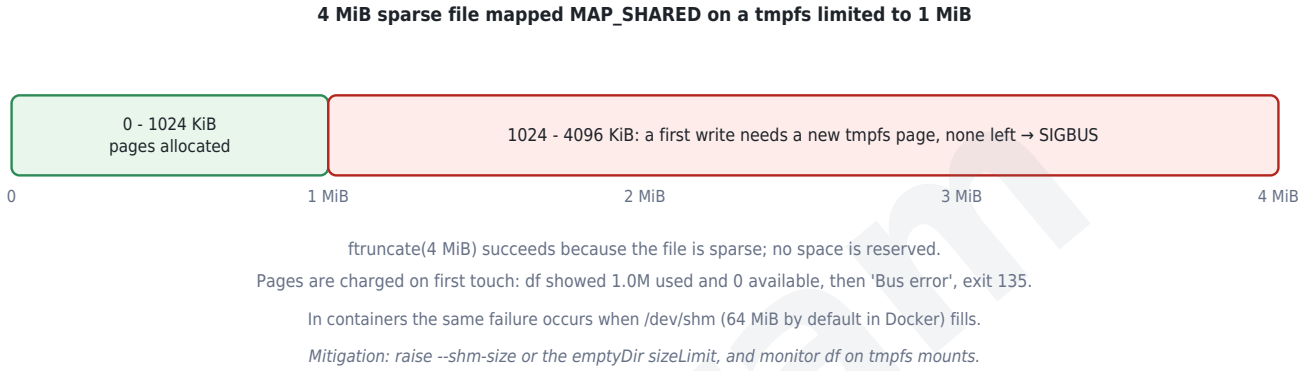


Figure 2.16. The mapping is larger than the file system that backs it; the error appears only when a page beyond the capacity is first touched.

Root cause. The size of the mapped region exceeded the capacity of the backing `tmpfs`. The same failure occurs in Docker, whose `/dev/shm` defaults to 64 MiB,. A JVM maps a performance-data file under the temporary directory, so a full file system there can cause the same signal; the option `-XX:-UsePerfData` disables that file.

Fix. Immediate: enlarge the mount (`--shm-size` for Docker, `emptyDir` with `medium: Memory` and `sizeLimit` in Kubernetes) and restart the workers. Permanent: reserve the space at creation time with `posix_fallocate` instead of `ftruncate`, so that a lack of capacity appears as an error at start-up and not as a signal in the middle of a request, and size the memory volume from the maximum cache size.

Prevention. Alert on the usage of every `tmpfs` mount, count `SIGBUS` terminations (status 135) separately from out-of-memory kills, and treat memory-backed volumes as part of the container's memory budget.

2.3.5 Questions to expect

Questions and model answers

1. **Name the main IPC mechanisms in Linux.** Pipes, FIFOs, Unix domain sockets, POSIX and System V message queues, shared memory, signals, regular files and network sockets.
2. **Why is shared memory the fastest mechanism?** After the mapping is set up the processes read and write the same physical frames without system calls or copies. It needs explicit synchronisation, because the kernel does not order the accesses.
3. **How does a shell implement `a | b`?** It calls `pipe`, forks two children, uses `dup2` to put the write end on the first child's descriptor 1 and the read end on the second child's descriptor 0, closes the unused ends and calls `exec` in both.
4. **When does `read` on a pipe return 0?** When the buffer is empty and no process holds a write end. A leaked write end in any process prevents end-of-file.
5. **What happens when a process writes to a pipe whose reader has exited?** The kernel sends `SIGPIPE`, which terminates the writer by default; if the signal is ignored, `write` fails with `EPIPE`.
6. **What is the capacity of a pipe and what follows from it?** 65536 bytes by default. A writer faster than its reader blocks, which gives pipelines natural flow control; a program that waits for the child before reading may deadlock.

7. **Why does a Java program that calls `Process.waitFor()` before reading output hang?** The child fills the pipe and blocks; the parent waits for its exit. The output must be consumed concurrently, merged or redirected to a file.
8. **Pipe versus Unix domain socket?** A pipe is unidirectional and requires a common ancestor (unless named); a Unix socket is bidirectional, can be connected by unrelated processes by path and can pass credentials and descriptors.
9. **What is the difference between `SIGSEGV` and `SIGBUS`?** `SIGSEGV` is an access to an unmapped or protected address; `SIGBUS` is an access to a valid mapping that cannot be backed, such as a page beyond the end of a file or of a full `tmpfs`.
10. **When would shared memory be a poor choice?** When the data is small or infrequent, when the processes are on different hosts, or when the team cannot guarantee correct locking; a socket or pipe is simpler and safer.

KEY POINTS

IPC mechanisms either pass data through a kernel buffer (pipes, sockets, queues) or map the same frames into several processes (shared memory).

- A pipe is a 64 KiB ring: full blocks the writer, empty blocks the reader, no write end gives end-of-file, no read end gives `SIGPIPE`.
- A shell builds a pipeline with `pipe`, `clone`, `dup2` and `exec`; the programs are unaware of the pipes.
- Never wait for a child before draining its output streams: the result is a circular wait with the child in `anon_pipe_write`.
- `SIGBUS` (exit 135) on a mapping means the backing store is smaller than the mapping; reserve space with `posix_fallocate`.

2.4 Threads

A process that serves many clients, or that overlaps computation with waiting for I/O, needs several flows of control. Using several processes isolates them completely but makes data sharing expensive. A **thread** is a flow of control inside a process that shares the address space and the open files with its siblings, and has its own stack, register set and scheduling state. A Java application server is a single process with hundreds of threads, and understanding how those threads exist in the kernel explains the limits on their number, the cost of switching between them and the behaviour of virtual threads.

2.4.1 Concept and mechanism

Threads are tasks that share resources

Linux has no separate thread object. Every schedulable entity is a `task_struct`, and the system call `clone` creates one with a set of flags that select which resources are shared with the creator. `fork` uses no sharing flags; `pthread_create` passes `CLONE_VM` (same address space), `CLONE_FS`, `CLONE_FILES`, `CLONE_SIGHAND` and `CLONE_THREAD` (same thread group, hence the same process ID as seen by user tools).

resource	fork() (child process)	pthread_create() (thread)
address space (mm)	copied, copy-on-write	shared CLONE_VM
open files (files)	table copied, objects shared	shared CLONE_FILES
signal handlers	copied	shared CLONE_SIGHAND
cwd, root (fs)	copied	shared CLONE_FS
PID / TGID	new PID = new TGID	new TID, same TGID: CLONE_THREAD
user stack	copy of parent stack	new stack mapped by libc
scheduling entity	one per task	one per task

Both end in the same kernel function; the flags decide what is shared (strace: CLONE_VM|CLONE_FS|CLONE_FILES|...).

Figure 2.17. The difference between a process and a thread in Linux is entirely in the clone flags: the kernel structures are the same, but a thread points to the creator's mm, files and handlers instead of copies.

A process has one **thread group identifier** (`Tgid`, what `getpid()` returns) and each thread has its own **thread identifier** (`Pid` in `/proc/PID/status`, `gettid()`). The first thread's TID equals the PID. The directory `/proc/PID/task` lists one entry per thread. Threads of a process share the heap and the global data, so one thread can corrupt the data of another; each has a private stack, and the library allocates it with `mmap` (default 8 MiB for C programs governed by `ulimit -s`, 1 MiB for Java threads, as set by `-Xss` or `-XX:ThreadStackSize`).

User threads, kernel threads and virtual threads

Three models exist in the literature. In the **one-to-one** model each user thread is a kernel task; Linux, with the POSIX library, and the HotSpot JVM use this model. In the **many-to-one** model a library multiplexes many user threads on one kernel thread, which gives cheap threads but no parallelism, and one blocking call blocks all. The **many-to-many** model multiplexes many user threads on a pool of kernel threads. Java 21 **virtual threads** use the third model: the JVM mounts a virtual thread on a **carrier** thread (a platform thread from a `ForkJoinPool`) and unmounts it when it blocks in a supported operation, so that the carrier can run another virtual thread. The kernel sees only the carriers.

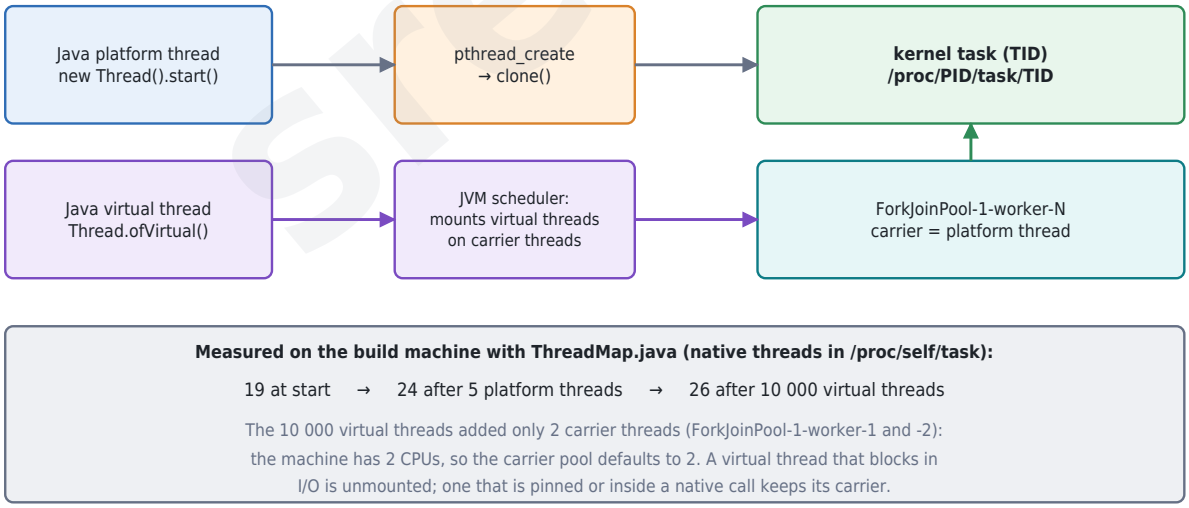
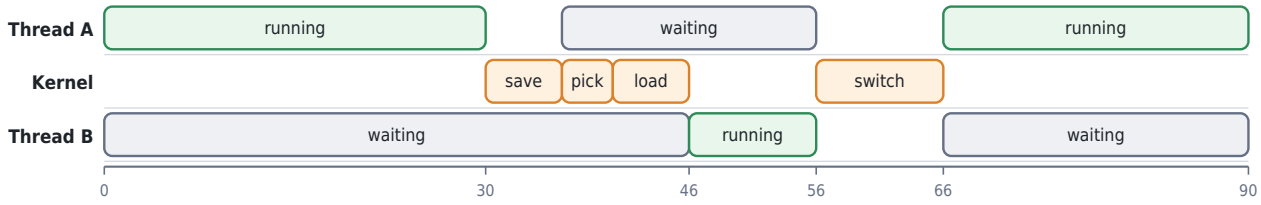


Figure 2.18. A Java platform thread maps one-to-one to a kernel task; 10 000 virtual threads were multiplexed onto two carrier tasks in the measured run.

Context switches

When the scheduler stops one task and runs another it performs a **context switch**: the processor registers and stack pointer of the old task are saved, the registers of the new task are loaded and, if the tasks belong to different processes, the page-table base register is changed. A switch between threads of the same process avoids the address-space change and its effects on the TLB, which is one of the reasons why threads are cheaper than processes. The direct cost is about a microsecond (measured below); the indirect cost is the loss of cache and TLB contents, which lengthens the following execution. Two kinds are distinguished: a **voluntary** switch happens when the task blocks (I/O, lock, sleep) and a **non-voluntary** switch when the kernel preempts a task that used its time slice. Both counters are in `/proc/PID/status`.



Switch: (1) a timer interrupt, blocking call or wake-up enters the kernel, (2) the old task's registers and stack pointer are saved, (3) the scheduler picks the next task, (4) the address space is switched if the tasks belong to different processes (CR3 write, TLB effects), (5) the new task's registers load.

Measured (ctxsw.c, two threads pinned to one CPU, pipe ping-pong): 1.0 to 1.3 µs per switch

including two system calls. The larger hidden cost is cold caches and TLB entries afterwards.

Figure 2.19. A switch from thread A to thread B saves and loads state inside the kernel; the processor does no user work during the grey kernel segments, so a workload with many switches pays that overhead on every one.

Concurrency models for servers

A server may give each request a new thread, a fixed pool of threads fed by a queue, or a small number of threads running an event loop over non-blocking descriptors. The trade-off is between code simplicity and resource use.

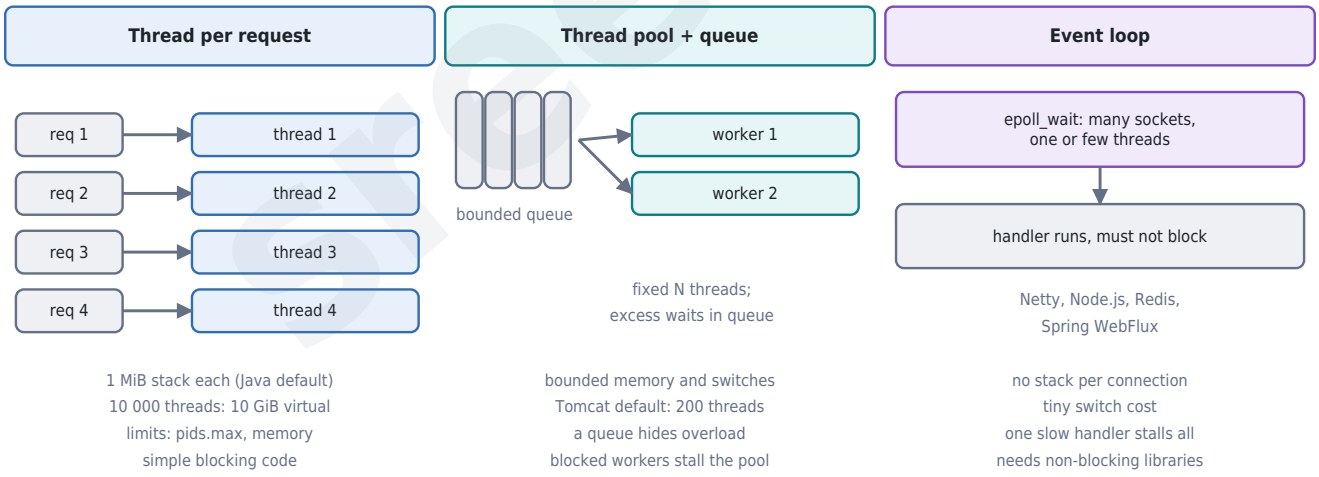


Figure 2.20. Thread per request is simple but costs one stack and one task per request; a pool bounds both and queues the excess; an event loop uses almost no memory per connection but forbids blocking calls in handlers.

Tomcat uses a pool (200 worker threads by default in Spring Boot), Netty and Spring WebFlux use event loops, and virtual threads restore the simplicity of thread-per-request code at the cost of a user-space scheduler, because a virtual thread blocked on a socket does not occupy a kernel task.

2.4.2 Worked trace

The program `pth.c` creates one thread and prints the process and thread identifiers of both, the address of a stack variable in each, and the entries in `/proc/PID/task`.

pth.c C

```c
#define _GNU_SOURCE
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/syscall.h>
#include <unistd.h>

static int shared = 0;                    /* one variable, one address space */

static void *worker(void *arg) {
    shared += 10;
    printf("worker : getpid()=%d gettid()=%ld shared=%d stack var at %p\n",
           getpid(), syscall(SYS_gettid), shared, (void *)&arg);
    fflush(stdout);
    sleep(3);
    return NULL;
}

int main(void) {
    pthread_t t;
    int local = 0;
    printf("main   : getpid()=%d gettid()=%ld stack var at %p\n",
           getpid(), syscall(SYS_gettid), (void *)&local);
    fflush(stdout);
    pthread_create(&t, NULL, worker, NULL);
    sleep(1);
    printf("main   : /proc/%d/task contains:", getpid());
    char cmd[64];
    snprintf(cmd, sizeof cmd, "ls /proc/%d/task | tr '\\n' ' '", getpid());
    fflush(stdout);
    int rc = system(cmd); (void)rc;
    printf("\nmain   : shared=%d after worker ran\n", shared);
    pthread_join(t, NULL);
    return 0;
}
```

Output and the clone call behind pthread_create TERMINAL

```
$ gcc -Wall -pthread -o pth pth.c && ./pth
main   : getpid()=4126 gettid()=4126 stack var at 0x7ffc768be5c0
worker : getpid()=4126 gettid()=4127 shared=10 stack var at 0x7f3acebfeea8
main   : /proc/4126/task contains:4126 4127
main   : shared=10 after worker ran
$ strace -f -e trace=clone3 ./pth 2>&1 >/dev/null | head -1 | cut -c1-210
clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|
  CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID, child_tid=0x7fb0559ff990, ...
```

Interpretation:

1. `getpid()` is 4126 in both threads, whereas `gettid()` differs (4126 and 4127): one process, two tasks.
2. The worker's change to `shared` is visible in the main thread (`shared=10`), unlike the child of `fork`, whose changes were private.
3. The two stack addresses are far apart: the main thread uses the process stack near `0x7ffc...`, the worker a region mapped by the library near `0x7f3a...`.
4. `/proc/4126/task` lists both TIDs.
5. The trace shows `CLONE_VM|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD`, which is the full set of sharing flags of the figure; the stack size recorded in the call (`0x7fff80`, close to 8 MiB) is the default for C threads.

2.4.3 Hands-on

Native threads of a JVM

ThreadMap.java — JAVA

```java
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.Executors;
import java.util.stream.IntStream;

public class ThreadMap {
    static int nativeThreads() throws Exception {
        try (var s = Files.list(Path.of("/proc/self/task"))) { return (int) s.count(); }
    }

    public static void main(String[] args) throws Exception {
        System.out.println("pid " + ProcessHandle.current().pid()
            + ", native threads at start: " + nativeThreads());
        CountDownLatch release = new CountDownLatch(1);
        for (int i = 0; i < 5; i++) {
            Thread.ofPlatform().name("worker-" + i).start(() -> {
                try { release.await(); } catch (InterruptedException e) { }
            });
        }
        System.out.println("after 5 platform threads    : " + nativeThreads());
        try (var exec = Executors.newVirtualThreadPerTaskExecutor()) {
            IntStream.range(0, 10_000).forEach(i -> exec.submit(() -> {
                try { release.await(); } catch (InterruptedException e) { }
                return i;
            }));
            Thread.sleep(500);
            System.out.println("after 10000 virtual threads : " + nativeThreads());
            try (var s = Files.list(Path.of("/proc/self/task"))) {
                s.map(p -> { try { return Files.readString(p.resolve("comm")).trim(); }
                             catch (Exception e) { return "?"; } })
                 .sorted().forEach(n -> System.out.print(n + "  "));
            }
            System.out.println();
            release.countDown();
        }
    }
}
```

Typical output on the 2-CPU build machine (JDK 21, G1) — TERMINAL

```
$ java ThreadMap.java
pid 4517, native threads at start: 19
after 5 platform threads    : 24
after 10000 virtual threads : 26
C1 CompilerThre  C2 CompilerThre  Common-Cleaner  Finalizer  ForkJoinPool-1-  ForkJoinPool-1-
G1 Conc#0  G1 Main Marker  G1 Refine#0  G1 Service  GC Thread#0  GC Thread#1  Monitor Deflati
Notification Th  Reference Handl  Service Thread  Signal Dispatch  VM Periodic Tas  VM Thread
java  java  worker-0  worker-1  worker-2  worker-3  worker-4
```

The nineteen threads at start are JVM internal: the VM thread, GC and concurrent-mark threads, compiler threads, the signal dispatcher, the finalizer and others (the file is run through the single-file launcher, so two threads named `java` belong to the launcher). The kernel limits thread names to 15 characters, which is why names are truncated. Platform threads appear one for one; the ten thousand virtual threads added only the two carriers `ForkJoinPool-1-worker-N`.

Stack size and recursion depth

Default thread stack size and the effect of -Xss on a Java recursion (typical, varies between runs) — TERMINAL

```
$ java -XX:+PrintFlagsFinal -version 2>/dev/null | grep -E "ThreadStackSize" | tr -s ' '
 intx CompilerThreadStackSize = 1024 {pd product} {default}
 intx ThreadStackSize = 1024 {pd product} {default}
 intx VMThreadStackSize = 1024 {pd product} {default}
$ for x in 256k 512k 1m 4m; do echo -n "-Xss$x : "; java -Xss$x Depth.java; done
-Xss256k : StackOverflowError at depth 2376
-Xss512k : StackOverflowError at depth 4772
-Xss1m : StackOverflowError at depth 21744
-Xss4m : StackOverflowError at depth 98982
```

`Depth.java` recurses in a thread created with stack size 0 (meaning "use the default"). The depth grows with the stack size but not exactly proportionally, since interpreted and compiled frames differ in size. Stack memory is virtual until touched: a thread reserves 1 MiB of address space but only the touched pages become resident, so 10 000 platform threads reserve about 10 GiB of virtual memory and a much smaller resident amount. The limits that stop thread creation first are usually the `pids` cgroup limit and `vm.max_map_count`, not memory.

The cost of a context switch and of creating a thread

ctxsw.c C

```c
#define _GNU_SOURCE
#include <pthread.h>
#include <sched.h>
#include <stdio.h>
#include <time.h>
#include <unistd.h>

#define ROUNDS 200000
static int ab[2], ba[2];

static void pin(void) {
    cpu_set_t set; CPU_ZERO(&set); CPU_SET(0, &set);
    sched_setaffinity(0, sizeof set, &set);      /* 0 = calling thread */
}
static double now(void) {
    struct timespec ts; clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec * 1e9 + ts.tv_nsec;
}
static void *echo(void *arg) {
    pin(); char c;
    for (int i = 0; i < ROUNDS; i++) { read(ab[0], &c, 1); write(ba[1], &c, 1); }
    return NULL;
}
int main(void) {
    pipe(ab); pipe(ba);
    pthread_t t; pthread_create(&t, NULL, echo, NULL);
    pin(); char c = 'x';
    double t0 = now();
    for (int i = 0; i < ROUNDS; i++) { write(ab[1], &c, 1); read(ba[0], &c, 1); }
    double t1 = now();
    pthread_join(t, NULL);
    /* each round trip = 2 context switches (main -> echo -> main) */
    printf("%d round trips in %.1f ms: %.0f ns per context switch (incl. 2 syscalls)\n",
           ROUNDS, (t1 - t0) / 1e6, (t1 - t0) / ROUNDS / 2);
    return 0;
}
```

Three runs on the build machine (a shared virtual machine, so values vary) TERMINAL

```
$ gcc -O2 -pthread -o ctxsw ctxsw.c && for i in 1 2 3; do ./ctxsw; done
200000 round trips in 406.2 ms: 1016 ns per context switch (incl. 2 syscalls)
200000 round trips in 506.2 ms: 1265 ns per context switch (incl. 2 syscalls)
200000 round trips in 434.2 ms: 1085 ns per context switch (incl. 2 syscalls)
```

Both threads are pinned to one CPU, so each message forces a real switch: the sender blocks on the reply, and the kernel runs the other thread. The figure includes the two system calls per switch, so the pure switch is below one microsecond. The result has two consequences: a thread pool with thousands of busy threads spends a measurable share of its time switching, and 100 000 switches per second cost roughly 10 percent of one core.

The cost of creating a platform thread is far higher than of reusing one, as the measurement `SpawnCost.java` shows for 10 000 trivial tasks:

Typical per-task cost (measured on the build machine, three rounds) TERMINAL

```
$ java SpawnCost.java
round 0: thread-per-task  163.0 us/task | pool(4)   3.6 us/task | virtual   6.9 us/task
round 1: thread-per-task  287.9 us/task | pool(4)   1.2 us/task | virtual   2.5 us/task
round 2: thread-per-task  195.7 us/task | pool(4)   4.6 us/task | virtual   1.7 us/task
```

Creating and joining a platform thread costs on the order of 100 to 300 microseconds here, two orders of magnitude more than handing a task to a pool or starting a virtual thread. This is the quantitative reason for pooling.

2.4.4 Real-world incidents

Incident: an unbounded cached thread pool exhausts the process limit

Scenario. A Spring Boot service calls a slow downstream API through an `ExecutorService` created with `Executors.newCachedThreadPool()`, because the example in a tutorial used it. During a downstream brownout each call blocks for twenty seconds, while the incoming rate stays constant.

Symptoms. The thread count of the service climbs steadily. Latency grows, and then the log shows `java.lang.OutOfMemoryError: unable to create native thread: possibly out of memory or process/resource limits reached`. The heap is nearly empty and the pod's memory use is moderate. Health checks that need a thread begin to fail, and the orchestrator restarts the pod.

Diagnosis. The message names the cause as a limit, not a lack of heap. The count of tasks in the process shows the growth directly; the cgroup limit shows the ceiling. The reproduction creates tasks that block for 20 seconds inside a `pids` cgroup limited to 60 tasks:

ThreadBomb.java JAVA

```java
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.ThreadPoolExecutor;

public class ThreadBomb {
    public static void main(String[] args) throws Exception {
        // newCachedThreadPool: unbounded thread count, 60 s keep-alive
        ExecutorService pool = Executors.newCachedThreadPool();
        ThreadPoolExecutor tpe = (ThreadPoolExecutor) pool;
        try {
            for (int i = 1; i <= 100_000; i++) {
                pool.submit(() -> {                    // each task blocks on a slow backend
                    try { Thread.sleep(20_000); } catch (InterruptedException e) { }
                });
                if (i % 20 == 0) {
                    System.out.println("submitted " + i + ", pool size " + tpe.getPoolSize());
                }
            }
        } catch (Throwable t) {
            System.out.println("after " + tpe.getPoolSize() + " pool threads: " + t);
        }
        System.exit(0);
    }
}
```

Reproduction (cgroup v1 pids controller, limit 60) TERMINAL

```
$ mkdir /sys/fs/cgroup/pids/osw_o2 && echo 60 > /sys/fs/cgroup/pids/osw_o2/pids.max
$ ( echo $BASHPID > /sys/fs/cgroup/pids/osw_o2/cgroup.procs; exec java -Xmx128m ThreadBomb.java )
submitted 20, pool size 20
[0.712s][warning][os,thread] Failed to start thread "Unknown thread" - pthread_create
  failed (EAGAIN) for attributes: stacksize: 1024k, guardsize: 0k, detached.
[0.712s][warning][os,thread] Failed to start the native thread for java.lang.Thread
  "pool-1-thread-38"
after 37 pool threads: java.lang.OutOfMemoryError: unable to create native ...
$ cat /sys/fs/cgroup/pids/osw_o2/pids.events
max 4
```

The JVM log names the failing system call: `pthread_create failed (EAGAIN)`. The pool stopped at 37 threads because the other 23 tasks of the 60 allowed were the JVM's own threads and the source launcher's. In production the commands are `ls /proc/PID/task | wc -l` (or `jcmd PID Thread.print | grep -c '^"'`) to count threads and `cat pids.current pids.max` for the limit; a thread dump shows hundreds of identical `pool-N-thread-M` threads in `TIMED_WAITING` inside the same client call.

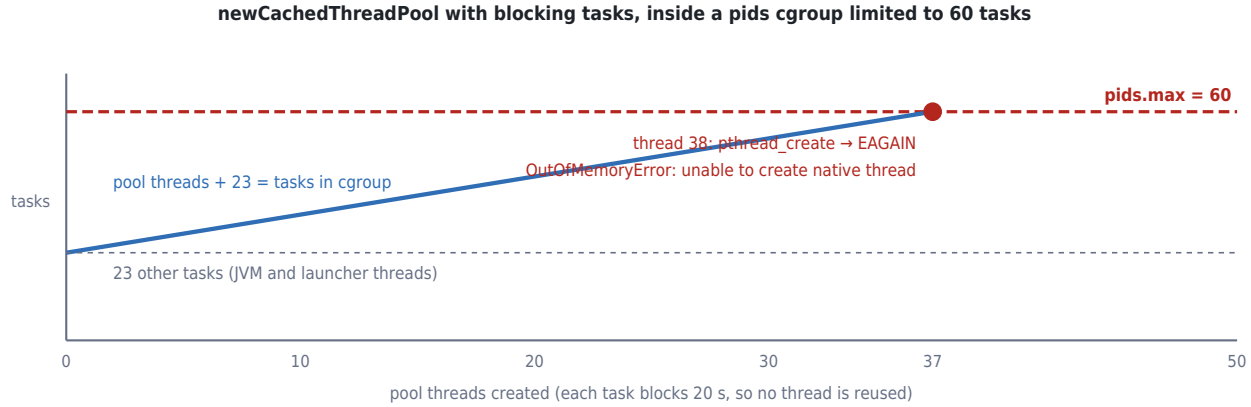


Figure 2.21. The cached pool creates one thread per blocked task, so the task count grows linearly with the arrival rate until the pids limit rejects thread 38; the heap is irrelevant to the failure.

Root cause. `newCachedThreadPool` has no upper bound on threads. When tasks block longer than they arrive, each new task creates a thread, and the number of threads equals the number of outstanding calls. The failure is thus a missing bound on concurrency, combined with a downstream without a timeout.

Fix. Immediate: restart the pod, reduce the inbound rate or break the circuit to the failing dependency. Permanent: replace the pool by a bounded `ThreadPoolExecutor` with a bounded queue and a rejection policy (`CallerRunsPolicy` or an explicit error), apply timeouts to the client (connection and read timeouts) and use a bulkhead per dependency. On Java 21, virtual threads remove the cost of a thread per blocked call, but the number of concurrent calls to the dependency must still be limited with a semaphore.

Prevention. Export `jvm_threads_live_threads` and `pids.current`, alert on a rising thread count, forbid unbounded executors in static analysis, and test the service against a slow dependency.

2.4.5 Questions to expect

Questions and model answers

1. **What is the difference between a process and a thread?** A process owns an address space, descriptors and handlers; threads of a process share them and have their own stack, registers and scheduling state. In Linux both are tasks, created by `clone` with different sharing flags.
2. **Why are threads cheaper than processes?** Creation does not copy page tables, switching between threads of one process does not change the address space, and communication through shared memory needs no system call.
3. **Why is a thread more dangerous than a process?** All threads share the heap, so a race or a wild write in one corrupts the others, and a crash of one kills the whole process.
4. **How does a Java platform thread map to the operating system?** One-to-one: `Thread.start` calls `pthread_create`, which calls `clone`; the thread appears as an entry in `/proc/PID/task` and is scheduled independently.
5. **How do virtual threads differ?** The JVM schedules them on a small pool of carrier threads and unmounts them when they block in supported operations; the kernel sees only the carriers, so ten thousand virtual threads need a few tasks.
6. **What does `-Xss` set and what limits the number of threads?** The stack size of new Java threads (default 1 MiB). The number of threads is limited by the `pids` cgroup, `RLIMIT_NPROC`, `kernel.threads-max`, `vm.max_map_count` and memory for stacks that are touched.
7. **What is a context switch and why is it expensive?** Saving and restoring processor state under kernel control, about one microsecond directly, plus the lost cache and TLB contents. Switching between processes also changes the address space.
8. **What are voluntary and non-voluntary context switches?** A voluntary switch occurs when a task blocks; a non-voluntary one when the scheduler preempts it. Many non-voluntary switches indicate CPU contention, many voluntary ones indicate blocking.
9. **When is a thread pool better than thread-per-request, and when an event loop?** A pool bounds memory and switches; an event loop suits very many idle connections but needs non-blocking code. Virtual threads give thread-per-request code without the cost.

10. **How do you find out how many threads a JVM has?** `ls /proc/PID/task | wc -l`, `jcmd PID Thread.print`, `ps -L -p PID`, or the metric `jvm.threads.live`.

KEY POINTS

A Linux thread is a task created by `clone` that shares the address space, descriptors and handlers of its process; the kernel schedules threads, not processes.

- Java platform threads are one-to-one with kernel tasks (`/proc/PID/task`); virtual threads are multiplexed on a few carrier tasks.
- A context switch costs about a microsecond directly plus cache and TLB losses; creating a platform thread costs 100 to 300 microseconds, which justifies pooling.
- The default Java stack is 1 MiB of virtual address space; the practical limit on threads is usually the `pids` cgroup or `RLIMIT_NPROC`.
- An unbounded pool with blocking tasks turns slowness of a dependency into thread exhaustion; bound the pool and the calls.

2.5 CPU Scheduling

A computer has few processors and many runnable threads. The part of the kernel that decides which thread runs on which processor, and for how long, is the **scheduler**. Its decisions determine the response time of an interactive service, the throughput of a batch job and the fairness between tenants of one machine. This chapter defines the criteria by which scheduling policies are judged, works through the classical algorithms with numbers that a short program verifies, and then explains how Linux schedules ordinary and real-time tasks, what the load average measures, and how to diagnose the incidents that scheduling produces.

2.5.1 Concept and mechanism

Goals and metrics

A scheduler is evaluated with a small set of measurable quantities, which conflict with each other. For one job with arrival time `a`, first-run time `f` and completion time `c`, and a total CPU demand (burst) `b`:

- **Turnaround time** is `c - a`: how long the job takes from submission to completion.
- **Waiting time** is `c - a - b`: the time spent runnable but not running.
- **Response time** is `f - a`: how long until the job first receives the CPU. It is the quantity that interactive users perceive.
- **Throughput** is the number of jobs completed per unit of time, and **CPU utilisation** the fraction of time the processors do useful work.
- **Fairness** requires that jobs of equal importance receive equal shares and that no job waits indefinitely (**starvation**).

A scheduler may be **non-preemptive**, so that a running job keeps the CPU until it blocks or ends, or **preemptive**, so that the kernel can take the CPU away, on a timer interrupt or when a more important job becomes runnable. General-purpose systems are preemptive. Every switch costs time, measured earlier at about a microsecond plus the cache effects, so a good policy limits the number of switches.

The classical algorithms

First-come, first-served (FCFS) runs jobs in arrival order, without preemption. It is simple and has no starvation, but a long job delays all jobs behind it (the convoy effect).

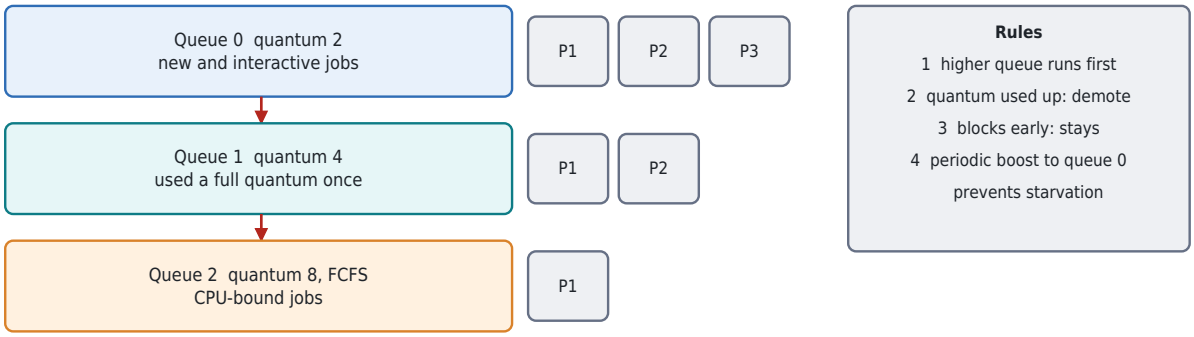
Shortest job first (SJF) runs the job with the smallest burst among those that are ready. It minimises the average waiting time of a given set of jobs, and its preemptive form, **shortest remaining time first (SRTF)**, does so even better. Its weakness is that burst lengths are not known in advance (systems estimate them from past behaviour), and long jobs can starve.

Priority scheduling runs the job with the highest priority. Priorities may be fixed or computed. A low priority job can starve, which is countered by **aging**: the priority of a waiting job increases with time.

Round robin (RR) gives each ready job a fixed **time quantum** in turn and puts it at the end of the queue when the quantum expires. It bounds the response time to about `(n - 1)` quanta for `n` jobs. If the quantum is very large it degenerates to FCFS; if it is very small the switching overhead dominates. A quantum of several milliseconds is typical.

sreeram

Multilevel feedback queues (MLFQ) combine these ideas without knowing burst lengths. A job starts in the top queue, which has the shortest quantum. A job that uses its whole quantum is demoted to the next queue; a job that blocks early stays. Short and interactive jobs therefore finish in the upper queues, and CPU-bound jobs sink to the bottom. A periodic boost to the top queue prevents starvation.



Short and I/O-bound jobs finish in the upper queues; long jobs sink. The scheduler needs no knowledge of burst lengths.

Linux does not use MLFQ for normal tasks; it appears in other schedulers and in the interview syllabus.

Figure 2.22. Three queues with growing quanta: new jobs enter at the top, a job that consumes its whole quantum moves down, and a job that blocks before the quantum ends keeps its level.

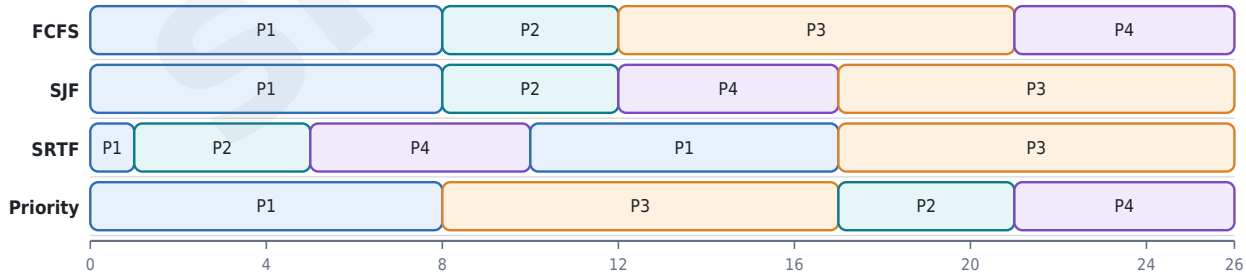
2.5.2 Worked trace

Four jobs arrive as in the table. The priority column is used by the priority algorithm only (1 is the best).

Job	Arrival	Burst	Priority
P1	0	8	3
P2	1	4	2
P3	2	9	1
P4	3	5	4

Workload for the scheduling examples (time units).

The schedules were computed by the program `sched_sim.py` (output and the round-robin function are in the hands-on section), and the Gantt charts are drawn from its output.

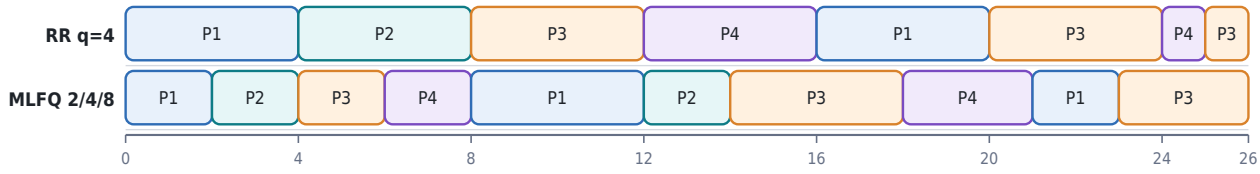


Average waiting time: FCFS 8.75 SJF 7.75 SRTF 6.50 Priority 10.00 (computed by sched_sim.py)

P1 arrives 0 (burst 8), P2 at 1 (4), P3 at 2 (9), P4 at 3 (5); priority order P3 > P2 > P1 > P4.

Figure 2.23. The same four jobs scheduled by four policies; SRTF achieves the lowest average waiting time because P2 and P4 overtake the long P1, whereas FCFS delays them behind it.

Verification by hand for FCFS: P1 runs from 0 to 8 without waiting. P2 arrives at 1 and starts at 8, so it waits 7. P3 arrives at 2 and starts at 12 (after P2), so it waits 10. P4 arrives at 3 and starts at 21, so it waits 18. The average is (0 + 7 + 10 + 18) / 4 = 8.75. For SRTF: at time 1 P2 arrives with burst 4 against the 7 that P1 has left, so P1 is preempted; P2 runs 1 to 5; at 5 the candidates are P1 (7 left), P3 (9) and P4 (5), so P4 runs 5 to 10; P1 then runs to 17 and P3 to 26. P1 waited 9 (it ran 0 to 1 and 10 to 17, arrival 0, completion 17, burst 8).



RR q=4: average wait 11.75, average response 4.50 **MLFQ: average wait 13.00, average response 1.50**

Round robin and MLFQ lose on turnaround but give every job its first CPU time within 0 to 9 time units.

Figure 2.24. Round robin (quantum 4) and a three-level MLFQ give the worst averages for waiting time but the best response: in the MLFQ run every job receives the CPU within three time units of arrival.

The comparison teaches the central trade-off. SRTF minimises waiting but needs to know the future and starves long jobs. Round robin and MLFQ lose on average turnaround because of interleaving, but bound the time to first service. A server prefers the second property, because users notice a request that does not start.

2.5.3 Hands-on

The program `sched_sim.py` implements FCFS, SJF, SRTF, priority, round robin and the MLFQ, prints the segments and the metrics, and was used to check the figures above. The round-robin core is listed; the convention is that jobs arriving while another runs are queued before the preempted job.

rr.py PYTHON

```python
from collections import deque

def round_robin(procs, q):
    """procs: {name: (arrival, burst)}; returns [(name, start, end)] segments."""
    rem = {p: b for p, (a, b) in procs.items()}
    order = sorted(procs, key=lambda k: procs[k][0])
    t, gantt, queue, admitted = 0, [], deque(), set()

    def admit(upto):                        # arrivals up to time t join the queue first
        for p in order:
            if p not in admitted and procs[p][0] <= upto:
                queue.append(p); admitted.add(p)

    admit(0)
    while queue:
        p = queue.popleft()
        run = min(q, rem[p])
        gantt.append((p, t, t + run)); t += run; rem[p] -= run
        admit(t)                            # newcomers queue before the preempted job
        if rem[p] > 0:
            queue.append(p)
    return gantt

jobs = {"P1": (0, 8), "P2": (1, 4), "P3": (2, 9), "P4": (3, 5)}
print(round_robin(jobs, 4))
```

Output of sched_sim.py (waiting = completion - arrival - burst) TERMINAL

```
$ python3 sched_sim.py
FCFS: P1[0-8] P2[8-12] P3[12-21] P4[21-26]
   wait    {'P1': 0, 'P2': 7, 'P3': 10, 'P4': 18} avg 8.75
SJF (non-preemptive): P1[0-8] P2[8-12] P4[12-17] P3[17-26]
   wait    {'P1': 0, 'P2': 7, 'P3': 15, 'P4': 9} avg 7.75
SRTF (preemptive SJF): P1[0-1] P2[1-5] P4[5-10] P1[10-17] P3[17-26]
   wait    {'P1': 9, 'P2': 0, 'P3': 15, 'P4': 2} avg 6.50
Priority (non-preemptive): P1[0-8] P3[8-17] P2[17-21] P4[21-26]
   wait    {'P1': 0, 'P2': 16, 'P3': 6, 'P4': 18} avg 10.00
RR q=4: P1[0-4] P2[4-8] P3[8-12] P4[12-16] P1[16-20] P3[20-24] P4[24-25] P3[25-26]
   wait    {'P1': 12, 'P2': 3, 'P3': 15, 'P4': 17} avg 11.75
   resp    {'P1': 0, 'P2': 3, 'P3': 6, 'P4': 9} avg 4.50
MLFQ 2/4/8: P1[0-2] P2[2-4] P3[4-6] P4[6-8] P1[8-12] P2[12-14] P3[14-18]
   P4[18-21] P1[21-23] P3[23-26]
   wait    {'P1': 15, 'P2': 9, 'P3': 15, 'P4': 13} avg 13.00
   resp    {'P1': 0, 'P2': 1, 'P3': 2, 'P4': 3} avg 1.50
```

sreeram

2.5.4 Questions to expect

Questions and model answers

1. **What do turnaround, waiting and response time mean?** Turnaround is completion minus arrival; waiting is turnaround minus burst; response is the time from arrival to the first run.
2. **Which algorithm minimises average waiting time, and what is its drawback?** SJF (and SRTF when preemption is allowed). It needs burst lengths in advance and can starve long jobs.
3. **What is the convoy effect?** Short jobs queue behind a long CPU-bound job under FCFS, which inflates average waiting.
4. **What determines the time quantum in round robin?** A compromise: too long approaches FCFS and hurts response, too short wastes time on context switches. Several milliseconds is typical.
5. **How does MLFQ approximate SJF without knowing burst lengths?** New jobs start at the top level with a short quantum; a job that uses its full quantum is demoted, so short and interactive jobs finish early. A periodic boost prevents starvation.

KEY POINTS

A scheduler trades waiting, response, throughput and fairness; the classical policies show each trade-off in numbers.

- FCFS is simple but suffers the convoy effect; SJF and SRTF minimise average waiting but need burst knowledge and can starve long jobs.
- Round robin bounds response time by the quantum; MLFQ gets the same effect adaptively, demoting jobs that use full quanta.
- In the worked example the average waiting time was 8.75 (FCFS), 7.75 (SJF), 6.50 (SRTF), 10.00 (priority), 11.75 (RR) and 13.00 (MLFQ), while the average response time was 8.75, 7.75, 4.25, 10.00, 4.50 and 1.50.

2.6 Scheduling in Linux

The algorithms of the previous chapter are textbook models. Linux combines proportional sharing, per-CPU queues, priority classes and cgroup accounting, and exposes the result through `nice`, `taskset`, `/proc` and a load average that is widely misread. This chapter describes the Linux scheduler on the kernel version of the build machine, derives the CPU shares that nice values produce, explains the load average and applies the knowledge to four incidents.

2.6.1 Concept and mechanism

The Linux scheduler

Linux separates scheduling into **classes**, which the kernel examines in a fixed order: the deadline class (`SCHED_DEADLINE`), the real-time class (`SCHED_FIFO` and `SCHED_RR`), the fair class for ordinary tasks (`SCHED_OTHER`, also called `SCHED_NORMAL`, with the variants `SCHED_BATCH` and `SCHED_IDLE`). A runnable task of a higher class always runs before any task of a lower class. Nearly all server processes, including every JVM, belong to the fair class.

The fair class was the **Completely Fair Scheduler (CFS)** until Linux 6.5. Since Linux 6.6 it is the **Earliest Eligible Virtual Deadline First (EEVDF)** scheduler; the build machine runs kernel 6.18 and therefore EEVDF. Both share the idea of **proportional sharing**: every task has a weight derived from its nice value, and over time a task receives a share of the CPU proportional to its weight divided by the sum of the weights of the runnable tasks on that CPU. EEVDF adds a rule for latency. The kernel tracks for each task its **lag**, the difference between the CPU time it deserved and the time it received; a task is **eligible** when its lag is zero or positive. Each eligible task has a **virtual deadline**, roughly the point in virtual time at which its current slice should be complete, and the scheduler runs the eligible task with the earliest deadline. A task that asks for a shorter slice therefore gets an earlier deadline and runs sooner, which helps latency-sensitive tasks without a separate priority.

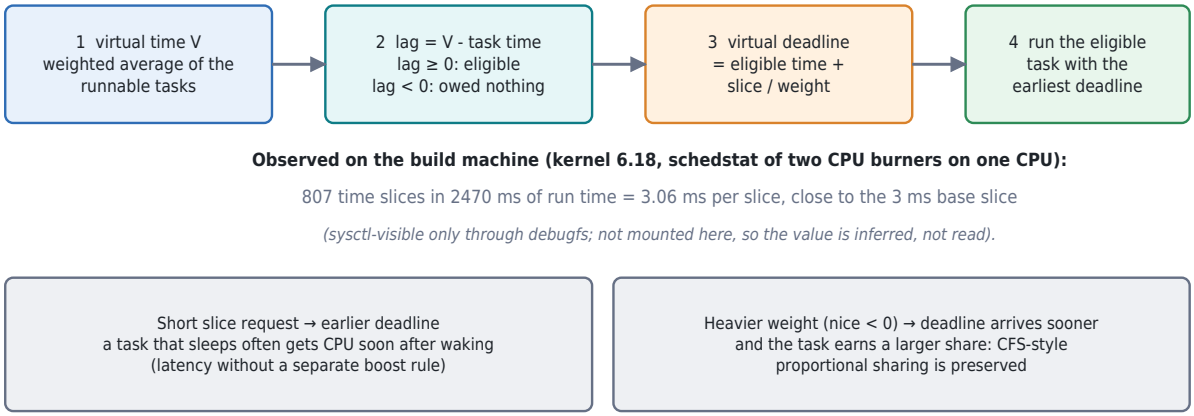


Figure 2.25. The fair scheduler picks the eligible task with the earliest virtual deadline; the measured average slice of two competing CPU burners was 3.06 ms.

Every CPU has its own **run queue**, so that scheduling decisions need no global lock. A periodic **load balancer** migrates tasks from busy queues to idle ones. A task's **CPU affinity** (a mask in `Cpus_allowed_list`) restricts the CPUs that it may use, which `taskset` and `sched_setaffinity` set.

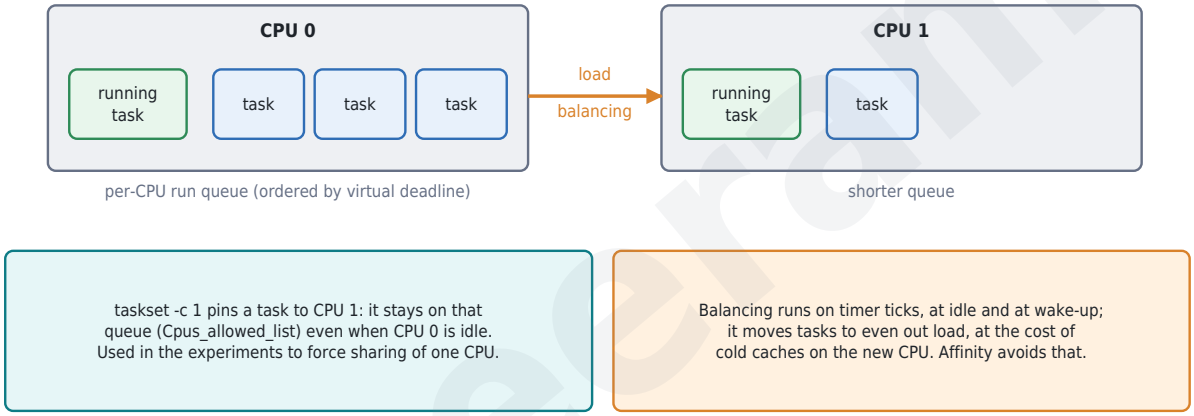
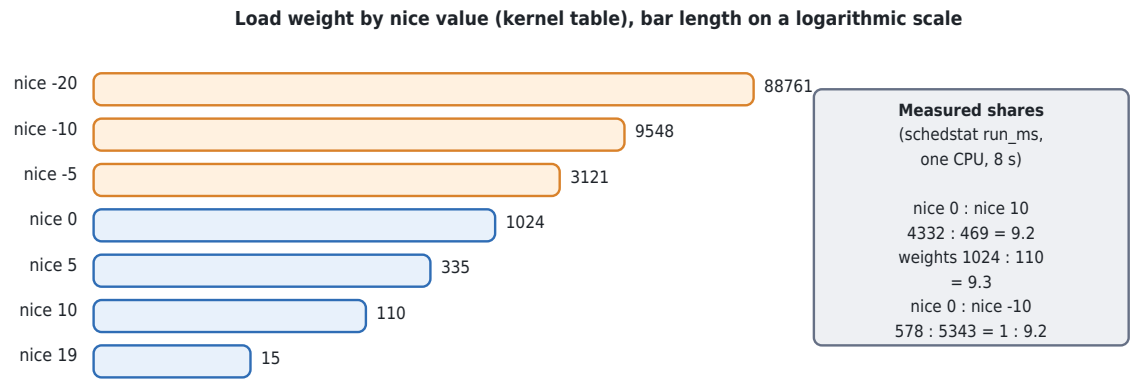


Figure 2.26. Each CPU orders its runnable tasks in its own queue; the balancer moves tasks between queues, and affinity pins a task to a queue.

Nice values and weights

The nice value ranges from -19 (most favoured) to 19 and is changed with `nice` and `renice`; a negative value requires `CAP_SYS_NICE` (root). Each nice step changes a task's weight by about 25 percent, from the kernel table in which nice 0 has weight 1024, nice 10 has 110 and nice -10 has 9548. Two CPU-bound tasks on one CPU therefore share it in the ratio of their weights.



Each nice step changes the weight by about 25 percent; ten steps change it by a factor of about 9.

Figure 2.27. Weights follow a geometric series; nice 0 against nice 10 gives about 9.3 : 1, which the measured CPU times of two competing burners reproduce within 1 percent.

Real-time policies and the other classes

`SCHED_FIFO` and `SCHED_RR` use static priorities from 1 to 99 (`chrt -m` lists the ranges on the build machine: FIFO and RR 1 to 99, the others 0). A real-time task preempts every fair task and runs until it blocks (FIFO) or exhausts a quantum among tasks of the same priority (RR). A real-time task that never blocks would lock a CPU, so the kernel limits the real-time share: `/proc/sys/kernel/sched_rt_runtime_us` is 950000 per `sched_rt_period_us` of 1000000, which reserves 5 percent for ordinary tasks. `SCHED_BATCH` marks CPU-bound non-interactive tasks, and `SCHED_IDLE` runs a task only when nothing else is runnable. The experiment below changes only a sleeping process, because a spinning real-time process could freeze a shared machine.

Inspecting and changing the policy of a sleeping process TERMINAL

```
$ chrt -m
SCHED_OTHER min/max priority    : 0/0
SCHED_FIFO min/max priority     : 1/99
SCHED_RR min/max priority       : 1/99
SCHED_BATCH min/max priority    : 0/0
SCHED_IDLE min/max priority     : 0/0
SCHED_DEADLINE min/max priority : 0/0
$ sleep 30 & chrt -r -p 5 $! && ps -o pid,cls,pri,ni,rtprio,comm -p $!
  PID CLS PRI  NI RTPRIO COMMAND
 7064  RR  45   -      5 sleep
$ chrt -b -p 0 7064 && chrt -p 7064
pid 7064's current scheduling policy: SCHED_BATCH
pid 7064's current scheduling priority: 0
```

Load average

The three numbers printed by `uptime` and `/proc/loadavg` are exponentially damped averages, over 1, 5 and 15 minutes, of the number of tasks that are runnable (state R) or in uninterruptible sleep (state D), sampled every five seconds. The one-minute value is updated as `load = load * e + n * (1 - e)` with `e = exp(-5/60)`, about 0.92, where `n` is the instantaneous count. If `n` stays constant at 4, the one-minute load reaches 63 percent of the way to 4 after one minute and 95 percent after three minutes.

Two points make the value frequently misread. It counts demand, not utilisation: with two CPUs, a load of 4 means that on average two tasks are waiting for a CPU. And it includes tasks in D state, so that a machine whose processes wait for a slow disk shows a high load with idle CPUs. On Linux the load average therefore means "tasks that want a CPU or a disk", and must be read together with the CPU states. The kernel also exports **pressure stall information** (`/proc/pressure/cpu`, `io`, `memory`), which reports the share of time in which tasks were stalled and is the more direct saturation measure.

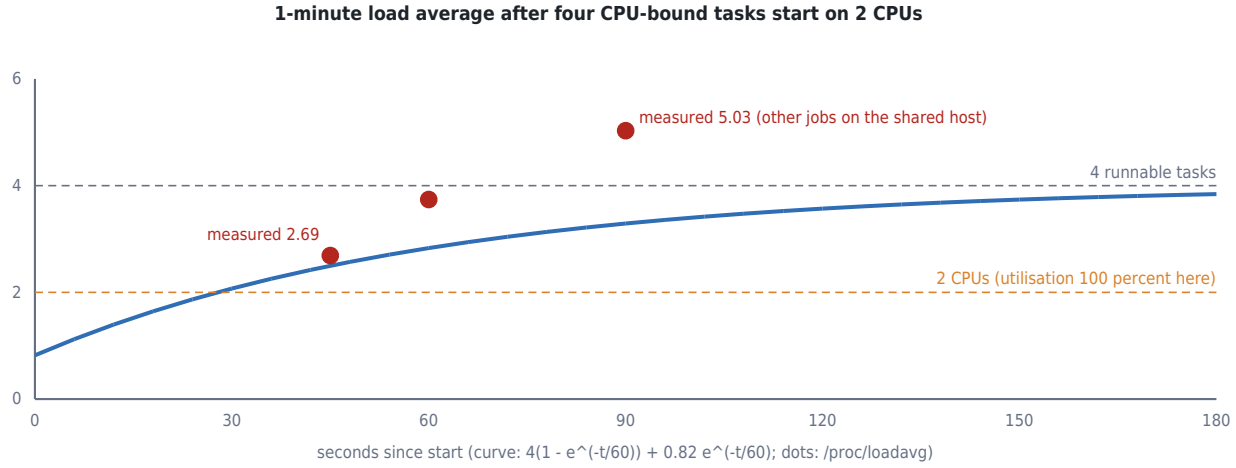


Figure 2.28. Four CPU-bound tasks on two CPUs push the one-minute load toward 4 along an exponential curve; the measured points lie on or above the curve because other jobs shared the build machine.

2.6.2 Worked trace

Two numerical derivations connect the mechanism to the observations of the hands-on section.

CPU share from weights. Two CPU-bound tasks on one CPU have nice 0 (weight 1024) and nice 10 (weight 110). The fair scheduler gives each a share of `weight / (sum of weights)`:

1. Share of the nice 0 task: 1024 / (1024 + 110) = 0.903.
2. Share of the nice 10 task: 110 / 1134 = 0.097.
3. Ratio: 1024 / 110 = 9.31.
4. Measured run times in 8 s: 4332 ms and 469 ms, shares 0.902 and 0.098, ratio 9.24.

The prediction and the measurement agree to within one percent. For the pair nice 0 and nice -10 the weights are 1024 and 9548, the predicted ratio is 1 : 9.32 and the measured run times were 578 ms and 5343 ms, a ratio of 1 : 9.24.

Load average update. The one-minute average is updated every five seconds. With `e = exp(-5/60) = 0.9200` and an instantaneous count of `n = 4` runnable tasks, starting from a load of 0:

1. After one sample: `0 * 0.92 + 4 * 0.08 = 0.32`.
2. After two samples: `0.32 * 0.92 + 0.32 = 0.61`.
3. After twelve samples (60 s): `4 * (1 - e^-1) = 2.53`, the value in the figure.
4. After thirty-six samples (180 s): `4 * (1 - e^-3) = 3.80`.

The five- and fifteen-minute values use `exp(-5/300)` and `exp(-5/900)` and therefore respond slowly. A one-minute value far above the 15-minute value shows a recent rise; the reverse shows a decline.

2.6.3 Hands-on

CPU share by nice value

The program `spin.c` is `int main(void){ for(;;); }`. Two instances are pinned to CPU 1 with `taskset` and given different nice values; the kernel's accounting file `/proc/PID/schedstat` records the run time, the wait time and the number of time slices.

Two burners on one CPU, nice 0 and nice 10, after 8 seconds TERMINAL

```
$ cat nice.sh
taskset -c 1 nice -n 0 ./spin & A=$!
taskset -c 1 nice -n 10 ./spin & B=$!
sleep 8
echo "PID   NI  run_ms  wait_ms  timeslices"
for p in $A $B; do
  read run wait slices < /proc/$p/schedstat
  echo "$p $(ps -o ni= -p $p)  $((run/1000000))  $((wait/1000000))  $slices"
done
kill $A $B; wait 2>/dev/null
```

Sreeram

Two burners on one CPU, nice 0 and nice 10, after 8 seconds (continued) TERMINAL

```
$ ./nice.sh
PID   NI  run_ms  wait_ms  timeslices
3531   0  4332  3631  3746
3532  10  469  7522  388
```

The run times are in the ratio 4332 : 469 = 9.2 : 1, against a weight ratio of 1024 : 110 = 9.3 : 1. The two tasks together received 4801 ms of the 8000 ms, since other processes of the shared build machine used part of CPU 1; only the ratio is meaningful. The task with nice 10 still ran: nice sets a share, not an exclusion.

Voluntary and non-voluntary context switches

Two spinning tasks and one task that wakes every 10 ms share CPU 1. The counters in `/proc/PID/status` and `schedstat` show how each behaves.

Two CPU-bound tasks and one interactive task on one CPU, after 5 seconds TERMINAL

```
$ for p in $A $B $C; do echo "$(ps -o comm= -p $p) $p $(grep ctxt /proc/$p/status|xargs)"; done
spin 7347 voluntary_ctxt_switches: 1 nonvoluntary_ctxt_switches: 805
spin 7348 voluntary_ctxt_switches: 0 nonvoluntary_ctxt_switches: 823
ticker 7349 voluntary_ctxt_switches: 497 nonvoluntary_ctxt_switches: 1
$ cat /proc/7347/schedstat      # run ns, wait ns, time slices
2470013000 2516000000 807
```

The burners are always preempted by the timer (about 800 non-voluntary switches, each slice 2470 ms / 807 = 3.06 ms), the sleeper blocks voluntarily 497 times in 5 seconds (once per 10 ms) and was preempted once. A thread with many non-voluntary switches competes for CPU; a thread with many voluntary switches mostly waits for something else. The sleeper accumulated only 8 ms of run time and 24 ms of waiting in five seconds: it was always scheduled promptly after waking.

Load average against utilisation

Four spinning tasks on two CPUs, load average sampled while they run (approximate times) TERMINAL

```
$ cat /proc/loadavg
0.82 1.70 1.22 1/109 7483
$ for i in 1 2 3 4; do (ulimit -t 62; exec ./spin) & done
$ cat /proc/loadavg        # after about 45 s
2.69 2.08 1.37 5/97 7838
$ cat /proc/loadavg        # after about 60 s
3.74 2.34 1.47 5/98 7955
$ cat /proc/loadavg        # after about 90 s (the spinners had already stopped at 62 s of CPU)
5.03 2.79 1.64 5/97 8225
```

The one-minute value rose from 0.82 along the curve of the figure. The last value is above 4 because other writers' jobs ran on the shared machine, and shows the limit of the experiment: load is a host-wide count, so on a shared host it cannot be attributed to one workload. The fourth field, `5/97`, is the instantaneous count of runnable tasks over the total number of tasks.

2.6.4 Real-world incidents

Incident: load average of 20 on a host with idle CPUs

Scenario. A host that runs several Spring Boot batch services alerts on a one-minute load average above 20 on 8 CPUs. Dashboards show CPU use at 10 percent, and the services respond slowly.

Symptoms. `uptime` reports a high and rising load, `top` shows a large `wa` value and few running tasks. Requests that touch the local database or write temporary files are slow, while pure in-memory endpoints are fast.

Diagnosis. The load counts R and D tasks, so with idle CPUs the missing term is D. The first command shows the states of all tasks; the following ones identify what they wait for. The reproduction uses four `dd` processes that write synchronously to a file, which saturates the disk.

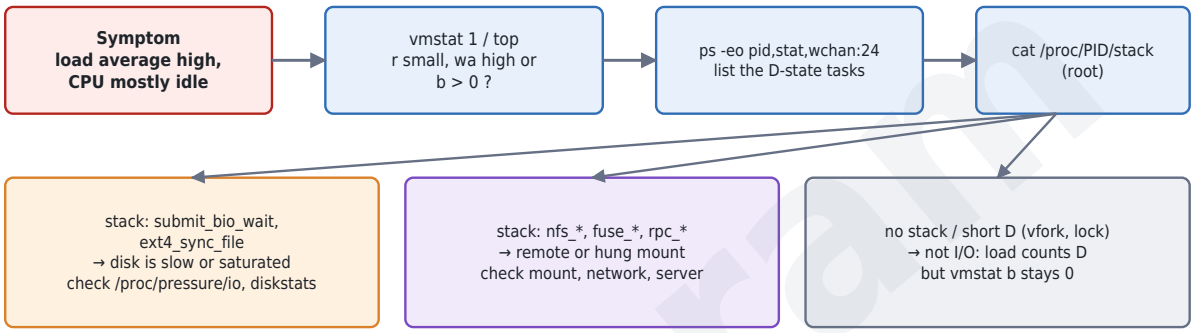
Reproduction: four synchronous writers; commands in the order of use TERMINAL

```
$ vmstat 1 3
procs -----------memory---------- ---swap-- -----io---- -system-- -------cpu-------
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st gu
 5  1      0 7146740  13092 758856    0    0  2558  9263 4474  176 25  6 67  2  0  0
 5  0      0 7084244  13092 797300    0    0     0 99200 26434 75333 13 35  2 49  1  0
```

Reproduction: four synchronous writers; commands in the order of use (continued) TERMINAL

```
 5  2      0 7021276  13092 835824    0    0     0 96260 24568 73475 30 36  6 28  1  0
$ ps -eo pid,stat,wchan:24,comm | awk '$2 ~ /^D/'
 5476 D    -                        dd
$ cat /proc/5476/stack
[<0>] submit_bio_wait+0x74/0xc0
[<0>] blkdev_issue_flush+0x38/0x60
[<0>] ext4_sync_file+0x23f/0x2d0
$ cat /proc/pressure/io
some avg10=28.45 avg60=14.84 avg300=5.20 total=22604424
full avg10=16.23 avg60=9.41 avg300=3.53 total=16037391
```

The `wa` column (28 to 49 percent) is the share of time in which CPUs were idle while tasks waited for I/O, and `b` counts tasks blocked on I/O. The kernel stack of a D-state process is the decisive evidence: `ext4_sync_file` and `submit_bio_wait` mean that the task waits for the block device to complete a flush. The `some` line of `/proc/pressure/io` is the share of time in which at least one task was stalled on I/O. (The `r` column is high in this run because other jobs ran on the shared machine.) Note that `vmstat`'s `b` counts only I/O waits: a task in D for another reason, such as the parent of `vfork`, raises the load average but leaves `b` at zero, as an extra experiment with eight such tasks showed.



Load average counts runnable tasks (R) and uninterruptible tasks (D); utilisation counts only CPU time.

Reproduction: four dd processes with oflag=dsync gave 21 to 49 percent iowait; /proc/PID/stack showed submit_bio_wait → blkdev_issue_flush → ext4_sync_file. Eight vfork parents showed D but vmstat b = 0.

Figure 2.29. Decision path for high load with idle CPUs: confirm D-state tasks, read what they wait for in the kernel stack, and branch to the disk, the network file system or a non-I/O wait.

Root cause. The batch services issued synchronous writes (`fsync` after every record) to a volume whose throughput was shared with a backup. Tasks queued behind the device in state D, which raised the load average without using CPU time.

Fix. Immediate: pause the competing backup or move it off-peak, and reduce the number of concurrent writers. Permanent: batch the commits (group several records per `fsync`), place the write-ahead data on a faster volume, and make the application report the time spent in I/O waits.

Prevention. Alert on pressure (`/proc/pressure/io some avg60`) and on device utilisation and latency (computed from `/proc/diskstats`, see the part on storage), not on the load average alone. Document that a load average above the CPU count has two possible causes.

Incident: one Java thread uses 100 percent of a core

Scenario. A Spring Boot service that consumes jobs from an in-memory queue shows one core fully busy after a deployment, although the request rate is unchanged. Response times degrade slowly, since one of the two cores of the container is unavailable to request threads.

Symptoms. `top` shows the Java process at about 100 percent CPU (one core); heap and GC are normal; no error appears in the log.

Diagnosis. A process-level percentage does not say which thread consumes it. The method is to list the threads, convert the hottest thread's ID and find that thread in a thread dump. The reproduction uses `HotThread.java`, which contains a busy-wait loop on `ConcurrentLinkedQueue.poll()` in a thread named `order-consumer`.

HotThread.java JAVA

```java
import java.util.concurrent.ConcurrentLinkedQueue;

public class HotThread {
    static final ConcurrentLinkedQueue<String> jobs = new ConcurrentLinkedQueue<>();

    public static void main(String[] args) throws Exception {
        Thread idle = new Thread(() -> {
            try { Thread.sleep(600_000); } catch (InterruptedException e) { }
        }, "http-nio-8080-exec-1");
        idle.start();
        Thread hot = new Thread(HotThread::consume, "order-consumer");
        hot.start();
        hot.join();
    }

    // Bug: a busy-wait consumer. poll() returns null immediately when the queue is empty,
    // so the loop never blocks and one core is fully used.
    static void consume() {
        long handled = 0;
        while (true) {
            String job = jobs.poll();
            if (job != null) {
                handled++;
            }
        }
    }
}
```

Find the hot thread (pid 1228 is the JVM; the output is from the real reproduction) TERMINAL

```
$ top -H -b -n2 -d1 -o %CPU -p 1228 -w 120 | awk '/^top -/{n++} n==2' | sed -n 7,10p
  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
 1248 root      20   0 4147972  40620  23144 R  99.9   0.5   0:05.17 order-consumer
 1228 root      20   0 4147972  40620  23144 S   0.0   0.5   0:00.00 java
 1230 root      20   0 4147972  40620  23144 S   0.0   0.5   0:00.02 java
$ printf '%x\n' 1248
4e0
$ jstack 1228 | grep -A4 'order-consumer'
"order-consumer" #19 [1248] prio=5 os_prio=0 cpu=5451.36ms elapsed=5.56s tid=0x00007f6e8012e360
  nid=1248 runnable
   java.lang.Thread.State: RUNNABLE
        at HotThread.consume(HotThread.java:25)
        at HotThread$$Lambda/0x00007f6e10000c28.run(Unknown Source)
        at java.lang.Thread.runWith(java.base@21.0.12.1/Thread.java:1596)
$ grep -E '^(State|nonvoluntary)' /proc/1228/task/1248/status
State:  R (running)
nonvoluntary_ctxt_switches:     286
```

The `-H` option of `top` lists threads, and the thread ID of the hot one is 1248, which is the `Pid` field of a thread in `/proc`. A thread dump identifies each thread by its native ID, written `nid`. Older JDK versions print `nid` in hexadecimal (`nid=0x4e0`), so the conversion with `printf '%x'` is required; the JDK 21 build used here prints it in decimal (`nid=1248`), and the `[1248]` in the header gives the same number. The check is therefore to match the number in the dump format actually produced. The `cpu=` field in the dump is the thread's accumulated CPU time and, compared between two dumps, confirms the hot thread. The thread is `RUNNABLE` at line 25 of `consume`, inside a loop. The thread never sleeps, so `voluntary_ctxt_switches` is almost zero, and the preemptions by the timer (`nonvoluntary`) are the only switches.

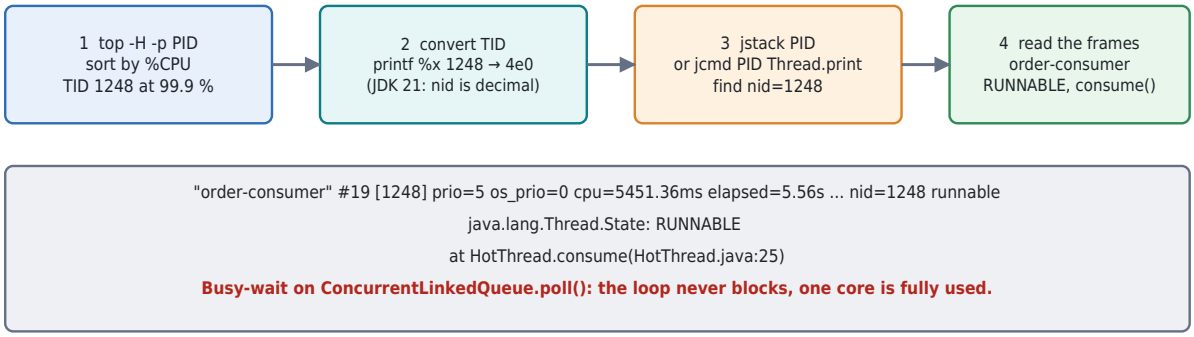


Figure 2.30. The four steps map the symptom from process to thread to source line: list threads by CPU, convert the ID if needed, find it in a thread dump, read the frames.

Root cause. A consumer loop polled a non-blocking queue without waiting, so it spun at full speed when the queue was empty.

Fix. Immediate: restart the instance after taking two thread dumps for the record; the CPU returns to normal. Permanent: replace the busy-wait by a blocking operation (`BlockingQueue.take()` or `poll(timeout)`), or use an executor that is fed by the queue. Add a pause or back-off only if blocking is impossible.

Prevention. Alert on per-thread CPU (a JFR or an APM thread-CPU view), add a load test with an empty queue to the release checks, and review loops that call `poll()` or test a flag without a wait.

Incident: a batch job with negative nice starves the API on the same host

Scenario. To make a nightly export finish earlier, an engineer starts it with `nice -n -10`. The export shares a two-CPU virtual machine with a latency-sensitive API.

Symptoms. While the export runs, API latency grows by an order of magnitude, although total CPU use is the same as on other nights, and the host is not memory-constrained.

Diagnosis. The scheduling statistics of the two processes show who receives the CPU and who waits. The reproduction pins a nice 0 task (the API) and a nice -10 task (the export) to one CPU:

CPU time received in 6 seconds on one CPU — TERMINAL

```
$ ./nice10.sh              # same layout as nice.sh, with nice -n -10 for the second task
PID   NI  run_ms  wait_ms
7304   0  578  5397
7305 -10  5343  591
```

The nice 0 task received 578 ms and waited 5397 ms, about one tenth of the CPU against the favoured task. The ratio, 1 : 9.2, equals the weight ratio 1024 : 9548. On a real host the API threads are runnable only in bursts, so they would still run, but each burst is slowed by the larger competitor on the same queue.

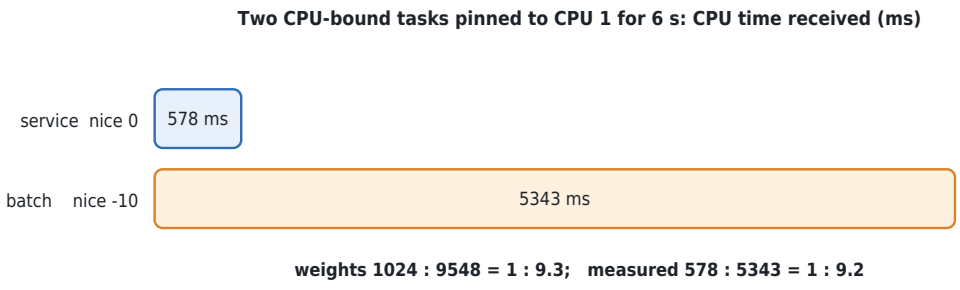


Figure 2.31. A task with nice -10 has about nine times the weight of a nice 0 task; on a shared CPU the favoured task takes about 90 percent of the time while the other waits.

Root cause. A negative nice value was assigned to a background job that shares CPUs with an interactive service, inverting the intended priority.

Fix. Immediate: `renice -n 19 -p PID` on the export (any user may lower its own priority), or stop the job. Permanent: run batch work at positive nice (or `SCHED_BATCH`/`SCHED_IDLE`), in a separate cgroup with a lower `cpu.shares`, or on a separate node; never raise a batch job above the service.

Prevention. Allow negative nice only for audited services, record the nice value of long jobs in the job metadata, and use cgroup weights in orchestrators instead of per-process nice.

Incident: a cloud instance slows down without any change in its own load

Scenario. A Java service on a shared-tenancy cloud virtual machine, unchanged for weeks, becomes 30 to 40 percent slower in the afternoon. Application metrics show normal request rates and no GC change.

Symptoms. CPU use appears moderate, but response times rise; `top` shows the field `st` (steal) at 15 to 25 percent.

Diagnosis. Steal time is the time during which the virtual CPU was ready to run but the hypervisor ran another guest. The guest cannot see the cause; it can see the amount. The sources are the `st` column of `vmstat` and `top` and the eighth value on the `cpu` line of `/proc/stat`, in clock ticks.

Steal on the build machine and how to compute it over an interval TERMINAL

```
$ head -1 /proc/stat
cpu  41837 85 13194 120910 4151 0 1953 812 0 0
$ python3 -c "..."          # two readings of /proc/stat five seconds apart, deltas in percent
{'user': 5.5, 'nice': 0.0, 'system': 1.1, 'idle': 92.5, 'iowait': 0.0, 'irq': 0.0,
 'softirq': 0.6, 'steal': 0.3}
```

The fields after the label are user, nice, system, idle, iowait, irq, softirq, steal, guest and guest_nice. On the build machine steal is 0.3 percent (812 ticks since boot). In the incident the same computation would show about 20 percent. The decision is made by comparing: if `st` is high while the run queue is short and `us` plus `sy` are moderate, the guest is waiting for physical CPU time, and application changes will not help.

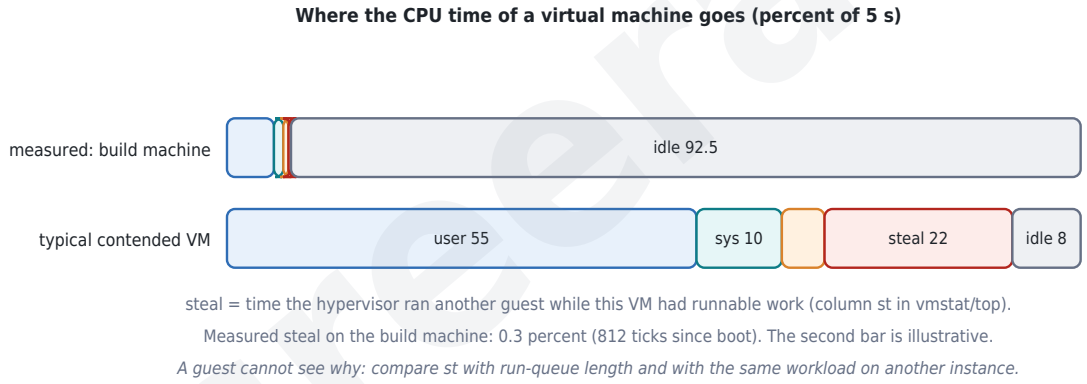


Figure 2.32. Steal appears as a separate slice of the CPU time of a virtual machine; the first bar is measured, the second illustrates an affected instance.

Root cause. Another tenant on the same physical host used CPU time that the provider had overcommitted, or the instance type uses burstable credits that were exhausted.

Fix. Immediate: move the instance to another host (stop and start it), or to a larger or dedicated instance type, and shift traffic away. Permanent: use instance types with dedicated or guaranteed CPUs for latency-sensitive services, avoid burstable types for steady load, and add autoscaling triggered by steal-adjusted utilisation.

Prevention. Record `st` as a metric per node, alert when it exceeds 5 to 10 percent for several minutes, and include it in the first checks of any latency incident before changing application code.

2.6.5 Questions to expect

Questions and model answers

1. **Which scheduler does Linux use for normal tasks?** CFS until 6.5 and EEVDF since 6.6, both proportional-share schedulers with per-CPU run queues. Weights come from nice values; EEVDF adds lag and virtual deadlines for latency.
2. **What does nice do?** It sets a task's weight; each step is about 25 percent, and nice 0 against nice 10 is about 9 :
   1. It changes the share under contention, not whether the task runs.

3. **What does a load average of 8 mean on a 4-CPU machine?** On average eight tasks were runnable or in uninterruptible sleep, so about four were waiting for a CPU or for I/O. The CPU states decide which.
4. **High load, idle CPUs: what is the cause and how do you find it?** Tasks in D state, usually waiting for disk or network file systems. `ps` with state and `wchan`, then `/proc/PID/stack` and the pressure files.
5. **How do you find the Java thread that uses 100 percent of a core?** `top -H` for the thread ID, convert to hexadecimal where the dump uses it, locate it in `jstack` or `jcmd Thread.print` and read the frames.
6. **What are voluntary and non-voluntary context switches in `/proc/PID/status`?** Voluntary: the task blocked. Non-voluntary: the scheduler preempted it. A high non-voluntary count signals CPU competition.
7. **What is steal time?** CPU time that a virtual machine wanted but the hypervisor gave to another guest; visible as `st` in `top` and `vmstat`.

KEY POINTS

Linux schedules ordinary tasks by weight with one run queue per CPU; the load average counts demand, not utilisation.

- Linux schedules ordinary tasks with a proportional-share fair scheduler (EEVDF since 6.6) with one run queue per CPU; real-time classes preempt it and are capped at 95 percent.
- Nice values set weights: about 25 percent per step, 9 : 1 between nice 0 and nice 10 (measured 9.2 : 1).
- Load average counts runnable and D-state tasks, not utilisation; combine it with `vmstat`, per-state task lists and pressure files.
- A hot Java thread is found with `top -H`, the thread ID in the dump (decimal in JDK 21, hexadecimal in older dumps) and two dumps to confirm.
- Steal and negative nice values are scheduling problems that no application change fixes.

2.7 CPU Limits in Containers

A container does not own a processor. It is a group of processes that the kernel places in a **control group** (cgroup), and the CPU controller of that cgroup decides how much processor time the group may use. Kubernetes translates the `requests` and `limits` of a container into two different cgroup settings, and the difference between them explains one of the most frequent causes of unexplained latency in containerised Java services: a service that is throttled while its average CPU use is far below its limit. This chapter defines both mechanisms, shows them with measurements on the build machine, and traces the throttling incident.

2.7.1 Concept and mechanism

Shares and quotas

The CPU controller offers two independent controls.

- **Weight (shares).** In cgroup v1, `cpu.shares` (default 1024) sets the relative weight of the group among its siblings. Weights matter only when CPUs are contended: an idle machine lets any group use any amount of CPU. In cgroup v2 the file is `cpu.weight`. Kubernetes derives the value from the container's CPU **request**: a request of 250 millicores becomes 256 shares.
- **Bandwidth (quota).** In cgroup v1, `cpu.cfs_period_us` (default 100000, that is 100 ms) and `cpu.cfs_quota_us` set a hard cap: in every period the group may consume at most `quota` microseconds of CPU time, summed over all its threads and CPUs. In cgroup v2 the pair is the single file `cpu.max` (`quota period`, or `max period` for no limit). Kubernetes derives the quota from the **limit**: a limit of 500 millicores becomes a quota of 50000 per period of 100000.

The mechanism is **CFS bandwidth control**. The group holds a pool of runtime for the current period; each CPU draws runtime from the pool in slices (5 ms by default, `sched_cfs_bandwidth_slice_us`) as its threads run. When the pool is empty, every thread of the group is **throttled**, removed from the run queues, until the next period begins and the pool is refilled. The statistics file `cpu.stat` counts `nr_periods`, `nr_throttled` and `throttled_time` (nanoseconds) and is the primary diagnostic.

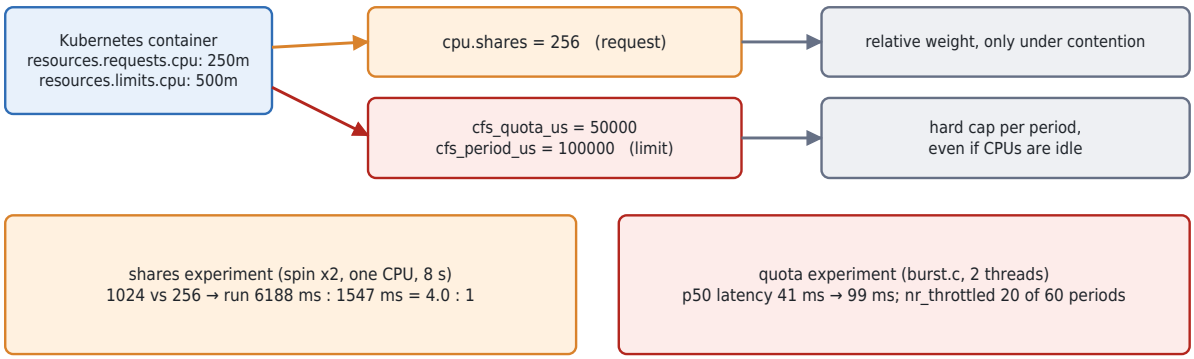


Figure 2.33. A request becomes a relative weight that applies only under contention; a limit becomes a hard per-period cap that applies even when CPUs are idle; the JVM derives its CPU count from the quota.

Why a quota causes latency below the average limit

The quota is spent in wall-clock time at a rate equal to the number of running threads. A group with a quota of 50 ms per 100 ms that runs two threads in parallel exhausts the quota after 25 ms; the threads then wait for the remaining 75 ms of the period, even if the average use over a second is low. A request that needs 40 ms of CPU on each of two threads, which would take 40 ms unthrottled, takes 100 ms or more. Multi-threaded services, which process one request on several threads or many requests at once, hit this effect hardest, and so do JVMs whose garbage collector and compiler threads run in parallel with the application threads and consume quota in short bursts.

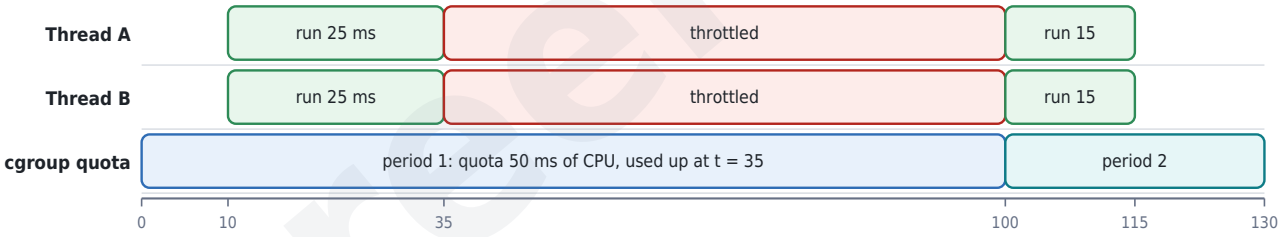


Figure 2.34. Two threads consume the 50 ms quota of a 100 ms period within 25 ms of wall time and are throttled until the next period; a request that needs 40 ms of CPU per thread completes after about 105 ms instead of 40 ms.

What the JVM sees

HotSpot reads the cgroup limits at start-up. It computes the number of available processors as the quota divided by the period, rounded up, and bounded by the CPUs of the host; it uses this number for `Runtime.availableProcessors()`, for the sizes of the common `ForkJoinPool`, the garbage collector's worker threads and the compiler threads. It also chooses the collector: the JVM selects G1 on a "server-class" machine (at least two processors and 1792 MB of memory) and Serial otherwise, and the CPU count that decides this is the container's. The flag `-XX:ActiveProcessorCount=N` overrides the detection.

2.7.2 Worked trace

A quota of 50000 per period of 100000 and two threads that each need 40 ms of CPU, arriving together at time 10 ms of period 1 (the chart above).

1. At t = 10 ms both threads start on two CPUs. Each runs at full speed, consuming CPU time at 2 ms per ms of wall time.
2. The CPUs draw 5 ms slices from the pool of 50 ms. The pool is exhausted when the total CPU time reaches 50 ms, that is after 25 ms of wall time, at t = 35 ms.
3. Each thread has then run 25 ms and needs 15 ms more. Both are throttled; the CPUs are idle although the threads are runnable.
4. At t = 100 ms the period ends and the pool is refilled. The threads resume and finish their remaining 15 ms by t = 115 ms.
5. The latency of the request is 105 ms. Without a limit it is 40 ms.
6. The average use of the cgroup over a longer window can still be low: with one such request per 240 ms the group uses 80 ms of CPU in 240 ms, 0.33 CPU, less than the 0.5 CPU limit.

The last point is why a dashboard of average CPU against the limit does not reveal the problem; only the throttling counters do.

2.7.3 Hands-on

Relative weights with cpu.shares

Two spinning tasks in sibling cgroups with 1024 and 256 shares, both pinned to CPU 1 (cgroup v1 layout) — TERMINAL

```
$ R=/sys/fs/cgroup/cpu; mkdir $R/osw_o2a $R/osw_o2b
$ echo 1024 > $R/osw_o2a/cpu.shares; echo 256 > $R/osw_o2b/cpu.shares
$ ( echo $BASHPID > $R/osw_o2a/cgroup.procs; exec taskset -c 1 ./spin ) &
$ ( echo $BASHPID > $R/osw_o2b/cgroup.procs; exec taskset -c 1 ./spin ) &
$ sleep 8; for p in $(jobs -p); do read run wait sl < /proc/$p/schedstat; \
    echo "pid $p run_ms=$((run/1000000))"; done
pid 6766 run_ms=6188
pid 6767 run_ms=1547
```

The ratio 6188 : 1547 is 4.0 : 1, equal to 1024 : 256. Shares divide the CPU in proportion under contention. Had the second task run alone, it would have used a whole CPU.

A quota and its effect on latency

The program `burst.c` starts two threads that, together, simulate requests: both threads wait at a barrier, then each consumes 40 ms of CPU (measured with the thread CPU clock), and the wall-clock time is the latency; a 200 ms pause separates the bursts.

burst.c — C

```c
#define _GNU_SOURCE
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <unistd.h>

#define THREADS 2
#define ROUNDS 20
#define WORK_MS 40

static double wall_ms(void) {
    struct timespec ts; clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec * 1e3 + ts.tv_nsec / 1e6;
}
static double cpu_ms(void) {
    struct timespec ts; clock_gettime(CLOCK_THREAD_CPUTIME_ID, &ts);
    return ts.tv_sec * 1e3 + ts.tv_nsec / 1e6;
}
static double lat[ROUNDS][THREADS];
static pthread_barrier_t barrier;

static void *worker(void *arg) {
    int id = (int)(long)arg;
```

burst.c (continued) C

```c
    for (int r = 0; r < ROUNDS; r++) {
        pthread_barrier_wait(&barrier);            /* all "requests" arrive together */
        double w0 = wall_ms(), c0 = cpu_ms();
        while (cpu_ms() - c0 < WORK_MS) { }        /* WORK_MS of pure CPU work */
        lat[r][id] = wall_ms() - w0;
        pthread_barrier_wait(&barrier);
        if (id == 0) usleep(200000);               /* idle gap between bursts */
    }
    return NULL;
}
static int cmp(const void *a, const void *b) {
    double x = *(double *)a, y = *(double *)b;
    return (x > y) - (x < y);
}

int main(void) {
    pthread_t t[THREADS];
    pthread_barrier_init(&barrier, NULL, THREADS);
    for (long i = 0; i < THREADS; i++) pthread_create(&t[i], NULL, worker, (void *)i);
    for (int i = 0; i < THREADS; i++) pthread_join(t[i], NULL);
    double all[ROUNDS * THREADS]; int n = 0;
    for (int r = 0; r < ROUNDS; r++) for (int i = 0; i < THREADS; i++) all[n++] = lat[r][i];
    qsort(all, n, sizeof(double), cmp);
    printf("requests=%d  work=%d ms CPU each  latency ms: p50=%.0f p90=%.0f max=%.0f\n",
           n, WORK_MS, all[n / 2], all[n * 9 / 10], all[n - 1]);
    return 0;
}
```

Without a limit, then with a quota of 50000 per 100000 microseconds TERMINAL

```
$ gcc -O2 -pthread -o burst burst.c
$ ./burst
requests=40  work=40 ms CPU each  latency ms: p50=41 p90=75 max=83
$ G=/sys/fs/cgroup/cpu/osw_o2; mkdir $G
$ echo 100000 > $G/cpu.cfs_period_us; echo 50000 > $G/cpu.cfs_quota_us
$ ( echo $BASHPID > $G/cgroup.procs; exec ./burst )
requests=40  work=40 ms CPU each  latency ms: p50=99 p90=104 max=120
$ cat $G/cpu.stat
nr_periods 60
nr_throttled 20
throttled_time 1960110985
nr_bursts 0
burst_time 0
```

What to notice. The median latency rose from 41 to 99 ms, and the spread collapsed, since every request now waits for the next period boundary. `nr_throttled` is 20 of 60 periods and `throttled_time` is 1.96 s (nanoseconds) in a run of about 5 s. The total CPU use is 80 ms per 240 ms cycle, about a third of one CPU, below the limit of half a CPU. On a cgroup v2 system the file is `cpu.stat` with `nr_throttled` and `throttled_usec`, and the limit is set by writing `50000 100000` to `cpu.max`.

The JVM under a quota

Processors and collector chosen by the JVM for three quotas (period 100000) TERMINAL

```
$ java -cp . Cpus                       # no quota
availableProcessors = 2, ForkJoin common pool parallelism = 1
$ echo 50000 > $G/cpu.cfs_quota_us      # run the same command inside the cgroup
availableProcessors = 1, ForkJoin common pool parallelism = 1
$ echo 150000 > $G/cpu.cfs_quota_us
availableProcessors = 2, ForkJoin common pool parallelism = 1
$ java -XX:+PrintFlagsFinal -version | grep -E " (UseSerialGC|UseG1GC|ParallelGCThreads) "
quota -1:     ParallelGCThreads=2  UseG1GC=true  UseSerialGC=false
quota 50000:  ParallelGCThreads=0  UseG1GC=false  UseSerialGC=true
quota 200000: ParallelGCThreads=2  UseG1GC=true  UseSerialGC=false
```

A quota of 0.5 CPU produced one available processor and made the JVM choose the Serial collector, which stops all application threads for every collection and uses no parallel GC threads; a quota of 1.5 CPUs rounds up to 2. (The last three lines combine three runs of the flag listing, one per quota, into one block.) The consequence is that a container limit below two CPUs silently changes the garbage collector and the thread-pool sizes that depend on the processor count, independent of the application code.

2.7.4 Real-world incidents

Incident: latency spikes of 100 ms in a Kubernetes pod with low average CPU

Scenario. A Spring Boot service runs in Kubernetes with `requests.cpu: 500m` and `limits.cpu: 1`. Its 95th-percentile latency is 40 ms most of the time but shows periodic spikes at 100 to 200 ms, and the 99th percentile is worse. The dashboard shows average CPU use of 30 percent of the limit. There is no memory pressure and GC pauses are short.

Symptoms. Latency spikes correlate with traffic bursts, not with deployments. They appear as multiples of about 100 ms. Thread dumps taken during a spike show request threads in `RUNNABLE` state, so the application is not blocked on a lock or I/O. A node with spare CPU shows the same spikes.

Diagnosis. Runnable threads that make no progress while CPUs are free point to throttling. The cgroup statistics of the pod confirm it. In a container the file is under the cgroup of the pod (`/sys/fs/cgroup/cpu.stat` on v2, `/sys/fs/cgroup/cpu/cpu.stat` on v1):

Throttling counters (output of the reproduction, quota 0.5 CPU, run of about 5 seconds) TERMINAL

```
$ cat /sys/fs/cgroup/cpu/cpu.cfs_quota_us /sys/fs/cgroup/cpu/cpu.cfs_period_us
50000
100000
$ cat /sys/fs/cgroup/cpu/cpu.stat
nr_periods 60
nr_throttled 20
throttled_time 1960110985
$ python3 -c "print('throttled periods: %.0f%%' % (100 * 20 / 60))"
throttled periods: 33%
```

One third of the periods ended with the group throttled, in total 1.96 s of throttled wall time over about 5 s. In Prometheus the same information is the ratio `container_cpu_cfs_throttled_periods_total / container_cpu_cfs_periods_total` and the counter `container_cpu_cfs_throttled_seconds_total`. A throttle ratio above a few percent on a latency-sensitive service is significant. The decision path is: runnable threads and spare node CPU, then `nr_throttled` rising, then the relation between threads and limit: the service has more concurrently running threads (request threads, GC and compiler threads, the common pool) than the limit allows it to run at once.

The timeline of the worked trace applies unchanged: concurrency above the quota exhausts the pool early in the period and every thread waits for the refill.

Root cause. The limit of one CPU is a budget of 100 ms of CPU time per 100 ms period, spent in wall-clock time at a rate equal to the number of busy threads. A burst of requests that keeps four threads busy exhausts the budget in 25 ms, after which all threads stand still for 75 ms. The average use stays low because most periods have no burst.

Fix. Immediate options, in order of safety: raise or remove the CPU limit for the affected workload (keep the request, which provides the fair share and the scheduling guarantee); reduce the number of concurrent busy threads (for example the Tomcat maximum threads and the size of the pools). Permanent: choose the limit from the measured peak concurrency, not from the average use; set `-XX:ActiveProcessorCount` explicitly if the limit must stay below the thread demand and the pools should be sized for it; on kernels that support it, use a burst allowance (`cpu.cfs_burst_us` in v1, `cpu.max.burst` in v2) so that unused quota carries over to the next period; and spread load over more replicas with smaller bursts.

Prevention. Alert on the throttle ratio per pod and show it beside latency in the service dashboard. Load-test with the production limits, not on an unconstrained node. Make a platform policy that latency-sensitive services declare requests and omit hard CPU limits unless a measured reason exists, and that batch services are limited.

2.7.5 Questions to expect

Questions and model answers

1. **What is the difference between `cpu.shares` and CFS quota?** Shares are a relative weight that applies only under contention; quota is an absolute cap per period that applies even when CPUs are idle.
2. **How does Kubernetes map `requests.cpu` and `limits.cpu` to cgroups?** The request becomes `cpu.shares` (millicores times 1024 / 1000, so 250m gives 256); the limit becomes `cpu.cfs_quota_us` with a 100 ms period (500m gives 50000 per 100000). In cgroup v2 these are `cpu.weight` and `cpu.max`.
3. **What is CPU throttling?** When a cgroup has used its quota within the current period, the kernel stops running its threads until the next period, even though CPUs may be idle.

4. **Why can a pod be throttled at 30 percent average CPU use?** The quota is consumed at a rate equal to the number of running threads, so a short parallel burst exhausts it early in the period; the average over a longer window hides this.
5. **Which counters show throttling?** `nr_periods`, `nr_throttled` and `throttled_time` in `cpu.stat` (`throttled_usec` in v2); in Prometheus the `container_cpu_cfs_throttled_*` series.
6. **How does the JVM determine the number of CPUs in a container?** It reads the cgroup quota and period and uses the quota divided by the period, rounded up, bounded by the host CPUs; `-XX:ActiveProcessorCount` overrides it. The result drives GC threads, the common pool and the collector selection.
7. **What happens to the JVM if the limit is under two CPUs?** It sees one processor, is not a server-class machine, and selects the Serial collector, whose collections stop all application threads.
8. **Should every container have a CPU limit?** Not necessarily. Requests give fair sharing; limits give a hard ceiling at the price of throttling. Latency-sensitive services often run with requests only, batch jobs with limits.
9. **How would you reproduce throttling on a laptop or a build machine?** Create a cgroup, write a quota to it, move a multi-threaded CPU burst program into it and read `cpu.stat`, as in this chapter.
10. **How do shares behave on an idle node?** They have no effect: a group with few shares can use all CPUs when no one else wants them.

KEY POINTS

Requests become relative weights (`cpu.shares`, `cpu.weight`); limits become a hard quota per 100 ms period (`cpu.cfs_quota_us`, `cpu.max`).

- Shares were verified as 4.0 : 1 for 1024 : 256; the quota raised the median latency of a bursty two-thread workload from 41 ms to 99 ms.
- A multi-threaded group spends its quota faster than wall time, so throttling occurs at low average use; `nr_throttled` and `throttled_time` reveal it.
- The JVM derives processor count and collector from the quota: 0.5 CPU gave one processor and the Serial collector.
- Fix by sizing the limit from peak concurrency, removing a needless limit, bounding thread pools or using burst; always monitor the throttle ratio.

sreram

PART 3

Memory Management

Virtual memory, paging, replacement, the page cache, the OOM killer and the memory of a JVM process, with measurements taken on Linux 6.18 and OpenJDK 21.

In this part

- **3.1** Virtual Memory and Address Spaces
- **3.2** Paging and Address Translation
- **3.3** Page Replacement, Working Sets and Swapping
- **3.4** The Page Cache and Memory Used for File I/O
- **3.5** Memory Limits, Overcommit and the OOM Killer
- **3.6** The Memory of a JVM Process

3.1 Virtual Memory and Address Spaces

Physical memory is a single, finite resource shared by every process, the kernel and the devices of a machine. If programs addressed that memory directly, three problems would follow immediately. A program could read or overwrite the memory of another program. A program would have to be compiled for the exact physical addresses it will occupy, which are unknown until it runs. The total size of all running programs could never exceed the installed RAM. Virtual memory removes all three problems with one mechanism: every process addresses a private, contiguous range of virtual addresses, and hardware together with the kernel translates each virtual address to a physical address on every access.

This chapter defines the mechanism, shows the layout of the address space of a small C program and of a JVM on the build machine, and explains how `malloc` and the JVM obtain memory from the kernel. The next chapter describes the translation hardware itself.

3.1.1 Concept and mechanism

Definitions

DEFINITION

The range of addresses a process can generate. On x86-64 Linux with 4-level paging the usable range is 48 bits: 128 TiB for user space (low half) and 128 TiB reserved for the kernel (high half). Each process has its own virtual address space; a virtual address has no meaning outside the process that generated it.

Physical memory is divided into fixed-size **frames** (4 KiB on x86-64) and the virtual address space into **pages** of the same size. A **page table** maps a virtual page number to a physical frame number. The memory management unit (MMU) of the CPU consults the page table of the running process on every instruction fetch, load and store. A page that has no valid mapping raises a **page fault**, a synchronous exception handled by the kernel.

sreerag

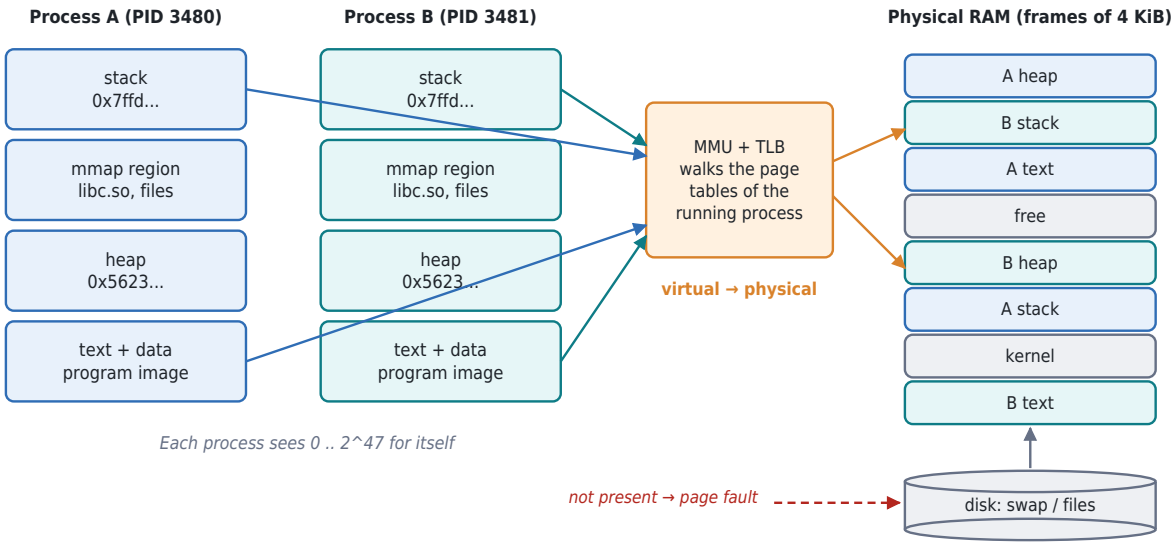


Figure 3.1. Two processes use overlapping virtual addresses, yet the MMU maps each page to a different frame in physical RAM; a page without a mapping raises a page fault.

What virtual memory provides

Service	Mechanism
Isolation	Each process has its own page tables; a process cannot name a frame that is not mapped into its tables.
Relocation	A program is linked once against virtual addresses; the kernel decides which frames back them.
Overcommitment	The sum of the virtual sizes may exceed RAM; frames are assigned when a page is first touched.
Sharing	The same frame may appear in several page tables: shared libraries, shared memory, the page cache.
Protection	Each page-table entry carries read, write, execute and user/supervisor bits enforced by the MMU.
Lazy allocation and file mapping	Pages are created on first access (demand paging) or loaded from a file on first access.

Services of virtual memory.

Virtual memory areas

The kernel does not track individual pages of a process in one list. It records the address space as a set of **virtual memory areas** (VMAs). Each VMA is a contiguous range of pages with identical permissions and the same backing: anonymous (zero-filled, swap-backed) or a region of a file. The VMAs of a process are the lines of `/proc/PID/maps`. A page-table entry exists only for pages that were actually touched. The VMA answers the question "is this address legal and what should be there"; the page table answers "which frame holds it now".

Each line of `/proc/PID/maps` has the format `start-end perms offset device inode path`. The permission field has four characters: `r` read, `w` write, `x` execute and `p` private (copy-on-write) or `s` shared. A mapping with permissions `---p` reserves address space without allowing any access; the JVM uses it for memory that it has reserved but not yet committed.

Segments of a process image

Region	Contents	Grows
`text`	Machine code (r-x), read-only data (r--)	fixed size
`data / bss`	Initialised and zero-initialised globals (rw-)	fixed size
`heap`	Memory obtained by `brk`; used by `malloc` for small blocks	upward
`mmap area`	Shared libraries, file mappings, large `malloc` blocks, thread stacks	downward from below the stack
`stack`	Call frames, local variables of the main thread; limit `ulimit -s` (8192 KiB here)	downward
`vdso / vvar`	Kernel-provided pages for fast system calls such as `clock_gettime`	fixed

Standard regions of a Linux process.

Address-space layout randomisation

With address-space layout randomisation (ASLR) the kernel adds a random offset to the base of the executable (for position-independent executables), the heap, the mmap area and the stack at every `exec`. Exploits that rely on fixed addresses therefore fail. The setting is `kernel.randomize_va_space`: 0 disables ASLR, 1 randomises stack, mmap and vdso, 2 also randomises the heap start. On the build machine the value is 2. Two consecutive runs of the same program below printed stack addresses `0x7fff2d3219f4` and `0x7ffff57c1fe4`; the heap start differed as well.

3.1.2 Worked trace: the address space of a C program

The program below prints one address from each region and then dumps its own `/proc/PID/maps`.

layout.c C

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/mman.h>
#include <unistd.h>

int global_init = 42;
int global_bss[1024];
const char *msg = "hello";

int main(void) {
    int local = 7;
    void *small = malloc(100);
    void *big = malloc(1 << 20);
    void *map = mmap(NULL, 1 << 20, PROT_READ | PROT_WRITE,
                     MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
    printf("code   main      %p\n", (void *)main);
    printf("rodata msg       %p\n", (const void *)msg);
    printf("data   global    %p\n", (void *)&global_init);
    printf("bss    global    %p\n", (void *)global_bss);
    printf("heap   malloc100 %p\n", small);
    printf("mmap   malloc1M  %p\n", big);
    printf("mmap   anon 1M   %p\n", map);
    printf("stack  local     %p\n", (void *)&local);
    printf("\n--- /proc/self/maps ---\n");
    fflush(stdout);
    char cmd[64];
    snprintf(cmd, sizeof cmd, "cat /proc/%d/maps", getpid());
    system(cmd);
    return 0;
}
```

Compile and run (typical output; addresses differ on every run because of ASLR) TERMINAL

```
$ gcc -O0 -o layout layout.c && ./layout
code   main      0x5623b6b5c249
rodata msg       0x5623b6b5d004
data   global    0x5623b6b5f010
bss    global    0x5623b6b5f040
heap   malloc100 0x5623d23d22a0
```

Compile and run (typical output; addresses differ on every run because of ASLR) (continued) TERMINAL

```
mmap   malloc1M  0x7f7e58621010
mmap   anon 1M   0x7f7e58300000
stack  local     0x7ffee013e074

--- /proc/self/maps ---
5623b6b5b000-5623b6b5c000 r--p 00000000 fe:00 966868    /tmp/osw_o3/layout
5623b6b5c000-5623b6b5d000 r-xp 00001000 fe:00 966868    /tmp/osw_o3/layout
5623b6b5d000-5623b6b5e000 r--p 00002000 fe:00 966868    /tmp/osw_o3/layout
5623b6b5e000-5623b6b5f000 r--p 00002000 fe:00 966868    /tmp/osw_o3/layout
5623b6b5f000-5623b6b60000 rw-p 00003000 fe:00 966868    /tmp/osw_o3/layout
5623b6b60000-5623b6b61000 rw-p 00000000 00:00 0
5623d23d2000-5623d23f3000 rw-p 00000000 00:00 0         [heap]
7f7e58300000-7f7e58400000 rw-p 00000000 00:00 0
7f7e58400000-7f7e58428000 r--p 00000000 fe:00 344580    .../libc.so.6
7f7e58428000-7f7e585b1000 r-xp 00028000 fe:00 344580    .../libc.so.6
7f7e585b1000-7f7e58600000 r--p 001b1000 fe:00 344580    .../libc.so.6
7f7e58600000-7f7e58604000 r--p 001ff000 fe:00 344580    .../libc.so.6
7f7e58604000-7f7e58606000 rw-p 00203000 fe:00 344580    .../libc.so.6
7f7e58606000-7f7e58613000 rw-p 00000000 00:00 0
7f7e58621000-7f7e58725000 rw-p 00000000 00:00 0
7f7e58735000-7f7e58739000 r--p 00000000 00:00 0         [vvar]
7f7e5873b000-7f7e5873d000 r-xp 00000000 00:00 0         [vdso]
7f7e5873d000-7f7e5873e000 r--p 00000000 fe:00 344560    .../ld-linux-x86-64.so.2
7f7e5873e000-7f7e58769000 r-xp 00001000 fe:00 344560    .../ld-linux-x86-64.so.2
7ffee011f000-7ffee0145000 rw-p 00000000 00:00 0         [stack]
ffffffffff600000-ffffffffff601000 --xp 00000000 00:00 0 [vsyscall]
```

The trace is read as follows.

1. `main` (`0x5623b6b5c249`) lies in the `r-xp` mapping `5623b6b5c000-5623b6b5d000`, the text segment of the executable. The string literal `msg` points into the following `r--p` page, the read-only data.
2. `global_init` (`...f010`) and `global_bss` (`...f040`) lie in the `rw-p` page starting at `...f000`. The executable occupies five consecutive VMAs at the low end of the address space (`0x5623...`, a randomised base for a position-independent executable).
3. `malloc(100)` returned `0x5623d23d22a0`, inside `[heap]` (`...d2000-...f3000`, 132 KiB = 0x21000 bytes). The heap starts at a random offset after the executable.
4. `malloc(1 << 20)` returned `0x7f7e58621010`, inside an anonymous 1 MiB + 4 KiB mapping at `7f7e58621000-7f7e58725000`. The 16 bytes before the returned address hold the allocator header. The explicit `mmap` returned the page-aligned `0x7f7e58300000`.
5. The libc mapping is split into four VMAs with different permissions: `r--p` headers, `r-xp` code, `r--p` read-only data, `rw-p` writable data. The file offset column shows which part of `libc.so.6` backs each VMA.
6. The stack VMA `7ffee011f000-7ffee0145000` is 152 KiB at this moment (0x26000 bytes). It grows downward on demand up to the `ulimit -s` limit of 8192 KiB. `local` (`0x7ffee013e074`) is near its upper end.
7. `[vdso]` and `[vvar]` are pages supplied by the kernel; the vDSO implements `clock_gettime` and `gettimeofday` without a system call. `[vsyscall]` is a legacy fixed page at the top of the address space.

srgram

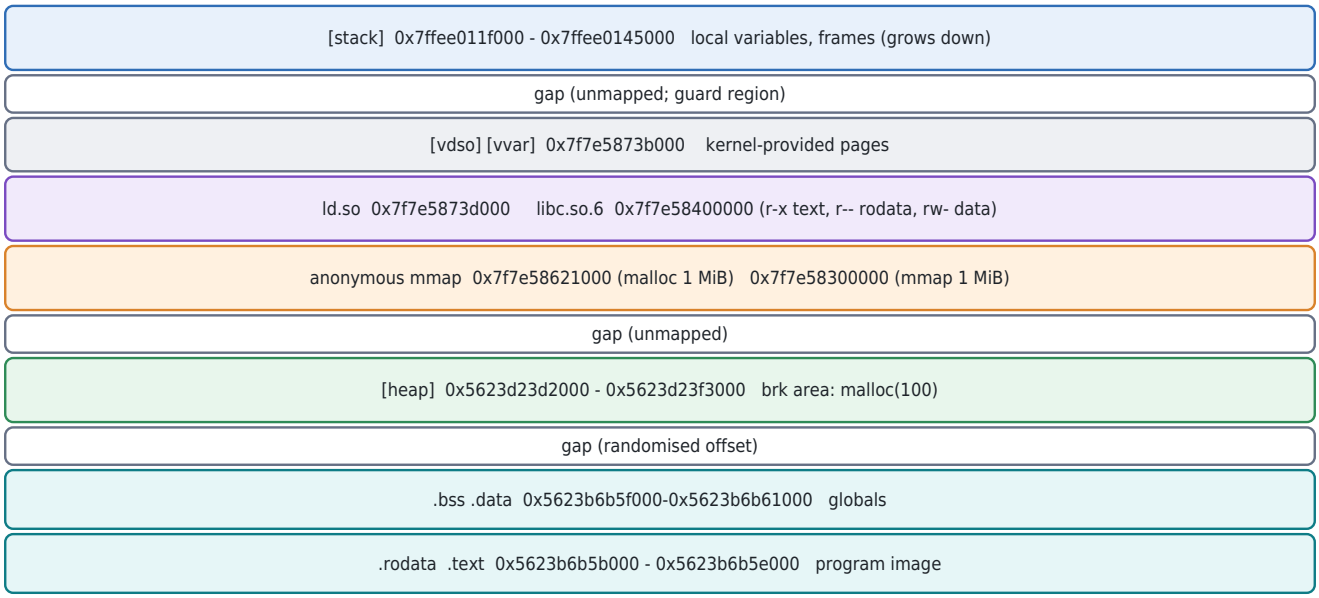


Figure 3.2. The regions of the C program placed on one vertical axis with the real addresses from the run: text and data at the bottom, the brk heap above them, the mmap region with libc, and the stack at the top.

3.1.3 How the heap and the mmap area grow

User space obtains virtual memory from the kernel with two system calls. `brk` moves the end of the **program break**, the upper boundary of the heap VMA. `mmap` creates an arbitrary new VMA. The C library allocator (`malloc` in glibc) uses `brk` for small requests and `mmap` for requests above a threshold (128 KiB by default, adjusted dynamically up to 32 MiB). The allocator also keeps per-thread arenas, each a reserved 64 MiB region, to reduce lock contention in multithreaded programs; this is the origin of the many `---p` regions in a JVM, described later.

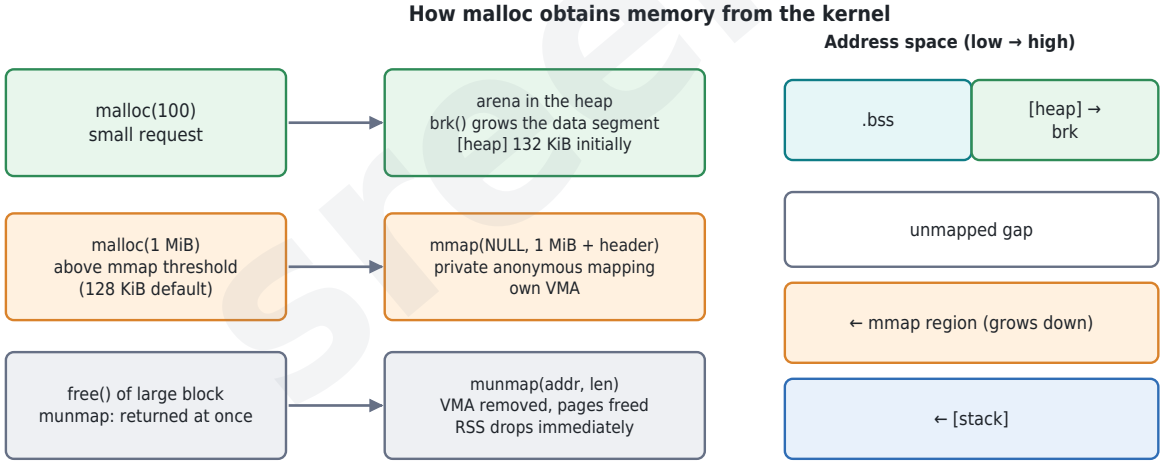


Figure 3.3. The allocator takes small requests from the brk heap and large requests from private anonymous mmap regions; freeing a large block returns the pages to the kernel at once.

The system calls of a program that performs `malloc(100)`, `malloc(1 << 20)` and frees both can be observed directly.

Trace brk, mmap and munmap of mal.c (loader calls omitted) TERMINAL

```
$ strace -e trace=brk,mmap,munmap ./mal 2>&1 | tail -4
brk(NULL)                               = 0x56495da2f000
brk(0x56495da50000)                     = 0x56495da50000
mmap(NULL, 1052672, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f2b63aff000
munmap(0x7f2b63aff000, 1052672)         = 0
```

The first `brk(NULL)` queries the current break. The second call extends it by 0x21000 bytes (132 KiB), which provides the arena for `malloc(100)`. The 1 MiB request exceeds the mmap threshold and becomes a separate mapping of 1052672 bytes (1 MiB plus one page for the header). `free` of the large block calls `munmap`, so the resident set drops

immediately; `free` of a small block normally keeps the memory in the arena for reuse, which is why the resident size of a long-running C or native-library process rarely shrinks.

3.1.4 Hands-on: the address space of a JVM

A JVM is an ordinary process; its address space contains the same regions plus areas that the JVM reserves for its own purposes. The program `Idle.java` allocates 100 MiB of live arrays and a 16 MiB direct buffer and then sleeps. It was started with `java -Xmx256m -Xms64m -XX:NativeMemoryTracking=summary` (default G1 collector on this 2-CPU, 8 GiB machine; the JDK is OpenJDK 21.0.12).

Size and shape of the JVM address space (PID 1777, typical output) TERMINAL

```
$ wc -l < /proc/1777/maps
182
$ grep -E "^(VmPeak|VmSize|VmRSS|RssAnon|RssFile|VmData|VmStk|Threads)" /proc/1777/status
VmPeak:  2913852 kB
VmSize:  2868616 kB
VmRSS:    162952 kB
RssAnon:          139352 kB
RssFile:           23600 kB
VmData:   230668 kB
VmStk:       152 kB
Threads:        19
```

The process has 182 VMAs, a virtual size of 2.74 GiB and a resident set of 159 MiB. The virtual size is 17.6 times the resident size, which is normal for a JVM. The large regions, listed by the script below, explain it.

bigmaps.py PYTHON

```python
import sys

# Print the VMAs of a process that are at least 8 MiB, and the total of the reserved (---p) ones.
reserved = 0
for line in open("/proc/%s/maps" % sys.argv[1]):
    f = line.split()
    start, end = (int(x, 16) for x in f[0].split("-"))
    kb = (end - start) // 1024
    if f[1] == "---p":
        reserved += kb
    if kb >= 8192:
        print(f[0], f[1], "%d KB" % kb, f[5] if len(f) > 5 else "")
print(reserved, "KB reserved, no access")
```

Regions of 8 MiB or more in /proc/1777/maps (selected lines of the real output) TERMINAL

```
$ python3 bigmaps.py 1777
f0000000-f7c00000 rw-p 126976 KB
f7c00000-ffe00000 ---p 133120 KB
7f2b90050000-7f2bd0000000 ---p 1048256 KB
7f2bfb800000-7f2c03e54000 r--s 137552 KB /usr/lib/jvm/java-21-openjdk-amd64/lib/modules
7f2c090a3000-7f2c09f1f000 r-xp 14832 KB /usr/lib/jvm/java-21-openjdk-amd64/lib/server/libjvm.so
7f2b8f000000-7f2b8fc9c000 rw-p 12912 KB /usr/lib/jvm/java-21-openjdk-amd64/lib/server/classes.jsa
7f2beca70000-7f2bf3d38000 ---p 117536 KB
7f2b64030000-7f2b68000000 ---p 65344 KB
2471884 KB reserved, no access
```

The regions are, in order: the committed part of the Java heap, its reserved remainder, the 1 GiB compressed class space, the mapped JDK class image, the VM library itself, the class-data-sharing archive, a reservation of the code cache, and one of the 64 MiB malloc arenas.

Three observations follow. First, the Java heap starts at `0xf0000000` and its reserved size is exactly `-Xmx256m` (0xf0000000 to 0x100000000). The heap ends exactly at the 4 GiB boundary (0x100000000), which allows the JVM to use compressed object pointers without any base or shift (32-bit references). Second, `---p` regions occupy more than 2 GiB of virtual size but consume no frames and no swap; they are reservations. The 1 GiB compressed class space is reserved once at start-up and committed in small pieces as classes load. Third, the JDK class image `lib/modules` is mapped read-only and shared; its pages enter the resident set (RssFile) only when classes are read from it, and they can be dropped and re-read at any time because they are backed by a file.

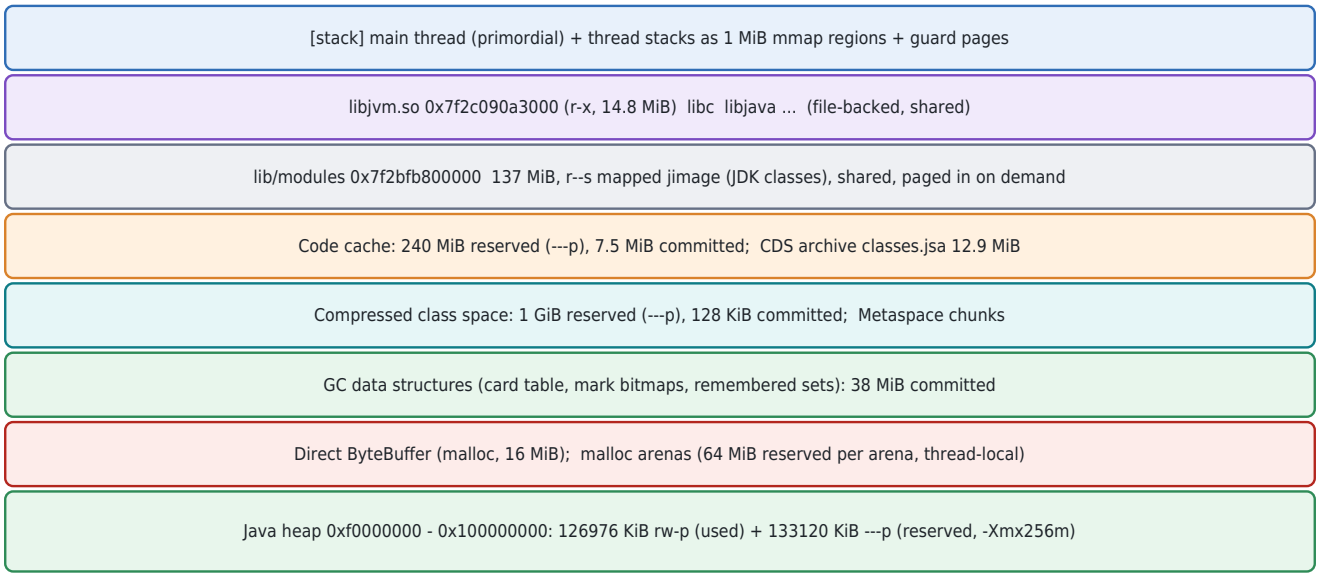


Figure 3.4. The JVM address space on one vertical axis: the 256 MiB heap reservation, class space, code cache, GC structures, direct and malloc memory, mapped libraries and thread stacks.

The system also contains the JVM threads: `ls /proc/1777/task | wc -l` gave 19 during the run (20 after `jcmd` attached). Each is a Linux task with its own stack VMA; the next chapters of this part and the following part treat them.

3.1.5 Real-world incidents

Incident: alert on virtual memory size of a Java service

Scenario. A monitoring template for a fleet of Spring Boot services raises a warning when `VSZ` (virtual size) of a process exceeds 1.5 times the container memory limit. A service with `-Xmx256m` in a 512 MiB container is reported with a virtual size of 2.8 GiB and the alert fires on every instance, although the services are healthy.

Symptoms. The alert dashboard shows `VmSize` of 2.8 GiB against a 512 MiB limit. No restarts, no `OutOfMemoryError`, no kernel OOM messages occur. Resident size stays near 160 MiB.

Diagnosis. The two sizes are compared directly, then the large regions are listed by permission.

Compare virtual and resident size, then total the reservations (PID 1777) TERMINAL

```
$ grep -E "VmSize|VmRSS" /proc/1777/status
VmSize:  2868616 kB
VmRSS:    162952 kB
$ python3 bigmaps.py 1777 | tail -1
2471884 KB reserved, no access
```

The `---p` regions (heap tail, class space, code cache, arenas) total 2.36 GiB, most of the 2.74 GiB virtual size. The `VmSize` metric counts address space, not memory. Neither the kernel limit nor the OOM killer considers it: a memory cgroup limits resident pages (anonymous, page cache and kernel memory charged to the cgroup), and a limit on address space exists only if `ulimit -v` or `RLIMIT_AS` is set.

Root cause. The alert uses a metric that measures reservation instead of consumption.

Fix. Immediate: silence the rule for JVM processes. Permanent: alert on resident memory (`VmRSS`, `container_memory_working_set_bytes` in Kubernetes) relative to the container limit, and on the NMT committed total when a finer view is required.

Prevention. Document in the monitoring standard that virtual size is informational. Review any use of `ulimit -v` for JVMs: a limit smaller than the reservations (about 1.25 GiB for the class space and code cache alone) prevents the JVM from starting.

Incident: JVM fails to start with a low virtual-memory limit

Scenario. A hardening script sets `ulimit -v 1000000` (about 1 GB) before starting a service to prevent runaway memory use.

Symptoms. The JVM exits during start-up with `Error occurred during initialization of VM` and a message that it could not reserve enough space for the object heap or the compressed class space.

Diagnosis. The limit is verified and the minimum reservation estimated from the maps of a running JVM.

Reproduction with a virtual-memory limit of about 1 GB, then 3 GB TERMINAL

```
$ (ulimit -v 1000000; java -Xmx256m -version 2>&1 | grep -v Picked)
Error occurred during initialization of VM
Could not allocate compressed class space: 1073741824 bytes
$ (ulimit -v 3000000; java -Xmx256m -version 2>&1 | grep -v Picked | head -1)
openjdk version "21.0.12.1" 2026-08-18
```

The failing reservation is the 1 GiB compressed class space; the message names the first reservation that does not fit, and the exact text can differ between JDK builds.

Root cause. `RLIMIT_AS` limits the total size of all VMAs, including the `---p` reservations; the JVM reserves the 1 GiB class space and the code cache before running any user code.

Fix. Remove `ulimit -v` and limit memory with a cgroup (`MemoryMax=` in a systemd unit, or the container limit), which accounts for resident pages.

Prevention. Test every start-up wrapper with the real JVM flags in a pre-production environment.

3.1.6 Questions to expect

Questions and model answers

1. **What is the difference between a virtual address and a physical address?** A virtual address is generated by a process and is translated by the MMU through the page tables of that process to a physical address, which identifies a byte of RAM. Two processes can use the same virtual address for different data.
2. **Why do the virtual sizes of Java processes exceed the heap size by gigabytes?** The JVM reserves address space (class space, code cache, heap up to `-Xmx`, per-thread arenas and stacks) without committing it. Reserved pages have no page-table entries and use no RAM.
3. **What is a VMA?** A virtual memory area is a contiguous range of pages with the same permissions and backing; the kernel keeps VMAs in a tree to validate addresses on page faults. `/proc/PID/maps` lists them.
4. **How do `brk` and `mmap` differ?** `brk` moves the end of the single heap segment; `mmap` creates an independent mapping anywhere in the address space and can be unmapped separately, which allows `free` to return large blocks to the kernel.
5. **What does ASLR protect against and what does it cost?** It makes the addresses of code, stack and heap unpredictable per run, which hinders exploits that jump to known addresses. The cost is negligible; position-independent code needs one extra register on 32-bit x86 and has little overhead on x86-64.
6. **Why does the resident size of a process not shrink after `free`?** Small blocks stay in the allocator arena for reuse; only whole top-of-heap regions or `mmap`-allocated blocks are returned.
7. **What does a mapping with permissions `---p` mean?** A reserved range with no access rights. Touching it raises `SIGSEGV`; the JVM later changes the permissions with `mprotect` or `mmap` to commit it.
8. **A segmentation fault occurs. What does the kernel actually do?** The access hits an address without a VMA or violates the VMA permissions. The page-fault handler cannot resolve it and sends `SIGSEGV` to the thread.

KEY POINTS

Virtual memory gives each process a private address space translated page by page by the MMU.

- The kernel records the address space as VMAs (`/proc/PID/maps`); page-table entries exist only for touched pages.
- `brk` extends the heap; `mmap` creates all other regions; `malloc` uses both.
- ASLR randomises the base addresses at every `exec`; compare two runs to verify it.
- A JVM reserves far more address space than it uses; judge consumption by resident size and NMT, not by `VmSize`.
- A `---p` mapping reserves addresses without frames; `RLIMIT_AS` counts these reservations.

3.2 Paging and Address Translation

A virtual address is useless until it is translated to a physical address. Translation must be fast, because it happens for every instruction fetch and every memory operand, and it must be compact, because a map of 2^48 bytes of address space cannot be stored as a flat table. Paging solves both requirements with a hierarchy of small tables and a cache of recent translations. This chapter derives the structure, performs a complete translation with real values, measures page faults with a C program, and relates the results to the JVM.

3.2.1 Concept and mechanism

Why a flat page table is impossible

With 4 KiB pages a 48-bit address contains a 36-bit virtual page number. A flat table with one 8-byte entry per page would need 2^36 × 8 bytes = 512 GiB per process, almost all of it describing unused addresses. A multi-level table allocates a lower-level table only if some address below it is in use. A process whose mappings occupy a few hundred megabytes needs a few hundred kilobytes of page tables.

The four-level structure

On x86-64, the 48 bits of a virtual address are split into four 9-bit indices and a 12-bit offset. Each table is exactly one 4 KiB page holding 512 entries of 8 bytes.

Level	Virtual bits	Entry points to	Region covered by one entry
`PGD (page global directory)`	47..39	a PUD table	512 GiB
`PUD (page upper directory)`	38..30	a PMD table or a 1 GiB page	1 GiB
`PMD (page middle directory)`	29..21	a PTE table or a 2 MiB page	2 MiB
`PTE (page table entry)`	20..12	a 4 KiB frame	4 KiB

Levels of the x86-64 page table (Linux names).

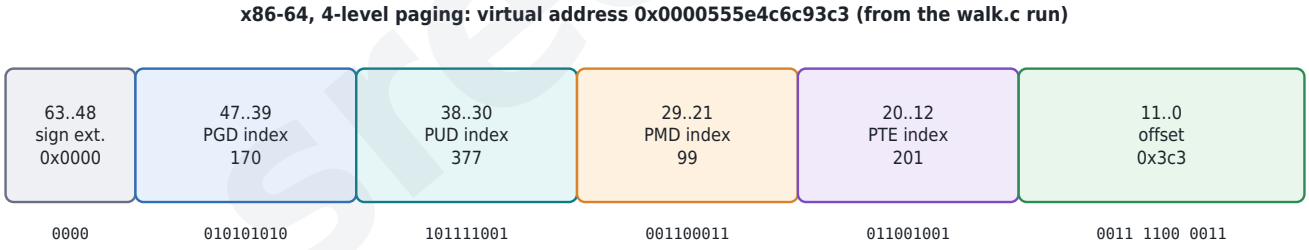


Figure 3.5. A 48-bit virtual address divided into four 9-bit table indices and a 12-bit page offset, with the values of the address used in the worked trace below.

The register `CR3` holds the physical address of the PGD of the running process. A context switch to another process loads a different `CR3`. Linux 6.x can also be built with five levels (a P4D level between PGD and PUD) for 57-bit address spaces; the default user address space remains 47 bits.

The page-table entry

An entry is 64 bits. The low bits carry control flags and the middle bits carry the physical frame number (PFN). The flags relevant for the operating system are: **P** (present), **R/W** (writable), **U/S** (accessible from user mode), **A** (accessed, set by hardware on any access), **D** (dirty, set by hardware on a write), **PS** (page size: the entry maps a huge page) and **NX** (no execute). The kernel reads **A** and **D** to decide which pages to evict and which must be written back. If **P** is zero the remaining bits are free for the kernel; Linux stores a swap location or a file offset there.

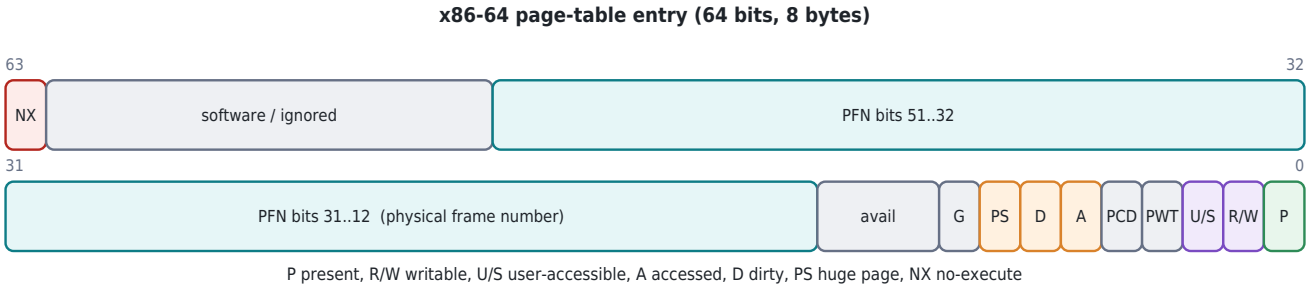


Figure 3.6. The 64-bit page-table entry: the frame number occupies bits 51..12; present, writable, user, accessed, dirty, page-size and no-execute flags control the access.

The translation lookaside buffer

A walk costs four dependent memory reads. The CPU therefore caches complete translations in the **translation lookaside buffer** (TLB). The TLB is indexed by the virtual page number (and an address-space tag, the PCID on x86) and stores the frame number and permissions. A hit adds almost no latency. A miss starts the hardware page walker, which reads the page tables through the normal data caches; page-table entries of recently used regions are usually in the L1 or L2 cache, so most walks cost far less than four main-memory accesses.

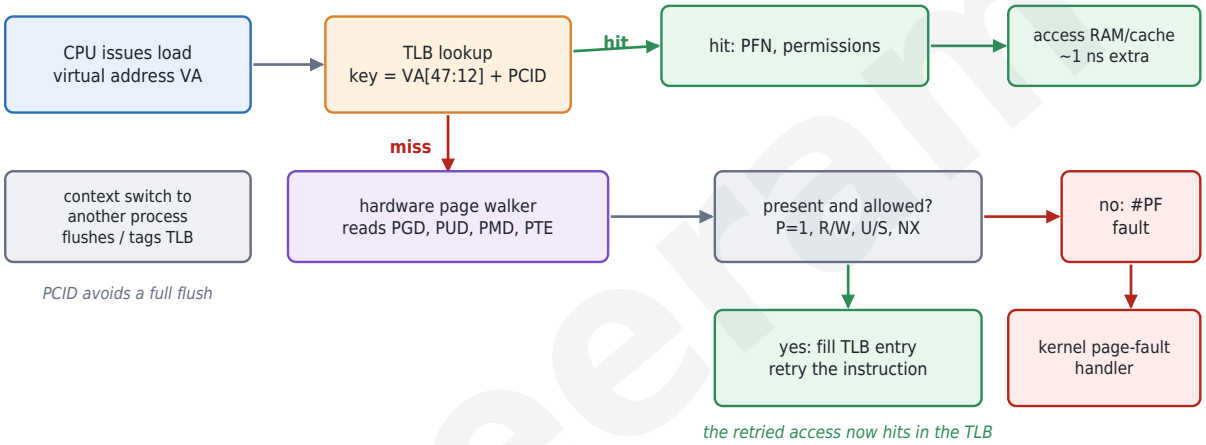


Figure 3.7. The translation path: a TLB hit supplies the frame immediately, a miss triggers the hardware walker, and a walk that finds a missing or forbidden entry raises a page fault for the kernel.

The **TLB reach** is the number of entries multiplied by the page size. The sizes vary by CPU model and are available from `cpuid`; a typical recent Intel core has a first-level data TLB of 64 entries and a second-level TLB of 1,024 to 2,048 entries (these figures are typical and not measured here). With 4 KiB pages 1,536 entries cover 6 MiB; with 2 MiB pages the same number of entries covers 3 GiB. A process that randomly touches more memory than its TLB reach spends a measurable part of its time in page walks. A context switch changes the address space; the PCID tag lets the TLB keep entries of several address spaces and avoids a full flush.

Effective access time

If the TLB hit ratio is h, the TLB lookup costs t and a memory access costs m, a translation that hits costs t + m for the access. On a miss the walk performs four additional memory reads before the data access: t + 4m + m. The effective access time is therefore h(t + m) + (1 − h)(t + 5m).

Huge pages

Setting the PS bit in a PMD entry maps 2 MiB with a single entry and no PTE level; a PUD entry can map 1 GiB. Benefits: one TLB entry covers 512 times as much memory, the walk is one level shorter and the number of page faults falls. Costs: a fault must zero 2 MiB, a partly used huge page wastes memory, and a free 2 MiB contiguous region is needed, which may require compaction. Linux offers two mechanisms: explicit huge pages reserved with `vm.nr_hugepages` (used through `hugetlbfs` or `MAP_HUGETLB`) and **transparent huge pages** (THP), where the kernel promotes regions automatically. The system setting is `/sys/kernel/mm/transparent_hugepage/enabled` with the values `always`, `madvise` and `never`; this machine reports `madvise`, so only regions that call `madvise(MADV_HUGEPAGE)` are eligible.

Page faults

A page fault is raised by the MMU when a translation fails: the entry is not present, or the access violates the entry permissions. The kernel handler classifies the fault by looking up the VMA that contains the address.

Cause	Kernel action	Class
Address in no VMA, or access not permitted by the VMA	Send `SIGSEGV`	fatal for the process
First touch of an anonymous page	Allocate a zeroed frame, map it	minor
Page is in the page cache but not mapped	Map the cached frame	minor
Page is not in the cache	Read it from the file, then map it (task sleeps)	major
Page was swapped out	Read from the swap device, then map	major
Write to a present, write-protected private page	Copy the frame, map the copy writable	minor (copy-on-write)

Classification of page faults.

A **minor** fault is resolved without disk I/O; a **major** fault requires it. Both are counted per process (`min_flt` and `maj_flt` in `/proc/PID/stat`, `ru_minflt` and `ru_majflt` in `getrusage`).

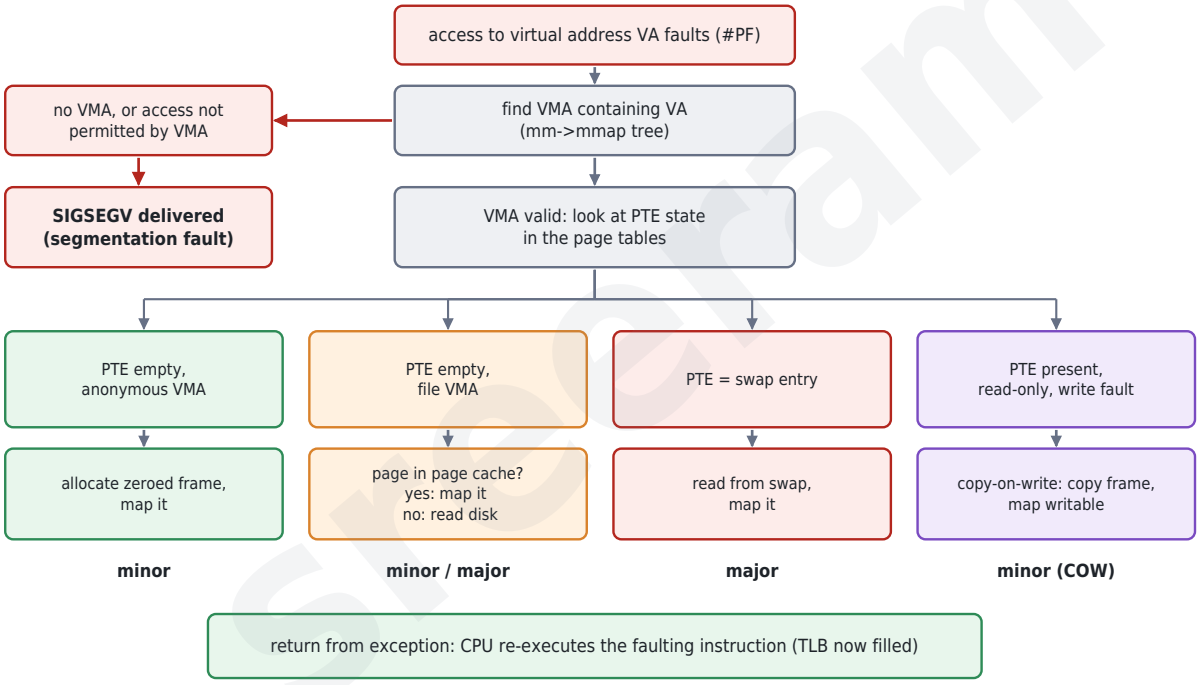


Figure 3.8. The page-fault handler: the VMA lookup separates illegal accesses (SIGSEGV) from legal ones, and the state of the page-table entry selects among zero-fill, page-cache mapping, disk read, swap-in and copy-on-write.

Demand paging

When `exec` or `mmap` creates a mapping, no frames are allocated and no data is read. The first access to each page faults, and the handler supplies the page. A 64 MiB anonymous mapping that is never touched costs nothing; a 1 GiB executable that executes only a fraction of its code loads only that fraction. The price is a fault at the first access of every page. The sequence for a major fault on a file-backed page is shown below.

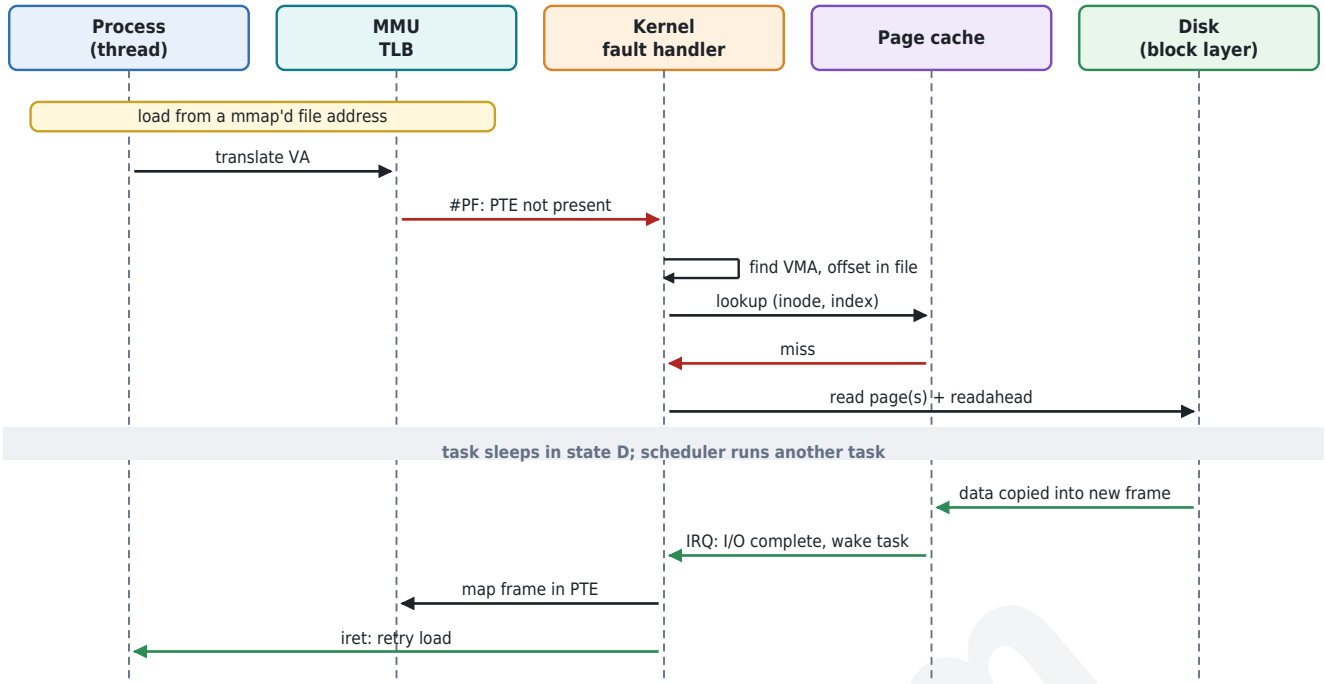


Figure 3.9. A major fault on a file mapping: the MMU raises the fault, the kernel finds the page absent from the page cache, starts the read, the task sleeps while the scheduler runs other work, and the interrupt at completion wakes the task, which retries the instruction.

Copy-on-write

`fork` must give the child a copy of the address space of the parent. Copying every page would be wasteful because most children call `exec` immediately. Linux copies only the page tables and marks all private writable pages read-only in both processes; the frames are shared and their reference counts are increased. The first write by either process faults. The handler allocates a new frame, copies the 4 KiB, updates the writer's entry to point to the copy with write permission, and decrements the reference count of the original. The same mechanism makes the all-zero page, shared libraries mapped privately, and snapshots by `fork` in databases efficient.

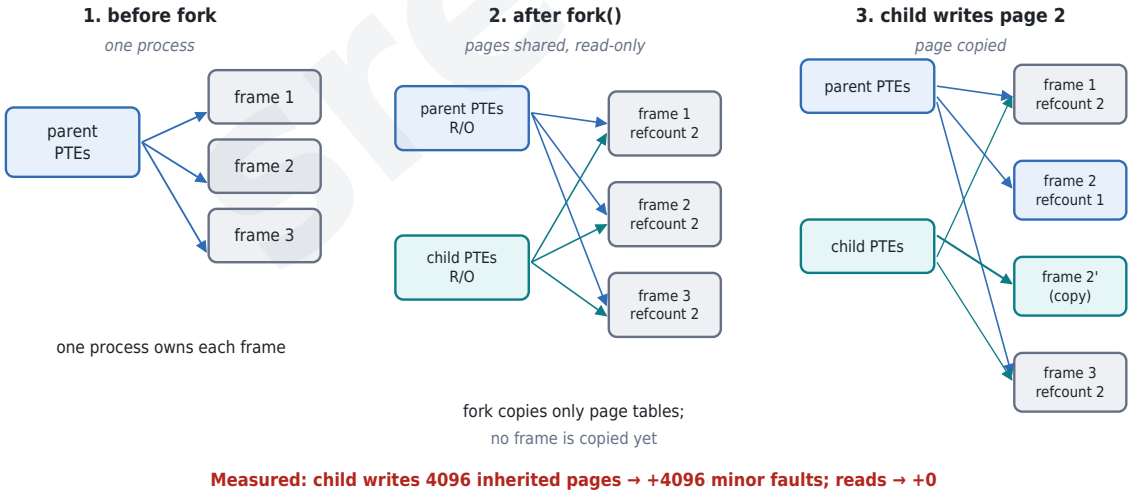


Figure 3.10. Copy-on-write after fork: both page tables initially point at the same read-only frames; a write by the child allocates frame 2', copies the data and changes only the child's entry.

3.2.2 Worked trace: translating 0x555e4c6c93c3

The program `walk.c` allocates memory, forces a page into existence, splits an address into its table indices and asks the kernel for the real frame number through `/proc/self/pagemap` (readable by root; unprivileged users receive zero frame numbers).

walk.c C

```c
#include <stdio.h>
#include <stdint.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>

int main(void) {
    char *p = malloc(4096 * 4);
    strcpy(p, "touch");                     /* force the page to exist */
    uint64_t va = (uint64_t)p + 0x123;
    printf("virtual address   0x%016lx\n", va);
    printf("bits 47..39 (PGD) %lu\n", (va >> 39) & 0x1ff);
    printf("bits 38..30 (PUD) %lu\n", (va >> 30) & 0x1ff);
    printf("bits 29..21 (PMD) %lu\n", (va >> 21) & 0x1ff);
    printf("bits 20..12 (PTE) %lu\n", (va >> 12) & 0x1ff);
    printf("bits 11..0  (off) 0x%lx\n", va & 0xfff);
    int fd = open("/proc/self/pagemap", O_RDONLY);
    uint64_t e;
    pread(fd, &e, 8, (va >> 12) * 8);
    printf("pagemap entry     0x%016lx\n", e);
    uint64_t pfn = e & ((1ULL << 55) - 1);
    printf("present=%lu swapped=%lu soft-dirty=%lu exclusive=%lu pfn=0x%lx\n",
           (unsigned long)((e >> 63) & 1), (unsigned long)((e >> 62) & 1),
           (unsigned long)((e >> 55) & 1), (unsigned long)((e >> 56) & 1), (unsigned long)pfn);
    printf("physical address  0x%lx\n", (unsigned long)((pfn << 12) | (va & 0xfff)));
    return 0;
}
```

Run walk.c (typical output; the address and the frame number differ on every run) TERMINAL

```
$ gcc -Wall -o walk walk.c && ./walk
virtual address   0x0000555e4c6c93c3
bits 47..39 (PGD) 170
bits 38..30 (PUD) 377
bits 29..21 (PMD) 99
bits 20..12 (PTE) 201
bits 11..0  (off) 0x3c3
pagemap entry     0x8100000000141c7c
present=1 swapped=0 soft-dirty=0 exclusive=1 pfn=0x141c7c
physical address  0x141c7c3c3
```

The binary form of the 48 low bits is `010101010 101111001 001100011 011001001 001111000011`, which gives the indices 170, 377, 99 and 201 and the offset `0x3c3`. The hardware performs these steps.

1. Read `CR3`: the physical address of the PGD of the process.
2. PGD: read the 8-byte entry at byte offset 170 × 8 = 0x550. It is present and holds the physical address of a PUD table.
3. PUD: read the entry at offset 377 × 8 = 0xBC8. It points to a PMD table.
4. PMD: read the entry at offset 99 × 8 = 0x318. The PS bit is zero, so it points to a PTE table.
5. PTE: read the entry at offset 201 × 8 = 0x648. It holds the frame number `0x141c7c` with the present, writable and user bits set. The kernel exposes this entry through the pagemap file as `0x8100000000141c7c`: bit 63 (present) and bit 56 (exclusively mapped) are set, and bits 54..0 hold the frame number.
6. Physical address = (`0x141c7c` << 12) | `0x3c3` = `0x141c7c3c3`.
7. The TLB stores the pair (virtual page `0x555e4c6c9`, frame `0x141c7c`); the next access to any byte of this page needs no walk.

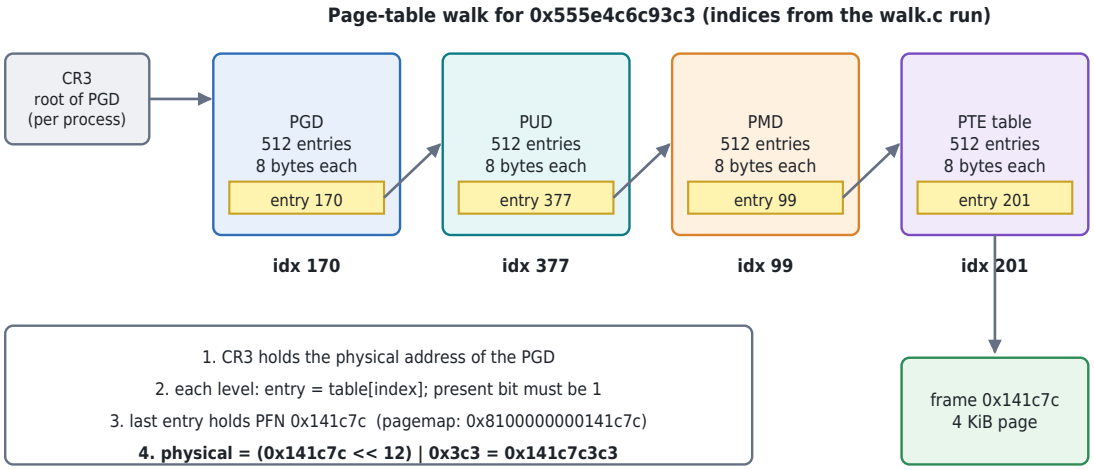


Figure 3.11. The four table lookups for the address of the trace: each index selects one 8-byte entry in a 4 KiB table, and the final entry supplies the frame number that is combined with the offset.

TLB hit ratio	Effective access time	Computation
100%	101 ns	t + m
99%	105 ns	0.99 × 101 + 0.01 × 501
90%	141 ns	0.90 × 101 + 0.10 × 501
50%	301 ns	0.50 × 101 + 0.50 × 501

Worked cost of translation (assumed memory time m = 100 ns, TLB time t = 1 ns).

The table assumes that all four page-table reads miss every cache; in practice they usually hit the cache hierarchy and the penalty is smaller. The conclusion remains: translation cost depends on the TLB hit ratio, and the working set of a program should fit within the TLB reach where possible.

3.2.3 Hands-on: measuring page faults

Faults per page and copy-on-write

The program `faults.c` counts the faults of the current process with `getrusage` around each action.

faults.c C

```c
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/resource.h>
#include <sys/wait.h>

static long minf(void) {
    struct rusage r; getrusage(RUSAGE_SELF, &r); return r.ru_minflt;
}
static long majf(void) {
    struct rusage r; getrusage(RUSAGE_SELF, &r); return r.ru_majflt;
}

int main(void) {
    const size_t MB = 1 << 20, PG = 4096;
    long m0, M0;

    size_t n = 64 * MB;
    char *a = mmap(NULL, n, PROT_READ | PROT_WRITE,
                   MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
    m0 = minf();
```

faults.c (continued) C

```c
    printf("anonymous 64 MiB mapped, no page touched: +%ld minor\n",
           minf() - m0);
    m0 = minf();
    for (size_t i = 0; i < n; i += PG) a[i] = 1;
    printf("write one byte per page (%zu pages):      +%ld minor\n",
           n / PG, minf() - m0);
    m0 = minf();
    for (size_t i = 0; i < n; i += PG) a[i] = 2;
    printf("write the same pages again:               +%ld minor\n",
           minf() - m0);

    /* file-backed mapping */
    size_t fn = 32 * MB;
    int fd = open("/tmp/osw_o3/data.bin", O_RDWR | O_CREAT | O_TRUNC, 0644);
    char *buf = malloc(MB);
    memset(buf, 'x', MB);
    for (int i = 0; i < 32; i++) write(fd, buf, MB);
    fsync(fd);
    posix_fadvise(fd, 0, 0, POSIX_FADV_DONTNEED);   /* evict from page cache */
    char *f = mmap(NULL, fn, PROT_READ, MAP_PRIVATE, fd, 0);
    long sum = 0;
    m0 = minf(); M0 = majf();
    for (size_t i = 0; i < fn; i += PG) sum += f[i];
    printf("file 32 MiB, cache cold, read each page:  +%ld minor, +%ld major\n",
           minf() - m0, majf() - M0);
    munmap(f, fn);
    f = mmap(NULL, fn, PROT_READ, MAP_PRIVATE, fd, 0);
    m0 = minf(); M0 = majf();
    for (size_t i = 0; i < fn; i += PG) sum += f[i];
    printf("file 32 MiB, cache warm, read each page:  +%ld minor, +%ld major\n",
           minf() - m0, majf() - M0);
    munmap(f, fn);
    posix_fadvise(fd, 0, 0, POSIX_FADV_DONTNEED);
    f = mmap(NULL, fn, PROT_READ, MAP_PRIVATE, fd, 0);
    madvise(f, fn, MADV_RANDOM);              /* no readahead, no fault-around */
    m0 = minf(); M0 = majf();
    for (size_t i = 0; i < 1000; i++)
        sum += f[((i * 7919) % 8192) * PG];
    printf("file cold, MADV_RANDOM, 1000 pages:       +%ld minor, +%ld major\n",
           minf() - m0, majf() - M0);
    fflush(stdout);

    /* copy-on-write after fork */
    pid_t pid = fork();
    if (pid == 0) {
        long c0 = minf();
        for (size_t i = 0; i < 16 * MB; i += PG) a[i] = 3;      /* 4096 pages */
        printf("child writes 4096 inherited pages (COW):  +%ld minor\n",
               minf() - c0);
        c0 = minf();
        for (size_t i = 32 * MB; i < 48 * MB; i += PG) sum += a[i];  /* read */
        printf("child reads 4096 inherited pages:         +%ld minor\n",
               minf() - c0);
        fflush(stdout); _exit(0);
    }
    wait(NULL);
    return sum == 42;
}
```

Run faults.c under a 600 MB address-space limit (typical output) TERMINAL

```
$ gcc -O1 -o faults faults.c && (ulimit -v 600000; ./faults)
anonymous 64 MiB mapped, no page touched: +0 minor
write one byte per page (16384 pages):      +16384 minor
write the same pages again:                 +0 minor
file 32 MiB, cache cold, read each page:  +260 minor, +1 major
file 32 MiB, cache warm, read each page:  +260 minor, +0 major
file cold, MADV_RANDOM, 1000 pages:       +0 minor, +1000 major
child writes 4096 inherited pages (COW):  +4096 minor
child reads 4096 inherited pages:         +0 minor
```

What to notice:

- Mapping 64 MiB costs no fault. Writing one byte to each of the 16,384 pages costs exactly 16,384 minor faults. A second pass costs none, because the entries are now present.
- Reading a cold 32 MiB file through `mmap` caused one major fault and 260 minor faults, not 8,192 faults. The kernel reads ahead around the faulting page and its fault-around logic maps neighbouring pages that are already in the page cache in a single fault. After eviction with `posix_fadvise(POSIX_FADV_DONTNEED)` the first access paid disk I/O once; the remaining requests were served from the read-ahead window.
- With `MADV_RANDOM` (no read-ahead, no fault-around) each of 1,000 distinct pages produced one major fault, which is the worst case for a database or an index that is accessed randomly.
- The child that wrote to 4,096 inherited pages took 4,096 minor faults, each including a 4 KiB copy. The child that only read took none, because the page-table entries were copied at `fork`.

Faults of a JVM heap

The following run starts a JVM with a 256 MiB heap and `-version`, with and without `-XX:+AlwaysPreTouch`, which writes to every heap page during start-up. Fault counts were read from `getrusage` of the child process.

Minor faults and start-up time (measured on the build machine, typical) TERMINAL

```
$ python3 pretouch.py
-Xms256m -Xmx256m                          0.07 s  minor faults   4754
-Xms256m -Xmx256m -XX:+AlwaysPreTouch      0.29 s  minor faults  73150
-Xms256m -Xmx256m -XX:+AlwaysPreTouch THP  0.29 s  minor faults   4890
```

(The third line adds `-XX:+UseTransparentHugePages`; it was 0.19 s in a second run, so the timing is noisy.) A start without pre-touch takes about 4,750 faults, mostly for the JVM and library images. Pre-touching 256 MiB adds about 68,400 faults, which is 65,536 heap pages plus some auxiliary structures, and multiplies the start-up time by about four. With transparent huge pages the extra faults disappear (4,890 in total), because the 256 MiB heap is mapped by huge pages.

Transparent huge pages

thp.c C

```c
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/mman.h>
#include <sys/resource.h>
#include <time.h>

static long anon_huge_kb(void) {
    FILE *f = fopen("/proc/self/smaps_rollup", "r"); char l[128]; long v = -1;
    while (fgets(l, sizeof l, f)) if (sscanf(l, "AnonHugePages: %ld kB", &v) == 1) break;
    fclose(f); return v;
}

static void run(int huge) {
    size_t n = 32UL << 20;
    size_t two = 2UL << 20;
    char *raw = mmap(NULL, n + two, PROT_READ | PROT_WRITE,
                     MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
    char *p = (char *)(((unsigned long)raw + two - 1) & ~(two - 1));   /* 2 MiB aligned */
    if (huge) madvise(p, n, MADV_HUGEPAGE);
    struct rusage a, b; getrusage(RUSAGE_SELF, &a);
    struct timespec t0, t1; clock_gettime(CLOCK_MONOTONIC, &t0);
    for (size_t i = 0; i < n; i += 4096) p[i] = 1;
    clock_gettime(CLOCK_MONOTONIC, &t1); getrusage(RUSAGE_SELF, &b);
    double ms = (t1.tv_sec - t0.tv_sec) * 1e3 + (t1.tv_nsec - t0.tv_nsec) / 1e6;
    printf("%-22s: %6ld faults, AnonHugePages %6ld kB, first touch %.1f ms\n",
           huge ? "MADV_HUGEPAGE" : "default (madvise mode)",
           b.ru_minflt - a.ru_minflt, anon_huge_kb(), ms);
}

int main(void) { run(0); run(1); return 0; }
```

Run thp.c under a 400 MB address-space limit (typical output) TERMINAL

```
$ gcc -O1 -o thp thp.c && (ulimit -v 400000; ./thp)
default (madvise mode):   8194 faults, AnonHugePages      0 kB, first touch 28.6 ms
MADV_HUGEPAGE         :     16 faults, AnonHugePages  32768 kB, first touch 15.3 ms
```

The same 32 MiB took 8,194 faults of 4 KiB in the default configuration and 16 faults of 2 MiB with `MADV_HUGEPAGE`; `AnonHugePages` in `/proc/self/smaps_rollup` confirms that all 32 MiB were mapped by huge pages. Per-fault cost is about 2 to 3.5 microseconds for a 4 KiB page and about 1 millisecond for a 2 MiB page (dominated by zeroing 2 MiB). The total first-touch time is similar; the benefit appears later as fewer TLB misses, the cost as longer individual faults.

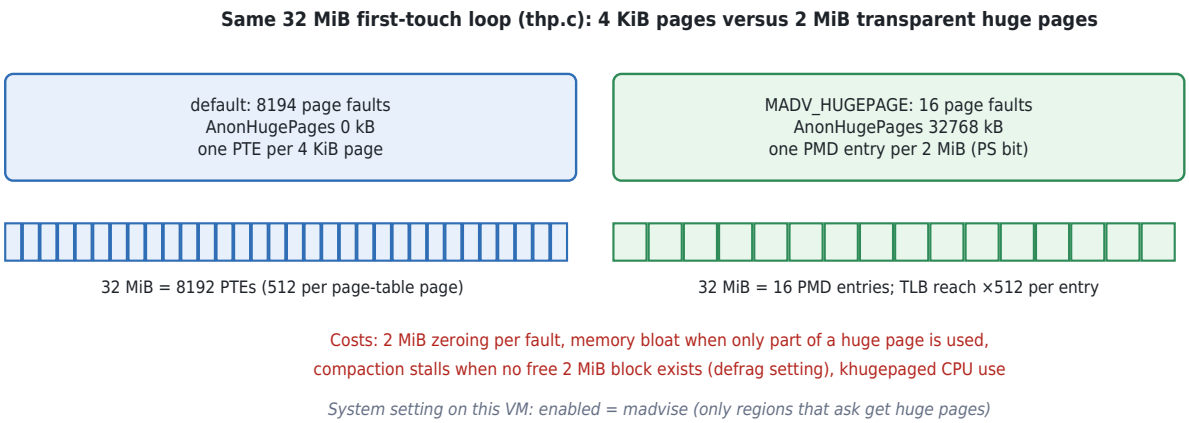


Figure 3.12. Thirty-two mebibytes mapped by 8,192 page-table entries versus 16 huge-page entries, with the costs that make transparent huge pages a trade-off.

TLB and cache effects

The program `tlb.c` (not listed) builds a random cyclic list in which every element lies on a different page and chases it for 20 million steps. It reports the time per step as the number of pages grows.

Pointer chase over N pages, one cache line per page (measured, typical) TERMINAL

```
$ gcc -O1 -o tlb tlb.c && (ulimit -v 300000; ./tlb)
pages  working set  ns per access
   16      64 KiB     3.1
   64     256 KiB     2.6
  256    1024 KiB     5.0
 1024    4096 KiB     8.4
 4096   16384 KiB    18.2
16384   65536 KiB    39.3
```

The time per access rises from 2.6 ns to 39.3 ns, a factor of 15, although the program executes the same instructions. The increase combines two effects that the program does not separate: the data no longer fits in the caches, and the translations no longer fit in the TLB. The measurement shows why the layout and the size of the working set matter even when no page fault occurs.

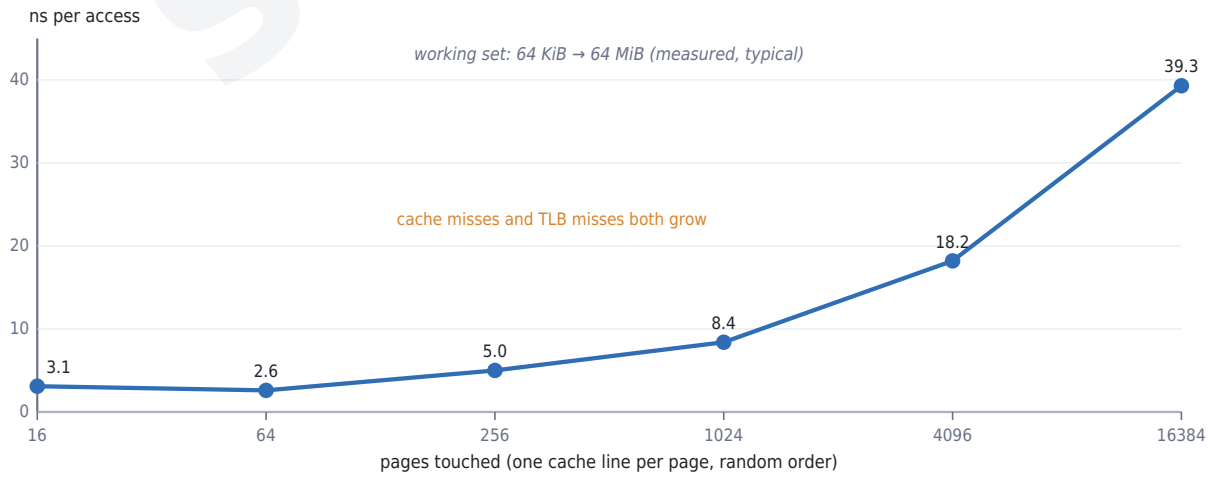


Figure 3.13. The measured time per memory access grows with the number of distinct pages touched, from 2.6 ns for 64 pages to 39.3 ns for 16384 pages.

Observing faults of a live process

Fault counters of a process and of the system — TERMINAL

```
$ awk '{print "minflt=" $10, "majflt=" $12}' /proc/self/stat
minflt=110 majflt=0
$ grep -E "^(pgfault|pgmajfault|thp_fault_alloc|thp_fault_fallback|compact_stall) " /proc/vmstat
pgfault 8507769
pgmajfault 325506
compact_stall 0
thp_fault_alloc 304
thp_fault_fallback 0
```

The fields 10 and 12 of `/proc/PID/stat` are the minor and major fault counters of the process; `/proc/vmstat` contains system-wide totals (`pgfault`, `pgmajfault`). A rising `pgmajfault` on a server that should be serving from memory is the primary sign of memory pressure.

3.2.4 Real-world incidents

Incident: latency spikes from transparent huge page compaction

Scenario. A latency-sensitive Spring Boot service with a 4 GiB heap runs on a VM image whose THP setting was changed to `always` and defragmentation to `direct compaction` by a performance-tuning script. After several days of uptime the 99.9th percentile latency shows isolated stalls of 100 ms to several seconds.

Symptoms. Latency histograms show periodic outliers unrelated to request size. GC logs show pauses with a high system time (`sys=`) component. CPU profiles show time in `compact_zone` or `khugepaged`.

Diagnosis. The THP policy and activity counters are read first.

THP configuration and counters (build machine values; the stall counters are the ones to watch) — TERMINAL

```
$ cat /sys/kernel/mm/transparent_hugepage/enabled
always [madvise] never
$ cat /sys/kernel/mm/transparent_hugepage/defrag
always defer defer+madvise [madvise] never
$ grep -E "^(thp_fault_alloc|thp_fault_fallback|compact_stall) " /proc/vmstat
compact_stall 0
thp_fault_alloc 304
thp_fault_fallback 0
```

On the build machine the settings are conservative and `compact_stall` is 0. On the affected host the combination `enabled = always` and `defrag = always` with a rising `compact_stall` and `thp_fault_fallback` indicates that page faults wait for memory compaction because no free 2 MiB block is available; the counters are cumulative since boot, so the growth rate between two samples is what matters.

Root cause. With `defrag = always` a fault that cannot find a free huge page compacts memory synchronously in the context of the application thread. Fragmentation grows with uptime and the page cache, so the problem appears only after days.

Fix. Immediate: `echo madvise > /sys/kernel/mm/transparent_hugepage/enabled` and `echo defer+madvise > /sys/kernel/mm/transparent_hugepage/defrag` (as root; both take effect at once). Permanent: persist the setting in the image (kernel command line `transparent_hugepage=madvise` or a systemd unit) and, if the JVM benefits from huge pages, enable `-XX:+UseTransparentHugePages` together with `-XX:+AlwaysPreTouch` so that the pages are obtained at start-up, not on the request path.

Prevention. Alert on the growth rate of `compact_stall`. Many database vendors (Redis, MongoDB, Oracle) recommend `madvise` or `never`; follow the guidance of each product.

Incident: slow first requests after a deployment

Scenario. After each rolling deployment of a service with `-Xms256m -Xmx2g`, the first minutes show elevated latency while the service is already receiving traffic. Warm-up runs of the load balancer are short.

Symptoms. CPU system time is high during the first minutes; the minor fault rate of the new process is in the tens of thousands per second; the latency falls without any change.

Diagnosis. The fault rate is sampled from the process counters.

Fault rate of a JVM whose live heap grows by 20 MiB per second (reproduction, Grow.java) TERMINAL

```
$ while [ -d /proc/$PID ]; do \
    awk '{print "minflt=" $10 " majflt=" $12}' /proc/$PID/stat; sleep 2; done
minflt=10283 majflt=0
minflt=21232 majflt=0
minflt=31497 majflt=0
minflt=41814 majflt=0
minflt=52139 majflt=0
```

The counter rises by about 10,300 every two seconds: 5,120 pages per 20 MiB step, which is one minor fault per newly used 4 KiB page. A heap that grows from 256 MiB to 2 GiB commits new pages as the allocation front advances. At 2 to 3.5 microseconds per first touch (measured above), 1.75 GiB of new heap means about 460,000 faults and roughly one to two seconds of CPU time spread over the warm-up.

Root cause. Lazy commit of the heap by demand paging, combined with a start-up traffic burst.

Fix. Set `-Xms` equal to `-Xmx` for steady services so that the heap does not resize, and use `-XX:+AlwaysPreTouch` when start-up time is acceptable (measured above: 0.07 s to 0.29 s for 256 MiB; it scales with the heap size, about 1 s per GiB on this machine). Use a readiness probe that opens only after warm-up requests.

Prevention. Measure start-up time with and without pre-touch before enabling it for large heaps: extrapolating the measured rate of about 1 second per GiB, pre-touching a 32 GiB heap can take about half a minute and may exceed the liveness probe timeout of the orchestrator.

3.2.5 Questions to expect

Questions and model answers

1. **Why are page tables multi-level?** A single-level table for a 48-bit space needs 512 GiB per process. A multi-level table creates lower-level tables only for ranges in use, so a process pays for its mappings, not for the whole address space.
2. **Walk through the translation of a virtual address on x86-64.** Split the address into four 9-bit indices and a 12-bit offset. Starting at the PGD whose address is in `CR3`, each entry selects the next table; the PTE yields the frame number; the physical address is frame × 4096 + offset. The TLB caches the result.
3. **What is the TLB and what happens on a context switch?** A cache of virtual-to-physical translations. Without tags it is flushed on a switch; with PCID the entries of several address spaces coexist, and only changed entries need invalidation.
4. **What is the difference between a minor and a major page fault?** A minor fault is resolved from memory (zero-fill, page cache hit, copy-on-write); a major fault needs I/O from a file or from swap and the task sleeps.
5. **How does `fork` avoid copying memory?** It copies the page tables, marks private pages read-only and shares the frames; the first write faults and the kernel copies that single page (copy-on-write).
6. **What is demand paging and why does `mmap` of a large file return immediately?** Mapping creates only a VMA; pages are read on first access. The cost is paid at the faults.
7. **When would you use huge pages, and what are the risks?** For large, long-lived heaps and databases with TLB-bound access patterns. Risks are memory bloat, compaction stalls and longer individual faults.
8. **A process shows `maj_flt` rising quickly. What does it mean?** The process is repeatedly reading pages that are not in memory: a cold file cache, a working set larger than available memory, or swapping. Check `vmstat` (`si`, `so`, `bi`) and `/proc/pressure/memory`.
9. **What happens on a write to a page mapped read-only?** If the VMA allows writing (copy-on-write or a page tracked for dirtiness), the kernel makes the page writable or copies it and resumes; otherwise `SIGSEGV` is sent.

KEY POINTS

Paging translates virtual to physical addresses with a four-level tree of 4 KiB tables and caches results in the TLB.

- A 48-bit address is four 9-bit indices plus a 12-bit offset; `CR3` locates the root table.
- The effective access time depends on the TLB hit ratio; working sets beyond the TLB reach cost time even without faults.
- Page faults are minor (no I/O) or major (I/O); first touch of anonymous memory costs one minor fault per 4 KiB page (measured 2 to 3.5 microseconds).
- Copy-on-write lets `fork` share frames; a child writing 4,096 inherited pages caused 4,096 faults in the measurement.
- Transparent huge pages reduce faults and TLB misses but can cause compaction stalls; the setting `madvise` is the conservative default.

3.3 Page Replacement, Working Sets and Swapping

Physical memory is smaller than the sum of the virtual memory in use. When a page fault needs a frame and no frame is free, the kernel must evict a resident page. The choice of the victim decides how many future faults occur. A good choice removes a page that will not be needed soon; a bad choice removes a page that is needed immediately, and the system spends its time moving pages between memory and disk instead of running programs. This chapter compares the classical algorithms on reference strings with verified counts, explains the working-set model and thrashing, and describes how Linux reclaims memory and uses swap.

3.3.1 Concept and mechanism

The replacement problem

A **reference string** is the sequence of page numbers accessed by a program. With *n* frames the memory holds at most *n* pages. A reference to a resident page is a hit; a reference to a non-resident page is a page fault. If all frames are in use, the algorithm chooses a victim. The goal is to minimise the number of faults. A **dirty** page (modified since it was loaded) must be written to its backing store before the frame is reused, which makes evicting clean pages cheaper.

Algorithm	Rule	Remarks
OPT (Belady's MIN)	Evict the page whose next use lies farthest in the future	Optimal; needs knowledge of the future; used as a yardstick
FIFO	Evict the page that has been resident longest	Simple; ignores use; suffers Belady's anomaly
LRU	Evict the page unused for the longest time	Good approximation of OPT; exact LRU needs a timestamp or list update on every access
Clock (second chance)	A hand sweeps a circular list; a page with reference bit 1 gets the bit cleared, a page with 0 is evicted	Approximates LRU using the hardware accessed bit
NRU	Classes by (referenced, modified); evict from the lowest non-empty class	Cheap; the bits are cleared periodically
LFU	Evict the least frequently used page	Old frequent pages stay too long without ageing

Classical replacement algorithms.

Belady's anomaly

Intuition says that more frames cannot cause more faults. For FIFO this is false. On the string 1 2 3 4 1 2 5 1 2 3 4 5, FIFO produces 9 faults with three frames and 10 faults with four frames. The cause is that FIFO does not have the **stack property**: the set of pages resident with *n* frames is not always a subset of the set resident with *n+1* frames. LRU and OPT do have this property, so their fault count never rises when frames are added.

sreeram

FIFO, 3 fr.

1	2	3	4	1	2	5	1	2	3	4	5
1	1	1	2	3	4	1	1	1	2	5	5
	2	2	3	4	1	2	2	2	5	3	3
		3	4	1	2	5	5	5	3	4	4
F	F	F	F	F	F	F	hit	hit	F	F	hit

9 page faults (F)

FIFO, 4 fr.

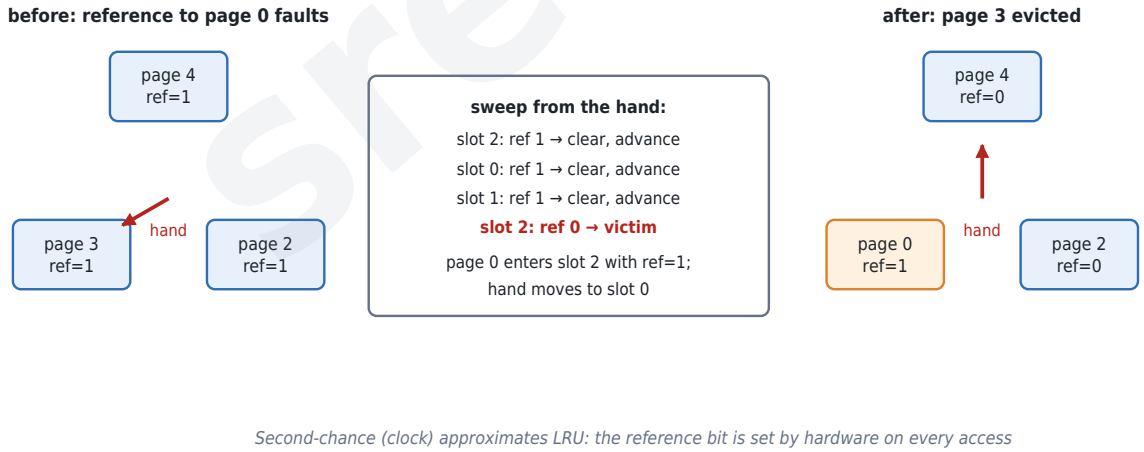
1	2	3	4	1	2	5	1	2	3	4	5
1	1	1	1	1	1	2	3	4	5	1	2
	2	2	2	2	2	3	4	5	1	2	3
		3	3	3	3	4	5	1	2	3	4
			4	4	4	5	1	2	3	4	5
F	F	F	F	hit	hit	F	F	F	F	F	F

10 page faults: more frames, more faults (Belady's anomaly)

Figure 3.14. FIFO on the string 1 2 3 4 1 2 5 1 2 3 4 5: the table shows the resident pages after every reference (oldest at the top); three frames produce nine faults and four frames produce ten.

The clock algorithm

Maintaining exact LRU order would require updating a list on every memory access, which is impossible in software because ordinary loads and stores do not trap. The hardware sets the **accessed bit** in the page-table entry for free. The clock algorithm uses it: the resident pages form a circle with a hand. When a victim is needed, the hand examines the page it points to. If the bit is 1, the page was used since the last sweep, so the bit is cleared and the hand advances (the page gets a second chance). If the bit is 0, the page is evicted and replaced by the new page, which enters with its bit set.



Second-chance (clock) approximates LRU: the reference bit is set by hardware on every access

Figure 3.15. One clock step from the trace below: all three reference bits are 1, so the hand clears them one by one, returns to slot 2 and evicts page 3 in favour of page 0.

Working sets and thrashing

The **working set** W(t, Δ) of a process is the set of pages it referenced during the last Δ memory references. A process runs efficiently if its working set is resident. If the sum of the working sets of all runnable processes exceeds the number of frames, every process faults continuously: each fault evicts a page that another process (or the same one) needs next. The CPU waits for the disk, CPU utilisation falls, and a scheduler that reacts to low utilisation by starting more processes makes the situation worse. This collapse is **thrashing**.

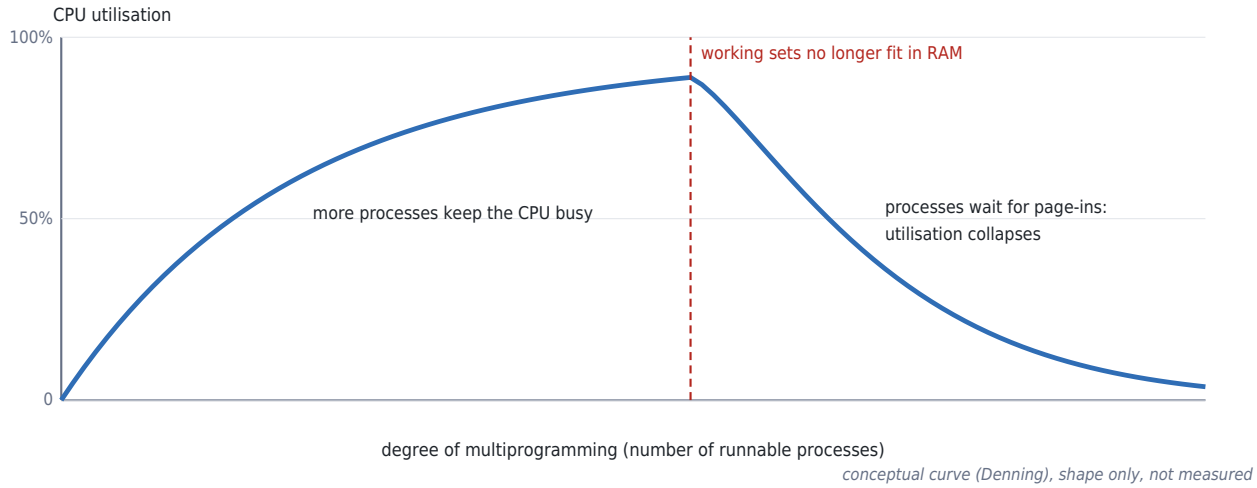


Figure 3.16. CPU utilisation first rises with the number of runnable processes and then collapses once the working sets no longer fit in memory (conceptual shape).

Remedies are to reduce the degree of multiprogramming (suspend or swap out whole processes), to add memory, or to reduce the working set (a smaller heap, a smaller data structure, a better access pattern). A page-fault-frequency policy gives a process more frames when its fault rate is above an upper threshold and takes frames away below a lower one.

Reclaim in Linux

Linux does not run FIFO or pure clock. Anonymous and file-backed pages are kept on **active** and **inactive** lists per memory zone and per cgroup. A page enters the inactive list; if it is referenced again while on that list it moves to the active list; when the active list becomes too large, its oldest pages are demoted. Reclaim scans the inactive list: clean file pages are dropped, dirty file pages are written back first, anonymous pages are written to swap (if swap exists) and then freed. Newer kernels (6.1 and later) can replace this with a multi-generational LRU (`CONFIG_LRU_GEN`); the build machine kernel does not expose `/sys/kernel/mm/lru_gen`, so the classical lists are in use.

Reclaim runs in two contexts. The background thread `kswapd` wakes when free memory falls below the low watermark and works until the high watermark is reached. If an allocation cannot be satisfied, the allocating task performs **direct reclaim** itself and waits; this is where applications observe memory pressure as latency.

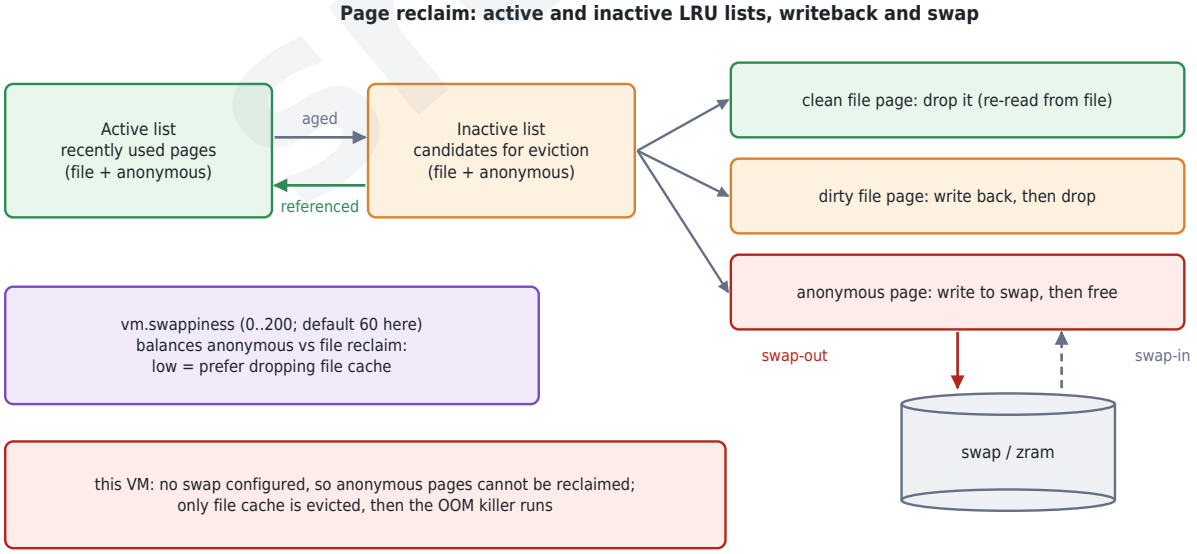


Figure 3.17. Reclaim moves pages from the active to the inactive list; clean file pages are dropped, dirty file pages are written back and anonymous pages are written to swap, with vm.swappiness weighing anonymous against file reclaim.

Swap and swappiness

Swap space is a partition or file that holds anonymous pages that have no other backing store. Without swap, the kernel can reclaim only file-backed pages; anonymous memory can be freed only by killing processes. The parameter `vm.swappiness` (0 to 200 on current kernels, default 60) sets the relative cost of reclaiming anonymous pages compared with file pages: values near 0 make the kernel prefer dropping file cache; high values make it swap anonymous pages earlier to keep file cache. Each memory cgroup has its own `memory.swappiness`. The value 0 does not disable swapping; it avoids it until memory is nearly exhausted.

zram is a compressed block device in RAM used as swap. Pages that are swapped out are compressed and stored in memory, which costs CPU time but no disk I/O and typically saves memory when the data compresses well. It is often enabled on small machines and desktops.

On servers, swap has two conflicting effects. It allows inactive anonymous pages (for example, an idle part of a heap) to leave memory, which leaves more room for the page cache. It also converts memory exhaustion into severe slowdowns instead of a quick kill, because a swapped-out heap region must be read back when the garbage collector or the application touches it. Container platforms often run with swap disabled, so that a memory limit produces a clear OOM kill instead of slow thrashing.

3.3.2 Worked trace: reference string 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

The trace uses LRU with three frames. The frame column lists the resident pages from least recently used (left) to most recently used (right).

Step	Ref	Frames (LRU to MRU)	Result
1	7	7	fault
2	0	7 0	fault
3	1	7 0 1	fault
4	2	0 1 2	fault, 7 evicted
5	0	1 2 0	hit
6	3	2 0 3	fault, 1 evicted
7	0	2 3 0	hit
8	4	3 0 4	fault, 2 evicted
9	2	0 4 2	fault, 3 evicted
10	3	4 2 3	fault, 0 evicted
11	0	2 3 0	fault, 4 evicted
12	3	2 0 3	hit
13	2	0 3 2	hit
14	1	3 2 1	fault, 0 evicted
15	2	3 1 2	hit
16	0	1 2 0	fault, 3 evicted
17	1	2 0 1	hit
18	7	0 1 7	fault, 2 evicted
19	0	1 7 0	hit
20	1	7 0 1	hit

LRU, three frames, string B.

LRU incurs 12 faults. The clock trace of the same string with three frames (slots 0 to 2, hand starting at slot 0) reaches the state of the diagram at step 11: the frames hold pages 4, 2, 3 with all reference bits 1; the reference to page 0 faults, the hand sweeps once around clearing the bits, and slot 2 (page 3) is replaced by page 0 with bit 1. The clock run totals 14 faults.

sreram

3.3.3 Hands-on: verified counts and a thrashing run

The algorithms in code

The program implements the four algorithms and runs both reference strings with three and four frames. Python 3 is used because the logic is short; the counts are verified by the run itself.

replacement.py PYTHON

```
def fifo(refs, n):
    frames, faults = [], 0
    for r in refs:
        if r not in frames:
            faults += 1
            if len(frames) == n:
                frames.pop(0)                   # oldest arrival leaves
            frames.append(r)
    return faults


def lru(refs, n):
    frames, faults = [], 0                      # list ordered from least to most recent
    for r in refs:
        if r in frames:
            frames.remove(r)
        else:
            faults += 1
            if len(frames) == n:
                frames.pop(0)                   # least recently used leaves
        frames.append(r)
    return faults


def clock(refs, n):
    frames, bits, hand, faults = [None] * n, [0] * n, 0, 0
    for r in refs:
        if r in frames:
            bits[frames.index(r)] = 1           # hardware sets the reference bit
            continue
        faults += 1
        while bits[hand]:                       # second chance: clear and move on
            bits[hand] = 0
            hand = (hand + 1) % n
        frames[hand], bits[hand] = r, 1
        hand = (hand + 1) % n
    return faults


def opt(refs, n):
    frames, faults = [], 0
    for i, r in enumerate(refs):
        if r in frames:
            continue
        faults += 1
        if len(frames) == n:
            future = refs[i + 1:]
            # evict the page whose next use is farthest away (never used again: infinity)
            far = lambda p: future.index(p) if p in future else len(refs) + 1
            frames.remove(max(frames, key=far))
        frames.append(r)
    return faults


STRINGS = {
    "A (Belady)": [1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5],
    "B (textbook)": [7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1],
}
print("%-14s %-6s %5s %5s %6s %5s" % ("string", "frames", "FIFO", "LRU", "CLOCK", "OPT"))
for name, refs in STRINGS.items():
    for n in (3, 4):
        print("%-14s %-6d %5d %5d %6d %5d" %
              (name, n, fifo(refs, n), lru(refs, n), clock(refs, n), opt(refs, n)))
```

sreram

Run replacement.py TERMINAL

```
$ python3 replacement.py
string         frames  FIFO   LRU  CLOCK   OPT
A (Belady)     3          9    10      9     7
A (Belady)     4         10     8     10     6
B (textbook)   3         15    12     14     9
B (textbook)   4         10     8      9     8
```

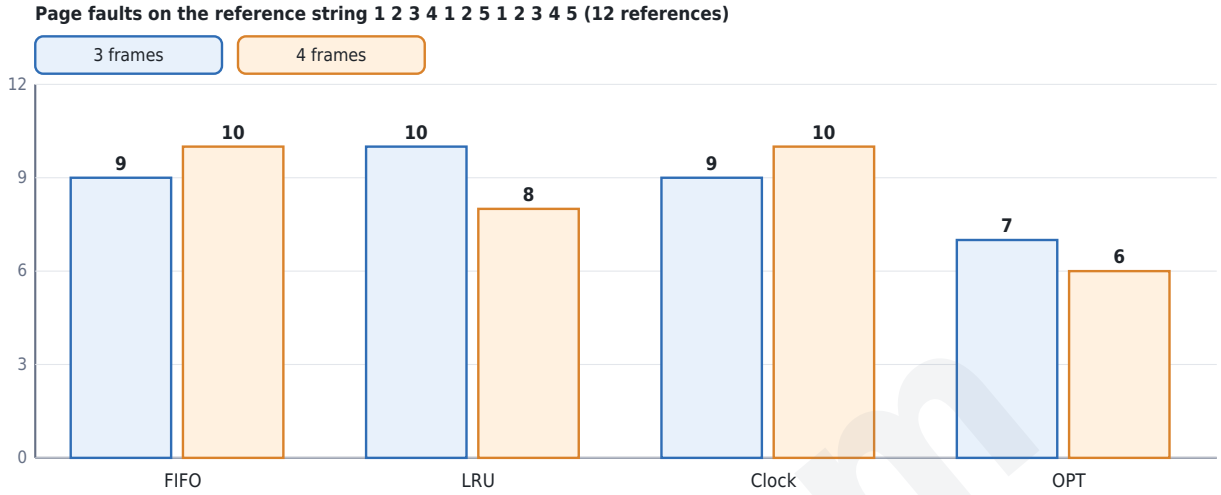


Figure 3.18. Fault counts of the four algorithms on string A with three and four frames: FIFO and Clock rise with more frames, LRU and OPT fall.

What to notice:

- String A with FIFO reproduces the anomaly: 9 faults with three frames, 10 with four. Clock, which behaves like FIFO on this string, shows it as well (9 and 10).
- LRU on string A is worse than FIFO with three frames (10 against 9) although it is better with four frames (8). No online algorithm wins on every string; the guarantee of LRU is the stack property, not a lower count on each input.
- OPT is the lower bound in every row: 7 and 6 for string A, 9 and 8 for string B. On string B with three frames the ordering is OPT 9, LRU 12, Clock 14, FIFO 15, which is the usual ranking.

Thrashing in a memory cgroup

The program `thrash.c` maps a file of a given size and performs 300,000 random reads of one byte each through the mapping, with `MADV_RANDOM` so that no read-ahead hides the effect. It was run in a memory cgroup with a 40 MiB limit and without swap. A file-backed working set is used so that eviction does not kill the process: the kernel simply drops and re-reads clean pages. A fresh file was created for each run.

Random reads in a 40 MiB memory cgroup (measured, typical) TERMINAL

```
$ for ws in 16 32 48 64; do rm -f ws.bin; ./thrash $ws; done
working set  16 MiB: 300000 random reads in   0.04 s, major faults   4096, minor     74
working set  32 MiB: 300000 random reads in   0.07 s, major faults   8192, minor     74
working set  48 MiB: 300000 random reads in   0.57 s, major faults  60595, minor     76
working set  64 MiB: 300000 random reads in   0.54 s, major faults 120062, minor     75
```

When the working set fits (16 and 32 MiB) the number of major faults equals the number of pages: one per page for the first touch, none afterwards. When it exceeds the limit (48 and 64 MiB) the kernel evicts pages that are needed again, and the counts jump to 60,595 and 120,062 faults, which is 4.9 and 7.3 times the number of pages (12,288 and 16,384). The run time rises from 0.07 s to 0.57 s. Because the storage of the build machine is fast and cached by the host, the slowdown is only a factor of eight; on a network disk or a rotating disk each major fault costs milliseconds and the same pattern produces a standstill.

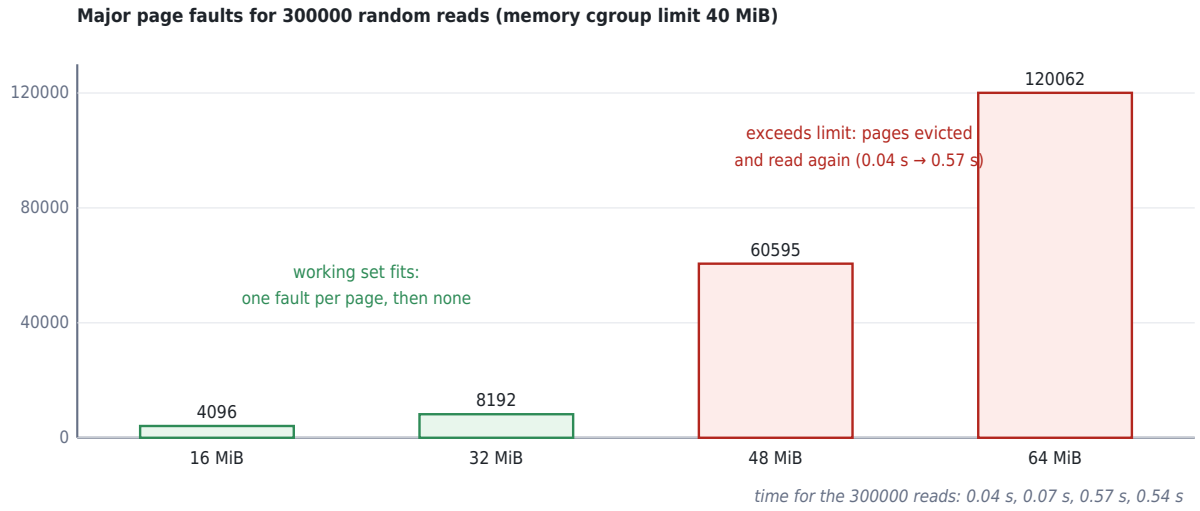


Figure 3.19. The major-fault count is flat while the working set fits within the 40 MiB limit and rises by an order of magnitude when it exceeds the limit.

The pressure stall information of the kernel recorded the episode. The counters in `/proc/pressure/memory` are cumulative microseconds during which tasks were stalled on memory.

Pressure stall information before and after the 64 MiB run (typical output) TERMINAL

```
$ cat /proc/pressure/memory
some avg10=0.00 avg60=0.05 avg300=0.35 total=3063149
full avg10=0.00 avg60=0.00 avg300=0.00 total=163924
$ ./thrash 64
working set  64 MiB: 300000 random reads in   0.43 s, major faults 120576, minor     75
$ cat /proc/pressure/memory
some avg10=0.00 avg60=0.05 avg300=0.35 total=3148784
full avg10=0.00 avg60=0.00 avg300=0.00 total=249024
```

During a run of 0.43 s the `some` total grew by 85.6 ms and the `full` total by 85.1 ms. `some` is the time at least one task was stalled on memory; `full` is the time all non-idle tasks were stalled at once. (The averages did not change visibly because the episode was shorter than the 10 second window.) PSI is the most direct indicator of memory pressure available on a modern kernel.

Swap configuration of the machine

Swap and swappiness on the build machine TERMINAL

```
$ swapon --show; cat /proc/sys/vm/swappiness
60
```

`swapon --show` prints nothing: no swap is configured on the build machine. All reclaim therefore concerns file pages, and anonymous memory is protected only by the OOM killer (see the chapter on memory limits and the OOM killer).

3.3.4 Real-world incidents

Incident: thrashing after a traffic increase

Scenario. A batch of report-generation jobs shares a 16 GiB virtual machine with a Spring Boot service. A new report type loads a larger data set, and the combined working sets of the service and the jobs exceed physical memory. The machine has a 4 GiB swap partition.

Symptoms. Response times rise from milliseconds to tens of seconds. Load average grows far above the CPU count while CPU utilisation is low. Disk reads and writes are high even though the application performs little I/O. `ssh` is sluggish.

Diagnosis. The commands are run in the order below. The output shown is typical for this condition (the build machine has no swap and cannot reproduce a swap storm; the page-fault and pressure mechanism was reproduced above).

Typical diagnosis of a swap storm (illustrative values) TERMINAL

```
$ vmstat 2 3
procs -----------memory---------- ---swap-- -----io---- -system-- ------cpu-----
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st
 3 18 3984112  61204   1520  93216 4180 5320  4890  6122 1840 3211  4  6 12 78  0
 2 21 3991008  58800   1488  90112 3920 6010  4410  6890 1790 3004  3  7  9 81  0
$ cat /proc/pressure/memory
some avg10=87.41 avg60=79.10 avg300=52.03 total=912330140
full avg10=71.22 avg60=64.30 avg300=40.11 total=745100233
```

Reading the output: `b = 18` tasks are in uninterruptible sleep (waiting for I/O); `si` and `so` (4,180 and 5,320 KiB per second) show continuous swap-in and swap-out; `swpd` is almost the whole swap space; `free` and `cache` are tiny, so the page cache cannot absorb the load; `wa` (I/O wait) is 78 to 81 percent; PSI `full` of 71 percent means that for most of the time all non-idle tasks were stalled on memory. Together these identify thrashing, not a CPU or disk hardware problem. `ps -eo pid,rss,comm --sort=-rss | head` then names the processes with the largest resident sets.

Root cause. The sum of the working sets exceeds RAM, the scheduler keeps all processes runnable, and each fault evicts a page needed by another process.

Fix. Immediate: stop or pause the report jobs (`kill -STOP PID` freezes a process and lets its pages be reclaimed without further faults; `kill -CONT` resumes it), or restart the service with a smaller heap. Permanent: separate the batch workload from the service (different hosts or cgroups with `memory.high` and `memory.max`), size the heap so that heap, metaspace, stacks and the page cache working set fit, and set `-Xmx` below the memory the host can provide.

Prevention. Alert on PSI `full` above a few percent for a sustained period, on `si/so` greater than zero for more than a minute, and on the major fault rate. Run batch work under cgroup limits so that it is reclaimed before the service is.

Incident: garbage-collection pauses caused by swapped-out heap

Scenario. A Java service with a 6 GiB heap runs on a host with swap enabled and `vm.swappiness = 60`. Overnight the service is idle and a backup job reads large files. In the morning the first requests trigger a full collection.

Symptoms. The first full GC takes 40 seconds instead of 2 seconds. GC logs show a large difference between real time and CPU time (`real` much greater than `user + sys`). `vmstat` shows `si` high during the pause.

Diagnosis. The swapped-out size of the process is read from its status and the pause is correlated with swap-in.

Check how much of the process is in swap (healthy JVM of the build machine, no swap present) TERMINAL

```
$ grep -E "VmRSS|VmSwap" /proc/PID/status
VmRSS:    163188 kB
VmSwap:        0 kB
```

On the affected host `VmSwap` was several gigabytes while `VmRSS` was small; a large `VmSwap` value confirms that heap pages were swapped out. A garbage collector reads every live object, so it touches every page of the heap and the kernel must read the swapped pages back one fault at a time.

Root cause. The page cache of the backup competed with the idle heap, and the kernel moved cold anonymous pages to swap to enlarge the cache. The collector then faulted them all back.

Fix. Immediate: `swapoff -a && swapon -a` (only when enough free memory exists) forces the pages back before traffic arrives. Permanent: lower `vm.swappiness` (for example 1 to 10) on hosts that run JVMs, or disable swap for the service cgroup with `memory.swappiness=0` (cgroup v1) or `memory.swap.max=0` (cgroup v2); lock the heap with `-XX:+AlwaysPreTouch` and size the heap to the working set; throttle the backup with `ionice` and `posix_fadvise(DONTNEED)`.

Prevention. Monitor `VmSwap` of the main process; alert when it is not zero for a latency-sensitive service. Review the swappiness value in the host image.

3.3.5 Questions to expect

Questions and model answers

1. **State Belady's anomaly and name an algorithm that is immune.** For some reference strings FIFO produces more faults with more frames (string 1 2 3 4 1 2 5 1 2 3 4 5: 9 faults with three frames, 10 with four). LRU and OPT are immune because they are stack algorithms.
2. **Why is LRU not implemented exactly?** It needs an ordering update on every reference, and the CPU does not trap ordinary accesses. Systems approximate LRU with the hardware accessed bit (clock, active/inactive lists).

3. **Describe the clock algorithm.** Pages form a circular list with a reference bit. The hand advances; a page with bit 1 has its bit cleared and is skipped, a page with bit 0 is evicted. The sweep approximates least recently used.
4. **What is the working set and how does it relate to thrashing?** The set of pages referenced in the last Δ references. If the combined working sets exceed memory, processes fault constantly and CPU utilisation collapses; this is thrashing.
5. **How do you detect thrashing on a Linux server?** High `b` and `wa` in `vmstat`, non-zero `si` and `so`, rising `pgmajfault`, high load with idle CPU, and `/proc/pressure/memory` showing high `some` and `full` values.
6. **What does `vm.swappiness` control?** The balance between reclaiming anonymous pages (swap) and file pages (cache drop). It is a preference, not a switch: 0 does not disable swap.
7. **Why do many container platforms disable swap?** A limit with swap hides memory exhaustion behind severe slowdowns and makes latency unpredictable; an OOM kill at the limit is a clear failure that the orchestrator can handle by restarting.
8. **What is direct reclaim and why does it matter?** The allocating thread reclaims memory itself when `kswapd` cannot keep up. It adds latency to the application's own allocation, which shows up as pauses and as `pgscan_direct` in `/proc/vmstat`.
9. **Does adding RAM always reduce page faults?** For stack algorithms such as LRU yes (never more); for FIFO not necessarily. In practice more memory lets the page cache hold more of the working set, so faults usually fall.

KEY POINTS

Replacement chooses which resident page to evict; the fault count depends on the algorithm and the access pattern.

- OPT is the unreachable optimum; LRU approximates it; FIFO can show Belady's anomaly (9 then 10 faults on the reference string above).
- Hardware reference bits enable clock and the active/inactive lists used by Linux.
- Thrashing occurs when the working sets exceed memory; measured here as 4,096 and 8,192 faults for fitting working sets against 60,595 and 120,062 beyond the limit.
- `vmstat` (`b`, `wa`, `si`, `so`), `pgmajfault` and PSI (`/proc/pressure/memory`) detect memory pressure.
- Swap trades OOM kills for slowdowns; tune `vm.swappiness` and cgroup swap limits for latency-sensitive Java services.

3.4 The Page Cache and Memory Used for File I/O

Disks are three to five orders of magnitude slower than memory. Linux hides this gap by keeping the contents of recently used files in otherwise idle RAM, the **page cache**. Every `read`, every `write` and every file-backed `mmap` passes through it. As a consequence, a Linux machine that has been running for a while shows almost no free memory, and an inexperienced reader of `free` concludes that memory is exhausted. This chapter explains what the cache holds, how data moves through it, how the fields of `free` and `/proc/meminfo` are to be read, and why the cache matters for Java services and containers.

3.4.1 Concept and mechanism

What the page cache stores

The page cache holds pages of file data keyed by (inode, page index) and also the metadata of block devices (the former "buffers"). All processes share it: the second process that reads a file finds the pages the first one loaded. Since Linux 2.4 the former buffer cache is unified with the page cache: a block-device buffer is a page of the cache with attached buffer heads, and the `Buffers` field of `/proc/meminfo` counts the part that holds raw block-device data such as file-system metadata.

Path	Behaviour
`read()`	The kernel copies from a cache page to the user buffer; a miss reads the page (with read-ahead) from the device first.
`write()`	The kernel copies the user buffer into a cache page and marks it dirty; the call returns before any I/O happens.
`mmap()`	The page table points directly at the cache page; no copy, no system call per access; a fault loads a missing page.
`O_DIRECT`	Bypasses the cache; the device transfers to and from the user buffer; the caller handles alignment and caching.

Path	Behaviour
`fsync()`	Forces the dirty pages of the file (and the journal) to the device and waits.

Access paths to file data.

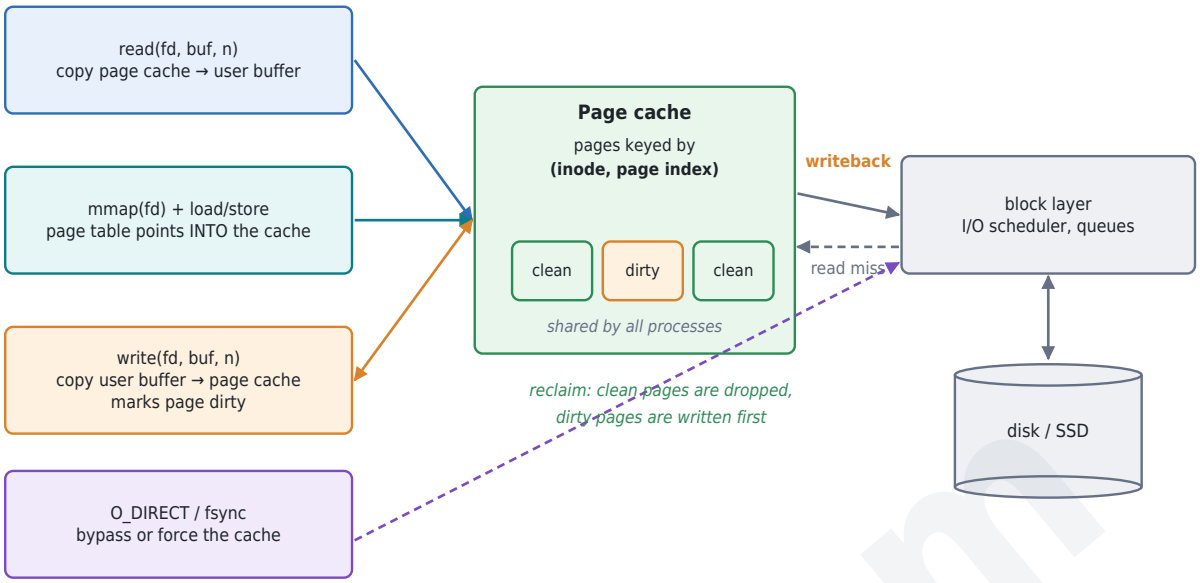


Figure 3.20. The four access paths meet in the page cache: read and write copy data, mmap maps the cache pages themselves, and writeback and read misses connect the cache to the block layer.

Read-ahead

When the kernel detects sequential access it reads additional pages before they are requested. The default window starts small and grows up to the device read-ahead setting (`blockdev --getra`, typically 128 KiB). The measurement in the chapter on paging showed the effect: reading a cold 32 MiB file through `mmap` caused one major fault, not thousands, and a `pread` of 16 MiB left 17 MiB in the cache (below).

Dirty pages and writeback

A page that has been modified in memory and not yet written to the device is **dirty**. Kernel flusher threads write dirty pages back in three situations: the page has been dirty longer than `vm.dirty_expire_centisecs` (30 seconds here), the amount of dirty memory exceeds `vm.dirty_background_ratio` percent of the reclaimable memory (10 here), or an application calls `fsync`. If dirty memory reaches `vm.dirty_ratio` (20 here), the writing process itself is throttled and must help write back before it may dirty more pages. These percentages refer to available memory (free plus reclaimable), not to total RAM. The flusher wakes every `vm.dirty_writeback_centisecs` (5 seconds here).

Writeback tunables of the build machine — TERMINAL

```
$ for f in dirty_ratio dirty_background_ratio dirty_expire_centisecs dirty_writeback_centisecs; do \
    echo "$f=$(cat /proc/sys/vm/$f)"; done
dirty_ratio=20
dirty_background_ratio=10
dirty_expire_centisecs=3000
dirty_writeback_centisecs=500
```

The consequence for durability is that a successful `write` does not mean that the data is on the device. Between the write and the writeback the data exists only in RAM and is lost on a power failure or a kernel crash. The chapter on buffered I/O and durability in the storage part covers `fsync`, `fdatasync` and the log-structured systems that depend on them.

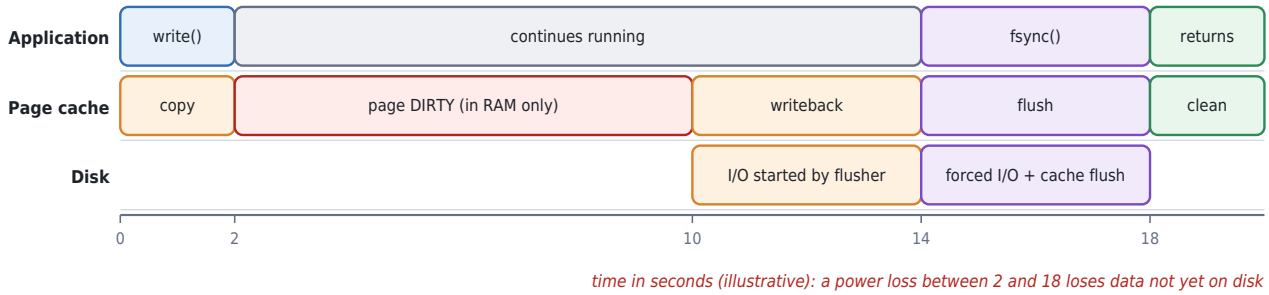


Figure 3.21. Lifecycle of written data: the page is dirty in RAM until the flusher or fsync writes it; the window between the write and the flush is the crash-loss window.

Reclaim of cache pages

Clean cache pages are the cheapest memory to reclaim: the kernel simply drops them, and a later access reads them again. Dirty pages must be written first. Therefore memory used by the cache is "free in an emergency", with two exceptions. Pages of `tmpfs` and shared anonymous memory (`Shmem`) appear in the `Cached` field but cannot be dropped without swap. Pages that are mapped and actively used by processes are kept by the active list.

3.4.2 Worked trace: reading the output of free

The following figures were taken from the build machine after the cache experiment described later. `free -m` prints mebibytes.

free and /proc/meminfo (build machine, typical) TERMINAL

```
$ free -m
               total        used        free      shared  buff/cache   available
Mem:            8029         580        6959          14         714        7449
Swap:              0           0           0
$ grep -E "^(MemTotal|MemFree|MemAvailable|Buffers|Cached|Dirty|Shmem|AnonPages|Mapped):" \
    /proc/meminfo
MemTotal:        8222320 kB
MemFree:         7125860 kB
MemAvailable:    7627648 kB
Buffers:           13408 kB
Cached:           706036 kB
Dirty:                 0 kB
Shmem:             14684 kB
AnonPages:        296256 kB
Mapped:           196448 kB
```

Field	Meaning
`total`	`MemTotal` (8222320 kB = 8029 MiB): RAM usable by the kernel (hardware reservations excluded).
`free`	`MemFree`: pages that hold nothing at all. Unused memory is wasted, so this value is expected to be small on a busy host.
`buff/cache`	`Buffers + Cached + SReclaimable` (13408 + 706036 + slab 12468 kB, about 714 MiB): file data, block metadata and reclaimable kernel caches.
`shared`	`Shmem`: tmpfs and shared memory (14 MiB); counted in `Cached` but not reclaimable without swap.
`used`	`total − available` in current procps-ng (older versions: `total − free − buff/cache`). It approximates memory that cannot be dropped without consequence: anonymous pages (`AnonPages`, 289 MiB), kernel stacks, unreclaimable slab and page tables.
`available`	`MemAvailable`: the kernel estimate of memory obtainable without swapping: free pages plus the reclaimable part of the cache, minus the watermarks that must be kept. Use this value, not `free`.

Meaning of the fields.

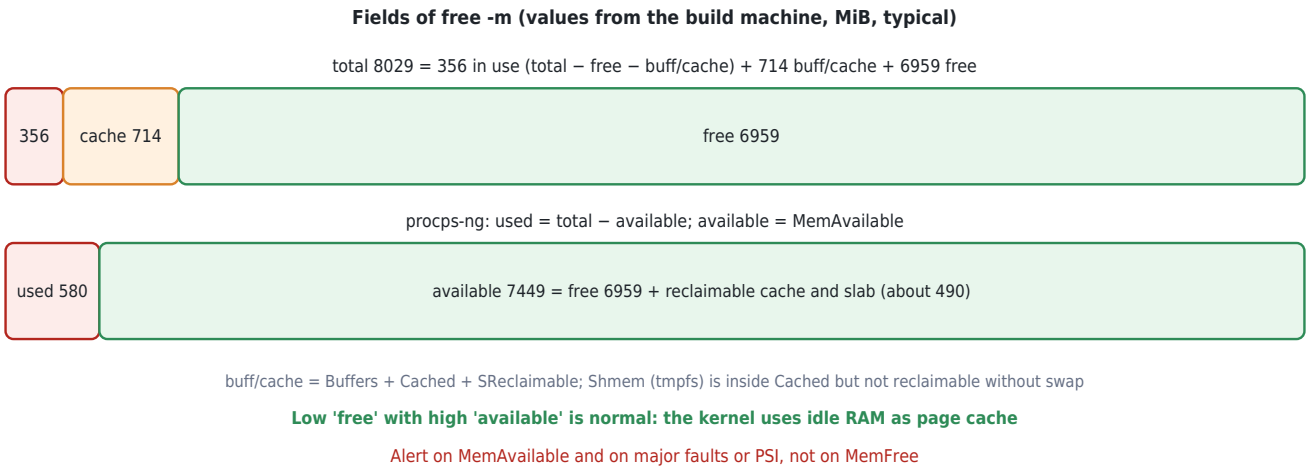


Figure 3.22. The relation between the fields of free: total is the sum of used, free and buff/cache, and available adds the reclaimable part of the cache to the free pages.

The columns do not add up to the total in current procps-ng: 580 (used) + 6959 (free) + 714 (buff/cache) = 8253, which exceeds 8029, because `used` is defined as `total − available` and the cache is partly counted in both `used` and `buff/cache`. Under the older definition, `total − free − buff/cache`, the used memory would be 356 MiB. For capacity decisions only `available` is needed.

3.4.3 Hands-on: the cache in action

Which pages of a file are cached

The program `cached.c` writes a 64 MiB file, evicts it, reads part of it and asks the kernel which pages are resident with the system call `mincore`.

cached.c C

```c
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>

static void report(const char *tag, int fd, size_t size) {
    void *m = mmap(NULL, size, PROT_READ, MAP_SHARED, fd, 0);
    size_t pages = size / 4096, n = 0;
    unsigned char *vec = malloc(pages);
    mincore(m, size, vec);
    for (size_t i = 0; i < pages; i++) n += vec[i] & 1;
    printf("%-30s %5zu of %zu pages in the page cache\n", tag, n, pages);
    munmap(m, size);
    free(vec);
}

int main(void) {
    size_t size = 64UL << 20;
    int fd = open("/tmp/osw_o3/fc.bin", O_RDWR | O_CREAT | O_TRUNC, 0644);
    char *buf = calloc(1, 1 << 20);
    for (int i = 0; i < 64; i++) if (write(fd, buf, 1 << 20) < 0) return 1;
    fsync(fd);
    report("after writing 64 MiB:", fd, size);
    posix_fadvise(fd, 0, 0, POSIX_FADV_DONTNEED);
    report("after POSIX_FADV_DONTNEED:", fd, size);
    for (int i = 0; i < 16; i++) if (pread(fd, buf, 1 << 20, (off_t)i << 20) < 0) return 1;
    report("after reading the first 16 MiB:", fd, size);
    unlink("/tmp/osw_o3/fc.bin");
    return 0;
}
```

Run cached.c (typical output) TERMINAL

```
$ gcc -O1 -o cached cached.c && ./cached
after writing 64 MiB:          16384 of 16384 pages in the page cache
after POSIX_FADV_DONTNEED:         0 of 16384 pages in the page cache
after reading the first 16 MiB:  4352 of 16384 pages in the page cache
```

The written file stays completely in the cache. `posix_fadvise(DONTNEED)` drops the clean pages. Reading 16 MiB (4,096 pages) leaves 4,352 pages resident: the extra 256 pages (1 MiB) are the read-ahead window beyond the last request.

Cost of a cold versus a warm read

The script reads a 128 MiB file after writing it, after evicting it and again, and reports the cache size and the time.

Page cache effect on a 128 MiB file (measured, typical; the cache columns include other activity of the machine) TERMINAL

```
$ bash pc.sh
after write+sync:        Cached 800 MiB   free=6577 buff/cache=821 available=7175
after eviction (fadvise): Cached 672 MiB   free=6707 buff/cache=690 available=7175
after cold read:         Cached 800 MiB   free=6580 buff/cache=818 available=7176   [.044624081 s]
after warm read:         Cached 800 MiB   free=6580 buff/cache=818 available=7177   [.021703637 s]
after rm:                Cached 673 MiB   free=6706 buff/cache=690 available=7175
```

What to notice:

- `Cached` rises by 128 MiB after the write and falls by the same amount after the eviction. The `free` column falls and rises by the same 130 MiB, while `available` does not change at all (7175 to 7177 MiB): the cache is memory that remains available to applications.
- The cold read took 45 ms and the warm read 22 ms. The storage of the build machine is a virtual disk with host-side caching, which makes the cold read unusually fast (2.9 GB/s); on a physical SSD or a network volume the difference is one to three orders of magnitude.
- `rm` returns the pages immediately because the file no longer exists.

The cache inside a memory cgroup

Page-cache pages are charged to the memory cgroup of the process that first touched them. A limit therefore bounds the cache as well, and the kernel reclaims cache pages of the group when the limit is approached. A cgroup with a 64 MiB limit was used to write 40 MiB and then 150 MiB more.

Cache charged to a 64 MiB memory cgroup (cgroup v1; typical output) TERMINAL

```
$ dd if=/dev/zero of=cg.bin bs=1M count=40; sync
$ grep -E "^(cache|rss|inactive_file) " memory.stat; cat memory.usage_in_bytes
cache 41947136
rss 286720
inactive_file 41947136
43864064
$ dd if=/dev/zero of=cg2.bin bs=1M count=150; sync
$ grep -E "^(cache|rss) " memory.stat; cat memory.usage_in_bytes; cat memory.failcnt
cache 62918656
rss 278528
65548288
130
```

After the first write the usage was 41.9 MB of cache and only 0.29 MB of anonymous memory. After the second write, which exceeded the limit by 86 MiB, the usage stayed at 65.5 MB, the cache was held at 62.9 MB, `failcnt` counted 130 limit hits and no process was killed: the kernel reclaimed clean cache pages to make room. This explains two facts about containers. The `memory.usage_in_bytes` value (and Docker's reported memory usage) includes the cache and therefore approaches the limit on any container that does file I/O, without indicating a problem. Kubernetes monitors the **working set**, defined as usage minus inactive file pages, to separate reclaimable cache from memory in use.

Mapping files and the JVM

The JVM maps the JDK class image (`lib/modules`, 137 MiB), its own shared library and the class-data-sharing archive; these pages are part of the page cache and of the process resident set (`RssFile`, 23,600 kB in the run shown in the first chapter of this part), and they are shared by all JVMs on the host. Java applications can map files explicitly with `FileChannel.map`, which returns a `MappedByteBuffer` backed by cache pages; the data is then accessed without `read` calls and without a copy into the Java heap. A mapped page that is not cached costs a major fault inside what looks like an ordinary memory access, which is a frequent cause of unexplained pauses in programs using memory-mapped files, because a JVM safepoint cannot interrupt a thread that waits in a page fault.

3.4.4 Real-world incidents

Incident: false alarm on low free memory

Scenario. An alert named "Memory exhausted" fires when `MemFree / MemTotal` is below 10 percent. A search service that reads large index files has been running for two weeks on a 16 GiB host.

Symptoms. The alert fires daily. `free` shows 300 MiB free. The service responds normally, no swapping occurs and there are no `OutOfMemoryError` messages or kernel OOM records.

Diagnosis. The three values are compared and the use of the cache is established.

Distinguish a full cache from a lack of memory (values from the build machine) TERMINAL

```
$ free -m | sed -n 1,2p
               total        used        free      shared  buff/cache   available
Mem:            8029         580        6959          14         714        7449
$ vmstat 1 3 | tail -1 | awk '{print "si=" $7, "so=" $8, "bi=" $9}'
si=0 so=0 bi=0
$ head -1 /proc/pressure/memory
some avg10=0.00 avg60=0.00 avg300=0.18 total=3157741
```

The build machine is idle and shows the normal pattern for a healthy host: `available` close to `total`, no swap activity (`si`, `so` zero) and PSI near zero. A host in the described state shows `free` near zero and `buff/cache` near 15 GiB, but the same healthy signs: `available` large, `si` and `so` zero, PSI averages close to zero. The memory is in use as cache and is returned on demand.

Root cause. The alert measures the wrong quantity. Unused memory is not a goal; the page cache is meant to fill all free memory.

Fix. Replace the rule with `MemAvailable / MemTotal` below 10 percent, combined with PSI `some avg60` above a threshold or a nonzero swap-in rate. In Prometheus node exporter terms: `node_memory_MemAvailable_bytes`, not `node_memory_MemFree_bytes`.

Prevention. Teach the three-column rule to the on-call team: low `free` with high `available` is normal; low `available` is the problem. Do not run `echo 3 > /proc/sys/vm/drop_caches` to "free memory": it discards useful cache, forces the next requests to read from disk and can create the latency problem that it was meant to prevent.

Incident: latency after a nightly batch evicts the cache

Scenario. A nightly job scans a 200 GiB data file with ordinary reads on the same 32 GiB host that serves a database-backed API whose hot data is 20 GiB. At 02:00 the job starts; at 02:30 the API latency doubles and remains high for an hour after the job ends.

Symptoms. The API's read latency rises; `pgmajfault` and disk read throughput rise during the job; disk reads remain high for some time after the job; no process memory changes.

Diagnosis. The cache content is compared before and after the job.

Fields to sample before, during and after the scan (values of the idle build machine) TERMINAL

```
$ grep -E "^(Cached|Active\(file\)|Inactive\(file\)):" /proc/meminfo
Cached:           706036 kB
Active(file):     157504 kB
Inactive(file):   547264 kB
$ grep pgmajfault /proc/vmstat
pgmajfault 446084
```

`pgmajfault` is cumulative: subtracting two samples taken a minute apart gives the major-fault rate. During the scan the `Inactive(file)` list receives the scanned pages and the hot pages of the database are pushed out; afterwards the API

reads its own data from disk until the cache has been rebuilt, which appears as a sustained major-fault rate and disk reads without any change in process memory.

Root cause. A one-pass scan pollutes the shared page cache. The active/inactive lists of Linux protect pages that were referenced twice, but the database's pages are touched less often than the scan streams new ones, and under a tight memory budget they are evicted.

Fix. Immediate: stop the job or lower its priority. Permanent: run the job with `posix_fadvise(POSIX_FADV_DONTNEED)` after each chunk, with `O_DIRECT`, with `nocache` or in a memory cgroup of its own (`memory.limit_in_bytes` at 2 GiB, so that its cache is confined), and with `ionice -c3`.

Prevention. Alert on the major fault rate and on the cache hit ratio of the database; schedule scans on replicas; keep the hot data set smaller than the memory that the cache may use.

3.4.5 Questions to expect

Questions and model answers

1. **What is the page cache and why does Linux use all free memory for it?** A cache of file data in RAM, shared by all processes. Free memory has no value; filling it with recently used file data avoids disk reads, and clean pages can be dropped instantly when memory is needed.
2. **What is the difference between `free` and `available` in `free`?** `free` counts pages holding nothing. `available` estimates the memory that can be given to applications without swapping, including reclaimable cache. Capacity decisions use `available`.
3. **What is the difference between buffers and cache?** Cache holds file contents; buffers hold block-device metadata (superblocks, inode tables, directory blocks). Both are page-cache pages today; `free` reports their sum.
4. **What happens when a process calls `write` and returns? Is the data on disk?** No. The data is in a dirty cache page. The flusher writes it within about 30 seconds, or at once on `fsync`. A crash before that loses it.
5. **How does `mmap` of a file differ from `read`?** `mmap` maps the cache pages into the address space: no copy, no system call per access, page faults load the data. `read` copies the data into a user buffer on each call.
6. **When is `drop_caches` appropriate?** Almost never in production; it is a benchmarking and debugging tool for creating a cold-cache state. It removes useful pages and does not fix memory leaks.
7. **Does the page cache count against a container's memory limit?** Yes, charged to the cgroup that loaded the pages. The kernel reclaims clean cache before the limit is enforced by an OOM kill, so high usage from cache is normal; the working-set metric excludes the inactive part.
8. **Why can a mapped-file Java application show unexplained latency?** A page fault that needs I/O blocks the thread for the duration of the read, and the JVM cannot reach a safepoint while the thread is inside the fault.
9. **What are the effects of `vm.dirty_ratio` set too high or too low?** Too high allows large bursts of dirty data and long stalls when it is finally flushed and more data lost on a crash; too low throttles writers early and lowers write throughput.

KEY POINTS

The page cache stores file data in otherwise unused RAM and is shared by all processes.

- `read` copies out of the cache, `write` copies in and marks the page dirty, `mmap` maps the cache pages directly.
- Read `available`, not `free`: in the measurement `available` stayed at 7175 MiB while the cache gained and lost 128 MiB.
- Dirty pages are written back after 30 seconds, at the background ratio or by `fsync`; until then a crash loses them.
- The cache is charged to the memory cgroup; the measured cgroup stayed at its 64 MiB limit with 62.9 MB of cache and no OOM kill.
- Do not use `drop_caches` as a remedy; scans that pollute the cache are controlled with `fadvise`, `O_DIRECT`, ionice and cgroups.

3.5 Memory Limits, Overcommit and the OOM Killer

A computer has a finite amount of memory, but the kernel promises more than it has: processes reserve address space that is never used, `fork` duplicates address spaces lazily, and sparse data structures touch only part of what they allocate. If every promise had to be backed by memory at the moment it was made, most programs could not run. Linux therefore **overcommits**. When the promises are called in, and no memory can be reclaimed, the kernel has one remaining option: kill a process. This chapter describes the policies that control overcommit, the selection logic of the OOM killer, the report that it writes, the difference between a global and a cgroup OOM, and the way these mechanisms appear in Java services on Kubernetes.

3.5.1 Concept and mechanism

Overcommit modes

The sysctl `vm.overcommit_memory` selects how the kernel answers requests for address space (`malloc`, `mmap`, `brk`, `fork`).

Mode	Behaviour	Consequence
`0` (default)	Heuristic: refuse a single request that is obviously larger than the total RAM plus swap	Allocations normally succeed; the shortage shows up later as an OOM kill
`1`	Always grant	Maximum flexibility; `fork` of large processes never fails; OOM kills are the only limit
`2`	Strict: refuse when `Committed_AS` would exceed `CommitLimit` (swap + `vm.overcommit_ratio` percent of RAM, default 50)	`malloc` returns `NULL` instead of an OOM kill; large heaps may fail to start

Overcommit policies.

`Committed_AS` in `/proc/meminfo` is the sum of the private writable address space that could be touched; `CommitLimit` is the ceiling used in mode 2. In mode 0 only the check for a single huge request applies. The program `overcommit.c` requests blocks of 1, 4, 16 and 64 GiB and never writes to them.

overcommit.c C

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

static void show(const char *tag) {
    FILE *f = fopen("/proc/meminfo", "r"); char l[128];
    while (fgets(l, sizeof l, f))
        if (!strncmp(l, "Committed_AS", 12)) printf("%-22s %s", tag, l);
    fclose(f);
}

int main(void) {
    size_t gib[] = {1, 4, 16, 64};
    show("start");
    for (int i = 0; i < 4; i++) {
        void *p = malloc(gib[i] << 30);                 /* reserved, never touched */
        printf("malloc(%2zu GiB) -> %s\n", gib[i], p ? "success" : "NULL");
        if (p) show("after malloc");
        /* deliberately not freed and not written: no page is ever allocated */
    }
    return 0;
}
```

Run overcommit.c on the 8 GiB build machine (mode 0) TERMINAL

```
$ gcc -o overcommit overcommit.c && ./overcommit
start                 Committed_AS:    1118592 kB
malloc( 1 GiB) -> success
after malloc          Committed_AS:    2167172 kB
malloc( 4 GiB) -> success
after malloc          Committed_AS:    6361480 kB
malloc(16 GiB) -> NULL
malloc(64 GiB) -> NULL
$ grep -E "CommitLimit|MemTotal" /proc/meminfo; cat /proc/sys/vm/overcommit_memory
MemTotal:        8222320 kB
CommitLimit:     4111160 kB
0
```

What to notice: a 4 GiB request succeeded although only 8 GiB of RAM exist and 1 GiB was already committed, because heuristic mode refuses only requests that alone exceed the RAM plus swap; the 16 GiB request failed. `Committed_AS` grew by the size of each granted block (1 GiB, then 4 GiB) while the resident memory of the process did not change at all, because no page was touched. The printed `CommitLimit` of 4,111,160 kB (50 percent of `MemTotal`, no swap) is not enforced in mode 0; it would be the ceiling in mode 2.

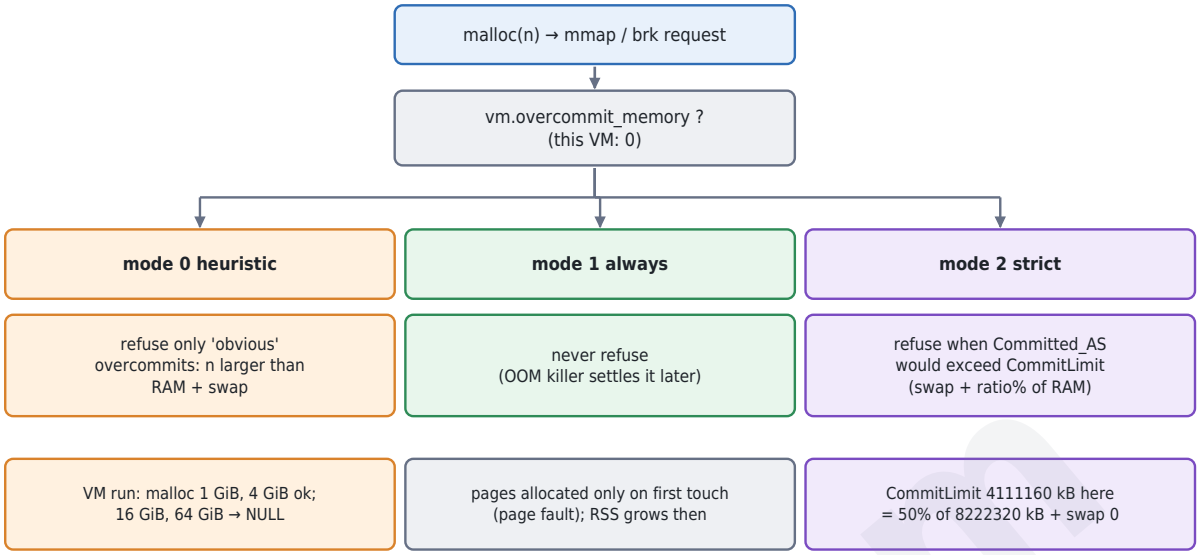


Figure 3.23. The decision made for an allocation request under the three overcommit modes, with the results of the overcommit.c run on the build machine.

When memory is really exhausted

Allocation of a physical page fails only when a process touches memory and no free page exists after reclaim. The kernel then runs the **OOM killer** (`out_of_memory()`), which chooses a victim, sends it `SIGKILL` and waits for its memory to be freed. Two kinds of OOM exist.

- **Global OOM**: the machine as a whole has no reclaimable memory. Any process may be chosen.
- **Memory cgroup OOM**: a cgroup reached its `memory.limit_in_bytes` (cgroup v1) or `memory.max` (cgroup v2) and reclaim within the group failed. Only processes of that cgroup are candidates, and the rest of the machine is not affected. This is the mechanism behind a container limit.

Selecting the victim

For each candidate task the kernel computes a **badness** value: the resident pages, the swap pages and the page-table pages of the task, scaled to a range of 0 to 1000 relative to the memory that is available to it (the machine, or the cgroup limit), plus the value `oom_score_adj` of the task (range -1000 to +1000). A value of -1000 exempts a task entirely; +1000 makes it the first choice. The task with the highest value is killed; the kernel prefers to kill a child with a nonzero badness before the parent. Kernel threads and PID 1 of the machine are never selected. The current values are visible per process:

File	Meaning
`/proc/PID/oom_score`	The badness value that the kernel would use now.
`/proc/PID/oom_score_adj`	Adjustment from -1000 to 1000, inherited by children. Lowering it below its initial value needs the capability `CAP_SYS_RESOURCE`.
`/proc/PID/oom_adj`	Legacy interface; superseded by `oom_score_adj`.

Per-process OOM files.

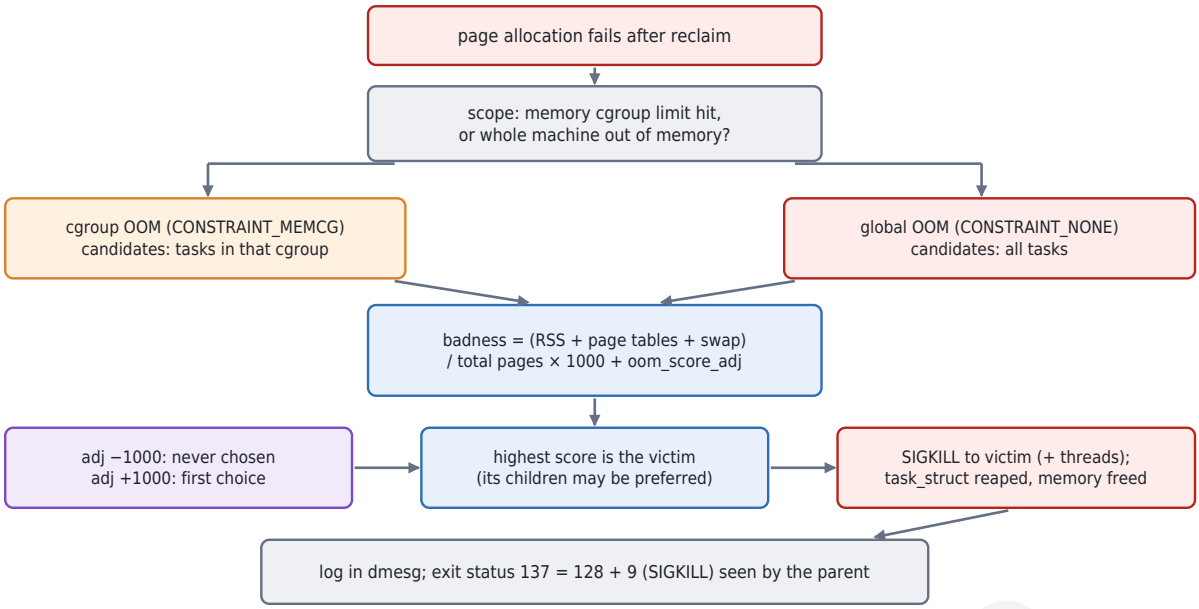


Figure 3.24. The path from a failed page allocation to a SIGKILL: scope (cgroup or global), badness ranking, effect of oom_score_adj, and the exit status 137 seen by the parent.

Control through cgroups

Cgroup v1 (used on the build machine: `/sys/fs/cgroup/memory`) provides `memory.limit_in_bytes`, `memory.usage_in_bytes`, `memory.failcnt` (number of times the limit was hit), `memory.oom_control` (with the counter `oom_kill`) and `memory.stat`. Cgroup v2 provides `memory.max` (hard limit), `memory.high` (throttling threshold), `memory.min` and `memory.low` (protection), `memory.swap.max`, `memory.events` (counters `high`, `max`, `oom`, `oom_kill`) and `memory.oom.group` (kill the whole group together). The limit applies to anonymous memory, page cache, kernel memory such as stacks and page tables, and tmpfs.

Kubernetes translates the container `resources.limits.memory` into the cgroup limit and `resources.requests.memory` into scheduling and `oom_score_adj` values.

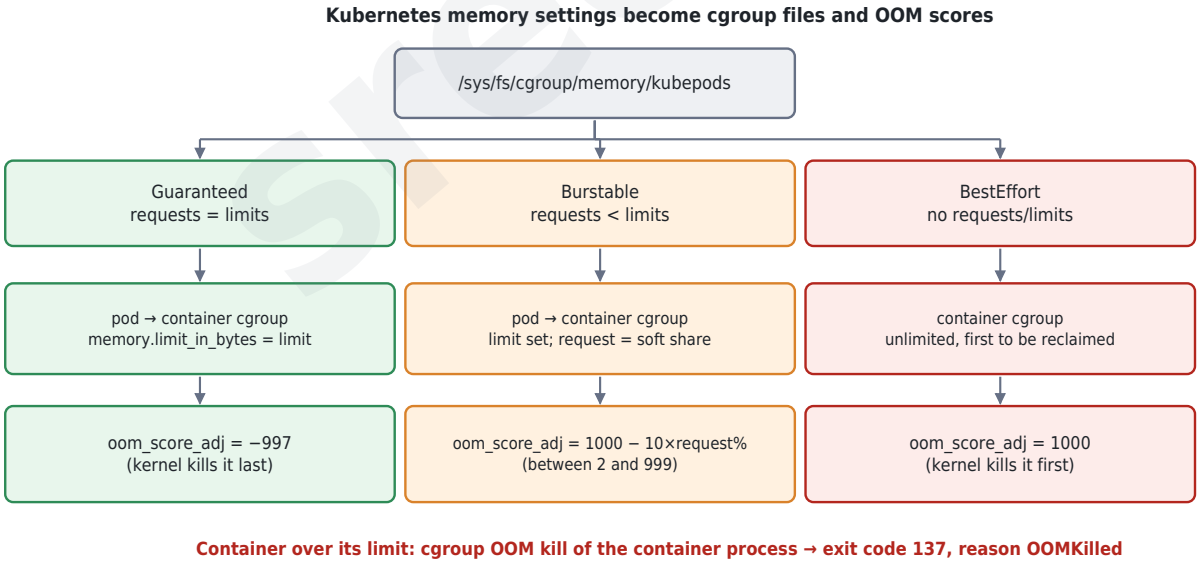


Figure 3.25. The Kubernetes quality-of-service classes map to cgroups and OOM score adjustments: Guaranteed pods are killed last (-997), BestEffort pods first (1000).

3.5.2 Worked trace: a cgroup OOM kill, step by step

The program `hog.c` allocates and writes to memory in 8 MiB steps and stops after 128 MiB.

hog.c C

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

int main(void) {
    const size_t chunk = 8UL << 20;
    for (int i = 1; i <= 16; i++) {          /* hard cap: 128 MiB even if the limit fails */
        char *p = malloc(chunk);
        if (!p) { puts("malloc failed"); return 2; }
        memset(p, 1, chunk);
        printf("touched %3d MiB\n", i * 8);
        fflush(stdout);
    }
    puts("finished without being killed");
    return 0;
}
```

A memory cgroup with a limit of 48 MiB (50,331,648 bytes) was created, the shell was moved into it and the program was started under `timeout`.

Reproduce a cgroup OOM kill (real run) TERMINAL

```
$ mkdir /sys/fs/cgroup/memory/osw_o3
$ echo $((48*1024*1024)) > /sys/fs/cgroup/memory/osw_o3/memory.limit_in_bytes
$ ( echo $BASHPID > /sys/fs/cgroup/memory/osw_o3/cgroup.procs; timeout 20 ./hog; \
    echo "exit status: $?" )
touched   8 MiB
touched  16 MiB
touched  24 MiB
touched  32 MiB
touched  40 MiB
/bin/bash: line 1:  2868 Killed                  timeout 20 ./hog
exit status: 137
$ G=/sys/fs/cgroup/memory/osw_o3; cat $G/memory.failcnt $G/memory.max_usage_in_bytes
28
50331648
$ grep oom_kill $G/memory.oom_control
oom_kill 1
```

The trace of events is as follows.

1. The program touches 8 MiB per step; memory is charged to the cgroup. After the step that reaches 40 MiB (plus about 1.5 MiB of code and libraries) the usage is close to the limit.
2. The next step touches new pages: the cgroup is at its limit of 48 MiB. The kernel first tries to reclaim from the group. There is no swap, and the process memory is anonymous, so nothing can be reclaimed; `failcnt` increments (28 times in total, for repeated attempts).
3. The kernel invokes the OOM killer for the cgroup. The only candidates are `bash` (581 pages), `timeout` (431 pages) and `hog` (12,506 pages); `hog` has by far the highest badness.
4. The kernel sends `SIGKILL` to PID 2869. The program does not print "touched 48 MiB" because it is killed during the `memset` that would have used the 48 MiB step.
5. `timeout` observes that its child died from signal 9 and exits with status 137, that is 128 + 9, which the shell reports.

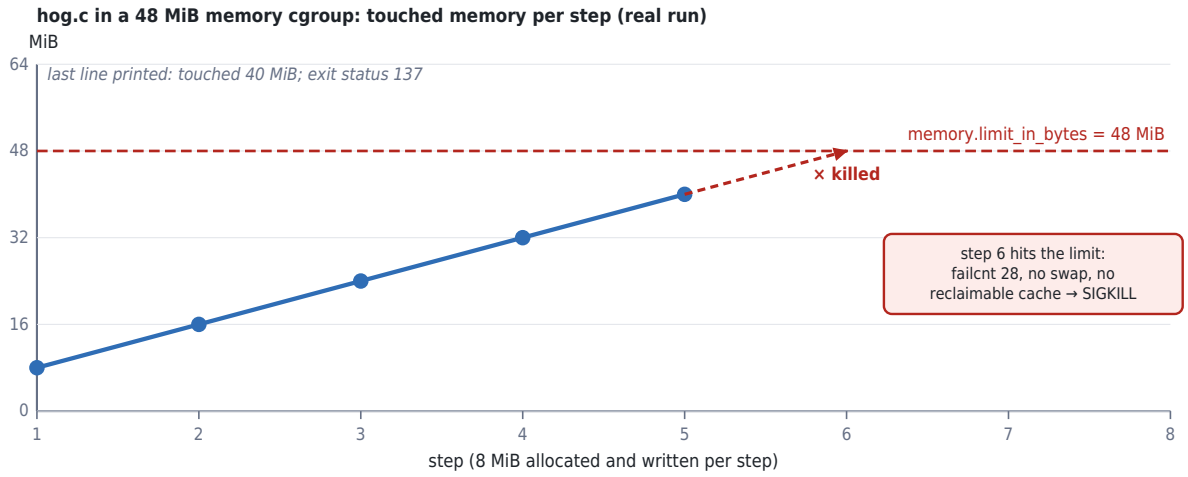


Figure 3.26. Memory touched per step of hog.c against the 48 MiB limit: five steps complete, the sixth reaches the limit and the process is killed.

Anatomy of the kernel report

The kernel writes the report into the ring buffer; `dmesg` shows it. An excerpt of the real report follows (timestamps and lines omitted; the stack dump of 30 lines is cut).

Excerpt of the dmesg OOM report (real output, trimmed) TERMINAL

```
$ dmesg | grep -E "oom|Memory cgroup|memory:|^\[ *[0-9.]+\] \[ *2869\]"
[  422.559257] hog invoked oom-killer: gfp_mask=0xcc0(GFP_KERNEL), order=0, oom_score_adj=0
[  422.561100] memory: usage 49152kB, limit 49152kB, failcnt 28
[  422.599046] memory+swap: usage 49152kB, limit 9007199254740988kB, failcnt 0
[  422.601764] Memory cgroup stats for /osw_o3:
[  422.645992] [   2866]     0  2866     1122      581      115      466 ...  bash
[  422.649033] [   2868]     0  2868      684      431        0      431 ...  timeout
[  422.651747] [   2869]     0  2869    12972    12506    12128      378 ...  hog
[  422.654255] oom-kill:constraint=CONSTRAINT_MEMCG,...,oom_memcg=/osw_o3,task=hog,pid=2869
[  422.657942] Memory cgroup out of memory: Killed process 2869 (hog) total-vm:51888kB
```

Line	Information
`hog invoked oom-killer: gfp_mask=...`	The process whose allocation triggered the killer and the allocation flags. The victim need not be this process.
`memory: usage 49152kB, limit 49152kB, failcnt 28`	Cgroup counters: usage equal to the limit; 28 failed charge attempts.
`memory+swap ... limit 9007199254740988kB`	No swap limit: the value is the maximum.
`Memory cgroup stats for /osw_o3`	Breakdown by type: `cache`, `rss`, `shmem`, `mapped_file`. Here the cache is 0 and `rss` is 49,942,528 bytes: the usage is all anonymous memory.
Task table (`pid uid tgid total_vm rss pgtables_bytes swapents oom_score_adj name`)	Memory in pages (4 KiB) per candidate. The 12,506 resident pages of `hog` equal 48.9 MiB.
`oom-kill:constraint=CONSTRAINT_MEMCG`	Scope. `CONSTRAINT_NONE` means a global OOM.
`Memory cgroup out of memory: Killed process 2869 (hog)`	The verdict with `anon-rss`, `file-rss` and `shmem-rss` of the victim.

Reading the report from top to bottom.

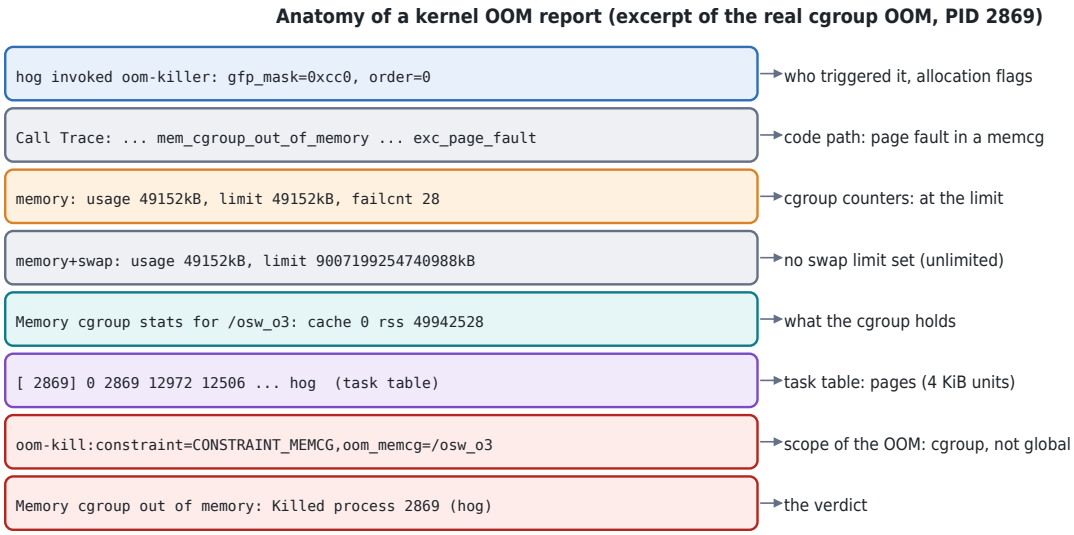


Figure 3.27. The lines of the OOM report that identify the trigger, the cgroup counters at the limit, the candidate table, the constraint type and the victim.

Observing oom_score

Two processes holding 10 MiB and 100 MiB were started and their scores read. A score adjustment of +500 was then written to the first one.

Scores of two processes and effect of oom_score_adj (real run) TERMINAL

```
$ ./hold 100 8 & ./hold 10 8 & sleep 1
$ for p in 1 $SMALL $BIG; do echo "pid $p $(cat /proc/$p/comm) \
    rss=$(awk '/VmRSS/{print $2" kB"}' /proc/$p/status) \
    score=$(cat /proc/$p/oom_score) adj=$(cat /proc/$p/oom_score_adj)"; done
pid 1 process_api rss=5952 kB score=0 adj=0
pid 3481 hold rss=11932 kB score=667 adj=0
pid 3480 hold rss=104092 kB score=674 adj=0
$ echo 500 > /proc/$SMALL/oom_score_adj
$ cat /proc/$SMALL/oom_score
1000
$ echo -500 > /proc/$BIG/oom_score_adj
bash: echo: write error: Permission denied
```

The larger process has the higher score, and the adjustment of +500 raises the other to the maximum 1000 (the sum is capped). Writing -500 was refused because the session lacks `CAP_SYS_RESOURCE` (the effective capability mask `000001fffeffffff` has bit 24 cleared). On this virtual machine the absolute scores of ordinary processes are high (667 and 674 for 12 MB and 104 MB) and not proportional to the resident size; the documented formula would produce values near 1 and 13 for these sizes. The numbers are therefore used here only for their ordering and for the effect of the adjustment, and the base value should not be extrapolated to other systems.

3.5.3 Hands-on: a JVM inside a limit

Heap size ergonomics in a container

The JVM reads the cgroup limit and derives the default maximum heap from it. The default is 25 percent of the memory for large limits, but for small ones a larger fraction (`MinRAMPercentage`, 50 percent) is used. The flags were printed inside cgroups of 128 MiB and 256 MiB.

Default and requested maximum heap inside cgroups (real output) TERMINAL

```
$ ( echo $BASHPID > $G/cgroup.procs; java -XX:+PrintFlagsFinal -version | grep -E " MaxHeapSize" )
128 MiB limit, default flags:                MaxHeapSize = 67108864   (64 MiB)
128 MiB limit, -XX:MaxRAMPercentage=90:      MaxHeapSize = 67108864   (64 MiB)
256 MiB limit, default flags:                MaxHeapSize = 132120576  (126 MiB)
256 MiB limit, -XX:MaxRAMPercentage=90:      MaxHeapSize = 243269632  (232 MiB)
```

In both containers the JVM selected the serial collector (`UseSerialGC` true), because fewer than two available processors or less than about 1.8 GiB of memory classify the machine as not server-class; with the container limit of this experiment only the memory criterion applies. In the 128 MiB container the setting `-XX:MaxRAMPercentage=90` had no effect and the heap remained at 64 MiB, whereas in the 256 MiB container it was applied (232 MiB). The ergonomics for very small memory sizes differ from those for ordinary sizes; the effective value must always be verified with `-XX:+PrintFlagsFinal` or `jcmd PID VM.flags`.

A Java OOM and a kernel OOM compared

The program `NativeHog.java` allocates 16 MiB direct buffers in a loop and writes to each page; the heap is limited to 32 MiB. The cgroup limit is 128 MiB.

NativeHog.java JAVA

```java
import java.nio.ByteBuffer;
import java.util.ArrayList;
import java.util.List;

public class NativeHog {
    public static void main(String[] args) throws Exception {
        List<ByteBuffer> keep = new ArrayList<>();
        Runtime rt = Runtime.getRuntime();
        for (int i = 1; i <= 12; i++) {                       // at most 192 MiB of direct memory
            ByteBuffer b = ByteBuffer.allocateDirect(16 << 20);
            for (int p = 0; p < b.capacity(); p += 4096) b.put(p, (byte) 1);
            keep.add(b);
            long heapUsedMb = (rt.totalMemory() - rt.freeMemory()) >> 20;
            System.out.printf("direct=%3d MiB  heapUsed=%2d MiB  heapMax=%d MiB%n",
                    i * 16, heapUsedMb, rt.maxMemory() >> 20);
            System.out.flush();
        }
        System.out.println("survived");
    }
}
```

Run NativeHog in a 128 MiB cgroup (real run) TERMINAL

```
$ ( echo $BASHPID > $G/cgroup.procs; \
    java -Xmx32m -XX:MaxDirectMemorySize=1g -cp . NativeHog; echo "exit status: $?" )
direct= 16 MiB  heapUsed= 1 MiB  heapMax=30 MiB
direct= 32 MiB  heapUsed= 1 MiB  heapMax=30 MiB
direct= 48 MiB  heapUsed= 1 MiB  heapMax=30 MiB
direct= 64 MiB  heapUsed= 1 MiB  heapMax=30 MiB
direct= 80 MiB  heapUsed= 1 MiB  heapMax=30 MiB
direct= 96 MiB  heapUsed= 1 MiB  heapMax=30 MiB
/bin/bash: line 47:  3594 Killed                  timeout 40 java -Xmx32m ... -cp . NativeHog
exit status: 137
$ dmesg | grep "Memory cgroup out of memory" | tail -1
[  453.135051] Memory cgroup out of memory: Killed process 3595 (java)
  total-vm:2493016kB, anon-rss:129792kB, file-rss:21964kB
```

The heap stayed at 1 MiB of 30 MiB. The process was killed after about 96 to 112 MiB of direct buffers because the cgroup was charged for the anonymous memory of the buffers (129,792 kB anonymous resident at the kill), the mapped JDK files (21,964 kB) and the other JVM structures. No `OutOfMemoryError` was thrown and no Java stack trace or heap dump exists: the process received `SIGKILL`. The garbage collector never had a reason to run, because the Java heap was empty; it cannot see native memory.

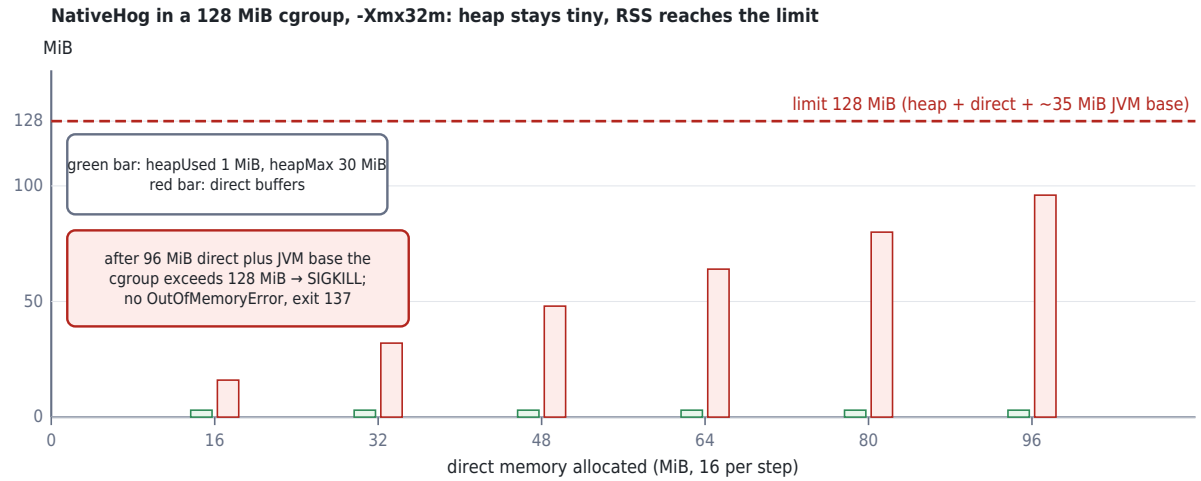


Figure 3.28. Heap use (green) stays below 2 MiB while direct buffers (red) approach the 128 MiB cgroup limit; the kernel kills the JVM without any Java-level error.

A Java-level OOM, in contrast, is an exception inside a live JVM. Reproduced with a 32 MiB heap:

Java heap OutOfMemoryError (real run, trimmed) TERMINAL

```
$ java -Xmx32m -cp . OomKinds heap; echo "exit=$?"
caught: java.lang.OutOfMemoryError: Java heap space
objects/threads created: 15
exit=0
```

The program caught the error, printed it and exited normally. The status is not 137, a stack trace is available and, with `-XX:+HeapDumpOnOutOfMemoryError`, a heap dump is written.

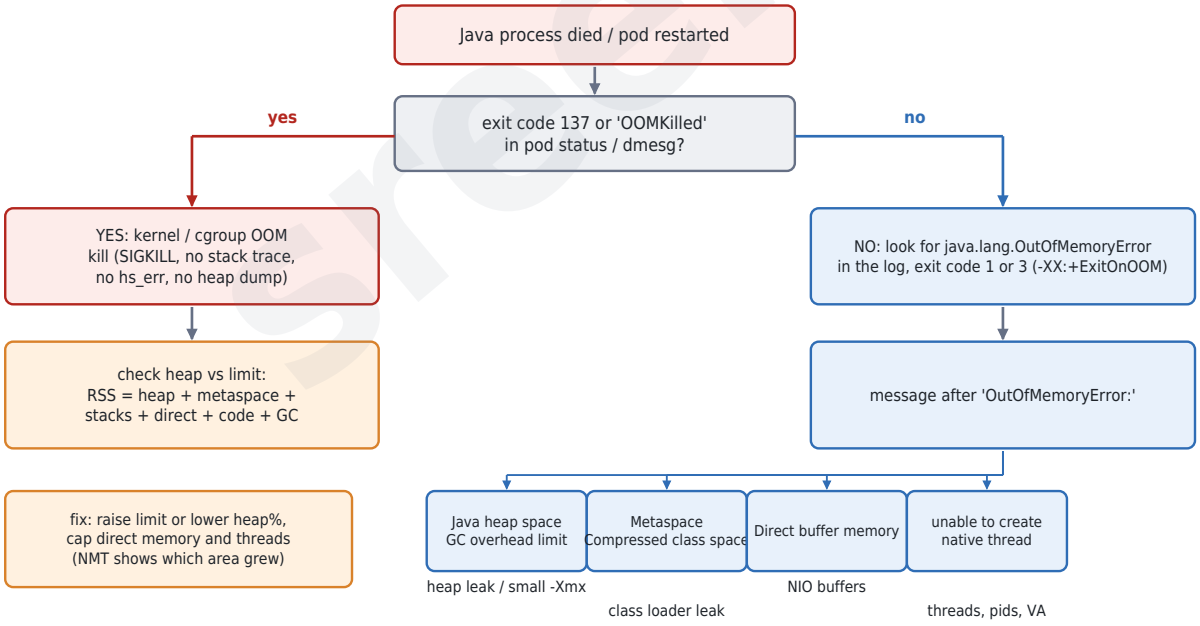


Figure 3.29. The decision path that separates a kernel or cgroup OOM kill (exit status 137, no Java error) from a Java-level OutOfMemoryError, with the message text that identifies the exhausted area.

3.5.4 Real-world incidents

Incident: pod OOMKilled while the heap is healthy

Scenario. A Spring Boot service runs in Kubernetes with `resources.limits.memory: 512Mi` and `-Xmx384m`. The service uses Netty-based HTTP clients with direct buffers. After a traffic increase the pods restart every few hours.

Symptoms. `kubectl get pods` shows `RESTARTS` increasing. `kubectl describe pod` shows `Last State: Terminated, Reason: OOMKilled, Exit Code: 137`. The application log ends abruptly without an exception. Heap dashboards show a utilisation of 40 to 60 percent, with no sign of exhaustion.

Diagnosis. The cause is established in three steps: confirm the kernel kill, compare heap and total usage, and find the growing area. The reproduction above (NativeHog) is a model of the case.

1. `kubectl describe pod POD | grep -A5 "Last State"` shows (typical output) `OOMKilled` and exit code 137. On the node, `dmesg | grep -i "killed process"` shows the matching report with `anon-rss` close to the limit.
2. The memory of the process is compared with the heap limit: `ps -o rss= -p PID` (or `container_memory_working_set_bytes`) against `jcmd PID GC.heap_info`. In the reproduction: heap 1 MiB, resident size at the limit.
3. Native memory tracking identifies the area. The JVM is restarted with `-XX:NativeMemoryTracking=summary` and `jcmd PID VM.native_memory summary` is sampled over time; `jcmd PID VM.native_memory baseline` followed by `summary.diff` shows which category grows.

Root cause. The container limit (512 MiB) is smaller than heap (384 MiB) plus native memory: metaspace, thread stacks, code cache, GC structures (about 40 MiB of the 210 MiB committed in the NMT sample of the next chapter) and direct buffers. The sum exceeded the limit, and the kernel killed the process.

Fix. Immediate: raise the limit (for example to 1 GiB) or reduce `-Xmx` so that the heap is at most 50 to 70 percent of the limit. Permanent: set `-XX:MaxDirectMemorySize` so that the JVM throws `OutOfMemoryError: Direct buffer memory` (a recoverable, loggable error) before the kernel intervenes; use `-XX:MaxRAMPercentage=60` instead of a fixed `-Xmx` so that heap follows the limit; enable `-XX:+ExitOnOutOfMemoryError` and `-XX:+HeapDumpOnOutOfMemoryError` with a writable dump path.

Prevention. Alert on `container_memory_working_set_bytes / limit` above 85 percent and on the restart counter. Review the budget: limit = heap + metaspace + code cache + threads × stack + direct memory + 10 to 20 percent headroom.

Incident: Java heap OOM or kernel OOM

Scenario. A service restarts and the team disagrees: one engineer reads the log as a heap leak, another expects a container limit problem. Both symptoms look the same on the dashboard: the process restarted.

Symptoms. Pod restart count rises. Heap usage dashboards show high values before the restart.

Diagnosis. The two cases differ in evidence, as the triage diagram shows. The reproduction above produced both.

Evidence for each case (real output of the reproductions) TERMINAL

```
$ java -Xmx32m -cp . OomKinds heap; echo "exit=$?"
caught: java.lang.OutOfMemoryError: Java heap space
objects/threads created: 15
exit=0
$ ( echo $BASHPID > $G/cgroup.procs; java -Xmx32m -XX:MaxDirectMemorySize=1g -cp . NativeHog \
    > /dev/null 2>&1; echo "exit=$?" )
exit=137
```

A Java-level OOM leaves an `OutOfMemoryError` in the log, an exit status of 1 (uncaught) or the chosen status, and optionally a heap dump. A kernel-level kill leaves status 137, `OOMKilled` in the pod status, a report in `dmesg` (`Memory cgroup out of memory`) and no Java exception.

Root cause. Different for each case: a Java OOM means that live objects fill the heap (leak or an undersized `-Xmx`); a kernel OOM means that the process footprint exceeds the container limit.

Fix. Java OOM: heap dump analysis (see the incident on the unbounded cache in the JVM chapters), then fix the leak or enlarge `-Xmx` within the limit. Kernel OOM: reduce the footprint or raise the limit as in the previous incident.

Prevention. Add `-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/dumps` with a volume, log the exit reason in the supervision tooling, and track both heap usage and working set per pod.

Incident: heap ergonomics ignore the percentage in a tiny container

Scenario. A sidecar service runs in a 128 MiB container. The engineer sets `-XX:MaxRAMPercentage=90` to give the heap more room and sees `OutOfMemoryError: Java heap space` at about 60 MiB of live data.

Symptoms. The error occurs although 90 percent of 128 MiB would be 115 MiB. The heap dashboard shows a maximum of 64 MiB.

Diagnosis. The effective flag value is printed inside the container (see the real output above): `MaxHeapSize = 67108864` for the 128 MiB limit with and without the percentage flag. A direct reproduction with `HeapFill` (a program that retains 1 MiB arrays) in a 256 MiB container with `-XX:MaxRAMPercentage=50` gave `OutOfMemoryError after 122 MiB: Java heap space`.

Root cause. The observed behaviour is consistent with a minimum-fraction rule of the HotSpot ergonomics for very small memory sizes (`MinRAMPercentage`, 50 percent): the percentage flag does not take effect as it does for ordinary sizes. The engineer assumed that heap sizing by percentage works uniformly.

Fix. Set an explicit `-Xmx` for tiny containers and verify the result with `-XX:+PrintFlagsFinal`. Leave room for metaspace, code cache and stacks: in a 128 MiB container a heap of 64 MiB is already a conservative choice.

Prevention. Include a start-up check in CI that prints `MaxHeapSize` inside the target container size and fails the build if it deviates from the intended value.

3.5.5 Questions to expect

Questions and model answers

1. **What is memory overcommit?** The kernel grants address space beyond the physical memory plus swap, because most of it is never touched. The shortage appears when pages are touched and no free page exists, and the OOM killer resolves it.
2. **Explain `vm.overcommit_memory` 0, 1 and 2.** 0: heuristic refusal of obviously excessive single requests; 1: never refuse; 2: refuse beyond `CommitLimit` (swap plus ratio percent of RAM). Mode 2 turns OOM kills into allocation failures.
3. **How does the OOM killer choose its victim?** It computes a badness from resident, swap and page-table pages normalised to 0 to 1000, adds `oom_score_adj`, and kills the highest. Tasks with -1000 are exempt.
4. **How do you protect an important process?** Set `oom_score_adj` to a negative value (needs `CAP_SYS_RESOURCE`), typically via the service manager (`OOMScoreAdjust=` in systemd), and give it a guaranteed QoS class in Kubernetes. Do not set -1000 for ordinary services: a leak would then kill other processes.
5. **What does exit code 137 mean?** 128 + 9: the process was terminated by `SIGKILL`. In containers it is usually the OOM killer, but `docker stop` after the grace period and `kill -9` also produce it, so check `OOMKilled` and `dmesg`.
6. **What is the difference between a global OOM and a cgroup OOM?** A global OOM is triggered by exhaustion of the whole machine and can kill any process; a cgroup OOM is triggered by a group limit and kills only inside the group.
7. **Why can a JVM be OOM-killed although its heap is far below `-Xmx`?** The limit applies to the whole process memory: native memory, stacks, code cache, metaspace, direct buffers and mapped files count. The heap is only one component.
8. **How would you make a Java service fail with a Java error instead of being killed?** Limit the native areas that can be limited (`-XX:MaxDirectMemorySize`, `-XX:MaxMetaspaceSize`, thread counts), keep the heap well below the container limit and leave headroom.
9. **How do Kubernetes QoS classes relate to the OOM killer?** The kubelet sets `oom_score_adj` per class: Guaranteed -997, BestEffort 1000, Burstable between 2 and 999 depending on the memory request. Under node pressure BestEffort and large-overshoot Burstable pods die first.
10. **How do you tell that `dmesg` reports a cgroup OOM?** The line `oom-kill:constraint=CONSTRAINT_MEMCG` and the text `Memory cgroup out of memory`; a global OOM shows `CONSTRAINT_NONE` and `Out of memory: Killed process`.

Steram

KEY POINTS

Linux grants more memory than it owns; the OOM killer settles the account when touched pages exceed what exists.

- Mode 0 refuses only single requests that exceed RAM plus swap: 1 GiB and 4 GiB succeeded, 16 GiB and 64 GiB failed on an 8 GiB machine.
- The victim has the highest badness (resident, swap, page tables, normalised) plus `oom_score_adj`; the report is in `dmesg`.
- A cgroup limit produces a cgroup OOM: the measured process was killed at 48 MiB, `failcnt` 28, exit status 137.
- A JVM can be killed with a nearly empty heap, because native memory counts; no Java exception exists in that case.
- Distinguish Java OOM (exception, log, dump) from kernel OOM (137, `OOMKilled`, `dmesg`) before choosing a remedy.

3.6 The Memory of a JVM Process

A Java developer sets `-Xmx` and expects the process to use at most that much memory. The operating system does not see the heap at all: it sees a process whose resident set, virtual size and cgroup charge are the sum of many components, of which the Java heap is only the largest. The difference between the configured heap and the real footprint is the origin of most memory incidents of Java services in containers. This chapter lists the components, shows how to measure each of them with `jcmd`, `pmap` and `/proc`, reconciles a real process against its configuration, and describes how the JVM reads container limits.

3.6.1 Concept and mechanism

Components of the process footprint

Area	Contents	Controlled by
Java heap	Objects; young and old generations	`-Xms`, `-Xmx`, `-XX:MaxRAMPercentage`
Metaspace	Class metadata (native memory, outside the heap)	`-XX:MaxMetaspaceSize` (unlimited by default)
Compressed class space	Class pointers metadata, reserved as 1 GiB	`-XX:CompressedClassSpaceSize`
Thread stacks	One stack per Java and VM thread (1 MiB reserved by default on Linux x64)	`-Xss`, `-XX:ThreadStackSize`, thread count
Code cache	JIT-compiled code, interpreter stubs (240 MiB reserved)	`-XX:ReservedCodeCacheSize`
GC data structures	Card table, mark bitmaps, remembered sets, work queues	collector, heap size
Direct buffers	`ByteBuffer.allocateDirect` memory, outside the heap	`-XX:MaxDirectMemorySize` (default: the maximum heap size)
Mapped files	Class image `lib/modules`, `libjvm.so`, JAR files, `MappedByteBuffer`	files in use
Native allocations	malloc arenas for the VM, JNI libraries, compiler working memory, symbol tables	libraries, compiler threads

JVM memory areas visible to the operating system.

Only the Java heap and, indirectly, the number of objects are visible to the garbage collector. All other areas are invisible to it: a garbage collection does not free a direct buffer until the owning `ByteBuffer` object itself is collected, and nothing frees native memory that a JNI library leaks.

Four different sizes

Size	Meaning
Virtual size (`VmSize`)	All address space of all VMAs, including reservations. Not a measure of consumption.
Resident set size (`VmRSS`)	Pages currently in RAM and mapped by the process: anonymous (`RssAnon`), file-backed (`RssFile`) and shared memory (`RssShmem`).
Committed (NMT)	Memory the JVM has asked the OS to make usable. Part of it may not have been touched yet, so committed can exceed resident.
cgroup charge	Anonymous pages plus the page cache and kernel memory attributed to the cgroup. The quantity that the OOM killer compares with the limit.

Terms used for the memory of a process.

For the heap the three sizes differ strongly. With `-Xmx256m` the heap is a 256 MiB reservation (virtual); `-Xms` of 64 MiB commits 64 MiB at start; pages become resident only when the allocator touches them (unless `-XX:+AlwaysPreTouch`). After a garbage collection the JVM may or may not return committed regions to the OS, depending on the collector and flags; the resident set therefore often stays near the peak heap use.

Native Memory Tracking

The JVM can account for its own native allocations by category. NMT is enabled at start with `-XX:NativeMemoryTracking=summary` (or `detail`) and queried with `jcmd`. The command `jcmd PID VM.native_memory summary` prints reserved and committed memory per category; `baseline` followed by `summary.diff` prints the growth since the baseline, which is the standard method to find a native leak. NMT adds a small overhead and must be enabled at start; it cannot be switched on in a running JVM. It does not track memory allocated by third-party native libraries (a JNI library that calls `malloc` directly) or by the C library arenas as such.

Container awareness

Since JDK 10 (and with fixes backported to 8u191) the JVM reads the limits of the cgroup in which it runs (`-XX:+UseContainerSupport`, on by default). It uses the memory limit to size the default heap and the CPU quota to compute the number of available processors, which in turn selects the collector (serial collector below two processors or below roughly 1.8 GiB, otherwise G1) and the number of GC and compiler threads. On the build machine the JDK reports `Provider: cgroupv1` in `java -XshowSettings:system -version`. The heap-sizing flags are fractions of the container memory:

Flag	Default	Meaning
`-XX:MaxRAMPercentage`	25	Maximum heap as a percentage of the memory limit
`-XX:MinRAMPercentage`	50	Used instead of the maximum fraction for small memory sizes
`-XX:InitialRAMPercentage`	1.5625	Initial heap as a percentage
`-Xmx` / `-Xms`	unset	Absolute values; override the percentages

Heap-sizing flags (JDK 21).

On the build host (a cgroup limit of 5.84 GiB) the default maximum heap was 1,568,669,696 bytes, which is 25 percent of the limit; the machine has 8 GiB, so the JVM correctly used the cgroup limit, not the installed RAM.

A budget for the container limit

A container limit has to cover every row of the area table. A practical budget is

`limit = heap + metaspace + code cache + threads × stack + direct buffers + GC overhead + 10 to 20 percent`

In the measured process below the non-heap part of the NMT committed memory was 81,136 KiB (79 MiB) for a 126 MiB committed heap with an idle 19-thread application: a committed footprint of 1.6 times the heap. Applications with large class counts, many threads, Netty buffers or large heaps (G1 structures grow with the heap: 39,913 KiB, 31 percent of the committed heap, in the measured process) need more.

3.6.2 Worked trace: reconciling a real process

The program `Idle.java` holds 100 MiB of live arrays in the heap and a 16 MiB direct buffer, then sleeps. It was started with `-Xmx256m -Xms64m -XX:NativeMemoryTracking=summary` (G1 collector).

Idle.java JAVA

```java
import java.nio.ByteBuffer;
import java.util.Arrays;

public class Idle {
    public static void main(String[] a) throws Exception {
        byte[][] keep = new byte[20][];
        for (int i = 0; i < 20; i++) {                // 100 MiB of live data
            keep[i] = new byte[5 << 20];
            Arrays.fill(keep[i], (byte) 1);
        }
        ByteBuffer d = ByteBuffer.allocateDirect(16 << 20);   // 16 MiB outside the heap
        for (int i = 0; i < d.capacity(); i += 4096) d.put(i, (byte) 1);
```

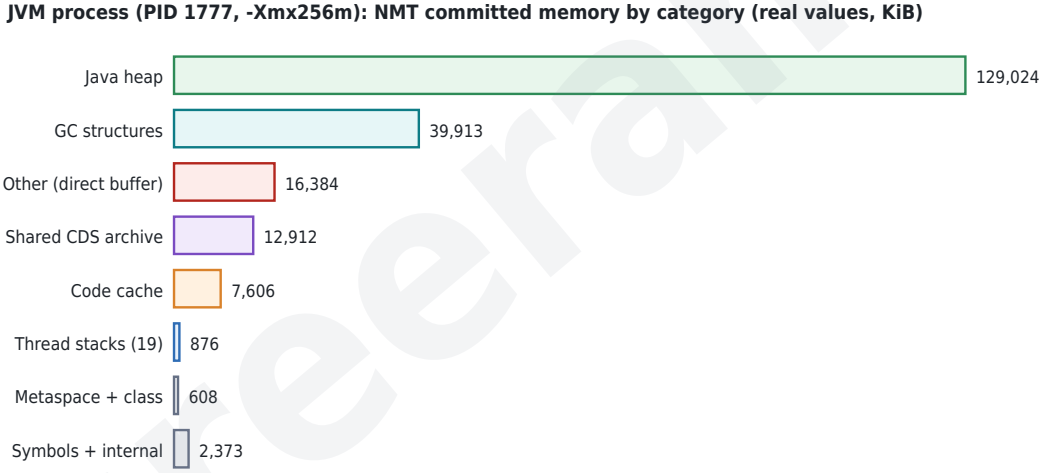
Idle.java (continued) JAVA

```
        System.out.println("pid " + ProcessHandle.current().pid());
        Thread.sleep(Long.parseLong(a[0]));          // argument in milliseconds
        System.out.println(keep.length + d.capacity());
    }
}
```

Native memory summary of the process (real output, categories with the largest values; PID 1777) TERMINAL

```
$ jcmd 1777 VM.native_memory summary
Total: reserved=1721728KB, committed=210160KB
       malloc: 23976KB #7795
       mmap:   reserved=1697752KB, committed=186184KB

-                 Java Heap (reserved=262144KB, committed=129024KB)
-                     Class (reserved=1048672KB, committed=224KB)
                            (classes #716)
-                    Thread (reserved=19512KB, committed=876KB)
                            (threads #19)
                            (stack: reserved=19456KB, committed=820KB, peak=820KB)
-                      Code (reserved=247746KB, committed=7606KB)
-                        GC (reserved=42513KB, committed=39913KB)
-                  Compiler (reserved=181KB, committed=181KB)
-                     Other (reserved=16384KB, committed=16384KB)
-                    Symbol (reserved=1142KB, committed=1142KB)
-        Shared class space (reserved=16384KB, committed=12912KB, readonly=0KB)
```



NMT total committed 210,160 KiB; heap -Xmx 262,144 KiB; VmSize 2,868,616 KiB; VmRSS 162,952 KiB
RssAnon 139,352 + RssFile 23,600; virtual size is about 17.6 times the resident size

Figure 3.30. The committed memory of the process by NMT category: the heap dominates, but GC structures (39,913 KiB), the direct buffer (16,384 KiB) and the shared class archive are each larger than the code cache or all stacks.

The resident set can then be read from the kernel and from `pmap`.

Kernel view of the same process (real output) TERMINAL

```
$ grep -E "^(VmSize|VmRSS|RssAnon|RssFile|Threads)" /proc/1777/status
VmSize:  2868616 kB
VmRSS:    162952 kB
RssAnon:          139352 kB
RssFile:           23600 kB
Threads:              19
$ pmap -x 1777 | sort -k3 -n -r | head -8
00000000f0000000  126976  102944  102944 rw---   [ anon ]
00007f88c9fff000   16388   16388   16388 rw---   [ anon ]
00007f88cb000000   12912   12764   10676 rw--- classes.jsa
00007f89406a3000   14832   11932       0 r-x-- libjvm.so
00007f8940400000    2700    2700       0 r---- libjvm.so
00007f8941a28000    1572    1492       0 r-x-- libc.so.6
00007f88a0000000    1304    1304    1304 rw---   [ anon ]
total kB         2868620  162904  139340
```

(The `pmap` columns are address, mapped size, resident size and dirty size in kB. The listing combines two separate runs of the same program; the sizes agree to within a few kilobytes, the addresses differ.)

The reconciliation follows.

Component	Resident	Source
Java heap (126,976 kB committed in the VMA)	102,944	`pmap` first line: the 100 MiB array data plus runtime objects
Direct buffer	16,388	anonymous mapping of 16 MiB
Class-data-sharing archive	12,764	file mapping, mostly clean pages
`libjvm.so` code and data	14,632	`r-x` 11,932 and `r----` 2,700
`libc` and other items listed	2,796	selected lines
Sum of listed items	149,524	of the 162,904 kB total in `pmap`; the remainder is many small mappings (stacks, malloc arenas, GC structures actually touched, jimage pages read)

Reconciliation of the resident set (kB).

Observations: the process mapped 2.74 GiB, committed 205 MiB according to NMT and holds 159 MiB resident. The heap is committed to 129,024 kB but only about 100 MiB are resident, since committed pages are made resident on first touch. The 39,913 kB of GC structures are mostly reserved and committed for the card table and bitmaps of the 256 MiB heap but only partly touched. The stacks of 19 threads have reserved 19,456 kB (about 1 MiB each) but committed only 820 kB (about 43 kB each).

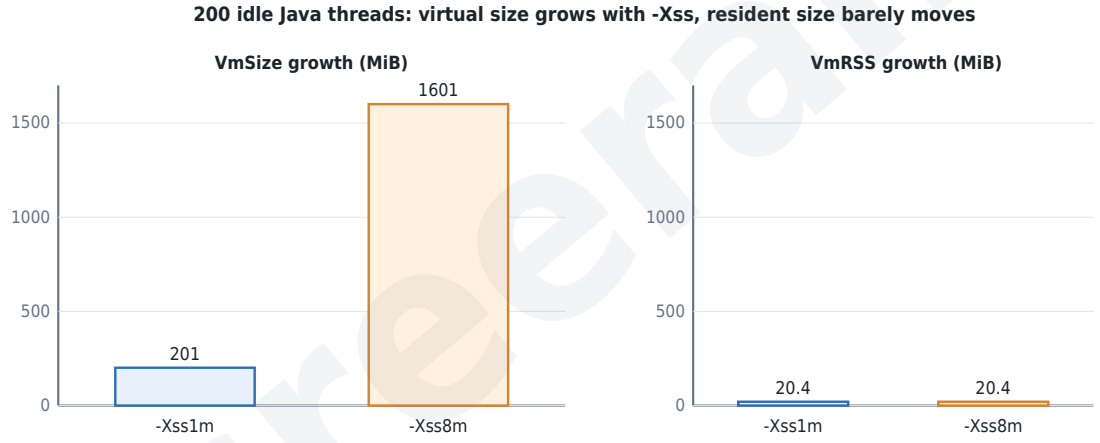


Figure 3.31. Two hundred idle threads: reserving 8 MiB instead of 1 MiB per stack raises the virtual size by 1.6 GiB instead of 0.2 GiB, while the resident size grows by the same 20 MiB in both cases.

The thread experiment behind the diagram uses `Stacks.java` (it starts 200 daemon threads that block, then prints `VmSize`, `VmRSS` and the thread count from `/proc/self/status`).

Memory effect of 200 idle Java threads (real output) TERMINAL

```
$ java -Xmx64m -Xss1m -cp . Stacks
before : VmSize: 2650876 kB  VmRSS: 40180 kB  Threads: 18
after  : VmSize: 2856704 kB  VmRSS: 61060 kB  Threads: 219
$ java -Xmx64m -Xss8m -cp . Stacks
before : VmSize: 2708220 kB  VmRSS: 40032 kB  Threads: 18
after  : VmSize: 4347648 kB  VmRSS: 60956 kB  Threads: 219
```

With 1 MiB stacks the virtual size grew by 200.9 MiB; with 8 MiB stacks by 1,601 MiB. The resident size grew by about 20 MiB in both cases, roughly 100 kB per thread: only the touched pages of each stack (the frames, the guard area and the kernel-side bookkeeping) are resident. The virtual reservation matters when address space or an `RLIMIT_AS` is restricted; the resident part matters for the cgroup charge. In a service with 2,000 threads the resident cost of stacks is about 200 MiB, and a deep recursion that touches a whole stack adds up to `-Xss` per thread.

Heap dumps and histograms

A heap dump is taken with `jcmd PID GC.heap_dump /path/file.hprof` (or `jmap -dump:live,format=b,file=...`). On the process above the dump of the 100 MiB live heap produced a 108,559,423 byte file in 0.055 seconds. The dump stops the application (a safepoint) for its duration and writes about as much as the live heap; the target volume needs that free space, and a container that writes the dump into its own limit-charged page cache can be killed while dumping. `jcmd PID GC.class_histogram` prints live instances and bytes per class after a full collection and is much cheaper than a dump.

3.6.3 Hands-on: three kinds of exhaustion

The program `OomKinds.java` provokes four different `OutOfMemoryError` situations selected by an argument.

OomKinds.java JAVA

```java
import java.nio.ByteBuffer;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.CountDownLatch;

public class OomKinds {
    static class Loader extends ClassLoader {
        final byte[] bytes;
        Loader(byte[] bytes) { this.bytes = bytes; }
        Class<?> make() { return defineClass("Dummy", bytes, 0, bytes.length); }
    }

    public static void main(String[] args) throws Exception {
        List<Object> keep = new ArrayList<>();
        try {
            switch (args[0]) {
                case "heap" -> { while (true) keep.add(new byte[1 << 20]); }
                case "direct" -> { while (true) keep.add(ByteBuffer.allocateDirect(1 << 20)); }
                case "meta" -> {
                    byte[] bytes = Files.readAllBytes(Path.of("Dummy.class"));
                    while (true) keep.add(new Loader(bytes).make());   // one class per loader
                }
                case "threads" -> {
                    CountDownLatch never = new CountDownLatch(1);
                    while (true) {
                        Thread t = new Thread(() -> {
                            try { never.await(); } catch (InterruptedException e) { }
                        });
                        t.setDaemon(true);
                        t.start();
                        keep.add(t);
                    }
                }
                default -> throw new IllegalArgumentException(args[0]);
            }
        } catch (OutOfMemoryError e) {
            int n = keep.size();
            keep.clear();
            System.out.println("caught: " + e);
            System.out.println("objects/threads created: " + n);
        }
    }
}
```

(The class `Dummy` is a trivial class compiled beside it; the `meta` mode defines it again in a new class loader for every iteration.)

Java-level exhaustion of three areas (real output) TERMINAL

```
$ java -Xmx32m -cp . OomKinds heap
caught: java.lang.OutOfMemoryError: Java heap space
objects/threads created: 15
$ java -Xmx64m -XX:MaxMetaspaceSize=16m -cp . OomKinds meta
caught: java.lang.OutOfMemoryError: Metaspace
objects/threads created: 2638
$ java -Xmx64m -XX:MaxDirectMemorySize=16m -cp . OomKinds direct
caught: java.lang.OutOfMemoryError: Cannot reserve 1048576 bytes of direct buffer memory
  (allocated: 16777216, limit: 16777216)
objects/threads created: 16
```

Each message names the exhausted area: `Java heap space`, `Metaspace` and `direct buffer memory`. The heap run retained 15 arrays of 1 MiB before the 32 MiB heap (of which about 30 MiB are usable) was full. The metaspace run loaded 2,638 copies of a tiny class (each in its own loader, so no sharing is possible) before the 16 MiB limit was reached. The direct buffer run stopped after exactly 16 buffers of 1 MiB at the configured limit.

Thread exhaustion under a pids limit of 40 (real output) TERMINAL

```
$ mkdir /sys/fs/cgroup/pids/osw_o3; echo 40 > /sys/fs/cgroup/pids/osw_o3/pids.max
$ ( echo $BASHPID > /sys/fs/cgroup/pids/osw_o3/cgroup.procs; \
    java -Xmx64m -cp . OomKinds threads 2>&1 )
[0.042s][warning][os,thread] Failed to start thread "Unknown thread" - pthread_create failed
[0.043s][warning][os,thread] Failed to start the native thread for java.lang.Thread "Thread-18"
caught: java.lang.OutOfMemoryError: unable to create native thread:
  possibly out of memory or process/resource limits reached
objects/threads created: 18
$ cat /sys/fs/cgroup/pids/osw_o3/pids.events
max 4
```

The cgroup allowed 40 tasks; the JVM itself uses about 20 for its own threads, so 18 application threads were created. The 19th `pthread_create` returned `EAGAIN` and the JVM converted it into `OutOfMemoryError: unable to create native thread`. The error does not mean that memory is exhausted; the message lists both causes. `pids.events` counted four refused forks.

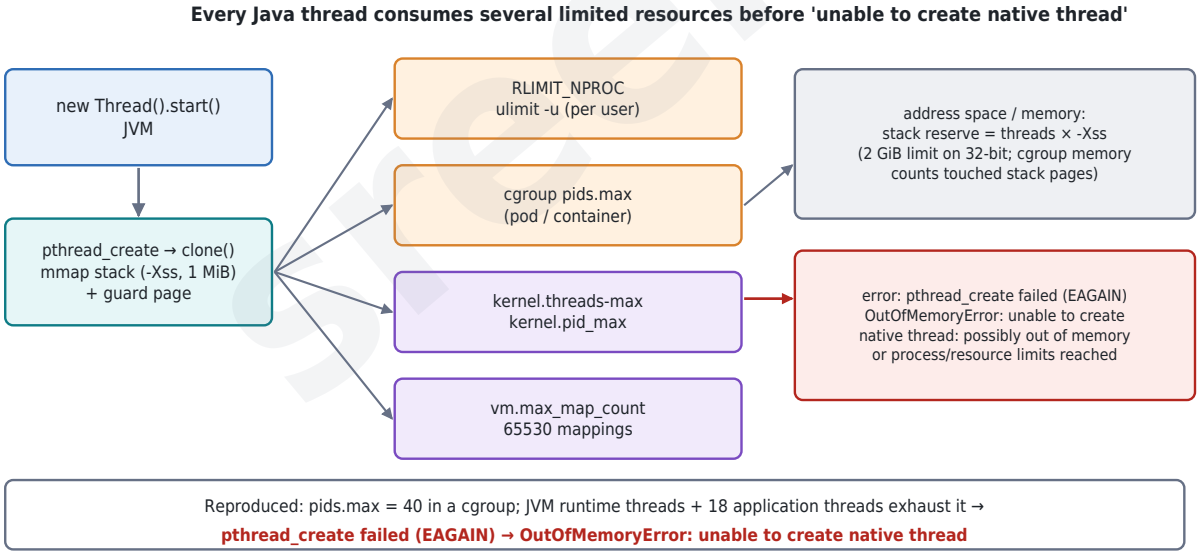


Figure 3.32. The resources a new Java thread needs before pthread_create can succeed: process and task limits, pids cgroup, kernel thread and map-count limits, and stack address space.

The unbounded cache

`LeakDemo.java` stores a record with a 4 KiB array in a static `ConcurrentHashMap` roughly 800 times per second and never removes entries. It runs with `-Xmx256m`.

LeakDemo.java JAVA

```java
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

public class LeakDemo {
    record Profile(String userId, byte[] avatar) { }
```

LeakDemo.java (continued) JAVA

```java
    // Unbounded cache: entries are added on every request and never evicted.
    static final Map<String, Profile> CACHE = new ConcurrentHashMap<>();

    public static void main(String[] args) throws Exception {
        System.out.println("pid " + ProcessHandle.current().pid());
        long id = 0;
        long end = System.currentTimeMillis() + Long.parseLong(args[0]);
        while (System.currentTimeMillis() < end) {
            String key = "user-" + (id++);
            CACHE.put(key, new Profile(key, new byte[4096]));
            Thread.sleep(1);   // about 700-900 requests per second
        }
        System.out.println("entries " + CACHE.size());
    }
}
```

Observing the leak (real output at about 6 s and 14 s after start; PID 4591) TERMINAL

```
$ jcmd 4591 GC.class_histogram | head -6
 num     #instances         #bytes  class name (module)
-------------------------------------------------------
   1:         18804       22440128  [B (java.base@21.0.12.1)
   2:         13395         321480  java.lang.String (java.base@21.0.12.1)
   3:          6436         205952  java.util.concurrent.ConcurrentHashMap$Node (java.base@21)
$ sleep 8; jcmd 4591 GC.class_histogram | head -6
   1:         33467       52819824  [B (java.base@21.0.12.1)
   2:         20727         497448  java.lang.String (java.base@21.0.12.1)
   3:         13767         440544  java.util.concurrent.ConcurrentHashMap$Node (java.base@21)
$ jstat -gcutil 4591
  S0     S1     E      O      M     CCS    YGC     YGCT    FGC    FGCT    CGC    CGCT     GCT
    -      -   0.00  66.66  65.11  20.56      6    0.049      2   0.024      0   0.000   0.073
```

Live `byte[]` grew from 22.4 MB to 52.8 MB in eight seconds while the collector ran (histograms are taken after a full collection, so the numbers are live bytes). The old generation (`O`) was 66.66 percent full with two full collections (`FGC` 2), which could not free it: this is the signature of a leak, as opposed to a high but stable use.

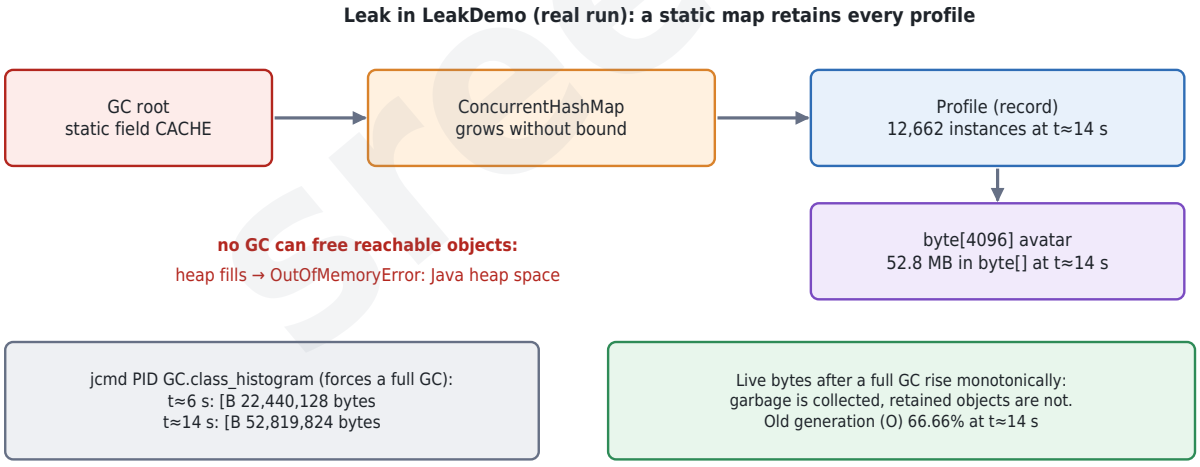


Figure 3.33. The retention path of the leak: a static field holds a map that holds every Profile and its byte array, so no collection can free them; the histogram counters rise monotonically.

3.6.4 Real-world incidents

Incident: heap growth from an unbounded cache

Scenario. A Spring Boot service caches user profiles in a static `ConcurrentHashMap` populated by a request filter. The map has no size bound and no expiry. The service runs with `-Xmx256m` in a container with a 512 MiB limit and is restarted weekly by the deployment pipeline, which hides the problem until traffic grows and the restart interval falls to a day.

Symptoms. The heap graph shows a sawtooth whose lower envelope (live data after each collection) rises steadily. Collection pauses become longer and more frequent. Eventually requests fail and the log shows `java.lang.OutOfMemoryError: Java heap space`.

Diagnosis. The commands below are run in this order; the output is from the reproduction `LeakDemo` described earlier.

1. `jstat -gcutil PID 1000 5` shows the old generation (`O`) and the number of full collections (`FGC`). A full collection that does not reduce `O` indicates retained data. Reproduction: `O` 66.66 percent with `FGC` 2 at 14 seconds.
2. `jcmd PID GC.class_histogram | head -20` shows which classes occupy the heap. Two samples a minute apart show growth. Reproduction: `[B` rose from 22,440,128 to 52,819,824 bytes and `LeakDemo$Profile` reached 12,662 instances.
3. `jcmd PID GC.heap_dump /dumps/h.hprof` writes a dump (108,559,423 bytes for 100 MiB of live data) that a heap analyser opens. The dominator tree shows that one `ConcurrentHashMap` retains nearly all of the heap and that the path to it starts at the static field `LeakDemo.CACHE`.

Root cause. A reachable collection grows without bound. The collector is working correctly: the objects are reachable and cannot be reclaimed.

Fix. Immediate: restart the instance and, if possible, reduce the cache TTL by configuration. Permanent: replace the map by a bounded cache with expiry (for example Caffeine with `maximumSize` and `expireAfterWrite`), and add the metric of the cache size.

Prevention. Alert on the live-set after collections (`jvm_memory_pool_bytes_used` of the old generation after GC) rising for several hours. Run a soak test in which memory after a full GC must be flat. Enable `-XX:+HeapDumpOnOutOfMemoryError` with a persistent dump path.

Incident: Metaspace growth from class-loader leaks

Scenario. An application server hosts a web application that is redeployed several times a day without restarting the JVM. A library that registers a shutdown hook and holds a static reference to the web application class loader prevents the loader from being collected after each redeploy.

Symptoms. After a few redeploys the log shows `java.lang.OutOfMemoryError: Metaspace`, while heap graphs are normal. The resident size of the process grows slowly with each deployment.

Diagnosis. Metaspace use is read directly and compared with the number of loaded classes.

Reproduction: one class in a new class loader per iteration, with a 16 MiB metaspace limit TERMINAL

```
$ java -Xmx64m -XX:MaxMetaspaceSize=16m -cp . OomKinds meta
caught: java.lang.OutOfMemoryError: Metaspace
objects/threads created: 2638
```

On a live process: `jcmd PID VM.metaspace` and `jcmd PID VM.classloader_stats` list the class loaders with their class counts; the leaked loaders appear in numbers equal to the number of redeploys. `jstat -gc PID` shows the `MU` column (metaspace used) rising after each deploy and not falling after a full collection.

Root cause. Each class loader keeps its classes and their metadata alive as long as the loader is reachable. A single stray reference to the loader (a thread, a static field in a parent loader, a `ThreadLocal`, a JDBC driver registration) keeps every class of the application in Metaspace. Metaspace is native memory: by default it is limited only by the available memory, so it grows until the container is killed or the JVM is stopped.

Fix. Immediate: restart the JVM. Permanent: find the reference path to the leaked loader in a heap dump (select instances of `ClassLoader`, path to GC roots), deregister drivers and threads on undeploy, upgrade the offending library, or stop redeploying in place and replace the whole process in the deployment (the container model).

Prevention. Set `-XX:MaxMetaspaceSize` (for example 256m) so that a leak produces a clear `OutOfMemoryError: Metaspace` instead of an OOM kill; alert on metaspace used after full GC; monitor loaded class counts (`jvm_classes_loaded_classes`).

Incident: direct buffer exhaustion in a network service

Scenario. A gateway service built on Netty serves large responses. Netty uses pooled direct buffers, which are allocated outside the heap. The service runs with `-Xmx512m` and no `-XX:MaxDirectMemorySize`, in a 1 GiB container.

Symptoms. Two failure modes appear, depending on load. Under moderate load the log contains `java.lang.OutOfMemoryError: Direct buffer memory`. Under heavier load the pod is `OOMKilled` (exit 137) without a Java error. In both cases heap dashboards are normal.

Diagnosis. The JVM is restarted with `-XX:NativeMemoryTracking=summary` and the categories are compared over time: `jcmd PID VM.native_memory summary.diff` after a baseline shows `Other` (direct buffers allocated by `ByteBuffer`) or Netty-internal counters growing. The effect of the limit is reproduced as follows.

Direct memory limit and the resulting error (real output) TERMINAL

```
$ java -Xmx64m -XX:MaxDirectMemorySize=16m -cp . OomKinds direct
caught: java.lang.OutOfMemoryError: Cannot reserve 1048576 bytes of direct buffer memory
  (allocated: 16777216, limit: 16777216)
objects/threads created: 16
```

When the limit is the default (the maximum heap size, 512 MiB here), direct memory may legitimately reach 512 MiB in addition to the heap, for a possible 1 GiB plus the JVM base; this exceeds the 1 GiB container and the kernel kills the process first (see the NativeHog reproduction in the chapter on the OOM killer).

Root cause. Direct buffers are native memory freed only when their owning objects are collected or released by the library; a heap that is never under pressure does not collect them. The default limit of direct memory (equal to the heap) was not part of the container budget.

Fix. Immediate: restart; lower the traffic that holds buffers. Permanent: set `-XX:MaxDirectMemorySize` to a budgeted value (for example 256m) so that the JVM throws a loggable error instead of being killed; configure the Netty pool (`io.netty.maxDirectMemory`, smaller pool arenas) and release buffers (`ReferenceCountUtil.release`); leak detection with `-Dio.netty.leakDetection.level=advanced` in a test environment.

Prevention. Budget the container as in the formula of this chapter, export `jvm_buffer_memory_used_bytes{id="direct"}` and alert when it exceeds 70 percent of the limit.

Incident: unable to create native thread

Scenario. A service creates a thread per outbound call with `new Thread(...)` and a bug in a retry loop leaves threads blocked forever. The pod runs with a `pids` limit set by the platform (1,024 in many Kubernetes clusters by `podPidsLimit`).

Symptoms. The log shows `java.lang.OutOfMemoryError: unable to create native thread: possibly out of memory or process/resource limits reached`. Heap and RSS are modest. New requests fail; existing ones continue.

Diagnosis. The thread count and the limit are compared, then the stacks are classified.

1. `ls /proc/PID/task | wc -l` (or the `Threads:` line of `/proc/PID/status`) gives the thread count of the process.
2. The limits applicable to the process are `cat /sys/fs/cgroup/pids/.../pids.max` and `pids.current`, `ulimit -u` and `/proc/sys/kernel/threads-max`. The refused attempts are counted in `pids.events`: in the reproduction `max 4`.
3. `jcmd PID Thread.print` (or `jstack`) shows what the threads wait for; hundreds of threads with an identical stack identify the leaking code path.

Reproduction under a pids limit of 40 (real output) TERMINAL

```
$ java -Xmx64m -cp . OomKinds threads 2>&1
[0.042s][warning][os,thread] Failed to start thread "Unknown thread" - pthread_create failed
[0.043s][warning][os,thread] Failed to start the native thread for java.lang.Thread "Thread-18"
caught: java.lang.OutOfMemoryError: unable to create native thread:
  possibly out of memory or process/resource limits reached
objects/threads created: 18
```

Root cause. The number of threads reached a limit (pids cgroup, `RLIMIT_NPROC`, kernel thread limit, `vm.max_map_count` or address space); each Java thread is a kernel task with its own stack mapping and guard page, and the message is the generic JVM translation of `EAGAIN` from `pthread_create`.

Fix. Immediate: restart the pod; raise the limit only if the thread count is legitimate. Permanent: replace unbounded thread creation by a bounded `ExecutorService` (or virtual threads, which do not use one kernel task each), fix the blocked code path and add timeouts.

Prevention. Export `jvm_threads_live_threads` and alert on growth; set `pids` limits deliberately; budget stacks (`threads × -Xss`) in the container limit; load-test failure modes of downstream services, since they are the usual trigger for blocked threads.

3.6.5 Questions to expect

Questions and model answers

1. **A JVM has `-Xmx2g` and a container limit of 2 GiB. Is that a correct configuration?** No. The heap is only one component; metaspace, code cache, thread stacks, direct buffers and GC structures add several hundred megabytes. The process will be OOM-killed when the total exceeds 2 GiB. A heap of 50 to 70 percent of the limit is a usual starting point.
2. **Why is the RSS of a JVM larger than `-Xmx`?** The resident set includes the non-heap areas (metaspace, code cache, stacks, direct buffers, GC structures, mapped libraries and class files, malloc arenas). In the measured process the NMT committed total was 210,160 KiB for a 129,024 KiB committed heap.
3. **What is Native Memory Tracking and how do you use it?** A JVM facility (`-XX:NativeMemoryTracking=summary|detail`) that counts native allocations by category. Query with `jcmd PID VM.native_memory summary`; use `baseline` and `summary.diff` to find growing categories.
4. **How does the JVM determine its heap size in a container?** It reads the cgroup memory limit (`UseContainerSupport`) and applies `MaxRAMPercentage` (25 percent by default; `MinRAMPercentage` 50 percent for small sizes). `-Xmx` overrides. Verify with `-XX:+PrintFlagsFinal`.
5. **What is the difference between virtual size and resident size for a thread stack?** The JVM reserves `-Xss` of address space per thread, but only touched pages are resident. 200 threads raised `VmSize` by 201 MiB but `VmRSS` by 20 MiB.
6. **How do you find a memory leak in a Java service?** Observe live data after full GCs (rising), take class histograms at intervals to see which class grows, take a heap dump and inspect the dominator tree and the path to the GC root.
7. **How do you find a native memory leak?** Compare NMT baselines for categories that grow; if NMT is flat while RSS grows, suspect JNI libraries or the malloc arenas (`MALLOC_ARENA_MAX`, an alternative allocator such as jemalloc) and inspect `pmap` and `smaps`.
8. **What does `OutOfMemoryError: unable to create native thread` indicate?** A failure of `pthread_create`: a limit on tasks (pids cgroup, `ulimit -u`, `threads-max`), the number of memory mappings, or address space. It is a resource limit, not necessarily a heap problem.
9. **Why is `-XX:MaxDirectMemorySize` worth setting?** Direct memory is outside the heap and by default as large as the heap; unbounded it can push the process beyond the container limit and cause a kill without a Java error. A set limit produces a catchable error.
10. **What does a heap dump cost and what are the risks?** A pause proportional to the live heap, a file as large as the live heap (108 MB for 100 MiB here) and potential exposure of sensitive data in the file; the volume must have the space and the pod limit must allow for the page cache of the write.

KEY POINTS

A JVM process is the heap plus a dozen native areas; the operating system limits the sum.

- Virtual size is reservation, committed is the JVM request, resident is what counts; measured: 2.74 GiB virtual, 205 MiB committed, 159 MiB resident.
- Budget the container: heap + metaspace + code cache + threads × stack + direct memory + GC structures + headroom.
- `jcmd` with `VM.native_memory`, `GC.class_histogram`, `GC.heap_info` and `GC.heap_dump`, together with `pmap` and `/proc/PID/status`, identify the growing area.
- Each exhaustion has its own message: heap space, Metaspace, direct buffer memory, unable to create native thread; a kernel kill has none.
- Bound what can be bounded (`-XX:MaxMetaspaceSize`, `-XX:MaxDirectMemorySize`, thread pools) so that failures are Java errors, not SIGKILLs.

PART 4

Concurrency and Synchronisation

How concurrent threads share state safely, how the kernel and the JVM implement waiting, and how to diagnose failures in production.

In this part

4.1 Race Conditions and Critical Sections
4.2 Mutexes, Semaphores, Condition Variables and Spinlocks
4.3 Deadlock, Livelock, Starvation and Priority Inversion
4.4 Locks in Java and Their Mapping to the Operating System
4.5 Diagnosing Concurrency Problems in Production

4.1 Race Conditions and Critical Sections

Every thread of a process shares the same address space. Two threads that read and write the same variable without coordination produce results that depend on the exact interleaving of their instructions, and the operating system scheduler, not the programmer, decides that interleaving. A defect of this kind is called a race condition. It is the root cause of lost updates, duplicate database rows, corrupted data structures and, indirectly, of every deadlock discussed later in this Part.

This chapter defines the problem precisely, states the properties that any solution must have, and describes the hardware primitives (atomic read-modify-write instructions, compare-and-swap and memory barriers) on which every lock, semaphore and concurrent collection is built. All experiments were run on a Linux 6.18 virtual machine with 2 CPUs and OpenJDK 21.0.12; JDK 25 behaves identically for the examples shown unless stated otherwise.

4.1.1 Concept and mechanism

Race condition and data race

A **race condition** is a flaw in which the correctness of a computation depends on the relative timing of events in several threads. A **data race** is the narrower, mechanically detectable case: two threads access the same memory location, at least one access is a write, and no synchronisation orders the accesses. A program can contain a race condition without a data race (two correctly locked updates may still happen in the wrong order for the business rule), and a data race is always a defect in languages with a defined memory model: it is undefined behaviour in C11 and a source of unspecified (though memory-safe) results in the Java Memory Model.

> **DEFINITION**
> A critical section A critical section is a region of code that accesses a shared resource and must not be executed by more than one thread at a time. Protecting it with mutual exclusion makes the sequence of instructions inside it appear to execute as one indivisible step to every other thread.

Why counter++ is not one step

The statement `counter++` looks atomic in source code. In machine code it is a read-modify-write sequence of three operations: load the value into a register, add one, store the register back. The JVM bytecode shows the same structure:

Bytecode of a static counter increment — TERMINAL

```
$ javap -c -p Counter
  static void inc();
    Code:
       0: getstatic     #7                  // Field count:I
       3: iconst_1
       4: iadd
       5: putstatic     #7                  // Field count:I
       8: return
```

Between the load and the store, the thread may be preempted, or another CPU core may execute its own load of the same, not yet updated, value. Both threads then compute the same result and both store it; one increment disappears. This is a **lost update**.

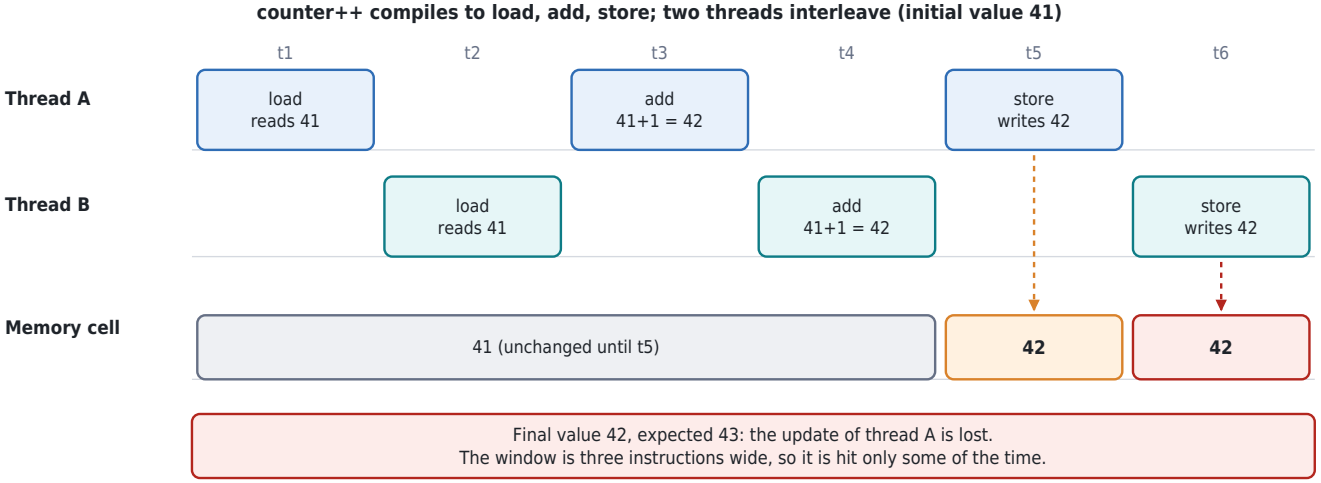


Figure 4.1. Two threads interleave the three steps of counter++: both read 41, both compute 42, and the second store overwrites the first, so one increment is lost.

Requirements for a solution

A correct protocol for entering and leaving a critical section must satisfy three requirements, formulated in the classical operating-systems literature:

- **Mutual exclusion.** At most one thread is inside the critical section at any time.
- **Progress.** If no thread is inside and some threads wish to enter, the choice of the next thread depends only on those threads and cannot be postponed indefinitely.
- **Bounded waiting.** After a thread has requested entry, only a bounded number of other threads can enter before it does. This excludes starvation.

No assumption may be made about relative CPU speeds or the number of processors. A solution that works only on one core, or only when threads are fair, is not a solution.

Hardware support: atomic instructions

Pure software protocols exist (Dekker, Peterson) but they assume that memory operations become visible in program order, which modern CPUs do not guarantee. Practical systems rely on instructions that perform a read-modify-write on one memory location as a single indivisible step with respect to all other cores:

Operation	x86-64	AArch64
`test-and-set / exchange`	`` `xchg reg, [mem]` (implicitly locked) ``	`swp` (v8.1) or `ldxr`/`stxr` loop
`fetch-and-add`	`` `lock xadd [mem], reg` ``	`ldadd` (v8.1) or `ldxr`/`stxr` loop
`compare-and-swap`	`` `lock cmpxchg [mem], reg` ``	`cas` (v8.1) or `ldxr`/`stxr` loop

Atomic read-modify-write primitives (x86-64 and AArch64 shown).

On x86 the `lock` prefix makes the instruction atomic and also acts as a full memory barrier. On older ARM cores atomicity is built from a load-linked/store-conditional pair: `ldxr` loads and marks the address, `stxr` stores only if no other core wrote the address in the meantime, and the sequence is retried on failure.

Compare-and-swap

Compare-and-swap (CAS) takes an address, an expected value and a new value. If the memory still holds the expected value, the new value is stored and the operation reports success; otherwise nothing is stored and failure is reported. All of this is one atomic step. A CAS loop builds an arbitrary update from it: read the current value, compute the new value from it, attempt the CAS, and retry from the read if it failed.

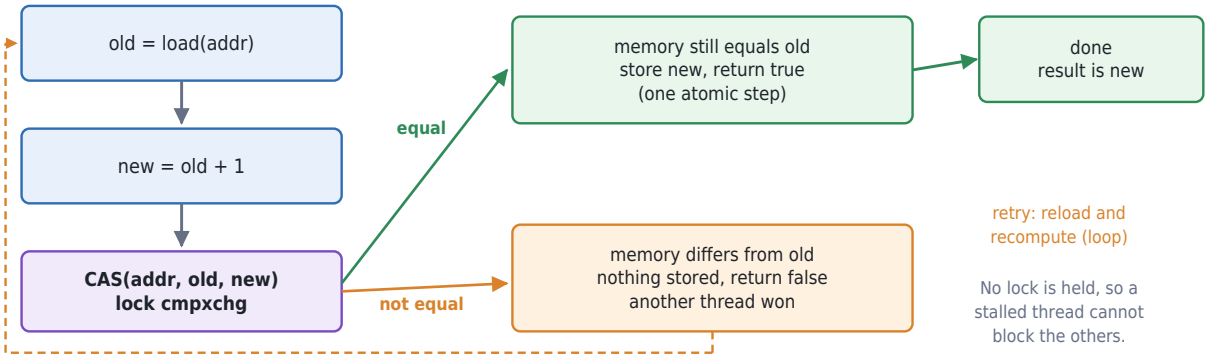


Figure 4.2. A CAS loop recomputes the update whenever another thread changed the location between the read and the CAS; no lock is held, so a descheduled thread never blocks the others.

`java.util.concurrent.atomic.AtomicInteger.incrementAndGet()` compiles to a single `lock xadd` on x86; `AtomicInteger.updateAndGet(f)` and `compareAndSet` use the CAS loop directly. CAS has one well-known weakness, the **ABA problem**: a location holding A is changed to B and back to A, and a CAS that expected A succeeds although the state changed in between. For counters this is harmless; for pointer-based structures it is solved by pairing the value with a version stamp (`AtomicStampedReference`).

Memory ordering and barriers

Atomicity solves only half of the problem. Modern processors and compilers reorder memory operations for speed, and each core keeps pending stores in a **store buffer** before they reach the shared cache. As a result, another core can observe the effects of one thread in an order that no sequential reading of the source code predicts.

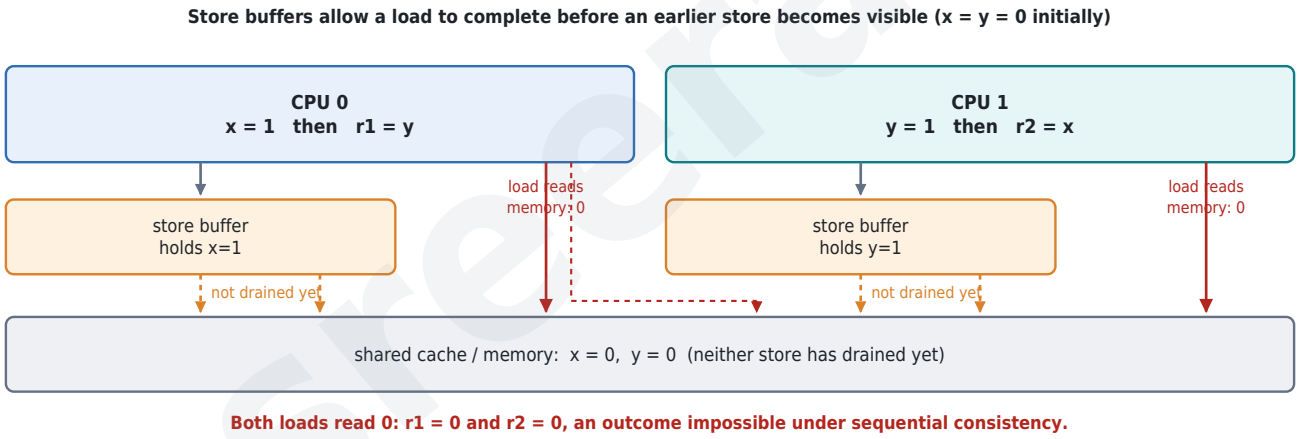


Figure 4.3. Each CPU places its store in a private buffer and then loads the other variable from memory, where it still reads 0; both threads see the other's store as not yet performed.

x86-64 implements a comparatively strong model (total store order) in which only the reordering of a store followed by a load to a different address is visible. ARM and POWER permit many more reorderings. A **memory barrier** (fence) instruction restricts reordering: stores before the barrier become visible before loads and stores after it. Higher-level primitives hide this: acquiring a mutex acts as an acquire barrier and releasing it as a release barrier, so everything written inside a critical section is visible to the next thread that acquires the same lock. In Java the same guarantees are expressed as happens-before edges: unlock then lock of the same monitor, a write then a read of the same `volatile` field, `Thread.start` and `Thread.join`. Part 4 later returns to these edges when thread dumps and `volatile` misuse are discussed; the full model is specified in the Java Memory Model chapter of the Java Language Specification.

GOTCHA

`volatile` guarantees visibility and ordering, not atomicity. A `volatile int` incremented with `++` still performs a separate read and write. In the experiment of this chapter a volatile counter lost between 8 and 32 percent of 10 million increments, depending on the run. Use `AtomicInteger`, `LongAdder` or a lock when the update depends on the previous value.

Sreeram

4.1.2 Worked trace

The table follows the interleaving of the previous figure instruction by instruction. The counter starts at 41; thread A and thread B each execute `counter++` once. The notation `rA` and `rB` denotes a register private to each thread.

Step	Thread A	Thread B	Memory counter	Remark
1	`rA = load` (41)	-	41	A reads the old value
2	-	`rB = load` (41)	41	B reads the same old value
3	`rA = rA + 1` (42)	-	41	A computes in its register
4	-	`rB = rB + 1` (42)	41	B computes in its register
5	`store rA`	-	42	memory shows A's result
6	-	`store rB`	42	B overwrites with the same value

Step-by-step lost update (counter starts at 41).

Two increments were requested and the value rose by one. With a mutex around steps 1 to 5 of each thread, thread B cannot execute its load until thread A has stored, and the final value is 43. With an atomic fetch-and-add, steps 1 to 3 collapse into one indivisible instruction with the same result.

4.1.3 Hands-on

A race in C

The program creates two POSIX threads that each increment a shared `long` one million times without synchronisation. It is compiled with `-O0` so that the compiler keeps the load, add and store visible in the machine code.

race.c — C

```c
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

#define N 1000000
static long counter = 0;

static void *worker(void *arg) {
    (void)arg;
    for (int i = 0; i < N; i++) {
        counter++;
    }
    return NULL;
}

int main(void) {
    pthread_t a, b;
    pthread_create(&a, NULL, worker, NULL);
    pthread_create(&b, NULL, worker, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    printf("expected %d, got %ld\n", 2 * N, counter);
    return 0;
}
```

The disassembly of the worker loop shows the three steps, with the counter in memory between each of them:

Disassembly of the increment (typical output) — TERMINAL

```
$ objdump -d race | grep -A12 "<worker>:" | sed -n 9,13p
    11be:       48 8b 05 53 2e 00 00    mov    0x2e53(%rip),%rax        # 4018 <counter>
    11c5:       48 83 c0 01             add    $0x1,%rax
    11c9:       48 89 05 48 2e 00 00    mov    %rax,0x2e48(%rip)        # 4018 <counter>
```

Four consecutive runs lost between 41 and 49 percent of the increments, and no two runs agreed:

sreeram

Unsynchronised increments, four runs (typical output) TERMINAL

```
$ gcc -O0 -pthread -o race race.c && for i in 1 2 3 4; do ./race; done
expected 2000000, got 1175080
expected 2000000, got 1042279
expected 2000000, got 1073544
expected 2000000, got 1028390
```

With optimisation (`-O2`) the compiler may keep `counter` in a register for the whole loop and write it back once; the program is then still wrong, but wrong in a different way. This illustrates why a data race is undefined behaviour: the compiler is entitled to assume that it does not occur.

Two correct fixes in C

The next program runs the same workload twice: once guarded by a `pthread_mutex_t` and once through a C11 atomic with `atomic_fetch_add`.

fix.c C

```c
#include <pthread.h>
#include <stdatomic.h>
#include <stdio.h>
#include <time.h>

#define N 1000000
static long plain = 0;
static atomic_long atom = 0;
static pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
static int mode;

static void *worker(void *arg) {
    (void)arg;
    for (int i = 0; i < N; i++) {
        if (mode == 0) {
            pthread_mutex_lock(&lock);
            plain++;
            pthread_mutex_unlock(&lock);
        } else {
            atomic_fetch_add(&atom, 1);
        }
    }
    return NULL;
}

static double now(void) {
    struct timespec t;
    clock_gettime(CLOCK_MONOTONIC, &t);
    return t.tv_sec + t.tv_nsec / 1e9;
}

int main(void) {
    for (mode = 0; mode < 2; mode++) {
        pthread_t a, b;
        double t0 = now();
        pthread_create(&a, NULL, worker, NULL);
        pthread_create(&b, NULL, worker, NULL);
        pthread_join(a, NULL);
        pthread_join(b, NULL);
        printf("%-7s result=%ld  time=%.0f ms\n", mode == 0 ? "mutex" : "atomic",
               mode == 0 ? plain : (long)atom, (now() - t0) * 1000);
    }
    return 0;
}
```

Mutex and atomic versions (measured on the build machine, typical) TERMINAL

```
$ gcc -O0 -pthread -o fix fix.c && ./fix && ./fix
mutex   result=2000000  time=139 ms
atomic  result=2000000  time=39 ms
mutex   result=2000000  time=126 ms
atomic  result=2000000  time=36 ms
```

Both versions are correct; the atomic is roughly three to four times faster here because the single `lock xadd` instruction never enters the kernel and never sleeps. The mutex is not slow in itself (an uncontended lock is also a user-space atomic), but two threads that collide on it pay for queueing and futex calls, described in the next chapter.

sreeram

The same experiment in Java

RaceDemo.java — JAVA

```java
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.atomic.LongAdder;

public class RaceDemo {
    static int plain = 0;
    static int locked = 0;
    static final Object LOCK = new Object();
    static final AtomicInteger atomic = new AtomicInteger();
    static final LongAdder adder = new LongAdder();
    static volatile int vol = 0;

    static void run(String name, Runnable body) throws InterruptedException {
        Thread a = new Thread(body);
        Thread b = new Thread(body);
        long t0 = System.nanoTime();
        a.start(); b.start();
        a.join(); b.join();
        System.out.printf("%-12s %.0f ms%n", name, (System.nanoTime() - t0) / 1e6);
    }

    public static void main(String[] args) throws Exception {
        final int n = 5_000_000;
        run("plain int", () -> { for (int i = 0; i < n; i++) plain++; });
        run("volatile", () -> { for (int i = 0; i < n; i++) vol++; });
        run("synchronized", () -> {
            for (int i = 0; i < n; i++) {
                synchronized (LOCK) { locked++; }
            }
        });
        run("AtomicInt", () -> { for (int i = 0; i < n; i++) atomic.incrementAndGet(); });
        run("LongAdder", () -> { for (int i = 0; i < n; i++) adder.increment(); });
        System.out.println("plain       = " + plain);
        System.out.println("volatile    = " + vol);
        System.out.println("synchronized = " + locked);
        System.out.println("AtomicInt   = " + atomic.get());
        System.out.println("LongAdder   = " + adder.sum());
    }
}
```

Run on the build machine, the unsafe variants lost updates in most runs, while the three safe variants always produced 10,000,000:

RaceDemo, one typical run — TERMINAL

```
$ java RaceDemo.java
plain int    19 ms
volatile     266 ms
synchronized 475 ms
AtomicInt    65 ms
LongAdder    124 ms
plain        = 10000000
volatile     = 7098011
synchronized = 10000000
AtomicInt    = 10000000
LongAdder    = 10000000
```

In this particular run the plain `int` was correct; in other runs of the same program it ended at 9993835 and 9980288. That variability is the defining property of a race: the defective run may pass every test and fail in production. Two further observations follow from the numbers. First, the JIT compiler may keep the plain field in a register, which makes the unsafe loop very fast and the loss rare in short runs. Second, the volatile counter lost far more, because every increment is forced to memory where the two cores constantly interfere. A `LongAdder` is slower than an `AtomicInteger` in this two-thread test because it is designed for many threads: it spreads contention over several cells and sums them on read.

4.1.4 Real-world incidents

Incident: duplicate orders from a check-then-act sequence

Scenario. A Spring Boot order service runs three replicas behind a load balancer. The handler checks whether an order with the client-supplied request identifier already exists and, if not, inserts it. A mobile client retries after a timeout, and the retry reaches a second replica while the first request is still in flight.

Symptoms. Customers receive two confirmation e-mails and two shipments for one purchase. The error rate is zero; no exception appears in any log. Support tickets cluster in periods of network latency.

Diagnosis. The duplicates are visible in the database as soon as the right question is asked. Rows sharing a request identifier prove that the application check did not serialise the two requests:

Finding duplicates (illustrative query; output typical) — TERMINAL

```
$ psql orders -c "SELECT request_id, COUNT(*) FROM orders GROUP BY request_id HAVING COUNT(*) > 1"
 request_id | count
------------+-------
 req-42     |     2
```

Application locks cannot explain or repair this: a `synchronized` block protects one JVM, while three replicas run three JVMs. The reproduction below uses SQLite and five threads, each with its own connection, to model five concurrent requests. Each thread executes the check, waits 50 ms (network and service latency) and then inserts.

orders.py — PYTHON

```python
import os, sqlite3, threading, time

DB = "/tmp/osw_o4/orders.db"

def connect():
    # autocommit: every statement is its own transaction
    c = sqlite3.connect(DB, timeout=10, isolation_level=None)
    return c

def setup(unique):
    if os.path.exists(DB):
        os.remove(DB)
    c = connect()
    c.execute("CREATE TABLE orders (id INTEGER PRIMARY KEY, request_id TEXT NOT NULL, sku TEXT)")
    if unique:
        c.execute("CREATE UNIQUE INDEX uq_request ON orders(request_id)")
    c.execute("CREATE TABLE inventory (sku TEXT PRIMARY KEY, qty INTEGER)")
    c.execute("INSERT INTO inventory VALUES ('book', 1)")
    c.close()

def place_check_then_act(results):
    c = connect()
    seen = c.execute(
        "SELECT COUNT(*) FROM orders WHERE request_id = 'req-42'").fetchone()[0]   # check
    if seen == 0:
        time.sleep(0.05)                      # latency between the check and the act
        try:
            c.execute("INSERT INTO orders (request_id, sku) VALUES ('req-42', 'book')")
            results.append("inserted")
        except sqlite3.IntegrityError:
            results.append("rejected by unique index")
    else:
        results.append("duplicate detected by check")
    c.close()

def run(unique):
    setup(unique)
    res = []
    ts = [threading.Thread(target=place_check_then_act, args=(res,)) for _ in range(5)]
    for t in ts: t.start()
    for t in ts: t.join()
    c = connect()
    n = c.execute("SELECT COUNT(*) FROM orders").fetchone()[0]
    print("unique index=%-5s rows in orders=%d  inserted=%d rejected=%d"
          % (unique, n, res.count("inserted"), res.count("rejected by unique index")))

def oversell(atomic):
    setup(False)
```

orders.py (continued) PYTHON

```
    sold = []
    def buy():
        c = connect()
        if atomic:
            cur = c.execute("UPDATE inventory SET qty = qty - 1 WHERE sku = 'book' AND qty > 0")
            if cur.rowcount == 1: sold.append(1)
        else:
            qty = c.execute("SELECT qty FROM inventory WHERE sku = 'book'").fetchone()[0]
            if qty > 0:
                time.sleep(0.05)
                c.execute("UPDATE inventory SET qty = ? WHERE sku = 'book'", (qty - 1,))
                sold.append(1)
        c.close()
    ts = [threading.Thread(target=buy) for _ in range(5)]
    for t in ts: t.start()
    for t in ts: t.join()
    print("atomic UPDATE=%-5s copies sold=%d (stock was 1)" % (atomic, len(sold)))

run(False)
run(True)
oversell(False)
oversell(True)
```

Reproduction of the race and of both fixes (measured) TERMINAL

```
$ python3 orders.py
unique index=False rows in orders=5  inserted=5 rejected=0
unique index=True  rows in orders=1  inserted=1 rejected=4
atomic UPDATE=False copies sold=5 (stock was 1)
atomic UPDATE=True  copies sold=1 (stock was 1)
```

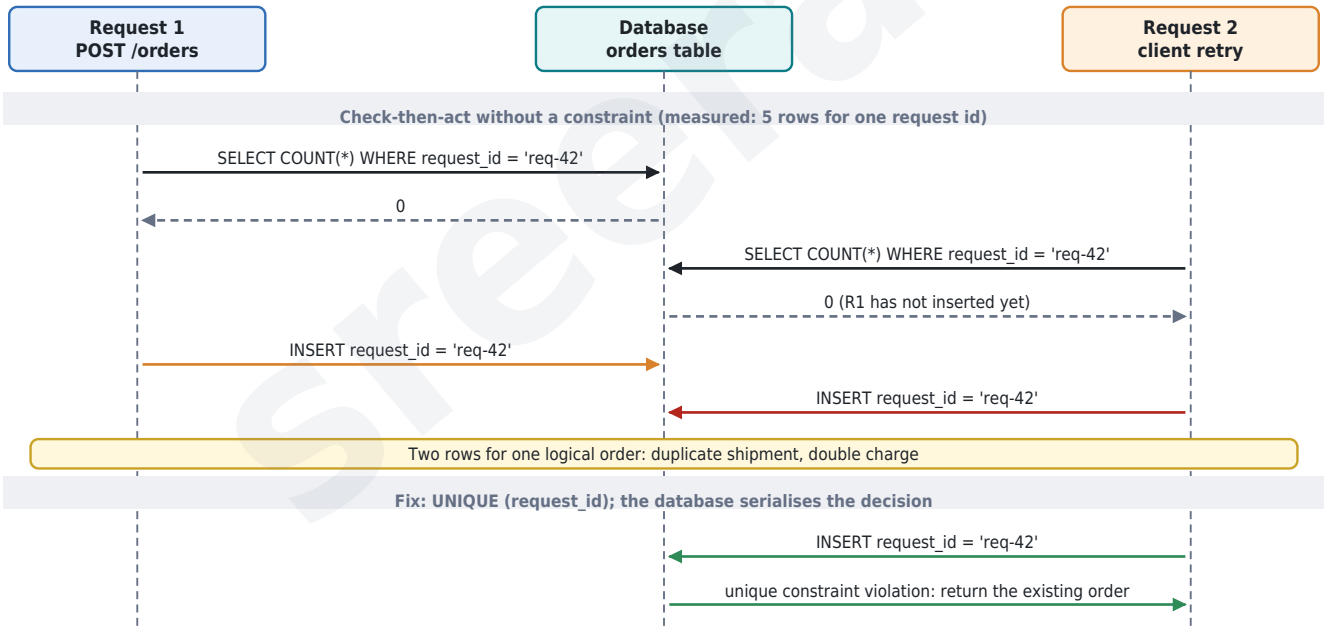


Figure 4.4. The check by request 2 happens before the insert by request 1 is visible, so both inserts proceed; a unique constraint turns the second insert into an error that the application can handle.

Root cause. A check-then-act sequence executed as two separate database statements. The check and the act are not one atomic step, so the window between them is a critical section that no lock protects. The stock example shows the same defect as a read-modify-write: five buyers each read quantity 1 and each decrement it.

Fix. Immediate: delete the duplicate rows and, if payments are affected, reconcile them. Permanent: let the database arbitrate. Declare `UNIQUE (request_id)`, treat the resulting constraint violation (`DataIntegrityViolationException` in Spring) as "already processed" and return the stored order. For inventory, replace the read-modify-write by one conditional statement, `UPDATE inventory SET qty = qty - 1 WHERE sku = ? AND qty > 0`, and check that exactly one row was updated. Where a transaction spans several statements, use a locking read (`SELECT ... FOR UPDATE`) or an isolation level that prevents the anomaly, after confirming how the chosen database implements it.

sreeram

Prevention. Require an idempotency key on every state-changing endpoint, enforce uniqueness in the schema, add a concurrent integration test that fires the same request from several threads, and alert on `GROUP BY ... HAVING COUNT(*) > 1` queries run periodically against business keys.

Incident: under-counting metrics in a singleton bean

Scenario. A controller in a Spring application keeps `private int requestCount` in a field and increments it per request. Spring beans are singletons, so every Tomcat worker thread shares the field.

Symptoms. The dashboard built from the field reports 9,980,288 requests while the access log, the load balancer and the database all agree on 10,000,000. The discrepancy varies from one deployment to the next and grows with traffic.

Diagnosis. Comparing an independent count (access-log lines, `wc -l`) with the application counter shows a deficit that is never negative. A deficit that appears only under concurrent load, never under a single-threaded test, and differs between runs is the signature of lost updates. The `RaceDemo` run above reproduces the same pattern: the plain counter ended at 9980288 in one run and at 10000000 in another.

Root cause. A non-atomic increment of shared mutable state by multiple threads, with no happens-before relationship between them.

Fix. Immediate: restart does not help; accept the metric as approximate. Permanent: replace the field by a `java.util.concurrent.atomic.LongAdder`, or, preferably, by a Micrometer `Counter`, which is already thread-safe.

Prevention. Treat every mutable field of a singleton as shared. Prefer immutable beans and thread-safe types; run static analysis or review for non-final fields in `@Component`, `@Service` and `@Controller` classes.

4.1.5 Questions to expect

1. **What is a race condition, and how does it differ from a data race?** A race condition is incorrect behaviour that depends on thread timing. A data race is two unsynchronised accesses to one location, at least one a write. A program can have either without the other.
2. **Why is `i++` not thread-safe even for an `int`?** It is a load, an add and a store. Two threads can load the same value and both store the same result, losing one update. Only a lock or an atomic read-modify-write makes it indivisible.
3. **What three properties must a critical-section solution have?** Mutual exclusion, progress and bounded waiting.
4. **How does compare-and-swap work, and what is the ABA problem?** CAS atomically stores a new value only if the location still holds the expected value. ABA occurs when the value changes from A to B and back to A, so a CAS succeeds although the state changed; version stamps solve it.
5. **Does `volatile` make `counter++` safe?** No. It makes each read and write visible and ordered, but the increment remains two separate operations. In the experiment it lost up to about 32 percent of the increments in a run.
6. **When is `AtomicInteger` preferable to `synchronized`, and when is it not?** For a single variable updated independently it is simpler and avoids blocking. When an invariant spans several variables, a lock is required because CAS protects one location only.
7. **Why can a lock in a Java service not prevent duplicate orders across replicas?** The lock lives in one JVM. State shared across processes must be arbitrated by a shared system, normally the database through unique constraints or conditional updates.
8. **What does a memory barrier do?** It limits reordering of memory operations around it, so that stores before it become visible before loads and stores after it. Lock acquire and release imply the necessary barriers.

KEY POINTS

- A race condition arises when correctness depends on the interleaving of threads; `counter++` is three steps (load, add, store), so concurrent increments lose updates.
- A critical-section solution needs mutual exclusion, progress and bounded waiting, and cannot assume anything about CPU speed.
- Atomic read-modify-write instructions (`lock xadd`, `lock cmpxchg`, `xchg`) are the hardware basis of all locks and atomics; CAS loops build lock-free updates.
- Memory barriers constrain the reordering caused by store buffers and compilers; lock operations and `volatile` provide them, but `volatile` does not provide atomicity.
- Races are nondeterministic: a test may pass for months. Reproduce them with many threads and long loops, and fix them structurally (unique constraints, atomics, immutable state).

4.2 Mutexes, Semaphores, Condition Variables and Spinlocks

A critical section needs an entry protocol, and the thread that finds the section occupied needs a policy: wait by repeatedly testing, or give up the CPU until the section is free. The operating system and the C library provide a small family of synchronisation primitives that implement these policies. The same primitives underlie Java: `synchronized`, `ReentrantLock`, `Semaphore` and `Condition` are all built on the mechanisms described here, and every contended Java lock ends in a Linux system call named `futex`.

4.2.1 Concept and mechanism

Mutex

A **mutex** (mutual-exclusion lock) has two operations, lock and unlock, and an owner. The thread that locked it must unlock it. A thread that finds the mutex held is put to sleep by the kernel and woken when the mutex is released, so a waiting thread consumes no CPU time. POSIX offers several types: the default (normal) mutex, whose behaviour on self-deadlock or on unlock by a non-owner is undefined; the error-checking mutex, which returns an error; and the recursive mutex, which counts nested locks by the same thread. Java's `synchronized` and `ReentrantLock` are recursive.

Spinlock

A **spinlock** is a lock whose waiters do not sleep: they loop on an atomic test-and-set until the lock is free. The wait costs CPU cycles but avoids the context switch. It is appropriate when the critical section is shorter than a context switch, when the holder cannot be preempted, and when at least as many CPUs as contenders exist. These conditions hold inside the Linux kernel, where a spinlock holder runs with preemption (and often interrupts) disabled, and rarely hold in ordinary user programs. If a user-space thread holding a spinlock is preempted, every spinning thread burns its whole time slice without progress, as the measurement later in this chapter shows.

Property	Mutex (sleep)	Spinlock (busy-wait)
Waiter state	Sleeping in the kernel, no CPU used	Running, 100 percent of a CPU
Cost of waiting	Two context switches and a futex call	Cycles burned while spinning
Best for	Critical sections longer than a few microseconds	Very short sections, non-preemptible holder
Safe with more threads than CPUs	Yes	No: lock-holder preemption
Typical use	Application code, Java monitors	Kernel data structures, lock-free retry loops

Sleeping lock against spinning lock.

Semaphore

A **semaphore**, introduced by Dijkstra, is a non-negative integer with two atomic operations. `wait` (P) decrements the integer and blocks while it is zero. `post` (V) increments it and wakes one waiter. A semaphore initialised to 1 acts as a mutex, but it has no owner: any thread may post, which makes semaphores suitable for signalling between threads ("one item is now available") and unsuitable for protecting data against accidental unlocking. A counting semaphore initialised to N limits concurrency to N, which is the model of a connection pool or of `java.util.concurrent.Semaphore`.

Condition variable and monitor

A mutex answers "may I enter?" but not "has the state I need arrived?". A **condition variable** lets a thread wait for a predicate on shared state. `wait(cond, mutex)` atomically releases the mutex and sleeps; on wake-up it reacquires the mutex before returning. `signal` wakes one waiter and `broadcast` wakes all. Three rules apply in every language:

- The predicate is always tested in a `while` loop, never an `if`. A wake-up does not guarantee that the predicate is true: another thread may have consumed the state first, and POSIX permits spurious wake-ups.
- The state change and the signal are made while holding the same mutex that the waiter uses, which prevents a signal from falling between the waiter's test and its sleep (a lost wake-up).
- `broadcast` is the safe default when waiters wait for different predicates on one condition variable.

A **monitor** packages a mutex with its condition variables and the data they guard. Java builds a monitor into every object: `synchronized` is the lock, `wait`, `notify` and `notifyAll` are the condition operations.

How a mutex is implemented: the futex

Entering the kernel for every lock operation would make locking expensive, and the great majority of lock operations are uncontended. Linux therefore implements the mutex in two layers, joined by the **futex** ("fast user-space mutex") system call. The lock state is one 32-bit integer in user memory. The fast path is an atomic instruction on that integer and involves no kernel. Only a thread that must sleep, or a thread that releases a lock that has sleepers, calls `futex`. The two operations used by mutexes are:

- `FUTEX_WAIT(addr, expected)`: if the integer at `addr` still equals `expected`, put the caller to sleep on a kernel queue keyed by that address; otherwise return immediately. The re-check, performed under the kernel's own lock, closes the race between a thread deciding to sleep and another thread releasing the lock.
- `FUTEX_WAKE(addr, n)`: wake at most `n` threads sleeping on `addr`.

The glibc normal mutex encodes three states in the word: 0 (unlocked), 1 (locked, no waiters), 2 (locked, waiters may exist). The flag `FUTEX_PRIVATE_FLAG` marks a futex shared only by threads of one process, which lets the kernel skip the cross-process address lookup.

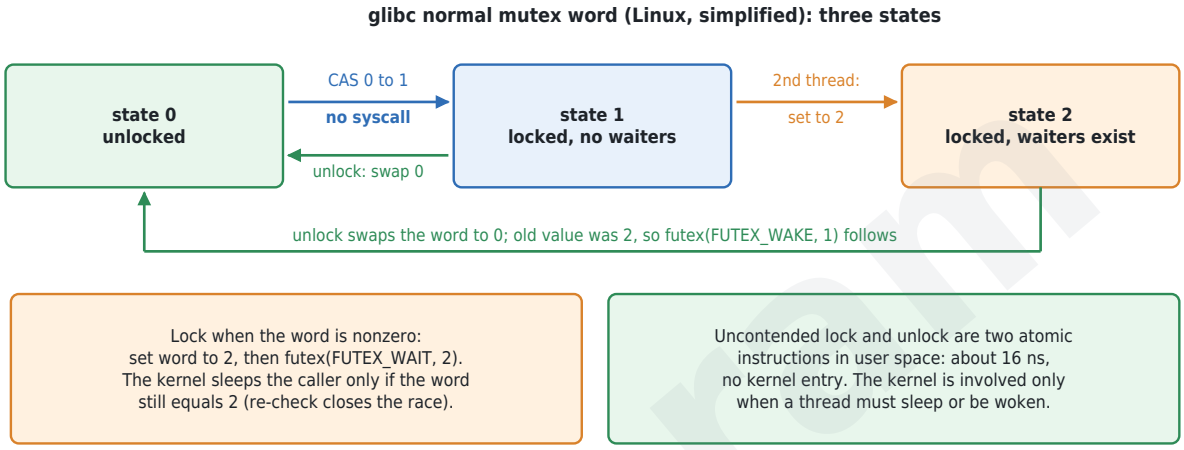


Figure 4.5. The mutex word moves between three states; the kernel is entered only on the transitions that put a thread to sleep (word set to 2, then FUTEX_WAIT) or wake one (unlock from state 2, then FUTEX_WAKE).

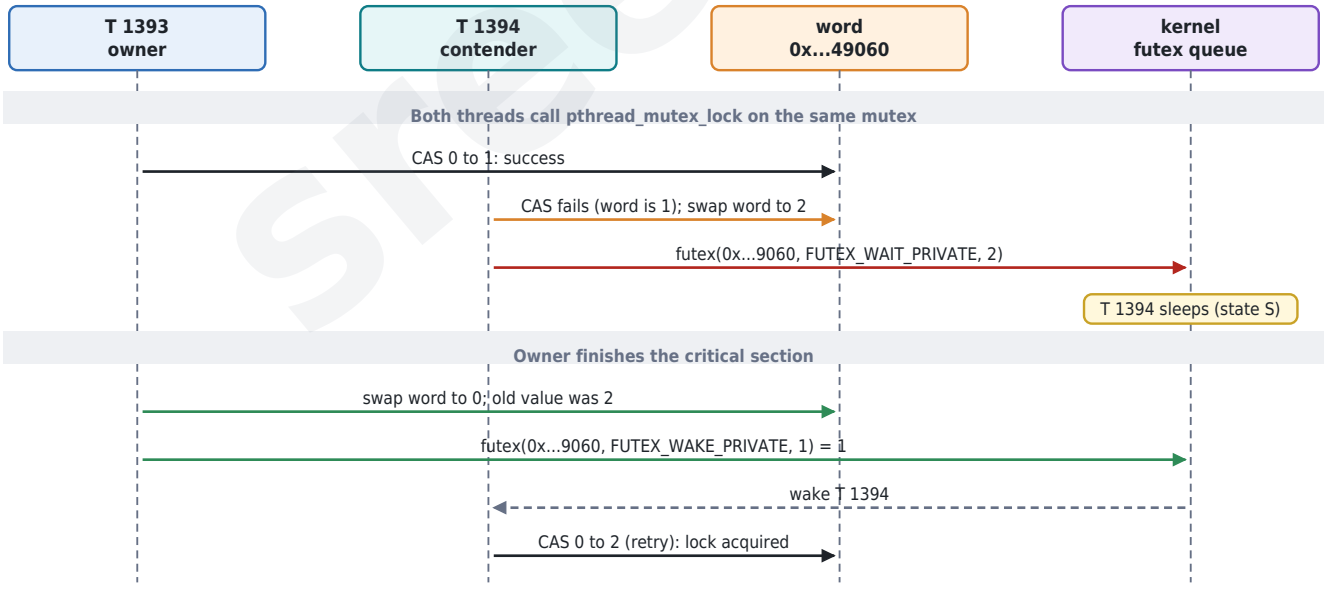


Figure 4.6. Sequence of a contended lock as recorded by strace: thread 1393 owns the mutex, thread 1394 sleeps in FUTEX_WAIT with expected value 2, and the unlock performs FUTEX_WAKE.

Condition variables, semaphores and the parking primitives of the JVM use the same system call with different words and different protocols; only the mutex word is described here.

4.2.2 Classical synchronisation problems

Bounded buffer (producer and consumer)

Producers add items to a buffer of fixed capacity, consumers remove them. A producer must wait when the buffer is full, a consumer when it is empty, and access to the buffer indices must be exclusive. The standard solution uses two counting semaphores, `empty` (free slots, initially the capacity) and `full` (filled slots, initially 0), plus one mutex for the indices. The order of the semaphore wait before the mutex matters: reversing it lets a producer hold the mutex while it sleeps for a free slot that only a consumer, blocked on that mutex, could create, which is a deadlock.

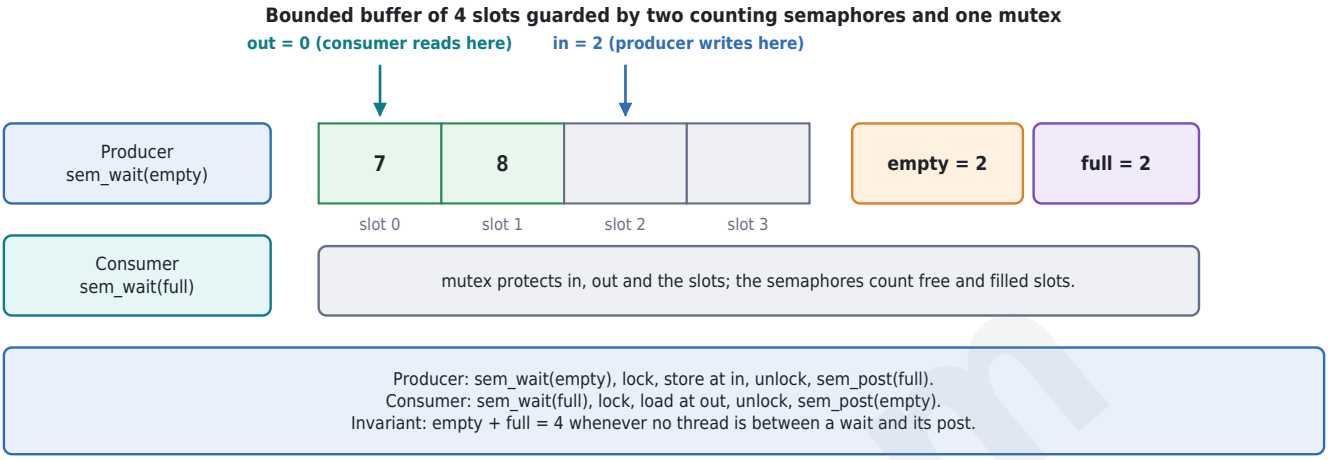


Figure 4.7. Two semaphores count free and filled slots while the mutex protects the indices; the invariant empty + full = 4 holds whenever no thread is between a wait and its post.

The following C program runs one producer and one consumer over four slots and verifies the sum of 100,000 items.

pc.c C

```c
#include <pthread.h>
#include <semaphore.h>
#include <stdio.h>

#define CAP 4
#define ITEMS 100000

static int buf[CAP];
static int in = 0, out = 0;
static sem_t empty, full;
static pthread_mutex_t mu = PTHREAD_MUTEX_INITIALIZER;

static void *producer(void *arg) {
    (void)arg;
    for (int i = 1; i <= ITEMS; i++) {
        sem_wait(&empty);               /* wait for a free slot */
        pthread_mutex_lock(&mu);
        buf[in] = i;
        in = (in + 1) % CAP;
        pthread_mutex_unlock(&mu);
        sem_post(&full);                /* announce one more item */
    }
    return NULL;
}

static void *consumer(void *arg) {
    long *sum = arg;
    for (int i = 1; i <= ITEMS; i++) {
        sem_wait(&full);                /* wait for an item */
        pthread_mutex_lock(&mu);
        *sum += buf[out];
        out = (out + 1) % CAP;
        pthread_mutex_unlock(&mu);
        sem_post(&empty);               /* announce one more free slot */
    }
    return NULL;
}

int main(void) {
```

pc.c (continued) C

```c
    long sum = 0;
    pthread_t p, c;
    sem_init(&empty, 0, CAP);
    sem_init(&full, 0, 0);
    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, &sum);
    pthread_join(p, NULL);
    pthread_join(c, NULL);
    printf("consumed sum=%ld expected=%ld\n", sum, (long)ITEMS * (ITEMS + 1) / 2);
    return 0;
}
```

Bounded buffer in C (measured) TERMINAL

```
$ gcc -O1 -pthread -o pc pc.c && ./pc
consumed sum=5000050000 expected=5000050000
```

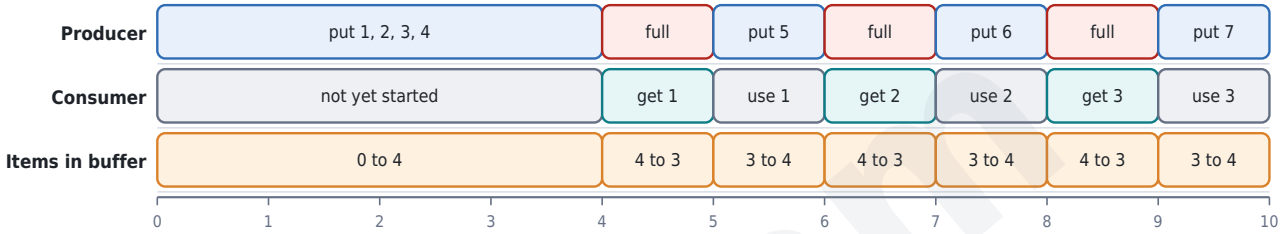


Figure 4.8. A slow consumer and a fast producer: the producer fills four slots, then is blocked on the full buffer and advances only when the consumer posts a free slot.

With a four-slot buffer the two threads alternate constantly and almost every operation sleeps and wakes a thread. Under `strace -f -c -e trace=futex` the same run recorded 182,870 futex calls (the tracer slows the program but does not change the count order of magnitude). This is why throughput-sensitive systems use larger buffers or batch several items per hand-off.

In Java the same structure uses a lock with two conditions. The listing shows the rules above: both predicates are tested in `while` loops and the signal is sent while the lock is held.

BoundedBuffer.java JAVA

```java
import java.util.ArrayDeque;
import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.ReentrantLock;

public class BoundedBuffer {
    private final ArrayDeque<Integer> items = new ArrayDeque<>();
    private final int capacity;
    private final ReentrantLock lock = new ReentrantLock();
    private final Condition notFull = lock.newCondition();
    private final Condition notEmpty = lock.newCondition();

    BoundedBuffer(int capacity) { this.capacity = capacity; }

    void put(int v) throws InterruptedException {
        lock.lock();
        try {
            while (items.size() == capacity) {
                notFull.await();                // loop guards against spurious wake-ups
            }
            items.addLast(v);
            notEmpty.signal();
        } finally {
            lock.unlock();
        }
    }

    int take() throws InterruptedException {
        lock.lock();
        try {
            while (items.isEmpty()) notEmpty.await();
            int v = items.removeFirst();
            notFull.signal();
```

BoundedBuffer.java (continued) JAVA

```
            return v;
        } finally {
            lock.unlock();
        }
    }

    public static void main(String[] args) throws Exception {
        BoundedBuffer buf = new BoundedBuffer(4);
        int producers = 3;
        int consumers = 3;
        int perProducer = 100_000;
        long[] sums = new long[consumers];
        Thread[] ts = new Thread[producers + consumers];
        for (int p = 0; p < producers; p++) {
            ts[p] = new Thread(() -> {
                try {
                    for (int i = 1; i <= perProducer; i++) buf.put(i);
                } catch (InterruptedException e) { }
            });
        }
        for (int c = 0; c < consumers; c++) {
            final int id = c;
            ts[producers + c] = new Thread(() -> {
                try {
                    for (int i = 0; i < perProducer; i++) sums[id] += buf.take();
                } catch (InterruptedException e) { }
            });
        }
        for (Thread t : ts) t.start();
        for (Thread t : ts) t.join();
        long total = 0;
        for (long s : sums) total += s;
        long expected = (long) producers * perProducer * (perProducer + 1) / 2;
        System.out.println("consumed sum = " + total + ", expected = " + expected);
    }
}
```

Bounded buffer in Java, three producers and three consumers (measured) TERMINAL

```
$ java BoundedBuffer.java
consumed sum = 15000150000, expected = 15000150000
```

Readers and writers

Many threads may read a structure simultaneously, but a writer needs exclusive access. A read-write lock allows any number of concurrent readers or one writer. The policy question is fairness: reader preference can starve writers, writer preference can starve readers. `ReentrantReadWriteLock` in its default non-fair mode blocks a new reader when a writer is first in the queue, as the timeline shows for R3.

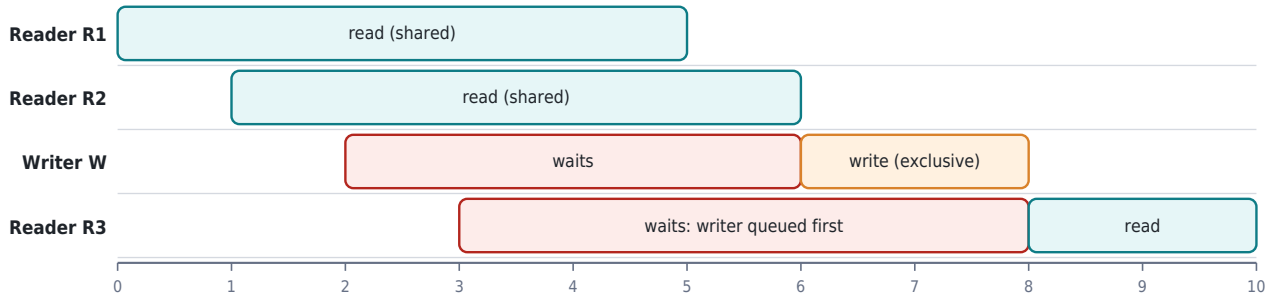


Figure 4.9. Readers R1 and R2 overlap; writer W waits until both leave; reader R3, arriving after W queued, waits behind W instead of overtaking it.

The program below has one writer that updates two fields `a` and `b` non-atomically (the invariant is `a == b`) and two readers that check the invariant. Without the lock readers observe the half-updated state constantly; with the read-write lock they never do.

ReadersWriters.java JAVA

```java
import java.util.concurrent.atomic.AtomicLong;
import java.util.concurrent.locks.ReentrantReadWriteLock;

public class ReadersWriters {
    static int a = 0;                           // invariant: a == b when no writer is active
    static int b = 0;
    static final ReentrantReadWriteLock RW = new ReentrantReadWriteLock();
    static final AtomicLong violations = new AtomicLong();
    static final AtomicLong reads = new AtomicLong();

    static void write(int v, boolean locked) {
        if (locked) RW.writeLock().lock();
        try { a = v; Thread.onSpinWait(); b = v; } finally { if (locked) RW.writeLock().unlock(); }
    }

    static void read(boolean locked) {
        if (locked) RW.readLock().lock();
        try {
            int x = a;
            Thread.onSpinWait();
            int y = b;
            reads.incrementAndGet();
            if (x != y) violations.incrementAndGet();
        } finally { if (locked) RW.readLock().unlock(); }
    }

    static void trial(boolean locked) throws Exception {
        violations.set(0); reads.set(0); a = 0; b = 0;
        long end = System.nanoTime() + 1_000_000_000L;
        Thread w = new Thread(() -> {
            int v = 0;
            while (System.nanoTime() < end) write(++v, locked);
        });
        Thread r1 = new Thread(() -> { while (System.nanoTime() < end) read(locked); });
        w.start(); r1.start();
        while (System.nanoTime() < end) read(locked);       // main thread is the second reader
        w.join(); r1.join();
        System.out.printf("locked=%-5s reads=%8d  inconsistent reads=%d%n",
                locked, reads.get(), violations.get());
    }

    public static void main(String[] args) throws Exception {
        trial(false);
        trial(true);
    }
}
```

Readers and writers, 1 second per trial (measured) TERMINAL

```
$ java ReadersWriters.java
locked=false reads= 8893636  inconsistent reads=1977213
locked=true  reads= 6338937  inconsistent reads=0
```

Dining philosophers

Five philosophers sit at a round table with one fork between each pair of neighbours. Each needs both adjacent forks to eat. If every philosopher takes the left fork first, all can hold one fork and wait forever for the other: the system is deadlocked. The problem models a set of threads that each need two locks.

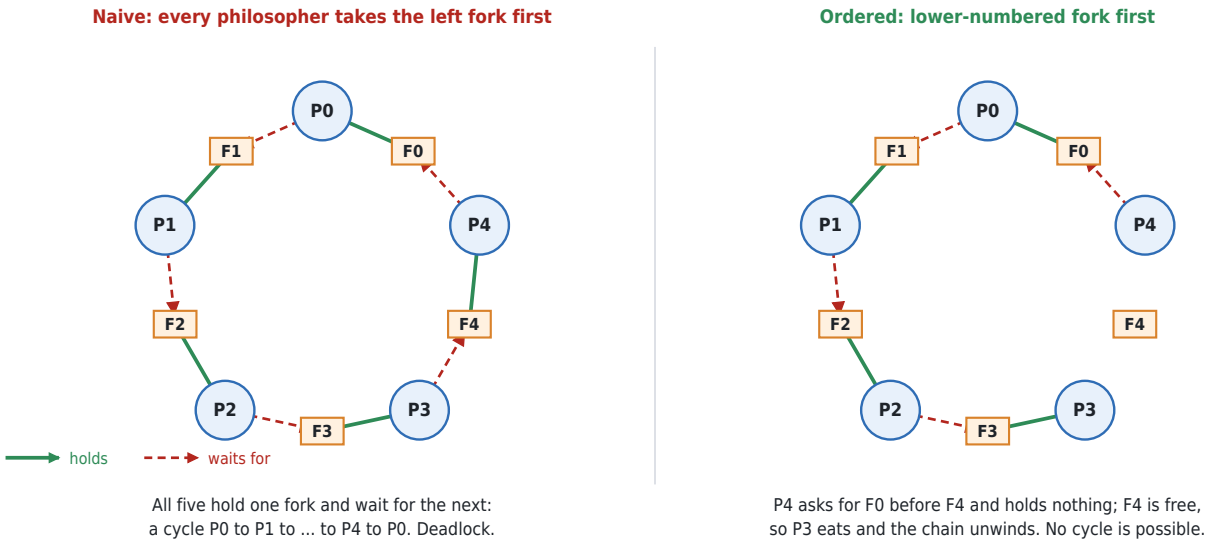


Figure 4.10. Left: each philosopher holds fork i and waits for fork i+1, forming a cycle. Right: numbering the forks and always taking the lower number first makes philosopher 4 request fork 0 before fork 4, which breaks the cycle.

The program runs the naive protocol first, with a 50 ms pause between the two pick-ups to make the deadlock certain, and uses `ThreadMXBean.findDeadlockedThreads()` to detect it. It then runs the ordered protocol for 100,000 meals per philosopher.

Philosophers.java JAVA

```
import java.lang.management.ManagementFactory;
import java.util.concurrent.locks.ReentrantLock;

public class Philosophers {
    static final int N = 5;
    static final ReentrantLock[] fork = new ReentrantLock[N];
    static final int[] meals = new int[N];

    static void eat(int id, boolean ordered, int rounds, boolean pause) {
        int left = id, right = (id + 1) % N;
        int first = left, second = right;
        if (ordered && left > right) { first = right; second = left; }   // global lock order
        for (int r = 0; r < rounds; r++) {
            fork[first].lock();
            try {
                if (pause) { try { Thread.sleep(50); } catch (InterruptedException e) { return; } }
                fork[second].lock();
                try { meals[id]++; } finally { fork[second].unlock(); }
            } finally { fork[first].unlock(); }
        }
    }

    static Thread[] start(boolean ordered, int rounds, boolean pause) {
        Thread[] t = new Thread[N];
        for (int i = 0; i < N; i++) {
            final int id = i;
            t[i] = new Thread(() -> eat(id, ordered, rounds, pause), "philosopher-" + i);
            t[i].setDaemon(true);
            t[i].start();
        }
        return t;
    }

    public static void main(String[] args) throws Exception {
        for (int i = 0; i < N; i++) fork[i] = new ReentrantLock();

        // 1. naive: everybody takes the left fork first, then pauses -> deadlock
        Thread[] naive = start(false, 1000, true);
        Thread.sleep(500);
        long[] dead = ManagementFactory.getThreadMXBean().findDeadlockedThreads();
        System.out.println("naive:   deadlocked threads = " + (dead == null ? 0 : dead.length));
        System.out.println("naive:   meals = " + java.util.Arrays.toString(meals));

        // 2. ordered: lower-numbered fork first -> no cycle possible
```

Philosophers.java (continued) JAVA

```
        java.util.Arrays.fill(meals, 0);
        for (int i = 0; i < N; i++) fork[i] = new ReentrantLock();
        Thread[] ok = start(true, 100_000, false);
        for (Thread t : ok) t.join();
        System.out.println("ordered: meals = " + java.util.Arrays.toString(meals));
    }
}
```

Dining philosophers (measured) TERMINAL

```
$ java Philosophers.java
naive:   deadlocked threads = 5
naive:   meals = [0, 0, 0, 0, 0]
ordered: meals = [100000, 100000, 100000, 100000, 100000]
```

The ordered solution is deadlock-free and every philosopher eats the same number of times in this run; it does not by itself guarantee fairness, which would require an additional mechanism such as a fair lock.

4.2.3 Worked trace

The following trace shows the futex protocol on a real mutex. The program creates three threads that each take the same mutex 200 times and sleep 500 microseconds while holding it, so the lock is contended on almost every acquisition.

futex2.c C

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

static long counter = 0;
static pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
static int n, hold_us;

static void *worker(void *arg) {
    (void)arg;
    for (int i = 0; i < n; i++) {
        pthread_mutex_lock(&lock);
        counter++;
        if (hold_us > 0) usleep(hold_us);   /* simulate work inside the critical section */
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

int main(int argc, char **argv) {
    int threads = atoi(argv[1]);
    n = atoi(argv[2]);
    hold_us = atoi(argv[3]);
    pthread_t t[8];
    for (int i = 0; i < threads; i++) pthread_create(&t[i], NULL, worker, NULL);
    for (int i = 0; i < threads; i++) pthread_join(t[i], NULL);
    printf("threads=%d counter=%ld\n", threads, counter);
    return 0;
}
```

First the uncontended case: one thread, 200,000 lock and unlock pairs. Then the contended case: three threads, 200 iterations each, summarised with `strace -c`.

Futex calls with and without contention (measured) TERMINAL

```
$ strace -f -c -e trace=futex ./futexdemo 1 200000
threads=1 counter=200000
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
  0.00    0.000000           0         1           futex
$ strace -f -c -e trace=futex,clock_nanosleep ./futex2 3 200 500
threads=3 counter=600
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 76.65    0.015095          13      1120           futex
 23.35    0.004598           7       600           clock_nanosleep
```

The single-threaded run performed 200,000 lock and unlock pairs with one futex call in total, which belongs to thread start-up. The contended run needed 1,120 futex calls for 600 critical sections, almost two per section: one `FUTEX_WAIT` by a loser and one `FUTEX_WAKE` by the owner. A shorter trace with two threads and three iterations makes the individual calls visible (thread numbers are the Linux thread ids, addresses abbreviated):

Individual futex calls of a contended mutex (typical output, trimmed) TERMINAL

```
$ strace -f -T -e trace=futex ./futex2 2 3 2000
1394  futex(0x562753449060, FUTEX_WAIT_PRIVATE, 2, NULL <unfinished ...>
1393  futex(0x562753449060, FUTEX_WAKE_PRIVATE, 1) = 1 <0.000006>
1394  futex(0x562753449060, FUTEX_WAIT_PRIVATE, 2, NULL <unfinished ...>
1393  futex(0x562753449060, FUTEX_WAKE_PRIVATE, 1) = 1 <0.000006>
1393  +++ exited with 0 +++
1394  futex(0x562753449060, FUTEX_WAKE_PRIVATE, 1) = 0 <0.000004>
```

Line	Meaning
1	Thread 1394 found the word nonzero, set it to 2 and sleeps: expected value 2, no timeout.
2	Thread 1393 unlocks: the word was 2, so it wakes one sleeper; the return value 1 reports one thread woken.
3-4	Thread 1394 acquires, then on the next iteration waits again; thread 1393 wakes it again.
5	Thread 1393 finishes its last iteration and exits.
6	Thread 1394 unlocks with state 2 left by its earlier contention; no sleeper remains, so the wake returns 0.

Reading the trace.

4.2.4 Hands-on

Cost of the fast path

Without tracing, one thread performing two million lock and unlock pairs of a `pthread_mutex_t` finished in 33 ms of user time, about 16 ns per pair, and no system time: the uncontended path never enters the kernel. Four threads performing the same total work took 112 ms elapsed and 63 ms of system time, spent mostly in futex calls. Contention converts a user-space operation into a kernel operation.

Spinlock against mutex

The program `spin.c` runs the same increment under a test-and-set spinlock and under a mutex and performs a fixed amount of busy work inside the critical section (an inner loop of 2,000 iterations). The machine has 2 CPUs.

spin.c C

```c
#include <pthread.h>
#include <stdatomic.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

static atomic_flag flag = ATOMIC_FLAG_INIT;
static pthread_mutex_t mu = PTHREAD_MUTEX_INITIALIZER;
static long counter;
static int use_spin, iters, work;
static volatile long sink;

static void *worker(void *arg) {
    (void)arg;
    for (int i = 0; i < iters; i++) {
        if (use_spin) {
            while (atomic_flag_test_and_set_explicit(&flag, memory_order_acquire)) { }   /* spin */
            counter++;
            for (int k = 0; k < work; k++) sink += k;   /* work inside the critical section */
            atomic_flag_clear_explicit(&flag, memory_order_release);
        } else {
            pthread_mutex_lock(&mu);
            counter++;
            for (int k = 0; k < work; k++) sink += k;
            pthread_mutex_unlock(&mu);
        }
    }
    return NULL;
}
```

spin.c (continued) C

```c

static double now(void) {
    struct timespec t;
    clock_gettime(CLOCK_MONOTONIC, &t);
    return t.tv_sec * 1e3 + t.tv_nsec / 1e6;
}

int main(int argc, char **argv) {
    int threads = atoi(argv[1]);
    iters = atoi(argv[2]);
    work = atoi(argv[3]);
    for (use_spin = 0; use_spin < 2; use_spin++) {
        pthread_t t[16];
        counter = 0;
        double t0 = now();
        for (int i = 0; i < threads; i++) pthread_create(&t[i], NULL, worker, NULL);
        for (int i = 0; i < threads; i++) pthread_join(t[i], NULL);
        printf("threads=%d %-8s counter=%ld  %6.0f ms\n", threads, use_spin ? "spinlock" : "mutex",
               counter, now() - t0);
    }
    return 0;
}
```

Spinlock against mutex on 2 CPUs (measured on a shared, loaded VM; typical) TERMINAL

```
$ for n in 2 4 8; do ./spin $n 20000 2000; done
threads=2 mutex    counter=40000     209 ms
threads=2 spinlock counter=40000     313 ms
threads=4 mutex    counter=80000     360 ms
threads=4 spinlock counter=80000     914 ms
threads=8 mutex    counter=160000     753 ms
threads=8 spinlock counter=160000    4229 ms
```

With as many threads as CPUs the two locks are comparable. With more threads than CPUs the spinlock degrades by a factor of 2.5 at four threads and 5.6 at eight: the kernel preempts a thread that holds the lock, and every other thread then spins through its whole time slice (several milliseconds) before the holder runs again. The mutex waiters sleep, so the CPU goes to the holder immediately. The lesson is general: user-space spinning is justified only with a guarantee that the holder is running.

4.2.5 Real-world incidents

Incident: 100 percent CPU from a spin-wait loop

Scenario. A Spring Boot batch component waits for a background initialisation task by polling a flag in a tight loop (`while (!ready) { }`), written to avoid the cost of a lock. The service is deployed to Kubernetes with a CPU limit of one core.

Symptoms. CPU utilisation sits at the limit within seconds of start-up and does not fall; request latency is high because the throttled container shares its single core between the spinning thread and the request threads. Heap and garbage collection are normal.

Diagnosis. The first question is which thread consumes the CPU. `top -H` lists threads instead of processes; the thread id in its first column is the Linux thread id, and `jcmd` prints the same number as `nid` in the thread dump (decimal on JDK 21; JDK 17 and earlier print it in hexadecimal, so convert with `printf '%x'`).

Locate the busy thread (reproduction with StateDemo, typical output) TERMINAL

```
$ top -H -b -n1 -p 9117 | sed -n 7,10p
  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
 9142 root      20   0 4154288  97156  34896 R  99.9   1.2   0:04.31 spinner
 9117 root      20   0 4154288  97156  34896 S   0.0   1.2   0:00.00 java
 9118 root      20   0 4154288  97156  34896 S   0.0   1.2   0:00.46 java
$ jcmd 9117 Thread.print | grep -A6 '"spinner"'
"spinner" #25 [9142] daemon prio=5 os_prio=0 cpu=4627.50ms elapsed=4.69s tid=0x... nid=9142 runnable
   java.lang.Thread.State: RUNNABLE
        at StateDemo.lambda$main$6(StateDemo.java:24)
```

The thread is RUNNABLE, in state `R` at 99.9 percent CPU, and the stack ends in application code that performs no call into the kernel. A thread that waits correctly (on a lock, a latch or a queue) would appear as `S` with 0 percent. The experiment below measures the difference directly.

SpinWait.java JAVA

```java
import java.lang.management.ManagementFactory;
import java.lang.management.ThreadMXBean;
import java.util.concurrent.CountDownLatch;

public class SpinWait {
    static volatile boolean ready = false;

    static long cpuMs(Thread t) {
        ThreadMXBean mx = ManagementFactory.getThreadMXBean();
        return mx.getThreadCpuTime(t.threadId()) / 1_000_000;
    }

    public static void main(String[] args) throws Exception {
        Thread spin = new Thread(() -> { while (!ready) { } }, "spin-waiter");
        spin.setDaemon(true);
        spin.start();

        CountDownLatch latch = new CountDownLatch(1);
        Thread block = new Thread(() -> {
            try { latch.await(); } catch (InterruptedException e) { }
        }, "latch-waiter");
        block.setDaemon(true);
        block.start();

        Thread.sleep(2000);
        System.out.println("after 2000 ms of waiting for an event that has not happened yet:");
        System.out.println("  spin-waiter  cpu = " + cpuMs(spin) + " ms");
        System.out.println("  latch-waiter cpu = " + cpuMs(block) + " ms");
        ready = true;
        latch.countDown();
    }
}
```

CPU time while waiting 2000 ms (measured) TERMINAL

```
$ java SpinWait.java
after 2000 ms of waiting for an event that has not happened yet:
  spin-waiter  cpu = 1378 ms
  latch-waiter cpu = 0 ms
```

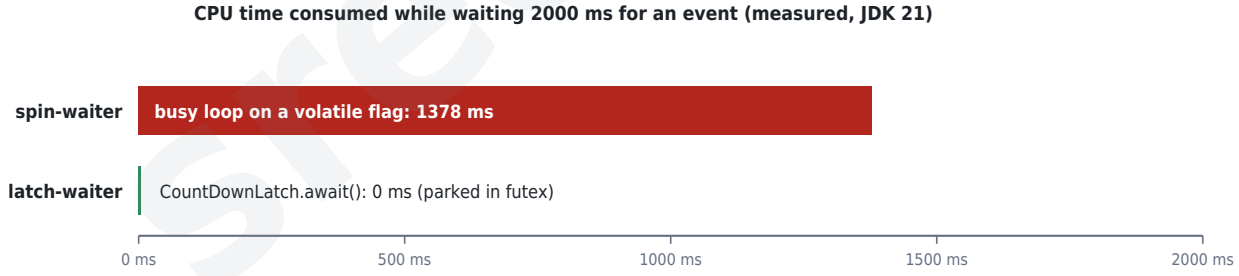


Figure 4.11. The spinning thread consumed 1378 ms of CPU in a 2000 ms wait (limited by competition for the two CPUs); the thread parked in a latch consumed none.

Root cause. Busy-waiting on a condition, which keeps a CPU occupied for a wait whose duration depends on an external event.

Fix. Immediate: restart does not cure it; raising the CPU limit only hides it. Permanent: replace the loop by a blocking construct: `CountDownLatch`, `CompletableFuture.get()`, a `BlockingQueue` or `Object.wait()` in a loop. Where a very short spin is justified, call `Thread.onSpinWait()` inside the loop and bound the number of iterations before parking.

Prevention. Alert on per-thread CPU (a single thread at 100 percent for minutes), review any `while` loop whose body is empty, and take a thread dump whenever CPU is high without matching traffic.

Incident: throughput falls after a deploy because a lock is held too long

Scenario. A release adds audit logging inside an existing `synchronized` method that updates a shared in-memory ledger. The log call formats a string and writes to a file, which is several orders of magnitude slower than the update it sits next to. Eight Tomcat threads call the method.

Symptoms. Throughput does not scale with load, p99 latency rises with concurrency, and CPU use remains below the machine capacity although requests queue. Adding replicas helps only linearly with cost.

Diagnosis. CPU below capacity with high latency means that threads wait rather than compute. A thread dump shows how they wait: one thread holds the monitor and does the slow work, the others sit in the entry list.

Sampling thread states of eight workers (measured, Contention.java x) — TERMINAL

```
$ java Contention.java x
sampling states of 8 workers:
  {RUNNABLE=1, BLOCKED=7}
```

In a dump, each blocked thread names the monitor it waits for, and the same address appears as "locked" in exactly one other thread, the owner. Here seven workers wait for `0x00000000a7eae7d0`, and `worker-5` owns it while executing the slow logging routine:

Waiter and owner in a thread dump (measured; tid and stack addresses trimmed) — TERMINAL

```
$ jcmd 13823 Thread.print
"worker-1" #20 [13845] prio=5 os_prio=0 cpu=421.97ms ... nid=13845 waiting for monitor entry
   java.lang.Thread.State: BLOCKED (on object monitor)
        at Contention.lambda$run$0(Contention.java:25)
        - waiting to lock <0x00000000a7eae7d0> (a java.lang.Object)
"worker-5" #24 [13849] prio=5 os_prio=0 cpu=395.16ms ... nid=13849 runnable
   java.lang.Thread.State: RUNNABLE
        at Contention.slowLog(Contention.java:12)
        at Contention.lambda$run$0(Contention.java:26)
        - locked <0x00000000a7eae7d0> (a java.lang.Object)
```

The benchmark below compares one lock held during the slow work ("wide") with the same work moved outside the lock ("narrow"). The wide lock does not just fail to scale; each hand-off from one blocked thread to the next costs a futex wake and a context switch, so throughput fell 17 percent below the single-thread rate with two threads and 25 percent with eight.

Contention.java — JAVA

```
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.locks.ReentrantLock;

public class Contention {
    static final Object LOCK = new Object();
    static long counter;
    static volatile long sink;

    static void slowLog() {                    // stands for formatting and writing a log line
        long x = 0;
        for (int i = 0; i < 20_000; i++) x += i ^ (x >>> 3);
        sink = x;
    }

    static long run(int threads, boolean wide, long millis) throws Exception {
        long[] done = new long[threads];
        List<Thread> ts = new ArrayList<>();
        long end = System.nanoTime() + millis * 1_000_000;
        for (int t = 0; t < threads; t++) {
            final int id = t;
            Thread th = new Thread(() -> {
                while (System.nanoTime() < end) {
                    if (wide) {
                        synchronized (LOCK) {                       // slow work inside the lock
                            slowLog();
                            counter++;
                        }
                    } else {
                        slowLog();                                  // slow work outside the lock
                        synchronized (LOCK) { counter++; }
                    }
                    done[id]++;
                }
            }, "worker-" + t);
            ts.add(th);
            th.start();
        }
        if (millis > 3000) {
            Thread.sleep(1500);
            System.out.println("sampling states of " + threads + " workers:");
            java.util.Map<Thread.State, Integer> m = new java.util.EnumMap<>(Thread.State.class);
```

Contention.java (continued) JAVA

```
            for (Thread th : ts) m.merge(th.getState(), 1, Integer::sum);
            System.out.println("  " + m);
        }
        for (Thread th : ts) th.join();
        long total = 0;
        for (long d : done) total += d;
        return total;
    }

    public static void main(String[] args) throws Exception {
        if (args.length > 0) { run(8, true, 4000); return; }
        run(2, false, 500);                                          // warm-up
        for (int threads : new int[]{1, 2, 8}) {
            long wide = run(threads, true, 2000);
            long narrow = run(threads, false, 2000);
            System.out.printf("threads=%d  wide lock: %6d ops   narrow lock: %6d ops%n",
                    threads, wide, narrow);
        }
    }
}
```

Operations in 2 seconds on 2 CPUs (measured, quiet machine) TERMINAL

```
$ java Contention.java
threads=1  wide lock: 106776 ops   narrow lock: 116766 ops
threads=2  wide lock:  89053 ops   narrow lock: 208982 ops
threads=8  wide lock:  80132 ops   narrow lock: 207592 ops
```

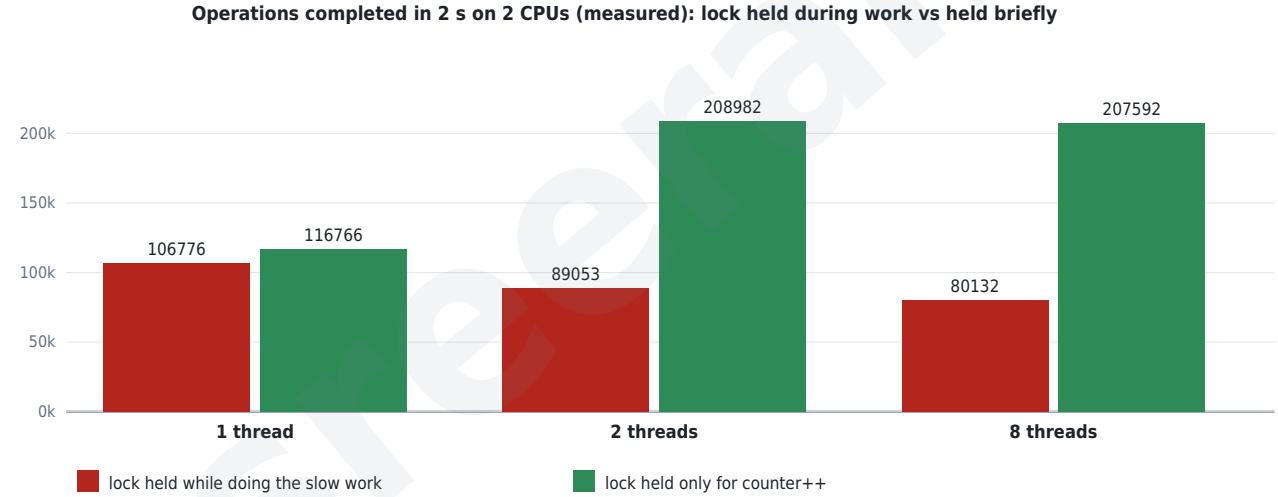


Figure 4.12. With the slow work outside the lock, two threads reach 1.8 times the single-thread throughput; with the lock held during the work, two and eight threads are slower than one (17 and 25 percent lower).

Root cause. The critical section contains work that does not need protection (formatting and I/O). The monitor serialises all threads for the duration of that work; contended hand-offs add kernel-level wake-ups, a pattern known as a lock convoy.

Fix. Immediate: roll back or disable the audit call, or move it to an asynchronous appender. Permanent: keep only the shared-state update inside the lock, copy what the log needs to a local variable, log after releasing the lock, and prefer a concurrent structure (`LongAdder`, `ConcurrentHashMap.merge`) that needs no external lock.

Prevention. Load-test with concurrency equal to production, record lock wait time (JFR event `jdk.JavaMonitorEnter`), and review for I/O or remote calls inside `synchronized` blocks and under `ReentrantLock`.

4.2.6 Questions to expect

1. **What is the difference between a mutex and a binary semaphore?** A mutex has an owner and must be unlocked by the thread that locked it; a binary semaphore has no owner and can be posted by any thread. Mutexes protect data, semaphores signal events.
2. **When is a spinlock appropriate?** When the critical section is very short, the holder cannot be preempted, and there are at least as many CPUs as contenders, as in kernel code. In user space with more threads than CPUs it collapses because the preempted holder keeps all spinners busy.

3. **Why must a condition variable wait be in a `while` loop?** A wake-up does not guarantee the predicate: another thread may consume the state first, and spurious wake-ups are permitted. The loop re-tests the predicate under the mutex.
4. **What is a lost wake-up and how is it prevented?** A signal sent between a waiter's test of the predicate and its sleep is lost. Changing the state and signalling under the same mutex, with `wait` atomically releasing that mutex, prevents it.
5. **What is a futex and why does it make mutexes fast?** A futex is a user-space integer plus a kernel wait queue. Uncontended lock and unlock are atomic instructions on the integer with no system call; the kernel is entered only to sleep (`FUTEX_WAIT`) or to wake (`FUTEX_WAKE`).
6. **How can the producer and consumer deadlock?** If a producer takes the buffer mutex before waiting on the `empty` semaphore, it can sleep holding the mutex while the consumer that would free a slot waits for that mutex.
7. **How is the dining-philosophers deadlock avoided?** By breaking one of the four deadlock conditions: ordering the forks globally (no circular wait), limiting the table to four philosophers with a counting semaphore (one of them can always obtain both forks), or using `tryLock` with back-off (preemption).
8. **What does `top -H` show that `top` does not?** One line per thread instead of per process, with the thread id that `jstack` and `jcmd` print as `nid`; it identifies the one thread that burns the CPU.
9. **Why can a wider lock lower throughput even below the single-thread rate?** Contended hand-offs involve futex wake-ups and context switches, and the waiters make no progress during the critical section, so extra threads add overhead without parallelism.

KEY POINTS

- A mutex puts waiters to sleep; a spinlock burns CPU and is safe only when the holder cannot be preempted; a semaphore counts and has no owner; a condition variable waits for a predicate, always tested in a loop under the mutex.
- On Linux, lock state lives in a user-space integer; `futex(FUTEX_WAIT)` and `futex(FUTEX_WAKE)` are the only kernel involvement, and uncontended locking costs about 16 ns with no system call.
- Producer-consumer, readers-writers and dining philosophers each demonstrate one rule: order semaphore waits before the mutex, choose a read-write fairness policy, and impose a global lock order.
- Spin-waiting shows as one thread at 100 percent CPU in `top -H` with `RUNNABLE` in the dump; blocking correctly shows 0 percent.
- Never perform I/O or slow work inside a critical section: it serialises all threads and converts contention into futex traffic.

4.3 Deadlock, Livelock, Starvation and Priority Inversion

A set of threads is deadlocked when each thread in the set waits for an event that only another thread in the set can cause. None can proceed, none releases what it holds, and the program stays in that state until it is restarted. A deadlocked server does not crash and does not log an error: its threads are alive and its process is healthy, but requests that reach the affected code never complete. This makes deadlock one of the costliest classes of production defect to detect.

This chapter states the four conditions that make deadlock possible, shows how to represent and detect it, derives the standard prevention and avoidance strategies (including the Banker's algorithm, verified by program), and treats its relatives: livelock, starvation and priority inversion.

4.3.1 Concept and mechanism

The four necessary conditions

Coffman, Elphick and Shoshani showed in 1971 that deadlock can occur only if four conditions hold simultaneously:

- **Mutual exclusion.** At least one resource is held in a non-shareable mode.
- **Hold and wait.** A thread holds at least one resource while waiting for another.
- **No preemption.** A resource can be released only voluntarily by its holder.
- **Circular wait.** A cycle of threads exists in which each waits for a resource held by the next.

The conditions are necessary. For single-instance resources such as locks, a circular wait is also sufficient. Every prevention technique removes one of the four.

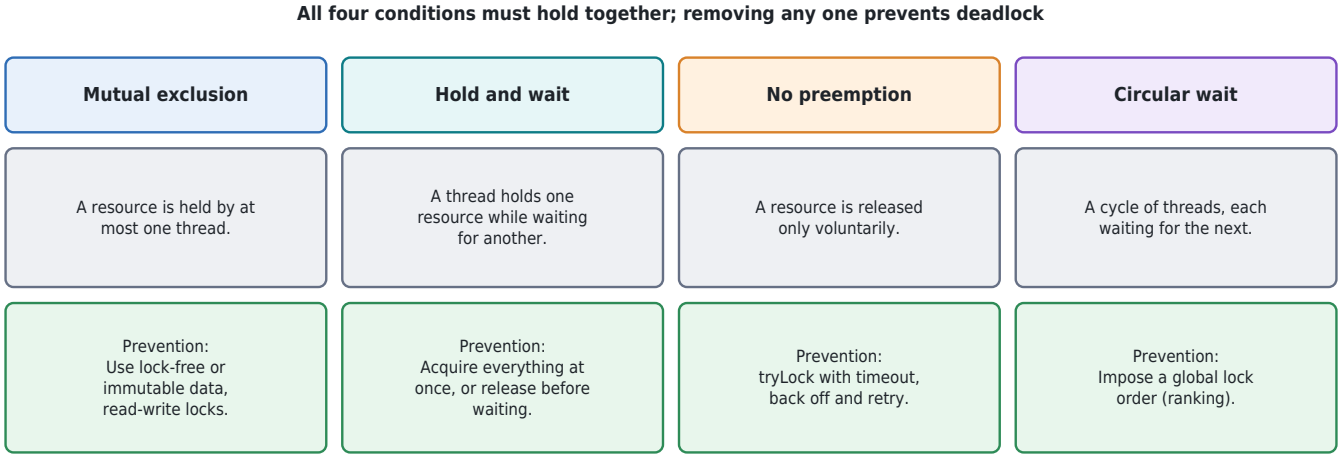


Figure 4.13. The four conditions with the prevention technique that removes each; in application code the cheapest is almost always a global lock order, which removes circular wait.

Resource-allocation graph

A **resource-allocation graph** has a vertex for every thread and every resource. An edge from a resource to a thread means that the thread holds the resource; an edge from a thread to a resource means that the thread is waiting for it. When every resource has a single instance (a lock), a cycle in the graph is both necessary and sufficient for deadlock. For resources with several instances (a pool, a semaphore with N permits) a cycle is necessary but not sufficient, and a reduction algorithm such as the Banker's safety check is needed.

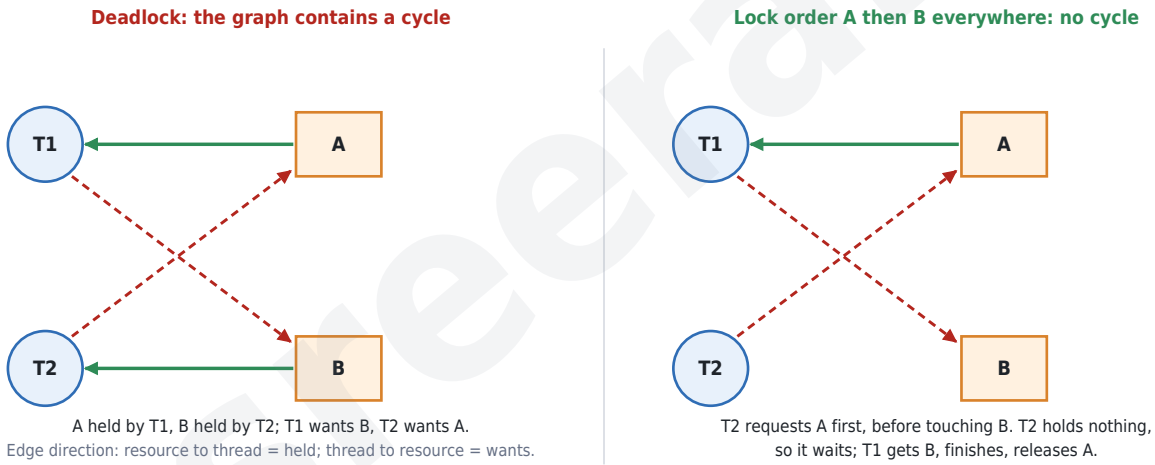


Figure 4.14. Left: T1 holds A and wants B while T2 holds B and wants A, a cycle. Right: when both threads request A first, T2 waits without holding anything, no cycle exists, and T1 finishes and releases A.

Strategies

Strategy	Idea	Cost
Prevention	Design so that one necessary condition can never hold (global lock order, acquire all at once)	Restricts program structure; may reduce concurrency
Avoidance	Grant a request only if the resulting state is safe (Banker's algorithm)	Needs maximum claims in advance; rarely practical for general locks
Detection and recovery	Allow deadlock, find the cycle periodically, abort or roll back a victim	Needs a detector and a safe way to abort (databases have both)
Ignore	Assume deadlock is rare and restart when it happens (the "ostrich algorithm")	Cheap; used by many general-purpose operating systems for user locks

Four ways to deal with deadlock.

Linux does not detect deadlocks among user-space mutexes. The kernel has a debugging facility, lockdep, that validates lock ordering of kernel locks, but a deadlock between two application threads is left to the application. The JVM supplies the detector for Java: `jstack`, `jcmd Thread.print` and `ThreadMXBean.findDeadlockedThreads()` find

cycles among `synchronized` monitors and `java.util.concurrent` locks, as shown in the hands-on section. Database engines run their own detectors on row and table locks.

Banker's algorithm

The **Banker's algorithm** (Dijkstra, generalised by Habermann) avoids deadlock for resources with several instances. It requires every process to declare its maximum claim in advance. The system keeps four structures for n processes and m resource types: `Available` (free instances of each type), `Max` (n by m maximum claims), `Allocation` (n by m currently held) and `Need = Max - Allocation`.

A state is **safe** if some order of the processes (a safe sequence) lets each one obtain its remaining need and finish, using what is free plus what earlier processes in the order release. The safety algorithm finds such an order:

1. Set `Work = Available` and mark every process unfinished.
2. Find an unfinished process `i` with `Need[i] <= Work`. If none exists, go to step 4.
3. Set `Work = Work + Allocation[i]`, mark `i` finished and return to step 2.
4. The state is safe if and only if every process is finished.

A request is granted only if it does not exceed the process's declared need, does not exceed `Available`, and the state that would result from granting it is safe. An unsafe state is not yet a deadlock; it is a state from which deadlock cannot be ruled out, so the request is delayed. The algorithm is of limited practical use because processes rarely know their maximum claims and the cost is O(m n squared) per request, but it is the standard exercise that makes the notion of safety precise.

Livelock and starvation

In a **livelock** threads are not blocked, yet make no progress because each keeps reacting to the others. The standard example is two threads that each `tryLock` a pair of locks, release the first when the second is unavailable, and retry immediately in lockstep. The remedy is randomised back-off, as used by Ethernet and by well-designed retry loops. In **starvation** a thread is runnable or eligible but is never selected, for example a low-priority thread under a stream of high-priority work, or a writer under a reader-preference lock. Starvation is solved by aging, fair queueing or bounded-waiting protocols; Java offers fair locks (`new ReentrantLock(true)`) at a throughput cost.

Priority inversion

Priority inversion occurs when a high-priority thread is delayed by a lower-priority one. The basic case is unavoidable and harmless: a high-priority thread waits for a lock briefly held by a low-priority thread. The pathological case arises when a medium-priority thread that does not use the lock preempts the low-priority holder. The high-priority thread now waits not for the holder's short critical section but for the medium thread to finish, which can take arbitrarily long.

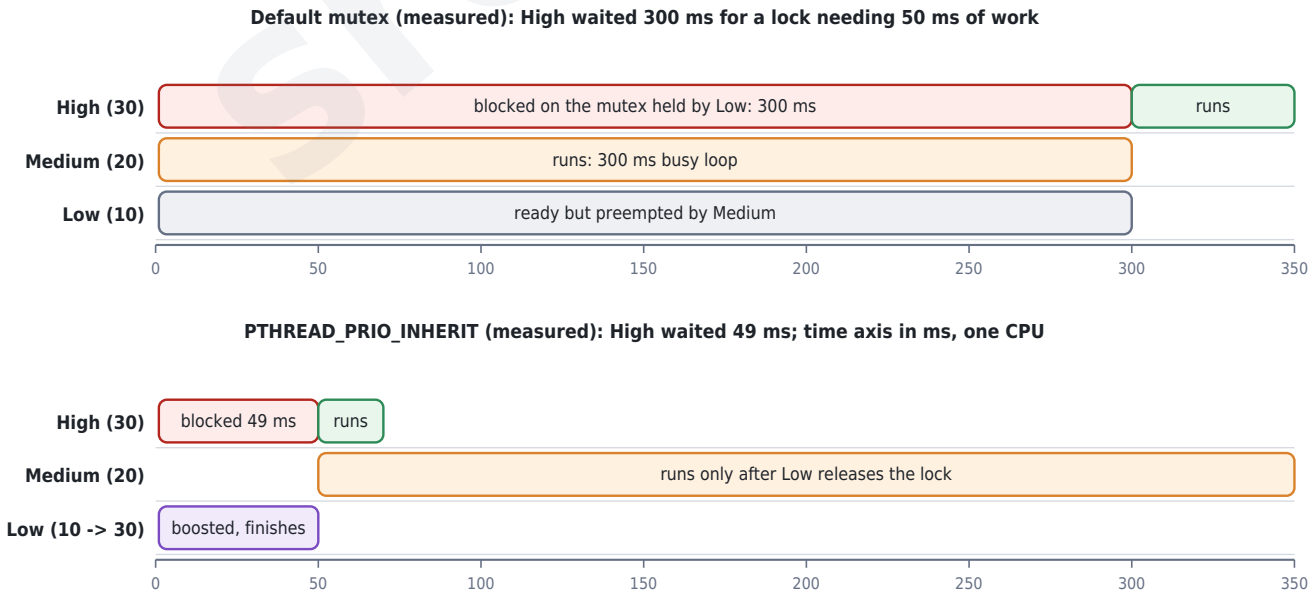


Figure 4.15. Measured on one CPU with real-time scheduling: with a default mutex High waits 300 ms because Medium runs while Low holds the lock; with priority inheritance Low runs at High's priority and High waits 49 ms.

Two protocols bound the delay. In **priority inheritance** the lock holder temporarily runs at the priority of the highest-priority waiter. In the **priority ceiling** protocol each lock has a ceiling equal to the highest priority of any thread that may use it, and the holder runs at that ceiling. POSIX exposes them as `PTHREAD_PRIO_INHERIT` and `PTHREAD_PRIO_PROTECT`; on Linux the former is implemented in the kernel by the priority-inheritance futex operations (`FUTEX_LOCK_PI`) and real-time mutexes.

The best-known case is the 1997 Mars Pathfinder mission. As reported by the flight-software team, a low-priority meteorological task held a mutex protecting the shared information bus, a high-priority bus-management task needed the same mutex, and a medium-priority communications task preempted the low-priority holder. A watchdog noticed that the bus-management task had not completed and reset the computer. The cause was found by reproducing the situation on a ground replica with tracing, and the system was repaired by enabling the priority-inheritance option of the VxWorks mutex, which had been available but disabled. The story is cited so often because it shows that the defect lay in a configuration flag and that tracing the real system was what exposed it.

> **NOTE**
> Priority inversion needs strict priorities. Under the default Linux time-sharing policy (EEVDF from kernel 6.6, CFS before) every runnable thread receives CPU time in proportion to its weight, so a medium-weight thread cannot starve a low-weight one indefinitely. The pathological case appears with real-time policies (`SCHED_FIFO`, `SCHED_RR`) and in embedded kernels. Java thread priorities have little or no effect on a default Linux JVM.

4.3.2 Worked trace

Safety check on a concrete state

The state below is the classical five-process, three-resource example: total instances A = 10, B = 5, C = 7. The program `Banker.java` implements the safety algorithm and the request algorithm.

Process	Allocation	Max	Need	Available at start
P0	0,1,0	7,5,3	7,4,3	3,3,2
P1	2,0,0	3,2,2	1,2,2	-
P2	3,0,2	9,0,2	6,0,0	-
P3	2,1,1	2,2,2	0,1,1	-
P4	0,0,2	4,3,3	4,3,1	-

State of the Banker's example (vectors are A,B,C).

The safety algorithm starts with `Work = (3,3,2)`. P0 does not fit (7,4,3 against 3,3,2); P1 fits (1,2,2), finishes and returns its allocation, giving `Work = (5,3,2)`. The scan continues with P3, P4, P0 and P2, as the figure shows. After P2 the work vector equals the system total (10,5,7), the check that every resource was returned.

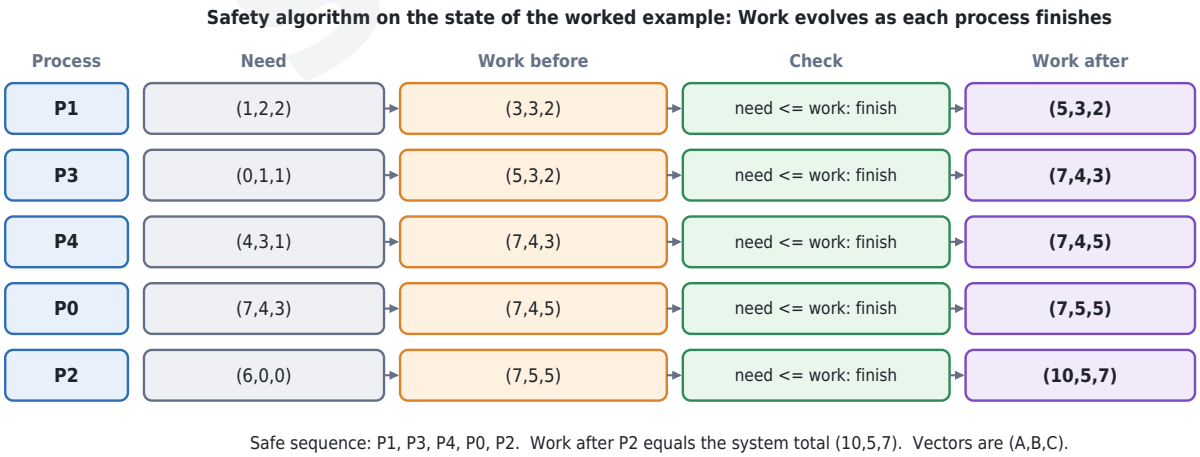


Figure 4.16. Each row applies the test Need <= Work and then adds the finished process's allocation to Work; the order P1, P3, P4, P0, P2 is one safe sequence.

Three requests

P1 requests (1,0,2). It does not exceed P1's need (1,2,2) or the available vector (3,3,2). Pretending to grant it yields `Available = (2,3,0)`, P1 allocation (3,0,2) and P1 need (0,2,0); the safety algorithm still finds the sequence P1, P3, P4, P0, P2, so the request is granted. P4 then requests (3,3,0): the first component exceeds `Available = (2,3,0)`, so P4 must wait. P0 requests (0,2,0): the resources exist, but granting leaves `Available = (2,1,0)` and no process has a need that fits, so the resulting state is unsafe and the request is delayed.

Banker.java JAVA

```java
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class Banker {
    static int[][] alloc = {{0,1,0},{2,0,0},{3,0,2},{2,1,1},{0,0,2}};
    static int[][] max   = {{7,5,3},{3,2,2},{9,0,2},{2,2,2},{4,3,3}};
    static int[] avail   = {3,3,2};

    static boolean le(int[] a, int[] b) {
        for (int i = 0; i < a.length; i++) if (a[i] > b[i]) return false;
        return true;
    }

    /** Returns a safe sequence, or null when the state is unsafe. */
    static List<Integer> safeSequence(int[][] alloc, int[][] need, int[] avail) {
        int n = alloc.length;
        int[] work = avail.clone();
        boolean[] done = new boolean[n];
        List<Integer> seq = new ArrayList<>();
        boolean progress = true;
        while (seq.size() < n && progress) {
            progress = false;
            for (int p = 0; p < n; p++) {
                if (!done[p] && le(need[p], work)) {
                    for (int r = 0; r < work.length; r++) work[r] += alloc[p][r];
                    done[p] = true;
                    seq.add(p);
                    progress = true;
                }
            }
        }
        return seq.size() == n ? seq : null;
    }

    static int[][] needOf() {
        int[][] need = new int[alloc.length][3];
        for (int p = 0; p < alloc.length; p++)
            for (int r = 0; r < 3; r++) need[p][r] = max[p][r] - alloc[p][r];
        return need;
    }

    static void request(int p, int[] req) {
        int[][] need = needOf();
        String head = "P" + p + " requests " + Arrays.toString(req) + ": ";
        if (!le(req, need[p])) {
            System.out.println(head + "error, exceeds declared maximum");
            return;
        }
        if (!le(req, avail)) { System.out.println(head + "must wait, not available"); return; }
        int[][] a2 = new int[alloc.length][];
        for (int i = 0; i < alloc.length; i++) a2[i] = alloc[i].clone();
        int[] v2 = avail.clone();
        for (int r = 0; r < 3; r++) { a2[p][r] += req[r]; v2[r] -= req[r]; need[p][r] -= req[r]; }
        List<Integer> s = safeSequence(a2, need, v2);
        if (s == null) { System.out.println(head + "denied, resulting state is unsafe"); return; }
        alloc = a2; avail = v2;
        System.out.println(head + "granted, safe sequence " + s);
    }

    public static void main(String[] args) {
        System.out.println("need = " + Arrays.deepToString(needOf()));
        System.out.println("initial safe sequence: " + safeSequence(alloc, needOf(), avail));
        request(1, new int[]{1, 0, 2});
```

sreeram

Banker.java (continued) JAVA

```
        request(4, new int[]{3, 3, 0});
        request(0, new int[]{0, 2, 0});
    }
}
```

Banker's algorithm (measured) TERMINAL

```
$ java Banker.java
need = [[7, 4, 3], [1, 2, 2], [6, 0, 0], [0, 1, 1], [4, 3, 1]]
initial safe sequence: [1, 3, 4, 0, 2]
P1 requests [1, 0, 2]: granted, safe sequence [1, 3, 4, 0, 2]
P4 requests [3, 3, 0]: must wait, not available
P0 requests [0, 2, 0]: denied, resulting state is unsafe
```

4.3.3 Hands-on

Reproducing a deadlock and reading the dump

The next program is the classical transfer defect: `transfer(from, to)` locks `from` and then `to`. Two threads that transfer in opposite directions acquire the two accounts in opposite orders. The 100 ms sleep stands for slow work (a remote call, a log write) between the two acquisitions and makes the interleaving certain.

TransferDeadlock.java JAVA

```
public class TransferDeadlock {
    static class Account {
        final String id;
        long balance = 1000;
        Account(String id) { this.id = id; }
    }

    static void transfer(Account from, Account to, long amount) throws InterruptedException {
        synchronized (from) {
            Thread.sleep(100);          // widens the window, as slow I/O would in production
            synchronized (to) {
                from.balance -= amount;
                to.balance += amount;
            }
        }
    }

    static void run(Account from, Account to, long amount) {
        try {
            transfer(from, to, amount);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }

    public static void main(String[] args) throws Exception {
        Account a = new Account("A"), b = new Account("B");
        Thread t1 = new Thread(() -> run(a, b, 10), "transfer-A-to-B");
        Thread t2 = new Thread(() -> run(b, a, 20), "transfer-B-to-A");
        t1.start(); t2.start();
        t1.join(); t2.join();               // never returns
        System.out.println("done");
    }
}
```

The program never prints `done`. A process in this state shows no CPU use; both threads are asleep in the kernel on a futex, which `ps -L` confirms by its wait-channel column:

The deadlocked threads at kernel level (measured) TERMINAL

```
$ ps -L -o tid,stat,wchan:20,comm -p 14168 | grep -E "TID|transfer"
  TID STAT WCHAN                COMMAND
14187 Sl   futex_do_wait        transfer-A-to-B
14188 Sl   futex_do_wait        transfer-B-to-A
```

The JVM detector prints the cycle at the end of the thread dump. The section lists, for each thread, the monitor it waits for and its owner, and then the stacks:

sreeram

Deadlock report from jcmd Thread.print (JDK 21.0.12, measured) TERMINAL

```
$ jcmd 14168 Thread.print
Found one Java-level deadlock:
=============================
"transfer-A-to-B":
  waiting to lock monitor 0x...a0001f70 (object 0x00000000a7b27130, a TransferDeadlock$Account),
  which is held by "transfer-B-to-A"

"transfer-B-to-A":
  waiting to lock monitor 0x...9c0015b0 (object 0x00000000a7b27118, a TransferDeadlock$Account),
  which is held by "transfer-A-to-B"

Java stack information for the threads listed above:
===================================================
"transfer-A-to-B":
        at TransferDeadlock.transfer(TransferDeadlock.java:12)
        - waiting to lock <0x00000000a7b27130> (a TransferDeadlock$Account)
        - locked <0x00000000a7b27118> (a TransferDeadlock$Account)
        at TransferDeadlock.run(TransferDeadlock.java:20)
"transfer-B-to-A":
        at TransferDeadlock.transfer(TransferDeadlock.java:12)
        - waiting to lock <0x00000000a7b27118> (a TransferDeadlock$Account)
        - locked <0x00000000a7b27130> (a TransferDeadlock$Account)
        at TransferDeadlock.run(TransferDeadlock.java:20)

Found 1 deadlock.
```

Time	transfer-A-to-B	transfer-B-to-A	Resulting graph
0 ms	locks Account A (...a7b27118)	locks Account B (...a7b27130)	A held by T1, B held by T2
0-100 ms	sleeps holding A	sleeps holding B	no waits yet
100 ms	requests B: blocked	requests A: blocked	T1 waits B, T2 waits A: cycle
after	state BLOCKED, futex wait	state BLOCKED, futex wait	permanent

Timeline of the reproduction (the cycle forms at about 100 ms).

Two repairs

The first repair removes circular wait by acquiring the two locks in a global order, here the account identifier. The second removes the unbounded wait by using `tryLock` with a timeout, which lets the caller fail, release what it holds and report an error. The listing contains both and a small test: the ordered version completes 200,000 opposing transfers, and the timed version fails fast when a lock is unavailable instead of hanging.

OrderedTransfer.java JAVA

```java
import java.util.concurrent.TimeUnit;
import java.util.concurrent.locks.ReentrantLock;

public class OrderedTransfer {
    static class Account {
        final long id;
        final ReentrantLock lock = new ReentrantLock();
        long balance = 1000;
        Account(long id) { this.id = id; }
    }

    /** Fix 1: acquire locks in a global order (by account id). */
    static void transferOrdered(Account from, Account to, long amount) {
        Account first = from.id < to.id ? from : to;
        Account second = first == from ? to : from;
        first.lock.lock();
        try {
            second.lock.lock();
            try { from.balance -= amount; to.balance += amount; } finally { second.lock.unlock(); }
        } finally { first.lock.unlock(); }
    }

    /** Fix 2: bounded wait; give up instead of waiting forever. */
    static boolean transferTimed(Account from, Account to, long amount)
            throws InterruptedException {
        if (!from.lock.tryLock(100, TimeUnit.MILLISECONDS)) return false;
        try {
```

OrderedTransfer.java (continued) JAVA

```
            if (!to.lock.tryLock(100, TimeUnit.MILLISECONDS)) return false;
            try {
                from.balance -= amount;
                to.balance += amount;
                return true;
            } finally {
                to.lock.unlock();
            }
        } finally { from.lock.unlock(); }
    }

    public static void main(String[] args) throws Exception {
        Account a = new Account(1), b = new Account(2);
        Thread t1 = new Thread(() -> {
            for (int i = 0; i < 100_000; i++) transferOrdered(a, b, 1);
        });
        Thread t2 = new Thread(() -> {
            for (int i = 0; i < 100_000; i++) transferOrdered(b, a, 2);
        });
        t1.start(); t2.start(); t1.join(); t2.join();
        System.out.println("ordered: A=" + a.balance + " B=" + b.balance
                + " total=" + (a.balance + b.balance));

        a.lock.lock();                                  // simulate a stuck holder on this thread
        Thread t3 = new Thread(() -> {
            try { System.out.println("timed transfer succeeded = " + transferTimed(b, a, 5)); }
            catch (InterruptedException e) { }
        });
        t3.start(); t3.join();
        a.lock.unlock();
    }
}
```

Both repairs (measured) TERMINAL

```
$ java OrderedTransfer.java
ordered: A=101000 B=-99000 total=2000
timed transfer succeeded = false
```

The balances satisfy conservation (the total stays 2000). The ordered version is the permanent fix because it makes deadlock impossible; the timed version bounds the damage but needs a retry or failure policy and, used alone, can cause livelock when many threads retry in step.

Priority inversion on Linux

The program `inv.c` creates three `SCHED_FIFO` threads on one CPU (the process is pinned with `taskset -c 1`): Low (priority 10) takes a mutex and works for 50 ms, High (priority 30) then blocks on the same mutex, and Medium (priority 20) starts a 300 ms busy loop. Real-time policies require privileges, which the build machine provides. The kernel's real-time throttling (`sched_rt_runtime_us`, 950000 per second by default) limits the effect on the rest of the system, and the run is bounded to well under a second.

inv.c C

```
#define _GNU_SOURCE
#include <pthread.h>
#include <sched.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <unistd.h>

static pthread_mutex_t m;
static volatile int l_has_lock = 0;
static double h_wait_ms;

static double now_ms(void) {
    struct timespec t;
    clock_gettime(CLOCK_MONOTONIC, &t);
    return t.tv_sec * 1e3 + t.tv_nsec / 1e6;
}

/* busy loop on the wall clock; the process is pinned to one CPU with taskset */
static void burn_ms(double ms) {
```

inv.c (continued) C

```c
    double end = now_ms() + ms;
    while (now_ms() < end) { }
}

static void *low(void *a)  { (void)a; pthread_mutex_lock(&m); l_has_lock = 1; burn_ms(50);
                             pthread_mutex_unlock(&m); return NULL; }
static void *med(void *a)  { (void)a; burn_ms(300); return NULL; }
static void *high(void *a) { (void)a; double t0 = now_ms(); pthread_mutex_lock(&m);
                             h_wait_ms = now_ms() - t0; pthread_mutex_unlock(&m); return NULL; }

static pthread_t spawn(void *(*fn)(void *), int prio) {
    pthread_attr_t at;
    struct sched_param sp = { .sched_priority = prio };
    pthread_t t;
    pthread_attr_init(&at);
    pthread_attr_setinheritsched(&at, PTHREAD_EXPLICIT_SCHED);
    pthread_attr_setschedpolicy(&at, SCHED_FIFO);
    pthread_attr_setschedparam(&at, &sp);
    int rc = pthread_create(&t, &at, fn, NULL);
    if (rc) { fprintf(stderr, "pthread_create: %d\n", rc); exit(1); }
    return t;
}

int main(int argc, char **argv) {
    int inherit = argc > 1;
    struct sched_param me = { .sched_priority = 40 };
    if (sched_setscheduler(0, SCHED_FIFO, &me)) { perror("sched_setscheduler"); return 1; }
    pthread_mutexattr_t ma;
    pthread_mutexattr_init(&ma);
    if (inherit) pthread_mutexattr_setprotocol(&ma, PTHREAD_PRIO_INHERIT);
    pthread_mutex_init(&m, &ma);

    pthread_t l = spawn(low, 10);
    while (!l_has_lock) usleep(1000);       /* main (prio 40) sleeps; L runs and takes the lock */
    pthread_t h = spawn(high, 30);          /* H preempts L, then blocks on the mutex */
    pthread_t md = spawn(med, 20);          /* M becomes runnable and outranks L */
    pthread_join(h, NULL); pthread_join(md, NULL); pthread_join(l, NULL);
    printf("%s: high-priority thread waited %.0f ms for a lock held for 50 ms of work\n",
           inherit ? "PTHREAD_PRIO_INHERIT" : "default mutex       ", h_wait_ms);
    return 0;
}
```

Priority inversion and its cure (measured) TERMINAL

```
$ gcc -O1 -pthread -o inv inv.c
$ taskset -c 1 ./inv
default mutex       : high-priority thread waited 300 ms for a lock held for 50 ms of work
$ taskset -c 1 ./inv inherit
PTHREAD_PRIO_INHERIT: high-priority thread waited 49 ms for a lock held for 50 ms of work
```

The default run shows 300 ms rather than 350 ms because the 50 ms of Low's work is measured on the wall clock and had already elapsed when Medium finished; the point stands: High waited for Medium's entire run. With `PTHREAD_PRIO_INHERIT`, Low is boosted to priority 30 when High blocks, Medium cannot preempt it, and High waits only for the remainder of Low's critical section.

4.3.4 Real-world incidents

Incident: a Java deadlock in a payment service

Scenario. A Spring Boot payment service exposes `POST /transfers`. The service method locks the source account object and then the destination account object (an in-memory lock used to serialise updates before a batched write). Two customers send money to each other at the same moment.

Symptoms. Requests to `/transfers` begin to time out after a deploy; the load balancer marks the instance unhealthy only after its readiness probe, which uses a different code path, also times out on the exhausted thread pool. CPU is at 1 percent, the heap is stable and the log contains nothing at ERROR level. Each timed-out request leaves one more Tomcat worker blocked.

Diagnosis. In order: (1) `top -H -p PID` shows no busy thread, so the problem is waiting, not computing. (2) `jcmd PID Thread.print > dump.txt` captures the threads; the file ends with `Found one Java-level deadlock`. (3) The report

names two threads, the two lock addresses and the owner of each; the source lines in the stacks (`TransferDeadlock.java:12`) point to the second `synchronized`. (4) Every worker that later entered `transfer` is also BLOCKED on one of the same two accounts, which explains the growth. The reproduction in this chapter shows exactly this output; the report in the dump leaves nothing to interpret.

Root cause. Circular wait: the lock order depends on the argument order, so opposite transfers acquire the same pair of locks in opposite orders, while the hold-and-wait, mutual-exclusion and no-preemption conditions are satisfied by `synchronized`.

Fix. Immediate: restart the instance (a deadlocked JVM never recovers; take the dump first, since it is the evidence). Permanent: acquire the locks in a canonical order (for example by account id), as in `transferOrdered`; or avoid the two-lock design by moving the arbitration to the database with one transaction and `SELECT ... FOR UPDATE` in id order. Add a timeout on lock acquisition where an order cannot be imposed.

Prevention. Document the lock hierarchy; add a test that runs opposite transfers concurrently with a timeout; run a watchdog that calls `ThreadMXBean.findDeadlockedThreads()` every few seconds and reports through the metrics system (a non-null result raises an alert); make the readiness probe exercise the same thread pool as real requests, so that an exhausted pool takes the instance out of rotation quickly.

Incident: latency spikes from priority inversion

Scenario. A real-time audio gateway (a native service running on a Linux host with `SCHED_FIFO` threads) uses a high-priority thread to deliver samples every 5 ms, a medium-priority thread for packet processing and a low-priority housekeeping thread that updates a shared configuration structure under a mutex also read by the delivery thread. Occasionally the delivery thread misses its deadline by hundreds of milliseconds.

Symptoms. Audible gaps without any CPU saturation; the gaps correlate with bursts of packet processing; a watchdog, if present, restarts the service.

Diagnosis. Latency tracing (`ftrace` or `trace-cmd` with the scheduler events) is the right instrument. It shows the delivery thread blocked in `futex_wait` while the medium-priority thread runs continuously on the CPU where the housekeeping thread is runnable but not running. In a system without such tools, the reproduction `inv.c` shows the signature: the waiting time of the high-priority thread equals the run time of the medium-priority thread, not the length of the critical section.

Root cause. Unbounded priority inversion: an ordinary mutex shared by threads of different real-time priorities.

Fix. Immediate: lower the priority of the medium thread or give the housekeeping thread the delivery thread's priority. Permanent: create the mutex with `pthread_mutexattr_setprotocol(PTHREAD_PRIO_INHERIT)`, as in the second run of `inv.c`, and keep critical sections short.

Prevention. Review every mutex shared across real-time priorities; test under load with the medium-priority thread saturated; expose lock wait time as a metric.

Incident: an application lock and a database transaction deadlock each other

Scenario. A service updates stock inside a database transaction and, within the same transaction, calls a method synchronised on a cache object to refresh the cached stock. Another code path takes the cache lock first and then writes to the database. Neither lock manager sees the other.

Symptoms. Occasional requests hang until the database reports a lock-wait timeout (`innodb_lock_wait_timeout` defaults to 50 seconds in MySQL; PostgreSQL's `lock_timeout` is off by default and a statement waits indefinitely). The database deadlock detector reports nothing, because from its point of view one transaction is waiting for a lock held by a transaction that is merely slow. `jcmd Thread.print` reports no deadlock either, because the JVM detector sees only one side.

Diagnosis. The thread dump shows one thread BLOCKED on the cache monitor and another RUNNABLE or WAITING inside the JDBC driver (a socket read) while holding that monitor. The database shows a session waiting for a lock and a second session idle in transaction. Matching the two by connection, or by thread name in the log, gives the cycle that crosses both systems. The reproduction below uses SQLite: one thread holds the database write lock and wants the application lock, another holds the application lock and wants the write lock.

mixed.py PYTHON

```python
import os, sqlite3, threading, time

DB = "/tmp/osw_o4/mixed.db"
if os.path.exists(DB):
    os.remove(DB)
```

mixed.py (continued) PYTHON

```python
setup = sqlite3.connect(DB)
setup.execute("CREATE TABLE stock (sku TEXT PRIMARY KEY, qty INTEGER)")
setup.execute("INSERT INTO stock VALUES ('book', 10)")
setup.commit()
setup.close()

# stands for a synchronized block or ReentrantLock in the service
app_lock = threading.Lock()
t0 = time.time()

def log(who, msg):
    print("%5.2fs  %-8s %s" % (time.time() - t0, who, msg), flush=True)

def tx_then_lock():
    c = sqlite3.connect(DB, timeout=3, isolation_level=None)
    c.execute("BEGIN IMMEDIATE")                       # takes the database write lock
    log("tx-first", "database write lock acquired")
    time.sleep(0.3)
    log("tx-first", "waiting for the application lock")
    with app_lock:                                     # blocks: the other thread holds it
        c.execute("UPDATE stock SET qty = qty - 1")
    c.execute("COMMIT")

def lock_then_tx():
    c = sqlite3.connect(DB, timeout=3, isolation_level=None)
    with app_lock:
        log("lock-first", "application lock acquired")
        time.sleep(0.3)
        log("lock-first", "waiting for the database write lock")
        try:
            c.execute("BEGIN IMMEDIATE")               # blocks inside SQLite for up to 3 s
        except sqlite3.OperationalError as e:
            log("lock-first", "gave up: " + str(e))

a = threading.Thread(target=tx_then_lock, daemon=True)
b = threading.Thread(target=lock_then_tx, daemon=True)
a.start(); time.sleep(0.05); b.start()
b.join()
log("main", "tx-first still alive after the timeout: %s" % a.is_alive())
```

A cycle across two lock managers (measured) TERMINAL

```
$ python3 mixed.py
 0.00s  tx-first database write lock acquired
 0.05s  lock-first application lock acquired
 0.30s  tx-first waiting for the application lock
 0.35s  lock-first waiting for the database write lock
 3.36s  lock-first gave up: database is locked
 3.36s  main     tx-first still alive after the timeout: True
```

The only thing that ends the wait is a timeout: the database gave up after its 3 second busy timeout. The thread that waits for the application lock has no timeout at all and is still blocked.

A deadlock that spans two lock managers: neither detector sees the whole cycle

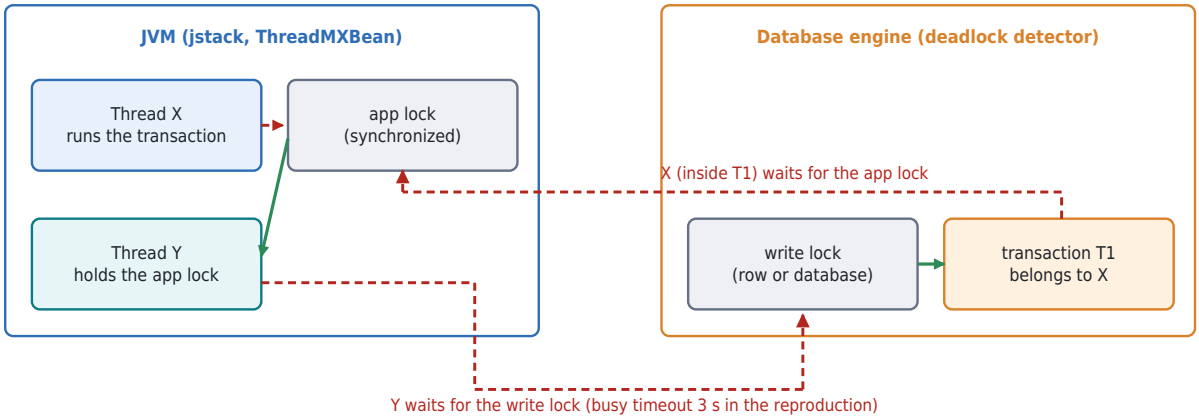


Figure 4.17. Neither the JVM detector nor the database detector sees the complete cycle, which passes through an application lock and a database lock; only timeouts break it.

Root cause. Two lock hierarchies (application locks and database locks) acquired in opposite orders, with no detector spanning both.

Fix. Immediate: kill the stuck database sessions (`KILL`, `pg_terminate_backend`) and restart the blocked instance. Permanent: never hold an application lock while waiting for the database, and never call code that takes an application lock from inside a transaction; derive one global order that includes the database (application lock first, then transaction), or remove the application lock in favour of optimistic concurrency (a version column) and database constraints. Set explicit timeouts at both layers (`lock_timeout`, `innodb_lock_wait_timeout`, `tryLock(timeout)`).

Prevention. Alert on sessions idle in transaction for more than a few seconds, on lock waits in the database, and on threads BLOCKED for more than a threshold; set the JDBC and Hikari timeouts below the request timeout so a stuck request fails and releases its resources.

4.3.5 Questions to expect

1. **State the four necessary conditions for deadlock.** Mutual exclusion, hold and wait, no preemption and circular wait. All four must hold; removing any one prevents deadlock.
2. **How do you prevent deadlock among application locks?** Impose a global order on the locks and always acquire in that order (removes circular wait); alternatively acquire all locks atomically, or use `tryLock` with a timeout and back off (removes hold-and-wait or no-preemption).
3. **What is the difference between deadlock, livelock and starvation?** In deadlock threads are blocked forever in a cycle. In livelock they keep running and reacting but make no progress. In starvation a thread is eligible but is never chosen.
4. **Explain a safe state in the Banker's algorithm.** A state is safe if some ordering of the processes lets each obtain its maximum need and finish using the free resources plus those released by earlier processes. An unsafe state may not deadlock, but deadlock cannot be excluded.
5. **How is deadlock detected in a running JVM?** `jcmd PID Thread.print` or `jstack` print "Found one Java-level deadlock" with the cycle; programmatically `ThreadMXBean.findDeadlockedThreads()` covers monitors and ownable synchronizers such as `ReentrantLock`.
6. **Can the JVM detector miss a deadlock?** Yes. It does not see threads parked on non-ownable constructs (futures, latches, semaphore waiters), starvation of a thread pool, or cycles that include database or remote locks.
7. **What is priority inversion and how is it solved?** A high-priority thread waits for a lock held by a low-priority thread that a medium-priority thread keeps preempting. Priority inheritance (or ceiling) raises the holder's priority; on Linux this is `PTHREAD_PRIO_INHERIT`.
8. **What happened on Mars Pathfinder?** A low-priority task held the mutex of the information bus, a medium-priority task preempted it, the high-priority bus task missed its deadline and a watchdog reset the computer. Enabling priority inheritance on the mutex fixed it.
9. **How do databases handle deadlocks?** They keep a waits-for graph or use a timeout, choose a victim, roll back its transaction and return an error (SQLSTATE 40P01 in PostgreSQL, error 1213 in MySQL); the application must retry the transaction.

10. **Why is the ordered-lock fix better than a timeout?** Ordering makes the cycle impossible, so no thread ever fails. A timeout only bounds the wait, turns a hang into an error that must be handled, and can cause livelock if retries synchronise.

KEY POINTS

- Deadlock needs mutual exclusion, hold and wait, no preemption and circular wait; the practical cure is a global lock order, which removes the cycle.
- A resource-allocation graph with a cycle on single-instance locks is deadlock; the JVM reports it in thread dumps as "Found one Java-level deadlock" with owners and stack lines.
- The Banker's algorithm decides whether a request leaves the system in a safe state; the worked example grants P1's request and refuses P0's.
- Priority inversion needs strict priorities; measured, a high-priority thread waited 300 ms for a 50 ms lock until `PTHREAD_PRIO_INHERIT` reduced it to 49 ms.
- A cycle that spans application locks and database locks is invisible to both detectors; set timeouts at every layer and never hold an application lock across a database call.

4.4 Locks in Java and Their Mapping to the Operating System

Java programs never call `futex` directly, yet every contended `synchronized` block, every `ReentrantLock`, every blocking queue and every `Thread.sleep` ends in the kernel services described in the previous chapters. Understanding the layers explains what a thread dump means, why a thread that waits shows as `S` in `ps`, why a `RUNNABLE` thread may use no CPU at all, and why virtual threads can be blocked by `synchronized` on JDK 21. All traces in this chapter come from OpenJDK 21.0.12 on Linux 6.18.

4.4.1 Concept and mechanism

synchronized and the monitor

Every Java object can serve as a **monitor**: a lock combined with a wait set. The compiler translates a `synchronized` block into the bytecode instructions `monitorenter` and `monitorexit`; a `synchronized` method instead carries the flag `ACC_SYNCHRONIZED` and the JVM acquires the monitor on entry and releases it on exit.

Bytecode of a synchronized block (javap -c -p Sync) TERMINAL

```
$ javap -c -p Sync
  void inc();
    Code:
       0: aload_0
       1: dup
       2: astore_1
       3: monitorenter
       4: aload_0
       5: dup
       6: getfield      #7                  // Field count:I
       9: iconst_1
      10: iadd
      11: putfield      #7                  // Field count:I
      14: aload_1
      15: monitorexit
      16: goto          24
      19: astore_2
      20: aload_1
      21: monitorexit
      22: aload_2
      23: athrow
      24: return
    Exception table:
       from    to  target type
           4    16    19   any
          19    22    19   any
```

The second `monitorexit` (offset 21) is on the exception path: the exception table covers the body so that the monitor is released even when an exception is thrown. Monitors are re-entrant (the owner can enter again and an entry count is kept), and exit establishes a happens-before edge with the next entry of the same monitor.

HotSpot implements the monitor in stages so that the common cases are cheap. The details differ between JDK releases; the following description is the stable outline.

- **Uncontended**: the lock state is recorded in the object header (the mark word) with a compare-and-swap; the owner holds a record on its stack (a thin or lightweight lock). No system call occurs. Biased locking, which removed even the CAS for single-thread use, was disabled by default in JDK 15 and has since been removed.
- **Contended**: when a second thread arrives, the monitor is **inflated** into an `ObjectMonitor` structure in native memory, which holds the owner, an entry list of threads waiting to acquire it and a wait set of threads that called `wait()`. A contender first spins briefly, in the hope that the owner releases soon, and then parks.
- **Deflation**: an idle inflated monitor is deflated back by a background thread (visible in dumps as `Monitor Deflation Thread`).

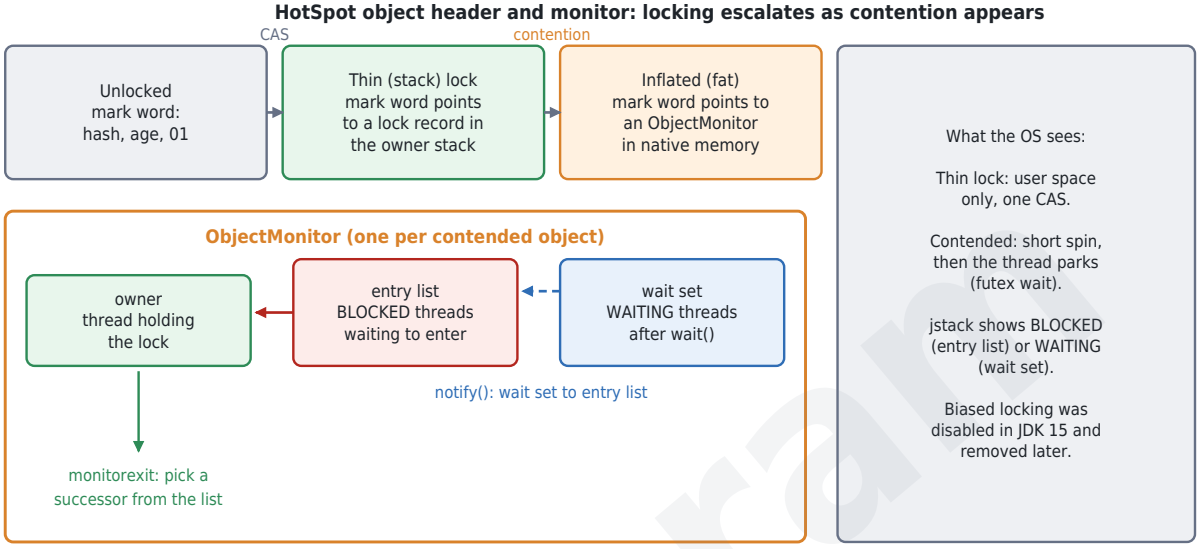


Figure 4.18. The header word escalates from unlocked to a thin lock to an inflated ObjectMonitor; `BLOCKED` threads sit in the entry list, `WAITING` threads in the wait set, and `notify()` moves one from the second to the first.

`wait()` releases the monitor, adds the thread to the wait set and parks it; `notify()` moves one waiter to the entry list, where it must still compete to reacquire the monitor, which is why `wait()` belongs in a loop that re-tests the predicate (the Java form of the condition-variable rule).

ReentrantLock and the AQS framework

`java.util.concurrent.locks.ReentrantLock` is implemented in Java on top of `AbstractQueuedSynchronizer` (AQS). The synchronizer keeps an integer `state` (the hold count) and a queue of waiting threads. `lock()` first attempts a CAS on `state`; on failure the thread enqueues a node and calls `LockSupport.park`. `unlock()` decrements the state and, when it reaches zero, unparks the successor in the queue. The class offers capabilities that `synchronized` lacks: `tryLock()` and `tryLock(timeout)` (the basis of the deadlock repair in the previous chapter), `lockInterruptibly()`, an optional **fair** mode that grants the lock in arrival order, and multiple `Condition` objects per lock. `Semaphore`, `CountDownLatch`, `ReentrantReadWriteLock` and `ThreadPoolExecutor`'s worker management are built on the same framework.

> **TIP**
> Prefer `synchronized` for simple mutual exclusion: it is shorter, cannot be forgotten (no `finally`), and the JVM can optimise it. Use `ReentrantLock` when a timed or interruptible acquire, fairness, or several conditions are required. Always release in `finally`.

park and unpark: the bridge to the kernel

Every blocking operation of `java.util.concurrent` reduces to `LockSupport.park()` and `LockSupport.unpark(thread)`. Each Java thread owns a native parker object containing a mutex and a condition variable. `park` waits on that condition variable; `unpark` signals it. glibc implements the condition variable with the futex system call. Consequently, a thread that waits for a `ReentrantLock`, a `Future`, a `BlockingQueue` or a timed sleep is, from the kernel's point of view, a thread in a futex wait.

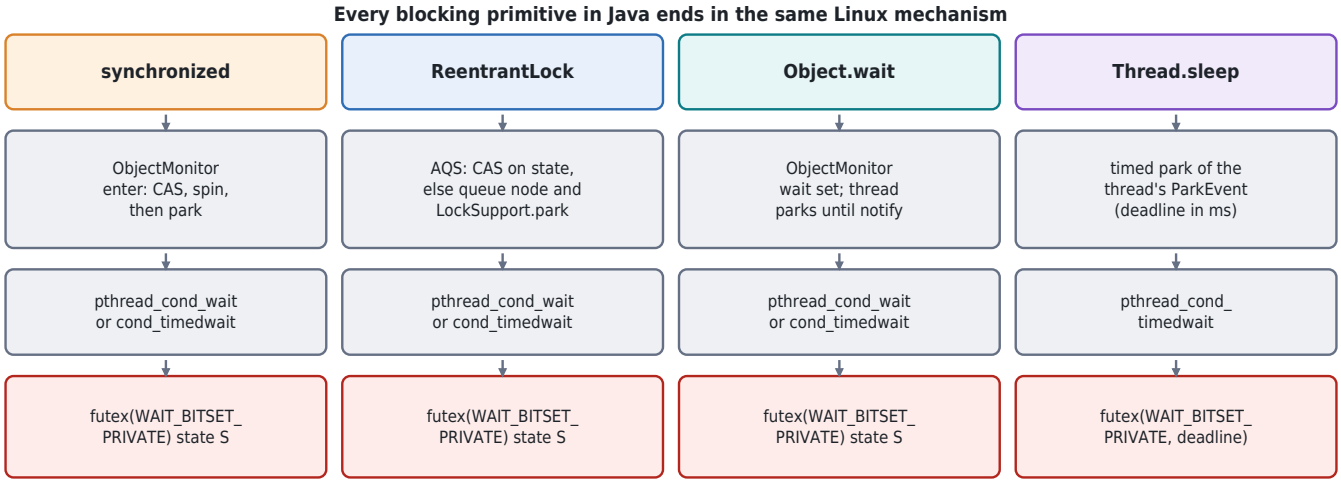


Figure 4.19. Four different Java waiting constructs converge on the same native condition variable and the same futex system call, which is why every blocked Java thread shows the kernel wait channel futex_do_wait.

Java thread states against Linux task states

`Thread.getState()` and thread dumps report six states: `NEW`, `RUNNABLE`, `BLOCKED`, `WAITING`, `TIMED_WAITING` and `TERMINATED`. They describe the position of the thread in the Java concurrency model, not the state of the kernel task. In particular `RUNNABLE` means "executing or eligible to execute in the JVM, including blocked in a native call such as a socket read", so a `RUNNABLE` thread may be sleeping in the kernel.

Thread (cause)	Java state in dump	Linux STAT	Kernel wait channel
`spinner (busy loop)`	`RUNNABLE`	R	-
`socket (ServerSocket.accept)`	`RUNNABLE`	S	inet_csk_accept
`blocked (monitor held by another)`	`BLOCKED (on object monitor)`	S	futex_do_wait
`waiting (Object.wait)`	`WAITING (on object monitor)`	S	futex_do_wait
`parked (LockSupport.park)`	`WAITING (parking)`	S	futex_do_wait
`timed (Thread.sleep)`	`TIMED_WAITING (sleeping)`	S	futex_do_wait

Java states against Linux states for threads in one JVM (measured, JDK 21.0.12).

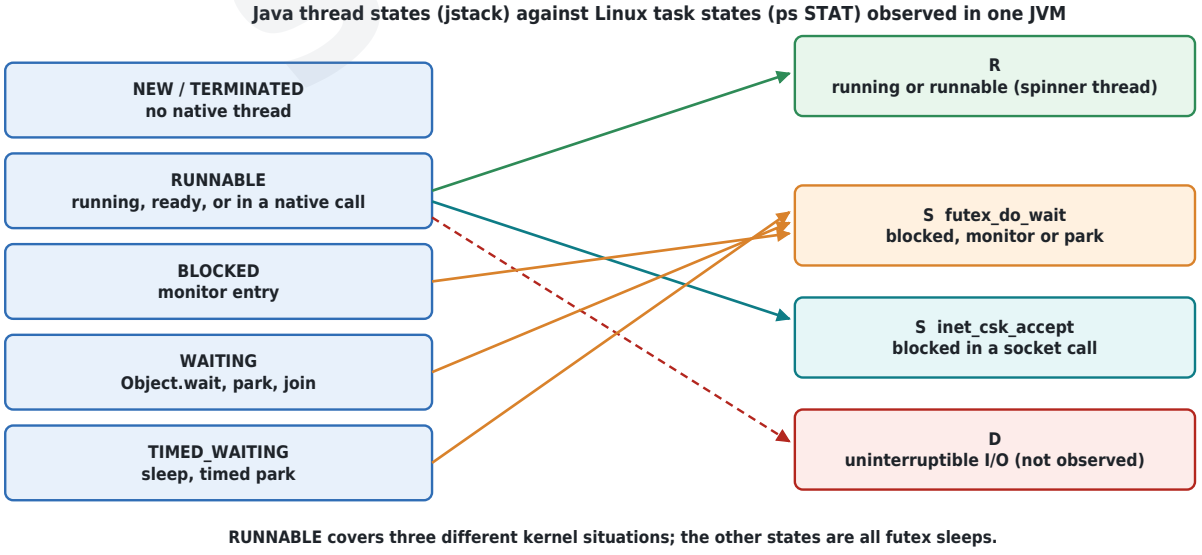


Figure 4.20. RUNNABLE covers a running thread and a thread blocked in a native socket call; all lock, monitor, park and sleep waits are Linux state S in a futex wait.

The practical consequences are direct. A dump with hundreds of `RUNNABLE` threads does not prove CPU saturation; `top -H` or the `STAT` and `WCHAN` columns of `ps -L` decide it. A thread that is `BLOCKED` or `WAITING` uses no CPU, so a service with low CPU and high latency is waiting on locks, pools or I/O, which is the premise of the diagnostic procedure in the next chapter.

Virtual threads and pinning

JDK 21 made **virtual threads** final (JEP 444). A virtual thread is a Java object scheduled by the JVM onto a small pool of platform **carrier threads**, by default a `ForkJoinPool` with as many carriers as CPUs. When a virtual thread blocks in a supported operation (`Thread.sleep`, `ReentrantLock.lock`, socket I/O, `BlockingQueue.take`), it is **unmounted**: its stack is copied to the heap and the carrier is free to run another virtual thread. The kernel sees only the carriers.

A virtual thread is **pinned** when it cannot be unmounted: while it executes inside a `synchronized` block or method, or inside a native method or foreign call. If it blocks while pinned, it blocks its carrier thread in a futex wait like any platform thread. With few carriers, a handful of pinned blocked virtual threads can stall the whole scheduler. JDK 24 removed the `synchronized` case (JEP 491, "Synchronize Virtual Threads without Pinning"); on JDK 21 and 23 it remains, native frames still pin on later releases, and the flag `-Djdk.tracePinnedThreads=short` prints a stack trace when a thread blocks while pinned.

4.4.2 Worked trace

Following one park and unpark through the kernel

The program `ParkDemo` starts a thread that calls `LockSupport.park()`; the main thread sleeps for 1.5 s, prints a message and calls `LockSupport.unpark`. It was run under `strace -f -tt -e trace=futex,write`, and the lines belonging to the two threads were extracted (timestamps are seconds within one minute; main is thread 3031, the waiter is thread 3054).

ParkDemo.java JAVA

```java
import java.util.concurrent.locks.LockSupport;

public class ParkDemo {
    public static void main(String[] args) throws Exception {
        Thread t = new Thread(() -> {
            LockSupport.park();
            System.out.println("waiter resumed");
        }, "waiter");
        t.start();
        Thread.sleep(1500);                 // waiter is parked
        System.out.println("main calls unpark");
        LockSupport.unpark(t);
        t.join();
    }
}
```

Kernel view of park and unpark (typical output, trimmed) TERMINAL

```
$ strace -f -tt -e trace=futex,write -o park.txt java ParkDemo.java
3054  33.256364 futex(0x7f69384d8220, FUTEX_WAIT_BITSET_PRIVATE|FUTEX_CLOCK_REALTIME, 0, NULL,
                FUTEX_BITSET_MATCH_ANY <unfinished ...>
3031  34.756813 write(1, "main calls unpark\n", 18) = 18
3031  34.756903 futex(0x7f69384d8220, FUTEX_WAKE_PRIVATE, 1) = 1
3054  34.756918 <... futex resumed>) = 0
```

Step	Meaning
1	The waiter calls `park`; the parker's condition variable becomes a futex wait on address `0x7f69384d8220` with no timeout (`NULL`). It sleeps for 1.5 s (33.256 to 34.756).
2	The main thread prints its message (`write`), then calls `unpark`.
3	`unpark` signals the condition variable: `FUTEX_WAKE_PRIVATE` on the same address, waking one thread (return value 1).
4	The waiter's `futex` call returns 0 at 34.756918, 15 microseconds after the wake-up was issued, and the thread resumes in Java.

Reading the trace.

The same address on the two calls is the link: the kernel queues the sleeper by address and the wake-up finds it by address. Nothing in the trace is specific to `park`; a contended `synchronized` or `ReentrantLock` produces the same two calls on different addresses.

4.4.3 Hands-on

Observing all six states in one process

`StateDemo` starts seven threads that wait in different ways: a thread holding a monitor while sleeping, a thread blocked on that monitor, one in `Object.wait`, one in `Thread.sleep`, one in `LockSupport.park`, one in `ServerSocket.accept` (on port 20401) and one in a busy loop.

StateDemo.java JAVA

```java
import java.net.ServerSocket;
import java.util.concurrent.locks.LockSupport;

public class StateDemo {
    static final Object MON = new Object();
    static final Object COND = new Object();
    static volatile boolean stop = false;

    static Thread start(String name, Runnable r) {
        Thread t = new Thread(r, name);
        t.setDaemon(true);
        t.start();
        return t;
    }

    public static void main(String[] args) throws Exception {
        start("holder", () -> { synchronized (MON) { sleep(60_000); } });
        Thread.sleep(200);
        start("blocked", () -> { synchronized (MON) { } });
        start("waiting", () -> {
            synchronized (COND) {
                try { COND.wait(); } catch (InterruptedException e) { }
            }
        });
        start("timed", () -> sleep(60_000));
        start("parked", () -> LockSupport.park());
        start("socket", () -> {
            try (ServerSocket s = new ServerSocket(20401)) {
                s.accept();
            } catch (Exception e) { }
        });
        start("spinner", () -> { long x = 0; while (!stop) { x++; } });
        System.out.println("ready pid=" + ProcessHandle.current().pid());
        Thread.sleep(25_000);
    }

    static void sleep(long ms) { try { Thread.sleep(ms); } catch (InterruptedException e) { } }
}
```

Java view and kernel view of the same threads (measured) TERMINAL

```
$ jcmd 2992 Thread.print | grep -A1 -E '^"(holder|blocked|waiting|timed|parked|socket|spinner)"'
"holder" #19 [3046] daemon prio=5 ... nid=3046 waiting on condition
   java.lang.Thread.State: TIMED_WAITING (sleeping)
"blocked" #20 [3047] daemon prio=5 ... nid=3047 waiting for monitor entry
   java.lang.Thread.State: BLOCKED (on object monitor)
"waiting" #21 [3048] daemon prio=5 ... nid=3048 in Object.wait()
   java.lang.Thread.State: WAITING (on object monitor)
"timed" #22 [3049] daemon prio=5 ... nid=3049 waiting on condition
   java.lang.Thread.State: TIMED_WAITING (sleeping)
"parked" #23 [3050] daemon prio=5 ... nid=3050 waiting on condition
   java.lang.Thread.State: WAITING (parking)
"socket" #24 [3051] daemon prio=5 ... nid=3051 runnable
   java.lang.Thread.State: RUNNABLE
"spinner" #25 [3052] daemon prio=5 ... nid=3052 runnable
   java.lang.Thread.State: RUNNABLE
$ ps -L -o tid,stat,wchan:22,comm -p 2992 \
    | grep -E "holder|blocked|waiting|timed|parked|socket|spinner"
 3046 Sl   futex_do_wait          holder
 3047 Sl   futex_do_wait          blocked
```

sreeram

Java view and kernel view of the same threads (measured) (continued) TERMINAL

```
 3048 Sl   futex_do_wait          waiting
 3049 Sl   futex_do_wait          timed
 3050 Sl   futex_do_wait          parked
 3051 Sl   inet_csk_accept        socket
 3052 Rl   -                      spinner
```

The join key is the number after `nid=`: on JDK 21 it equals the Linux thread id shown by `ps -L` and `top -H`. On JDK 17 and earlier the dump prints it in hexadecimal. The `socket` thread is `RUNNABLE` in Java and sleeping (`S`, `inet_csk_accept`) in the kernel; only the `spinner` is actually running.

Virtual-thread pinning

The program starts four virtual threads that each sleep 200 ms while holding a lock, once with a `ReentrantLock` and once with a `synchronized` block (each thread uses its own lock object, so no thread waits for another). The scheduler is restricted to one carrier with `-Djdk.virtualThreadScheduler.parallelism=1 -Djdk.virtualThreadScheduler.maxPoolSize=1` to make the effect exact.

Pinning.java JAVA

```java
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.locks.ReentrantLock;

public class Pinning {
    static void work(boolean useSynchronized) {
        Object mon = new Object();
        ReentrantLock rl = new ReentrantLock();
        try {
            if (useSynchronized) {
                synchronized (mon) { Thread.sleep(200); }
            } else {
                rl.lock();
                try { Thread.sleep(200); } finally { rl.unlock(); }
            }
        } catch (InterruptedException e) { }
    }

    static long time(boolean useSynchronized) throws Exception {
        List<Thread> ts = new ArrayList<>();
        long t0 = System.nanoTime();
        for (int i = 0; i < 4; i++) ts.add(Thread.ofVirtual().start(() -> work(useSynchronized)));
        for (Thread t : ts) t.join();
        return (System.nanoTime() - t0) / 1_000_000;
    }

    public static void main(String[] args) throws Exception {
        time(false);                                    // warm-up
        System.out.println("4 virtual threads, 1 carrier thread,"
                + " each sleeps 200 ms while holding a lock");
        System.out.println("  ReentrantLock: " + time(false) + " ms");
        System.out.println("  synchronized : " + time(true) + " ms");
    }
}
```

Pinning with one carrier thread (measured) TERMINAL

```
$ java -Djdk.virtualThreadScheduler.parallelism=1 \
    -Djdk.virtualThreadScheduler.maxPoolSize=1 Pinning.java
4 virtual threads, 1 carrier thread, each sleeps 200 ms while holding a lock
  ReentrantLock: 200 ms
  synchronized : 801 ms
$ java -Djdk.virtualThreadScheduler.parallelism=1 \
    -Djdk.virtualThreadScheduler.maxPoolSize=1 -Djdk.tracePinnedThreads=short Pinning.java
VirtualThread[#29]/runnable@ForkJoinPool-1-worker-1 reason:MONITOR
    Pinning.work(Pinning.java:11) <== monitors:1
```

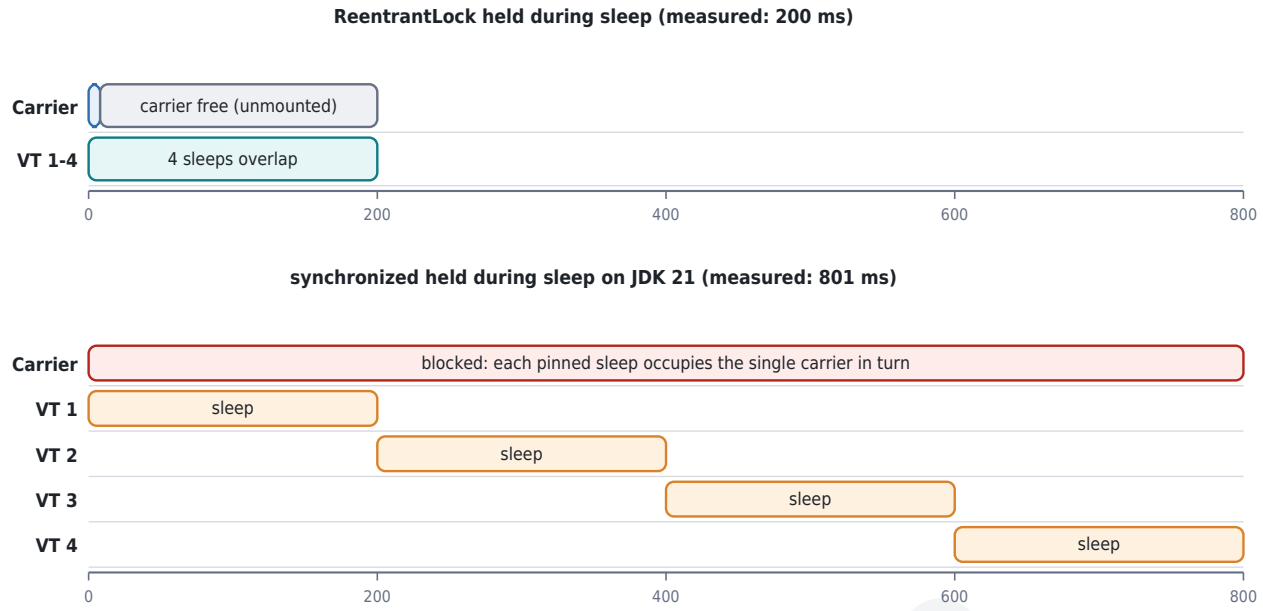


Figure 4.21. With the lock held during the sleep, the four sleeps overlap (200 ms) when the virtual threads can unmount, and run one after another (801 ms) when each is pinned to the single carrier by `synchronized`.

With the default scheduler on the two-CPU build machine (two carriers) the `synchronized` variant took 400 ms, twice the unpinned time: pinning does not make the program incorrect, it divides the available concurrency by the ratio of blocked pinned threads to carriers.

4.4.4 Real-world incidents

Incident: virtual threads enabled, throughput collapses under load

Scenario. A Spring Boot 3.2 service on JDK 21 is switched to virtual threads for request handling (`spring.threads.virtual.enabled=true`) to cope with many slow downstream calls. A third-party client library uses `synchronized` around a blocking HTTP call. The pod has a CPU limit of two cores, so the scheduler has two carrier threads.

Symptoms. Under moderate load every request that reaches the library is slow, and the service stops responding entirely when two or more concurrent calls to that library are slow: CPU is low, active virtual threads number in the thousands and the platform threads named `ForkJoinPool-1-worker-N` are all blocked.

Diagnosis. (1) `jcmd PID Thread.dump_to_file -format=json dump.json` writes a dump that includes virtual threads (the plain `Thread.print` lists only platform threads). (2) `top -H` shows two idle `ForkJoinPool-1-worker` threads, not a busy one. (3) Restart with `-Djdk.tracePinnedThreads=short`: the log prints `reason:MONITOR` with the frame of the `synchronized` block, as in the reproduction. (4) Counting pinned events per minute identifies how often it occurs. The reproduction in this chapter with one carrier shows the quantitative effect: four sleeps of 200 ms took 801 ms instead of 200 ms.

Root cause. A blocking call made while holding a monitor pins the virtual thread to its carrier on JDK 21. With as many pinned blocked virtual threads as carriers, no virtual thread can run.

Fix. Immediate: restore platform threads (set `spring.threads.virtual.enabled=false`) or raise `-Djdk.virtualThreadScheduler.maxPoolSize` and `parallelism` as a stop-gap. Permanent: replace the `synchronized` blocks that wrap blocking calls with `ReentrantLock` (own code or a patched library version), or upgrade to JDK 24 or later, where `synchronized` no longer pins; native frames still pin.

Prevention. Load-test with virtual threads enabled before release, keep `-Djdk.tracePinnedThreads=short` in the test environment, and monitor the `jdk.VirtualThreadPinned` JFR event (it fires when a virtual thread blocks while pinned for longer than a threshold, 20 ms by default).

Incident: hundreds of RUNNABLE threads but an idle CPU

Scenario. An engineer investigating slow responses takes a thread dump of a Spring Boot service and finds 180 threads in state `RUNNABLE`, most with a stack in `SocketInputStream.socketRead0`. The engineer concludes that the CPU is saturated and requests more cores.

Symptoms. Latency is high, `top` reports 5 percent CPU for the process, and adding cores changes nothing.

Diagnosis. (1) `top -H -p PID` shows that no thread exceeds a few percent. (2) `ps -L -o tid,stat,wchan:22 -p PID` shows the threads in state `S` with a network wait channel (in the reproduction `inet_csk_accept`; for an established connection a read waits in a socket receive routine). (3) The stack frames show the threads waiting for a downstream service, not computing. The reproduction table in this chapter shows the same contradiction for the `socket` thread: `RUNNABLE` in Java, `S` in the kernel.

Root cause. A misreading of `RUNNABLE`, which includes threads blocked in native I/O. The real bottleneck is a slow dependency whose responses the threads are waiting for.

Fix. Immediate: none from the service side; reduce the load on the dependency or fail fast. Permanent: set connect and read timeouts, bound concurrency to the dependency with a semaphore or bulkhead, and diagnose the dependency (see the following chapter).

Prevention. Teach the three-way comparison of dump state, `top -H` CPU and `ps` state and wait channel; record client-side latency histograms per dependency.

4.4.5 Questions to expect

1. **How does `synchronized` work at the bytecode level?** `monitorenter` and `monitorexit` around the block, with an exception-table entry that releases the monitor on exceptions; for methods the `ACC_SYNCHRONIZED` flag. The monitor is re-entrant.
2. **What happens when a second thread contends for a thin lock?** The monitor is inflated to an `ObjectMonitor`; the contender spins briefly and then parks in the entry list; the owner's release unparks a successor.
3. **How does `ReentrantLock` differ from `synchronized`?** It is a library class on AQS with `tryLock`, timed and interruptible acquire, optional fairness and several conditions; it must be released in `finally`. Both are re-entrant and give the same memory guarantees.
4. **How do `park` and `unpark` relate to the kernel?** Each thread has a native parker with a mutex and condition variable; park waits on it and unpark signals it. glibc implements both with `futex` calls, so a parked thread is in state `S` with wait channel `futex_do_wait`.
5. **What is the difference between `BLOCKED` and `WAITING`?** `BLOCKED` means waiting to enter a monitor held by another thread; `WAITING` means waiting for another thread's action (`wait`, `join`, `park`) with no timeout. `TIMED_WAITING` adds a timeout.
6. **Why can a `RUNNABLE` Java thread use no CPU?** `RUNNABLE` includes threads blocked in native calls such as socket reads; the kernel shows them as `S`. Only `top -H` or `ps` state `R` shows a running thread.
7. **What is virtual-thread pinning?** A virtual thread that blocks inside `synchronized` (JDK 21 to 23) or a native frame cannot unmount and occupies its carrier thread; with all carriers pinned the scheduler stalls.
8. **How do you map a thread-dump thread to an OS thread?** Use `nid`: it is the Linux thread id on JDK 21 (hexadecimal on earlier releases); match it with the first column of `top -H` or `ps -L` and with `/proc/PID/task`.
9. **Why must `wait()` be called in a loop?** After `notify()` the waiter must reacquire the monitor, and the condition may have changed or the wake-up may be spurious; the loop re-tests the predicate.

KEY POINTS

- `synchronized` compiles to `monitorenter` and `monitorexit`; HotSpot uses a thin lock when uncontended and an inflated `ObjectMonitor` (entry list and wait set) under contention.
- `ReentrantLock` and every `java.util.concurrent` waiting construct are built on AQS and `LockSupport.park`, which end in `futex(FUTEX_WAIT)` and `futex(FUTEX_WAKE)`.
- Java states are not Linux states: `BLOCKED`, `WAITING` and `TIMED_WAITING` threads are `S` in `futex_do_wait`; `RUNNABLE` may be running (`R`) or blocked in a native call (`S`).
- The `nid` value of a JDK 21 dump equals the Linux thread id seen in `top -H`, `ps -L` and `/proc/PID/task`.
- On JDK 21 a virtual thread that blocks inside `synchronized` pins its carrier; measured with one carrier, four 200 ms sleeps took 801 ms instead of 200 ms.

4.5 Diagnosing Concurrency Problems in Production

Concurrency defects rarely announce themselves with an exception. They appear as timeouts, falling throughput, a service that is up but does not answer, or a CPU that is oddly idle or oddly busy. The essential observation that organises the diagnosis is this: a service with high latency and low CPU is waiting, and the question is on what; a service with high latency and high CPU is computing, and the question is where. The instruments for both questions are the thread dump, per-thread CPU, and the kernel's view of each waiting thread.

4.5.1 Concept and mechanism

The procedure

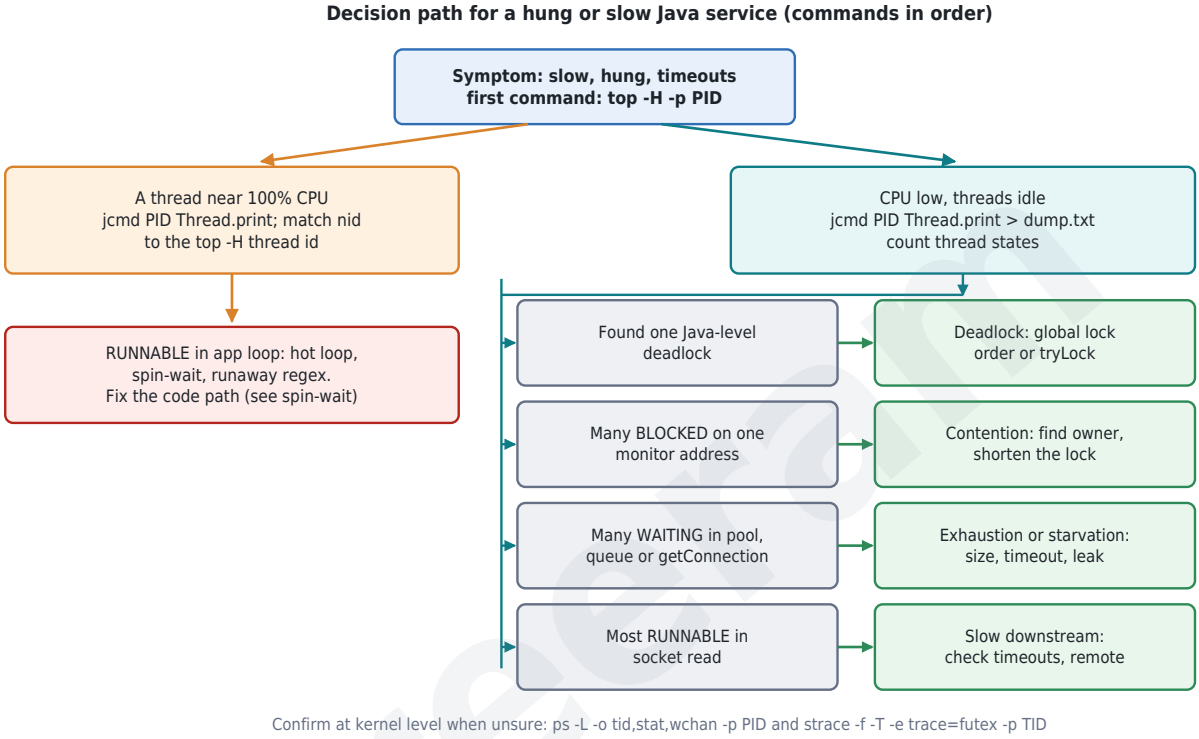


Figure 4.22. Start with per-thread CPU; a busy thread leads to the application loop that burns it, an idle process leads to a thread dump whose state counts and lock addresses name the cause.

The procedure has four steps, each answering one question.

1. **Is a thread computing?** `top -H -p PID` lists threads with their CPU share. One thread near 100 percent means a loop or spin; many threads at moderate load means genuine work.
2. **What are the threads doing?** `jcmd PID Thread.print` (or `jstack PID`, or `kill -3 PID` which prints to the JVM's standard output) writes all platform threads with state, owned and awaited locks and stacks. Take three dumps five to ten seconds apart: a thread that appears in the same place in all three is stuck; a thread that moves is merely busy. Add `-l` to include owned `java.util.concurrent` synchronizers. Virtual threads are not in the ordinary dump; use `jcmd PID Thread.dump_to_file -format=json file`.
3. **Count and group.** Count threads by state and by top frame. Hundreds of threads with an identical frame (`getConnection`, `FutureTask.get`, `monitorenter`) identify the saturated resource immediately.
4. **Confirm in the kernel.** `ps -L -o tid,stat,wchan:22 -p PID` shows whether a thread is really running (`R`) or asleep (`S`, `D`) and in which kernel function; `/proc/PID/task/TID/status` gives context-switch counters; `strace -f -T -e trace=futex -p TID` shows the individual waits and their durations.

Reading a thread dump

Anatomy of one thread-dump entry (JDK 21.0.12, reproduction from this chapter)

```
"transfer-A-to-B" #19 [14187] prio=5 os_prio=0 nid=14187 waiting for monitor entry   1
   java.lang.Thread.State: BLOCKED (on object monitor)                               2
   at TransferDeadlock.transfer(TransferDeadlock.java:12)                            3
   - waiting to lock <0x00000000a7b27130> (a TransferDeadlock$Account)               4
   - locked <0x00000000a7b27118> (a TransferDeadlock$Account)                        5
   at TransferDeadlock.run(TransferDeadlock.java:20)                                 6
```

1 thread name; nid is the OS thread id (decimal on JDK 21, hexadecimal on older JDKs)
2 Java state; BLOCKED (on object monitor) means waiting in the entry list
3 the source line where the thread is stopped
4 the monitor this thread is waiting for (address and class)
5 a monitor this thread already owns (the lock another thread is waiting for)
6 caller frames, newest first

Owner search: find the thread that prints 'locked' with the address printed after 'waiting to lock'.

Figure 4.23. Each line of a dump entry answers one question; the pair "waiting to lock" and "locked" with the same address connects a waiter to its owner.

The meaning of the main states for diagnosis is as follows.

State	Typical top frame	Diagnostic meaning
RUNNABLE	application method, `socketRead0`, `epollWait`	Running, or blocked in native I/O; check `top -H` before concluding that it consumes CPU
BLOCKED	`monitorenter`, "waiting to lock"	Contention on a `synchronized` monitor; find the owner by address
WAITING	`Unsafe.park`, `Object.wait`, `FutureTask.get`	Waiting for another thread: pool queue empty (healthy) or waiting for a result (suspect)
TIMED_WAITING	`Thread.sleep`, `parkNanos`, `Semaphore.tryAcquire`	Bounded wait: idle pool worker, retry loop, or a pool acquire with timeout

What each state indicates in a dump.

Idle pool workers are `WAITING` on the queue (`LinkedBlockingQueue.take`) and are normal; the same state in a thread that is processing a request, waiting on a `FutureTask` or a connection, is the signal.

Lock contention against CPU saturation

Contention and saturation both produce latency but have opposite signatures. Under saturation, threads are `R` and the run queue exceeds the CPU count. Under contention, most threads are asleep, voluntary context switches are high and CPU is below capacity. The per-thread counters in `/proc` show it directly: a thread that sleeps and is woken repeatedly accumulates voluntary context switches, a thread preempted while running accumulates non-voluntary ones.

Context switches of eight workers contending for one monitor (measured, 3 s after start) TERMINAL

```
$ grep -E 'voluntary_ctxt_switches' /proc/16021/task/16041/status
voluntary_ctxt_switches:        7158
nonvoluntary_ctxt_switches:     43
```

All eight workers showed between 7,000 and 11,000 voluntary and between 43 and 75 non-voluntary switches: thousands of sleep and wake cycles per second, almost no preemption. The futex calls of one worker show the waits one by one; `EAGAIN` means that the lock word changed before the thread went to sleep, so no sleep occurred:

Futex waits of one contending worker (measured, trimmed) TERMINAL

```
$ strace -T -e trace=futex -p 16041
futex(0x7fef4c567178, FUTEX_WAIT_BITSET_PRIVATE|..., 0, NULL, ...) = 0 <0.032501>
futex(0x7fef4c567128, FUTEX_WAKE_PRIVATE, 1) = 0 <0.000004>
futex(0x7fef4c56717c, FUTEX_WAIT_BITSET_PRIVATE|..., 0, NULL, ...) = -1 EAGAIN <0.000006>
futex(0x7fef4c567128, FUTEX_WAKE_PRIVATE, 1) = 0 <0.000005>
futex(0x7fef4c567178, FUTEX_WAIT_BITSET_PRIVATE|..., 0, NULL, ...) = 0 <0.000032>
```

The number in angle brackets is the time spent inside the call: the first wait lasted 32.5 ms, the later ones tens of microseconds. For a long-running service the more convenient instrument is Java Flight Recorder, which records every monitor wait above a threshold with the owner and the stack, without stopping the process:

Monitor contention events from JFR (measured, abbreviated) TERMINAL

```
$ java -XX:StartFlightRecording:filename=mon.jfr,jdk.JavaMonitorEnter#threshold=1ms \
    Contention.java x
$ jfr print --events jdk.JavaMonitorEnter --stack-depth 3 mon.jfr
jdk.JavaMonitorEnter {
  duration = 6.68 ms
  monitorClass = java.lang.Object (classLoader = bootstrap)
  previousOwner = "worker-0" (javaThreadId = 23)
  eventThread = "worker-4" (javaThreadId = 27)
  stackTrace = [
    Contention.lambda$run$0(long, boolean, long[], int) line: 26
    java.lang.Thread.run() line: 1583
  ]
}
```

The recording of this 4-second run contained 954 such events, each naming the waiting thread, its previous owner and the source line. On a production service the default threshold of 20 ms keeps the overhead low.

Pool saturation and queue growth

Every bounded resource (worker threads, database connections, semaphore permits) has a capacity in requests per second equal to size divided by service time. Little's law, concurrency = throughput x latency, gives the relation: a pool of 10 workers with a 50 ms task serves at most 200 tasks per second. When arrivals exceed that rate and the queue is unbounded, the queue length and the latency grow without limit; when the queue is bounded or the wait has a timeout, requests fail instead. Typical defaults to check in a Spring Boot service: Tomcat `server.tomcat.threads.max` 200, `server.tomcat.accept-count` 100 and `server.tomcat.max-connections` 8192; HikariCP `maximumPoolSize` 10, `connectionTimeout` 30,000 ms and `leakDetectionThreshold` 0 (disabled). The defaults should be confirmed for the framework version in use.

4.5.2 Worked trace

Saturation arithmetic checked by experiment

The program submits 300 tasks per second to a pool of 10 workers, each task sleeping 50 ms. The capacity is 10 / 0.05 = 200 tasks per second, so the expected surplus is 100 tasks per second, which accumulates in the queue and adds latency in proportion: each second of overload leaves 100 more tasks to be served at 200 per second, that is 0.5 s of extra waiting.

PoolSaturation.java JAVA

```
import java.util.concurrent.LinkedBlockingQueue;
import java.util.concurrent.ThreadPoolExecutor;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicLong;

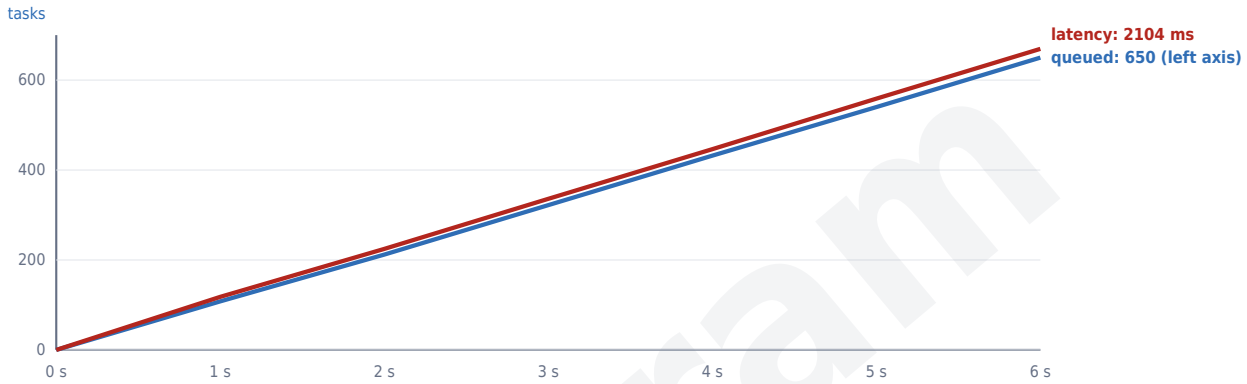
public class PoolSaturation {
    public static void main(String[] args) throws Exception {
        ThreadPoolExecutor pool = new ThreadPoolExecutor(10, 10, 0, TimeUnit.SECONDS,
                new LinkedBlockingQueue<>());
        AtomicLong lastLatencyMs = new AtomicLong();
        int perSecond = 300;               // arrival rate; capacity is 10 / 0.05 s = 200 per second
        for (int sec = 1; sec <= 6; sec++) {
            for (int i = 0; i < perSecond; i++) {
                final long submitted = System.nanoTime();
                pool.execute(() -> {
                    try {
                        Thread.sleep(50);
                    } catch (InterruptedException e) {
                        return;
                    }
                    lastLatencyMs.set((System.nanoTime() - submitted) / 1_000_000);
                });
                Thread.sleep(1000 / perSecond);
            }
            System.out.printf("t=%ds  active=%d  queued=%4d  completed=%4d  last latency=%d ms%n",
                    sec, pool.getActiveCount(), pool.getQueue().size(),
                    pool.getCompletedTaskCount(), lastLatencyMs.get());
        }
        pool.shutdownNow();
    }
}
```

Pool saturation (measured; achieved arrival rate is below 300 per second) TERMINAL

```
$ java PoolSaturation.java
t=1s  active=10  queued= 108  completed= 182  last latency=371 ms
t=2s  active=10  queued= 212  completed= 378  last latency=706 ms
t=3s  active=10  queued= 322  completed= 568  last latency=1056 ms
t=4s  active=10  queued= 432  completed= 758  last latency=1403 ms
t=5s  active=10  queued= 540  completed= 950  last latency=1756 ms
t=6s  active=10  queued= 650  completed=1140  last latency=2104 ms
```

The measured service rate is about 190 tasks per second (completed grows by 182 to 190 per second, below the ideal 200 because of scheduling overhead), the queue grows by about 108 per second and the latency by about 350 ms per second (the achieved arrival rate was below 300 per second because the loop sleeps 3 ms between submissions). The pool never shows an exception; the only symptoms are `active` pinned at its maximum, a growing queue and rising latency, which are the three numbers to export as metrics.

Queue length and latency when arrivals (300/s) exceed capacity (10 workers x 20/s)



Measured: active stays 10, the queue grows by about 108 tasks per second, latency by about 350 ms per second.

Figure 4.24. With arrivals above capacity, the active count stays at the pool size while the queue and the latency grow linearly with time.

4.5.3 Hands-on

A starvation deadlock in a thread pool

The program submits six "requests" to a pool of four threads. Each request submits a sub-task to the same pool and waits for its result with `get()`. Once all four workers are executing requests, no worker remains to run the sub-tasks.

PoolStarve.java JAVA

```java
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicInteger;

public class PoolStarve {
    public static void main(String[] args) throws Exception {
        AtomicInteger seq = new AtomicInteger();
        ThreadPoolExecutor pool = new ThreadPoolExecutor(4, 4, 0, TimeUnit.SECONDS,
                new LinkedBlockingQueue<>(), r -> {
                    Thread t = new Thread(r, "http-" + seq.incrementAndGet());
                    t.setDaemon(true);
                    return t;
                });

        // Each "request" submits a sub-task to the SAME pool and waits for it.
        for (int i = 0; i < 6; i++) {
            final int req = i;
            pool.execute(() -> {
                Future<String> sub = pool.submit(() -> "price-for-" + req);
                try { System.out.println(sub.get()); } catch (Exception e) { }
            });
        }
        Thread.sleep(1000);
        System.out.println("pool=" + pool.getPoolSize() + " active=" + pool.getActiveCount()
                + " queued=" + pool.getQueue().size()
                + " completed=" + pool.getCompletedTaskCount());
```

PoolStarve.java (continued) JAVA

```
        System.out.println("pid=" + ProcessHandle.current().pid());
        Thread.sleep(15_000);
    }
}
```

Starved pool (measured; the exact counts depend on timing) TERMINAL

```
$ java PoolStarve.java
pool=4 active=4 queued=6 completed=0
pid=30980
$ jcmd 30980 Thread.print | grep -A8 '^"http-1"'
"http-1" #19 [31402] daemon prio=5 os_prio=0 ... nid=31402 waiting on condition
   java.lang.Thread.State: WAITING (parking)
        at jdk.internal.misc.Unsafe.park(java.base@21.0.12.1/Native Method)
        - parking to wait for  <0x00000000a7ede090> (a java.util.concurrent.FutureTask)
        at java.util.concurrent.locks.LockSupport.park(java.base@21.0.12.1/LockSupport.java:221)
        at java.util.concurrent.FutureTask.awaitDone(java.base@21.0.12.1/FutureTask.java:500)
        at java.util.concurrent.FutureTask.get(java.base@21.0.12.1/FutureTask.java:190)
        at PoolStarve.lambda$main$2(PoolStarve.java:19)
```

In an earlier run of the same program the output was `pool=4 active=4 queued=4 completed=4`: two requests had slipped through with their sub-tasks before the pool filled. The stuck state is reached either way. Note that the dump contains no "Found one Java-level deadlock" section: `FutureTask` is not an ownable lock, so the JVM detector cannot see this cycle, although it is a deadlock in every other sense.

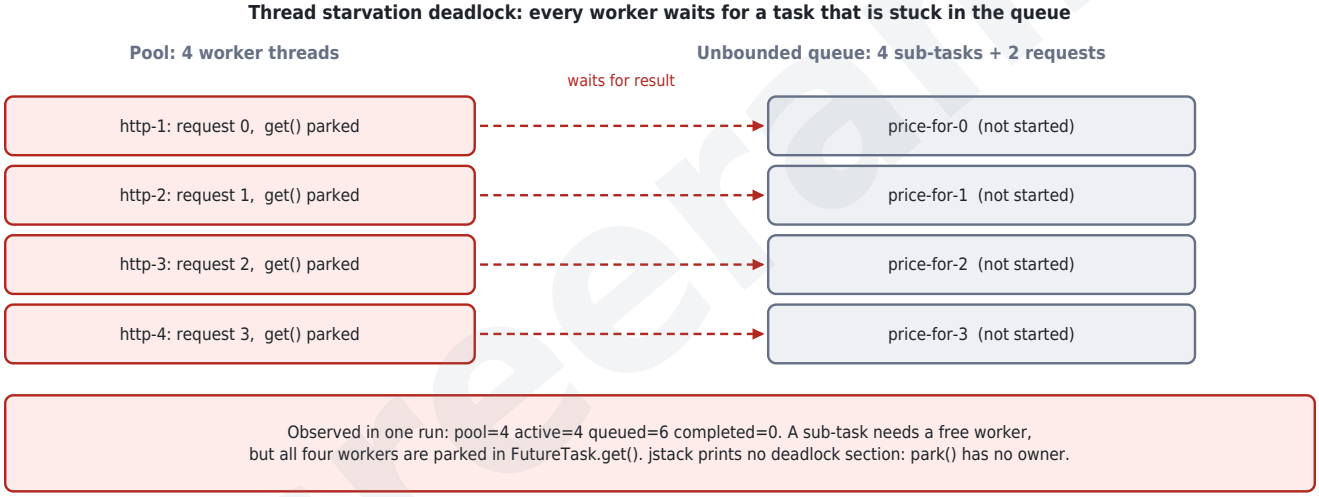


Figure 4.25. Every worker is parked in `FutureTask.get()` waiting for a sub-task that sits in the queue behind it; no worker is free to run it.

A leaking connection pool

The program models a connection pool of five connections with a 200 ms acquire timeout (the semantics of HikariCP's `connectionTimeout`, scaled down). The leaky handler releases the connection only on the success path; every third request throws a business exception after acquiring.

PoolLeak.java JAVA

```
import java.util.concurrent.*;

public class PoolLeak {
    /** Minimal bounded connection pool; acquire() waits up to the timeout. */
    static class Pool {
        private final Semaphore permits;
        private final long timeoutMs;
        Pool(int size, long timeoutMs) {
            permits = new Semaphore(size);
            this.timeoutMs = timeoutMs;
        }
        void acquire() throws Exception {
            if (!permits.tryAcquire(timeoutMs, TimeUnit.MILLISECONDS))
                throw new TimeoutException("Connection is not available, request timed out after "
                        + timeoutMs + "ms");
        }
```

PoolLeak.java (continued) JAVA

```java
        void release() { permits.release(); }
        int idle() { return permits.availablePermits(); }
    }

    static void handleLeaky(Pool pool, int i) throws Exception {
        pool.acquire();
        if (i % 3 == 0) throw new IllegalStateException("validation failed for order " + i);
        pool.release();                          // skipped when the exception is thrown
    }

    static void handleSafe(Pool pool, int i) throws Exception {
        pool.acquire();
        try {
            if (i % 3 == 0) throw new IllegalStateException("validation failed for order " + i);
        } finally {
            pool.release();                      // always runs
        }
    }

    interface Handler { void run(Pool p, int i) throws Exception; }

    static void drive(String name, Handler h) throws Exception {
        Pool pool = new Pool(5, 200);
        int ok = 0, bad = 0, timeouts = 0;
        for (int i = 1; i <= 30; i++) {
            try { h.run(pool, i); ok++; }
            catch (TimeoutException e) { timeouts++; }
            catch (IllegalStateException e) { bad++; }
        }
        System.out.printf("%-6s ok=%d business-errors=%d pool-timeouts=%d idle-connections=%d%n",
                name, ok, bad, timeouts, pool.idle());
    }

    public static void main(String[] args) throws Exception {
        drive("leaky", PoolLeak::handleLeaky);
        drive("safe", PoolLeak::handleSafe);
    }
}
```

Leaky and safe handlers, 30 requests each (measured) TERMINAL

```
$ java PoolLeak.java
leaky  ok=10 business-errors=5 pool-timeouts=15 idle-connections=0
safe   ok=20 business-errors=10 pool-timeouts=0 idle-connections=5
```



Figure 4.26. The leaky handler loses one connection per failed request; after the fifth failure (request 15) the pool is empty and every later request waits for the timeout and fails, although the business logic is healthy.

4.5.4 Real-world incidents

Incident: thread-pool exhaustion after a slow dependency

Scenario. A Spring Boot service calls a pricing service through a shared `ExecutorService` of 20 threads, used both for request processing and for the nested sub-calls of each request. The pricing service begins to answer slowly after a database maintenance window on its side.

Symptoms. p99 latency of the service rises from 120 ms to 30 seconds within minutes and clients receive gateway timeouts. CPU stays at 4 percent. After some time, requests that do not call the pricing service also time out. Heap and GC are normal.

Diagnosis. (1) `top -H -p PID` shows no busy thread: the process is waiting. (2) Three dumps taken ten seconds apart show the same pool workers in the same frames; `jcmd PID Thread.print | grep -c 'FutureTask.get'` returns 20, the pool size. (3) The pool metrics (`getActiveCount()` equals the maximum, `getQueue().size()` growing) confirm saturation, as the `PoolSaturation` run shows. (4) The frames after `FutureTask.get` point to the nested call, which identifies the starvation pattern of `PoolStarve`: callers occupy all workers while the callees wait in the queue. (5) `ps -L` shows all workers in `S` with `futex_do_wait`, ruling out CPU saturation.

Root cause. Two defects combine. A slow dependency holds workers for longer (capacity falls from size divided by 50 ms to size divided by 30 s) and nested tasks in the same bounded pool create a circular dependency: workers wait for queued work that only workers can execute.

Fix. Immediate: restart the instance to clear the stuck workers, and cut the traffic to the dependency (circuit breaker or feature flag). Permanent: use separate pools (bulkheads) for callers and callees, or avoid blocking on futures inside pool tasks (`CompletableFuture` composition); put a timeout on every `get()` and every HTTP call; bound the queue and reject with a clear error instead of queueing without limit.

Prevention. Export `active`, `queue size` and `rejected` per pool as metrics and alert when `active` equals the maximum for more than a minute; load-test with the dependency slowed; set client timeouts lower than the caller's timeout.

Incident: HikariCP pool starvation after a code change

Scenario. A release adds a validation step between obtaining a connection and executing a query in a Spring Boot service whose HikariCP pool has the default `maximumPoolSize` of 10. The validation throws an exception for malformed input; the connection is obtained with `dataSource.getConnection()` and closed at the end of the method, outside a `finally` block or try-with-resources.

Symptoms. After a few hours, requests fail with `HikariPool-1 - Connection is not available, request timed out after 30000ms`; each failure takes 30 seconds. Database CPU is low and the database shows ten idle connections from this service. Restarting the service fixes it for several hours.

Diagnosis. (1) A thread dump shows many request threads `TIMED_WAITING` in the pool's connection-acquire call (the Hikari `ConcurrentBag.borrow`, which waits with a timeout), while no thread is executing SQL. (2) The pool's own metrics (`hikaricp_connections_active` equal to the maximum, `hikaricp_connections_pending` growing, in Micrometer) show that connections are active yet no thread uses them. (3) In the database, `pg_stat_activity` (PostgreSQL) shows the service's sessions with state `idle` and an old `state_change`. (4) Enabling `spring.datasource.hikari.leak-detection-threshold=20000` makes Hikari log, for each connection held longer than 20 seconds, a warning with the stack trace of the code that obtained it. That stack points to the validation method. The `PoolLeak` reproduction has the same signature: `idle-connections=0` and every later request timing out.

Root cause. A connection leak on an exception path: the release was not guaranteed. Each failing request permanently removes one connection, until the pool is empty (request 15 in the reproduction, hours in production).

Fix. Immediate: restart the service; as a stop-gap raise `maximumPoolSize` only within what the database permits. Permanent: acquire connections with try-with-resources (or use Spring's `JdbcTemplate`, which manages them), release in `finally`, and reduce `connectionTimeout` so that failures are fast. Keep the leak-detection threshold enabled in production above the longest legitimate transaction.

Prevention. Alert on `connections_pending > 0` for more than a few seconds and on acquire time (`hikaricp_connections_acquire_seconds`); add a test that forces the exception path and asserts that the pool returns to full; size the pool by Little's law (connections = queries per second x query time), not by the number of threads.

4.5.5 Questions to expect

1. **A service has high latency and low CPU. What do you do?** Treat it as waiting. Take three thread dumps, group threads by state and top frame, and identify the shared resource (monitor, pool, connection, dependency). Confirm with `ps -L` that the threads are in `S`.
2. **How do you find which thread uses 100 percent CPU?** `top -H -p PID` for the thread id, then match it to the `nid` of `jcmd PID Thread.print` (decimal on JDK 21, hexadecimal on earlier releases) to read its stack.
3. **How do you find the owner of a contended monitor?** Take the address after "waiting to lock" in a BLOCKED thread and search the dump for the thread that prints "locked" with the same address; JFR's `jdk.JavaMonitorEnter` reports the previous owner directly.
4. **Why take several dumps?** One dump is a snapshot; the same thread in the same frame across dumps several seconds apart is stuck, while a changing frame means progress.
5. **How can a thread pool deadlock without the JVM reporting a deadlock?** Tasks that wait for sub-tasks queued in the same bounded pool occupy every worker; the wait is on a `FutureTask`, which has no owner, so the detector sees no cycle.
6. **How does a connection leak present, and how is it found?** The pool shows all connections active and requests timing out while the database shows idle sessions. Leak detection (`leakDetectionThreshold`) logs the stack of the code that obtained the unreturned connection.
7. **How do you size a pool?** With Little's law: required concurrency = arrival rate x service time. A pool smaller than that queues; a pool larger than the downstream can serve only moves the queue.
8. **What are the signs of lock contention at kernel level?** Many threads in `S` with `futex_do_wait`, high voluntary context-switch counts in `/proc/PID/task/TID/status`, futex calls with long durations under `strace -T`, and CPU below capacity.
9. **What does `EAGAIN` from a futex wait mean?** The futex word changed before the thread slept, so the kernel refused to sleep it; the thread retries without blocking.
10. **Why is `RUNNABLE` in a dump not proof of CPU load?** It includes threads in native I/O; `top -H` and the `STAT` column decide.

KEY POINTS

- High latency with low CPU means waiting: dump the threads, group by state and top frame, and find the resource that all of them share.
- `top -H` gives the thread id; `nid` in a JDK 21 dump is the same number; `ps -L -o tid,stat,wchan` and `/proc/PID/task/TID/status` confirm the kernel state.
- Contention shows as BLOCKED threads, high voluntary context switches and futex waits; JFR `jdk.JavaMonitorEnter` names the owner and the line.
- Pool saturation shows as `active` at the maximum with a growing queue; starvation deadlocks inside a pool and connection leaks are invisible to the JVM deadlock detector.
- Prevent with timeouts on every wait, separate pools for callers and callees, try-with-resources for connections, leak detection and pool metrics with alerts.

PART 5

Storage, File Systems and I/O

Every byte a Java service reads or writes passes through descriptors, the page cache, a file system and a block device; this Part follows that path and the failures along it.

In this part

- **5.1** Files and Directories
- **5.2** File System Internals
- **5.3** Storage Devices and the Block Layer
- **5.4** Buffered I/O, the Page Cache and Durability
- **5.5** I/O Models
- **5.6** Logging and Log Rotation in Production

5.1 Files and Directories

A running process cannot hold a disk, a socket or a terminal directly. The operating system therefore offers one narrow abstraction for all of them: the file, accessed through a small integer called a file descriptor. A Spring Boot service uses this abstraction for the jar it was started from, every log line, every configuration file, every HTTP connection and every database connection. Exhausting or misconfiguring the abstraction produces some of the most common production failures: `Too many open files`, `Permission denied` after a deployment and disks that are full for no visible reason. This chapter defines the model precisely, so that the incidents at the end of the chapter can be diagnosed from first principles.

5.1.1 Concept and mechanism

The file abstraction

In UNIX-derived systems a file is a sequence of bytes that supports a small set of operations: `open`, `read`, `write`, `lseek`, `close`, plus `fsync`, `mmap` and `ioctl` for special purposes. The kernel does not care whether the bytes live on a disk, in memory or on the far side of a network connection. Regular files, directories, symbolic links, character devices (`/dev/null`), block devices (`/dev/vda`), pipes, sockets and the pseudo files of `/proc` all use the same descriptor interface. The type of an object is the first character of the `ls -l` mode string: `-` regular file, `d` directory, `l` symbolic link, `c` character device, `b` block device, `p` named pipe, `s` socket.

Three tables between a descriptor and the data

A file descriptor is only an index. Between the index and the stored data lie three kernel structures, and the difference between them explains the behaviour of `dup`, `fork` and shared log files.

Structure	Scope	Contents
`Descriptor table`	one per process	index (0, 1, 2, ...) to open file description, plus the close-on-exec flag
`Open file description (OFD)`	system wide, reference counted	current offset, access mode (`O_RDONLY`, `O_RDWR`), status flags (`O_APPEND`, `O_NONBLOCK`), pointer to the inode
`Inode`	one per file object on a file system	type, owner, mode, size, timestamps, link count, location of the data blocks

The three levels of the open-file model.

Every successful `open` creates a new OFD and installs it in the lowest free slot of the descriptor table. The call `dup(fd)` installs a second slot that points to the same OFD, and `fork` copies the whole descriptor table, so parent and child share every OFD. Two descriptors that share an OFD share the offset: a write through one moves the position seen by the other. Two independent `open` calls on the same path create two OFDs with independent offsets, although both reach the same inode.

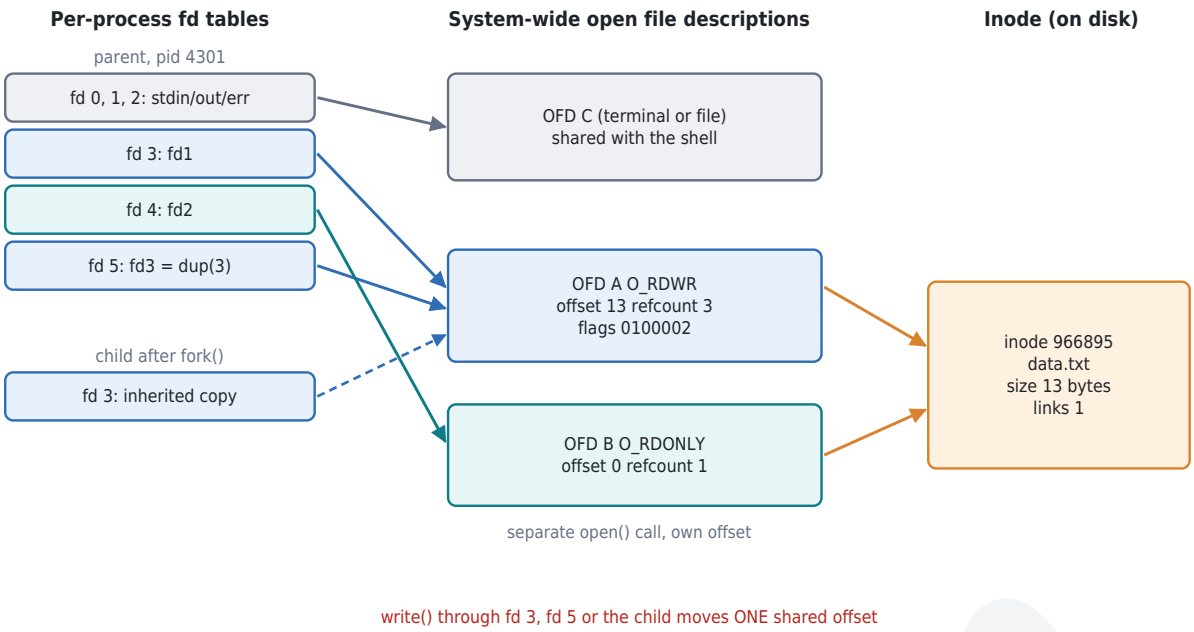


Figure 5.1. Three descriptors in a parent process and one in a forked child reach two open file descriptions and a single inode; only descriptors that share an OFD share a file offset.

Standard descriptors and redirection

Descriptor 0 is standard input, 1 is standard output and 2 is standard error. These numbers are conventions inherited across `fork` and `execve`, not special kernel objects. The shell implements the redirection `ls /nonexistent > out.txt 2>&1` by opening the file after `fork` and before `execve`, then copying the descriptor onto slots 1 and 2 with `dup2`. The program that is executed afterwards writes to descriptor 2 without knowing that it leads to a file. Pipelines work in the same way, with `pipe` supplying the two ends; the chapter on inter-process communication in the Part on processes, threads and scheduling shows the pipe case.

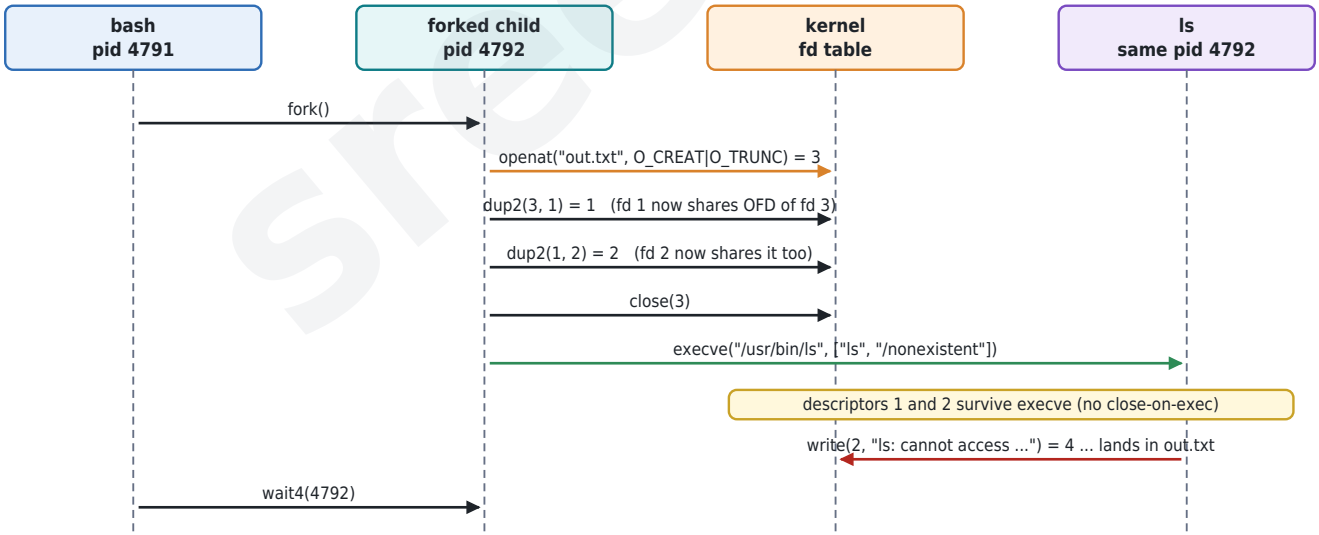


Figure 5.2. The shell child performs openat, dup2 and dup2 before execve, so the redirected descriptors are inherited by the new program unchanged.

The descriptor flag `FD_CLOEXEC` (set with `O_CLOEXEC` at open time) closes a descriptor automatically during `execve`. Well-behaved programs set this flag on files and sockets so that child processes do not inherit them; the JDK additionally closes inherited descriptors above 2 in a child started through `ProcessBuilder`. A child that does inherit a listening socket or a log file can keep it open after the parent has closed it, which later appears as a deleted-but-open file or a port that cannot be rebound.

Observing descriptors through /proc

For any process, `/proc/PID/fd` contains one symbolic link per open descriptor, and `/proc/PID/fdinfo/N` reports the OFD state: the current offset (`pos`), the octal flags, the mount identifier and the inode number. These two views are the primary diagnostic interface for descriptor leaks, because they require no agent and work for any process owned by the inspecting user. The symbolic link target shows the path, `socket:[inode]`, `pipe:[inode]` or `anon_inode:[eventpoll]` for epoll instances.

Limits on open files

Three independent limits apply, and a failure message does not say which one was reached.

- The per-process soft limit (`RLIMIT_NOFILE`, shown by `ulimit -Sn`) is the value against which `open` and `socket` return `EMFILE`, reported by Java as `Too many open files`.
- The per-process hard limit (`ulimit -Hn`) is the ceiling to which an unprivileged process may raise its own soft limit. The kernel parameter `fs.nr_open` bounds the hard limit itself.
- The system-wide limit `fs.file-max` bounds the total number of open file descriptions on the machine; exceeding it returns `ENFILE`. The file `/proc/sys/fs/file-nr` shows the allocated count, the unused count and the maximum.

On the build machine the kernel reported `fs.nr_open = 1048576`, `fs.file-max = 819675`, a soft and hard limit of 20000 for the shell, and `fs.file-nr` of `132 0 819675`. The HotSpot flag `-XX:+MaxFDLimit` is enabled by default and raises the soft limit to the hard limit when the JVM starts. The experiment later in this chapter confirmed it: a process started with a soft limit of 256 and a hard limit of 4096 reported a soft limit of 4096 inside the JVM.

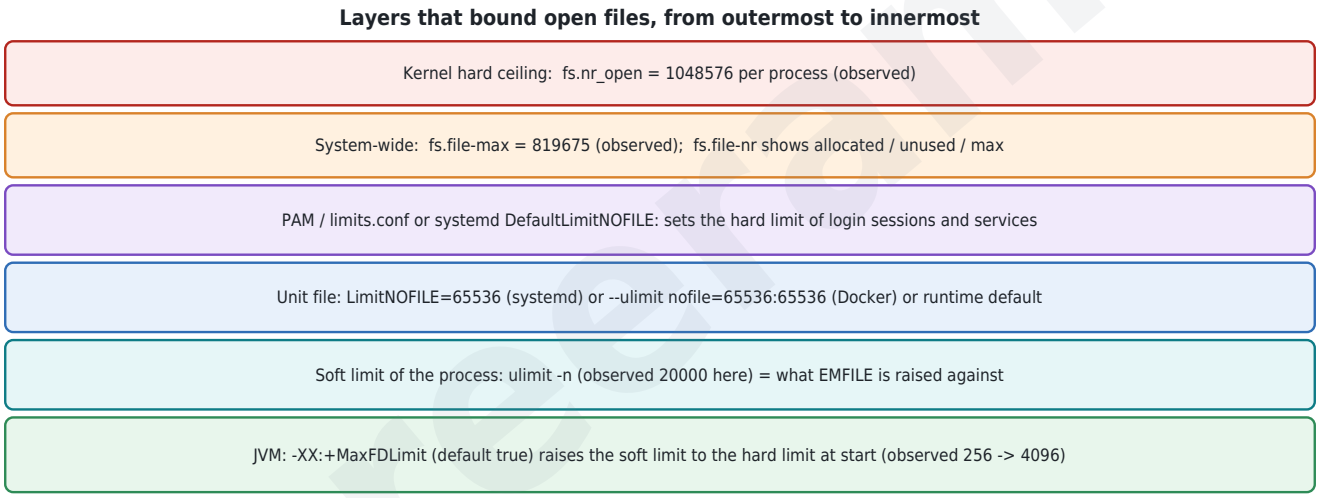


Figure 5.3. The limits form layers from the kernel ceiling down to the JVM; an `EMFILE` error is raised against the innermost soft limit, but the value must be set in whichever outer layer starts the process.

Permissions, umask and special bits

Each inode stores an owner uid, a group gid and a 12-bit mode. The nine permission bits form three classes: owner, group and other. The kernel selects exactly one class for a given process: owner if the effective uid matches, otherwise group if the effective gid or a supplementary group matches, otherwise other. The selected class is the only one consulted, so an owner with mode `0044` cannot read the file even though group and other can. The root user bypasses these checks through the capabilities `CAP_DAC_OVERRIDE` and `CAP_DAC_READ_SEARCH`.

For a directory the bits mean: `r` list names, `w` create, delete or rename entries (this requires `x` as well), `x` traverse the directory to reach entries inside. Deleting a file therefore depends on the permission of the directory, not of the file. The sticky bit on a directory (`/tmp` has mode `1777`) restricts deletion to the owner of the file or of the directory.

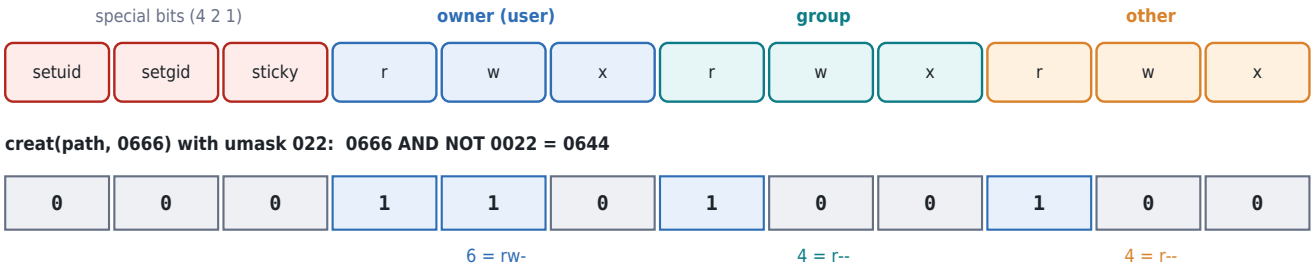


Figure 5.4. The mode word consists of three special bits and three rwx triplets; the umask removes bits from the mode requested by open, so a requested 0666 becomes 0644 under umask 022.

The special bits change who a process runs as. The setuid bit makes an executable run with the uid of its owner: on the build machine `/usr/bin/passwd` has mode `-rwsr-xr-x` and owner root, so any user can change a password entry that only root can write. The setgid bit on a directory makes new files inherit the directory group, which shared deployment directories use. Access control lists extend the owner, group and other triplets with entries for named users and groups, managed with `setfacl` and `getfacl`; the build machine had neither tool installed, so ACL syntax is not demonstrated here.

5.1.2 Worked trace: one file, three descriptors

The C program in the listing opens `data.txt` twice, duplicates the first descriptor, writes ten bytes, forks, lets the child write three more bytes and prints the offsets. Each step corresponds to a row in the figure above.

fdtable.c C

```c
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/wait.h>
#include <unistd.h>

static void show(const char *who, int fd) {
    printf("%-6s fd=%d offset=%ld\n", who, fd, (long)lseek(fd, 0, SEEK_CUR));
}

int main(void) {
    int fd1 = open("data.txt", O_RDWR | O_CREAT | O_TRUNC, 0644);
    int fd2 = open("data.txt", O_RDONLY);   /* separate open file description */
    int fd3 = dup(fd1);                     /* shares description with fd1 */
    write(fd1, "ABCDEFGHIJ", 10);
    show("fd1", fd1);
    show("fd2", fd2);
    show("fd3", fd3);
    if (fork() == 0) {                      /* child inherits the descriptor table */
        write(fd1, "xyz", 3);
        _exit(0);
    }
    wait(NULL);
    show("parent", fd1);
    char cmd[64];
    snprintf(cmd, sizeof cmd, "ls -l /proc/%d/fd", getpid());
    fflush(stdout);
    system(cmd);
    snprintf(cmd, sizeof cmd, "cat /proc/%d/fdinfo/%d", getpid(), fd1);
    system(cmd);
    return 0;
}
```

Compile and run (standard descriptors 0 to 2 omitted from the listing) TERMINAL

```
$ gcc -Wall -o fdtable fdtable.c && ./fdtable
fd1    fd=3 offset=10
fd2    fd=4 offset=0
fd3    fd=5 offset=10
parent fd=3 offset=13
```

Compile and run (standard descriptors 0 to 2 omitted from the listing) (continued) TERMINAL

```
total 0
lrwx------ 1 root root 64 Oct 18 02:15 3 -> /tmp/osw_o5/c/data.txt
lr-x------ 1 root root 64 Oct 18 02:15 4 -> /tmp/osw_o5/c/data.txt
lrwx------ 1 root root 64 Oct 18 02:15 5 -> /tmp/osw_o5/c/data.txt
pos:    13
flags:  0100002
mnt_id: 28
ino:    966895
$ cat data.txt
ABCDEFGHIJxyz
```

The numbered interpretation follows.

1. The three `open`/`dup` calls take the lowest free slots 3, 4 and 5, because 0 to 2 are occupied.
2. After the ten-byte write through descriptor 3, descriptors 3 and 5 report offset 10 because they share one OFD; descriptor 4 reports 0 because it owns a separate OFD.
3. The `fork` copies the table. The child writes `xyz` through its copy of descriptor 3, which points to the same OFD, so the shared offset advances from 10 to 13 and the parent observes 13 without having written anything.
4. The file contains `ABCDEFGHIJxyz`: the child appended after the parent data instead of overwriting it. This is the behaviour that lets a shell script and its children write one log file in sequence.
5. `fdinfo` confirms `pos: 13` and the flags `0100002`, which is octal for `O_RDWR` (2) together with `O_LARGEFILE` (0100000).

5.1.3 Hands-on

Redirection under strace

The following session traces the shell while it executes a redirected command. The trace was filtered to the calls that matter; the process identifiers are typical.

The shell opens, duplicates and executes TERMINAL

```
$ strace -f -e trace=openat,dup2,close,execve -o st.txt bash -c 'ls /nonexistent > out.txt 2>&1'
$ grep -E 'out.txt|dup2|execve' st.txt
4791  execve("/usr/bin/bash", ["bash", "-c", "ls /nonexistent > out.txt 2>&1"], ...) = 0
4792  openat(AT_FDCWD, "out.txt", O_WRONLY|O_CREAT|O_TRUNC, 0666) = 3
4792  dup2(3, 1)                        = 1
4792  dup2(1, 2)                        = 2
4792  execve("/usr/bin/ls", ["ls", "/nonexistent"], ...) = 0
$ cat out.txt
ls: cannot access '/nonexistent': No such file or directory
```

The order of the redirections matters. In `> out.txt 2>&1` descriptor 2 is copied from descriptor 1 after descriptor 1 already points to the file, so both reach the file. In `2>&1 > out.txt` descriptor 2 is copied first and still points to the terminal or pipe; error output then bypasses the file. Container log collection depends on this: a service whose error stream is not redirected to the same destination as its output loses stack traces.

Permissions and umask

Creation modes under umask 022 and umask 077 (run as root; files owned by uid 1000) TERMINAL

```
$ umask
0022
$ setpriv --reuid=1000 --regid=1000 --clear-groups bash -c 'touch data/f2; ls -ln data/f2'
-rw-r--r-- 1 1000 1000 0 Oct 18 02:19 data/f2
$ (umask 077; touch data/private; ls -ln data/private)
-rw------- 1 0 0 0 Oct 18 02:19 data/private
$ setpriv --reuid=1001 --regid=1001 --clear-groups bash -c 'cat data/private'
cat: data/private: Permission denied
```

Descriptors of a JVM

A JVM that serves many connections holds one socket descriptor per connection in addition to its jar, class library image and log files. The server in the chapter on I/O models held the following after 500 clients connected (typical output):

Descriptors of the NioEcho JVM with 500 connected clients — TERMINAL

```
$ ls /proc/$JP/fd | wc -l
508
$ ls -l /proc/$JP/fd | grep -E "eventpoll|socket" | awk '{print $NF}' \
    | sed 's/\[.*//' | sort | uniq -c
      1 anon_inode:
    502 socket:
```

5.1.4 Real-world incidents

Incident: Too many open files caused by a descriptor leak

Scenario. A Spring Boot service on Kubernetes reads a small tenant configuration file for every request through a helper that opens a `FileInputStream` and never closes it. The container runs with the default limit of 1024 descriptors on some runtimes and 1048576 on others; this instance had a soft limit of 20000 in production and was started with 256 in the reproduction below.

Symptoms. After several hours the error rate rises abruptly. Logs contain `java.io.IOException: ... (Too many open files)` for unrelated operations such as accepting HTTP connections, opening database sockets and writing logs. The readiness probe fails because the server cannot accept the probe connection. Heap usage and CPU are normal.

Diagnosis. The first step compares the current descriptor count with the limit of the process. The second classifies the descriptors by target, which identifies the leak source. The reproduction used `FdLeak.java` (listing below) under `ulimit -n 256`.

FdLeak.java — JAVA

```
import java.io.FileInputStream;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.List;

public class FdLeak {
    private static final List<FileInputStream> LEAKED = new ArrayList<>();

    // Bug: the stream is never closed, so each call leaks one descriptor.
    static String readConfig(Path p) throws IOException {
        FileInputStream in = new FileInputStream(p.toFile());
        LEAKED.add(in);            // simulates a reference kept by a pool or listener
        return new String(in.readAllBytes());
    }

    // Fix: Files.readString opens and closes the descriptor internally.
    static String readConfigFixed(Path p) throws IOException {
        return Files.readString(p);
    }

    public static void main(String[] args) throws Exception {
        Path p = Path.of("/etc/hostname");
        boolean fixed = args.length > 0 && args[0].equals("fixed");
        long pid = ProcessHandle.current().pid();
        for (int i = 1; i <= 400; i++) {
            try {
                if (fixed) readConfigFixed(p); else readConfig(p);
            } catch (IOException e) {
                System.out.println("call " + i + " failed: " + e.getMessage());
                System.out.println("pid " + pid + ", leaked streams: " + LEAKED.size());
                Thread.sleep(20000);      // keep the process alive for inspection
                return;
            }
        }
        System.out.println("all calls succeeded; open descriptors: "
                + Path.of("/proc/" + pid + "/fd").toFile().list().length);
    }
}
```

srcram

Reproduction and diagnosis (output from the build machine) TERMINAL

```
$ (ulimit -n 256; java FdLeak.java > leak.out 2>&1 &)
$ cat leak.out
call 253 failed: /etc/hostname (Too many open files)
pid 3327, leaked streams: 252
$ grep "open files" /proc/3327/limits
Max open files            256                  256                  files
$ ls /proc/3327/fd | wc -l
256
$ ls -l /proc/3327/fd | awk '{print $NF}' | sort | uniq -c | sort -rn | head -3
    252 /etc/hostname
      2 /tmp/osw_o5/j/leak.out
      1 /usr/lib/jvm/java-21-openjdk-amd64/lib/modules
$ java FdLeak.java fixed
all calls succeeded; open descriptors: 5
```

The count equals the limit (256), and 252 of the descriptors point to the same path. Descriptors pointing to one path indicate an unclosed file stream. If the majority were `socket:[...]`, the next commands would be `ss -tanp` to count connection states per remote address and `lsof -i -a -p PID` to identify the peer, which usually reveals a client library whose responses are not closed. The failing call is a symptom; the leak site is wherever the stream was created.

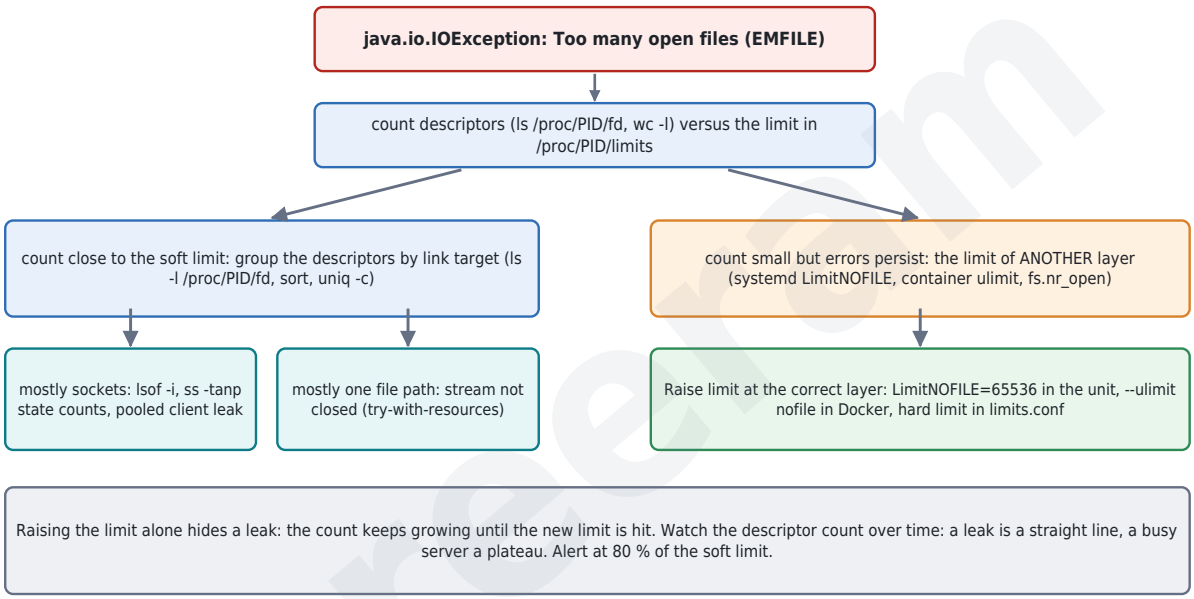


Figure 5.5. The diagnosis starts with the descriptor count against the limit and branches on what the descriptors point to.

Root cause. A code path obtained a descriptor and kept a reference without closing it. The garbage collector does not close descriptors promptly: finalisation is not guaranteed, and a heap with plenty of free space may not collect for a long time, so the leak is invisible to heap monitoring.

Fix. Immediate mitigation: restart the process to release the descriptors, and raise the limit temporarily only if restarts are impossible. Permanent fix: close every stream with try-with-resources, as in `readConfigFixed`, and reuse HTTP clients (for example one `HttpClient` or `RestTemplate` per application) rather than creating one per request.

Prevention. Export the descriptor count as a metric (Micrometer publishes `process.files.open` and `process.files.max`) and alert at 80 percent of the soft limit. A leak appears as a straight rising line, a legitimately busy server as a plateau. Add a load test that checks the count after traffic stops.

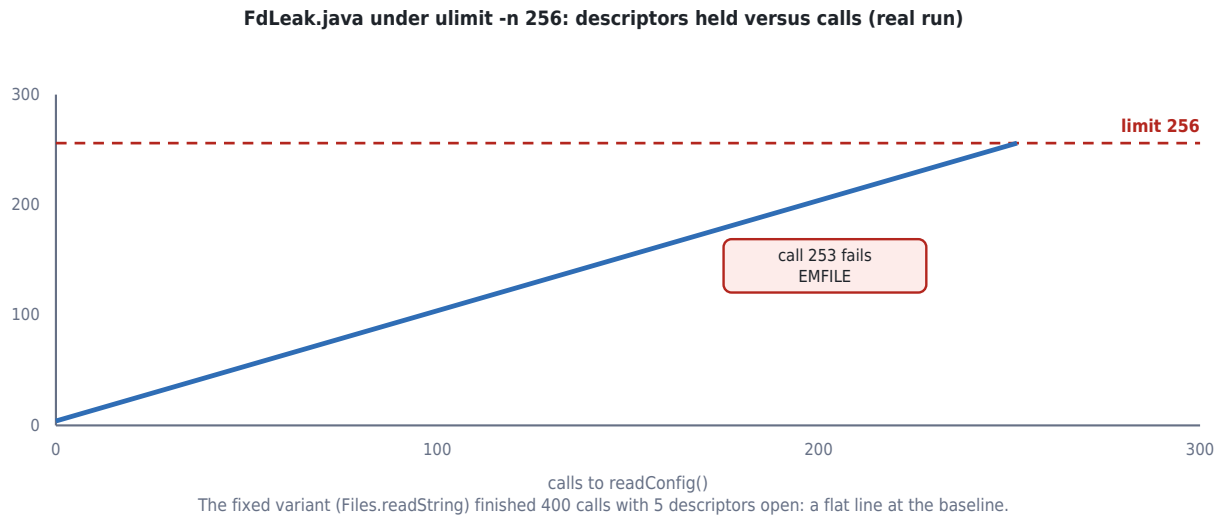


Figure 5.6. The descriptor count rises by one per call until it meets the limit of 256 at call 253, whereas the fixed variant stays at the baseline.

Incident: file descriptor limit too low in systemd or a container

Scenario. A service works on a developer machine and in a staging container but fails under load in production after migration to a systemd unit. The shell limit on the host is 20000, so the team assumes the same value applies to the service.

Symptoms. `Too many open files` appears at only about 1000 concurrent connections, far below the shell limit. Increasing `fs.file-max` has no effect.

Diagnosis. The limit of a running process is read from the process, never from the shell that was used to log in. The reproduction below shows that the limit follows the process and its parent chain.

The limit belongs to the process that was started with it — TERMINAL

```
$ (ulimit -n 1024; sleep 60 & echo $! > sl.pid)
$ grep "open files" /proc/$(cat sl.pid)/limits
Max open files            1024                 1024                 files
$ prlimit --pid $(cat sl.pid) --nofile=65536:65536
prlimit: failed to set the NOFILE resource limit: Operation not permitted
$ cat /proc/sys/fs/nr_open /proc/sys/fs/file-max
1048576
819675
```

The second command failed on the build machine because the container that hosts it lacks the capability `CAP_SYS_RESOURCE`; on an ordinary host, root can raise the limit of a live process with `prlimit`, which is useful as a mitigation. The decisive diagnostic is `grep "open files" /proc/PID/limits` against the production process, which here would show the value 1024 or 4096 granted by the service manager, not 20000.

Root cause. The limit was configured in the layer that does not apply: shell profiles and `limits.conf` do not affect services started by systemd, which applies `LimitNOFILE` from the unit or from `DefaultLimitNOFILE` in `system.conf`. In container platforms the limit comes from the container runtime defaults.

Fix. Set the limit in the layer that starts the process (typical configuration, not run on the build machine because systemd is not the init system there):

/etc/systemd/system/orders.service.d/limits.conf

```
[Service]
LimitNOFILE=65536
```

For Docker use `docker run --ulimit nofile=65536:65536`; for Kubernetes the limit is inherited from the container runtime configuration of the node and is raised through the runtime, not through the pod specification. After the change, restart the service and verify the new value in `/proc/PID/limits`.

Prevention. Document the limit as part of the service definition, log `ulimit -n` at application start and publish `process.files.max` as a metric next to `process.files.open`. Size the limit from the connection budget: each client connection consumes one descriptor, each upstream connection one more, plus database pool size, log files and a safety margin.

Incident: Permission denied after a deployment

Scenario. A hardened image replaces a base image that ran as root. The new image declares `USER 1000` and the pod sets `runAsNonRoot: true`. The service writes a lock file and temporary uploads to a mounted volume that was created earlier by the old image.

Symptoms. The new pods start, but every upload fails with `java.nio.file.AccessDeniedException: /data/app.lock`. The old pods continued to work with the same volume.

Diagnosis. The numeric identity of the process is compared with the numeric owner and mode of the directory. Names can be misleading because the same uid may map to different names in different images; `ls -ln` and `id` show numbers.

Reproduction with a root-owned directory (run as root; setpriv drops to uid 1000) TERMINAL

```
$ ls -ldn data
drwxr-xr-x 2 0 0 4096 Oct 18 02:19 data
$ setpriv --reuid=1000 --regid=1000 --clear-groups bash -c 'touch data/app.lock; echo rc=$?'
touch: cannot touch 'data/app.lock': Permission denied
rc=1
$ chown 1000:1000 data
$ setpriv --reuid=1000 --regid=1000 --clear-groups \
    bash -c 'touch data/app.lock && echo created; ls -ln data'
created
total 0
-rw-r--r-- 1 1000 1000 0 Oct 18 02:19 app.lock
```

The directory belongs to uid 0 with mode `755`, so uid 1000 falls in the "other" class, which has no write bit. Searching for the cause in application code would be wasted effort; the kernel returned `EACCES` from `openat`, and `strace -f -e trace=openat -p PID` would show it with the exact path.

Root cause. The image changed the runtime uid but the volume kept the ownership assigned under the previous uid.

Fix. Immediate: change ownership of the volume contents to the new uid (an init container running `chown -R 1000:1000 /data`). Permanent: in Kubernetes set `securityContext.fsGroup: 1000` so that the kubelet makes the volume group-owned by gid 1000 and group-writable; in Docker use a matching `--user` and a host directory with the same owner. Setting mode `777` or reverting to root removes the symptom while removing the isolation that motivated the change.

Prevention. Run the hardened image in staging with the real volume type. Include a startup self-test that creates and deletes a file in each writable directory and fails fast with a message naming the uid, the path and the mode. Keep the `umask` of the image consistent: with `umask 077` in a base image, files created by one container are unreadable to a backup or sidecar container that runs under another uid.

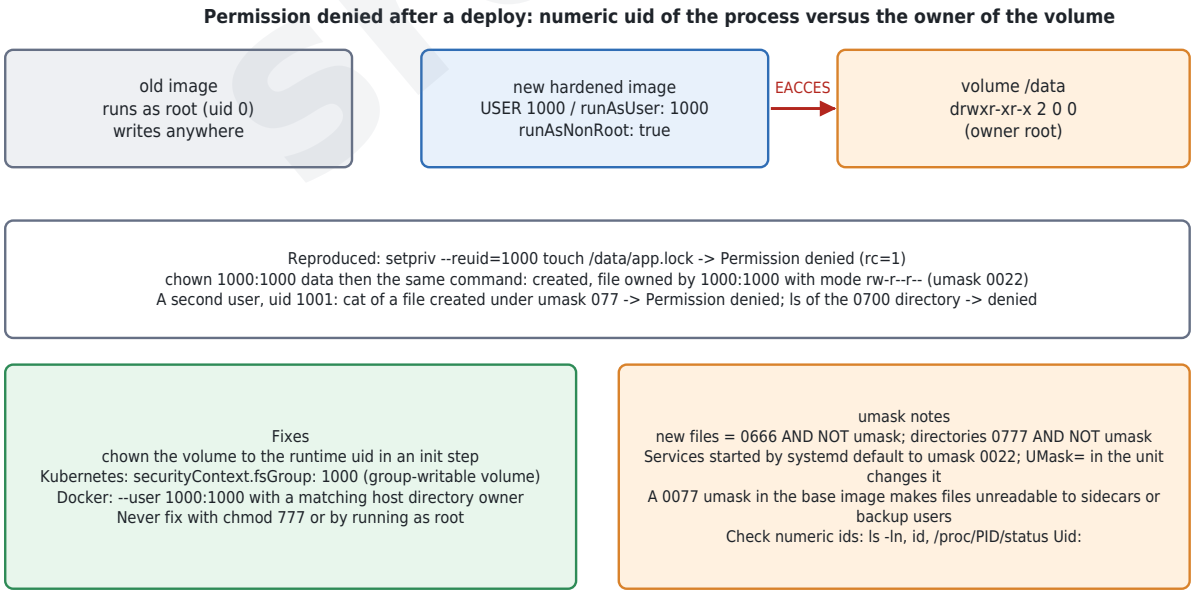


Figure 5.7. The failure is a mismatch between the numeric uid of the new process and the owner of the volume; the reproduced commands and the three fixes are listed under the boxes.

5.1.5 Questions to expect

What is the difference between a file descriptor, an open file description and an inode?

A descriptor is a per-process integer index. It points to an open file description, a system-wide structure holding the offset and the open flags. The description points to the inode, which stores the file metadata and the location of the data. `dup` and `fork` create several descriptors for one description (shared offset); two `open` calls create two descriptions for one inode.

What happens to descriptors on fork and on execve?

`fork` duplicates the descriptor table, so parent and child share the open file descriptions. `execve` keeps the descriptors except those with the close-on-exec flag. The shell relies on this to implement redirection and pipelines.

How does the shell implement `command > file 2>&1`?

After `fork` the child opens the file, which receives descriptor 3, then `dup2(3, 1)` and `dup2(1, 2)` make descriptors 1 and 2 refer to it, and the child executes the command. The order is significant: `2>&1 > file` would copy the original destination of descriptor 1 to descriptor 2 first.

Why can a process hit `Too many open files` when the system limit is far from reached?

The error refers to the per-process soft limit, `RLIMIT_NOFILE`, not to `fs.file-max`. The soft limit is inherited from the parent or set by the service manager (`LimitNOFILE`) or container runtime, so a shell limit of 20000 does not apply to a systemd service.

How is a descriptor leak found in a running JVM?

Compare `ls /proc/PID/fd | wc -l` with `/proc/PID/limits`, then group the link targets with `ls -l /proc/PID/fd | awk '{print $NF}' | sort | uniq -c`. Many identical file paths indicate unclosed streams; many `socket:` entries indicate unclosed connections, which `ss -tanp` and `lsof -i` attribute to remote peers.

Can a user delete a file that the user cannot write?

Yes, if the user has write and execute permission on the containing directory, because deletion modifies the directory entry. The sticky bit (for example on `/tmp`) restricts deletion to the owner of the file or directory.

What does the umask do?

The umask removes permission bits from the mode requested at creation. A requested mode of 0666 under umask 022 becomes 0644; a directory requested with 0777 becomes 0755. It does not alter existing files.

What does setuid do and why is it dangerous?

A setuid executable runs with the effective uid of its owner instead of the caller. It allows controlled privilege escalation (for example `passwd`), but any flaw in the program becomes a privilege escalation flaw, so setuid binaries are kept few and small; capabilities offer a narrower alternative.

KEY POINTS

Key points

- A descriptor indexes a per-process table; open file descriptions are shared by `dup` and `fork` and carry the offset and flags; inodes carry the metadata.
- Redirection is `dup2` between `fork` and `execve`; the order of redirections changes the result.
- Three limits bound open files: the soft limit (`EMFILE`), the hard limit and `fs.file-max` (`ENFILE`); the limit of a service comes from its starter, not from the login shell.
- `/proc/PID/fd` and `/proc/PID/limits` diagnose leaks without tools; descriptors with identical targets point to the leak site.
- Permission checks use exactly one class (owner, group or other) and directory bits govern name operations; numeric uids decide, not names.

5.2 File System Internals

A disk is an array of fixed-size blocks without names, ownership or structure. A file system imposes the structure that makes it usable: it maps names to files, files to blocks, tracks free space, enforces permissions and survives power failure without corrupting itself. The design decisions of the file system explain behaviours that production engineers meet daily: why the disk can be full while `du` shows little, why deleting a log file does not free space, why a crash does not destroy a database that used `fsync` and why a container can modify a read-only image.

5.2.1 Concept and mechanism

Blocks, inodes and directories

The file system divides the device into blocks (4096 bytes on ext4 by default). Four kinds of structure account for almost everything:

- The **superblock** records global parameters: block size, block count, inode count, feature flags and the state of the file system (clean or needing recovery).
- **Bitmaps** record which blocks and which inodes are free.
- The **inode table** holds one fixed-size inode per file object (256 bytes on the test image). An inode records type, mode, owner, size, timestamps, link count and the location of the data blocks. It does not record the file name.
- **Data blocks** hold file contents, directory contents and the journal.

A directory is a file whose contents are a list of entries, each pairing a name with an inode number. Names therefore live in directories, and one inode can have several names. The number of names is the link count of the inode; the inode and its blocks are released only when the link count and the count of open descriptors both reach zero.

The ext4 layout

ext4 divides large devices into block groups, each with its own bitmaps and inode table, so that an inode and its data stay physically close. The test image of 32 MiB had a single group. The values in the next figure are the real output of `dumpe2fs` and `debugfs` for that image.

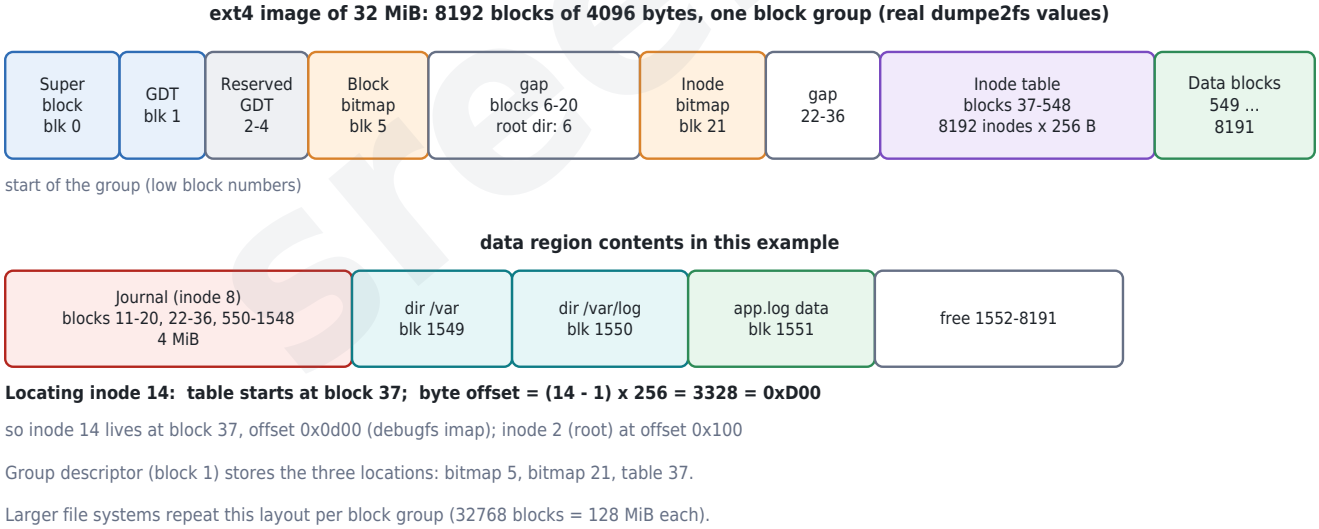


Figure 5.8. The superblock, group descriptors, bitmaps and the 512-block inode table precede the data area; inode 14 is found by arithmetic from the start of the table, without searching.

An ext4 inode locates its data through an extent tree. An extent is a triple of logical block, length and physical block, so a contiguous file needs one entry, and up to four extents fit in the inode itself. Older ext2 and ext3 used block pointers, which were slower for large files. The extent of `app.log` in the experiment was a single entry: logical block 0, length 1, physical block 1551.

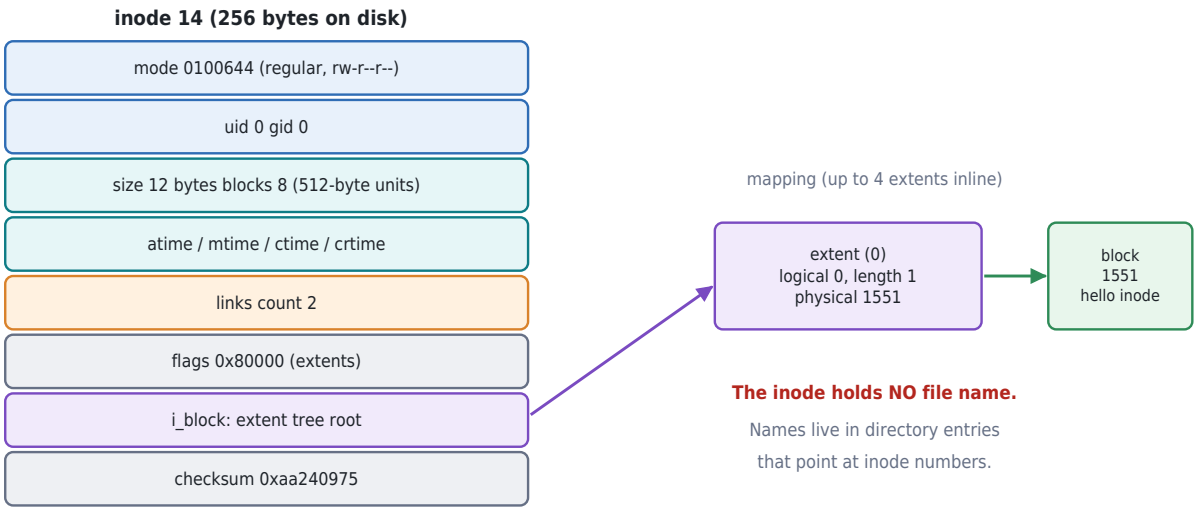


Figure 5.9. The inode stores ownership, mode, size, timestamps, link count and the extent root, but no name; the two extents and the data block shown are the real values for inode 14.

Hard links, symbolic links and special inodes

A hard link is a second directory entry for the same inode. Both names are equal; there is no original. A symbolic link is a separate inode of type link whose content is a path string that is resolved at each access. Hard links cannot cross file systems (inode numbers are only unique within one file system) and cannot point to directories; symbolic links can do both, and they may dangle if the target is removed.

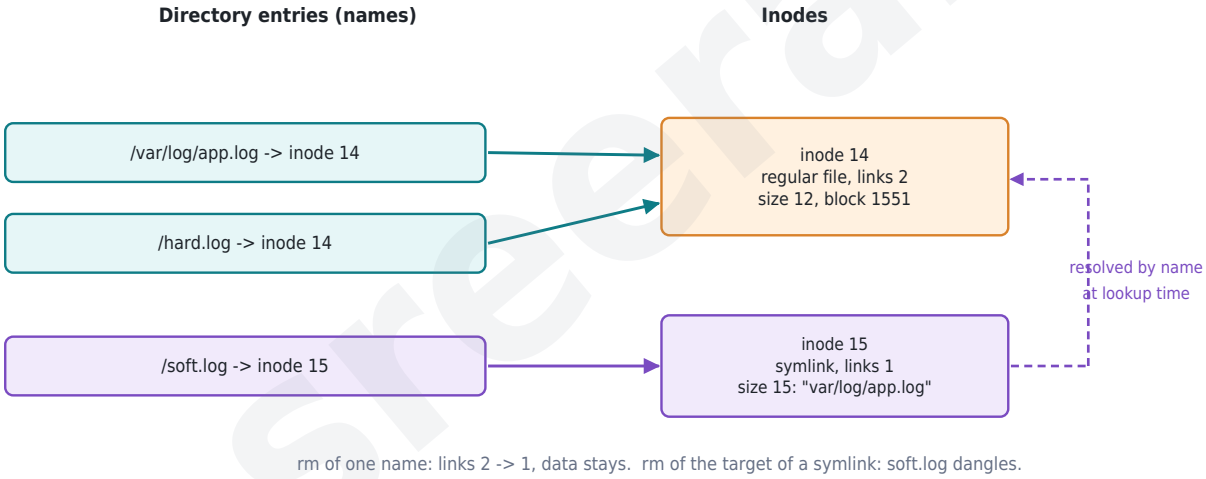


Figure 5.10. Two directory entries for inode 14 give it a link count of 2, while the symbolic link is a distinct inode 15 that stores the path text; removing one hard link leaves the data intact.

Several inode numbers are reserved. Inode 2 is the root directory, inode 8 is the journal and inode 11 is `lost+found`, the directory in which recovery places orphaned files. In the experiment, `lost+found` was inode 11, `/var` inode 12, `/var/log` inode 13 and `app.log` inode 14, the first free numbers in order of creation.

Sparse files

A file may have a size larger than the blocks allocated to it. Unwritten regions read as zeros and occupy no blocks. In the experiment a 1 GiB file with one byte written at offset 512 MiB occupied 8 sectors of 512 bytes, that is one 4 KiB block:

A sparse file: apparent size 1 GiB, allocated 4 KiB — TERMINAL

```
$ truncate -s 1G sparse.bin
$ echo x | dd of=sparse.bin bs=1 seek=536870912 conv=notrunc 2>/dev/null
$ ls -ls sparse.bin
4 -rw-r--r-- 1 root root 1073741824 Oct 18 02:20 sparse.bin
$ du -h sparse.bin; du -h --apparent-size sparse.bin
4.0K    sparse.bin
1.0G    sparse.bin
$ stat -c 'size=%s blocks=%b blocksize=%B' sparse.bin
size=1073741824 blocks=8 blocksize=512
```

The distinction explains differences between `ls -l` and `du`, and why copying a sparse file with a naive tool can fill a disk. Database files, virtual machine images and some core dumps are commonly sparse.

The VFS layer and the file system types

Applications call the same system calls regardless of the file system. The kernel dispatches them through the Virtual File System (VFS), which defines four object types that every file system implements or emulates: the superblock (one mounted instance), the inode, the dentry (a cached name-to-inode mapping) and the file (an open file description). The dentry cache is what makes repeated path lookups cheap.

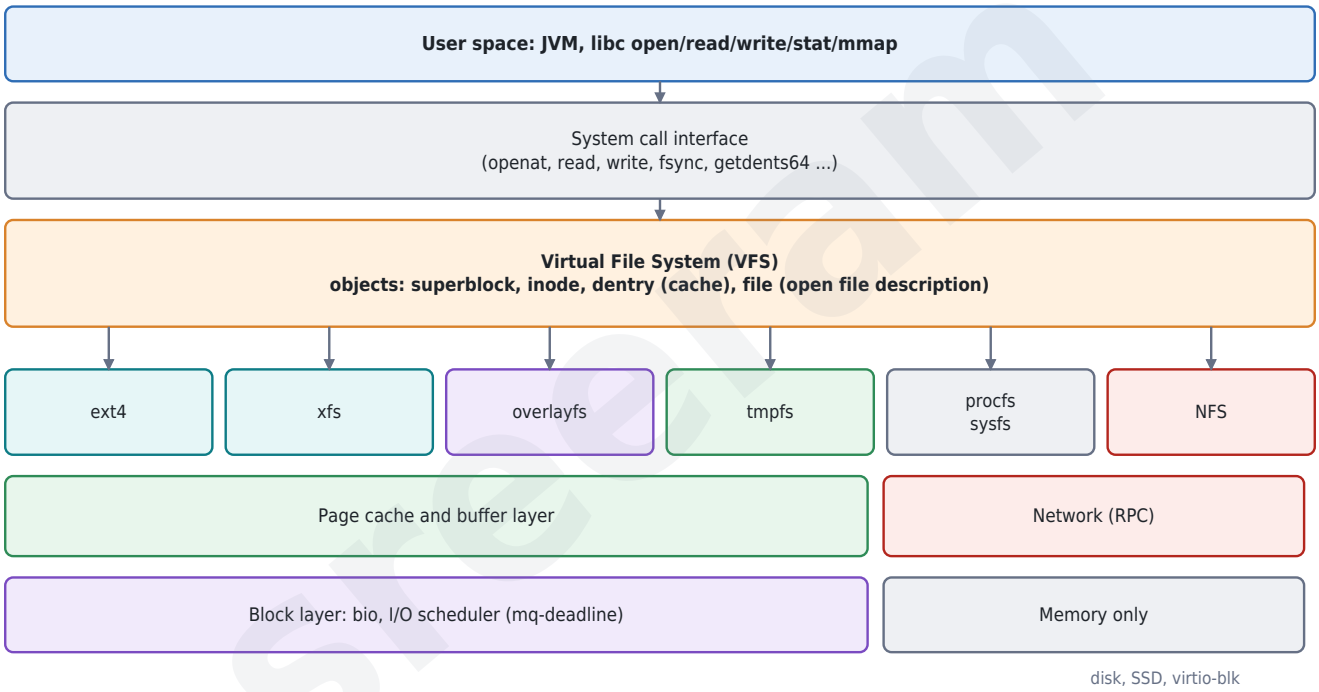


Figure 5.11. System calls enter the VFS, which dispatches to a file system driver; disk-backed file systems then use the page cache and block layer, while tmpfs and procfs use memory only and NFS uses the network.

Type	Characteristics	Typical use
`ext4`	journaling, extents, default on many distributions; fixed inode count chosen at creation	root volumes, general servers
`xfs`	journaling, scales to very large files and parallel I/O, dynamic inode allocation; can grow but not shrink	databases, large volumes, default on several enterprise distributions
`overlayfs`	union of read-only lower layers and a writable upper layer; copy-on-write per file	container images and writable layers
`tmpfs`	memory-backed, size-limited, contents lost at unmount or reboot; swappable	`/dev/shm`, `/run`, scratch space
`proc`, `sysfs`	kernel data exposed as files; no storage	diagnostics (`/proc/PID`, `/sys/block`)
`NFS`	remote file system over RPC; semantics depend on mount options	shared storage, legacy integrations

File systems that a Java engineer meets in production.

On the build machine `/proc/filesystems` listed ext2, ext3, ext4, xfs, overlay, tmpfs, squashfs, fuse and erofs among others, and the root file system was ext4 mounted on `/dev/vda` with options `rw,relatime,discard`.

Journaling and crash consistency

Creating a file changes several structures: the inode bitmap, the inode, the directory block and possibly the block bitmap. A power failure after some of these writes but before others leaves the file system inconsistent. Before journaling, the remedy was a full `fsck` scan at boot, which took hours on large disks. A journaling file system first writes a description of the planned metadata changes to a sequential log (the journal), marks the transaction complete with a commit record and only then writes the blocks to their home locations. After a crash, recovery replays committed transactions and discards incomplete ones in seconds.

ext4 offers three data modes, selected by the mount option `data=`:

- `data=ordered` (default): only metadata is journaled, but file data blocks are written to disk before the metadata that refers to them is committed. After a crash a file never exposes stale content from a previously deleted file, though recent appends may be missing.
- `data=writeback`: metadata is journaled and data is written in any order; faster, but a crash can expose old data inside a file whose size was updated.
- `data=journal`: data and metadata are both journaled; every block is written twice, which doubles write traffic but gives the strongest guarantee.

The kernel accepted all three modes on test images. After remounting with `data=journal`, `/proc/mounts` reported `rw,relatime,nodioread_nolock,nodelalloc,data=journal`, showing that the mode also disables delayed allocation; the writeback mode appeared as `data=writeback`.

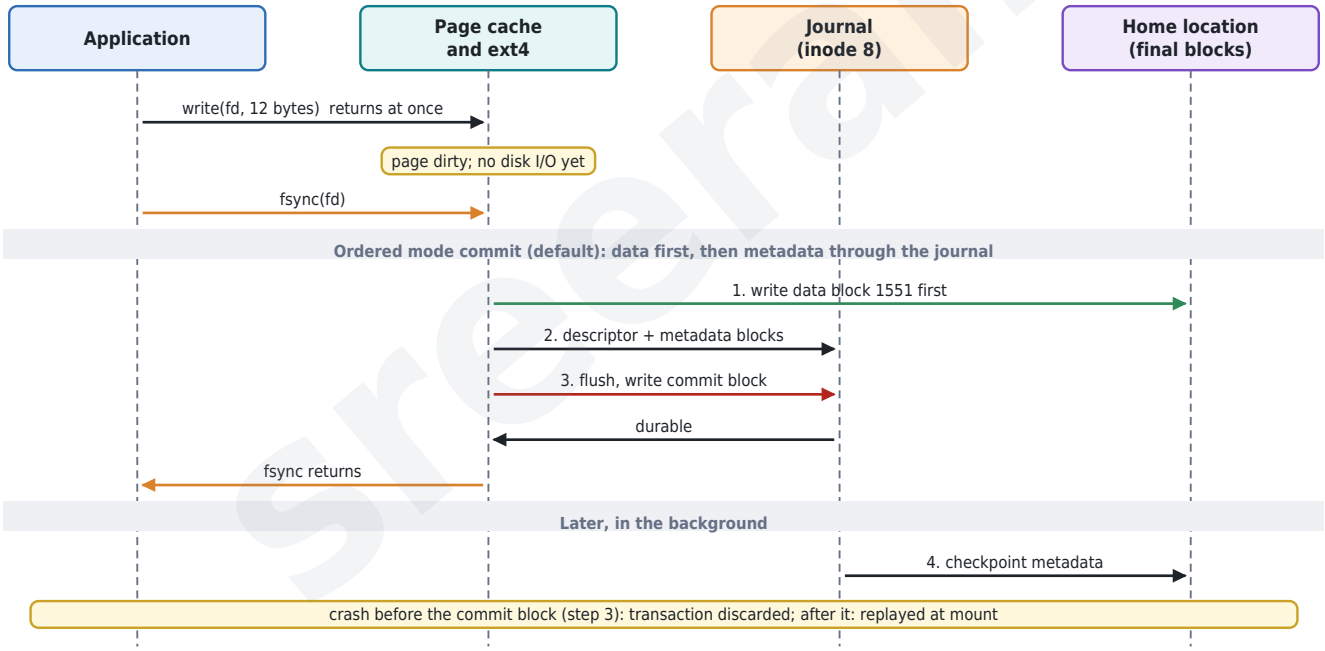


Figure 5.12. An fsync in ordered mode writes the data block first, then the journal descriptor and metadata, then a flushed commit block; only after the commit does fsync return, and the home-location copy of metadata happens later.

The journal protects the structure of the file system, not the application data. Metadata consistency after a crash does not mean that a multi-step application update is atomic. Applications that need atomicity use `fsync` at the right points, write-ahead logs or the atomic `rename` of a fully written temporary file; the next chapters return to this point.

Mounting and /etc/fstab

A file system becomes visible when it is attached to a directory with `mount`. The mount table is a tree: after the mount, the directory shows the root of the mounted file system, and the inode numbering restarts there. `/proc/mounts` and `findmnt` display the live table. The file `/etc/fstab` lists persistent mounts with device, mount point, type, options and the dump and fsck pass numbers. On the build machine `/etc/fstab` was an unconfigured placeholder because the machine was a minimal virtual environment; on a normal server it also names options such as `noatime` (avoid a metadata write on each read), `nofail` and `x-systemd.device-timeout`, whose absence is a cause of boot failures when a data disk is missing.

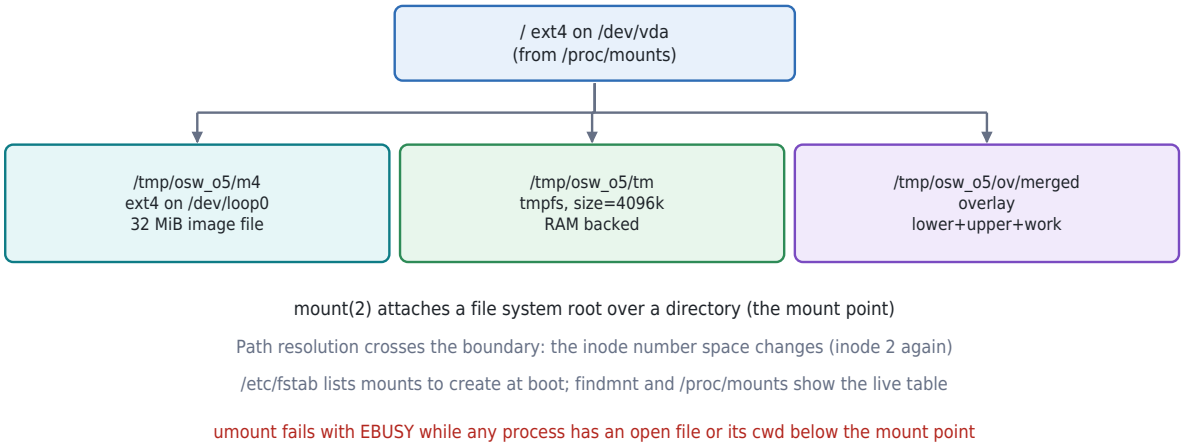


Figure 5.13. Three file systems are attached below the root file system in the experiment; crossing a mount point changes the device and restarts inode numbering.

5.2.2 Worked trace: resolving /var/log/app.log

The kernel resolves a path one component at a time, starting at the root dentry for an absolute path and at the current directory for a relative one. For every directory it checks that the caller has execute permission, looks up the next name in the directory contents and loads the inode. The trace below used the real structures of a 32 MiB test image; `debugfs` supplied the numbers.

The facts used in the trace TERMINAL

```
$ debugfs -R "ls -l <2>" img4.ext4
      2   40755 (2)      0      0    4096 18-Oct-2026 02:20 .
      2   40755 (2)      0      0    4096 18-Oct-2026 02:20 ..
     11   40700 (2)      0      0   16384 18-Oct-2026 02:20 lost+found
     12   40755 (2)      0      0    4096 18-Oct-2026 02:20 var
$ for p in /var /var/log /var/log/app.log; do \
    debugfs -R "stat $p" img4.ext4 | grep -E "^Inode|^\(0\)"; done
Inode: 12   Type: directory    Mode:  0755   Flags: 0x80000
(0):1549
Inode: 13   Type: directory    Mode:  0755   Flags: 0x80000
(0):1550
Inode: 14   Type: regular    Mode:  0644   Flags: 0x80000
(0):1551
$ debugfs -R "imap <14>" img4.ext4
Inode 14 is part of block group 0
        located at block 37, offset 0x0d00
$ debugfs -R "stat <2>" img4.ext4 | grep -E "^\(0\)"
(0):6
```

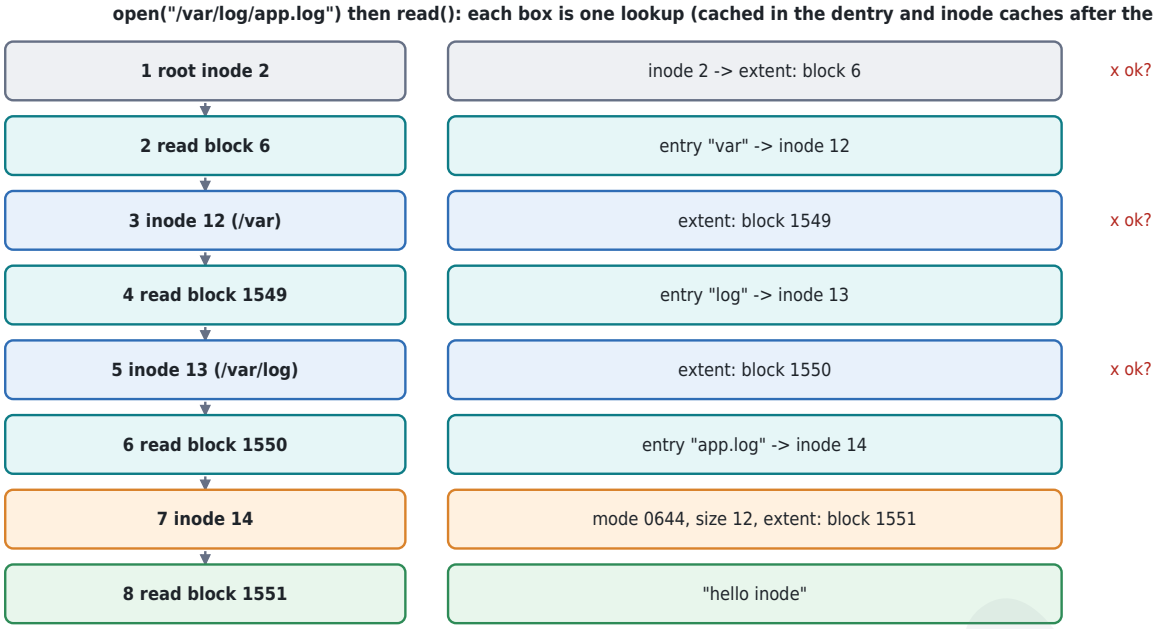


Figure 5.14. Eight lookups resolve the path: four reads of inodes (2, 12, 13, 14) interleaved with four reads of directory or data blocks, with an execute-permission check at each directory.

1. The kernel starts at the root inode, number 2. Its extent shows the directory contents at block 6. The root directory block lists `.`, `..`, `lost+found` (inode 11) and `var` (inode 12).
2. Inode 12 is located in the inode table. Group 0 has its inode table at block 37 and the inode size is 256 bytes, so inode 12 is at byte offset `(12 - 1) x 256 = 2816` within the table. Its mode is `0755` directory, so execute permission is checked for the caller. Its extent points to block 1549.
3. The block 1549 holds the entries of `/var`. A raw dump (`debugfs -R "bd 1549"`) shows the entry `log` with inode number 13, in little-endian bytes `0d 00 00 00`, a record length of `0x0fdc` and the name length 3.
4. Inode 13 (`/var/log`) is read; its directory block is 1550 and it holds `app.log` with inode 14.
5. Inode 14 is at block 37, byte offset `0x0d00 = 13 x 256`. Mode `0644`, size 12 and one extent at block 1551. The file name lookup is complete and the kernel allocates an open file description and a descriptor.
6. `read` then fetches block 1551 through the page cache and returns the 12 bytes `hello inode`.

On a running system, almost all of these reads are served from the dentry cache and inode cache in memory; the disk is touched only on a cold path. The cost of deep paths is therefore dominated by cache misses, not by the number of components.

5.2.3 Hands-on

Building and inspecting a file system without a spare disk

A regular file can serve as a block device through the loop driver, which makes real file systems available for experiments without touching any production disk. The sequence below creates, mounts and inspects an ext4 image. The commands require root; paths are shortened.

Create an ext4 image, mount it and read the block group layout TERMINAL

```
$ truncate -s 32M img.ext4 && mkfs.ext4 -q -F -L osw img.ext4
$ mount -o loop img.ext4 /mnt/t
$ dumpe2fs -h img.ext4 | grep -E "features|Inode count|Block count|Block size|Inode size|Journal"
Filesystem features:      has_journal ext_attr resize_inode dir_index filetype needs_recovery
                          extent
                          64bit flex_bg sparse_super large_file huge_file dir_nlink
                          extra_isize metadata_csum
Inode count:              8192
Block count:              8192
Block size:               4096
Inode size:               256
Journal inode:            8
Journal features:         journal_64bit journal_checksum_v3
$ dumpe2fs img.ext4 | sed -n '/^Group 0/,$p' | head -8
```

Create an ext4 image, mount it and read the block group layout (continued) TERMINAL

```
Group 0: (Blocks 0-8191) csum 0x4d71 [ITABLE_ZEROED]
  Primary superblock at 0, Group descriptors at 1-1
  Reserved GDT blocks at 2-4
  Block bitmap at 5 (+5), csum 0x47f48a4a
  Inode bitmap at 21 (+21), csum 0x1ffe63d9
  Inode table at 37-548 (+37)
  6640 free blocks, 8178 free inodes, 4 directories, 8178 unused inodes
  Free blocks: 1552-8191
```

The feature list contains `needs_recovery` because the image was inspected while mounted with a journal that had not been checkpointed. The journal itself is inode 8, a 4 MiB file whose extents were `(0-9):11-20, (10-24):22-36, (25-1023):550-1548`, which shows that a file system places its own metadata files in the gaps between the fixed structures.

Hard and symbolic links

Two names, one inode; one name that stores a path TERMINAL

```
$ mkdir -p var/log && echo "hello inode" > var/log/app.log
$ ln var/log/app.log hard.log && ln -s var/log/app.log soft.log
$ ls -li . var/log
.:
total 24
14 -rw-r--r-- 2 root root    12 Oct 18 02:14 hard.log
11 drwx------ 2 root root 16384 Oct 18 02:14 lost+found
15 lrwxrwxrwx 1 root root    15 Oct 18 02:14 soft.log -> var/log/app.log
12 drwxr-xr-x 3 root root  4096 Oct 18 02:14 var

var/log:
total 4
14 -rw-r--r-- 2 root root 12 Oct 18 02:14 app.log
$ stat var/log/app.log | head -4
  File: var/log/app.log
  Size: 12              Blocks: 8          IO Block: 4096   regular file
Device: 7,0     Inode: 14          Links: 2
Access: (0644/-rw-r--r--)  Uid: (    0/    root)   Gid: (    0/    root)
```

`hard.log` and `app.log` share inode 14 with a link count of 2. The symbolic link is inode 15 with size 15, the length of the stored text `var/log/app.log`. Note that `Blocks: 8` counts 512-byte units: the 12-byte file occupies one 4096-byte block.

An overlay file system

Container images use overlayfs. The experiment mounts a lower directory with two files, adds an empty upper directory and a work directory, then modifies the merged view.

Copy-up and whiteout in overlayfs TERMINAL

```
$ mount -t overlay overlay -o lowerdir=ov/lower,upperdir=ov/upper,workdir=ov/work ov/merged
$ echo "changed" >> ov/merged/a.txt; rm ov/merged/b.txt; echo new > ov/merged/c.txt
$ ls -l ov/upper ov/merged
ov/merged:
total 8
-rw-r--r-- 1 root root 19 Oct 18 02:14 a.txt
-rw-r--r-- 1 root root  4 Oct 18 02:14 c.txt

ov/upper:
total 8
-rw-r--r-- 1 root root   19 Oct 18 02:14 a.txt
c--------- 2 root root 0, 0 Oct 18 02:14 b.txt
-rw-r--r-- 1 root root    4 Oct 18 02:14 c.txt
$ cat ov/lower/a.txt
from lower
$ stat -c '%n %F %t:%T' ov/upper/b.txt
ov/upper/b.txt character special file 0:0
```

The first write to `a.txt` copied the entire file into the upper layer (the copy-up) and appended there, leaving the lower file untouched. The removal of `b.txt` created a whiteout, a character device with device number 0:0, which hides the lower file. Two consequences matter for operations. Writing a few bytes to a large file in an image copies the whole file on first write, which causes latency spikes for log files or databases placed in the container layer. Deleting a file that came from the image never frees space, so large data and logs belong on a volume, not in the writable layer.

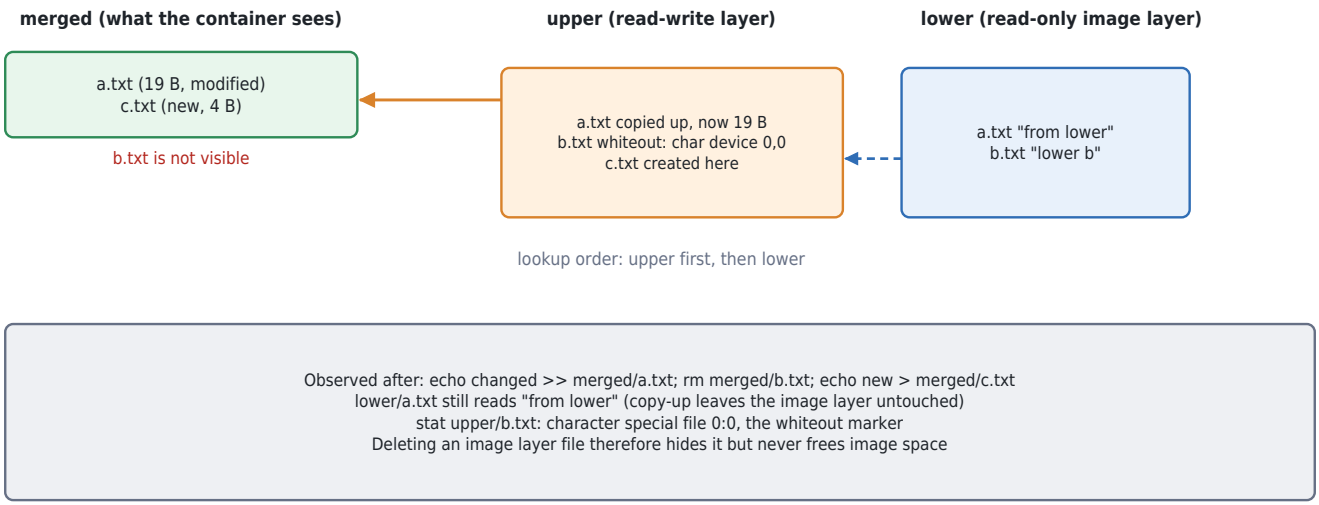


Figure 5.15. Reads fall through the upper layer to the lower layers, writes copy the file up, and deletions leave a whiteout marker in the upper layer.

Reading the journal

The journal of a freshly written image (typical) TERMINAL

```
$ debugfs -R "logdump" img4.ext4
Journal starts at block 1, transaction 2
Found expected sequence 2, type 1 (descriptor block) at block 1
Found expected sequence 2, type 2 (commit block) at block 10
No magic number at block 11: end of journal.
```

A transaction of sequence 2 consists of a descriptor block at journal block 1, the metadata blocks that follow it and a commit block at block 10. If the commit block had not reached the disk before a crash, replay would ignore the whole transaction. The journal size of 4 MiB (1024 blocks) bounds how much unwritten metadata can be pending.

5.2.4 Real-world incidents

Incident: No space left on device with free blocks (inode exhaustion)

Scenario. A batch component of an order platform writes one small marker file per processed event into a directory on an ext4 volume. After several weeks the writes begin to fail although monitoring shows the volume at 4 percent block usage.

Symptoms. Java throws `java.io.IOException: No space left on device` from `File.createNewFile`, and shell commands such as `touch` fail with the same message. `df -h` shows ample free space. Writes to existing files still succeed.

Diagnosis. When `df -h` disagrees with the error, the next command is `df -i`, which reports inode usage. The reproduction used an image formatted with only 256 inodes so that the condition appears within seconds.

Reproduction on a 16 MiB ext4 image with 256 inodes TERMINAL

```
$ mkfs.ext4 -q -N 256 -F img1.ext4 && mount -o loop img1.ext4 /mnt/t
$ df -h /mnt/t; df -i /mnt/t
Filesystem      Size  Used Avail Use% Mounted on
/dev/loop0       12M   24K   11M   1% /mnt/t
Filesystem     Inodes IUsed IFree IUse% Mounted on
/dev/loop0        256    11   245    5% /mnt/t
$ cd /mnt/t && mkdir t && cd t && for i in $(seq 1 300); do : > f$i || break; done
bash: f245: No space left on device
$ cd ..; df -h .; df -i .
Filesystem      Size  Used Avail Use% Mounted on
/dev/loop0       12M   28K   11M   1% /mnt/t
Filesystem     Inodes IUsed IFree IUse% Mounted on
/dev/loop0        256   256     0  100% /mnt/t
$ touch x
touch: cannot touch 'x': No space left on device
```

The block usage remained at 1 percent while inode usage reached 100 percent. The file system had 11 inodes in use at creation (the reserved inodes and `lost+found`) and refused the 245th new file.

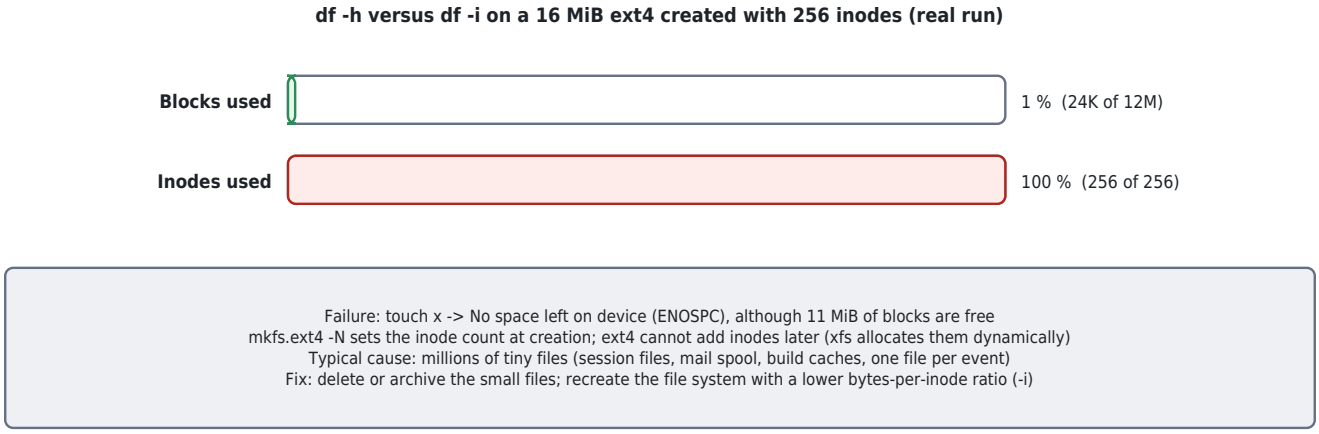


Figure 5.16. Block usage stays near 1 percent while inode usage reaches 100 percent; the kernel reports the same ENOSPC error for both conditions.

On a production volume the directory containing the millions of files is found with `find /data -xdev -type f -printf '%h\n' | sort | uniq -c | sort -rn | head`, which counts files per directory. The `-xdev` option prevents the search from crossing into other mounts.

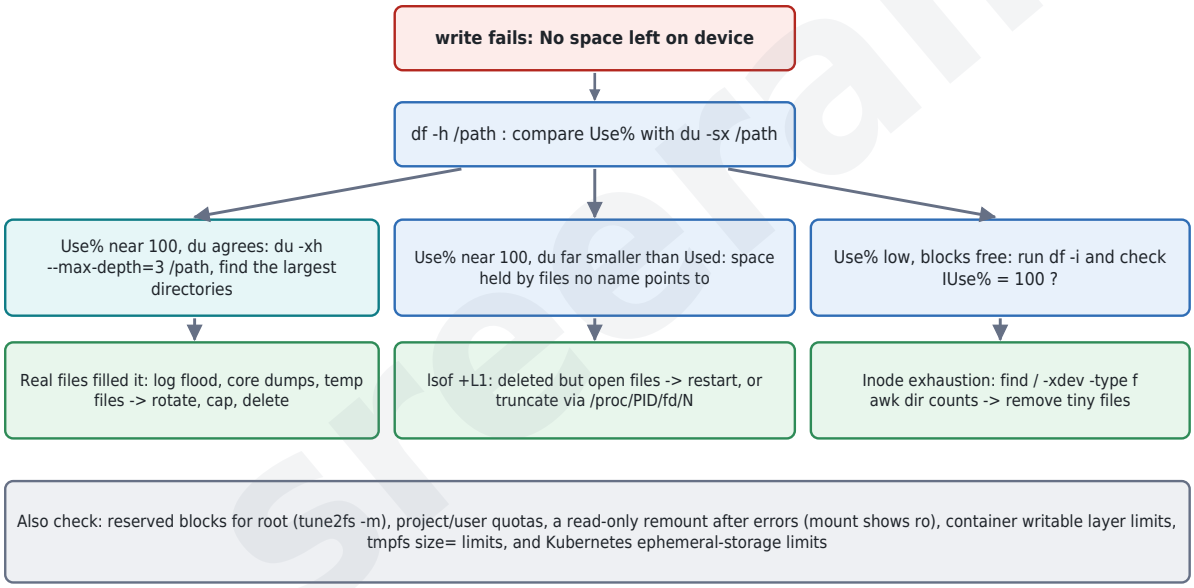


Figure 5.17. The same error message has four causes; `df -h`, `du`, `df -i` and `lsof +L1` separate them in four commands.

Root cause. ext4 fixes the number of inodes when the file system is created (by default one per 16 KiB of capacity), so a workload with many tiny files exhausts inodes long before blocks. The error code `ENOSPC` is shared by both conditions.

Fix. Immediate: delete or archive the small files (removing a directory with millions of entries is faster with `find dir -type f -delete` than with shell expansion, which fails with `Argument list too long`). Permanent: change the design so that events are written to one append-only file, to a database or to an object store; or recreate the file system with more inodes (`mkfs.ext4 -i 4096`) or use xfs, which allocates inodes dynamically.

Prevention. Alert on `node_filesystem_files_free` (or `df -i`) next to the block alert, with the same thresholds. Never use the file system as a key-value store with one file per key at scale.

Incident: hung process on a stale NFS mount

Scenario. A report generator reads input files from an NFS share mounted at `/mnt/share`. The storage server is restarted for maintenance and does not return for twenty minutes.

Symptoms. The service stops responding. Thread dumps show request threads inside `FileInputStream.readBytes` or `File.exists`. `kill -15` has no effect, the pod does not terminate gracefully and the load average rises while CPU usage stays near zero. Even `ls /mnt/share` hangs in the terminal.

Diagnosis. The decisive observation is the process state. Processes waiting for an NFS reply are in uninterruptible sleep, shown as `D`. The output below is typical for this failure; it was not reproduced because no NFS server exists on the build machine.

Typical diagnosis of a hard NFS mount that is not responding TERMINAL

```
$ ps -eo pid,stat,wchan:32,cmd | awk '$2 ~ /^D/'
 2210 D    nfs_wait_on_request   java -jar app.jar
 2305 D    rpc_wait_bit_killable ls /mnt/share
$ dmesg | tail -2
nfs: server nas01 not responding, still trying
nfs: server nas01 not responding, still trying
$ grep nfs /proc/mounts
nas01:/export/reports /mnt/share nfs4 rw,relatime,vers=4.2,hard,timeo=600,retrans=2 0 0
```

The `D` state, the `wchan` function names and the kernel message together identify the file system as the blocked layer. Tools that touch the mount, such as `df` and `lsof`, hang too; `df -l` (local file systems only) and `mount` are safe.

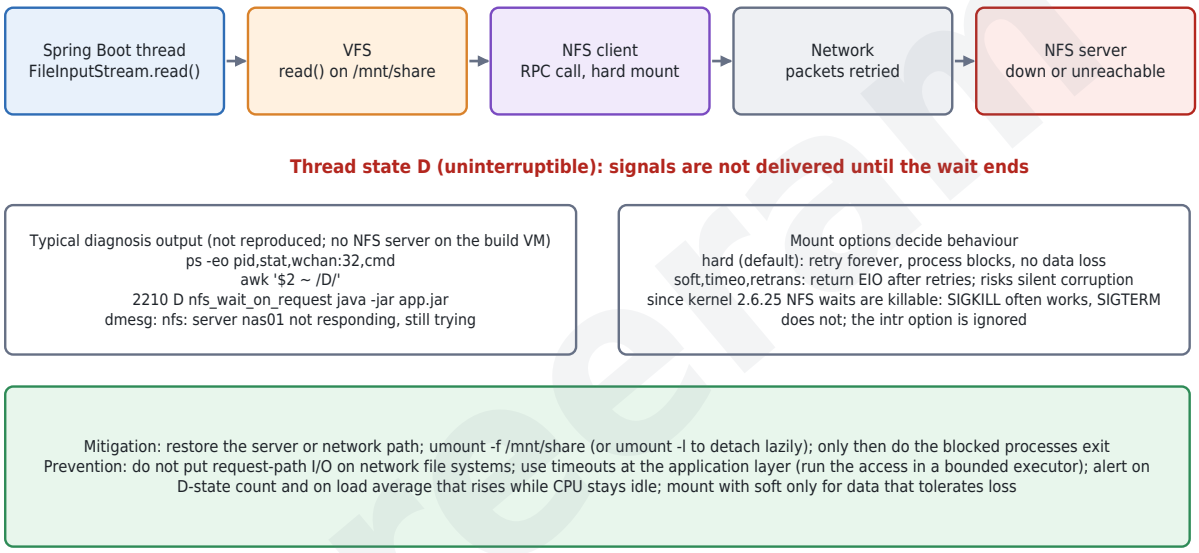


Figure 5.18. A hard NFS mount retries indefinitely, so every thread that touches the share is parked in uninterruptible sleep until the server returns.

Root cause. With the default `hard` option the NFS client retries a request forever so as not to lose data, and the blocked task cannot be interrupted by ordinary signals. The service placed network file access on its request path without a timeout.

Fix. Immediate: restore the server or its network path; blocked tasks then continue. If the server cannot return, `umount -f /mnt/share` aborts pending requests, and `umount -l` detaches the mount from the tree lazily. Permanent: keep request-path I/O off network file systems; perform the access in a bounded executor with a timeout so that a hang consumes one worker, not all threads; use `soft` with explicit `timeo` and `retrans` only for data that tolerates failure, because a soft mount can return `EIO` in the middle of a write and corrupt output.

Prevention. Alert on the count of processes in state `D` and on a load average that rises while CPU is idle. Mount shares with a health check, and make the readiness probe verify the dependency with a timeout rather than a plain `stat`.

5.2.5 Questions to expect

What does an inode contain, and what does it not contain?

It contains the file type, mode, owner, size, timestamps, link count and the locations of the data blocks (extents on ext4). It does not contain the file name; names are entries in directories that map a name to an inode number.

What is the difference between a hard link and a symbolic link?

A hard link is another directory entry for the same inode, indistinguishable from the original and valid only within one file system. A symbolic link is a separate inode containing a path that is resolved at access time; it may cross file systems and may dangle.

Why does deleting a large log file sometimes not free disk space?

Deletion removes the name and decrements the link count. The inode and blocks remain while any process holds the file open, so space is freed only when the last descriptor closes. `lsof +L1` lists such files.

How can a file system have free blocks and still return ENOSPC?

It may have run out of inodes (`df -i`), reached a quota or the reserved-block threshold, or the file may live on a small tmpfs or overlay. Inode exhaustion arises with very many small files.

What problem does journaling solve, and what does it not solve?

It keeps file system metadata consistent after a crash by replaying committed transactions, which avoids a full `fsck`. It does not make application updates atomic and, in the default ordered mode, does not guarantee that recently written data survives without `fsync`.

Describe the steps the kernel takes to open /var/log/app.log.

It starts at the root inode, checks execute permission on each directory, looks up the next name in the directory contents (using the dentry cache when possible), loads the inode and repeats until the final component, then allocates an open file description and a descriptor. Each directory lookup reads at most one inode and one directory block on a cold cache.

How does overlayfs support container images?

It presents a union of read-only lower layers and a writable upper layer. A first write copies the file into the upper layer, deletions are recorded as whiteouts and lower layers stay unchanged, so many containers share one image. Heavy writes belong on volumes because of the copy-up cost.

What is a sparse file?

A file whose size exceeds its allocated blocks because unwritten ranges are holes that read as zeros. `ls -l` shows the apparent size, `du` the allocated size.

What is the effect of a hard NFS mount when the server disappears?

Requests are retried indefinitely; threads touching the mount enter uninterruptible sleep and cannot be terminated by ordinary signals until the server returns or the mount is forced off.

KEY POINTS

Key points

- A file system maps names to inodes (directories), inodes to blocks (extents) and tracks free space in bitmaps; the inode count is fixed at creation on ext4.
- Path resolution is a sequence of directory lookups with permission checks; caches make it cheap, and the real cost is on a cold path.
- Hard links share an inode; the data is freed only when the link count and the open count reach zero, which explains deleted-but-open files.
- The journal protects metadata consistency; durability of application data still requires `fsync`, and ordered mode writes data before the metadata that refers to it.
- `df -h`, `df -i`, `du` and `lsof +L1` separate the four causes of ENOSPC; overlayfs copy-up and whiteouts explain container storage behaviour.

Sterna

5.3 Storage Devices and the Block Layer

A file system decides which blocks to read or write; the block layer and the device decide how fast that happens. The latency of a storage device is three to five orders of magnitude above that of memory, and the difference between a service that answers in 5 milliseconds and one that answers in 500 is frequently a queue in front of a disk. This chapter describes the device classes, the kernel path to them, the aggregation layers (RAID and LVM) and the counters that measure utilisation, so that a statement such as "the disk is the bottleneck" can be proved or refuted from `/proc/diskstats` with arithmetic.

5.3.1 Concept and mechanism

Hard disks and solid-state devices

A hard disk drive (HDD) reads and writes through a head that moves over spinning platters. A random access pays a seek (moving the head, typically a few to ten milliseconds) and a rotational delay (on average half a revolution: 4.17 ms at 7200 rpm). A sequential access pays these costs once and then streams at the platter speed. An HDD therefore delivers on the order of one hundred random operations per second but a hundred or more megabytes per second sequentially. The kernel and databases are designed to turn random access into sequential access wherever possible: merging adjacent requests, ordering by address, appending to logs.

A solid-state drive (SSD) stores data in flash memory without moving parts. Reads of a page take tens of microseconds and many chips work in parallel, so random reads are almost as fast as sequential reads and tens of thousands of operations per second are normal. Flash has two constraints that shape its behaviour. A page cannot be overwritten in place: it must be erased as part of a much larger erase block, so the controller remaps logical blocks to fresh pages (the flash translation layer) and later reclaims stale pages by garbage collection. The extra writes this causes are called write amplification, and the pauses it causes appear as latency spikes under sustained random writes. The `discard` (TRIM) command tells the device which blocks the file system no longer uses, so that reclamation is cheaper; the root file system of the build machine was mounted with `discard`.

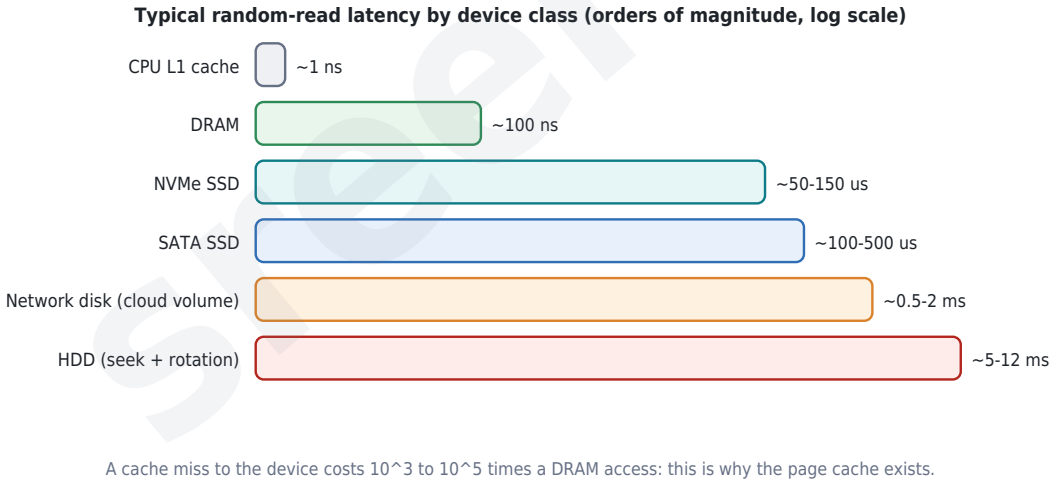


Figure 5.19. The access latency of a cloud volume or HDD is thousands of times that of DRAM, which is the reason for the page cache and for write-ahead logs; the values are orders of magnitude, not measurements of the build machine.

NVMe devices attach to the PCIe bus and support many hardware queues (up to 65535 queues of up to 65536 entries in the specification), so several CPUs can submit requests without sharing a lock. A SATA disk exposes one queue of 32 commands. A virtual machine disk (virtio-blk) is a software queue whose speed is set by the hypervisor and the backing storage; cloud volumes add network latency and a provisioned operations-per-second budget that the volume enforces by throttling.

The path through the kernel

A `write` to a file reaches the block layer only after the page cache and the file system. The block layer turns file system requests into `bio` structures (a list of memory pages and a target sector range), merges adjacent bios into larger requests, places them into queues and hands them to the driver. The I/O scheduler decides the order within a queue. The available schedulers are visible per device in `/sys/block/DEV/queue/scheduler`; the active one is in brackets.

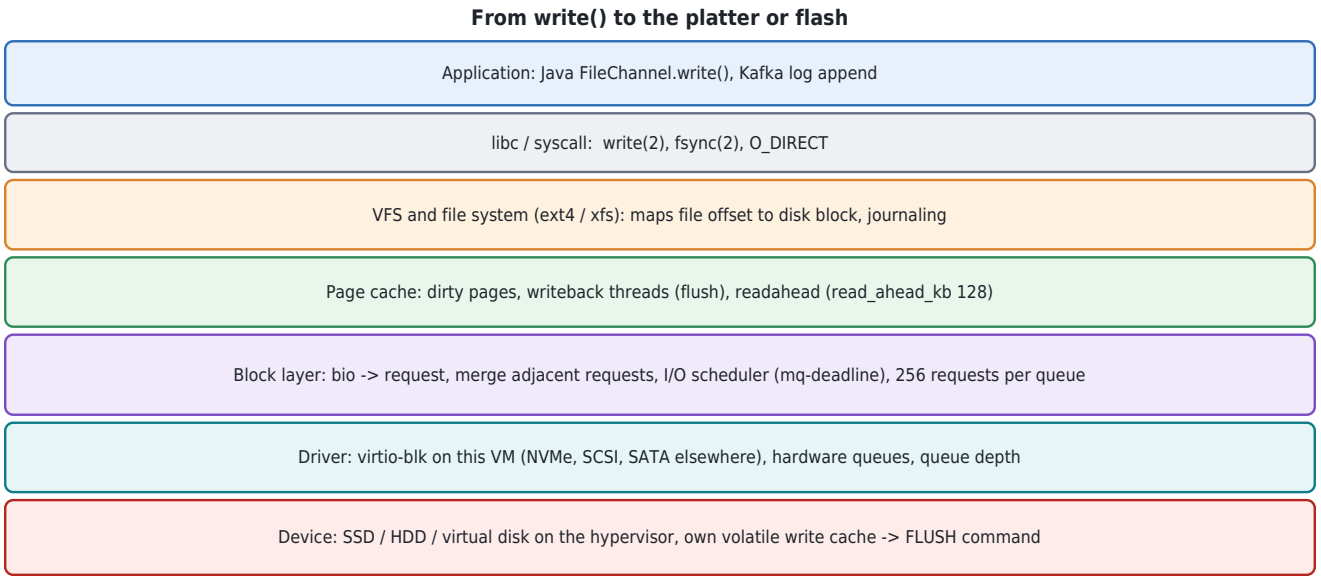


Figure 5.20. Each layer adds a queue or a cache between the application and the device; the numbers in the boxes (128 KiB readahead, 256 requests per queue, write-back device cache) were read from the build machine.

Scheduler	Behaviour	Typical choice
`none`	no reordering, FIFO into the hardware queues	NVMe and virtual disks where the device or host does the scheduling
`mq-deadline`	sorts by sector, gives every request an expiry (reads 500 ms, writes 5000 ms by default) so that writes cannot starve reads	SATA SSD and HDD, default on many systems; active on the build machine
`kyber`	adapts queue depth to meet read and write latency targets	fast devices with latency goals
`bfq`	proportional share per process or cgroup, favours interactive latency	desktops, mixed HDD workloads

I/O schedulers in the multi-queue block layer.

The build machine reported the following parameters for its virtual disk `vda`:

Block device parameters (build machine, typical for a virtio disk) TERMINAL

```
$ cd /sys/block/vda/queue
$ cat scheduler rotational nr_requests read_ahead_kb logical_block_size physical_block_size
none [mq-deadline] kyber bfq
1
256
128
512
4096
$ cat iosched/read_expire iosched/write_expire write_cache
500
5000
write back
```

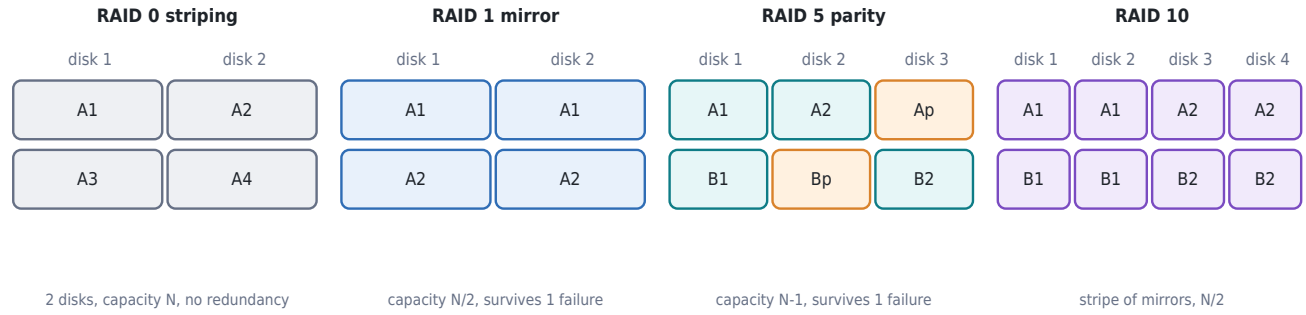
Three entries need interpretation. `rotational` is 1 although the disk is virtual: the flag is a default of the virtio driver and must not be used to decide whether a disk is an SSD. `write_cache` set to `write back` means that the device has a volatile cache, so the kernel must send explicit flush commands to make data durable; that cost appears in the measurement of `fsync` in the next chapter. `read_ahead_kb` of 128 means that after sequential reads the kernel prefetches up to 128 KiB.

Queue depth and parallelism

The queue depth is the number of requests outstanding at the device. A depth of one serialises operations and wastes the parallelism of an SSD; a very deep queue raises throughput but also raises latency, because a newly submitted request waits behind everything already queued. The relation is Little's law: the average number of requests in the system equals the arrival rate multiplied by the average time each spends in the system. A device that sustains 200 writes per second with a mean latency of 38 ms must have on average 7.6 requests in flight, which matches the measurement below.

RAID and LVM

Redundant arrays of independent disks combine several devices for capacity, speed and failure tolerance. A hardware RAID controller presents the array as one disk, often with a battery-backed write cache; software RAID (`md`) and file systems such as ZFS and btrfs implement the same ideas in the kernel.

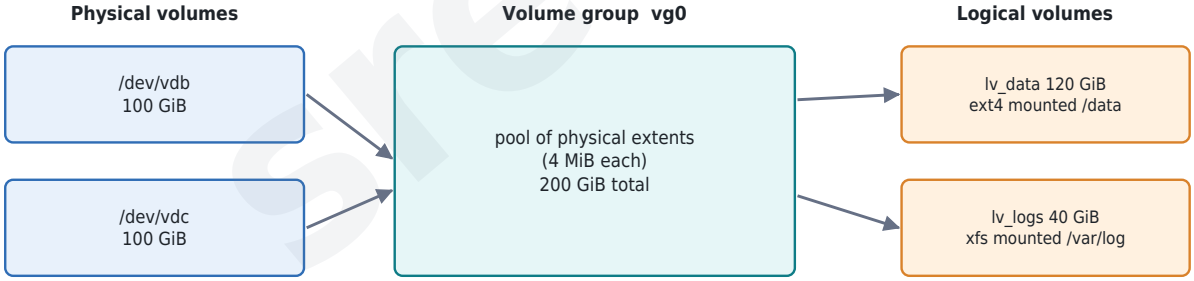


Write penalty: RAID 5 small write = read old data + read old parity + write data + write parity (4 I/Os)
RAID 1 and 10 write each block to every mirror (2 I/Os), reads can use either copy
Hardware RAID controllers add a battery-backed write cache; software RAID (md) uses the host page cache
RAID is availability, not backup: deletes and corruption replicate instantly

Figure 5.21. The four common levels differ in capacity, failure tolerance and write cost; RAID 5 turns a small write into four disk operations because parity must be read and updated.

RAID is availability, not backup: a deletion, a corruption or an application bug is replicated to every copy immediately. For database volumes in the cloud the relevant redundancy is usually provided by the storage service, and the guest only sees one device.

The Logical Volume Manager (LVM) inserts a layer between disks and file systems. Physical volumes are pooled into a volume group, from which logical volumes of arbitrary size are carved. A logical volume can be extended while mounted and followed by `resize2fs` (ext4) or `xfs_growfs` (xfs, which cannot shrink). Snapshots use copy-on-write extents and are the basis of consistent backups.



lvextend -L +20G /dev/vg0/lv_data then resize2fs (ext4, online) or xfs_growfs (xfs, grow only)
Snapshots use copy-on-write extents; striping and mirroring are available per LV
Not installed on the build VM (no lvs): the commands above are shown for reference, not run

Figure 5.22. Logical volumes are carved from a pool of extents contributed by several physical volumes, which is why a volume can grow without moving data; the LVM tools were not installed on the build machine, so the commands are shown for reference only.

5.3.2 Worked trace: utilisation and latency from /proc/diskstats

Every block device has one line in `/proc/diskstats` (and a shorter one in `/sys/block/DEV/stat`). The fields, after the major number, minor number and device name, are cumulative counters since boot. The tools `iostat` and `sar` merely subtract two samples; they were not installed on the build machine, so the calculation was done by hand, which is also the clearest way to learn the meaning of each field.

Field	Name	Meaning
1	reads completed	read requests finished
3	sectors read	in units of 512 bytes
4	ms reading	summed over all requests, so it can exceed wall time
5	writes completed	write requests finished
7	sectors written	in units of 512 bytes
8	ms writing	summed over all requests
9	I/Os in progress	instantaneous, not cumulative
10	ms doing I/O	wall time during which at least one request was in flight
11	weighted ms	requests in flight multiplied by time, accumulated

The fields used in the calculation (kernel 6.x documentation, Documentation/admin-guide/iostats).

The workload was `dd if=/dev/zero of=big bs=1M count=150 conv=fdatasync`, which writes 150 MiB and flushes it once. The counters of `vda` were captured immediately before and after; the elapsed time measured by the shell was 214.8 ms.

Before and after samples (the fields of interest, from the build machine) TERMINAL

```
$ grep ' vda ' /proc/diskstats        # before
 254       0 vda 3903 3578 365730 12355 5041 336 76640 1557 0 4352 14716 255 0 18240 678 1248 124
$ dd if=/dev/zero of=rootd/big bs=1M count=150 conv=fdatasync
157286400 bytes (157 MB, 150 MiB) copied, 0.208802 s, 753 MB/s
$ grep ' vda ' /proc/diskstats        # after
 254       0 vda 3905 3578 365874 12355 5083 336 383880 3153 0 4472 16394 257 0 18272 679 1249 205
```

/proc/diskstats for vda around dd bs=1M count=150 conv=fdatasync (interval 214.8 ms)

field	meaning	before	after	delta
5	writes completed	5041	5083	42
7	sectors written (512 B)	76640	383880	307240
8	ms spent writing	1557	3153	1596
10	ms doing I/O (busy)	4352	4472	120
11	weighted ms (queue x time)	14716	16394	1678

utilisation	120 ms busy / 214.8 ms = 55.9 %
write await	1596 ms / 42 writes = 38.0 ms per write
average queue size	1678 ms / 214.8 ms = 7.8 requests
average write size	307240 x 512 B / 42 = 3658 KiB
throughput	307240 x 512 B / 0.2148 s = 732 MB/s

Utilisation near 100 % means saturated only for a single-queue device; SSDs serve many requests in parallel.

Figure 5.23. Subtracting the two samples and dividing by the interval gives utilisation 55.9 percent, write latency 38.0 ms, an average queue of 7.8 requests and 732 MB/s.

1. Write requests completed: 5083 - 5041 = 42, which is 42 / 0.2148 s = 195.5 write operations per second.

2. Sectors written: 383880 - 76640 = 307240, which is 307240 x 512 B = 157.3 MB; divided by 0.2148 s the throughput is 732 MB/s. The average request therefore carried 307240 x 512 / 42 = 3.66 MB, far above the 128 KiB readahead size, because the block layer merged adjacent pages into large requests.
3. Time spent writing: 3153 - 1557 = 1596 ms summed over all requests; divided by 42 requests this is a mean latency (`await`) of 38.0 ms. The value includes the time in the queue and the time at the device. Because the requests were large and queued behind each other, a large latency here is expected and does not indicate a slow device.
4. Busy time: 4472 - 4352 = 120 ms during the 214.8 ms interval gives a utilisation of 55.9 percent. The field counts time with at least one request outstanding; it does not say how much more the device could have done.
5. Weighted time: 16394 - 14716 = 1678 ms; divided by the interval this is an average queue size of 7.81. Little's law check: 195.5 operations per second x 0.038 s = 7.4 requests in flight, with the remainder explained by the two reads and the flush.
6. The last two fields are flush counters: one flush request completed (1248 to 1249) and consumed 81 ms (124 to 205). That single cache flush, issued by `fdatasync`, accounts for 38 percent of the elapsed time.

The calculation illustrates why `%util` is misleading for devices that handle many requests in parallel: an NVMe drive can be at 100 percent utilisation (always at least one request in flight) while delivering a small part of its capacity. For such devices the queue size and the latency are more informative than utilisation.

5.3.3 Hands-on

Sampling a device over time

A short shell function turns the counters into rates; it is shown for the write side. The read side uses the neighbouring fields (indices 3, 5 and 6 of the array).

diskrate.sh BASH

```
#!/bin/bash
# usage: diskrate.sh DEVICE SECONDS
dev=$1; sec=$2
read -r -a a < <(awk -v d="$dev" '$3==d' /proc/diskstats)
sleep "$sec"
read -r -a b < <(awk -v d="$dev" '$3==d' /proc/diskstats)
# index: 7=writes completed, 9=sectors written, 10=ms writing, 12=ms doing I/O, 13=weighted
w=$(( b[7] - a[7] )); sw=$(( b[9] - a[9] )); mw=$(( b[10] - a[10] ))
busy=$(( b[12] - a[12] )); wt=$(( b[13] - a[13] ))
awk -v w="$w" -v sw="$sw" -v mw="$mw" -v busy="$busy" -v wt="$wt" -v s="$sec" 'BEGIN {
  printf "w/s=%.1f  MB/s=%.1f  await=%.2f ms  util=%.1f%%  avgq=%.2f\n",
    w/s, sw*512/1e6/s, (w ? mw/w : 0), busy/(s*10), wt/(s*1000) }'
```

Output of the script while FsyncLoop was running (build machine, typical) TERMINAL

```
$ ./diskrate.sh vda 3
w/s=7820.7  MB/s=181.6  await=0.07 ms  util=65.6%  avgq=0.99
```

The device as a process sees it

Per-process counters are in `/proc/PID/io`: `rchar` and `wchar` count bytes passed through `read` and `write` (including cached data), whereas `read_bytes` and `write_bytes` count bytes that were actually submitted to the block layer. The difference between the two shows how much of the I/O of a process was absorbed by the page cache. The incident below uses these fields.

5.3.4 Real-world incidents

Incident: high iowait and a slow service

Scenario. An order service in a Kubernetes pod writes a transaction journal to a persistent volume. After a release that reduced the batching of journal writes, request latency grows from 20 ms to over 300 ms during peak traffic. CPU graphs show moderate load.

Symptoms. The load average of the node is high compared with the CPU count while the `us` and `sy` columns of `vmstat` are low. Thread dumps show many threads in the journal-writing code, not in the database driver.

Diagnosis. The first question is whether threads wait for the disk. The commands below were run against `FsyncLoop.java`, a Java program that appends 64 KiB and calls `force(true)` in a loop, which reproduces the I/O pattern of a journal with one flush per record.

FsyncLoop.java JAVA

```java
import java.io.IOException;
import java.nio.ByteBuffer;
import java.nio.channels.FileChannel;
import java.nio.file.Path;
import java.nio.file.StandardOpenOption;

public class FsyncLoop {
    public static void main(String[] args) throws IOException {
        long end = System.nanoTime() + Long.parseLong(args[1]) * 1_000_000_000L;
        try (FileChannel ch = FileChannel.open(Path.of(args[0]), StandardOpenOption.CREATE,
                StandardOpenOption.WRITE, StandardOpenOption.TRUNCATE_EXISTING)) {
            ByteBuffer buf = ByteBuffer.allocate(64 * 1024);
            long n = 0, worst = 0;
            while (System.nanoTime() < end) {
                buf.clear();
                ch.write(buf);
                long t = System.nanoTime();
                ch.force(true);
                worst = Math.max(worst, System.nanoTime() - t);
                n++;
                if (n % 4096 == 0) ch.truncate(0).position(0);   // cap file at 256 MiB
            }
            System.out.printf("fsync calls: %d, worst fsync: %.1f ms%n", n, worst / 1e6);
        }
    }
}
```

Diagnosis during one 16-second run (build machine, typical; vmstat rows 1 to 4) TERMINAL

```
$ java FsyncLoop.java /data/wal.bin 16 &
$ ./diskrate.sh vda 3 &  vmstat 1 4
procs -----------memory---------- ---swap-- -----io---- -system-- -------cpu-------
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st gu
 4  0      0 6782760  14160 967556    0    0  1416  7647 4366  120 32  7 59  2  1  0
 0  1      0 6862024  14172 893744    0    0    12 202168 17420 27431  4 18 42 35  1  0
 0  1      0 6930864  14172 820992    0    0     0 203528 16583 28673  8 19 37 34  2  0
 3  1      0 6779928  14180 943484    0    0     0 126092 13754 22478 46 22 10 22  0  0
w/s=7820.7  MB/s=181.6  await=0.07 ms  util=65.6%  avgq=0.99
$ cat /proc/pressure/io
some avg10=22.61 avg60=5.57 avg300=2.18 total=39863304
full avg10=19.27 avg60=4.83 avg300=1.68 total=27278176
$ ps -L -o tid,stat,wchan:30,comm -p $JP | awk '$4=="java" && $2 ~ /D/'
14754 Dl   submit_bio_wait                java
14754 Dl   submit_bio_wait                java
14754 Dl   submit_bio_wait                java
$ grep -E "wchar|write_bytes|syscw" /proc/$JP/io
wchar: 1799946256
syscw: 27475
write_bytes: 1800048640
$ cat fl2.out
fsync calls: 41651, worst fsync: 9.9 ms
```

The reading of the output follows the decision path of the figure. The `b` column of `vmstat` shows one blocked process and the `wa` column 22 to 35 percent, so CPUs were idle while a task waited for I/O. The pressure stall information confirms it: `some avg10=22.61` means that for 22 percent of the last ten seconds at least one task was stalled on I/O, and `full` at 19 percent means that all non-idle tasks were stalled. The state of the Java thread was `D` (uninterruptible sleep) in `submit_bio_wait`, the kernel function in which a task waits for a block request to complete. The per-process counters show 1.8 GB written by 27475 write calls, with `write_bytes` equal to `wchar`: nothing was absorbed by the page cache because every write was forced.

The device counters give the decisive nuance. The disk was 65.6 percent busy, yet `await` was only 0.07 ms and the average queue 0.99: the device was not slow and not queued, it was kept busy by a stream of strictly serial tiny requests (a data write, a journal write and a flush per commit), 7820 per second. The thread, not the device, was the limit: 41651 commits in 16 s is one commit every 0.38 ms. Adding bandwidth would not help; fewer and larger commits would.

An earlier run of the same program, made while other processes kept both CPUs of the shared build machine busy, showed `wa` of only 0 to 13 percent and `b` of 0 although the thread was in `D` state. The reason is that `iowait` is counted only while a CPU is idle, so a busy machine hides blocked threads. The `D` state and the pressure values were reliable in both runs and are the primary indicators.

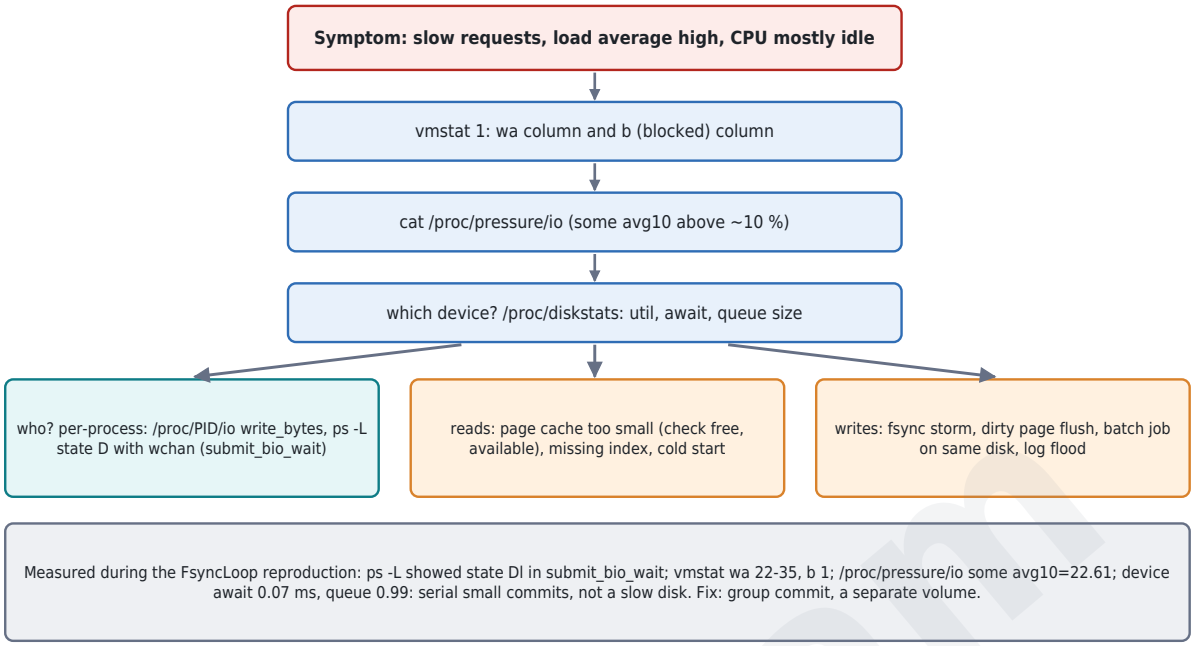


Figure 5.24. Diagnosis proceeds from the system-wide indicators to the device and finally to the process; the measured values of the reproduction are listed in the bottom box.

Root cause. The release replaced batched journal writes with one `force(true)` per record. Each record paid a full device round trip including a cache flush, and the volume had a limited number of operations per second, so threads queued behind the device.

Fix. Immediate: raise the batch size or the commit interval so that many records share one flush (group commit); move the journal to a faster or dedicated volume; scale out the number of volumes. Permanent: acknowledge records only after a group flush, with a bounded delay such as a few milliseconds, instead of one flush per record. Measure the achieved operations per second against the provisioned limit of the volume.

Prevention. Alert on pressure stall `io some avg10` above 10 percent, on the await of the data volume and on the proximity to the provisioned I/O budget. Include the commit mode in load tests: a test that writes to a RAM disk (tmpfs) cannot reveal flush costs, as the measurement in the next chapter shows.

5.3.5 Questions to expect

Why is random I/O slower than sequential I/O on an HDD, and does the same hold for an SSD?

On an HDD each random access needs a head seek and a rotational delay of several milliseconds, whereas sequential access pays them once. An SSD has no mechanical delay, so random reads are close to sequential speed; random writes still suffer from erase-block management and write amplification.

What does the I/O scheduler do, and is it needed on NVMe?

It orders and merges requests within the block layer and prevents starvation (mq-deadline gives every request an expiry). NVMe devices have many hardware queues and internal scheduling, so `none` is commonly used.

What does `iowait` mean?

It is the share of time in which a CPU was idle while at least one task was waiting for I/O. It is not a measure of disk load: a busy CPU hides waiting tasks, and a CPU with nothing to do shows waiting that is harmless. Pressure stall information and per-device latency are more reliable.

How are utilisation and latency of a disk computed from `/proc/diskstats`?

Take two samples. Utilisation is the increase of the busy-time field divided by the wall time; latency is the increase of the time spent reading or writing divided by the number of completed requests; average queue size is the increase of the weighted time divided by the wall time.

Why can a device at 100 percent utilisation still have spare capacity?

The busy-time field counts time with at least one request outstanding. An SSD or NVMe device processes many requests in parallel, so it can be busy all the time while serving only a fraction of its throughput.

What are the trade-offs between RAID 1, RAID 5 and RAID 10?

RAID 1 mirrors (half capacity, fast reads, tolerates one failure per pair). RAID 5 stripes with parity (capacity of N-1 disks, tolerates one failure, slow small writes because of the read-modify-write of parity). RAID 10 stripes over mirrors (half capacity, good read and write performance, tolerates one failure per mirror pair). None of them replaces backup.

What is the purpose of LVM?

It separates logical volumes from physical disks so that volumes can be resized online, spread over several disks and snapshotted. The file system on top must be grown separately.

What is write amplification?

The ratio of data physically written to flash to the data written by the host, caused by erase-block granularity and garbage collection. High amplification reduces speed and lifetime of the device.

KEY POINTS

Key points

- Device latency exceeds memory latency by orders of magnitude; the kernel hides it with the page cache, merging, read-ahead and queues, each of which adds its own latency under load.
- The counters of `/proc/diskstats` yield rate, latency, queue size and utilisation by simple subtraction; Little's law checks the result.
- `%util` is not capacity for parallel devices, and `iowait` is hidden by busy CPUs; pressure stall information, `D`-state threads and per-device await are better indicators.
- A device with a volatile write cache needs explicit flushes for durability; RAID 5 penalises small writes and no RAID level is a backup.
- Reduce the number of synchronous commits (group commit) before buying faster storage.

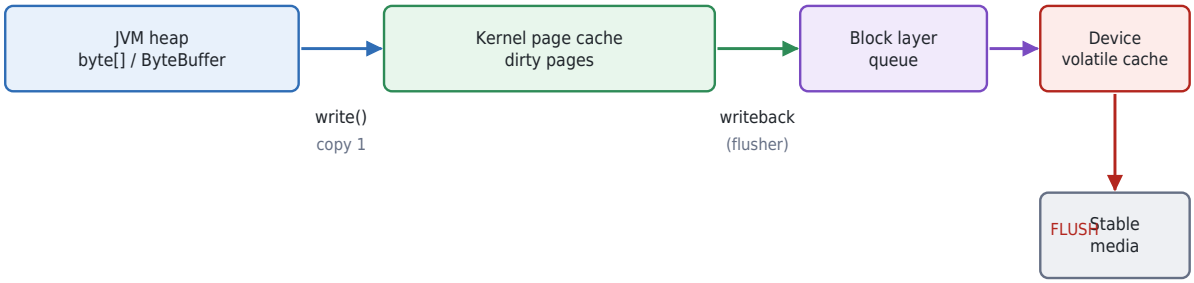
5.4 Buffered I/O, the Page Cache and Durability

A call to `write` returns in a few microseconds although a disk needs milliseconds. The difference is the page cache: the kernel copies the data into memory, marks the pages dirty and writes them out later. This design makes ordinary file I/O fast, but it also means that a successful `write` does not imply that the data is stored safely. Databases, message brokers and any service that promises to retain accepted data must therefore control when data reaches stable media. This chapter explains the write path, the system calls that control durability, their measured cost and the window of loss that remains when they are omitted.

5.4.1 Concept and mechanism

The write path

The sequence of a buffered write is as follows. The application passes a buffer to `write` (for Java, `FileOutputStream.write` or `FileChannel.write`, which end in the same system call; `BufferedOutputStream` merely reduces the number of calls). The kernel copies the bytes into pages of the page cache that belong to the file, marks those pages dirty and returns. At a later time kernel writeback threads select dirty pages, ask the file system to map them to disk blocks and submit them through the block layer. The device may hold the data in its own volatile cache before it reaches stable media.



Where does the data survive?

after write() returns	survives JVM crash (kernel holds it)	lost on power failure or kernel panic
after fsync() returns	survives power failure (data + metadata flushed)	needs honest FLUSH support in the device
fdatasync() / force(false)	data durable; skips unneeded metadata (mtime)	size change is still flushed
O_DIRECT	bypasses the page cache, not the device cache	still needs fsync or O_DSYNC for durability

Figure 5.25. A buffered write completes after the copy into the page cache; the data passes the block layer and the device cache later and is stable only after a flush, which the table below summarises for each strategy.

Reads use the same cache. A `read` first looks up the page; a hit is a memory copy, a miss submits a block request and blocks the thread. The kernel also detects sequential access and prefetches ahead of the reader. Pages of the cache that are clean can be dropped at any time when memory is needed, so the cache grows to use free memory without harming applications; the accounting of this memory in `free` is treated in the Part on memory management.

Writeback policy

The kernel starts writing dirty data when any of three conditions is reached. These are controlled by sysctl parameters; the build machine reported the common defaults:

Writeback parameters (build machine) TERMINAL

```
$ sysctl vm.dirty_ratio vm.dirty_background_ratio \
    vm.dirty_expire_centisecs vm.dirty_writeback_centisecs
vm.dirty_ratio = 20
vm.dirty_background_ratio = 10
vm.dirty_expire_centisecs = 3000
vm.dirty_writeback_centisecs = 500
```

- Background writeback starts when dirty pages exceed `vm.dirty_background_ratio` percent of reclaimable memory (10 percent); the process continues to write without being delayed.
- When dirty pages exceed `vm.dirty_ratio` percent (20 percent), the writing process itself is throttled and forced to write, which appears as sudden latency in `write` calls.
- Regardless of volume, the writeback threads wake every `vm.dirty_writeback_centisecs` (5 seconds) and write pages that have been dirty longer than `vm.dirty_expire_centisecs` (30 seconds).

The implication is a bound on the age of unwritten data: a dirty page is written within roughly 30 to 35 seconds unless the process forces it earlier. That interval is the window in which a power failure loses data that the application believed to be written.

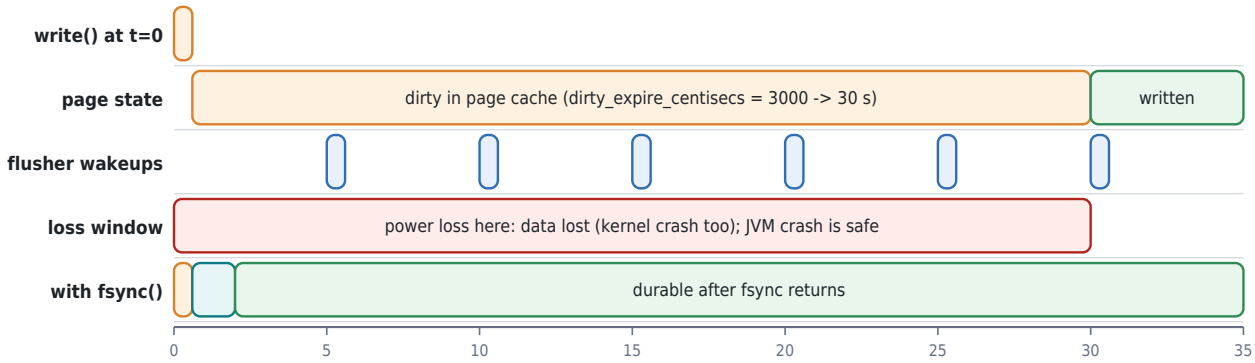


Figure 5.26. A page written at time 0 stays dirty until the 30-second expiry is noticed by one of the 5-second writeback wake-ups; an fsync closes the loss window at the moment it returns.

A process that crashes does not lose cached writes, because the kernel owns the pages: only a failure of the kernel, the machine or the device loses them. Applications and operators often confuse the two kinds of failure when they reason about durability.

The durability system calls

Mechanism	Effect	Java equivalent
`fsync(fd)`	blocks until the data and the metadata of the file are on stable storage	`FileChannel.force(true)`
`fdatasync(fd)`	as `fsync`, but skips metadata not needed to read the data (for example the modification time)	`FileChannel.force(false)`
`O_SYNC`	every `write` behaves as if followed by `fsync`	`StandardOpenOption.SYNC`
`O_DSYNC`	every `write` behaves as if followed by `fdatasync`	`StandardOpenOption.DSYNC`
`O_DIRECT`	bypasses the page cache; buffers and offsets must be block aligned; does not imply durability	no standard option; `ExtendedOpenOption.DIRECT` (JDK 10+, `com.sun.nio.file`)
`sync_file_range`, `msync`	write out a range or a mapped region	`MappedByteBuffer.force()`

Calls and flags that control when data becomes durable.

The mapping of the Java methods was verified with `strace`: a program calling `force(true)` produced `fsync(4)`, one calling `force(false)` produced `fdatasync(4)` (five calls each in the test run).

An `fsync` of a file does not make the directory entry of a newly created file durable. After creating a file, a program that needs the name to survive a crash must also `fsync` the containing directory; in Java this is `FileChannel.open(dir, READ).force(true)`, which works on Linux. The ordering rules of the journal give a common safe pattern for replacing a file atomically: write a temporary file in the same directory, `fsync` it, rename it over the target and `fsync` the directory.

The device cache and flush

Storage devices have volatile write caches. A completed block write means only that the device accepted the data. `fsync` therefore ends with a flush command (or issues writes with the force-unit-access flag) so that the device writes its cache to media. Enterprise devices with power-loss protection treat the flush as nearly free; consumer devices do not. A device or hypervisor that acknowledges flushes without honouring them breaks every durability guarantee above it, which is a documented cause of corrupted databases on misconfigured virtual disks. The build machine reported `write_cache` as `write back`, so flushes were required.

Why databases and brokers care

A relational database makes a transaction durable by appending its redo record to a write-ahead log and calling `fsync` before it reports the commit; the data pages are written lazily. PostgreSQL does this with `fsync` and `synchronous_commit` enabled by default, and MySQL InnoDB with `innodb_flush_log_at_trx_commit=1`. Relaxing the setting (to a one-second flush interval) multiplies throughput but accepts the loss of up to one second of committed transactions after a crash. Redis with `appendfsync everysec` has the same trade-off. Apache Kafka takes a different position: by default it does not call `fsync` per message (the flush interval properties default to unlimited) and relies on replication to several brokers for durability, leaving the page cache to write segments. A cluster in which all replicas lose power simultaneously can therefore lose acknowledged messages, whereas a single broker failure cannot.

A further subtlety is the failure of `fsync` itself. When a write to the device fails, Linux marks the affected pages clean and reports the error once; a repeated `fsync` of the same file may then return success although the data was never written. PostgreSQL treats an `fsync` failure as fatal for this reason since 2018, and applications that retry after an I/O error assume the data is safe at their peril.

5.4.2 Worked trace: the life of one 4 KiB durable record

The following steps follow one record of a journal written by `FileChannel.write(buf)` followed by `force(true)`; the durations are the medians measured on the build machine (typical, from the experiment below).

1. The JVM calls `write(fd, buf, 4096)`. The kernel copies 4096 bytes into a page cache page and marks it dirty. About 7.5 microseconds elapse; the file size in the inode grows.
2. The JVM calls `fsync(fd)`. The file system starts a journal transaction: it writes the data block first (ordered mode), then the journal descriptor with the changed metadata (inode size and timestamps, block bitmap, extent).
3. The kernel writes the commit block with a flush so that all earlier journal blocks are on media before it. The commit block is only valid if everything before it is durable.
4. The device flush completes, the transaction is committed and `fsync` returns. About 214 microseconds per call elapsed in the measurement on the root disk and about 720 microseconds on the loop-mounted image (which adds a second file system layer).
5. Later, the checkpoint writes the metadata from the journal to its home location, outside the latency path of the application.

The cost of step 2 to 4 is thirty times that of step 1, and the next experiment measures it directly.

5.4.3 Hands-on

Measuring the cost of synchronisation

The program writes 200 blocks of 4 KiB in three modes: without synchronisation, with `force(false)` and with `force(true)` after every write.

FsyncCost.java — JAVA

```
import java.io.IOException;
import java.nio.ByteBuffer;
import java.nio.channels.FileChannel;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardOpenOption;

public class FsyncCost {
    public static void main(String[] args) throws IOException {
        Path dir = Path.of(args[0]);
        int n = Integer.parseInt(args[1]);
        for (String mode : new String[] {"buffered", "force(false)", "force(true)"}) {
            Path p = dir.resolve("fsync-test.bin");
            try (FileChannel ch = FileChannel.open(p, StandardOpenOption.CREATE,
                    StandardOpenOption.WRITE, StandardOpenOption.TRUNCATE_EXISTING)) {
                ByteBuffer buf = ByteBuffer.allocate(4096);
                long t0 = System.nanoTime();
                for (int i = 0; i < n; i++) {
                    buf.clear();
                    ch.write(buf);
                    if (mode.equals("force(false)")) ch.force(false);
                    if (mode.equals("force(true)")) ch.force(true);
                }
                long us = (System.nanoTime() - t0) / 1000;
```

FsyncCost.java (continued) JAVA

```
                System.out.printf("%-13s %5d writes of 4 KiB: total %8d us, %8.1f us/write%n",
                        mode, n, us, us / (double) n);
            }
            Files.delete(p);
        }
    }
}
```

Results on three targets (measured on the build machine, typical) TERMINAL

```
$ java FsyncCost.java /data/rootd 200
buffered      200 writes of 4 KiB: total     1507 us,      7.5 us/write
force(false)  200 writes of 4 KiB: total    49723 us,    248.6 us/write
force(true)   200 writes of 4 KiB: total    42754 us,    213.8 us/write
$ java FsyncCost.java /mnt/loop 200
buffered      200 writes of 4 KiB: total     4253 us,     21.3 us/write
force(false)  200 writes of 4 KiB: total   164815 us,    824.1 us/write
force(true)   200 writes of 4 KiB: total   143828 us,    719.1 us/write
$ java FsyncCost.java /mnt/tmpfs 200
buffered      200 writes of 4 KiB: total     5531 us,     27.7 us/write
force(false)  200 writes of 4 KiB: total     1429 us,      7.1 us/write
force(true)   200 writes of 4 KiB: total     1185 us,      5.9 us/write
```

Three observations follow. Forced writes on the root disk cost between 214 and 249 microseconds, about thirty times the cost of the buffered write. The `force(false)` run was not faster than `force(true)` because every write extends the file, so the size change is metadata that `fdatasync` must flush as well; the difference between the two calls appears only for overwrites of already allocated blocks. On tmpfs a synchronisation call costs nothing because there is no device behind it, so a test on a RAM disk reports an optimistic result for any durable workload. The numbers on a different disk, a cloud volume or with a different load will differ; the ratio is the lesson.

Variants measured with dd

Writing 2000 blocks of 4 KiB with different flags (build machine, typical) TERMINAL

```
$ dd if=/dev/zero of=t.bin bs=4k count=2000
8192000 bytes (8.2 MB, 7.8 MiB) copied, 0.00846518 s, 968 MB/s
$ dd if=/dev/zero of=t.bin bs=4k count=2000 conv=fdatasync
8192000 bytes (8.2 MB, 7.8 MiB) copied, 0.0268795 s, 305 MB/s
$ dd if=/dev/zero of=t.bin bs=4k count=2000 oflag=direct
8192000 bytes (8.2 MB, 7.8 MiB) copied, 0.174004 s, 47.1 MB/s
$ dd if=/dev/zero of=t.bin bs=4k count=2000 oflag=dsync
8192000 bytes (8.2 MB, 7.8 MiB) copied, 3.3686 s, 2.4 MB/s
```

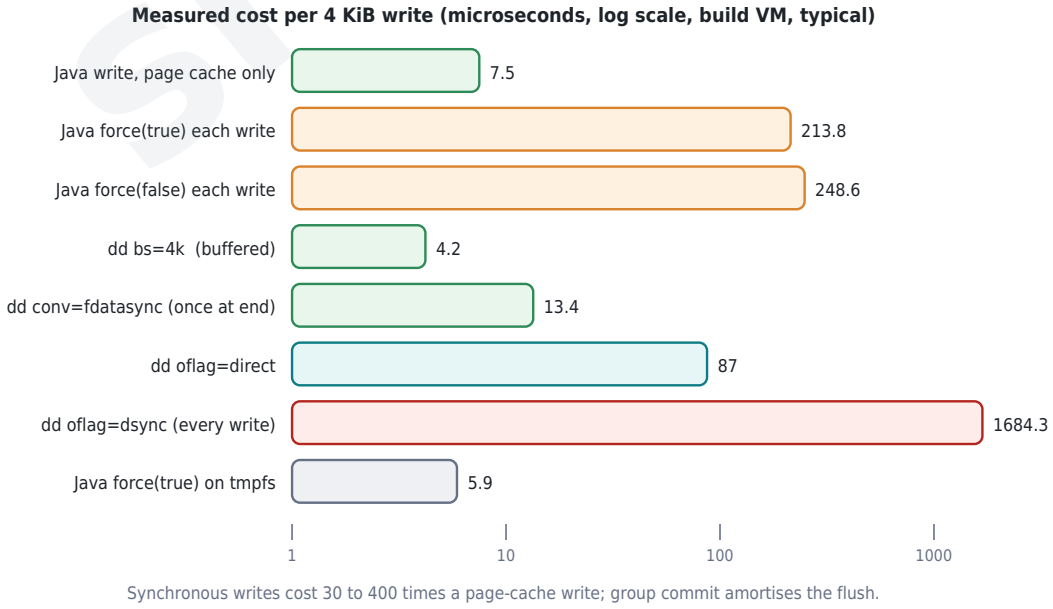


Figure 5.27. Per 4 KiB block the cost ranges from 4 microseconds (buffered) to 1684 microseconds (O_DSYNC on every write); a single flush at the end costs 13 microseconds per block.

The four runs differ by a factor of 400. `conv=fdatasync` flushes once at the end, so the 2000 writes share one flush. `oflag=direct` bypasses the page cache but is not durable by itself. `oflag=dsync` makes each of the 2000 writes wait for the device, 1.68 ms per write. The conclusion for design is that the number of flushes, not the number of bytes, dominates durable throughput, which is the reason for group commit: collect the records of many concurrent requests into one write and one flush.

Dirty pages become visible

Dirty memory after a buffered write of 64 MiB (build machine, typical) TERMINAL

```
$ dd if=/dev/zero of=t2.bin bs=1M count=64
67108864 bytes (67 MB, 64 MiB) copied, 0.0305604 s, 2.2 GB/s
$ grep -E "^(Dirty|Writeback):" /proc/meminfo
Dirty:             67000 kB
Writeback:             0 kB
$ sync; grep -E "^(Dirty|Writeback):" /proc/meminfo
Dirty:                72 kB
Writeback:             0 kB
```

The `dd` command reported 2.2 GB/s because it measured only the copy into memory. At the moment it finished, 67000 kB were dirty and nothing was written yet; `sync` forced the write and returned the counters to near zero. Monitoring the `Dirty` and `Writeback` fields shows whether the system is accumulating unwritten data.

The read side

Warm cache against direct reads of the same 128 MiB file (build machine, typical) TERMINAL

```
$ dd if=r.bin of=/dev/null bs=1M
134217728 bytes (134 MB, 128 MiB) copied, 0.0181431 s, 7.4 GB/s
$ dd if=r.bin of=/dev/null bs=1M iflag=direct
134217728 bytes (134 MB, 128 MiB) copied, 0.0563702 s, 2.4 GB/s
$ dd if=r.bin iflag=nocache count=0           # evict this file from the cache
$ dd if=r.bin of=/dev/null bs=1M              # cold read, fills the cache
134217728 bytes (134 MB, 128 MiB) copied, 0.0502132 s, 2.7 GB/s
$ dd if=r.bin of=/dev/null bs=1M              # warm read
134217728 bytes (134 MB, 128 MiB) copied, 0.0201377 s, 6.7 GB/s
```

The warm read was about 2.5 times faster than the cold read. The ratio on this machine is modest because the virtual disk is backed by the host cache; on a physical disk the difference is two to four orders of magnitude. System-wide eviction with `echo 3 > /proc/sys/vm/drop_caches` was not used because it affects every process on the machine and slows them all; it is acceptable for benchmarks on dedicated hosts and wrong as a production remedy.

sreram

Atomic file replacement

AtomicWrite.java JAVA

```java
import java.io.IOException;
import java.nio.ByteBuffer;
import java.nio.channels.FileChannel;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardCopyOption;
import java.nio.file.StandardOpenOption;

public class AtomicWrite {
    /** Replaces target with new content so that a crash leaves either the old or the new file. */
    static void replace(Path target, String content) throws IOException {
        Path dir = target.toAbsolutePath().getParent();
        Path tmp = Files.createTempFile(dir, "cfg", ".tmp");
        try (FileChannel ch = FileChannel.open(tmp, StandardOpenOption.WRITE)) {
            ch.write(ByteBuffer.wrap(content.getBytes(StandardCharsets.UTF_8)));
            ch.force(true);                                  // 1. data durable before the rename
        }
        Files.move(tmp, target, StandardCopyOption.ATOMIC_MOVE,
                StandardCopyOption.REPLACE_EXISTING);        // 2. atomic switch of the name
        try (FileChannel d = FileChannel.open(dir, StandardOpenOption.READ)) {
            d.force(true);                                   // 3. directory entry durable
        }
    }

    public static void main(String[] args) throws IOException {
        Path target = Path.of(args[0]);
        replace(target, "version=1\n");
        replace(target, "version=2\n");
        System.out.print(Files.readString(target));
        try (var names = Files.list(target.toAbsolutePath().getParent())) {
            names.forEach(p -> System.out.println("entry: " + p.getFileName()));
        }
    }
}
```

Run and system call trace of the pattern TERMINAL

```
$ java AtomicWrite.java /data/aw/app.conf
version=2
entry: app.conf
$ strace -f -e trace=fsync,rename -o aw.st java AtomicWrite.java /data/aw/app.conf
$ grep -E "fsync|rename" aw.st | head -6
fsync(6)                        = 0
rename("/data/aw/cfg17957079634848529826.tmp", "/data/aw/app.conf") = 0
fsync(6)                        = 0
fsync(6)                        = 0
rename("/data/aw/cfg4243081716996555018.tmp", "/data/aw/app.conf") = 0
fsync(6)                        = 0
```

The trace shows the required order: `fsync` of the temporary file, `rename`, `fsync` of the directory. Readers of the target file see either the old or the new content in full, and after a crash the same holds, because the rename is a metadata operation of a single journal transaction.

5.4.4 Real-world incidents

Incident: intermittent latency spikes from slow fsync

Scenario. A payment service commits each transaction to an embedded write-ahead log with `fsync`. It shares the volume with a nightly batch job that exports data to large files. Median latency is 2 ms, but several times per night the 99th percentile jumps to seconds and the circuit breakers of upstream services open.

Symptoms. Latency graphs show spikes that coincide with the start of the export. CPU and heap are normal. Application logs show nothing, but a thread dump taken during a spike shows commit threads inside `FileChannel.force`.

Diagnosis. The aim is to show that the `fsync` latency of the service rises when a bulk writer shares the device, and to find the writer. The reproduction used `FsyncLatency.java`, which measures every `force(true)` of a 4 KiB write, once alone and once while a `dd` loop wrote 200 MB files with `fsync` on the same disk.

FsyncLatency.java JAVA

```java
import java.io.IOException;
import java.nio.ByteBuffer;
import java.nio.channels.FileChannel;
import java.nio.file.Path;
import java.nio.file.StandardOpenOption;
import java.util.Arrays;

public class FsyncLatency {
    public static void main(String[] args) throws IOException {
        long end = System.nanoTime() + Long.parseLong(args[1]) * 1_000_000_000L;
        long[] lat = new long[200_000];
        int n = 0;
        try (FileChannel ch = FileChannel.open(Path.of(args[0]), StandardOpenOption.CREATE,
                StandardOpenOption.WRITE, StandardOpenOption.TRUNCATE_EXISTING)) {
            ByteBuffer buf = ByteBuffer.allocate(4096);
            while (System.nanoTime() < end && n < lat.length) {
                buf.clear();
                ch.write(buf);
                long t = System.nanoTime();
                ch.force(true);
                lat[n++] = System.nanoTime() - t;
                if (n % 2048 == 0) ch.truncate(0).position(0);
                try { Thread.sleep(2); } catch (InterruptedException e) { return; }
            }
        }
        long[] s = Arrays.copyOf(lat, n);
        Arrays.sort(s);
        System.out.printf("n=%d  p50=%.2f ms  p99=%.2f ms  max=%.2f ms%n", n,
                s[n / 2] / 1e6, s[(int) (n * 0.99)] / 1e6, s[n - 1] / 1e6);
    }
}
```

Commit latency alone and next to a bulk writer (build machine, shared VM, typical) TERMINAL

```
$ java FsyncLatency.java /data/commit.log 8
n=2959  p50=0.49 ms  p99=1.25 ms  max=35.87 ms
$ (for i in 1 2 3 4; do dd if=/dev/zero of=/data/bulk.bin bs=1M count=200 conv=fsync; done) &
$ java FsyncLatency.java /data/commit.log 8
n=2588  p50=0.63 ms  p99=5.87 ms  max=81.05 ms
```

The median moved little (0.49 to 0.63 ms) while the 99th percentile grew by a factor of 4.7 and the maximum by a factor of 2.3. This is the signature of queueing behind bulk I/O: most commits find a short queue, a few find the queue full of large requests. On the shared test machine the effect is small; on a volume with a provisioned throughput budget, a 3 GB export can fill the queue for seconds. To find the culprit on a real host, the per-process counters `write_bytes` in `/proc/PID/io` are sampled twice and subtracted, and `ps -L` with `wchan` shows which threads wait for the device.

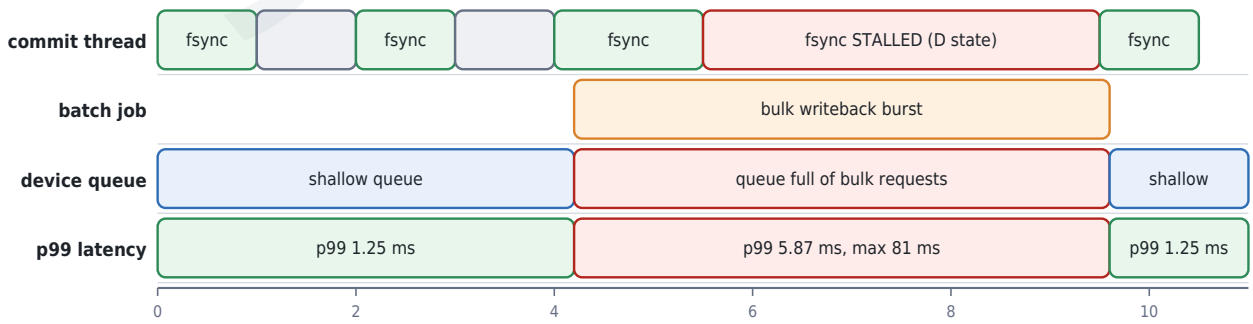


Figure 5.28. While the batch writer fills the device queue, commit threads wait in uninterruptible sleep and the tail latency rises; the p99 values are the measured ones, the time axis is schematic.

Root cause. The write-ahead log and a bulk writer shared one device and one queue. A synchronous flush has to wait for the requests already queued, and the file system journal can also serialise the commit behind the dirty data of the large file.

Fix. Immediate: move or throttle the export (for example with `ionice -c3` (honoured by `bfq` and, in recent kernels, by `mq-deadline`), a cgroup `io.max` limit, or by writing in smaller chunks with an explicit pause). Permanent: place the

write-ahead log on its own volume; limit the dirty memory a bulk writer can accumulate (`vm.dirty_background_bytes`, `vm.dirty_bytes`) so that writeback is continuous instead of bursty; use group commit to reduce the number of flushes.

Prevention. Track the 99th and 99.9th percentile of the commit duration as a metric of its own; alert on `fsync` duration, not only on request latency. Schedule batch work away from peak hours and give it a throttled, separate volume.

5.4.5 Questions to expect

What is the difference between `write` and `fsync`?

`write` copies data into the page cache and returns; the data is lost if the machine fails before writeback. `fsync` blocks until the data and the metadata of the file are on stable storage, including a device cache flush.

What is the difference between `fsync` and `fdatasync`?

`fdatasync` flushes only the metadata that is required to retrieve the data (for example a new file size), skipping others such as the modification time, so it can avoid one journal write. For appends the difference is small because the size changes.

What does `O_DIRECT` do, and does it make writes durable?

It bypasses the page cache and transfers between user buffers and the device, with alignment requirements. It does not by itself guarantee durability because the device cache may still be volatile; it is combined with `O_DSYNC` or `fsync`. Databases use it to avoid double caching.

Why does a database call `fsync` on its log but not on its data files at commit?

The log is sequential, so one flush makes the transaction recoverable. The data pages can be written lazily because recovery re-applies the log to them. This turns many random writes into one sequential flush.

When is a process crash different from a power failure for written data?

After a process crash the kernel still owns the dirty pages and writes them, so data passed to `write` survives. A power failure or kernel panic loses everything not yet written to stable storage.

How is a file replaced atomically?

Write a temporary file in the same directory, `fsync` it, `rename` it over the target (atomic on one file system), then `fsync` the directory. Readers and crash recovery see the old or the new file, never a mixture.

What controls when dirty pages are written?

Background writeback starts at `vm.dirty_background_ratio`, writers are throttled at `vm.dirty_ratio`, and pages older than `vm.dirty_expire_centisecs` (30 s by default) are written by threads that wake every `vm.dirty_writeback_centisecs` (5 s).

Why can a benchmark on tmpfs mislead?

There is no device, so `fsync` costs nothing; the measurement hides the dominant cost of durable workloads.

How does Kafka obtain durability without calling `fsync` per message?

By replicating each partition to several brokers and acknowledging according to `acks` and `min.insync.replicas`; the operating system writes segments from the page cache. Simultaneous loss of all replicas can still lose data.

KEY POINTS

Key points

- `write` copies into the page cache and returns; data is stable only after `fsync` or `fdatasync` has returned, which includes a device flush.
- Dirty pages are written within about 30 to 35 seconds by default; a process crash loses nothing, a power failure loses the unwritten window.
- Durable writes cost 30 to 400 times a cached write on the measured machine; the number of flushes dominates, so batch commits (group commit).
- A durable file replacement is temporary file, `fsync`, `rename`, `fsync` of the directory.
- Tests on tmpfs or with `fsync` disabled hide the cost and the risk; a bulk writer on the same device raises the tail latency of commits.

5.5 I/O Models

A network service must wait for data that arrives at moments it does not control, and it must wait for thousands of connections at once. The strategy for waiting determines how many threads a server needs, how much memory they consume and how the server behaves under load. The operating system offers five models, from the simplest blocking call to asynchronous submission, and every Java server framework is a choice among them. This chapter defines the models by the moments at which the calling thread waits, shows the readiness mechanism of `epoll` with a C server and a Java NIO server that were run on the build machine, and connects the results to Tomcat, Netty and Kafka.

5.5.1 Concept and mechanism

The two phases of an input operation

Every input operation has two phases: waiting until data is available (the kernel receives a packet or a disk block into its buffer) and copying the data from the kernel buffer into the buffer of the process. The models differ in what the calling thread does during each phase.

Model	Phase 1: waiting for data	Phase 2: copy	Linux interface
`Blocking`	thread blocked in the call	thread blocked	`read`, `recv` on a default socket
`Non-blocking`	call returns `EAGAIN` at once; the program polls	thread blocked during the copy	`O_NONBLOCK`
`Multiplexing`	one thread blocked on many descriptors	thread blocked during the copy	`select`, `poll`, `epoll_wait`
`Signal-driven`	process continues; signal when ready	thread blocked during the copy	`O_ASYNC` with `SIGIO`, rarely used
`Asynchronous`	process continues	kernel performs the copy; completion is reported	`io_uring`, Linux AIO (`io_submit`)

The five I/O models of UNIX (after W. Richard Stevens) as they appear on Linux.

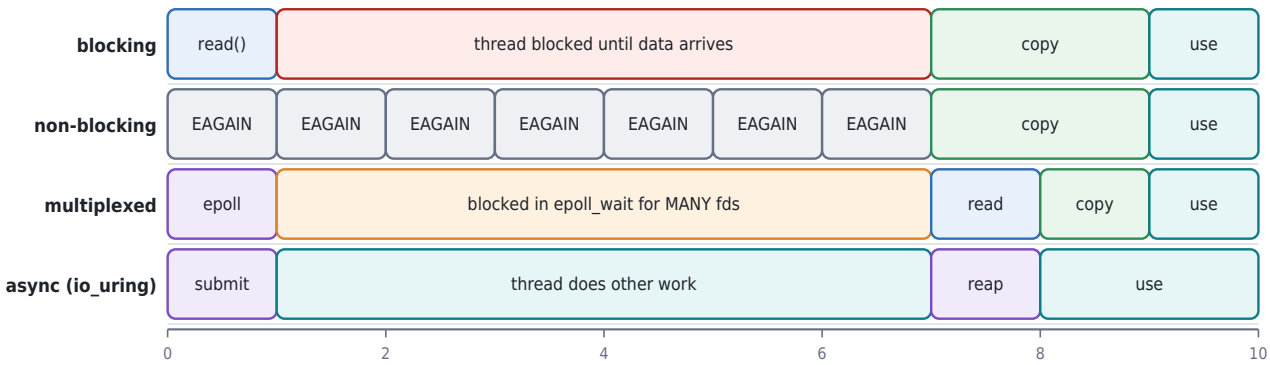


Figure 5.29. The blocking thread waits for the whole interval, the non-blocking thread spends it polling, the multiplexing thread waits for many descriptors at once and the asynchronous thread does other work until the completion arrives.

The word asynchronous is often misused. A non-blocking read with `epoll` is not asynchronous in the strict sense: the thread still performs the copy and decides when to read. In a truly asynchronous interface the thread submits the operation and the kernel completes it, including the copy. The distinction is irrelevant for most Java programs but is answered in interviews.

Blocking I/O and thread per connection

The simplest server accepts a connection and assigns a thread to it; the thread calls `read`, which blocks until data arrives. The code is sequential and easy to reason about, and the kernel schedules the threads. The cost is one thread per connection: each carries a stack (the default `-Xss` is 1 MiB of virtual address space on Linux x86-64, committed on use), kernel structures and a scheduling entry. With a few hundred connections this is acceptable. With tens of thousands of mostly idle connections the memory and the context switches become the limit, and the thread count itself is a risk (the thread explosion incident in the Part on processes, threads and scheduling).

Readiness: select, poll and epoll

A multiplexing call blocks one thread until any one of many descriptors is ready for reading or writing. Three system calls provide it:

- `select` takes bit sets of descriptors with a hard limit of `FD_SETSIZE` (1024) and rescans all of them on every call.
- `poll` takes an array of descriptors without that limit but still copies the whole array into the kernel and scans it on every call.
- `epoll` keeps the interest list inside the kernel. `epoll_ctl` adds and removes descriptors once; `epoll_wait` returns only the descriptors that are ready.

The cost of `select` and `poll` grows with the number of registered descriptors on every call, while the cost of `epoll_wait` grows with the number of ready descriptors.

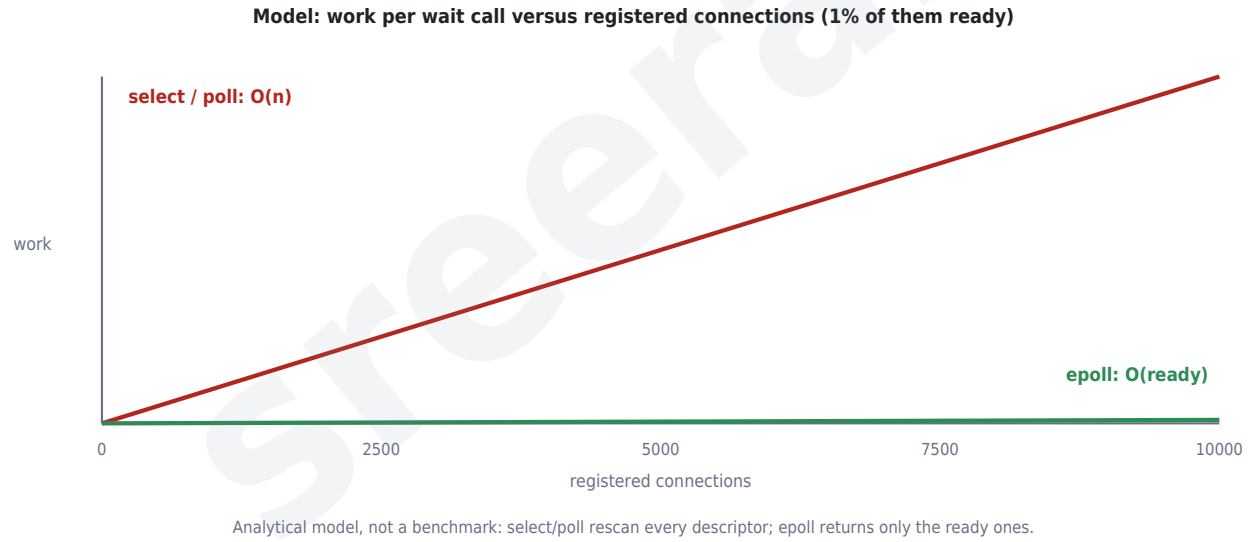
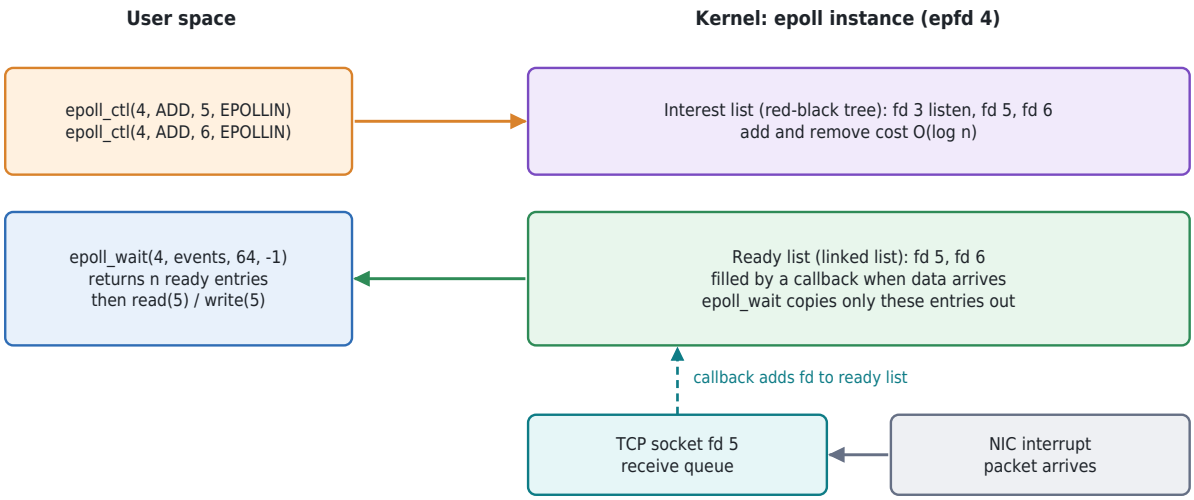


Figure 5.30. The work per wait call of select and poll grows linearly with the registered connections, whereas epoll depends on the ready ones; the figure is an analytical model, not a benchmark.

Inside the kernel an `epoll` instance has two structures. The interest list is a red-black tree keyed by descriptor, so adding and removing costs O(log n). The ready list is a linked list that the kernel appends to from a callback when a registered socket receives data. `epoll_wait` copies the entries of the ready list to the caller and sleeps if the list is empty.

sreeram



Cost of epoll_wait is proportional to the number of READY descriptors, not the number registered.

select/poll copy the whole fd set into the kernel and scan it on every call (O(n)).

Level-triggered (default): fd stays ready until drained. Edge-triggered: one event per new arrival.

Regular files are always ready for epoll: it helps only for sockets, pipes and similar descriptors.

Figure 5.31. The callback of the socket adds a descriptor to the ready list when data arrives; the waiting thread receives only ready entries, regardless of how many descriptors are registered.

`epoll` supports two trigger modes. In level-triggered mode (the default), a descriptor is reported as long as unread data remains, which makes the loop simple and safe. In edge-triggered mode (`EPOLLET`) a descriptor is reported once per arrival of new data and the program must read until `EAGAIN`; it reduces wake-ups but is easy to get wrong. Regular files are always reported as ready, so `epoll` helps for sockets, pipes and timers but not for disk files; this is why disk I/O in event-loop servers is moved to worker threads.

Asynchronous I/O and io_uring

Linux AIO (`io_submit`) is limited to files opened with `O_DIRECT` and has been rarely used outside databases. `io_uring`, available since Linux 5.1, supplies two ring buffers shared between user space and the kernel: a submission queue to which the program appends operation descriptions (read, write, accept, send, fsync) and a completion queue from which it reads results. Submission and completion need few or no system calls in steady state. It supports files and sockets uniformly. The JDK does not use `io_uring`; Netty offers an incubating transport for it. `io_uring` was not exercised on the build machine, so the description here is based on the kernel documentation only.

Java's `AsynchronousFileChannel` on Linux is implemented with a thread pool that performs ordinary blocking calls, and `AsynchronousSocketChannel` uses `epoll` with a pool of threads, so neither is kernel-level asynchronous I/O.

Zero-copy transfers

A conventional file-to-socket transfer in a loop of `read` and `write` copies data twice through user space: from the page cache into a user buffer, then from the buffer into a socket buffer. The `sendfile` call instructs the kernel to move the data between the two descriptors without the round trip to user space. `mmap` maps the page cache into the address space of the process so that reads need no copy into a separate buffer, and `splice` moves data between descriptors through a pipe.

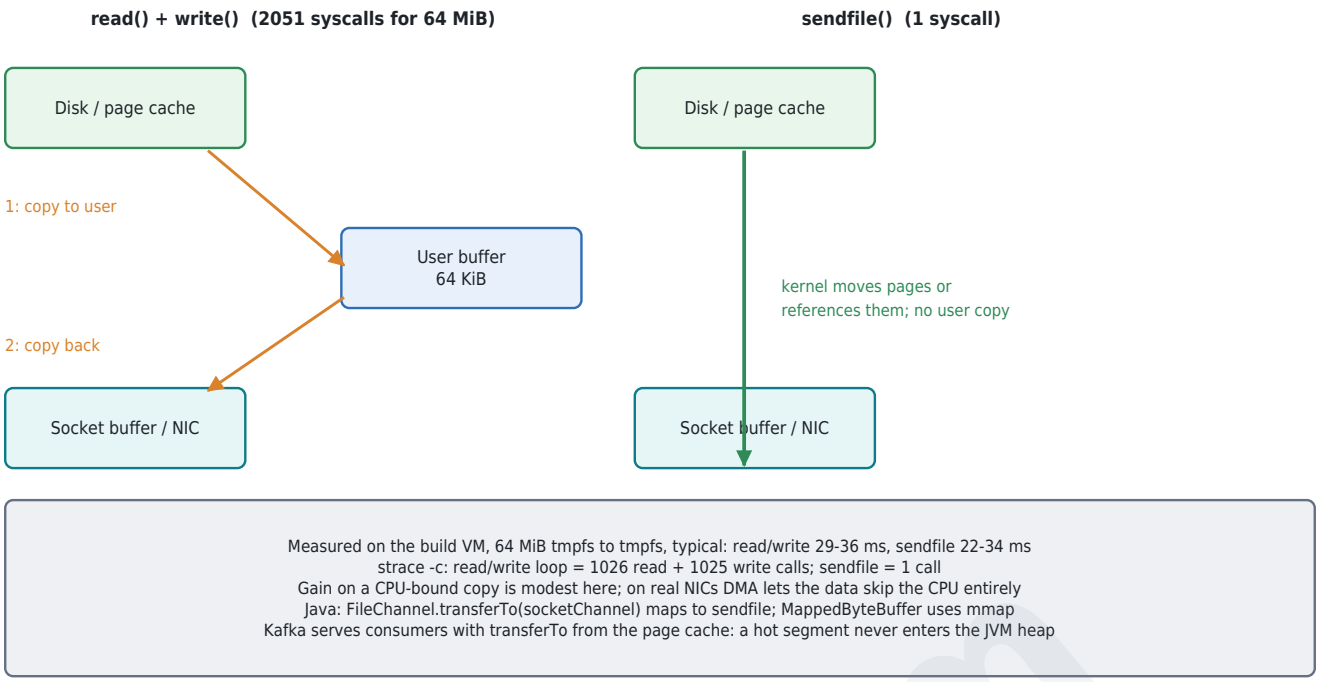


Figure 5.32. sendfile replaces about two thousand system calls and two copies with a single call; the strace counts and the timings are from the build machine.

In Java, `FileChannel.transferTo` uses `sendfile` where the kernel supports it, and `FileChannel.map` returns a `MappedByteBuffer`. Apache Kafka stores each partition as segment files and serves consumers by `transferTo` from the page cache to the socket, so that a record that a consumer reads shortly after it was written never enters the JVM heap. The advantage disappears when TLS is enabled between broker and client, because the data must be encrypted in user space.

5.5.2 Worked trace: an epoll echo server

The server in the listing is a complete single-threaded echo server. It registers the listening socket with `epoll`, accepts all pending connections when the listening socket is ready and echoes data from every ready client socket.

epoll_echo.c C

```
#include <arpa/inet.h>
#include <errno.h>
#include <fcntl.h>
#include <netinet/in.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/epoll.h>
#include <sys/socket.h>
#include <unistd.h>

#define MAX_EVENTS 64

static void set_nonblocking(int fd) {
    fcntl(fd, F_SETFL, fcntl(fd, F_GETFL, 0) | O_NONBLOCK);
}

int main(int argc, char **argv) {
    int port = argc > 1 ? atoi(argv[1]) : 20501;
    int lfd = socket(AF_INET, SOCK_STREAM, 0);
    int one = 1;
    setsockopt(lfd, SOL_SOCKET, SO_REUSEADDR, &one, sizeof one);
    struct sockaddr_in a = { .sin_family = AF_INET, .sin_port = htons(port),
                             .sin_addr.s_addr = htonl(INADDR_LOOPBACK) };
    if (bind(lfd, (struct sockaddr *)&a, sizeof a) < 0 || listen(lfd, 128) < 0) {
        perror("bind/listen");
        return 1;
    }
    set_nonblocking(lfd);
    int ep = epoll_create1(0);
    struct epoll_event ev = { .events = EPOLLIN, .data.fd = lfd };
```

epoll_echo.c (continued) C

```c
    epoll_ctl(ep, EPOLL_CTL_ADD, lfd, &ev);
    struct epoll_event events[MAX_EVENTS];
    for (;;) {
        int n = epoll_wait(ep, events, MAX_EVENTS, -1);
        for (int i = 0; i < n; i++) {
            int fd = events[i].data.fd;
            if (fd == lfd) {                        /* new connection(s) */
                int c;
                while ((c = accept(lfd, NULL, NULL)) >= 0) {
                    set_nonblocking(c);
                    struct epoll_event ce = { .events = EPOLLIN, .data.fd = c };
                    epoll_ctl(ep, EPOLL_CTL_ADD, c, &ce);
                }
            } else {                                /* readable client socket */
                char buf[4096];
                ssize_t r = read(fd, buf, sizeof buf);
                if (r > 0) {
                    if (write(fd, buf, r) < 0) perror("write");
                } else if (r == 0 || errno != EAGAIN) {
                    close(fd);                      /* close removes it from the epoll set */
                }
            }
        }
    }
}
```

The client is a short Python script that opens N connections, sends one line on each and checks the echo:

client.py PYTHON

```python
import socket, sys, time
port = int(sys.argv[1]); n = int(sys.argv[2])
socks = [socket.create_connection(("127.0.0.1", port)) for _ in range(n)]
for i, s in enumerate(socks):
    s.sendall(b"hello-%d\n" % i)
ok = 0
for i, s in enumerate(socks):
    if s.recv(64) == b"hello-%d\n" % i:
        ok += 1
print("connections", n, "echoed correctly", ok)
if len(sys.argv) > 3:
    time.sleep(float(sys.argv[3]))      # keep the connections open for inspection
```

Two clients against the server, traced with strace (typical output) TERMINAL

```
$ strace -o strace_ep.txt -e trace=epoll_create1,epoll_ctl,epoll_wait,accept,read,write,close \
    ./epoll_echo 20501 &
$ python3 client.py 20501 2 0.5
connections 2 echoed correctly 2
$ sed -n '4,19p' strace_ep.txt
epoll_create1(0)                        = 4
epoll_ctl(4, EPOLL_CTL_ADD, 3, {events=EPOLLIN, data={u32=3, u64=3}}) = 0
epoll_wait(4, [{events=EPOLLIN, data={u32=3, u64=3}}], 64, -1) = 1
accept(3, NULL, NULL)                   = 5
epoll_ctl(4, EPOLL_CTL_ADD, 5, {events=EPOLLIN, data={u32=5, u64=5}}) = 0
accept(3, NULL, NULL)                   = 6
epoll_ctl(4, EPOLL_CTL_ADD, 6, {events=EPOLLIN, data={u32=6, u64=6}}) = 0
accept(3, NULL, NULL)                   = -1 EAGAIN (Resource temporarily unavailable)
epoll_wait(4, [{events=EPOLLIN, data={u32=5, u64=5}},
    {events=EPOLLIN, data={u32=6, u64=6}}], 64, -1) = 2
read(5, "hello-0\n", 4096)              = 8
write(5, "hello-0\n", 8)                = 8
read(6, "hello-1\n", 4096)              = 8
write(6, "hello-1\n", 8)                = 8
epoll_wait(4, [{events=EPOLLIN, data={u32=5, u64=5}}], 64, -1) = 1
read(5, "", 4096)                       = 0
close(5)                                = 0
```

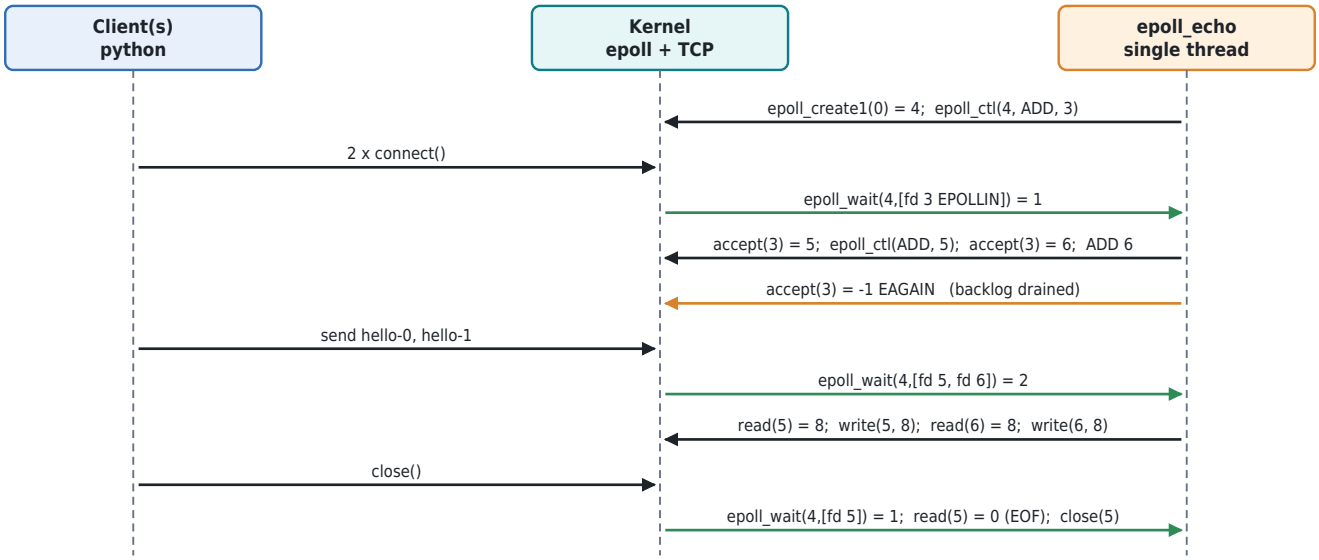


Figure 5.33. The server sleeps in epoll_wait and wakes only for the listening socket or for ready clients; one wake-up handled two clients, and the EAGAIN on the third accept shows that the backlog was drained.

1. The descriptors 0 to 2 exist already; the listening socket takes descriptor 3 and the `epoll` instance descriptor 4.
2. The first `epoll_wait` returns the listening socket (descriptor 3) because two connections are pending. The loop calls `accept` until it returns `EAGAIN`; the accepted sockets receive descriptors 5 and 6 and are added to the interest list.
3. The next `epoll_wait` returns both client sockets in a single call (`= 2`). One system call served two connections, which is how a single thread scales: the number of wake-ups depends on bursts, not on connections.
4. After a client closes, `read` returns 0 (end of file) and the server closes the descriptor; closing removes it from the interest list.
5. With 200 clients connected, the server had 205 descriptors (3 standard, listener, epoll instance and 200 sockets) and `Threads: 1` in `/proc/PID/status`.

5.5.3 Hands-on

The same design in Java NIO

The Java `Selector` is an `epoll` instance on Linux. The listing implements the same echo protocol; the program prints the implementation class to show that.

NioEcho.java JAVA

```
import java.io.IOException;
import java.net.InetSocketAddress;
import java.nio.ByteBuffer;
import java.nio.channels.SelectionKey;
import java.nio.channels.Selector;
import java.nio.channels.ServerSocketChannel;
import java.nio.channels.SocketChannel;
import java.util.Iterator;

public class NioEcho {
    public static void main(String[] args) throws IOException {
        int port = Integer.parseInt(args[0]);
        Selector selector = Selector.open();
        ServerSocketChannel server = ServerSocketChannel.open();
        server.bind(new InetSocketAddress("127.0.0.1", port));
        server.configureBlocking(false);
        server.register(selector, SelectionKey.OP_ACCEPT);
        System.out.println("listening on " + port + ", selector=" + selector.getClass().getName());
        ByteBuffer buf = ByteBuffer.allocate(4096);
        while (true) {
            selector.select();                      // blocks in epoll_wait
            Iterator<SelectionKey> it = selector.selectedKeys().iterator();
            while (it.hasNext()) {
                SelectionKey key = it.next();
                it.remove();
```

NioEcho.java (continued) JAVA

```
                if (key.isAcceptable()) {
                    SocketChannel c = server.accept();
                    if (c == null) continue;
                    c.configureBlocking(false);
                    c.register(selector, SelectionKey.OP_READ);
                } else if (key.isReadable()) {
                    SocketChannel c = (SocketChannel) key.channel();
                    buf.clear();
                    int n = c.read(buf);
                    if (n < 0) { c.close(); continue; }
                    buf.flip();
                    while (buf.hasRemaining()) c.write(buf);
                }
            }
        }
    }
}
```

The loop has the structure of the C program: one blocking call, then a dispatch on the ready keys. The `while (buf.hasRemaining())` write loop is a simplification; a production server registers `OP_WRITE` and keeps a per-connection output buffer, because a non-blocking write may accept only part of the data.

500 connections served by one selector thread (JDK 21, typical output) TERMINAL

```
$ java -Xmx64m NioEcho.java 20502 &
listening on 20502, selector=sun.nio.ch.EPollSelectorImpl
$ python3 client.py 20502 500 6 &
connections 500 echoed correctly 500
$ echo "fds: $(ls /proc/$JP/fd | wc -l)  threads: $(ls /proc/$JP/task | wc -l)"
fds: 508  threads: 19
$ grep -E "VmRSS|Threads" /proc/$JP/status
VmRSS:     89952 kB
Threads:        19
```

The 19 threads are the JVM's own: the main thread, garbage collector and compiler workers, the signal dispatcher and so on. None was added for the 500 connections. A blocking design with one platform thread per connection would have needed 500 more, each with a stack reservation of 1 MiB of address space.

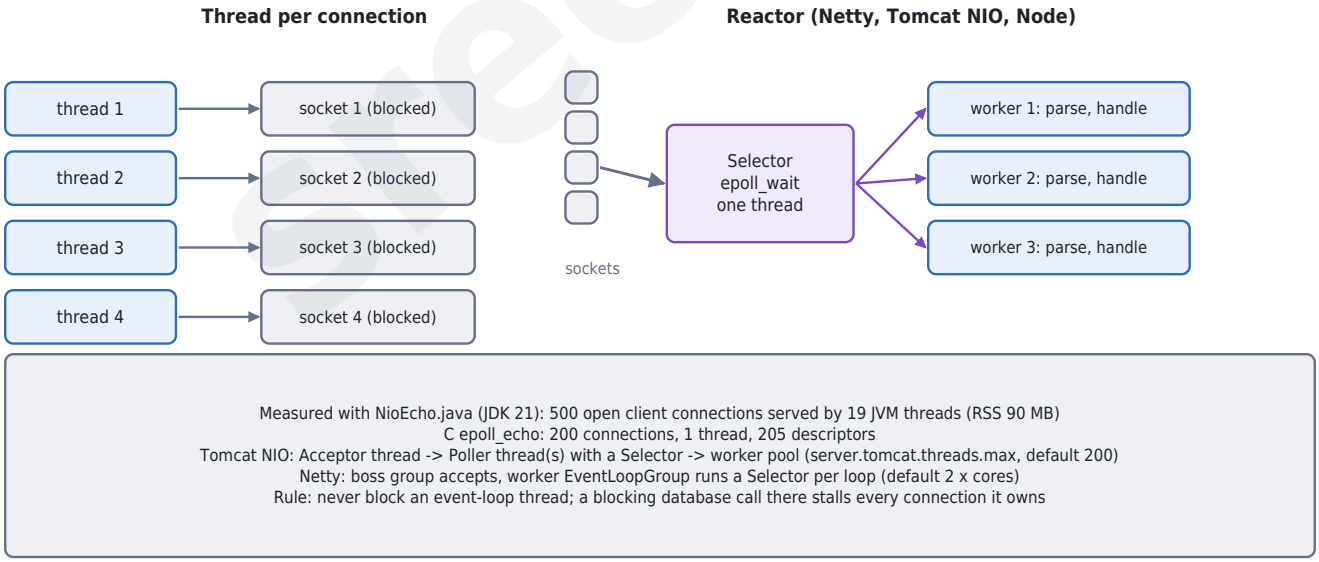


Figure 5.34. The thread-per-connection design needs one thread per socket; the reactor design lets one thread watch all sockets and hands the work to a small pool.

How the Java frameworks map onto the models

Tomcat in its default NIO connector (the one Spring Boot embeds) has an acceptor thread that accepts connections, one or more poller threads that run a `Selector` over idle connections and a worker pool that executes the servlet. A request waits in the poller until bytes arrive; a worker then runs the controller to completion and blocks on every database or HTTP call it makes. The relevant Spring Boot properties are `server.tomcat.threads.max` (default 200 workers), `server.tomcat.max-connections` (default 8192 open connections) and `server.tomcat.accept-count` (default 100 queued connections beyond that). Concurrency in this design is limited by the worker pool, not by the number of connections.

Netty, and Reactor Netty underneath Spring WebFlux, run an event loop per thread, each owning a `Selector`; a connection is bound to one loop for its life. The loops are few (twice the core count for plain Netty by default). The code on a loop must never block, because a blocked loop stops every connection assigned to it. With Java 21 virtual threads, blocking socket calls park the virtual thread and the JDK waits for readiness with an `epoll` poller internally, giving the blocking programming style on top of the multiplexing model.

5.5.4 Real-world incidents

Incident: a blocking call stalls every connection of an event loop

Scenario. A reactive gateway built on Netty validates requests by calling a licence database synchronously from the handler. After a database slowdown, latency of all clients, including those that never touch the licence check, rises from milliseconds to seconds.

Symptoms. CPU is nearly idle, the number of open connections grows and request timeouts occur for unrelated endpoints. Health checks served by the same loops time out and the orchestrator restarts healthy pods.

Diagnosis. A thread dump shows the event-loop threads, not the workers, sitting in a blocking call. The reproduction replaced the handler of `NioEcho` with one that sleeps two seconds when a message begins with the letter `s`, which stands in for a slow database call, and measured a second client.

A slow request blocks the selector thread (JDK 21.0.12, typical output) — TERMINAL

```
$ java NioEchoSlow.java 20503 &
$ python3 lat_client.py 20503
fast request answered after 1906 ms
slow request answered after 2006 ms
$ jstack $JP | grep -A5 '^"main"'
"main" #1 [15730] prio=5 os_prio=0 cpu=484.13ms elapsed=6.46s nid=15730 waiting on condition
   java.lang.Thread.State: TIMED_WAITING (sleeping)
        at java.lang.Thread.sleep0(java.base@21.0.12.1/Native Method)
        at java.lang.Thread.sleep(java.base@21.0.12.1/Thread.java:509)
        at NioEchoSlow.main(NioEchoSlow.java:37)
$ top -H -b -n1 -p $JP | sed -n 7,8p
  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
15730 root      20   0 4148144  92880  35176 S   0.0   1.1   0:00.47 java
```

The change to the server is one line, inserted after `buf.flip()` in `NioEcho` (the `main` method then declares `throws Exception` because of `Thread.sleep`), and the client sends the slow request on one connection and the fast request on another 100 ms later:

NioEchoSlow.java (fragment: line added to NioEcho) — JAVA

```java
buf.flip();
if (buf.get(0) == (byte) 0x73) Thread.sleep(2000);   // message starting with 's' = slow request
```

lat_client.py — PYTHON

```python
import socket, sys, time
port = int(sys.argv[1])
a = socket.create_connection(("127.0.0.1", port))
b = socket.create_connection(("127.0.0.1", port))
t0 = time.time()
a.sendall(b"slow-request\n")
time.sleep(0.1)
tb = time.time()
b.sendall(b"fast-request\n")
b.recv(64)
print("fast request answered after %.0f ms" % ((time.time() - tb) * 1000))
a.recv(64)
print("slow request answered after %.0f ms" % ((time.time() - t0) * 1000))
```

The fast request, sent 100 ms after the slow one on a different connection, was answered after 1906 ms: it waited for the loop thread to finish the 2000 ms call. The thread that owns the loop is in a sleeping or waiting state and uses 0.0 percent CPU, which explains why CPU graphs look healthy during the incident. In a real service the frame at the top of the stack would be a JDBC or blocking HTTP client method, which is the line to search for.

Root cause. Blocking work was executed on a thread whose only duty is to multiplex many connections. Every connection assigned to that loop waited for it.

Fix. Immediate: restart does not help; reduce the dependency timeout or bypass the check. Permanent: move blocking calls to a bounded worker pool (in Netty an `EventExecutorGroup`, in Reactor `publishOn(Schedulers.boundedElastic())`) or replace them with a non-blocking client; set timeouts on every outbound call. On Java 21, a virtual thread per request removes the restriction because blocking parks only that virtual thread.

Prevention. Run a blocking-call detector in tests (BlockHound for Reactor), alert on event-loop lag or pending-task counts, and keep the health endpoint independent of dependencies on the same loop.

5.5.5 Questions to expect

What is the difference between blocking and non-blocking I/O?

A blocking call suspends the thread until the operation can complete; a non-blocking call returns immediately with `EAGAIN` when it cannot. Non-blocking calls are combined with a readiness mechanism so that the program does not spin.

What is the difference between synchronous and asynchronous I/O?

In synchronous I/O (blocking, non-blocking and multiplexing) the calling thread performs the data transfer itself. In asynchronous I/O the program submits the operation and the kernel completes it, copy included, and signals completion.

Why is `epoll` more scalable than `select` and `poll`?

`select` and `poll` pass the entire descriptor set on every call and scan it, so the cost is proportional to the number of descriptors. `epoll` keeps the interest list in the kernel and returns only ready descriptors, so the cost is proportional to the number of events.

What is the difference between level-triggered and edge-triggered `epoll`?

Level-triggered reports a descriptor as long as it remains ready; edge-triggered reports only when its state changes and requires the program to read until `EAGAIN`. Edge-triggered reduces wake-ups but is easier to misuse.

Why does epoll not help for regular files?

Regular files are always reported as ready, so readiness carries no information. Event-loop frameworks run file I/O on separate threads.

How many threads does a Tomcat server need for 10000 idle keep-alive connections?

Few: the poller threads watch the idle sockets through a selector; a worker thread is used only while a request is processed. The number of workers (default 200) limits concurrent requests, and `max-connections` (default 8192) limits open connections.

What does `sendfile` do, and where is it used in Java?

It transfers data from a file descriptor to a socket inside the kernel, avoiding copies through user space. `FileChannel.transferTo` uses it, and Kafka relies on it to serve consumers from the page cache.

Why must code on a Netty event loop not block?

One loop thread serves many connections; a blocked thread delays all of them, as the reproduction showed with a 1906 ms delay for an unrelated request.

sneram

What is `io_uring`?

A Linux interface with shared submission and completion rings for asynchronous operations on files and sockets, which reduces system calls and supports real asynchronous completion.

KEY POINTS

Key points

- Every input has a wait phase and a copy phase; blocking, non-blocking, multiplexing, signal-driven and asynchronous models differ in who waits during each.
- `epoll` registers descriptors once and returns only ready ones; one thread served 500 connections in a JVM with 19 threads, and one C thread served 200.
- Zero-copy (`sendfile`, `mmap`) removes user-space copies; Kafka depends on it, TLS defeats it.
- Tomcat uses pollers plus a worker pool; Netty binds each connection to one event loop; blocking on a loop stalls every connection it owns.
- Virtual threads in Java 21 keep the blocking style while the JDK multiplexes underneath.

5.6 Logging and Log Rotation in Production

Logs are the file workload that production systems produce continuously and that nobody plans disk capacity for. A log file is written by one process, read by others, rotated by a third and deleted by a fourth, and each of these actors interacts with the file system semantics of the previous chapters: names versus inodes, open descriptors, ordered writes, synchronous cost. Failures in this area are among the most frequent operational incidents: the disk fills up, the space does not return after cleanup, the log disappears after rotation, or the service slows down because logging blocks.

5.6.1 Concept and mechanism

Where logs go

A Java service can send log events to a file, to standard output, to a network collector or to the system journal. The choices carry different failure modes.

Destination	Mechanism	Typical failure
File written by the framework	Logback `RollingFileAppender` or Log4j2 with its own rolling policy	disk full without a size cap, blocking when the disk is slow
File rotated by `logrotate`	external rename or copy-and-truncate	writer keeps the old inode, or lines lost at truncation
Standard output in a container	the runtime captures the stream into a file on the node and rotates it	node disk fills if the runtime limits are absent; `kubectl logs` loses rotated data
journald	structured binary journal written by `systemd-journald`, read with `journalctl`	rate limiting drops messages, journal size limits discard the oldest
Network (syslog, Fluentd, Kafka)	socket or agent	blocking or loss when the collector is down

Log destinations and their failure modes.

Rotation: rename versus truncate

A process that writes a log keeps a descriptor to the inode. Rotation must therefore decide what happens to that descriptor. Two strategies exist, and the experiment below used the same Java writer under both.

- **Rename and signal.** The rotating tool renames `app.log` to `app.log.1` and tells the process to reopen the path (a postrotate command such as `kill -HUP`, or a `create` directive for a new empty file). The renamed file keeps being written until the process reopens, because the descriptor follows the inode, not the name. Without the signal, the process writes to `app.log.1` forever and the new `app.log` stays empty.
- **Copy and truncate** (`copytruncate` in `logrotate`). The tool copies the file and truncates the original to zero length while the process keeps writing to it. No signal is needed, but the lines written between the copy and the truncate are lost, and the writer must have opened the file in append mode (`O_APPEND`) or its offset will leave a hole of zeros at the old position.

Java logging frameworks avoid both problems by rotating inside the process: Logback's `SizeAndTimeBasedRollingPolicy` and Log4j2's rolling policies close the old file and open the new one themselves. Spring Boot exposes `logging.logback.rollingpolicy.max-file-size` (default 10 MB), `max-history` (default 7) and `total-size-cap` (default 0, which means no cap and should be set explicitly).

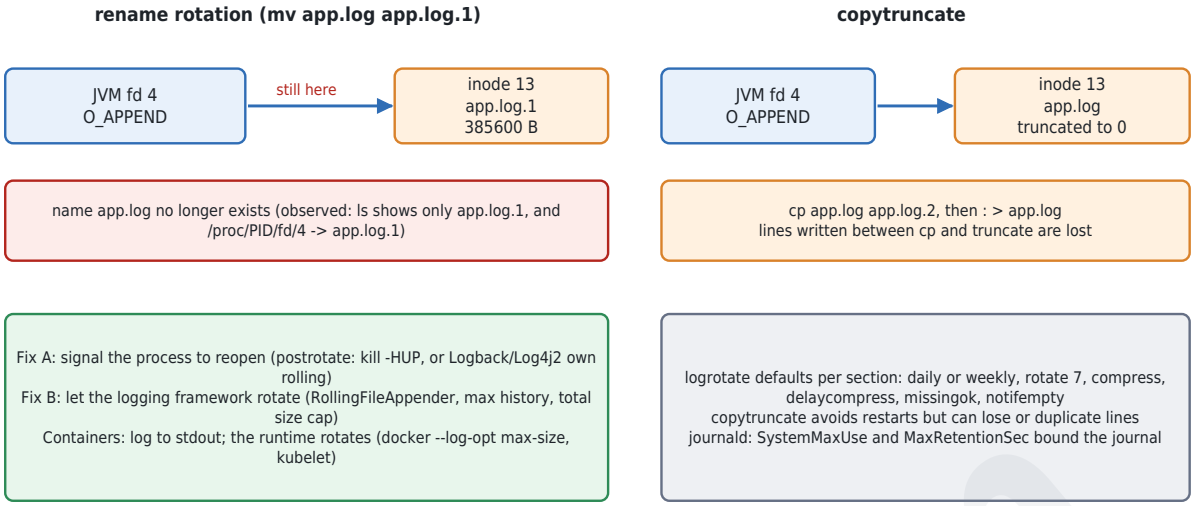
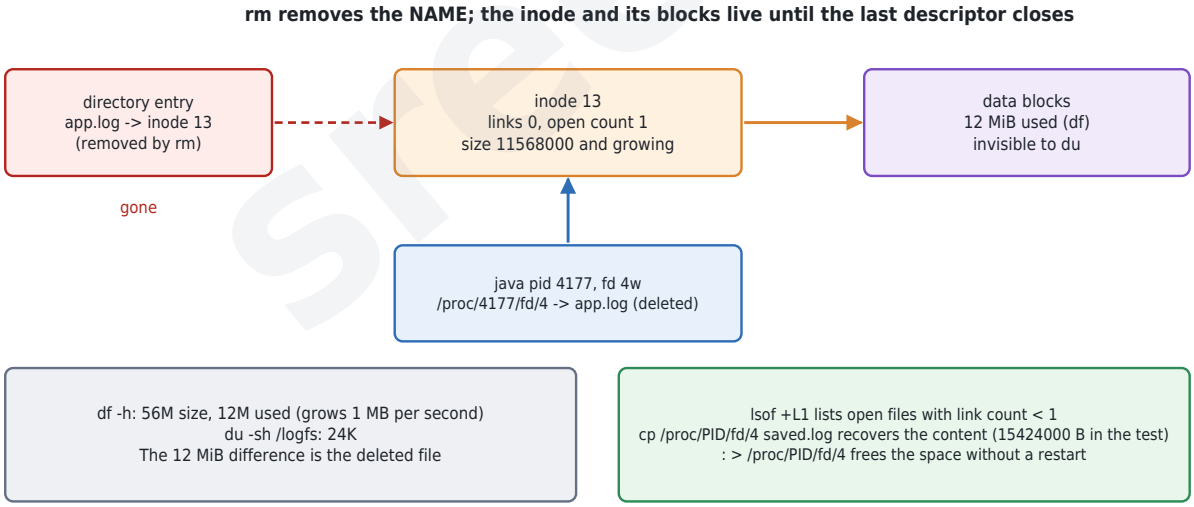


Figure 5.35. Rename rotation leaves the writer attached to the renamed inode; copytruncate keeps the name but can lose lines; in-process rolling and container stdout avoid both.

Deleted but open files

When a log file is deleted while a process holds it open, the directory entry disappears but the inode and its blocks remain until the last descriptor is closed. The process continues to write, the file keeps growing and no tool that walks directories can see it. This produces the situation in which `df` reports a full disk and `du` finds nothing to account for the space.



After truncate: df 15M -> 972K used, while lsof still showed the file as (deleted) with 964000 B.

After the process exited: 28K used. The space returns only when the open count reaches zero.

Figure 5.36. Deletion removes only the name; the inode survives with a link count of zero while the process holds a descriptor, and the blocks stay allocated.

The reverse view is a recovery technique: the deleted file is still reachable as `/proc/PID/fd/N`, so its contents can be copied out or truncated in place.

Journald and container logs

On systems with systemd, a service started as a unit writes standard output and standard error to the journal, which is queried with `journalctl -u NAME`, with options `-f` (follow), `--since`, `-p err` (priority) and `-o json`. The size of the journal is bounded by `SystemMaxUse` and `SystemKeepFree` in `journald.conf`, and the rate by `RateLimitIntervalSec` and `RateLimitBurst`; a service that logs in bursts can lose messages without any error. The build machine did not use systemd as its init system (PID 1 was a process-management shim), so `journalctl` output cannot be shown and the journald statements follow the manual pages.

For containers the convention is to log to standard output and let the platform handle the files. Docker's default `json-file` driver stores the stream on the host and has no size limit unless `--log-opt max-size` and `max-file` are set. Kubernetes writes container logs under `/var/log/pods` on the node and the kubelet rotates them (the settings `containerLogMaxSize` and `containerLogMaxFiles`, 10 MiB and 5 by default).

The cost of logging

A log call is a write. With a file appender it ends in a `write` call of a few hundred bytes, which is cheap while the page cache absorbs it and expensive when a thread must wait. Three situations make logging slow: a synchronous appender on a slow or saturated disk (every request thread blocks), a flush after every event (`immediateFlush` set to `true` costs one system call per event, but not an `fsync`) and a very high log level on a busy path. Asynchronous appenders (Logback `AsyncAppender`, Log4j2 asynchronous loggers) decouple the request thread from the write by a bounded queue; the policy for a full queue (block, drop) must be chosen deliberately.

5.6.2 Worked trace: what happens to a log file that is renamed and deleted

The test used `LogWriter.java`, which appends one kilobyte lines at a fixed rate through a `PrintWriter` opened in append mode.

LogWriter.java JAVA

```java
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;

public class LogWriter {
    public static void main(String[] args) throws Exception {
        String path = args[0];
        int kbPerSecond = Integer.parseInt(args[1]);
        String line = "2026-10-18T02:20:00 INFO  OrderService - processed order batch "
                + "x".repeat(900) + "\n";
        try (PrintWriter out = new PrintWriter(new FileWriter(path, true))) {
            for (int i = 0; i < 600; i++) {
                for (int k = 0; k < kbPerSecond; k++) out.print(line);
                out.flush();
                Thread.sleep(1000);
            }
        }
    }
}
```

Rename without a signal, then copy and truncate (writer rate 100 lines per second) TERMINAL

```
$ java LogWriter.java /logfs/var/log/app/app.log 100 &
$ ls -l /logfs/var/log/app
-rw-r--r-- 1 root root 192800 Oct 18 02:18 app.log
$ mv app.log app.log.1; sleep 2; ls -l; ls -l /proc/$JP/fd | grep log
-rw-r--r-- 1 root root 385600 Oct 18 02:18 app.log.1
l-wx------ 1 root root 64 Oct 18 02:18 4 -> /logfs/var/log/app/app.log.1
$ cp app.log.1 app.log.2; : > app.log.1; sleep 2; ls -l
-rw-r--r-- 1 root root 192800 Oct 18 02:18 app.log.1
-rw-r--r-- 1 root root 385600 Oct 18 02:18 app.log.2
```

After the `mv`, no file named `app.log` existed and the file kept growing under the new name, from 192800 to 385600 bytes; descriptor 4 of the process pointed to `app.log.1`. After the copy and truncate, `app.log.1` started again from zero and had grown back to 192800 bytes after two seconds, while `app.log.2` held the earlier 385600 bytes. Any lines written between `cp` and `: >` were in neither file. Because the writer opened the file with `O_APPEND`, no hole appeared.

5.6.3 Hands-on

Finding deleted files that are still open

Deleting an active log (the writer produces about 1 MB per second) — TERMINAL

```
$ df -h /logfs | tail -1
/dev/loop0       56M  9.3M   43M  18% /logfs
$ rm /logfs/app/app.log; sleep 2
$ df -h /logfs | tail -1
/dev/loop0       56M   12M   41M  22% /logfs
$ du -sh /logfs; ls -l /logfs/app
24K     /logfs
total 0
$ lsof +L1 | awk 'NR==1 || /logfs/'
COMMAND  PID USER  FD  TYPE DEVICE SIZE/OFF NLINK NODE NAME
java    4177 root  4w   REG    7,0 11568000     0   13 /logfs/app/app.log (deleted)
$ ls -l /proc/4177/fd | grep logfs
l-wx------ 1 root root 64 Oct 18 02:17 4 -> /logfs/app/app.log (deleted)
```

The inode number 13 and the link count 0 identify the file; `SIZE/OFF` of 11568000 bytes matches the 12 MiB difference between `df` and `du`. The size kept growing by about a megabyte per second while the directory was empty.

5.6.4 Real-world incidents

Incident: disk full because a deleted log file is still open

Scenario. A disk alert fires on a service host at 95 percent. An operator removes the large `app.log` and sees no change in `df`. The service writes to a 56 MiB volume in the reproduction; in production the volume is far larger and the effect is identical.

Symptoms. `df` stays high or keeps growing; `du -sh` on the mount point reports a few kilobytes; `ls` shows no large file; writes begin to fail with `No space left on device` later.

Diagnosis. The sign is a disagreement between `df` and `du`. The commands are the ones of the decision tree in the file system chapter.

Diagnosis and recovery (build machine, sizes as observed) — TERMINAL

```
$ lsof +L1 | awk 'NR==1 || /logfs/'
COMMAND  PID USER  FD  TYPE DEVICE SIZE/OFF NLINK NODE NAME
java    4177 root  4w   REG    7,0 11568000     0   13 /logfs/app/app.log (deleted)
$ cp /proc/4177/fd/4 /backup/recovered.log; ls -l /backup/recovered.log
-rw-r--r-- 1 root root 15424000 Oct 18 02:17 /backup/recovered.log
$ df -h /logfs | tail -1
/dev/loop0       56M   15M   37M  29% /logfs
$ : > /proc/4177/fd/4; sleep 1; df -h /logfs | tail -1
/dev/loop0       56M  972K   51M   2% /logfs
$ lsof +L1 | awk 'NR==1 || /logfs/'
COMMAND  PID USER  FD  TYPE DEVICE SIZE/OFF NLINK NODE NAME
java    4177 root  4w   REG    7,0   964000     0   13 /logfs/app/app.log (deleted)
$ kill 4177; sleep 1; df -h /logfs | tail -1
/dev/loop0       56M   28K   52M   1% /logfs
```

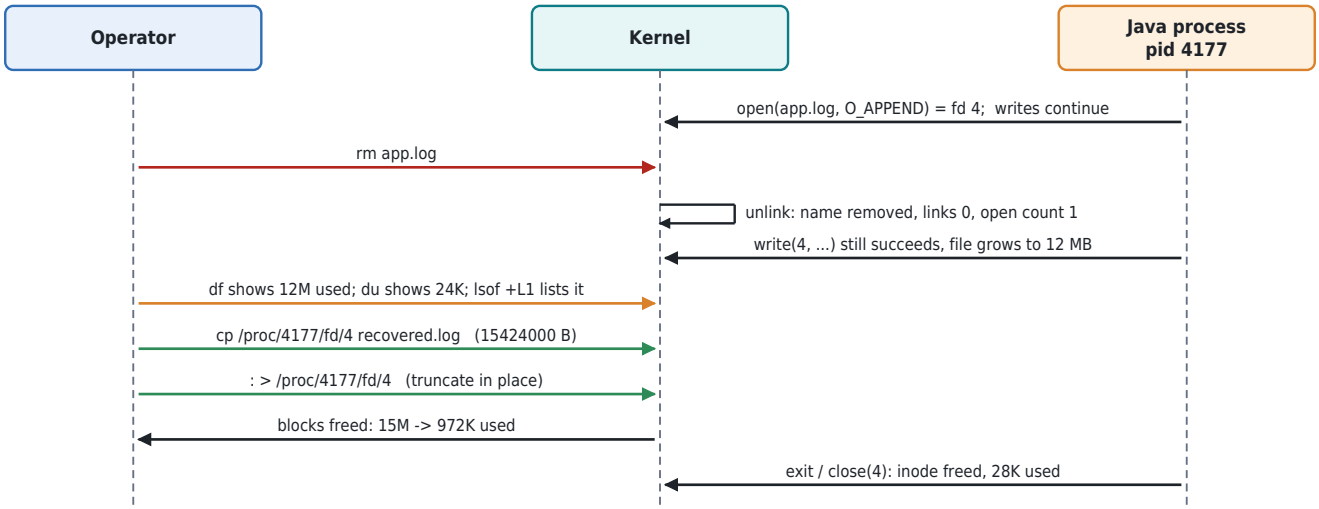


Figure 5.37. The sequence shows that the process keeps writing after the removal, that the content can be copied out of the descriptor and that the blocks are freed by the truncation or when the process exits.

Root cause. The file was removed by name while a process still held it open. The kernel keeps the inode until the last descriptor closes, so the blocks stayed allocated and the writer continued to add to them.

Fix. Immediate: truncate in place with `: > /proc/PID/fd/N`, which frees the blocks without restarting the process (after copying out the content if it is needed); or restart or signal the service so that it reopens its log. Permanent: never delete an active log; truncate it (`: > file`) or rotate it with a mechanism the writer cooperates with.

Prevention. Alert on the difference between `df` and the sum of `du` (or on `lsof +L1` returning large entries) and on file system usage with a margin that gives time to react. Cap the log size in the framework (`total-size-cap`) so that no log can fill a volume.

Incident: a log flood fills the disk

Scenario. A release enables `DEBUG` logging for a package to investigate a defect and the change is not reverted. Under production traffic the service writes tens of megabytes per second.

Symptoms. The disk fills within minutes. The service begins to fail writes and the host cannot create temporary files, so unrelated components fail as well. In the reproduction a writer producing 20 MB per second filled a 56 MiB volume within about three seconds.

Diagnosis. The questions are which volume, which directory and which file, and whether the growth continues.

The volume and the offending file (build machine; volume 56 MiB) TERMINAL

```
$ df -h /logfs | tail -1
/dev/loop0       56M   55M     0 100% /logfs
$ du -xh --max-depth=3 /logfs | sort -rh | head -4
55M     /logfs/var/log/app
55M     /logfs/var/log
55M     /logfs/var
55M     /logfs
$ ls -lh /logfs/var/log/app
total 55M
-rw-r--r-- 1 root root 55M Oct 18 02:18 debug.log
```

The volume is at 100 percent and the directory listing identifies a single file, `debug.log`, which accounts for the whole usage; the log level setting of the service confirms the cause.

Root cause. A log level change multiplied the log volume by several orders of magnitude, and nothing bounded the size of the log file or reserved space for the rest of the system.

Fix. Immediate: lower the level at runtime (Spring Boot Actuator `POST /actuator/loggers/PACKAGE` with `{"configuredLevel": "INFO"}`, or the JMX operation of the logging framework), then free space by truncating the file (`: > debug.log`) rather than deleting it while the process runs, as the previous incident showed. Permanent: configure a size-based rolling policy with a total cap, put logs on a separate volume or in a container with an `ephemeral-storage` limit, and require a change record and an expiry for temporary debug levels.

Prevention. Alert on the rate of growth of the log volume (bytes per minute), not only on the percentage used. Sample or rate-limit repetitive messages. Keep the disk that hosts the log separate from the one that hosts the application or the database.

Incident: accidental removal of a file that a process has open

Scenario. An operator cleaning a data directory removes `pricing.csv`, a lookup table that a long-running process keeps open. The process continues to work, but a restart would fail and no backup of the latest version exists.

Symptoms. The file no longer appears in `ls`. The service still answers, since it holds a descriptor. After a planned restart the service would fail to load its configuration.

Diagnosis and recovery. While the process runs, the content is still reachable. The reproduction used a `sleep` process that holds the file as its standard input, which is an open descriptor like any other.

Recovery through /proc (build machine) — TERMINAL

```
$ md5sum pricing.csv
9f3248a1cbf2e5be3d6309d4456b375f  pricing.csv
$ (sleep 120 < pricing.csv & echo $! > pid)
$ rm pricing.csv; ls
pid
$ ls -l /proc/$(cat pid)/fd/0
lr-x------ 1 root root 64 Oct 18 02:35 /proc/15496/fd/0 -> /tmp/osw_o5/rm_test/pricing.csv (deleted)
$ cp /proc/$(cat pid)/fd/0 pricing.csv; md5sum pricing.csv
9f3248a1cbf2e5be3d6309d4456b375f  pricing.csv
```

The checksum of the recovered file equals the original. The method works only while the descriptor is open and only for the part of the file that exists on disk; if the process has already exited, the blocks are free and only forensic tools or backups remain, with poor results on ext4.

Root cause. A removal by name on a file in use, without a backup or a snapshot.

Fix. Copy the file from `/proc/PID/fd/N` immediately, verify the checksum, restore the name and only then restart anything. Do not close or restart the holding process before the copy.

Prevention. Keep configuration and data under version control or in a backed-up volume; use snapshots (LVM, ZFS, cloud volume snapshots) before cleanups; run cleanups with `find ... -print` first, then `-delete`; restrict write permission of production directories so that ordinary operators cannot delete application data.

5.6.5 Questions to expect

Why does the free space not increase after a log file is deleted?

The process that writes the log still holds an open descriptor, so the inode and blocks are retained until it closes the file. `lsof +L1` lists such files, and truncating through `/proc/PID/fd/N` or restarting the process releases the space.

What is the difference between `logrotate` with `copytruncate` and rotation with a signal?

With a signal the file is renamed and the process reopens the path, which loses nothing but requires cooperation. With `copytruncate` the file is copied and truncated in place; no cooperation is needed but lines written between the copy and the truncate are lost.

Why does a service keep writing to `app.log.1` after rotation?

A descriptor refers to an inode, not to a name. After a rename the descriptor still refers to the renamed file, until the process reopens the path.

Where should a containerised Java service write its logs?

To standard output, so that the runtime collects and rotates them with configured limits. Logging to a file inside the writable layer costs copy-on-write overhead and disappears with the container.

How can logging slow a service down?

A synchronous appender blocks request threads when the disk or the collector is slow. An asynchronous appender with a bounded queue decouples them, but the policy for a full queue (block or drop) must be defined.

sreram

How is a disk-full condition diagnosed quickly?

`df -h` for the volume, `df -i` for inodes, `du -xh --max-depth=N` for the directory, `lsof +L1` for deleted open files. The disagreement between `df` and `du` points to open deleted files.

How can a deleted file that a process has open be recovered?

Copy it from `/proc/PID/fd/N` while the process is running. The kernel keeps the inode alive as long as the descriptor is open.

Which limits should bound logs in production?

The framework (`max-file-size`, `max-history`, `total-size-cap`), the runtime for container logs (`max-size`, `max-file`, kubelet limits) and the journal (`SystemMaxUse`), plus a volume or ephemeral-storage limit as the last barrier.

KEY POINTS

Key points

- A writer holds an inode, not a name: renaming does not redirect it and deleting does not free space while it is open.
- `df` greater than `du` means deleted-but-open files; `lsof +L1` finds them, `cp` or `: >` on `/proc/PID/fd/N` recovers or frees them without a restart.
- Rotate inside the framework or with a signal; `copytruncate` can lose lines; containers log to standard output with runtime-enforced limits.
- Bound every log: `total-size-cap`, rolling policy, separate volume, alert on growth rate.
- Logging is I/O on the request path unless it is asynchronous; size the queue and decide the full-queue policy.

sreeram

PART 6

Containers, Security and Production Playbook

Namespaces, cgroups, privilege, graceful shutdown and the incident method that ties every earlier Part together.

In this part

- **6.1** Containers from First Principles
- **6.2** Linux Security, Services and Graceful Shutdown
- **6.3** The Production Playbook
- **6.4** Interview Question Bank
- **6.5** Cheat Sheets and Study Plan
- **6.6** Putting It Together: One Request Through the Whole Stack

6.1 Containers from First Principles

A container is not a kernel object. The Linux kernel has no data structure named "container". What is called a container is an ordinary process (or a tree of processes) whose view of the system has been narrowed by **namespaces**, whose resource consumption has been bounded by **control groups** (cgroups), and whose root file system has been replaced by an image mounted through a **union file system**. Docker, containerd, CRI-O and Kubernetes automate the assembly of these three kernel features and add distribution, scheduling and lifecycle management on top.

This distinction determines how containers fail in production. A Spring Boot service in a pod still uses the host kernel, the host scheduler, the host page cache and the host network stack. A memory limit is enforced by the kernel out-of-memory killer, a CPU limit by the CFS bandwidth controller, and a process that ignores SIGTERM is terminated by the same signal rules as on a bare host. A diagnosis therefore starts from the Linux mechanisms described in the earlier Parts, applied to the cgroup that contains the process.

The chapter derives containers from the three kernel features, builds one by hand with `unshare` and `chroot`, measures the limits with a memory cgroup and a CPU quota on the build machine, and closes with the mapping from Kubernetes requests and limits to cgroup files and to the out-of-memory score.

6.1.1 Concept and mechanism

Namespaces: what a process can see

A namespace wraps a global kernel resource so that the processes inside observe a private instance of it. The kernel provides eight types. Each process belongs to exactly one namespace of each type; the membership is visible as symbolic links under `/proc/PID/ns/`.

Type	Created by	Effect inside the namespace
`pid`	`CLONE_NEWPID, unshare --pid`	Own PID numbering; the first process is PID 1 and acts as init.
`mnt`	`CLONE_NEWNS, unshare --mount`	Private mount table; a mount made inside is invisible outside.
`uts`	`CLONE_NEWUTS, unshare --uts`	Own hostname and NIS domain name.
`ipc`	`CLONE_NEWIPC, unshare --ipc`	Own System V IPC objects and POSIX message queues.
`net`	`CLONE_NEWNET, unshare --net`	Own interfaces, routing table, firewall rules and port space.
`user`	`CLONE_NEWUSER, unshare --user`	Own UID/GID mapping; root inside may be an unprivileged UID outside.
`cgroup`	`CLONE_NEWCGROUP`	Own view of the cgroup hierarchy root.
`time`	`CLONE_NEWTIME`	Own offsets for the monotonic and boot clocks.

Namespace types and the resource each one isolates.

A namespace is created with `clone()` flags at process creation, with `unshare()` for the calling process, or joined with `setns()`. The `nsenter` command joins the namespaces of an existing process, which is how `docker exec` and `kubectl exec` place a shell inside a running container.

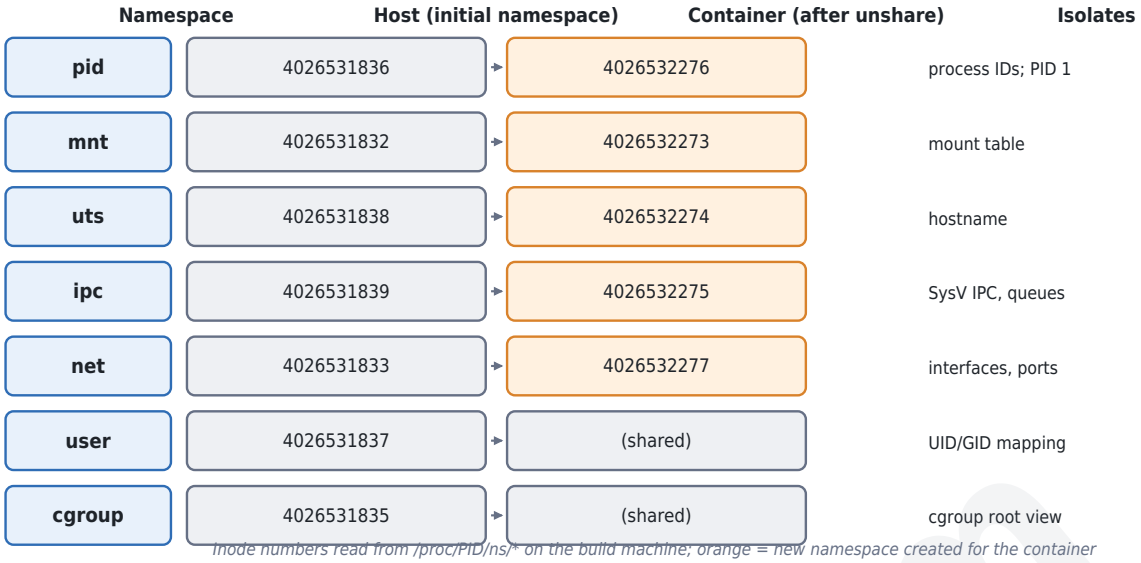


Figure 6.1. Namespaces are identified by inode numbers: the container created with unshare shares the user and cgroup namespaces with the host but owns new pid, mnt, uts, ipc and net namespaces.

The pid namespace deserves particular attention because it changes the meaning of process identity. A process has one PID per namespace level in which it is visible. The same task is PID 1275 in the host namespace and PID 1 inside the container; the field `NSpid` in `/proc/PID/status` lists both values. Signals sent from the host with the host PID reach the task; the task cannot see or signal host processes at all.

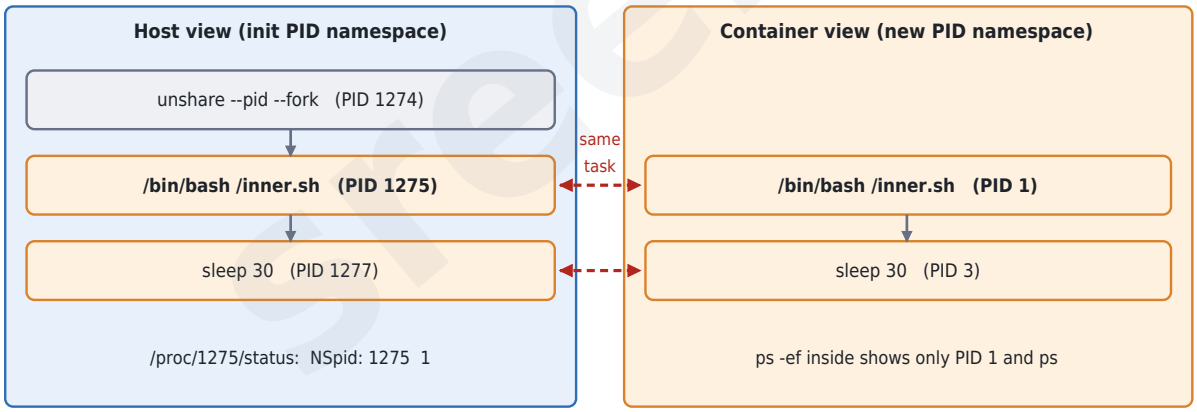


Figure 6.2. One process, two identities: the container init is PID 1 inside and PID 1275 on the host, and ps inside the container lists only the processes of its own namespace.

Control groups: how much a process can use

A cgroup is a node in a tree to which processes are attached and on which controllers enforce limits and accounting. The memory controller limits resident memory and page cache, the cpu controller limits and weights CPU time, the pids controller limits the number of tasks, and the blkio (v1) or io (v2) controller limits block device bandwidth.

Two interface generations coexist in the field. In **cgroup v1** each controller owns a separate hierarchy mounted under `/sys/fs/cgroup/<controller>/`. In **cgroup v2** there is a single unified hierarchy at `/sys/fs/cgroup/` and every controller appears as files in the same directory. The build machine used for the experiments in this Part runs a hybrid layout: v1 controller mounts plus a v2 mount at `/sys/fs/cgroup/unified` (`stat -fc %T` reports `cgroup2fs` for it). Linux 6.18 and JDK 21 were used throughout.

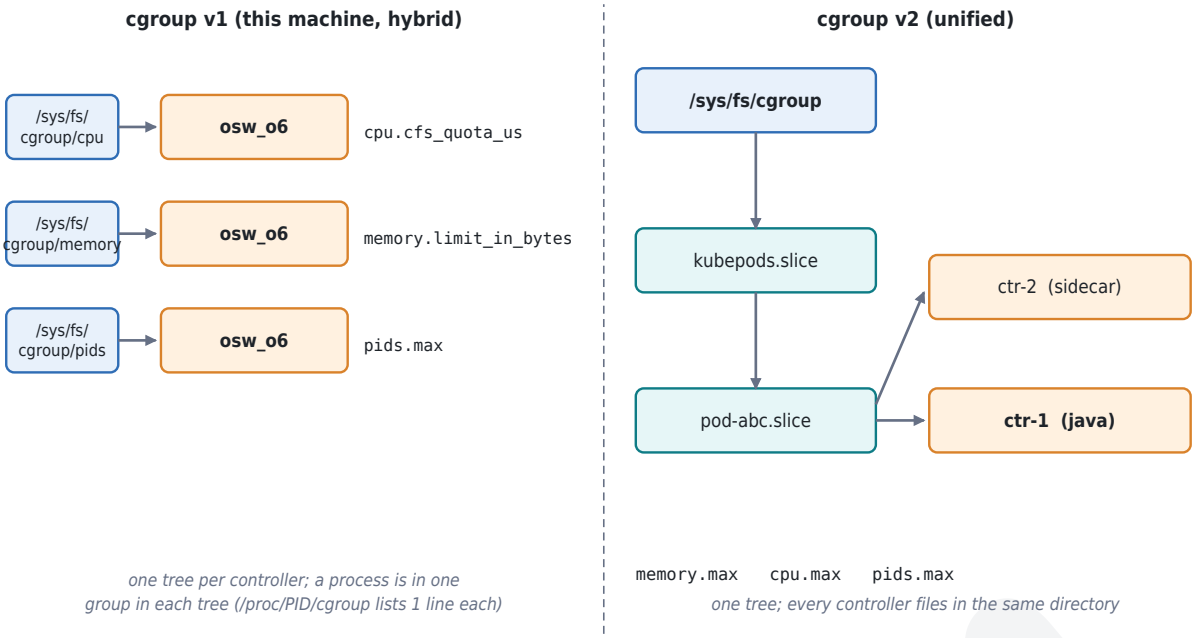


Figure 6.3. In v1 every controller has its own tree, so a process has one path per controller; in v2 a single tree carries all controllers and a pod directory contains one directory per container.

Purpose	cgroup v1	cgroup v2
Memory limit	`memory.limit_in_bytes`	`memory.max`
Memory in use	`memory.usage_in_bytes`	`memory.current`
CPU hard cap	`cpu.cfs_quota_us`, `cpu.cfs_period_us`	`cpu.max`
CPU relative weight	`cpu.shares` (default 1024)	`cpu.weight` (default 100)
Task limit	`pids.max`	`pids.max`
Throttling counters	`cpu.stat` (`nr_throttled`)	`cpu.stat` (`nr_throttled`)

Equivalent control files in cgroup v1 and v2.

Union file systems: what a process sees as its root

An image is a stack of read-only layers. A container adds one writable layer and presents the union as a single directory tree. Linux implements this with **overlayfs**: `lowerdir` holds the read-only layers, `upperdir` holds the writable layer, `workdir` is scratch space, and the mount exposes the union. A write to a file that exists only in a lower layer first **copies the whole file up** to the upper layer. A deletion creates a **whiteout**, a character device with major and minor number 0, in the upper layer.

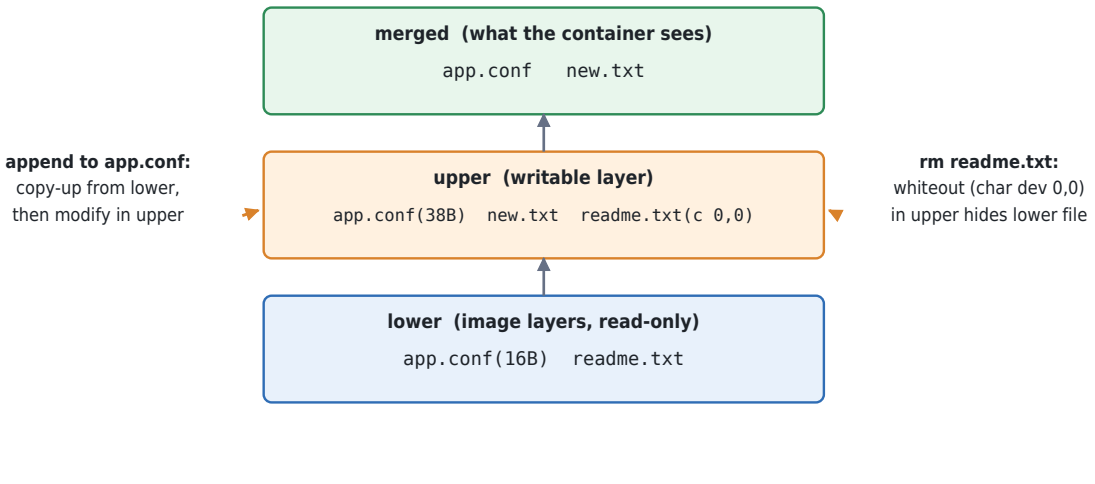


Figure 6.4. Writes and deletions land only in the upper layer: the lower layer stays unchanged and a deleted file becomes a whiteout device node.

Copy-up has a performance consequence: appending one byte to a 2 GB file in a lower layer first copies 2 GB. Databases and large log files therefore belong on a volume, which is a bind mount outside overlayfs, rather than in the writable layer.

Docker architecture and the runtime chain

The `docker` command is a client of the `dockerd` daemon through a REST API. `dockerd` manages images and networks and delegates container lifecycle to `containerd`. For each container `containerd` starts a `containerd-shim` process, which stays alive as the parent of the container process and holds its standard streams. The shim invokes `runc`, the reference implementation of the Open Container Initiative (OCI) runtime specification. `runc` creates the namespaces, joins the cgroup, applies capabilities and the seccomp profile, mounts the root file system, executes the entrypoint, and exits. Kubernetes talks to `containerd` or CRI-O directly through the Container Runtime Interface; `dockerd` is not required.

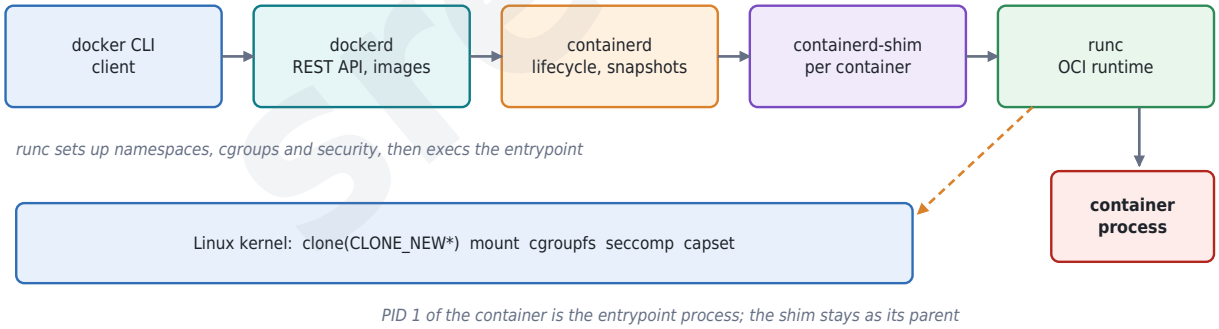


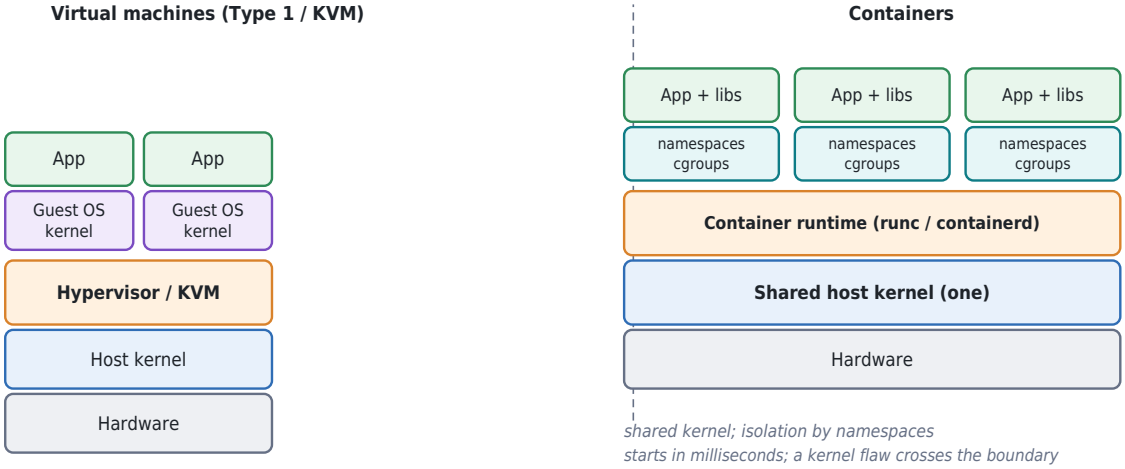
Figure 6.5. The runtime chain ends in runc, which configures the kernel features and then execs the entrypoint, which becomes PID 1 of the container.

Containers compared with virtual machines

A virtual machine virtualises hardware: a hypervisor presents virtual CPUs, memory and devices, and each guest runs its own kernel. A container virtualises the operating system interface: all containers share one kernel. The consequences are practical. A container starts in milliseconds and carries no kernel memory overhead, but a kernel vulnerability is shared by all containers on the host, and a container cannot run a different kernel version or a different operating system family than the host.

A **Type 1** hypervisor runs directly on the hardware (Xen, VMware ESXi, Hyper-V). **KVM** turns the Linux kernel itself into a Type 1 hypervisor: the `kvm` module uses the processor virtualisation extensions (Intel VT-x, AMD-V), and each guest is a user-space process (usually QEMU) whose virtual CPUs are ordinary threads scheduled by the host. A **Type 2** hypervisor is an application on a host operating system (VirtualBox, VMware Workstation). Most cloud virtual machines

are KVM or Xen guests; a Kubernetes node is therefore often a virtual machine, and the **steal** time reported by the guest kernel is the time its virtual CPUs were runnable while the hypervisor ran something else.



Type 1: runs on the hardware (KVM inside the Linux kernel, Xen, ESXi).
Type 2: runs as a process on a host OS (VirtualBox, VMware Workstation).

Figure 6.6. Virtual machines carry a guest kernel per instance above a hypervisor; containers share the single host kernel and differ only in namespaces and cgroups.

Kubernetes requests, limits and quality of service

A Kubernetes container declares `resources.requests` (used by the scheduler to place the pod) and `resources.limits` (enforced by the kernel). The kubelet translates them into cgroup settings and assigns a **quality of service (QoS) class** that determines the kernel out-of-memory score.

Setting	Kernel mechanism	Behaviour at the boundary
`limits.memory`	memory.limit_in_bytes or memory.max	Allocation beyond the limit triggers the cgroup OOM killer (exit 137).
`limits.cpu`	cpu.cfs_quota_us / period, or cpu.max	The group is throttled for the rest of the 100 ms period.
`requests.cpu`	cpu.shares or cpu.weight	Relative weight under contention; no cap when the node is idle.
`requests.memory`	Scheduling only (and the OOM score)	Not enforced as a floor; sets oom_score_adj for Burstable pods.

Mapping from pod resources to kernel settings.

sreram

QoS class	Rule (every container)	oom_score_adj	cgroup parent
Guaranteed	requests = limits (cpu and memory)	**-997**	kubepods/pod<uid>
Burstable	at least one request or limit set; not equal	**2 .. 999**	kubepods/burstable/pod<uid>
BestEffort	no requests, no limits	**1000**	kubepods/besteffort/pod<uid>

Burstable: adj = 1000 - (1000 x memory request / node capacity), clamped to 2 .. 999

Kernel OOM kill order: highest oom_score_adj first

Figure 6.7. Guaranteed pods have oom_score_adj of -997 and are killed last; BestEffort pods have 1000 and are killed first; Burstable pods fall between according to their memory request.

The Burstable formula is `oom_score_adj = 1000 - (1000 * memory_request / node_memory_capacity)`, clamped to the range 2 to 999. A container that requests 1 GiB on a node with 16 GiB receives about 938; a container that requests 8 GiB receives 500. The class is a statement about who is sacrificed first, not a guarantee of safety: a Guaranteed pod that exceeds its own limit is still killed by the cgroup OOM killer, because the limit applies regardless of the score.

6.1.2 Worked trace

The following sequence builds a container without Docker. The root file system is a directory holding `bash`, a handful of utilities and the shared libraries they need (copied with `ldd`). Each numbered step corresponds to a kernel action.

1. `unshare --pid --fork --mount-proc=ROOT/proc --uts --ipc --net --mount` calls `unshare(2)` with five `CLONE_NEW*` flags. Because a new pid namespace applies only to children, `--fork` makes `unshare` fork; the child becomes PID 1 of the new namespace.
2. `--mount-proc` mounts a fresh `proc` file system at `ROOT/proc` inside the new mount namespace so that `ps` reads the new PID table instead of the host table.
3. `chroot ROOT /bin/bash` changes the root directory of the process. From this point `/` is `ROOT`; `/etc`, `/home` and every host path are unreachable by name.
4. The script runs `hostname box1`. The change affects only the new uts namespace; the host keeps its name.
5. `ps -ef` reads `/proc` and finds two processes, PID 1 (the script) and the `ps` itself. The network namespace contains only a loopback interface in the down state, so the container has no connectivity.
6. From the host, `/proc/1275/status` shows `NSpid: 1275 1`, and `/proc/1275/root` resolves to the container root directory: the process is an ordinary host process observed through two views.

A real container runtime adds, to these steps, a cgroup assignment (so that limits apply), a capability bounding set, a seccomp filter, a pivot_root instead of chroot (to remove the old root from the mount table), and a virtual Ethernet pair connecting the network namespace to a bridge.

6.1.3 Hands-on

All experiments ran as root on the build machine (2 CPUs, 8 GB, Linux 6.18.44, JDK 21, hybrid cgroup v1). Each cgroup used a name private to the experiment, and every process was started with a time bound.

A container built by hand

The root file system was assembled from host binaries and their libraries; `inner.sh` is the script executed as PID 1.

inner.sh BASH

```bash
hostname box1
echo "hostname=$(hostname) pid=$$"
ps -ef
grep NSpid /proc/self/status
ls -l /proc/self/ns | head -4
ls /
sleep 30 &
sleep 30
```

Start the container and observe it from outside (PIDs are typical) TERMINAL

```
$ unshare --pid --fork --mount-proc=$ROOT/proc --uts --ipc --net --mount \
    chroot $ROOT /bin/bash /inner.sh
hostname=box1 pid=1
UID          PID    PPID  C STIME TTY          TIME CMD
0              1       0  0 02:14 ?        00:00:00 /bin/bash /inner.sh
0              4       1  0 02:14 ?        00:00:00 ps -ef
NSpid:  5
total 0
lrwxrwxrwx 1 0 0 0 Oct 18 02:14 cgroup -> cgroup:[4026531835]
lrwxrwxrwx 1 0 0 0 Oct 18 02:14 ipc -> ipc:[4026532275]
lrwxrwxrwx 1 0 0 0 Oct 18 02:14 mnt -> mnt:[4026532273]
bin  dev  etc  inner.sh  lib  lib64  proc  tmp
$ hostname
vm
$ grep NSpid /proc/1275/status
NSpid:  1275    1
$ readlink /proc/1275/root
/tmp/osw_06/rootfs
```

The container hostname is `box1` while the host remains `vm`. The `cgroup` namespace inode is the host value because `unshare` was not asked to create a cgroup namespace. Inside the container the user IDs print as `0` rather than `root` because `/etc/passwd` is absent from the minimal root.

A memory limit enforced by the kernel

A memory cgroup was created, limited to 64 MiB, and a C program that allocates and touches 10 MB every 50 ms was started inside it.

hog.c C

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

int main(void) {
    size_t chunk = 10UL << 20;
    int n = 0;
    for (;;) {
        char *p = malloc(chunk);
        if (!p) { puts("malloc failed"); return 1; }
        memset(p, 1, chunk);
        n++;
        printf("touched %d MB\n", n * 10);
        fflush(stdout);
        usleep(50000);
    }
}
```

Run the allocator inside a 64 MiB memory cgroup TERMINAL

```
$ M=/sys/fs/cgroup/memory/osw_06
$ echo $((64*1024*1024)) > $M/memory.limit_in_bytes
$ ( echo $BASHPID > $M/cgroup.procs; exec timeout 20 ./hog ) > hog.out
Killed
$ echo "exit=$?"
exit=137
$ tail -2 hog.out
touched 50 MB
```

sreeram

Run the allocator inside a 64 MiB memory cgroup (continued) TERMINAL

```
touched 60 MB
$ cat $M/memory.max_usage_in_bytes
67108864
$ grep oom_kill $M/memory.oom_control
oom_kill 1
$ dmesg | tail -2
oom-kill:constraint=CONSTRAINT_MEMCG,nodemask=(null),cpuset=/,mems_allowed=0,oom_memcg=/osw_o6,
  task_memcg=/osw_o6,task=hog,pid=1809,uid=0
Memory cgroup out of memory: Killed process 1809 (hog) total-vm:74420kB, anon-rss:65024kB,
  file-rss:1640kB, shmem-rss:0kB, UID:0 pgtables:180kB oom_score_adj:0
```

The program printed "touched 60 MB" and died while touching the next chunk. The usage counter stopped at exactly 67,108,864 bytes, the kernel log names `CONSTRAINT_MEMCG` (a cgroup limit, not host exhaustion), and the exit status 137 equals 128 plus signal 9. The line `constraint=CONSTRAINT_MEMCG` is the decisive evidence that distinguishes a container kill from a host-wide out-of-memory event, which reports `CONSTRAINT_NONE`.

A CPU quota and its effect on latency

A CPU cgroup with `cpu.cfs_quota_us` set to 25000 and the default period of 100000 microseconds allows 25 ms of CPU time per 100 ms. The Java program below executes 150 sequential requests, each requiring 10 ms of CPU, and prints percentile latencies.

Lat.java JAVA

```java
import java.util.Arrays;

public class Lat {
    static long spin(long nanos) {
        long end = System.nanoTime() + nanos;
        long x = 0;
        while (System.nanoTime() < end) { x++; }
        return x;
    }

    public static void main(String[] a) throws Exception {
        int n = 150;
        long[] lat = new long[n];
        for (int i = 0; i < n; i++) {
            long t0 = System.nanoTime();
            spin(10_000_000L);                  // 10 ms of CPU work
            lat[i] = (System.nanoTime() - t0) / 1_000_000;
        }
        Arrays.sort(lat);
        System.out.printf("requests=%d p50=%dms p99=%dms max=%dms%n",
            n, lat[n / 2], lat[(int) (n * 0.99)], lat[n - 1]);
    }
}
```

The same program without and with a 25% CPU quota (measured on the build machine) TERMINAL

```
$ java -cp . Lat
requests=150 p50=10ms p99=15ms max=23ms
$ echo 25000 > /sys/fs/cgroup/cpu/osw_o6/cpu.cfs_quota_us
$ ( echo $BASHPID > /sys/fs/cgroup/cpu/osw_o6/cgroup.procs; java -cp . Lat )
requests=150 p50=10ms p99=82ms max=84ms
$ cat /sys/fs/cgroup/cpu/osw_o6/cpu.stat
nr_periods 99
nr_throttled 85
throttled_time 4809441318
```

The median is unchanged, which is why average-based dashboards stay green, while the 99th percentile grows from 15 ms to 82 ms. A request that begins late in a period exhausts the quota, and the whole group sleeps until the next period starts. The counters are cumulative for the cgroup: an earlier 5 second CPU burner had already contributed 47 periods, 36 of them throttled, so the Java run added 52 periods, 49 throttled and 3.51 s of throttled time. The counters `nr_throttled` and `throttled_time` (nanoseconds) in `cpu.stat` expose the effect directly; the incident case studies in this Part use them as the primary signal.

sreram

Overlay mounts and copy-up

overlay.sh BASH

```
mkdir -p ov/lower ov/upper ov/work ov/merged
echo "base layer file" > ov/lower/app.conf
echo "only in lower" > ov/lower/readme.txt
unshare -m bash -c '
  mount -t overlay overlay -o lowerdir=ov/lower,upperdir=ov/upper,workdir=ov/work ov/merged
  echo "modified in container" >> ov/merged/app.conf
  rm ov/merged/readme.txt
  echo "new" > ov/merged/new.txt
  ls -l ov/upper; cat ov/upper/app.conf; ls ov/lower; ls ov/merged
'
```

Result of the overlay experiment TERMINAL

```
$ bash overlay.sh
total 8
-rw-r--r-- 1 root root   38 Oct 18 02:18 app.conf
-rw-r--r-- 1 root root    4 Oct 18 02:18 new.txt
c--------- 2 root root 0, 0 Oct 18 02:18 readme.txt
base layer file
modified in container
app.conf  readme.txt
app.conf  new.txt
```

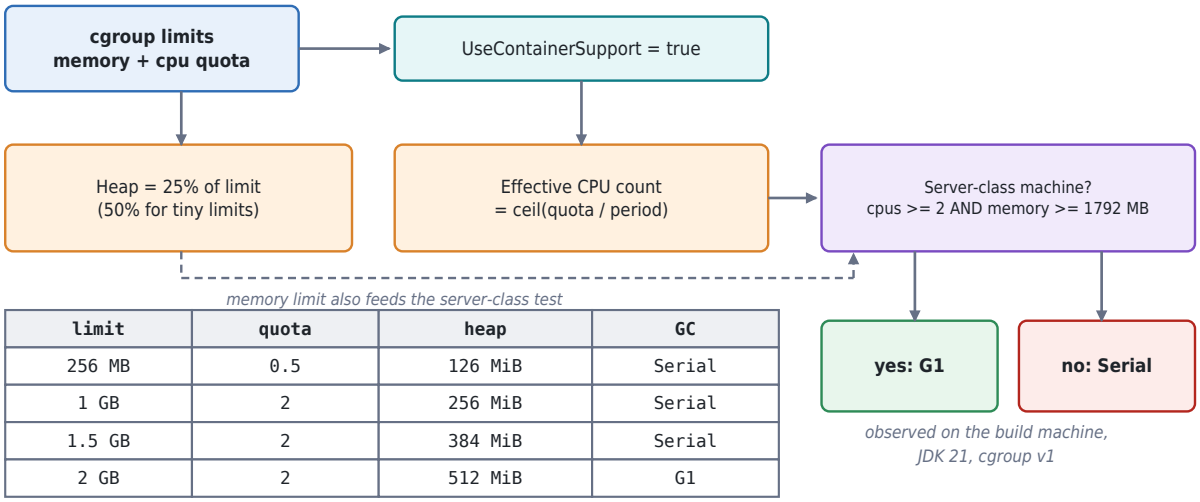
The upper directory holds the 38-byte copied-up `app.conf` (16 bytes of original content plus the 22 appended), the new file, and the whiteout `readme.txt` shown as a character device `0, 0`. The lower directory still lists both original files.

What the JVM derives from the cgroup

Since JDK 10 (and JDK 8u191) the JVM reads cgroup limits when `UseContainerSupport` is enabled, which is the default on Linux. The memory limit determines the maximum heap, the CPU quota determines the processor count, and both determine the collector. The experiment varied the limit and quota of one cgroup and ran `java -XX:+PrintFlagsFinal -XshowSettings:system -version`.

JVM ergonomics inside cgroups (observed, JDK 21, cgroup v1) TERMINAL

```
$ # limit 256 MB, quota 50000/100000
    Provider: cgroupv1
    Effective CPU Count: 1
    CPU Quota: 50000us
    Memory Limit: 256.00M
   size_t MaxHeapSize = 132120576      bool UseSerialGC = true
$ # limit 1 GB, quota 100000 / 150000 / 200000
    Effective CPU Count: 1 / 2 / 2      MaxHeapSize = 268435456   UseSerialGC = true
$ # limit 1.5 GB, quota 200000
   size_t MaxHeapSize = 402653184      bool UseSerialGC = true
$ # limit 2 GB, quota 200000
   size_t MaxHeapSize = 536870912      bool UseG1GC = true
```



limit	quota	heap	GC
256 MB	0.5	126 MiB	Serial
1 GB	2	256 MiB	Serial
1.5 GB	2	384 MiB	Serial
2 GB	2	512 MiB	G1

Figure 6.8. The heap is 25% of the limit (50% for very small limits) and G1 is selected only when the JVM sees at least two CPUs and about 1792 MB; otherwise the single-threaded Serial collector is used.

Three rules follow. The effective CPU count is the quota divided by the period, rounded up. The default maximum heap is 25% of the container memory (`MaxRAMPercentage=25`), which wastes memory in a container dedicated to one JVM; for a 256 MB limit the observed heap was 126 MiB, consistent with the larger `MinRAMPercentage` default (50) that the JVM applies to very small limits. The server-class test selects G1 only with two or more processors and at least 1792 MB; a 1 GB or 1.5 GB container silently receives the Serial collector.

6.1.4 Real-world incidents

Incident: pod killed with exit code 137 while the heap is healthy

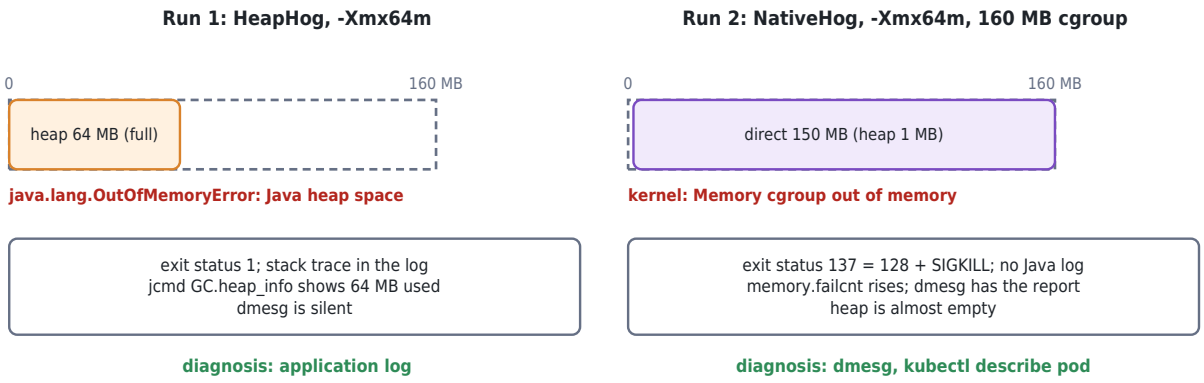
Scenario. A Spring Boot service with `-Xmx512m` runs in a pod with a 640 MiB memory limit. After a release that added a file-upload endpoint using a Netty-based HTTP client, the pod restarts several times a day. The heap graph in the monitoring system never exceeds 40%.

Symptoms. `kubectl get pods` shows `OOMKilled` with exit code 137 and a growing restart count. The application log ends abruptly without a stack trace and without a Java `OutOfMemoryError`. Heap, GC and thread metrics look normal at the moment of death.

Diagnosis. The absence of a Java error indicates that the JVM did not die from heap exhaustion. The commands run in order are listed below, together with a reproduction on the build machine in which a program allocates 10 MB direct buffers inside a 160 MiB cgroup with `-Xmx64m`, compared with a program that exhausts the heap.

Heap exhaustion versus kernel kill (real output, shortened) — TERMINAL

```
$ java -Xmx64m -cp . HeapHog
heap=50MB
Exception in thread "main" java.lang.OutOfMemoryError: Java heap space
        at HeapHog.main(HeapHog.java:7)
$ echo $?
1
$ java -Xmx64m -XX:MaxDirectMemorySize=512m -cp . NativeHog     # inside a 160 MiB cgroup
direct=130MB heapUsed=1MB
direct=140MB heapUsed=1MB
Killed
$ echo $?
137
$ dmesg | grep 'Memory cgroup out of memory' | tail -1
Memory cgroup out of memory: Killed process 4728 (java) total-vm:2564688kB, anon-rss:162432kB,
  file-rss:21620kB, shmem-rss:0kB, UID:0 pgtables:556kB oom_score_adj:0
```



dmesg line (real): oom-kill:constraint=CONSTRAINT_MEMCG ... task=java ... anon-rss:162432kB

Fix pattern: container limit >= Xmx + metaspace + threads x Xss + direct buffers + code cache + headroom

Figure 6.9. A heap overflow raises a Java exception and exits with status 1, whereas native memory growth beyond the cgroup limit is a kernel kill with status 137 and almost no heap in use.

In Kubernetes the corresponding evidence is `kubectl describe pod` (Last State: Terminated, Reason: OOMKilled, Exit Code: 137) and the node kernel log. Within the pod, `jcmd PID VM.native_memory summary` (enabled with `-XX:NativeMemoryTracking=summary`) breaks the footprint into heap, metaspace, threads, code cache, GC structures and `Internal` (direct buffers).

Root cause. The container limit (640 MiB) was sized from the heap alone. Resident memory of a JVM is heap plus metaspace plus thread stacks plus code cache plus GC data structures plus direct and mapped buffers plus the allocator's own overhead; the new HTTP client used direct buffers that were not bounded by `-Xmx`.

Fix. Immediate mitigation: raise the limit (for example to 1 GiB) and set `-XX:MaxDirectMemorySize=128m` so that a direct-buffer leak raises a catchable `OutOfMemoryError: Direct buffer memory` instead of a kernel kill. Permanent fix: derive the heap from the limit with `-XX:MaxRAMPercentage=60` instead of a fixed `-Xmx`, leave at least 25% of the limit for non-heap memory, and bound thread counts.

Prevention. Alert on `container_memory_working_set_bytes / limit` above 85%, track `OOMKilled` restarts, enable NMT in staging, and load-test with the production limit rather than a larger test limit.

Incident: a container with two CPUs and 1.5 GB runs a single-threaded collector

Scenario. A service migrates from virtual machines with 8 CPUs and 16 GB to pods with `limits.cpu: 2` and `limits.memory: 1536Mi`. Throughput is unchanged, but p99 latency doubles and GC pauses appear in the logs with a different format.

Symptoms. Pause times of 200 to 400 ms during young collections, a single `GC Thread` in thread dumps, and heap usage capped near 384 MiB.

Diagnosis. `java -XX:+PrintFlagsFinal -version | grep -E 'UseG1GC|UseSerialGC|MaxHeapSize'` run inside the pod reports `UseSerialGC = true` and `MaxHeapSize = 402653184`. `java -XshowSettings:system -version` shows `Memory Limit: 1.50G` and `Effective CPU Count: 2`. The reproduction above (limit 1.5 GB gives Serial, 2 GB gives G1) confirms the threshold.

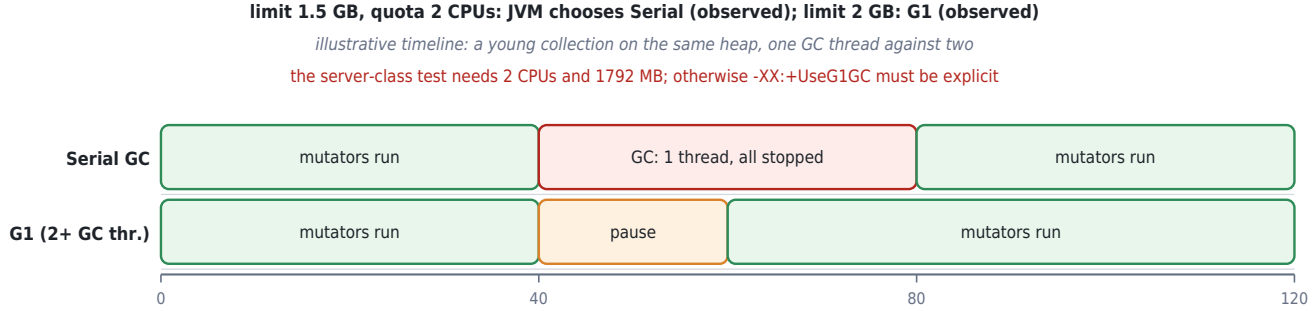


Figure 6.10. With less than about 1792 MB the JVM does not classify the container as a server-class machine, so a single thread performs each collection while every application thread is stopped.

Root cause. The JVM treats a machine as server-class only when it has at least 2 processors and at least 1792 MB of memory. A 1.5 GB container fails the test and receives `SerialGC`; the heap is 25% of the limit.

Fix. Mitigation: add `-XX:+UseG1GC` explicitly (or `-XX:+UseParallelGC` for throughput services) and `-XX:MaxRAMPercentage=65`. Permanent: set collector, heap percentage and `-XX:ActiveProcessorCount` explicitly in the deployment manifest so that behaviour does not depend on ergonomics, and record the flags in the application log at startup.

Prevention. Print `-XX:+PrintFlagsFinal` into the startup log in CI and compare it between environments; alert on GC pause p99.

6.1.5 Questions to expect

Interview questions and model answers

1. **What is a container in terms of kernel features?** A process (or process tree) isolated by namespaces (pid, mnt, net, uts, ipc, user, cgroup), constrained by cgroups, and given a root file system from an image, usually via overlayfs. The kernel has no container object.
2. **What is the difference between a namespace and a cgroup?** A namespace restricts what a process can see; a cgroup restricts how much of a resource the process can use. Both are needed: namespaces without cgroups give isolation without limits.
3. **Why does a container's PID 1 matter?** The first process of a pid namespace is its init: the kernel ignores signals with default dispositions for it, it must reap orphaned children, and its exit terminates every process in the namespace.
4. **How does a container differ from a virtual machine?** A VM runs a guest kernel on virtualised hardware; containers share the host kernel. Containers start faster and use less memory but isolate less strongly.
5. **What happens when a container exceeds its memory limit?** The kernel reclaims page cache first; when reclaim fails, the cgroup OOM killer terminates a process in that cgroup with SIGKILL. The exit status is 137 and `dmesg` reports `CONSTRAINT_MEMCG`.
6. **What happens when a container exceeds its CPU limit?** It is not killed. The CFS bandwidth controller throttles the group until the next period, which raises tail latency; `nr_throttled` in `cpu.stat` counts the events.
7. **What is the difference between requests and limits?** Requests are used for scheduling and set CPU weight and the OOM score; limits are enforced by cgroup files. Equal requests and limits give the Guaranteed QoS class.
8. **How does overlayfs implement copy-on-write?** Reads fall through the layers; the first write to a lower-layer file copies it to the upper layer; deletions are recorded as whiteouts.
9. **Why can the JVM size itself wrongly in a container?** Releases before JDK 8u191 read host resources; later releases honour cgroups, but defaults (25% heap, Serial below two CPUs or 1792 MB) are rarely appropriate and should be set explicitly.
10. **Is a Docker container a security boundary?** Only partially: the kernel is shared, so a kernel flaw or an excessive capability can cross the boundary. Capabilities, seccomp, read-only root and non-root users reduce exposure.

Check yourself

1. A container prints `NSpid: 4311 1` for its entrypoint. Which number would `kill` use on the host?
2. A pod shows exit code 137 and the heap graph is flat at 30%. Name two different causes.
3. Why does a 25% CPU quota not change the median latency of a 10 ms request but raise the 99th percentile?
4. A file of 1 GB lives in an image layer. A process appends 1 byte to it. How much data is written to the upper layer?
5. What is the Kubernetes QoS class of a pod whose containers all set requests equal to limits for CPU and memory, and what `oom_score_adj` does it receive?

Answers

1. 4311, the PID in the host namespace; 1 is the PID inside the container.
2. A kernel cgroup OOM kill from non-heap memory (direct buffers, thread stacks, metaspace) or a SIGKILL sent by the platform because the process ignored SIGTERM beyond the grace period.
3. Most requests finish inside the 25 ms that remain in the period; the request that crosses the quota boundary waits for the rest of the 100 ms period, so only the tail grows.
4. About 1 GB, because overlayfs copies the whole file up before modifying it.

5. Guaranteed; `oom_score_adj` is -997.

KEY POINTS

Containers are processes: namespaces limit visibility, cgroups limit consumption, overlayfs supplies the root file system.

- A memory limit ends in a SIGKILL (exit 137) with `CONSTRAINT_MEMCG` in the kernel log; a CPU limit ends in throttling, visible as `nr_throttled` and tail latency.
- The JVM derives heap, processor count and collector from the cgroup; the defaults (25% heap, Serial below 2 CPUs or 1792 MB) should be overridden deliberately.
- Requests drive scheduling, CPU weight and the OOM score; limits are the enforced boundary.
- A container image is read-only; writes go to the upper layer and large mutable data belongs on a volume.

6.2 Linux Security, Services and Graceful Shutdown

A production Java service must satisfy two requirements that are often treated separately: it must run with the minimum privilege that allows it to work, and it must start, stop and restart in a way that the platform can predict. Both requirements are enforced by kernel mechanisms (credentials, capabilities, seccomp, signals) and by a supervisor (systemd on a host, the kubelet in Kubernetes). Failures in either area produce incidents that look unrelated to security: refused port binds after hardening, dropped requests during every deployment, or processes that survive a restart request.

6.2.1 Concept and mechanism

Users, groups and root

Every process carries a real, effective and saved user ID and group ID, plus supplementary groups. Access to files is decided by comparing the effective IDs with the file owner, group and mode. UID 0 is special only because the kernel skips most permission checks for it; the privilege itself is decomposed into capabilities, so a process running as root without capabilities cannot perform privileged operations, and a non-root process holding a capability can.

Capabilities

A **capability** is one unit of the traditional root privilege. Linux defines about forty; the most relevant to Java services are listed below.

Capability	Permits
`CAP_NET_BIND_SERVICE`	Binding TCP and UDP ports below 1024.
`CAP_CHOWN, CAP_DAC_OVERRIDE`	Changing file ownership; bypassing file permission checks.
`CAP_KILL`	Sending signals to processes owned by other users.
`CAP_SETUID, CAP_SETGID`	Changing process credentials.
`CAP_SYS_PTRACE`	Attaching to other processes (debuggers, `strace`, some profilers).
`CAP_NET_ADMIN, CAP_NET_RAW`	Configuring interfaces and firewalls; raw sockets (`ping`).
`CAP_SYS_RESOURCE`	Raising resource limits; lowering oom_score_adj below the current value.
`CAP_SYS_ADMIN`	A very large set of operations (mount, namespaces, many ioctl calls); treated as nearly equal to root.

Capabilities that appear in service and container configurations.

Each thread holds five capability sets, visible in `/proc/PID/status` as `CapInh`, `CapPrm`, `CapEff`, `CapBnd` and `CapAmb`. The **effective** set is what the kernel checks. The **permitted** set is the ceiling to which the effective set may be raised. The **bounding** set limits what a process and its descendants can ever acquire. The **inheritable** and **ambient** sets control what survives `execve`: an ambient capability is preserved across `execve` of an unprivileged program, which is what allows a non-root service to keep `CAP_NET_BIND_SERVICE`.

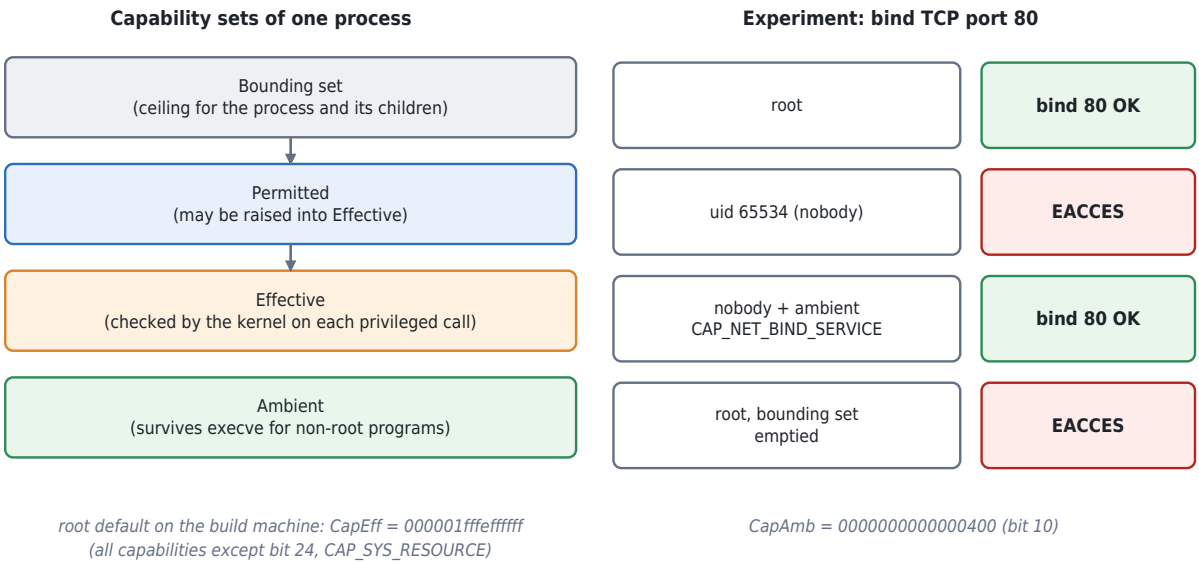


Figure 6.11. The bounding set is the ceiling for the other sets; the experiment shows that a non-root process binds port 80 only when the capability is ambient, and that root fails when its bounding set is empty.

Seccomp

Seccomp (secure computing mode) attaches a filter, written in classic BPF, that the kernel runs on every system call entry. The filter inspects the syscall number and arguments and returns a verdict: allow, fail with a chosen errno, kill the thread or process, or notify a supervisor. A filter is installed with `prctl(PR_SET_SECCOMP)` or `seccomp(2)` after `PR_SET_NO_NEW_PRIVS` is set, and it can only become stricter. Docker applies a default profile that blocks roughly fifty rarely needed calls; Kubernetes can select `RuntimeDefault` through `securityContext.seccompProfile`.

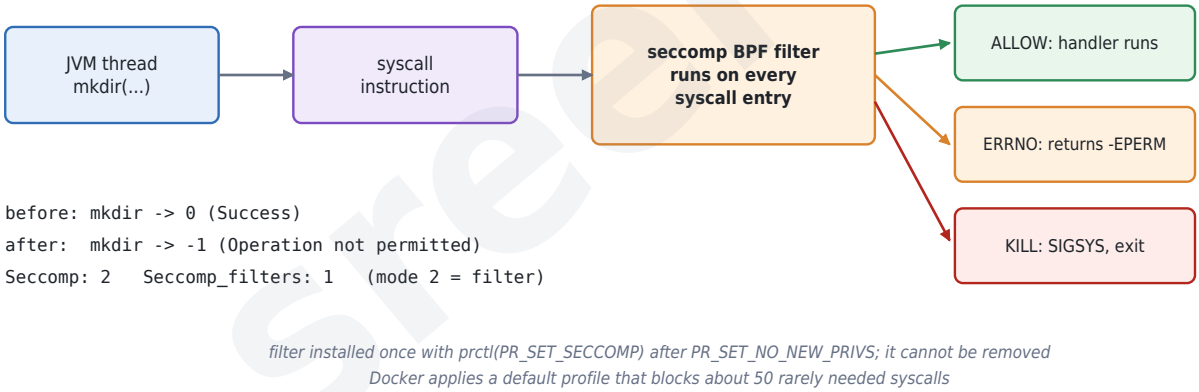


Figure 6.12. Every syscall passes through the installed filter; the experiment makes mkdir fail with EPERM while all other calls proceed.

Mandatory access control: AppArmor and SELinux

Discretionary access control (file modes) lets the owner decide. **Mandatory access control** adds a system-wide policy that the owner cannot override. **SELinux** (Red Hat family) labels every process and object with a security context and allows an access only when a policy rule permits that label pair. **AppArmor** (Ubuntu, SUSE) attaches a per-program profile that lists permitted paths and capabilities. Both confine a compromised process even when it runs as root. A frequent production symptom is a service that works as root on a development machine and fails with `Permission denied` under enforcement; the denial is recorded in `/var/log/audit/audit.log` (SELinux) or in the kernel log (AppArmor), and `setenforce 0` should be used only to confirm the diagnosis, never as the fix.

Least privilege for Java services

A Java web service needs no administrative capability. The recommended baseline is: a dedicated non-root user; a listening port above 1024 (the Kubernetes Service maps port 80 to 8080); all capabilities dropped; a read-only root file system with writable volumes only for `/tmp` and log directories; `no_new_privs`; and the runtime-default seccomp profile.

securityContext.yaml YAML

```yaml
securityContext:
  runAsNonRoot: true
  runAsUser: 10001
  readOnlyRootFilesystem: true
  allowPrivilegeEscalation: false      # sets no_new_privs
  capabilities:
    drop: ["ALL"]
  seccompProfile:
    type: RuntimeDefault
```

A read-only root file system breaks software that writes temporary files; the JVM writes `hsperfdata` under `/tmp`, so `/tmp` must be an `emptyDir` volume or the option `-XX:-UsePerfData` must be set.

systemd services

On a virtual machine, systemd is the supervisor. A **unit file** declares how to start and stop a service, under which identity, with which limits and restart policy. The unit below runs a Spring Boot jar.

orders.service INI

```ini
[Unit]
Description=Orders service (Spring Boot)
After=network-online.target
Wants=network-online.target

[Service]
Type=exec
User=orders
Group=orders
WorkingDirectory=/opt/orders
ExecStart=/usr/bin/java -XX:MaxRAMPercentage=60 -XX:+ExitOnOutOfMemoryError \
  -jar /opt/orders/orders.jar
SuccessExitStatus=143
Restart=on-failure
RestartSec=5
TimeoutStopSec=40
LimitNOFILE=65536
MemoryMax=1G
TasksMax=1024
NoNewPrivileges=true
ProtectSystem=strict
ProtectHome=true
PrivateTmp=true
ReadWritePaths=/var/log/orders
CapabilityBoundingSet=

[Install]
WantedBy=multi-user.target
```

The directives that matter in incidents are the following. `Type=exec` marks the unit started when `execve` succeeds. `SuccessExitStatus=143` treats the JVM exit status after SIGTERM (128 + 15) as success; without it, every stop is recorded as a failure and `Restart=on-failure` may restart a service that was stopped deliberately. `TimeoutStopSec` is the time between SIGTERM and SIGKILL and must exceed the application drain time. `LimitNOFILE` raises the open-file limit, whose default is far below what a connection-heavy service needs. `MemoryMax` and `TasksMax` apply cgroup limits to the service. The `Protect*` and `PrivateTmp` options construct a restricted mount namespace. Edits to a unit require `systemctl daemon-reload`.

sreeram

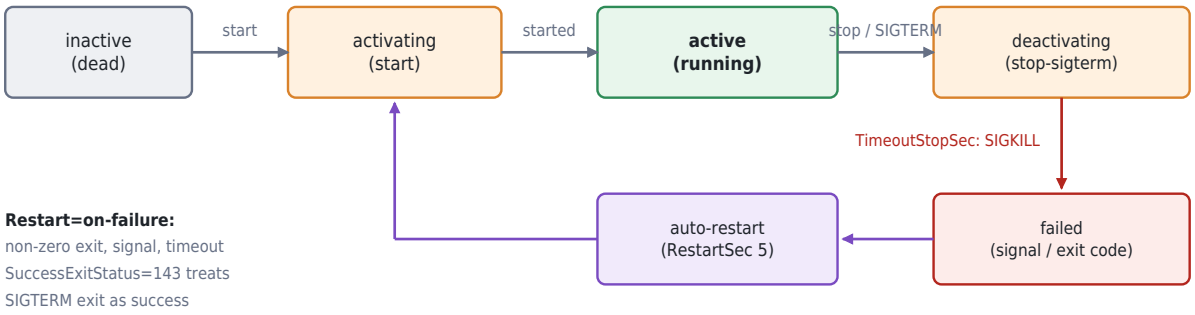


Figure 6.13. A unit moves through activating, active and deactivating; an exit status that is not listed as success sends the unit to failed, from which Restart schedules another start after RestartSec.

Standard output and error of the service are captured by **journald**, queried with `journalctl -u orders -f` (follow), `journalctl -u orders --since "10 min ago"` and `journalctl -u orders -p err`. The journal is size-limited (`SystemMaxUse` in `journald.conf`); a service that logs excessively evicts its own history.

Signals and the JVM

A **signal** is an asynchronous notification delivered to a process. `SIGTERM` (15) requests termination and can be handled; `SIGKILL` (9) and `SIGSTOP` cannot be caught or ignored; `SIGINT` (2) is the keyboard interrupt. The HotSpot JVM installs handlers for SIGTERM, SIGINT and SIGHUP: on receipt it starts an orderly shutdown, runs every registered shutdown hook concurrently, and exits with status 128 plus the signal number, so SIGTERM yields 143. `SIGKILL` bypasses all of this: no hook runs, buffers are not flushed and transactions are not closed.

Spring Boot registers a shutdown hook that closes the application context. With `server.shutdown=graceful` (the default is `immediate`) the embedded server stops accepting new requests and waits for active requests, up to `spring.lifecycle.timeout-per-shutdown-phase` (default 30 seconds), before the remaining lifecycle beans are stopped. The configuration is shown below; it was not run on the build machine (no Spring Boot distribution was available) and follows the Spring Boot reference documentation.

application.yml YAML

```yaml
server:
  shutdown: graceful
spring:
  lifecycle:
    timeout-per-shutdown-phase: 30s
```

Termination in Kubernetes

When a pod is deleted, two sequences start concurrently: the pod is removed from Service endpoints (which propagates asynchronously to proxies and ingress controllers), and the kubelet stops the containers. Stopping runs the optional `preStop` hook, then sends SIGTERM to PID 1 of each container, then waits `terminationGracePeriodSeconds` (default 30) and finally sends SIGKILL. Because endpoint removal and SIGTERM race, an application that exits immediately can still receive requests routed to it; the usual mitigation is a `preStop` sleep of a few seconds followed by graceful shutdown.

srecram

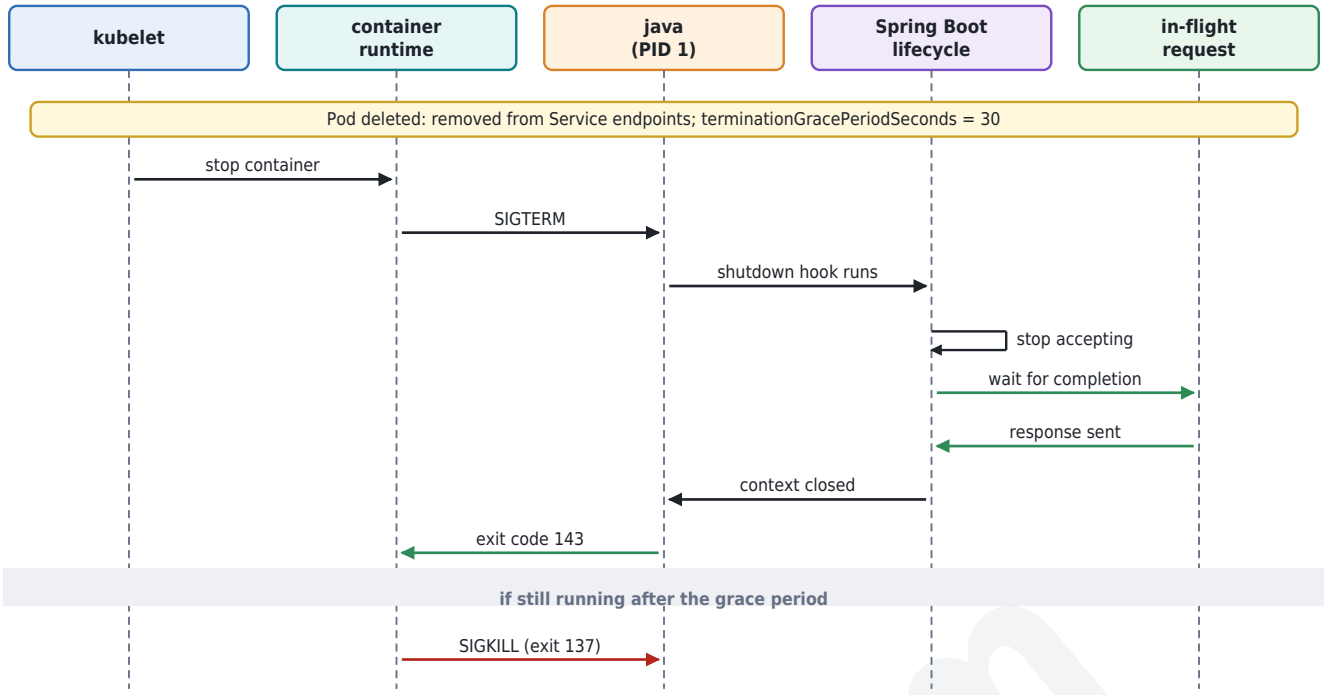


Figure 6.14. The runtime delivers SIGTERM to PID 1; the JVM hook drains in-flight requests and exits with 143; if the process outlives the grace period the runtime sends SIGKILL and the status becomes 137.

Why PID 1 matters

In a pid namespace, the kernel applies two special rules to PID 1. First, a signal sent from inside the namespace to PID 1 is ignored unless PID 1 has installed a handler for it (the same rule protects the real init); signals from the parent namespace follow the same rule for SIGTERM, so a shell as PID 1 without a handler never terminates on SIGTERM. Second, orphaned processes are re-parented to PID 1, which must call `wait()` to reap them; otherwise they remain as zombies. When PID 1 exits the kernel sends SIGKILL to every process in the namespace.

A container whose entrypoint is `sh -c "java -jar app.jar"` therefore has the shell as PID 1 and the JVM as its child. Two remedies exist: `exec java -jar app.jar` in the shell form replaces the shell with the JVM, and a minimal init such as `tini` (or `docker run --init`, or Kubernetes `shareProcessNamespace`) forwards signals and reaps children.

6.2.2 Worked trace

The sequence below follows one SIGTERM delivered to a Spring Boot container whose entrypoint is `exec java -jar app.jar`, with `terminationGracePeriodSeconds: 30` and a 5 second `preStop` sleep.

1. t = 0: the Deployment controller deletes the pod. The endpoints controller removes the pod IP from the Service; kube-proxy and the ingress controller update their rules over the following one to three seconds.
2. t = 0: the kubelet runs the `preStop` command (`sleep 5`). The application keeps serving requests that are still routed to it.
3. t = 5 s: `preStop` completes. The runtime delivers SIGTERM to PID 1, the JVM.
4. The JVM starts shutdown; the Spring hook marks the server as shutting down, stops accepting new connections and waits for active requests.
5. In-flight requests complete (for example after 1.2 s). The hook closes the HikariCP pool, flushes logs and stops the context.
6. The JVM exits with status 143 at t = 6.5 s. The kubelet observes the exit and removes the container.
7. Had a request run beyond t = 35 s (grace period measured from step 1, not from the preStop end), the runtime would send SIGKILL and the container status would be 137.

The grace period includes the `preStop` time. A common configuration error is a 30 second grace period, a 20 second `preStop` and a 20 second drain, which exceeds the budget by 10 seconds and ends in SIGKILL.

6.2.3 Hands-on

Capabilities and no_new_privs

The tool `setpriv` changes credentials and capability sets before executing a command. The helper script `bind.py` attempts to bind a TCP socket to the port given as argument.

Binding port 80 under different credentials (real output) TERMINAL

```
$ python3 -I bind.py 80
bind 80 OK
$ setpriv --reuid=65534 --regid=65534 --clear-groups python3 -I bind.py 80
bind 80 failed: [Errno 13] Permission denied
$ setpriv --reuid=65534 --regid=65534 --clear-groups python3 -I bind.py 20601
bind 20601 OK
$ setpriv --reuid=65534 --regid=65534 --clear-groups \
    --inh-caps=+net_bind_service --ambient-caps=+net_bind_service python3 -I bind.py 80
bind 80 OK
$ setpriv --reuid=65534 --regid=65534 --clear-groups \
    --inh-caps=+net_bind_service --ambient-caps=+net_bind_service grep Cap /proc/self/status
CapInh: 0000000000000400
CapPrm: 0000000000000400
CapEff: 0000000000000400
CapBnd: 000001fffeffffff
CapAmb: 0000000000000400
$ setpriv --bounding-set=-all python3 -I bind.py 80
bind 80 failed: [Errno 13] Permission denied
```

Bit 10 (`0x400`) is `CAP_NET_BIND_SERVICE`. UID 65534 (nobody) cannot bind port 80, binds port 20601 without any capability, and binds port 80 once the capability is made ambient. Root with an empty bounding set loses the ability, which shows that the capability, not the UID, is the controlling factor. A related observation on the build machine: writing `-997` to `/proc/PID/oom_score_adj` returned `Permission denied`, because the process lacked `CAP_SYS_RESOURCE` (bit 24 is absent from its `CapEff` of `000001fffeffffff`); raising the value is always permitted, lowering it is not.

A seccomp filter

The program installs a filter that returns EPERM for `mkdir` and allows everything else, then calls `mkdir` before and after.

sec.c C

```c
#include <stdio.h>
#include <errno.h>
#include <string.h>
#include <stddef.h>
#include <sys/stat.h>
#include <sys/prctl.h>
#include <sys/syscall.h>
#include <linux/seccomp.h>
#include <linux/filter.h>
#include <linux/audit.h>

int main(void) {
    struct sock_filter f[] = {
        BPF_STMT(BPF_LD | BPF_W | BPF_ABS, offsetof(struct seccomp_data, arch)),
        BPF_JUMP(BPF_JMP | BPF_JEQ | BPF_K, AUDIT_ARCH_X86_64, 1, 0),
        BPF_STMT(BPF_RET | BPF_K, SECCOMP_RET_KILL_PROCESS),
        BPF_STMT(BPF_LD | BPF_W | BPF_ABS, offsetof(struct seccomp_data, nr)),
        BPF_JUMP(BPF_JMP | BPF_JEQ | BPF_K, SYS_mkdir, 0, 1),
        BPF_STMT(BPF_RET | BPF_K, SECCOMP_RET_ERRNO | EPERM),
        BPF_STMT(BPF_RET | BPF_K, SECCOMP_RET_ALLOW),
    };
    struct sock_fprog p = { sizeof f / sizeof f[0], f };
    printf("before: mkdir -> %d (%s)\n", mkdir("/tmp/osw_o6/d1", 0755), strerror(errno));
    prctl(PR_SET_NO_NEW_PRIVS, 1, 0, 0, 0);
    if (prctl(PR_SET_SECCOMP, SECCOMP_MODE_FILTER, &p)) { perror("seccomp"); return 1; }
    errno = 0;
    int r = mkdir("/tmp/osw_o6/d2", 0755);
    printf("after:  mkdir -> %d (%s)\n", r, strerror(errno));
    FILE *s = fopen("/proc/self/status", "r");
    char l[256];
    while (fgets(l, sizeof l, s)) if (!strncmp(l, "Seccomp", 7)) fputs(l, stdout);
    return 0;
}
```

Compile and run (x86-64 only, real output) TERMINAL

```
$ gcc -o sec sec.c && ./sec
before: mkdir -> 0 (Success)
after:  mkdir -> -1 (Operation not permitted)
Seccomp:        2
Seccomp_filters:        1
```

`Seccomp: 2` means filter mode; the field is the quick way to check whether a container process runs under a profile (`grep Seccomp /proc/PID/status`).

SIGTERM handling and PID 1

The program registers a shutdown hook that simulates draining for 300 ms and then idles.

Graceful.java JAVA

```java
public class Graceful {
    public static void main(String[] args) throws Exception {
        Runtime.getRuntime().addShutdownHook(new Thread(() -> {
            System.out.println("hook: SIGTERM received, draining in-flight work");
            try { Thread.sleep(300); } catch (InterruptedException e) { }
            System.out.println("hook: drained, exiting");
        }));
        System.out.println("started pid=" + ProcessHandle.current().pid());
        while (true) { Thread.sleep(1000); }
    }
}
```

Three launch variants were tested with `unshare --pid --fork --mount-proc`, which provides a real pid namespace on the build machine.

A: plain process; B: shell as PID 1 with java as child; C: java as PID 1 (real output) TERMINAL

```
$ java -cp . Graceful & kill -TERM $!; wait $!; echo "exit=$?"
started pid=2498
hook: SIGTERM received, draining in-flight work
hook: drained, exiting
exit=143
$ unshare --pid --fork --mount-proc bash -c 'java -cp . Graceful; echo "java exited $?"' &
$ INIT=$(pgrep -P $!)        # host PID of the namespace init (the bash)
$ grep NSpid /proc/$INIT/status; kill -TERM $INIT; sleep 2; kill -0 $INIT && echo "still alive"
NSpid:  2532    1
still alive
$ kill -KILL $INIT           # SIGKILL from the parent namespace is honoured
started pid=2
$ unshare --pid --fork --mount-proc bash -c 'exec java -cp . Graceful' &
$ kill -TERM $(pgrep -P $!); wait; echo "exit=$?"
started pid=1
hook: SIGTERM received, draining in-flight work
hook: drained, exiting
exit=143
```

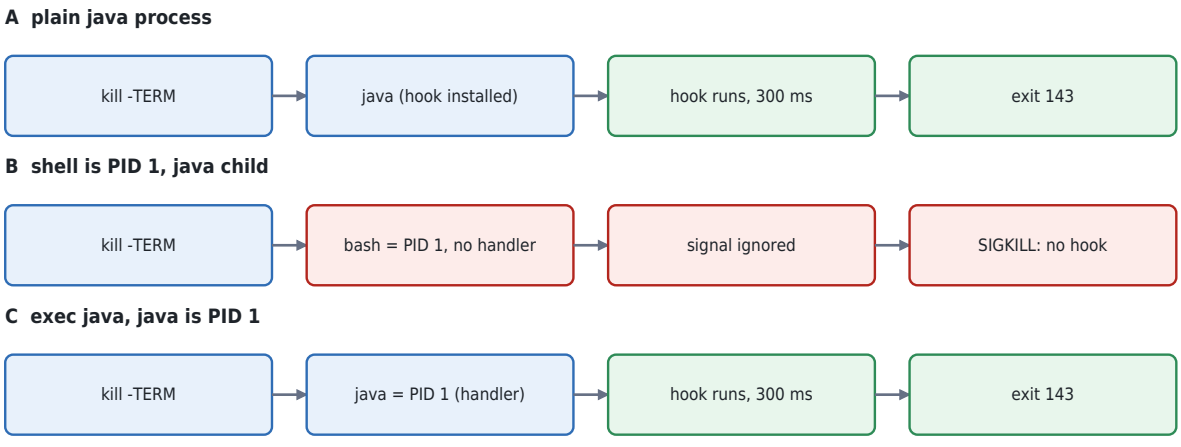


Figure 6.15. Variant B shows the failure: the shell is PID 1 without a SIGTERM handler, the signal is ignored, and only SIGKILL ends the container, killing the JVM before its hook runs.

In variant B the JVM printed `started pid=2` (a child of PID 1) and never printed the hook messages: SIGTERM was ignored by the shell, and the later SIGKILL of the namespace init made the kernel kill the JVM without running any hook. Variants A and C ended with status 143 after the hook completed.

Zombies when PID 1 does not reap

A Python process as PID 1 of a namespace forks three children that exit immediately and never calls `wait`.

Reaping versus not reaping (real output) TERMINAL

```
$ unshare --pid --fork --mount-proc python3 -I noinit.py
    PID    PPID STAT COMMAND
      2       1 Z    python3
      3       1 Z    python3
      4       1 Z    python3
      5       1 R    ps
$ unshare --pid --fork --mount-proc python3 -I withinit.py     # same, plus a wait() loop
    PID    PPID STAT COMMAND
      5       1 R    ps
```

6.2.4 Real-world incidents

Incident: 502 errors during every rolling deployment

Scenario. A Spring Boot API behind an ingress controller is deployed with a rolling update. Each deployment produces a burst of HTTP 502 responses lasting about three seconds per pod replaced, although readiness probes pass and no error appears in the application log.

Symptoms. The ingress access log shows `502` with upstream errors "connection refused" or "connection reset by peer" exactly at deployment times; the error rate returns to zero afterwards; the application's own request log contains no failed requests.

Diagnosis. (1) `kubectl rollout status` and the event stream confirm that the errors coincide with pod termination. (2) `kubectl describe pod` of a terminated pod shows `Exit Code: 143` or `0`, not 137, so the container exited promptly rather than being killed. (3) The container log's last line is the Spring shutdown message, followed by no drain wait. (4) `kubectl get pod -o yaml` shows no `preStop` hook, and `server.shutdown` is unset (default `immediate`). (5) The timeline in the figure explains the gap: the process exits at t = 10 s, but the endpoint removal reaches the ingress about three seconds later.

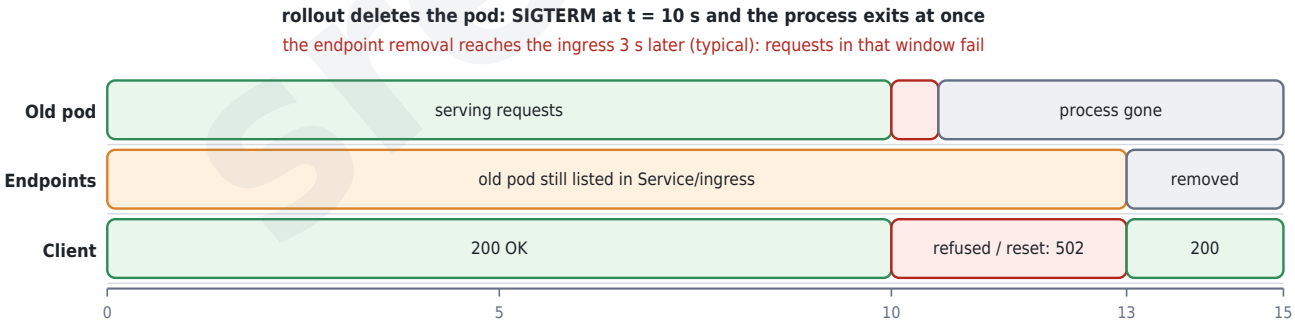


Figure 6.16. The process exits at once on SIGTERM while the ingress still routes to its address; requests sent in that window receive connection errors, which the proxy reports as 502.

Root cause. The application stops accepting connections immediately on SIGTERM, but endpoint removal propagates asynchronously. The window is typically one to five seconds.

Fix. Immediate: add a `preStop` hook (`sleep 5`) so that the pod keeps serving while endpoints update. Permanent: set `server.shutdown=graceful` with a drain timeout, set `terminationGracePeriodSeconds` larger than `preStop` plus drain time, and enable readiness failure on shutdown (Spring Boot does this automatically on graceful shutdown).

Prevention. Run a continuous load test against a staging cluster during deployments and fail the pipeline when any non-2xx response appears; alert on ingress 502 counts correlated with rollout events.

Incident: zombie processes accumulate in a container

Scenario. A service container launches an external converter (`ffmpeg`, `wkhtmltopdf` or a shell script) for every request through `Runtime.exec`, using a shell wrapper entrypoint. After several days the node reports `fork: Resource temporarily unavailable` and the pod cannot start new conversions.

Symptoms. `ps` inside the container lists thousands of processes in state `Z`, the pod's `pids.current` approaches `pids.max`, and conversion requests fail with `Cannot run program ... error=11, Resource temporarily unavailable`.

Diagnosis. `ps -eo pid,ppid,stat,comm | awk '$3 ~ /Z/'` lists the zombies and shows that their parent is PID 1, a shell or the JVM. `cat /proc/1/comm` identifies the process; `grep -E '^(State|PPid)' /proc/PID/status` for a zombie confirms state `Z`. `cat /sys/fs/cgroup/pids/.../pids.current` (v1) or `pids.current` (v2) compares the count with `pids.max`.

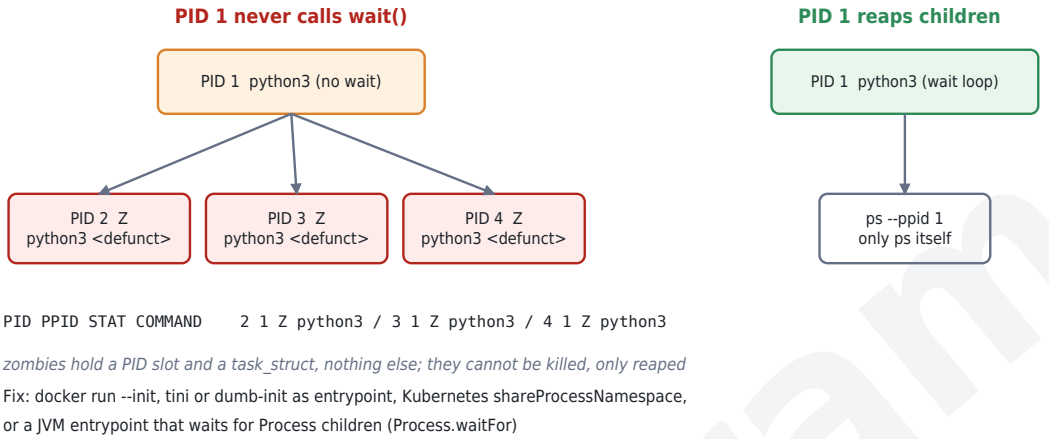


Figure 6.17. A zombie is a terminated process whose parent has not called wait; in a container PID 1 inherits orphans and must reap them, otherwise entries accumulate until the pids limit is reached.

Root cause. The converter's children were orphaned or exited while the parent never collected their status, and PID 1 (a shell) does not reap. Each zombie keeps a PID and a `task_struct` until reaped. The JVM reaps children it started through `ProcessBuilder`, but not unrelated grandchildren that are re-parented to PID 1.

Fix. Immediate: restart the pod (all zombies are released when PID 1 exits). Permanent: run an init as PID 1 (`tini` or `docker run --init`; Kubernetes `shareProcessNamespace: true` makes the pause container the reaper), call `Process.waitFor()` or `destroy()` for every started process, and set a pids limit that exposes leaks early.

Prevention. Alert on the count of zombie processes per node (`node_processes_state{state="Z"}` in node exporter) and on `pids.current / pids.max` above 80%.

Incident: Permission denied binding port 80 after hardening

Scenario. A security review requires the service to run as a non-root user. After the change the pod crashes at startup.

Symptoms. The log contains `java.net.SocketException: Permission denied` (or `Failed to bind to /0.0.0.0:80`) during server start and the pod enters `CrashLoopBackOff`.

Diagnosis. `id` inside the container shows the non-root UID. `grep Cap /proc/1/status` shows an empty `CapEff` after `drop: ["ALL"]`. The server property `server.port` is 80. The experiment in the Hands-on section shows the same failure for UID 65534.

Root cause. Ports below 1024 require `CAP_NET_BIND_SERVICE`; a non-root process without it receives EACCES.

Fix. Immediate: set `server.port=8080`. Permanent: expose port 80 on the Kubernetes Service (`port: 80`, `targetPort: 8080`); avoid re-adding capabilities.

Prevention. Run the container under its production `securityContext` in CI and fail the build on startup errors.

6.2.5 Questions to expect

Interview questions and model answers

1. **What is the difference between root and capabilities?** Root (UID 0) is an identity for which the kernel skips most checks; capabilities divide that privilege into units. A process can be root without capabilities (powerless) or non-root with one capability.
2. **How can a non-root Java process bind port 80?** Grant `CAP_NET_BIND_SERVICE` (ambient capability, `setcap` on the binary, or `AmbientCapabilities` in systemd), or listen on a high port and map it with a Service, iptables rule or proxy. The second option is preferred.
3. **What does seccomp do and how does it differ from capabilities?** Seccomp filters individual syscalls by number and arguments; capabilities gate privileged operations by kind. Seccomp reduces kernel attack surface even for capabilities the process legitimately holds.
4. **What does the JVM do on SIGTERM?** It starts an orderly shutdown, runs all shutdown hooks concurrently, and exits with status 143. SIGKILL cannot be handled, so no hook runs.
5. **Why does SIGTERM sometimes not stop a container?** The entrypoint is a shell that is PID 1 and has no handler, so the kernel ignores the signal. Use `exec`, an init such as `tini`, or a handler.
6. **What is the Kubernetes termination sequence?** Endpoint removal and `preStop` start together; SIGTERM follows `preStop`; SIGKILL follows after `terminationGracePeriodSeconds` (default 30 s, counted from the start of termination).
7. **What does exit status 143 mean? And 137?** 143 = 128 + 15 (SIGTERM, graceful). 137 = 128 + 9 (SIGKILL: grace period exceeded, or OOM kill).
8. **What is a zombie process and why is it harmless but dangerous in bulk?** A terminated process whose exit status has not been collected; it uses only a process table entry, but exhausting PIDs or the pids cgroup limit blocks new processes.
9. **What does `Restart=on-failure` do in systemd and why add `SuccessExitStatus=143`?** It restarts the unit after a non-zero exit, signal or timeout. A JVM stopped with SIGTERM exits 143, which would otherwise count as a failure.
10. **Why set `no_new_privs`?** It prevents a process and its children from gaining privileges through setuid binaries or file capabilities, and is required to install a seccomp filter without privilege.

Check yourself

1. A non-root process has `CapAmb: 0000000000000400`. What can it do that others cannot?
2. The entrypoint is `ENTRYPOINT sh -c "java -jar app.jar"`. A `docker stop` takes exactly 10 seconds and the application log has no shutdown message. Explain.
3. What exit status does the JVM return after `kill -TERM` when the hook completes?
4. Pod: `terminationGracePeriodSeconds: 30`, `preStop: sleep 20`, drain 20 s. What happens?
5. Which kernel setting causes `Seccomp: 2` to appear in `/proc/PID/status`?

Answers

1. Bind TCP and UDP ports below 1024 (bit 10, `CAP_NET_BIND_SERVICE`).
2. The shell is PID 1 and ignores SIGTERM; Docker waits the default 10 second stop timeout and then sends SIGKILL, so no hook runs. Use `exec java ...` or an init.
3. 143.
4. SIGTERM arrives at t = 20 s; the drain needs until t = 40 s, but SIGKILL arrives at t = 30 s, so in-flight requests are cut and the status is 137.
5. A seccomp filter installed with `PR_SET_SECCOMP` in filter mode.

KEY POINTS

Privilege is decomposed into capabilities; a service needs none of them if it listens on a high port and runs as a dedicated user.

- Seccomp filters syscalls, MAC (SELinux, AppArmor) confines by policy; both limit a compromised process.
- systemd supplies identity, limits, restart policy and journald logging; `SuccessExitStatus=143` and `TimeoutStopSec` must match the JVM shutdown behaviour.
- SIGTERM yields exit 143 after hooks run; SIGKILL (137) skips hooks. The grace period includes the `preStop` time.
- PID 1 ignores unhandled signals and must reap children: use `exec`, `tini` or `--init`.

Skeleton

6.3 The Production Playbook

Earlier Parts explained individual mechanisms: scheduling, virtual memory, locking, file systems, containers. A production incident presents none of them by name. It presents a symptom (latency, errors, restarts) and a clock. The playbook is the method for moving from the symptom to the responsible mechanism in minutes, and from the mechanism to a change that is safe to apply under pressure. It consists of a measurement discipline, a decision tree, a fixed opening sequence, a set of commands whose output is understood field by field, and a short list of tunables that are worth changing.

6.3.1 Concept and mechanism

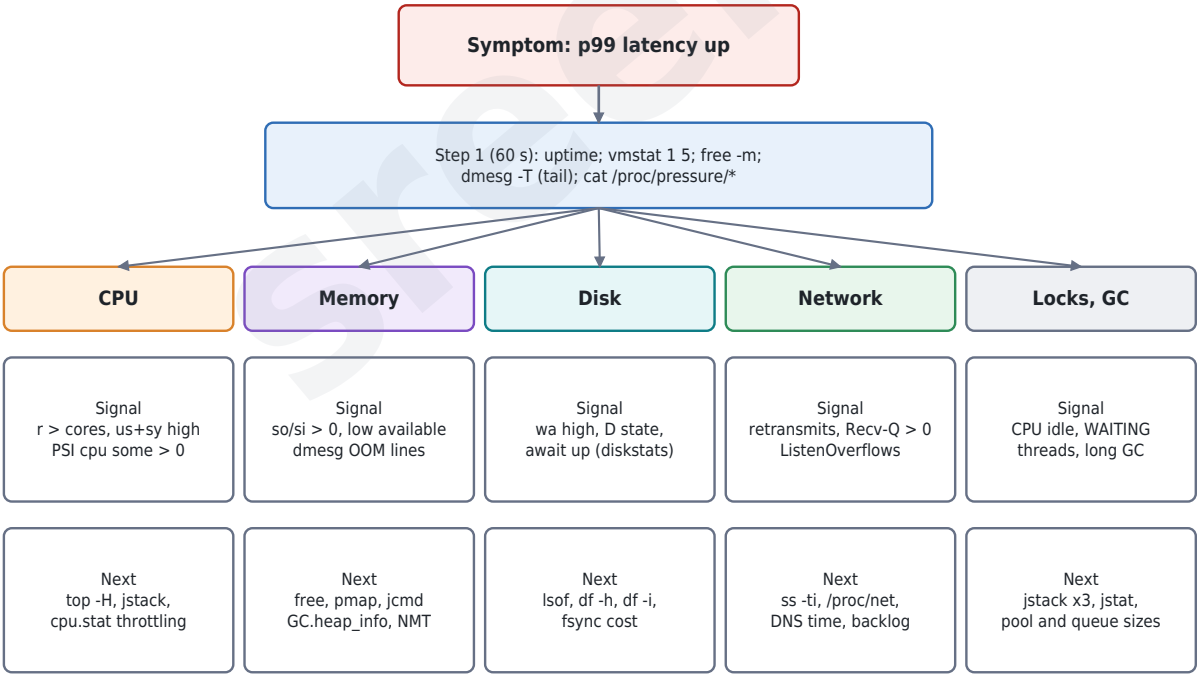
Methodology: measure resources, then confirm in the application

Three complementary methods cover most services.

- **USE** (Utilisation, Saturation, Errors) is applied to every resource: CPU, memory, storage devices, network interfaces and also software resources such as thread pools, connection pools and file descriptors. Utilisation is the fraction of time the resource is busy; saturation is the amount of queued work (run queue length, swap activity, I/O queue depth, pool wait); errors are discrete failures (dropped packets, OOM kills, `EMFILE`).
- **RED** (Rate, Errors, Duration) is applied to each service request path and tells where users feel the problem.
- The **four golden signals** (latency, traffic, errors, saturation) combine both views for dashboards and alerts.

A resource with high utilisation and zero saturation is rarely the cause; a resource with moderate utilisation and non-zero saturation or errors usually is. The method is deliberately mechanical: it prevents a diagnosis from stopping at the first plausible explanation.

The working loop has five steps: state the symptom with numbers (for example "p99 of `/orders` rose from 120 ms to 900 ms at 14:05"), list what changed in the preceding hours (deployments, configuration, traffic, platform), form one hypothesis, test it with a command that can disprove it, and change one variable at a time. Recording each command and result in the incident channel provides the timeline needed for the post-incident review.



Branch on the resource that is saturated first; confirm with a second tool before changing anything

Figure 6.18. The decision tree starts with a 60 second host-level look and branches by the resource that shows saturation; each branch names the signal that selects it and the next commands to run.

The first five minutes

The first five minutes establish impact, scope and the most recent change, take the host-level readings, isolate the failing layer, and apply the cheapest reversible mitigation. A rollback of the last change is usually cheaper and safer than a live diagnosis; the diagnosis can continue on a preserved instance or from collected data.

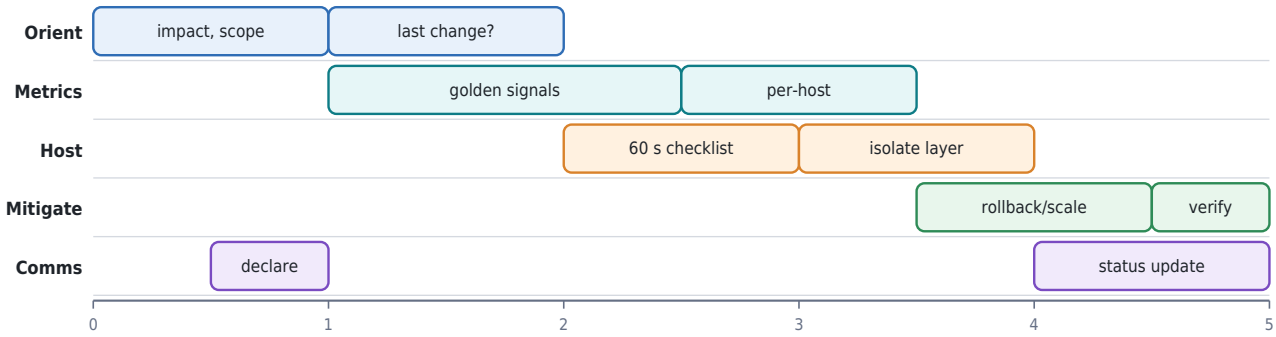


Figure 6.19. Orientation and metrics run in parallel with the declaration of the incident; the host checklist and layer isolation lead to a reversible mitigation by minute four or five.

Capacity planning basics

Three relations explain most capacity questions.

- **Little's law**: the mean number of requests in the system equals the arrival rate times the mean time in the system (`L = lambda x W`). A service receiving 200 requests per second with a mean latency of 50 ms holds 10 requests concurrently; the thread pool or connection pool must exceed that value with margin.
- **Utilisation law**: utilisation equals throughput times service demand. If each request consumes 8 ms of CPU, 200 requests per second consume 1.6 CPU-seconds per second, which is 80% of a 2 CPU allocation.
- **Queueing delay**: for a single server with random arrivals, the mean response time is `R = S / (1 - U)`, where S is the service time and U the utilisation. At 50% utilisation latency is twice the service time; at 80% it is five times; at 95% twenty times.

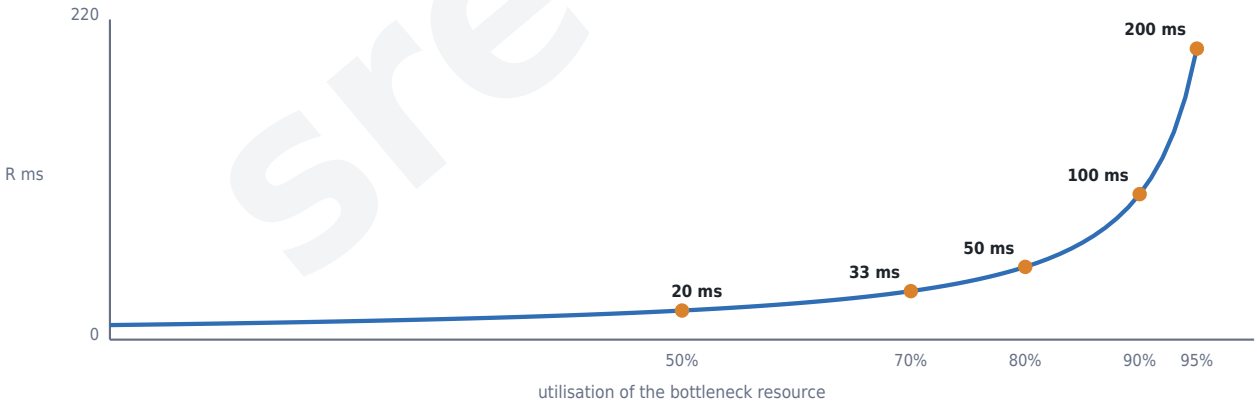


Figure 6.20. Response time grows hyperbolically with utilisation; the computed points for a 10 ms service time show why services are sized for 50 to 70% utilisation of the bottleneck resource and not for 95%.

The consequence for the example above is direct: a 2 CPU limit at 80% utilisation yields a mean response of about five times the 8 ms service demand under the single-server model, before any garbage collection or lock contention. Headroom is therefore planned from the latency objective, not from the point at which the CPU graph reaches 100%.

Tunables worth changing, and the caution that applies

Kernel parameters are changed with `sysctl -w name=value` for the running system and persisted in a file under `/etc/sysctl.d/`. Each change alters behaviour for every process on the host (or on the network namespace for namespaced parameters), so each requires a stated reason, a measured baseline and a rollback value.

sreeram

Parameter	Observed	Purpose and caution
`vm.swappiness`	`60`	Relative preference to swap anonymous memory versus reclaim page cache. Lowering it (10) on latency-sensitive nodes reduces swapping; 0 does not disable swap.
`net.core.somaxconn`	`4096`	Upper bound for the accept queue of any listening socket. Kernels before 5.4 defaulted to 128. The application backlog is capped by this value.
`net.ipv4.tcp_max_syn_backlog`	`512`	Half-open connection queue; raise together with somaxconn for connection bursts.
`fs.file-max`	`819675`	System-wide open file limit. Per-process limits (`ulimit -n`, `LimitNOFILE`) are usually the binding constraint.
`vm.max_map_count`	`65530`	Maximum memory mappings per process; JVMs with many threads or Elasticsearch need 262144.
`kernel.pid_max`	`32768`	Maximum PID value; a node running thousands of pods may require 4194304.
`net.ipv4.ip_local_port_range`	`32768 60999`	Ephemeral ports for outgoing connections; exhaustion appears as `Cannot assign requested address`.
`vm.overcommit_memory`	`0`	Heuristic overcommit; 2 enforces a commit limit and causes allocation failures instead of OOM kills.

Parameters that appear in incidents. The second column is the value read on the build machine.

Namespaced parameters (for example the `net.*` family) can be set per pod in Kubernetes, but some require the kubelet to allow them as unsafe sysctls; this depends on the Kubernetes version and should be verified before relying on it. Parameters such as `vm.swappiness` are global to the node.

Kernel and operating system upgrades

An upgrade changes behaviour that applications depend on without any change to the application. The recurring categories are: the **cgroup version** (v1 to v2 changes file names and, for old JVMs, container detection; see the case study below), the **scheduler** (the move from CFS to EEVDF in Linux 6.6 altered latency characteristics for some workloads), **default sysctl values** (for example `somaxconn`), the **C library** (changes in `malloc` or DNS resolver behaviour), the **JDK container support** and the **file system defaults**. Safe practice is a canary node pool running the new image for a full business cycle, a comparison of `sysctl -a` and `/proc/cmdline` between old and new nodes, a rollback path (the old image remains available), and a staggered rollout with latency and error alerts that page automatically.

6.3.2 Worked trace

The case: a release changed the pod manifest from `limits.cpu: 2` to `limits.cpu: 250m`. The `/orders` endpoint showed p99 of 900 ms after the deployment. The trace applies the five-minute sequence with the readings from the CPU quota experiment (quota 25000 per 100000 microseconds).

1. **Minute 0 to 1 (orient).** Impact: `/orders` p99 rose from 120 ms to 900 ms; p50 is unchanged at 40 ms; error rate is 0.2%. The only change in the last 24 hours is release 4.12.
2. **Minute 1 to 2 (metrics; values illustrative but consistent with the experiment).** CPU usage of the pod averages 22% of its limit, which seems healthy; the container metric `cpu_cfs_throttled_periods / cpu_cfs_periods` is 0.94.
3. **Minute 2 to 3 (host checklist).** `uptime` shows a load average below the CPU count; `vmstat 1` shows idle CPU on the node and no swap activity; `free -m` shows ample available memory. The node is not saturated, which points at a limit inside the pod.
4. **Minute 3 to 4 (isolate).** `cat /sys/fs/cgroup/cpu/.../cpu.stat` (v1) or `/sys/fs/cgroup/cpu.stat` (v2) shows `nr_throttled` rising by about 9 every 10 periods and `throttled_time` growing by nearly 750 ms per second. A request that needs 10 ms of CPU occasionally waits 75 ms for the next period.
5. **Minute 4 to 5 (mitigate).** Revert the limit to 2 CPUs (a one-line manifest change) and confirm that the throttled ratio falls to zero and p99 returns to 120 ms. The root cause (a cost-saving change applied without a load test) is then written up for the review.

6.3.3 Hands-on

The 60 second checklist on a real host

The following commands are run in this order at the start of any host-level investigation. The output is from the build machine, shared with other workloads, so the numbers describe a lightly loaded host. Each field is read as stated in the notes below the block.

Host-level opening sequence (real output, shortened) TERMINAL

```
$ uptime
 02:30:56 up 21 min,  0 user,  load average: 2.06, 3.11, 2.05
$ vmstat 1 4
procs -----------memory---------- ---swap-- -----io---- -system-- -------cpu-------
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st gu
 0  0      0 7088412  13400 721888    0    0  1415  7021 4515  135 33  7 57  2  1  0
 0  0      0 7087648  13400 723436    0    0     0  4500 3834 5499  3  5 85  7  1  0
 0  0      0 7086640  13400 724928    0    0     0  4344 3942 5353  1  5 84  9  1  0
$ free -m
               total        used        free      shared  buff/cache   available
Mem:            8029         612        6927          14         714        7417
Swap:              0           0           0
$ head -1 /proc/pressure/cpu /proc/pressure/memory /proc/pressure/io
==> /proc/pressure/cpu <==
some avg10=1.35 avg60=4.94 avg300=23.96 total=321637690

==> /proc/pressure/memory <==
some avg10=0.00 avg60=0.00 avg300=0.16 total=3157741

==> /proc/pressure/io <==
some avg10=0.00 avg60=0.27 avg300=1.11 total=36221006
```

- `uptime`: three load averages (1, 5, 15 minutes) count runnable plus uninterruptible tasks; they are compared with the CPU count (2). A value of 2.06 on 2 CPUs indicates full use without a standing queue.
- `vmstat` column `r` is the run queue (saturation of CPU), `b` is tasks blocked in uninterruptible sleep (I/O), `si` and `so` are swap-in and swap-out per second (any sustained non-zero value is memory saturation), `wa` is time waiting for I/O, and `st` is steal time. The first line is an average since boot and is discarded.
- `free -m`: the column `available` estimates memory obtainable without swapping and is the number to alert on; `free` alone is misleading because the page cache occupies free memory.
- The pressure stall information (PSI) files give the fraction of time tasks were stalled on a resource: `some avg10=1.35` means 1.35% of the last ten seconds had at least one task waiting for CPU.

Command crib sheet by symptom

Symptom	First commands	What confirms the hypothesis
High CPU	`top -H`, pidstat-style reading of `/proc/PID/stat`, `jstack PID`	A thread with high %CPU whose hex ID matches a `nid` in the dump.
Latency with low CPU	`cat cpu.stat`, `jstack` (3 times), `vmstat 1`	`nr_throttled` rising; threads blocked on one monitor.
Memory growth	`free -m`, `cat /proc/PID/status`, `jcmd PID GC.heap_info`	RSS above heap plus metaspace indicates native memory.
OOM kill	`dmesg -T`, `kubectl describe pod`, `cat memory.failcnt`	`CONSTRAINT_MEMCG` in the log; exit code 137.
Disk full	`df -h`, `df -i`, `du -xh --max-depth=1`, `lsof +L1`	100% blocks, or inodes, or deleted files still open.
Slow disk	`cat /proc/diskstats` twice, `vmstat 1` (`wa`, `b`), `cat /proc/pressure/io`	Rising weighted I/O time and `b` greater than zero.
Connection failures	`ss -s`, `ss -ltn`, `nstat`, `cat /proc/net/netstat`	ListenOverflows rising; `Recv-Q` near `Send-Q` on a listener.
Too many threads	`ls /proc/PID/task`, `cat pids.current pids.max`	Count near `pids.max`; `unable to create native thread`.
Shutdown problems	`kubectl describe pod`, `journalctl -u UNIT`, `kill -TERM PID`	Exit 143 versus 137; hook output present or absent.

Commands by symptom.

6.3.4 Real-world incidents

Twelve end-to-end case studies follow. Each combines more than one layer of the stack and each uses measurements taken on the build machine wherever a reproduction was possible; cases that could not be reproduced are labelled as typical.

Case 1: slow API after a release lowered the CPU limit

Scenario. A Spring Boot service in Kubernetes was released with `limits.cpu` reduced from 2 to 250m. The service uses Tomcat with 200 worker threads and a Hikari pool of 20.

Symptoms. p50 latency is unchanged, p99 rises by a factor of seven, and CPU utilisation relative to the limit looks moderate. Upstream callers log timeouts at the 1 second mark.

Diagnosis. (1) `cat cpu.stat` in the pod's cgroup twice, ten seconds apart; the throttled ratio is `delta(nr_throttled) / delta(nr_periods)`. (2) `jstack` shows many threads runnable but not running. (3) A reproduction with a 25% quota is below.

Throttling in a CPU cgroup (real output; counters are cumulative, see text) TERMINAL

```
$ cat /sys/fs/cgroup/cpu/osw_06/cpu.stat | head -3      # before the Lat run
nr_periods 47
nr_throttled 36
throttled_time 1298169543
$ java -cp . Lat                                         # quota 25000, period 100000
requests=150 p50=10ms p99=82ms max=84ms
$ cat /sys/fs/cgroup/cpu/osw_06/cpu.stat | head -3
nr_periods 99
nr_throttled 85
throttled_time 4809441318
```

During the run the group advanced by 52 periods, of which 49 (94%) were throttled, accumulating 3.51 seconds of throttled time.

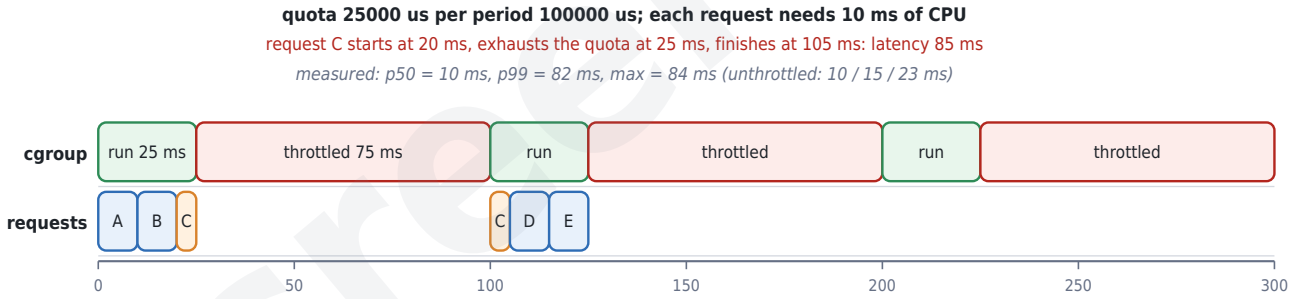


Figure 6.21. Each period allows 25 ms of CPU; a request that straddles the boundary (request C) waits 75 ms, which produces the p99 of 82 ms while the median stays at 10 ms.

Root cause. The CFS bandwidth controller grants `quota` microseconds per `period`; with 250m the process may run 25 ms per 100 ms, and all threads of the cgroup are stopped when the allowance is consumed, including the JIT compiler and GC threads.

Fix. Immediate: restore the previous limit. Permanent: size the limit from measured CPU demand with headroom (see capacity planning), consider omitting CPU limits while keeping requests (weights provide fairness without throttling), and bound the Tomcat and Hikari pools so that concurrency matches the CPU budget.

Prevention. Alert on the throttled-periods ratio above 25% for 5 minutes; require a load test at the production limit in the release pipeline.

Case 2: nightly batch job evicts the API's hot data from the page cache

Scenario. A reporting job reads 40 GB of export files every night between 02:00 and 03:30 on the node that also runs the order API, which reads a 6 GB index through memory-mapped files.

Symptoms. API p99 triples during the batch window; the node's disk read rate rises from near zero to 400 MB/s; `free -m` shows `available` unchanged because the cache is reclaimable.

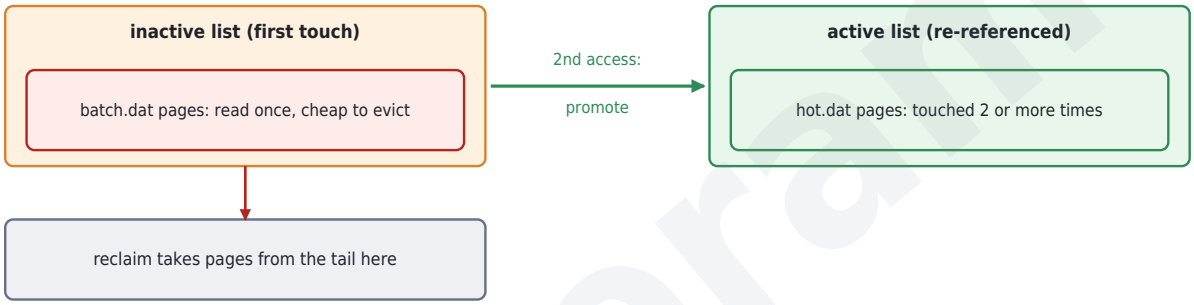
Diagnosis. (1) `vmstat 1`: column `bi` is high and `b` is non-zero. (2) `grep -E 'pgmajfault|pgpgin' /proc/vmstat` twice: major faults rise by thousands per second. (3) Residency of the index file measured with `mincore`; the helper below reports resident pages. (4) The reproduction confirms the mechanism and the remedy.

Cache residency before and after posix_fadvise(DONTNEED) (real output) TERMINAL

```
$ python3 -I resid.py hot3.dat batch3.dat
hot3.dat   resident  6144 of  6144 pages (100%)
batch3.dat resident 76800 of 76800 pages (100%)
$ python3 -I fadvise.py batch3.dat          # POSIX_FADV_DONTNEED on the whole file
fadvise DONTNEED issued
$ python3 -I resid.py hot3.dat batch3.dat
hot3.dat   resident  6144 of  6144 pages (100%)
batch3.dat resident     0 of 76800 pages (  0%)
```

In this experiment the 24 MB hot file was read twice and then a 300 MB file was scanned twice: the hot pages stayed resident, because pages referenced repeatedly are promoted to the **active** list and a streaming scan fills only the **inactive** list. The experiment did not reproduce eviction, which is the honest result: eviction of hot data occurs when it is referenced once per long interval (and therefore sits on the inactive list), when the cgroup or node memory is smaller than the active set, or when the scan itself is mapped and randomly accessed.

Page cache LRU lists: why a streaming scan does not evict hot data



Observed (cgroup 96 MB, files created outside it; resident pages by mincore):

```
batch.dat 300 MB: resident 76800 of 76800 pages -> fadvise DONTNEED -> 0 pages -> re-read -> 76800
hot.dat   24 MB: read twice, then batch scans: 6144 of 6144 pages stayed resident
```

Eviction hits hot data when it is referenced once, or the scan is large and uses mmap/random access.

Figure 6.22. Re-referenced pages are promoted to the active list and survive a sequential scan; pages touched once, such as an index read once per hour, are reclaimed first.

Root cause. The batch job and the API compete for one page cache; the index pages are cold between requests and fall to the inactive list, where the scan reclaims them. Every subsequent index read becomes a major fault served by the disk.

Fix. Immediate: stop or throttle the batch job. Permanent: have the batch job call `posix_fadvise(DONTNEED)` (or read with `O_DIRECT`) after each chunk; run it in a separate memory cgroup so that its page cache is charged and reclaimed inside its own limit; move it to a different node; pre-warm the index at start (`cat index > /dev/null` or `vmtouch -t`); schedule outside peak.

Prevention. Alert on major page faults per second for the API process (`majflt` in `/proc/PID/stat`) and on disk read throughput at peak hours.

Case 3: disk fills after a log level change

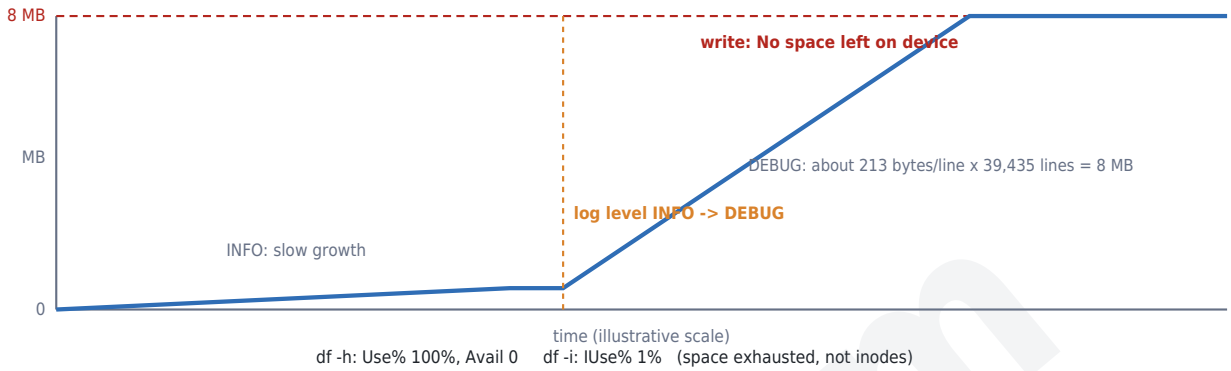
Scenario. To investigate a defect, an operator changes the root log level to DEBUG through the Spring Boot actuator at 14:10. The service logs request payloads at that level to a file on a small volume.

Symptoms. At 14:40 requests fail with `No space left on device`; health checks fail; the node reports `DiskPressure`; the pod is evicted.

Diagnosis. (1) `df -h` shows 100% used on the log volume; (2) `df -i` shows inode use at 1%, ruling out inode exhaustion; (3) `du -xh --max-depth=1 /var/log | sort -h` identifies the file; (4) `lsof +L1` lists deleted-but-open files in case rotation removed a file that the process still holds. The reproduction wrote a DEBUG-sized line repeatedly to an 8 MB tmpfs inside a private mount namespace.

Filling a size-capped file system (real output) TERMINAL

```
$ unshare -m bash fill.sh          # fill.sh mounts an 8 MB tmpfs and appends DEBUG lines
fill.sh: line 4: echo: write error: No space left on device
lines written: 39435
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           8.0M  8.0M     0 100% /tmp/osw_o6/mnt
Filesystem      Inodes IUsed   IFree IUse% Mounted on
tmpfs           1027790     2 1027788    1% /tmp/osw_o6/mnt
```



reproduction: 8 MB tmpfs in a private mount namespace; the 39,435-line count is the real result

Figure 6.23. The write path fails when blocks are exhausted although almost no inodes are used; the log volume grows linearly at the DEBUG rate until it reaches capacity.

Root cause. An unbounded log volume combined with a verbose level and no cap on total log size.

Fix. Immediate: set the level back with `POST /actuator/loggers/ROOT` and body `{"configuredLevel":"INFO"}`; truncate the active file with `: > app.log` (truncation frees blocks even while the file is open, deletion does not). Permanent: configure size-based rotation with a total cap (Logback `SizeAndTimeBasedRollingPolicy` with `maxFileSize`, `maxHistory` and `totalSizeCap`), log to stdout in containers, and set an ephemeral-storage limit.

Prevention. Alert on filesystem use above 80% and on `predict_linear` projections of exhaustion within four hours; restrict who may change log levels in production and expire them automatically.

Case 4: intermittent slowness on a cloud virtual machine caused by CPU steal

Scenario. A latency-sensitive service runs on a burstable cloud instance. During business hours the latency fluctuates without any change in traffic or code.

Symptoms. CPU utilisation inside the guest is moderate, yet requests run slower and load average is higher than usual; there are no errors.

Diagnosis. `top` shows a non-zero `st` figure; `vmstat 1` column `st` is the same value; the raw counters are in `/proc/stat`, where the eighth value of the `cpu` line is steal in clock ticks. Two samples give the rate.

Reading steal time (real output from the build machine) TERMINAL

```
$ head -1 /proc/stat
cpu  93253 85 16904 172748 4528 0 2617 1480 0 0
$ vmstat 1 3 | awk 'NR>2 {print "us="$13" sy="$14" id="$15" wa="$16" st="$17}'
us=32 sy=7 id=59 wa=2 st=1
us=27 sy=5 id=68 wa=0 st=0
us=2 sy=1 id=97 wa=0 st=1
```

The `/proc/stat` fields are: user, nice, system, idle, iowait, irq, softirq, steal, guest, guest_nice. Here steal is 1480 of 291,615 ticks, 0.5%, which is negligible; the same host in a noisy-neighbour condition would show 10 to 30%, as in the illustrative bar in the figure.

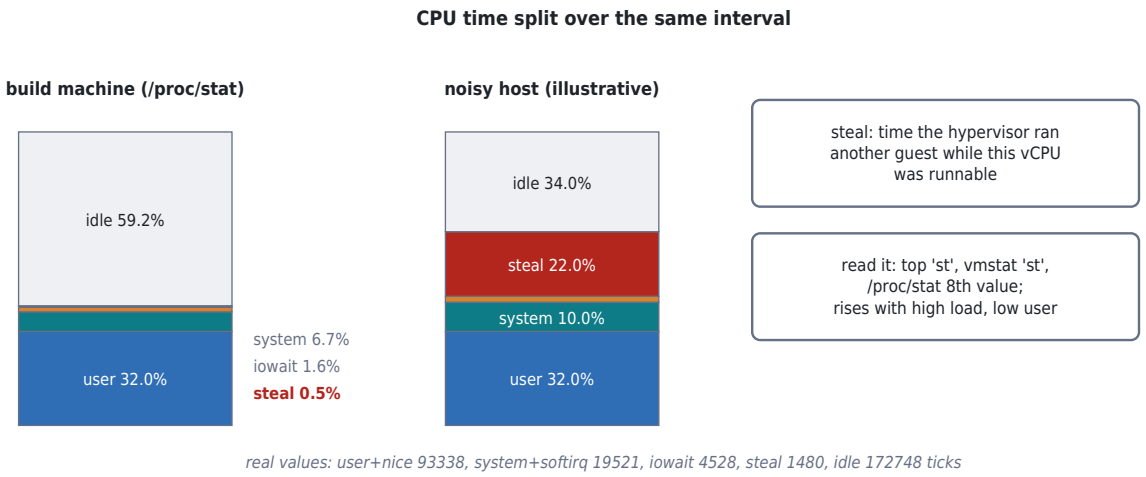


Figure 6.24. Steal is time the hypervisor ran another guest while this virtual CPU was runnable; the left bar is the measured host, the right bar an illustrative degraded host.

Root cause. The hypervisor oversubscribes physical cores, or the instance has exhausted its CPU credits and is capped. Neither is visible from inside the guest except through steal time.

Fix. Immediate: move the workload to another host or instance. Permanent: use dedicated or non-burstable instance types for latency-critical services, spread replicas across hosts, and avoid sizing thread pools from `availableProcessors()` alone.

Prevention. Export node steal as a metric (`node_cpu_seconds_total{mode="steal"}`) and alert above 10% for five minutes.

Case 5: connection timeouts from a full accept queue

Scenario. During a flash sale, clients receive connection timeouts although the service CPU is low and no error is logged. The load balancer reports failed health checks on some instances.

Symptoms. Clients wait the one second SYN retransmit interval and then fail with `connect timed out`; the server access log shows no corresponding requests.

Diagnosis. (1) `ss -ltn` for the listening port: `Recv-Q` is the current accept-queue length and `Send-Q` its limit. (2) `nstat -az TcpExtListenOverflows TcpExtListenDrops`, or the same counters from `/proc/net/netstat`, rise during the incident. (3) A reproduction with a listener whose backlog is 5 and which never calls `accept()`:

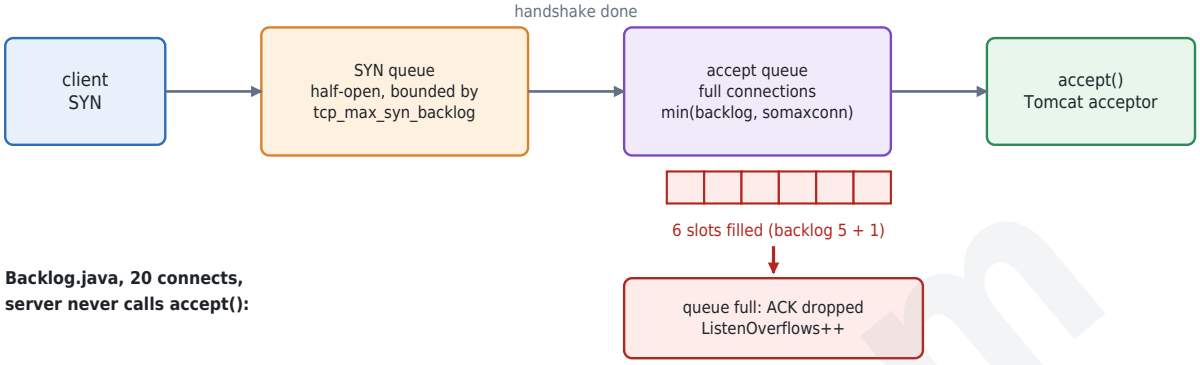
Backlog.java JAVA

```java
import java.net.InetAddress;
import java.net.InetSocketAddress;
import java.net.ServerSocket;
import java.net.Socket;
import java.net.SocketTimeoutException;
import java.util.ArrayList;
import java.util.List;

public class Backlog {
    public static void main(String[] a) throws Exception {
        int backlog = Integer.parseInt(a[0]);
        InetAddress lo = InetAddress.getLoopbackAddress();
        ServerSocket ss = new ServerSocket(20601, backlog, lo);
        List<Socket> cs = new ArrayList<>();
        int ok = 0, fail = 0;
        for (int i = 0; i < 20; i++) {          // the server never calls accept()
            Socket s = new Socket();
            try {
                s.connect(new InetSocketAddress(lo, 20601), 1000);
                ok++;
                cs.add(s);
            } catch (SocketTimeoutException e) { fail++; }
        }
        System.out.println("backlog=" + backlog + " connected=" + ok + " timed_out=" + fail);
        Thread.sleep(300);
    }
}
```

Accept queue overflow (real output; counters are host-wide) TERMINAL

```
$ sh nst.sh                       # prints ListenOverflows and ListenDrops from /proc/net/netstat
ListenOverflows 23
ListenDrops 23
$ java -cp . Backlog 5
backlog=5 connected=6 timed_out=14
$ sh nst.sh
ListenOverflows 38
ListenDrops 38
$ java -cp . Backlog 128
backlog=128 connected=20 timed_out=0
```



Backlog.java, 20 connects, server never calls accept():

```
backlog=5   -> connected=6   timed_out=14   ListenOverflows +15
backlog=128 -> connected=20  timed_out=0    ListenOverflows +0
```

client sees a 1 s connect timeout (retransmitted SYN); server logs nothing

Figure 6.25. A completed handshake waits in the accept queue until the application calls accept(); when the queue holds backlog plus one entries further connections are dropped and counted as ListenOverflows.

Root cause. The application accepts connections slower than they arrive, and the accept queue (limited by the smaller of the application backlog and `net.core.somaxconn`) overflows. In Tomcat the backlog is `server.tomcat.accept-count` (default 100) and the acceptor stops accepting at `server.tomcat.max-connections` (default 8192 for NIO).

Fix. Immediate: raise `accept-count` and `somaxconn`, and add capacity. Permanent: remove the reason the acceptor is slow (blocked worker pool, long GC pauses) and apply load shedding earlier with rate limits.

Prevention. Alert on any increase of `ListenOverflows` and on `Recv-Q` above half of `Send-Q`; load-test with burst traffic.

Case 6: thread leak after a library upgrade

Scenario. After a dependency upgrade, a service fails after about 40 minutes with `OutOfMemoryError: unable to create native thread` while heap use is normal.

Symptoms. Thread count in metrics rises linearly; the error appears at a constant thread count; the pod's memory limit is not reached.

Diagnosis. `ls /proc/PID/task | wc -l` gives the count; `jcmd PID Thread.print | grep '^"' | sed 's/-[0-9]*"/"/' | sort | uniq -c | sort -rn | head` groups threads by name and exposes the leaking pool; `cat pids.current pids.max` in the cgroup shows proximity to the limit. The reproduction creates threads in a cgroup with `pids.max` of 60.

Thread creation failing at the pids limit (real output) TERMINAL

```
$ java -Xss512k -cp . ThreadLeak       # inside a pids cgroup with pids.max = 60
[warning][os,thread] Failed to start thread "Unknown thread" - pthread_create failed (EAGAIN)
 for attributes: stacksize: 512k, guardsize: 0k, detached.
created 39 threads, then: java.lang.OutOfMemoryError: unable to create native thread:
 possibly out of memory or process/resource limits reached
pids.current=60
$ cat /sys/fs/cgroup/pids/osw_o6/pids.events
max 8
```

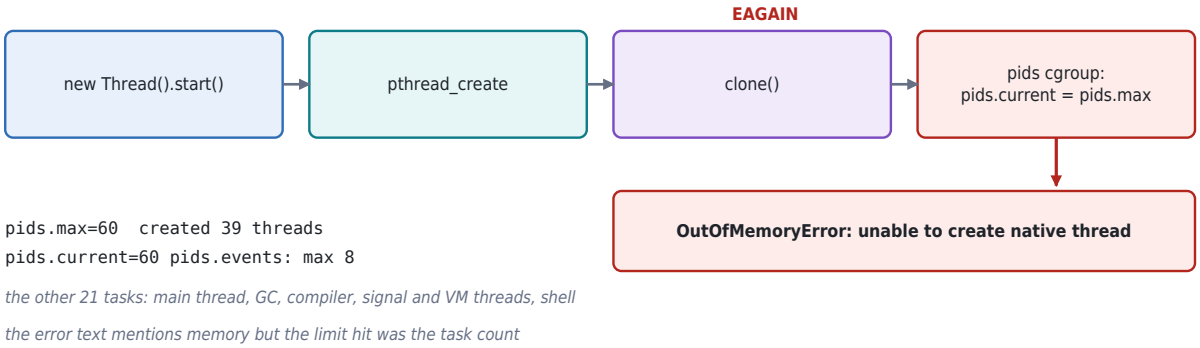


Figure 6.26. Thread creation passes from the JVM through pthread_create and clone to the pids controller, which returns EAGAIN; the JVM reports it as an out-of-memory error although the limit was a task count.

Root cause. The upgraded client library created an `ExecutorService` per request and never shut it down, so each request left worker threads parked forever. The JVM counts 21 of the 60 tasks in the experiment as its own threads, so the leak reached the limit after 39 threads.

Fix. Immediate: restart pods on a schedule while the fix is prepared. Permanent: reuse one shared executor or close the client in `finally`; configure `pids.max` (`podPidsLimit` in the kubelet) to expose leaks early; name pool threads with a factory so that they are identifiable in dumps.

Prevention. Alert on `jvm_threads_live_threads` growth and on `pids.current / pids.max` above 70%.

Case 7: requests delayed by five seconds because of name resolution

Scenario. A service calls an external API over HTTPS. A small fraction of calls takes slightly more than five seconds although the API responds in 100 ms.

Symptoms. Latency histogram shows a second mode at 5.0 to 5.1 seconds; CPU is idle; no errors.

Diagnosis (typical; not reproduced on the build machine, which has no controllable resolver). (1) `time getent hosts api.example.com` repeated shows a 5 second outlier. (2) `cat /etc/resolv.conf` lists two nameservers, the first unreachable, and `options ndots:5`. (3) `strace -f -tt -T -e trace=sendto,recvfrom,poll -p PID` shows a query to the first nameserver followed by a gap of five seconds before the second is tried.

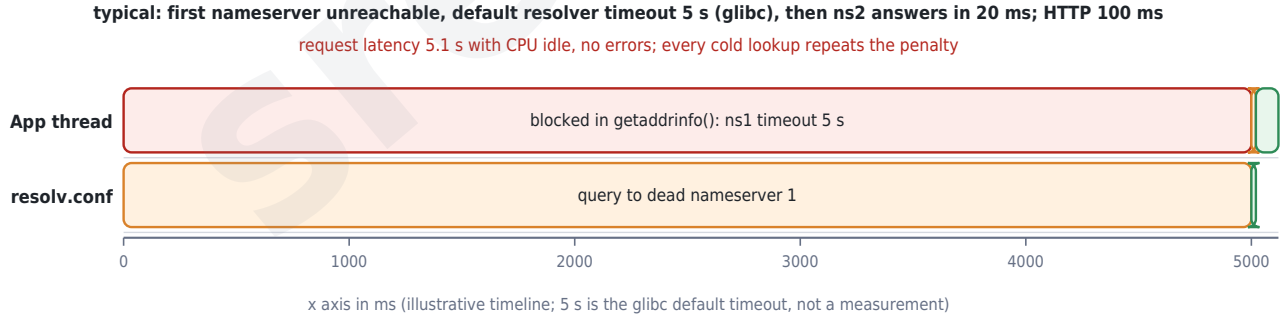


Figure 6.27. The application thread blocks inside getaddrinfo for the resolver timeout (5 seconds by default in the C library) before the second nameserver answers in milliseconds.

Root cause. The C library tries nameservers in order with a default timeout of 5 seconds per attempt; with `ndots:5` (the Kubernetes default) an external name is first tried against several cluster search domains, multiplying queries. The JVM caches successful lookups for 30 seconds by default (`networkaddress.cache.ttl` when no security manager is installed), so the penalty recurs periodically.

Fix. Immediate: remove the dead nameserver or point to a working resolver. Permanent: run a node-local DNS cache, use fully qualified names with a trailing dot for external hosts or set `ndots:2` through the pod `dnsConfig`, reuse HTTP connections, and set connect and resolve timeouts in the client. (The protocol itself belongs to the companion material on computer networks.)

Prevention. Monitor resolver latency (`kube-dns` or node-local-dns metrics) and alert on query failures and timeouts.

Case 8: valid tokens rejected because of clock skew

Scenario. After a hypervisor maintenance event, one node of a cluster rejects JWTs issued by the identity provider. Other nodes accept them.

Symptoms. HTTP 401 from one pod only; log text such as "Jwt expired at ..." or "Jwt used before ..."; the failure ratio is constant for that pod and unrelated to load.

Diagnosis. (1) `date -u` on the failing node compared with a reference host. (2) `timedatectl` (NTP synchronised: no) and `chronyc tracking` (typical; chrony is not installed on the build machine) show the offset. (3) The program below reproduces the validation logic for four clock offsets and prints the real decisions.

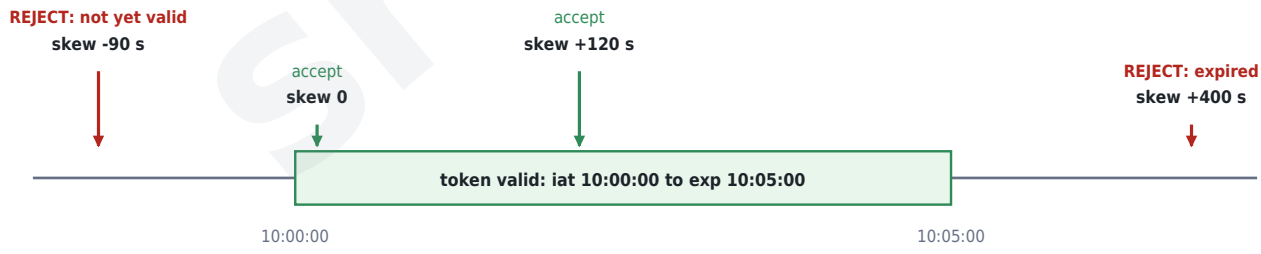
Jwt.java — JAVA

```java
import java.time.Clock;
import java.time.Duration;
import java.time.Instant;
import java.time.ZoneOffset;

public class Jwt {
    public static void main(String[] a) {
        Instant issuedAt = Instant.parse("2026-10-18T10:00:00Z");   // set by the auth server
        Instant exp = issuedAt.plusSeconds(300);                      // valid for 5 minutes
        for (long skew : new long[] {0, -90, 120, 400}) {
            Clock clk = Clock.offset(Clock.fixed(issuedAt.plusSeconds(10), ZoneOffset.UTC),
                                     Duration.ofSeconds(skew));
            Instant now = clk.instant();
            boolean notYet = now.isBefore(issuedAt);
            boolean expired = now.isAfter(exp);
            System.out.printf("resource-server clock skew %+5ds: now=%s -> %s%n", skew, now,
                notYet ? "REJECT (token used before iat/nbf)"
                       : expired ? "REJECT (expired)" : "accept");
        }
    }
}
```

Output of java Jwt.java (real; strict check without tolerance) — TERMINAL

```
$ java Jwt.java
resource-server clock skew    +0s: now=2026-10-18T10:00:10Z -> accept
resource-server clock skew   -90s: now=2026-10-18T09:58:40Z -> REJECT (token used before iat/nbf)
resource-server clock skew  +120s: now=2026-10-18T10:02:10Z -> accept
resource-server clock skew  +400s: now=2026-10-18T10:06:50Z -> REJECT (expired)
```



Jwt.java output (real): resource-server clock = issue time + 10 s + skew

default Spring Security JWT validation allows 60 s of clock skew; larger drift fails

fix: chrony/NTP on every node, alert on offset above 100 ms

Figure 6.28. A clock that runs behind sees the token as not yet valid; a clock that runs far ahead sees it as expired; both fail even though the token is correct.

Root cause. The node clock was not synchronised after the maintenance event. Spring Security's default JWT timestamp validation tolerates 60 seconds of skew, so drift beyond that fails validation; TLS certificate validation fails for the same reason when the offset exceeds the certificate validity margins.

Fix. Immediate: step the clock (`chronyc makestep`) or drain the node. Permanent: run `chronyd` or `systemd-timesyncd` on every node against reliable sources and keep the tolerance small and explicit.

Prevention. Alert when the clock offset exceeds 100 ms (`node_timex_offset_seconds`) and when NTP synchronisation status is lost.

sreram

Case 9: kernel image upgrade changes the cgroup version

Scenario. Nodes are upgraded to an image whose kernel boots with the unified cgroup hierarchy (v2) by default. Within hours some services restart repeatedly and the monitoring agent reports no container memory data.

Symptoms. Pods running an older JDK 8 build are killed with exit 137; dashboards for container memory go blank on upgraded nodes only.

Diagnosis. (1) `stat -fc %T /sys/fs/cgroup` prints `cgroup2fs` on v2 and `tmpfs` on v1; on the build machine the hybrid layout shows `tmpfs` for `/sys/fs/cgroup` and `cgroup2fs` for `/sys/fs/cgroup/unified`. (2) `cat /proc/self/cgroup` lists one line per controller on v1 and a single `0::/path` line on v2. (3) `java -XshowSettings:system -version` prints `Provider: cgroupv1` (as on the build machine) or `cgroupv2`; an old JVM prints nothing and sizes the heap from host memory. (4) `ls /sys/fs/cgroup/memory/` fails on v2.

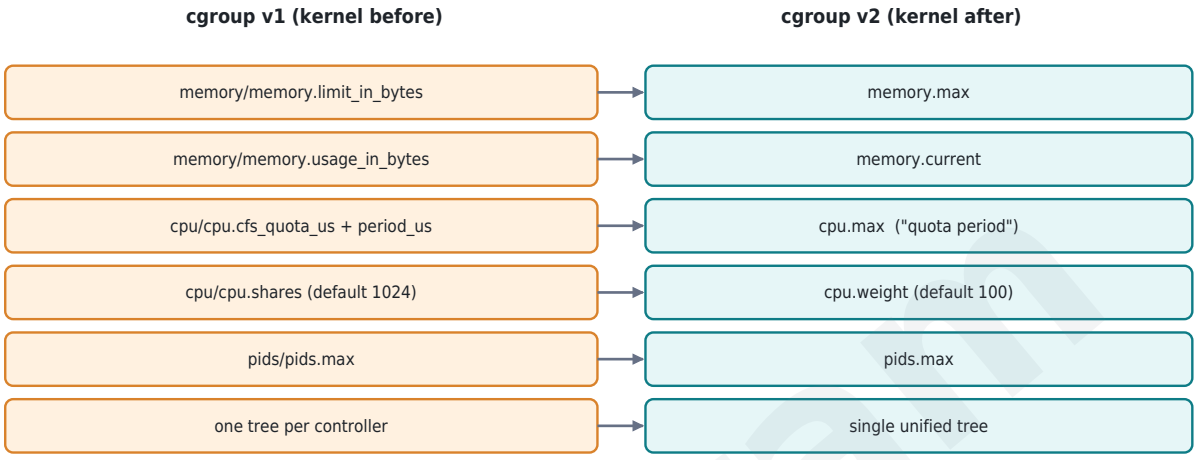


Figure 6.29. Files keep their meaning but change names and locations: scripts and agents that read the v1 paths find nothing on v2, and a JVM without v2 support cannot detect the container limit.

Root cause. JVMs before JDK 8u372, 11.0.16 and 15 do not recognise cgroup v2, so container support silently fails and the heap default (a quarter of host memory) exceeds the pod limit.

Fix. Immediate: pin affected workloads to nodes still on v1, or set `-Xmx` explicitly. Permanent: upgrade the JDK, update agents and scripts, and add the cgroup version to the node pool inventory.

Prevention. Run a canary node pool for any node image change and compare `PrintFlagsFinal` output between old and new nodes.

Case 10: memory growth and an OOMKilled restart loop

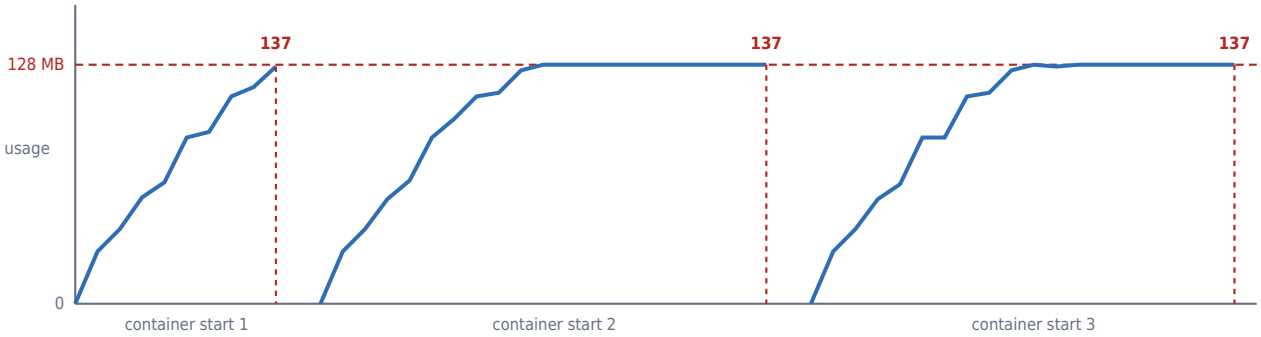
Scenario. A service holds a per-customer cache in a `HashMap` without eviction. It is configured with `-Xmx` equal to the container memory limit. Every few minutes the pod restarts; after each restart the cycle repeats.

Symptoms. Restart count increases; `OOMKilled` with exit 137; memory graphs show a sawtooth that ends at the limit; the heap-exhaustion error never appears in the log.

Diagnosis. The reproduction ran the cache leak (2 MB per 40 ms) with `-Xmx512m` inside a 128 MB cgroup, three times, sampling `memory.usage_in_bytes` every 0.25 s. A histogram of a running instance identifies the leaking type.

Three restart cycles and the histogram (real output) TERMINAL

```
run 3 exit=137 samples(MB every 0.25s): 0 28 40 56 64 89 89 111 113 125 128 127 128 128 128 128 128
$ jcmd PID GC.class_histogram | head -4       # separate run, -Xmx300m, after 2.5 s
 num     #instances         #bytes  class name (module)
-------------------------------------------------------
   1:          8084      153443936  [B (java.base@21.0.12.1)
$ grep VmRSS /proc/PID/status
VmRSS:    211884 kB
```



memory.usage_in_bytes sampled every 0.25 s; -Xmx512m inside a 128 MB cgroup; unbounded cache grows 2 MB per 40 ms

each cycle ends with SIGKILL (exit 137); the orchestrator then restarts the process and the cycle repeats

Figure 6.30. Memory climbs to the cgroup limit and the process is killed with exit 137; because the heap maximum exceeds the limit the JVM never raises its own OutOfMemoryError, and the restart repeats the cycle.

Root cause. The heap maximum (512 MB) exceeded the cgroup limit (128 MB), so the kernel reached the limit before the JVM reached its own, and the unbounded cache guaranteed that it would be reached.

Fix. Immediate: raise the limit or reduce the cache by restarting with a smaller cache size. Permanent: bound the cache (Caffeine with `maximumSize` and expiry), set `-XX:MaxRAMPercentage` below 75 so that the heap maximum is under the limit with room for non-heap memory, and enable `-XX:+HeapDumpOnOutOfMemoryError` on a volume that survives restarts.

Prevention. Alert on `OOMKilled` events and on heap occupancy after full collections trending upward.

Case 11: a batch job takes CPU from the API on the same node

Scenario. A nightly job and the API run on the same node. The job is CPU-bound and has the same CPU request as the API.

Symptoms. API latency increases whenever the job runs; node CPU is 100%; no throttling is reported because no limit applies.

Diagnosis. `top` on the node shows the job processes consuming most of the CPU; `cat cpu.shares` (v1) or `cpu.weight` (v2) of both cgroups shows equal weights; the reproduction pinned two cgroups to one core and compared throughput.

Two CPU-bound cgroups on one core, equal and unequal shares (real output) TERMINAL

```
$ # cpu.shares 1024 : 1024
batch iterations=80962
api iterations=81081
$ # cpu.shares 1024 (api) : 128 (batch)
batch iterations=18053
api iterations=144711
```

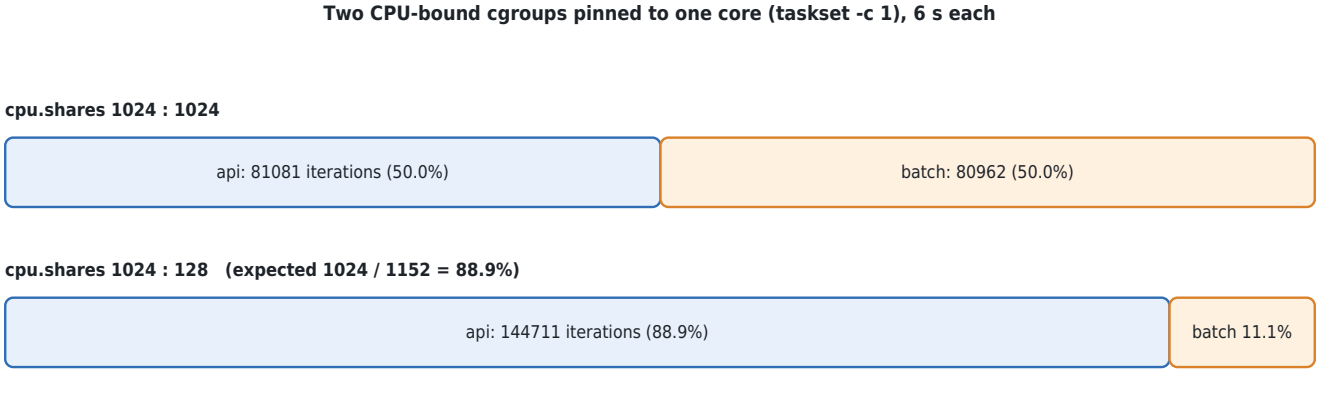


Figure 6.31. Equal shares split one core evenly; with weights 1024 to 128 the heavier group receives 88.9% of the iterations, matching 1024 / 1152.

Root cause. Equal weights give the batch job half of the contended CPU. Weights have no effect when the CPU is idle.

Fix. Immediate: `renice` the job or lower its weight. Permanent: give the batch workload a low request (small weight) or its own node pool with a CPU limit, and schedule it away from latency-critical pods using taints and priority classes.

Prevention. Separate workload classes by node pool; alert on node CPU saturation correlated with latency.

Case 12: rolling restart kills in-flight work because the grace period is too short

Scenario. A queue consumer processes messages in batches that take up to four seconds to commit. Every deployment produces duplicate records downstream.

Symptoms. Duplicated or half-processed batches immediately after each deployment; pods terminate with exit 137.

Diagnosis. `kubectl get pod -o yaml` shows `terminationGracePeriodSeconds: 2` (copied from a template for a stateless service). The reproduction below ran a worker whose shutdown hook needs four seconds, first with a 10 second kill delay and then with 2 seconds.

Shutdown hook versus kill delay (real output; timeout exits 124 after a clean TERM exit) TERMINAL

```
$ timeout -s TERM -k 10 3 java -cp . Batch
worker started pid=8073
hook: SIGTERM, finishing current batch (needs 4 s)
hook: batch committed, offsets saved, exit clean
exit=124 elapsed=7.03s
$ timeout -s TERM -k 2 3 java -cp . Batch
worker started pid=8122
hook: SIGTERM, finishing current batch (needs 4 s)
exit=137 elapsed=5.01s
```

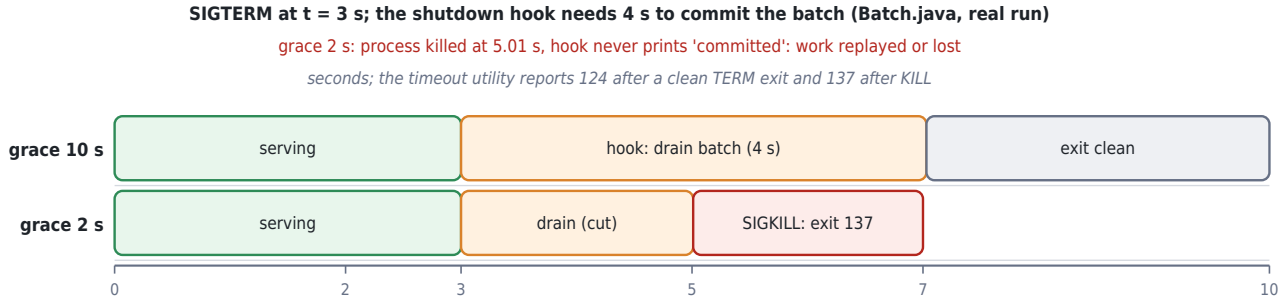


Figure 6.32. With a 10 second grace period the hook completes and the process exits cleanly at 7 seconds; with 2 seconds SIGKILL ends the process at 5 seconds before the batch is committed.

Root cause. The grace period was shorter than the drain time, so the runtime sent SIGKILL before the hook finished and the message offsets were never committed; the batch was reprocessed after restart.

Fix. Immediate: raise `terminationGracePeriodSeconds` above the worst-case drain time plus `preStop`. Permanent: make processing idempotent (deduplicate by message key), stop polling at the start of the hook, and commit offsets after processing.

Prevention. Test shutdown under load in staging by deleting a pod and checking for duplicates; alert on exit code 137 after deployments.

6.3.5 Questions to expect

Interview questions and model answers

1. **A service is slow. What are the first three things checked?** The scope and time of onset with the most recent change; the host-level readings (`uptime`, `vmstat`, `free`, pressure files); and the cgroup counters (throttling, OOM, pids) for containerised services. The resource showing saturation selects the next branch.
2. **What is the USE method?** For each resource, check utilisation, saturation and errors. It forces a complete sweep of CPU, memory, disks, network and software queues before a hypothesis is accepted.
3. **Why does a container with low average CPU show high tail latency?** CFS bandwidth throttling stops all threads of the group when the per-period quota is used, so a request that spans the boundary waits for the next period. `nr_throttled` and `throttled_time` quantify it.
4. **How is a heap `OutOfMemoryError` distinguished from a kernel OOM kill?** The first is a Java exception with a stack trace and exit status 1 (or whatever the handler chooses); the second is SIGKILL, exit 137, no Java log, and a kernel message with `CONSTRAINT_MEMCG` for a cgroup limit.
5. **What does Little's law say and how is it used?** Concurrency equals arrival rate times latency. It sizes thread and connection pools: 200 requests per second at 50 ms need about 10 concurrent slots plus headroom.
6. **Why not run a service at 95% CPU utilisation?** Queueing delay grows as 1 / (1 - U): at 95% the mean response is twenty times the service time, and bursts overflow every queue.
7. **How is CPU steal diagnosed?** The `st` column of `vmstat` or `top`, or the eighth counter of the `cpu` line in `/proc/stat`; sustained values above about 10% indicate hypervisor contention.
8. **What happens when the accept queue is full?** The kernel drops the completing handshake, increments `ListenOverflows`, and the client retransmits and may time out. Queue size is the smaller of the backlog and `somaxconn`.
9. **Why must a rollout include a `preStop` delay?** Endpoint removal and SIGTERM race; the delay keeps the pod serving until proxies have dropped it.
10. **What changes when nodes move from cgroup v1 to v2?** File names and locations, the single hierarchy, and the need for JVM and agent versions that understand v2.

Check yourself

1. `nr_periods 200`, `nr_throttled 150`. What ratio does this give and what does it suggest?
2. A service handles 300 requests per second with a mean latency of 80 ms. How many requests are in the system on average?
3. `vmstat` shows `r=6` on a 2 CPU host with `us` 60 and `sy` 30. Is the CPU saturated?
4. Exit status 137 appears one second after SIGTERM was sent. Is the grace period the likely cause?
5. Why is `free` memory a poor alert metric?

Answers

1. 75% of periods throttled: the container is CPU-limited far below its need; raise the limit or reduce concurrency.
2. 300 x 0.08 = 24 requests.
3. Yes: the run queue (6) is three times the CPU count and utilisation is 90%.
4. Unlikely. A one second delay suggests the process was killed by something else (OOM) or the grace period is 1 second; inspect `describe pod`.
5. The page cache consumes free memory by design; `available` (or working set) reflects what applications can obtain.

KEY POINTS

Method before tools: sweep resources with USE, confirm in the application, change one variable.

- The decision tree branches on the resource showing saturation; the 60 second host checklist selects the branch.
- Latency with low CPU and a CPU limit means throttling; read `cpu.stat`.
- A kernel OOM kill (exit 137, `CONSTRAINT_MEMCG`) is not a Java `OutOfMemoryError`; different logs, different fixes.
- Queueing explains non-linear latency: plan for 50 to 70% utilisation of the bottleneck.
- Kernel parameters, node images and cgroup versions are application dependencies; change them with a canary.

6.4 Interview Question Bank

The questions below are grouped by topic and arranged from definitions to scenario reasoning. Each model answer states the mechanism and, where relevant, the observable evidence (a command or a counter). The answers are intentionally brief; the chapters of the earlier Parts supply the derivations. A candidate should be able to give each answer in about a minute and to extend it with a command that demonstrates it.

6.4.1 Fundamentals

1. **What does an operating system do?** It multiplexes hardware among programs (CPU scheduling, memory management, I/O), abstracts hardware into processes, files and sockets, and enforces protection between users and programs.
2. **What is the difference between kernel space and user space?** The kernel runs in the privileged CPU mode with access to all memory and instructions; user programs run unprivileged and reach the kernel only through system calls, interrupts and exceptions.
3. **What is a system call and how is it executed?** A controlled entry into the kernel. On x86-64 the program loads the syscall number and arguments into registers and executes `syscall`; the CPU switches to kernel mode and jumps to the kernel entry point, which dispatches through the syscall table.
4. **Monolithic kernel versus microkernel?** A monolithic kernel (Linux) runs drivers, file systems and networking in one privileged address space: fast, but a driver fault affects the whole kernel. A microkernel keeps only minimal services privileged and runs the rest as user processes: more isolation, more message passing.
5. **How does a Java program reach the hardware?** Application code runs on the JVM, which calls the C library and native code; these issue system calls; the kernel drives the hardware. `strace` on a JVM shows the syscalls beneath `System.out.println`.
6. **What is an interrupt and how does it differ from an exception?** An interrupt is an asynchronous signal from hardware (timer, disk, network card); an exception is synchronous, caused by the executing instruction (page fault, divide by zero). Both transfer control to a kernel handler.
7. **What is the memory hierarchy and why does it matter?** Registers, L1 to L3 caches, DRAM, SSD, disk: each level is larger and slower by orders of magnitude. Locality of reference determines performance, which is why array traversal order and object layout affect speed.
8. **What happens at boot?** Firmware (UEFI) initialises hardware and loads the bootloader; the bootloader loads the kernel and initramfs; the kernel initialises subsystems and starts PID 1 (systemd), which brings up services according to targets.
9. **What does PID 1 do?** It is the ancestor of all processes: it starts services, adopts and reaps orphans, and its termination is fatal to the system (or to the container's pid namespace).
10. **What does "everything is a file" mean on Linux?** Devices, pipes, sockets and kernel information share the file interface (`open`, `read`, `write`, `close`) and appear in the namespace: `/dev`, `/proc`, `/sys`.
11. **What is the difference between `/proc` and `/sys`?** `/proc` exposes processes and kernel state (`/proc/PID/status`, `/proc/meminfo`); `/sys` exposes the device and driver model and kernel parameters in a structured tree.
12. **What is the USE method?** For every resource, check utilisation, saturation and errors; it is a systematic sweep for performance problems.

6.4.2 Processes and threads

1. **Program versus process?** A program is a file; a process is a running instance with an address space, open files, credentials and at least one thread of execution.
2. **What is in the process control block?** On Linux the `task_struct`: state, PID, credentials, scheduling data, memory descriptor, file table, signal handlers, and parent and child links.
3. **What do `fork` and `exec` do?** `fork` creates a child that is a copy-on-write duplicate of the parent; `exec` replaces the process image with a new program. The shell uses both to run commands.
4. **Why is `fork` cheap for large processes?** Copy-on-write: pages are shared read-only and copied only when written; only the page tables are duplicated.
5. **What are the Linux process states?** `R` running or runnable, `S` interruptible sleep, `D` uninterruptible sleep (usually I/O), `T` stopped, `Z` zombie.
6. **What is a zombie and what is an orphan?** A zombie has terminated but its parent has not called `wait`; an orphan is a live process whose parent died and which is re-parented to PID 1 (or a subreaper).
7. **How are signals delivered?** The kernel marks the signal pending; it is acted upon when the process next returns to user mode. `SIGKILL` and `SIGSTOP` cannot be caught.
8. **What is a thread on Linux?** A task created with `clone()` sharing the address space, file table and signal handlers with its creator; the kernel schedules threads like processes.
9. **Where are Java threads visible?** Each platform thread is a native thread listed in `/proc/PID/task`; `jstack` shows the `nid` (hex native ID) that matches the TID in `top -H`.
10. **Process versus thread: what is shared?** Threads share memory and descriptors, with separate stacks, registers and thread-local storage; processes share nothing except by explicit IPC or shared memory.
11. **Name the IPC mechanisms.** Pipes, named pipes, Unix domain sockets, TCP sockets, shared memory, message queues, signals and files.
12. **How does a shell pipeline `a | b` work?** The shell creates a pipe, forks twice, connects the write end to `a`'s stdout and the read end to `b`'s stdin with `dup2`, then executes both programs.

6.4.3 Scheduling

1. **What does a scheduler optimise?** Throughput, latency (response time), fairness and CPU utilisation; the weights differ between batch and interactive systems.
2. **Compare FCFS, SJF and round robin.** FCFS is simple but suffers the convoy effect; SJF minimises mean waiting time but needs burst estimates and can starve long jobs; round robin gives fair slices and bounded response time at the cost of context switches.
3. **What is a multilevel feedback queue?** Multiple priority queues where jobs move down when they use full quanta and up when they block, approximating SJF without knowing burst lengths.
4. **How does Linux schedule normal tasks?** Through a fair scheduler (CFS, replaced by EEVDF in Linux 6.6) that tracks virtual runtime or eligibility and deadlines so each runnable task receives a proportional share.
5. **What does `nice` do?** It changes a task's weight in the fair scheduler (range -20 to 19); a lower value receives a larger share under contention.
6. **What is load average?** An exponentially damped count of runnable plus uninterruptible tasks over 1, 5 and 15 minutes; it is compared with the number of CPUs and includes tasks blocked on I/O.
7. **High load average but idle CPU. Explanation?** Many tasks in `D` state (disk or NFS waits); check `vmstat` column `b`, `ps` state letters and pressure files.
8. **What is a context switch and what does it cost?** Saving one task's registers and switching to another's address space; direct cost is microseconds, indirect cost is cache and TLB warm-up.
9. **What is CPU affinity?** Restricting a task to certain CPUs (`taskset`, `sched_setaffinity`); useful for cache locality, harmful if it causes imbalance.
10. **What is CFS bandwidth throttling?** A cgroup receives `quota` microseconds per `period`; when exhausted all its tasks wait until the next period, so tail latency rises while average CPU appears low.
11. **What is `cpu.shares` or `cpu.weight`?** A relative weight applied only under contention; it is not a cap.

6.4.4 Memory

1. **Why virtual memory?** Isolation, a uniform address space, overcommit, demand paging and memory-mapped files.
2. **How does address translation work?** The CPU splits a virtual address into page number and offset, walks a multi-level page table (four levels for 48-bit addresses with 4 KiB pages) and caches results in the TLB.
3. **What is a page fault and what are its kinds?** An exception when a page is not mapped or permitted. Minor: resolved without I/O (first touch, copy-on-write). Major: requires reading from disk. Invalid: `SIGSEGV`.
4. **What is the TLB?** A small cache of recent translations; a miss costs a page-table walk; huge pages reduce misses.
5. **FIFO, LRU and Clock?** FIFO evicts the oldest page and exhibits Belady's anomaly; LRU evicts the least recently used and approximates the optimal policy; Clock approximates LRU with a reference bit.
6. **What is thrashing?** The working sets exceed memory, so the system spends its time paging; throughput collapses while `si` and `so` stay high.
7. **What does `free` show and which column matters?** `available` estimates memory obtainable without swapping; `free` excludes reclaimable page cache and is misleading.
8. **What is the page cache?** Kernel memory caching file contents; reads and writes go through it; dirty pages are written back asynchronously.
9. **What is overcommit?** Granting more virtual memory than exists, relying on pages not being touched; mode 0 uses a heuristic, mode 2 enforces a limit.
10. **How does the OOM killer choose a victim?** It computes a badness score from memory use adjusted by `oom_score_adj` (-1000 to 1000) and kills the highest; a cgroup limit restricts the choice to that cgroup.
11. **Why is JVM RSS larger than `-Xmx`?** RSS includes metaspace, thread stacks, code cache, GC structures, direct buffers, mapped files and allocator overhead.
12. **How is a native memory leak found in a JVM?** Enable `-XX:NativeMemoryTracking=summary`, compare `jcmd PID VM.native_memory summary` over time, and inspect `pmap -x` and `/proc/PID/smaps`.
13. **What do `MaxRAMPercentage` and `UseContainerSupport` do?** The JVM reads the cgroup memory limit and sets the default maximum heap to that percentage (default 25) of it.
14. **What is copy-on-write?** Sharing pages read-only between mappings and copying on the first write; used by `fork`, overlayfs and private file mappings.

6.4.5 Concurrency and deadlock

1. **What is a race condition?** An outcome depending on the interleaving of unsynchronised accesses to shared state, for example two threads incrementing a counter.
2. **What is a critical section and what must a solution guarantee?** Code accessing shared data; a solution must provide mutual exclusion, progress and bounded waiting.
3. **What is compare-and-swap?** An atomic instruction that writes a new value only if the current value equals an expected one; the basis of lock-free structures and `AtomicInteger`.
4. **Mutex versus semaphore?** A mutex has an owner and protects a critical section; a semaphore is a counter that signals availability or limits concurrency and has no owner.
5. **What is a condition variable?** A queue on which threads wait for a predicate to become true while atomically releasing a mutex; the predicate must be re-checked in a loop.
6. **What are the four conditions for deadlock?** Mutual exclusion, hold and wait, no preemption, circular wait. Removing any one prevents it.
7. **How is deadlock prevented in practice?** Impose a global lock order, use timeouts (`tryLock`), acquire all locks at once, or avoid nested locks.
8. **What does the Banker's algorithm do?** It grants a request only if the resulting state is safe, meaning an order exists in which every process can finish.
9. **What is priority inversion?** A high-priority task waits for a lock held by a low-priority task that a medium-priority task preempts; priority inheritance resolves it (the Mars Pathfinder case).
10. **How does Java implement `synchronized` on Linux?** Through monitors with lightweight uncontended paths; contended waiting parks threads via `pthread` mutex and condition primitives, which use `futex` system calls.
11. **How is a deadlock confirmed in a running JVM?** `jstack PID` or `jcmd PID Thread.print` prints "Found one Java-level deadlock" with the lock cycle.

12. **What is thread-pool exhaustion?** All workers are blocked (on I/O or on each other) and the queue grows; requests time out although CPU is idle. The cause is visible in repeated thread dumps.

6.4.6 File systems and I/O

1. **What is a file descriptor?** A per-process integer index into a table of open file descriptions, each referencing an inode; 0, 1 and 2 are stdin, stdout and stderr.
2. **What is an inode?** The metadata record of a file (owner, mode, size, timestamps, block pointers); the name is stored in directory entries, not in the inode.
3. **Hard link versus symbolic link?** A hard link is another name for the same inode within one file system; a symbolic link is a file containing a path.
4. **What does a journal provide?** Crash consistency: metadata (and optionally data) changes are logged before being applied, so recovery replays or discards incomplete transactions.
5. **What is the difference between `write`, `fsync` and `O_DIRECT`?** `write` copies to the page cache; `fsync` forces data to the device; `O_DIRECT` bypasses the cache.
6. **Why do databases call `fsync`?** To make a commit durable; without it a power failure loses acknowledged writes.
7. **Blocking versus non-blocking versus multiplexed I/O?** Blocking waits in the call; non-blocking returns at once with `EAGAIN`; multiplexing (`select`, `poll`, `epoll`) waits for readiness of many descriptors in one call.
8. **Why is `epoll` preferred over `select`?** Cost proportional to ready descriptors, not registered ones, and no descriptor limit of 1024.
9. **What is zero-copy?** Moving data between file and socket in the kernel (`sendfile`, `splice`) without copying through user space; Kafka uses it for consumers.
10. **How can a disk be full with `df` showing free space?** Inode exhaustion (`df -i`), reserved blocks, or deleted files held open by a process (`lsof +L1`).
11. **What does "Too many open files" mean?** The per-process limit `RLIMIT_NOFILE` was reached (`EMFILE`), usually from a descriptor leak or a low `ulimit -n`.
12. **What is log rotation and how does it interact with open files?** The log is renamed or truncated; a process that keeps the old descriptor continues writing to the renamed file unless it reopens (or `copytruncate` is used).

6.4.7 Containers

1. **What is a container?** A process isolated by namespaces, limited by cgroups, with a root file system from an image.
2. **Which namespaces exist?** pid, mnt, uts, ipc, net, user, cgroup, time.
3. **What is a cgroup?** A node in a hierarchy to which processes are attached and on which controllers limit and account CPU, memory, pids and I/O.
4. **cgroup v1 versus v2?** v1 has one hierarchy per controller; v2 has a unified hierarchy with different file names (`memory.max`, `cpu.max`).
5. **What is overlayfs?** A union file system with read-only lower layers and a writable upper layer; first write copies the file up.
6. **Container versus VM?** Containers share the host kernel and start in milliseconds; VMs run a separate guest kernel with stronger isolation.
7. **What do Kubernetes requests and limits map to?** Requests to scheduling and CPU weight, limits to cgroup quota and memory limit; the QoS class sets `oom_score_adj`.
8. **What are the QoS classes?** Guaranteed (requests equal limits), Burstable, BestEffort; killed in the reverse order under pressure.
9. **Why does exit code 137 appear?** The process received SIGKILL: OOM kill or grace period exceeded.
10. **Why must containers handle SIGTERM?** Kubernetes and Docker send SIGTERM first; without handling, the process is killed after the grace period and loses in-flight work.
11. **What does `--init` do in Docker?** Runs a minimal init as PID 1 that forwards signals and reaps zombies.
12. **How should a Java service be hardened in a container?** Non-root user, dropped capabilities, read-only root file system, no privilege escalation, runtime-default seccomp profile, resource limits.

sheram

6.4.8 Linux troubleshooting scenarios

1. **A service has high latency and low CPU. What is checked?** Throttling (`cpu.stat`), blocked threads (`jstack`), I/O waits (`vmstat b`, `wa`), GC pauses, connection pool waits.
2. **One CPU core is at 100% in a Java process. How is the thread found?** `top -H -p PID` for the TID, convert to hex, find `nid=0x...` in `jstack` output.
3. **A pod restarts with exit 137. What next?** `kubectl describe pod` for OOMKilled, node `dmesg` for `CONSTRAINT_MEMCG`, container memory working set against the limit, NMT for non-heap growth.
4. `java.lang.OutOfMemoryError: unable to create native thread`**?** Thread count against `pids.max`, `ulimit -u`, `kernel.threads-max`, available virtual memory for stacks; find the leaking pool in a dump.
5. **The disk is full but** `du` **shows less usage than** `df`**. Why?** Deleted files still open (`lsof +L1`), data in a hidden mount point, or snapshots; restart the holder or truncate through `/proc/PID/fd`.
6. **Connection timeouts under burst with idle CPU?** `ListenOverflows` and `ss -ltn` accept queue; raise backlog and `somaxconn`, fix slow accept.
7. **Intermittent 5 second delays?** DNS timeouts (`strace -T`, `getent hosts`), Nagle and delayed ACK interplay, retransmission timers.
8. **Load average is 30 on 4 CPUs but CPU is idle?** Tasks in `D` state: stuck NFS or slow storage; check `ps -eo state,wchan` and `/proc/pressure/io`.
9. **Memory looks full in** `free`**. Is it a problem?** Not if `available` is high; the page cache is reclaimable.
10. **A process cannot be killed with** `kill -9`**.** It is in `D` state waiting for I/O or is a zombie (already dead); fix the I/O path or the parent.
11. **After a deployment, 502 errors appear for seconds.** Missing graceful shutdown or `preStop` delay: the pod exits before endpoints are updated.
12. **JWT validation fails on one node.** Clock skew; check `chronyc tracking` and `timedatectl`.
13. **Latency doubles during a nightly batch on the same node.** Page cache eviction, CPU weights, I/O contention; separate the workloads or constrain the batch with cgroups and `fadvise`.
14. **The JVM uses G1 on the laptop and Serial in the pod.** The pod has fewer than 2 CPUs or less than about 1792 MB; set the collector explicitly.
15. **How would an incident be run in its first five minutes?** Declare, state impact and recent change, read golden signals, run the host checklist, isolate the layer, apply a reversible mitigation, communicate.

6.5 Cheat Sheets and Study Plan

These reference tables collect values and commands used throughout the guide. Values were checked on Linux 6.18 (x86-64) or taken from the Linux manual pages; architecture-specific numbers (signals, errno) are those of x86-64.

6.5.1 Process state letters and signals

Letter	Name	Meaning and typical cause
R	Running or runnable	On a CPU or in the run queue.
S	Interruptible sleep	Waiting for an event; wakes on a signal. Most idle processes.
D	Uninterruptible sleep	Waiting in the kernel for I/O; cannot be killed; counts toward load average.
T	Stopped	SIGSTOP or SIGTSTP received, or traced by a debugger.
Z	Zombie	Terminated; the parent has not called wait.
I	Idle kernel thread	Kernel threads that are idle.

Process states in the STAT column of ps and top.

Signal	No.	Default action and use
SIGHUP	1	Terminate; conventionally reload configuration for daemons.
SIGINT	2	Terminate; keyboard Ctrl-C.
SIGQUIT	3	Core dump; the JVM prints a thread dump to stdout instead.
SIGABRT	6	Core dump; abort().

Signal	No.	Default action and use
SIGKILL	9	Terminate; cannot be caught or ignored (exit status 137).
SIGUSR1	10	Terminate by default; user defined.
SIGSEGV	11	Core dump; invalid memory access.
SIGPIPE	13	Terminate; write to a pipe with no reader (the JVM ignores it).
SIGTERM	15	Terminate; the polite request (JVM exit status 143).
SIGCHLD	17	Ignored; a child changed state.
SIGCONT	18	Continue a stopped process.
SIGSTOP	19	Stop; cannot be caught or ignored.

Signals most used in operations (numbers for x86-64).

6.5.2 errno values

Name	No.	Typical cause
EPERM	1	Operation not permitted (missing capability, seccomp filter).
ENOENT	2	No such file or directory.
EINTR	4	System call interrupted by a signal.
EIO	5	Input/output error (failing device).
EBADF	9	Bad file descriptor (closed or never opened).
ECHILD	10	No child processes to wait for.
EAGAIN	11	Try again: non-blocking read with no data, or `fork`/thread limit reached.
ENOMEM	12	Out of memory (or a cgroup limit during allocation).
EACCES	13	Permission denied (file mode, port below 1024 without capability).
EEXIST	17	File exists.
EINVAL	22	Invalid argument.
ENFILE	23	System-wide open file table full.
EMFILE	24	Too many open files in this process.
ENOSPC	28	No space left on device (blocks, inodes or quota).
EPIPE	32	Broken pipe.
EADDRINUSE	98	Address already in use (port bound, or TIME_WAIT without reuse).
EADDRNOTAVAIL	99	Cannot assign requested address (ephemeral ports exhausted).
ECONNRESET	104	Connection reset by peer.
ETIMEDOUT	110	Connection timed out.
ECONNREFUSED	111	Connection refused (nothing listening).

Common errno values (Linux, x86-64) and typical causes in Java services.

sreram

6.5.3 Files under /proc and /sys

Path	Content
`/proc/PID/status`	State, threads, VmRSS, Cap* sets, voluntary and involuntary context switches, NSpid.
`/proc/PID/task`	One directory per thread; the count is the thread count.
`/proc/PID/fd`	Open descriptors as links; the number of entries is the descriptor count.
`/proc/PID/limits`	Effective resource limits including open files and processes.
`/proc/PID/smaps_rollup`	Summed memory accounting (Rss, Pss, Anonymous).
`/proc/PID/io`	Read and write byte counters.
`/proc/PID/cgroup`	cgroup membership (one line per controller on v1).
`/proc/PID/ns`	Namespace identities as links.
`/proc/meminfo`	MemTotal, MemAvailable, Cached, Dirty, Slab, Swap.
`/proc/stat`	CPU counters (user, nice, system, idle, iowait, irq, softirq, steal), context switches.
`/proc/diskstats`	Per-device I/O counters for utilisation and await.
`/proc/pressure/cpu, memory, io`	PSI stall percentages.
`/proc/net/netstat`	TcpExt counters such as ListenOverflows.
`/proc/sys/*`	Tunable kernel parameters (same as `sysctl`).
`/sys/fs/cgroup`	cgroup controllers and files.

Frequently read kernel interfaces.

6.5.4 ulimit and sysctl crib

Item	Notes
`ulimit -n`	Open files per process (soft and hard); persistent through `LimitNOFILE` in systemd or `limits.conf`.
`ulimit -u`	Processes per user; `nproc`; counts threads.
`ulimit -s`	Default thread stack size; the JVM uses `-Xss` instead.
`ulimit -c`	Core file size; 0 disables core dumps.
`prlimit --pid PID`	Show or change limits of a running process.
`sysctl -a`	List all kernel parameters; `sysctl -w name=value` changes one; persist in `/etc/sysctl.d/`.
`vm.swappiness, net.core.somaxconn, fs.file-max`	See the playbook table for observed values and cautions.
`kernel.core_pattern`	Destination of core dumps; in containers it is a host setting.

Limits and parameters, and where each is set.

sreeram

6.5.5 JVM flags for containers

Flag	Effect
`-XX:+UseContainerSupport`	Read cgroup limits (default on Linux).
`-XX:MaxRAMPercentage=65`	Maximum heap as a percentage of container memory (default 25).
`-XX:InitialRAMPercentage`	Initial heap percentage (default 1.5625).
`-XX:ActiveProcessorCount=N`	Override the processor count seen by the JVM.
`-XX:+UseG1GC, -XX:+UseParallelGC`	Select the collector explicitly; avoids ergonomics surprises.
`-XX:MaxMetaspaceSize, -XX:MaxDirectMemorySize`	Bound non-heap memory so leaks raise exceptions instead of OOM kills.
`-Xss`	Thread stack size; multiply by the thread count when sizing the limit.
`-XX:NativeMemoryTracking=summary`	Enable `jcmd PID VM.native_memory summary`.
`-XX:+ExitOnOutOfMemoryError`	Exit on heap exhaustion so the orchestrator restarts the process.
`-XX:+HeapDumpOnOutOfMemoryError, -XX:HeapDumpPath`	Write a dump on heap exhaustion to a persistent volume.
`-Xlog:gc*:stdout`	Unified GC logging to standard output.

Flags for Java 17 and later in containers.

6.5.6 Commands by symptom

Symptom	Command
Hot thread	`top -H -p PID; printf '%x' TID; jstack PID`
Thread count	`ls /proc/PID/task (count the entries)`
Descriptor count	`ls /proc/PID/fd (count the entries)`
Deleted open files	`lsof +L1`
Throttling	`cat /sys/fs/cgroup/cpu/.../cpu.stat`
OOM evidence	`dmesg -T (search for oom and killed process)`
Resident memory	`grep -E 'VmRSS|VmSwap' /proc/PID/status`
Listen queue	`ss -ltn`
Syscall profile	`strace -c -f -p PID (bounded with timeout)`
Heap histogram	`jcmd PID GC.class_histogram`
GC activity	`jstat -gcutil PID 1000`

One-line commands for the symptoms met most often.

6.5.7 Six-week study plan

The plan assumes about two hours per day. Every week ends with the interview questions of the matching topic answered aloud without notes, and with one reproduced experiment from the Hands-on sections.

sreram

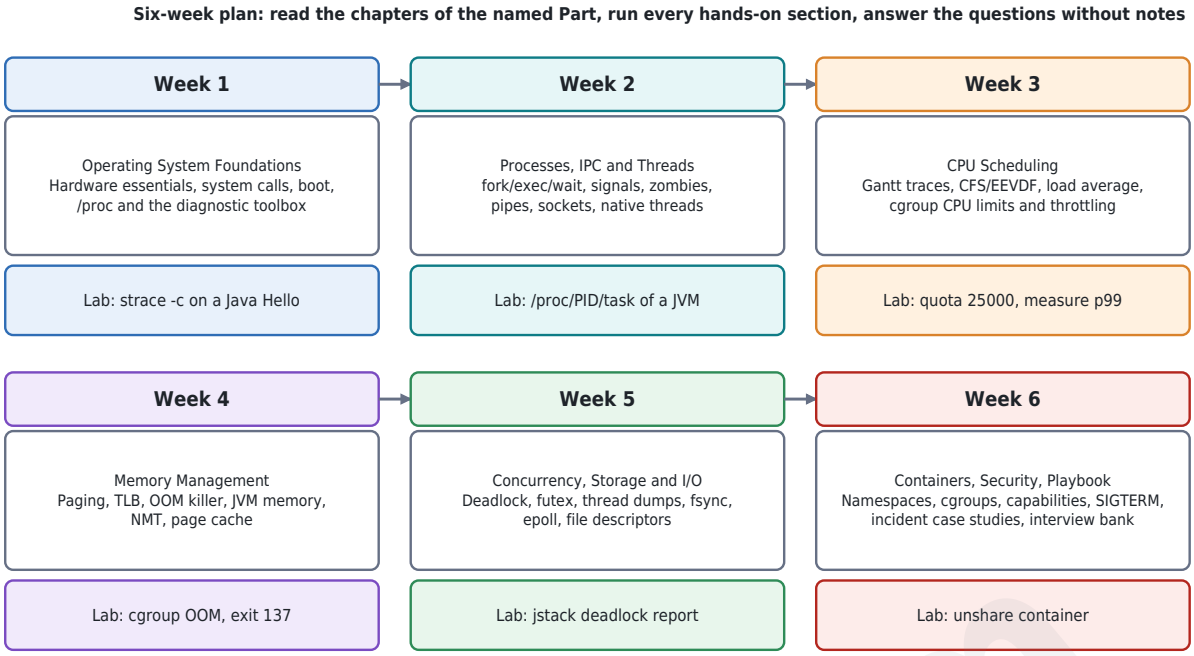


Figure 6.33. The six weeks follow the order of the Parts: foundations, processes and scheduling, memory, concurrency and storage, then containers and the production playbook; each week has one laboratory exercise.

Week	Chapters by name	Exercise
1	What an operating system does; Hardware essentials; System calls; Boot and init; The diagnostic toolbox	Trace a Java Hello with `strace -c` and read `/proc/PID/status`.
2	Processes; Inter-process communication; Threads; CPU scheduling	List the native threads of a JVM and reproduce a zombie.
3	CPU limits in containers; Virtual memory; Paging	Set a CPU quota and measure p99; read `/proc/PID/maps`.
4	Page replacement; The page cache; OOM killer; Memory of a JVM process	Reproduce a cgroup OOM kill and read the `dmesg` report.
5	Race conditions; Deadlock; Locks in Java; File systems; I/O models	Produce and read a Java deadlock dump; count descriptors.
6	Containers from first principles; Security and services; The production playbook; this interview bank	Build a container with `unshare`; run three case studies end to end.

Mapping of weeks to chapters and exercises.

6.6 Putting It Together: One Request Through the Whole Stack

A single HTTP request to a Spring Boot service crosses every subsystem described in this guide. Latency is the sum of the waits in each layer, and an incident is one layer whose wait has grown. The trace below follows `GET /orders` from the network card to the response, naming the kernel mechanism and the observable counter at each stage.

6.6.1 Concept and mechanism

The path has twelve stages.

1. **Network card and interrupt.** The packet arrives in a receive ring; the card raises an interrupt; the interrupt handler schedules a softirq. Counters: `/proc/interrupts`, `/proc/softirqs`.
2. **TCP stack.** The kernel processes the segment, completes the handshake for a new connection and places the connection in the accept queue, or appends payload to the socket receive buffer. Counters: `ListenOverflows`, `ss -ti`.
3. **Readiness notification.** The socket becomes readable; the `epoll` instance used by the Tomcat poller marks it ready and wakes the sleeping poller thread.

4. **Scheduler.** The woken task is placed on a run queue; it may wait for a CPU (run-queue latency) or be throttled by its cgroup. Counters: `/proc/PID/schedstat`, `cpu.stat`, PSI.
5. **Read system call.** The poller reads the request bytes (`read`), copying from the socket buffer to user memory, and hands the request to a worker thread through a queue (a `futex` wake).
6. **Worker thread executes.** The Spring controller runs on a CPU. Allocation touches heap pages: first touch causes minor page faults, and a young collection may stop all threads. Counters: `minflt` in `/proc/PID/stat`, GC logs.
7. **Memory access.** Data accesses pass through the CPU caches and the TLB; a TLB miss walks the page table.
8. **Database call.** The JDBC driver writes the query to a socket (`write`, TCP send path) and the worker blocks in `read` (state `S`) until the reply arrives, while the scheduler runs other threads. The connection comes from the Hikari pool, whose wait time is a separate queue.
9. **Logging.** The access log line is written with `write` into the page cache; the kernel flushes dirty pages to the block layer later. A full disk or slow `fsync` blocks this write.
10. **Response write.** The worker writes the response (`write`), the TCP stack segments it, and the card transmits it (DMA from kernel buffers).
11. **Connection reuse or close.** With keep-alive the descriptor stays registered in `epoll`; otherwise `close` releases the descriptor and the socket enters `TIME_WAIT`.
12. **Accounting.** The kernel updates CPU time, context switch counters and I/O counters visible in `/proc`.

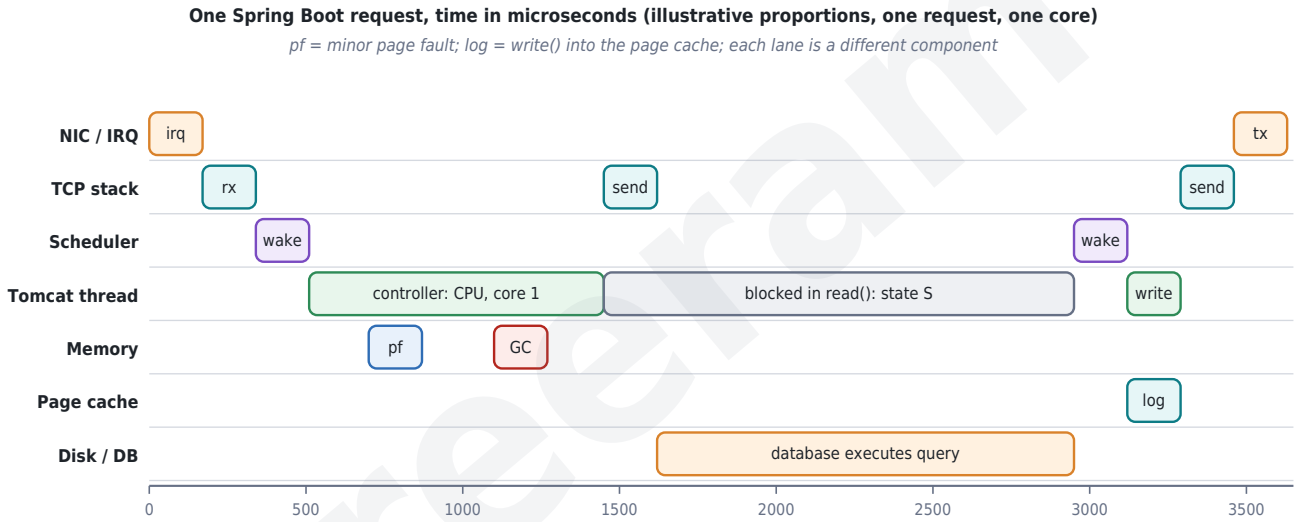


Figure 6.34. The request spends most of its time blocked on the database; the lanes show where interrupts, the TCP stack, the scheduler, memory and the page cache contribute and where a delay in any lane would add directly to the latency.

6.6.2 Worked trace

The following trace was taken from a minimal Java HTTP server (`com.sun.net.httpserver`) on the build machine under `strace -f -tt`, restricted to network and I/O calls, for one `curl` request to a freshly started JVM. The first request is a cold path: classes load and code is interpreted, so absolute times are larger than in steady state, while the sequence of calls is the same as in steady state.

One request seen by strace (real output, shortened; times are typical) TERMINAL

```
$ curl -s -o /dev/null -w "http=%{http_code} time=%{time_total}\n" http://127.0.0.1:20602/orders
http=200 time=0.133040
$ grep -E 'accept|epoll_ctl|read\(8|write\(8' mini.trace | cut -c1-120
15041 02:35:03.011419 accept(5, {sa_family=AF_INET, sin_port=htons(49980), ...}, [28 => 16]) = 8
15041 02:35:03.014019 epoll_ctl(6, EPOLL_CTL_ADD, 8, {events=EPOLLIN, data={u32=8, u64=8}}) = 0
15041 02:35:03.016579 epoll_ctl(6, EPOLL_CTL_DEL, 8, 0x7f1588efe304 <unfinished ...>
15041 02:35:03.018941 read(8, "GET /orders HTTP/1.1\r\nHost: 127."..., 8192) = 84
15041 02:35:03.140264 write(8, "HTTP/1.1 200 OK\r\nDate: Sun, 18 O"..., 75) = 75
15041 02:35:03.143812 write(8, "{\"id\":42}", 9) = 9
15041 02:35:03.145814 read(8, "", 8192) = 0
```

The sequence maps to the stages above: `accept` returns descriptor 8 (stages 2 and 3) after `epoll_wait` reported readiness of the listening socket 5; the descriptor is registered with `epoll_ctl` and removed again once the dispatcher hands the exchange to the handler; `read` obtains 84 bytes of request (stage 5); the interval of 121 ms between the `read` and the first `write` is the handler and header generation on a cold JVM (stages 6 and 7: class loading and

interpretation, which the JIT removes for later requests); the two `write` calls send the header and the 9-byte body (stage 10); the final `read` returning 0 is the client closing the connection (stage 11). In this server the default executor runs the handler on the dispatcher thread, so no `futex` hand-off to a worker appears; Tomcat, which uses a worker pool, adds that hand-off at stage 5.

6.6.3 Hands-on

Each stage can be observed with the tools of the earlier Parts. A complete exercise: start the server with `strace -f -c -p PID` under `timeout`, issue 1,000 requests with a loop, and compare the syscall counts with the stage list; then repeat inside a cgroup with a 25% CPU quota and compare the latency distribution with the figure in the playbook case on CPU throttling. The diagnosis of any latency complaint is the identification of the lane in the figure whose segment became longer.

6.6.4 Real-world incidents

The case studies in the playbook chapter each correspond to a stage of this path: throttling (stage 4), accept queue overflow (stage 2), page cache eviction and disk fill (stage 9), thread and pid limits (stage 6), DNS delay (stage 8, before the connection is opened), clock skew (the validation step inside the controller), memory limits (stage 6 and the cgroup), and shutdown behaviour (stages 1 to 3 for the next request). The practical skill is to place a symptom on this path before choosing a tool.

6.6.5 Questions to expect

Interview questions and model answers

1. **Describe what happens when a request reaches a Spring Boot service.** Interrupt and softirq, TCP processing and accept queue, `epoll` readiness, scheduler wake-up, `read` of the request, hand-off to a worker thread, execution with possible page faults and GC, a blocking database call, a log write into the page cache, and a `write` of the response.
2. **Where can the request wait without using CPU?** Accept queue, run queue or cgroup throttling, connection pool, database reply, lock monitors, disk write or `fsync`, DNS resolution.
3. **How is the slowest stage found?** Compare per-stage timings from tracing (`strace -T`, application spans) and the counters named for each stage; the saturated resource identifies the lane.
4. **Why is a first request slower than later requests?** Class loading, interpretation before JIT compilation, cold CPU caches and page-cache misses.
5. **Which counters distinguish CPU wait from I/O wait?** Run-queue latency, `cpu.stat` and PSI cpu for CPU; `wa`, the `D` state, `/proc/diskstats` and PSI io for I/O.

Check yourself

1. A worker thread is in state `S` during the database call. Does that thread consume CPU? What does a `futex` call at the hand-off indicate?
2. Name two places where the request can be delayed before any application code runs.
3. In the strace output, what does `read(8, "", 8192) = 0` mean?
4. Which stage does `nr_throttled` describe?

Answers

1. No; it sleeps until the reply arrives. A `futex` wake at the hand-off is the poller waking a worker through a queue.
2. The accept queue (overflow or slow `accept`) and the run queue or cgroup throttling.
3. End of file: the peer closed the connection.
4. Stage 4, the scheduler: time the cgroup's tasks were stopped because the quota was exhausted.

KEY POINTS

A request is a path through the interrupt handler, TCP stack, scheduler, memory system, page cache and block layer; each stage has a queue and a counter.

- Latency is the sum of waits; an incident is one stage whose wait has grown.
- Place the symptom on the path first, then choose the tool for that stage.
- Cold paths (class loading, first-touch faults, cold caches) explain first-request latency.
- The playbook cases are the same path with one stage degraded.

sreeram